O'REILLY®

Python 金融大数据分析

Python for Finance

[德] Yves Hilpisch 著
姚军 译

中国工信出版集团
人民邮电出版社
POSTS & TELECOM PRESS

O'REILLY®

Python 金融大数据分析

[德] Yves Hilpisch 著

姚 军 译

人 民 邮 电 出 版 社

北 京

图书在版编目（CIP）数据

Python金融大数据分析 /（德）希尔皮斯科著 ；姚军译. — 北京 ：人民邮电出版社，2015.12
ISBN 978-7-115-40445-9

Ⅰ. ①P… Ⅱ. ①希… ②姚… Ⅲ. ①软件工具－程序设计－应用－金融－分析 Ⅳ. ①F830.41-39

中国版本图书馆CIP数据核字(2015)第244879号

版权声明

Copyright © 2015 by O'Reilly Media.Inc.
Simplified Chinese Edition, jointly published by O'Reilly Media, Inc. and Posts & Telecom Press, 2015. Authorized translation of the English edition, 2015 O'Reilly Media, Inc., the owner of all rights to publish and sell the same.
All rights reserved including the rights of reproduction in whole or in part in any form.
本书中文简体字版由 O'Reilly Media, Inc. 授权人民邮电出版社出版。未经出版者书面许可，对本书的任何部分不得以任何方式复制或抄袭。
版权所有，侵权必究。

◆ 著　［德］Yves Hilpisch
译　姚　军
责任编辑　傅道坤
责任印制　张佳莹　焦志炜

◆ 人民邮电出版社出版发行　北京市丰台区成寿寺路 11 号
邮编　100164　电子邮件　315@ptpress.com.cn
网址　http://www.ptpress.com.cn
三河市海波印务有限公司印刷

◆ 开本：787×1000　1/16
印张：33
字数：687 千字　2015 年 12 月第 1 版
印数：1－2 500 册　2015 年 12 月河北第 1 次印刷
著作权合同登记号　图字：01-2014-7519 号

定价：99.00 元

读者服务热线：(010)81055410　印装质量热线：(010)81055316
反盗版热线：(010)81055315

内容提要

Python 凭借其简单、易读、可扩展性以及拥有巨大而活跃的科学计算社区，在需要分析、处理大量数据的金融行业得到了广泛而迅速的应用，并且成为该行业开发核心应用的首选编程语言。本书提供了使用 Python 进行数据分析，以及开发相关应用程序的技巧和工具。

本书总计分为 3 部分，共 19 章，第 1 部分介绍了 Python 在金融学中的应用，其内容涵盖了 Python 用于金融行业的原因、Python 的基础架构和工具，以及 Python 在计量金融学中的一些具体入门实例；第 2 部分介绍了金融分析和应用程序开发中最重要的 Python 库、技术和方法，其内容涵盖了 Python 的数据类型和结构、用 matplotlib 进行数据可视化、金融时间序列数据处理、高性能输入/输出操作、高性能的 Python 技术和库、金融学中需要的多种数学工具、随机数生成和随机过程模拟、Python 统计学应用、Python 和 Excel 的集成、Python 面向对象编程和 GUI 的开发、Python 与 Web 技术的集成，以及基于 Web 应用和 Web 服务的开发；第 3 部分关注的是蒙特卡洛模拟期权与衍生品定价实际应用的开发工作，其内容涵盖了估值框架的介绍、金融模型的模拟、衍生品的估值、投资组合的估值、波动率期权等知识。

本书适合对使用 Python 进行大数据分析、处理感兴趣的金融行业开发人员阅读。

O'Reilly Media, Inc.介绍

O'Reilly Media 通过图书、杂志、在线服务、调查研究和会议等方式传播创新知识。自 1978 年开始，O'Reilly 一直都是前沿发展的见证者和推动者。超级极客们正在开创着未来，而我们关注真正重要的技术趋势——通过放大那些“细微的信号”来刺激社会对新科技的应用。作为技术社区中活跃的参与者，O'Reilly 的发展充满了对创新的倡导、创造和发扬光大。

O'Reilly 为软件开发人员带来革命性的“动物书”；创建第一个商业网站（GNN）；组织了影响深远的开放源代码峰会，以至于开源软件运动以此命名；创立了 Make 杂志，从而成为 DIY 革命的主要先锋；公司一如既往地通过多种形式缔结信息与人的纽带。O'Reilly 的会议和峰会集聚了众多超级极客和高瞻远瞩的商业领袖，共同描绘出开创新产业的革命性思想。作为技术人士获取信息的选择，O'Reilly 现在还将先锋专家的知识传递给普通的计算机用户。无论是通过书籍出版，在线服务或者面授课程，每一项 O'Reilly 的产品都反映了公司不可动摇的理念——信息是激发创新的力量。

业界评论

“O'Reilly Radar 博客有口皆碑。”

—— Wired

“O'Reilly 凭借一系列（真希望当初我也想到了）非凡想法建立了数百万美元的业务。”

—— Business 2.0

“O'Reilly Conference 是聚集关键思想领袖的绝对典范。”

—— CRN

“一本 O'Reilly 的书就代表一个有用、有前途、需要学习的主题。”

—— Irish Times

“Tim 是位特立独行的商人，他不光放眼于最长远、最广阔的视野并且切实地按照 Yogi Berra 的建议去做了：‘如果你在路上遇到岔路口，走小路（岔路）。’回顾过去 Tim 似乎每一次都选择了小路，而且有几次都是一闪即逝的机会，尽管大路也不错。”

—— Linux Journal

前言

不久以前，在金融行业，Python 作为一种编程语言和平台技术还被视为异端。相比之下，2014 年有许多大型金融机构——如美国银行、美林证券的“石英”项目或者摩根大通的“雅典娜”项目——战略性地使用了 Python 和其他既定的技术，构建、改进和维护其核心 IT 系统。众多大大小小的对冲基金也大量使用 Python 的功能，进行高效的金融应用程序开发和金融分析工作。

同样，当今许多金融工程硕士课程（或者授予类似学位的课程）也使用 Python 作为核心语言之一，教授计量金融理论与可执行计算机代码之间的转换方法。针对金融专业人士的教育项目和培训也越来越多地在课程中加入 Python。有些课程将它作为主要实现语言。

Python 最近取得这样的成功，而且在未来似乎还会继续下去，这有许多原因。其中包括它的语法、Python 开发人员可用的科学生态系统和数据分析库、易于和几乎所有其他技术集成，以及其开源地位（更多这方面的深入探讨请参见第 1 章）。

因此，有许多好的书籍，从不同角度和焦点传授 Python。本书是最先介绍和传授 Python 金融应用的书籍之一，特别是将 Python 用于计量金融学和金融分析。书中采用的方法很实用，实现和说明先于理论细节，通常将焦点更多地放在大局上，而非某些类或者函数晦涩难懂的参数化选项。

本书的大部分是在基于浏览器的强大交互式环境 IPython Notebook（在第 2 章中有更详细的介绍）中编写的，因此有可能为读者提供本书中几乎所有例子的可执行、交互式版本。

希望立即开始使用完备的交互式 Python（以及 R 和 Julia）金融分析环境的读者，应该前往 http://oreilly.quant-platform.com，尝试 Python Quant 平台（结合本书提供的 IPython Notebook 文件）。你还应该关注基于 Python 的金融分析库 DX analytics（http://dx-analytics.com）。我的另一本书《Derivatives Analytics with Python》（Wiley Finance）更详细地介绍高级衍生品分析的理论和数值方法，书中也提供了丰富而易用的 Python 代码。进一步的材料，特别是有关 Python 计量金融学应用的幻灯片及视频，可以在我的私人网站上找到（http://hilpisch.com）。

如果你想参加 Python 计量金融学应用社区的活动，在世界上的金融中心有各种各样的机会。例如，我自己在伦敦（http://www.meetup.com/Python-for-Quant_finance-London/）和纽约（http://www.meetup.com/Python-for-Quant_finance-NYC/）组织以此为焦点的讨

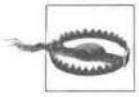
论组。每年还有多次 For Python Quants 会议和研讨（http://forpythonquants.com 和 http://pythonquants.com）。

对于 Python 确立金融行业中重要技术地位这一事实，我确实感到很兴奋。我敢肯定，它在未来将会起到更重要的作用，例如在衍生品和风险分析或者高性能计算领域中。我希望本书能够帮助专业人士、研究人员和学生在面对这一迷人领域中的挑战时，最大限度地利用 Python。

本书的惯例

提示

提示这个图标用来强调一个提示、建议或一般说明。



警告

警告这个图标用来表示一个警告或注意事项。

代码示例的使用

补充材料（特别是 IPython Notebooks 和 Python 脚本/模块）可以从 http://oreilly.quant-platform.com 下载。

本书的目的是为了帮助读者完成工作。一般而言，你可以在你的程序和文档中使用本书中的代码，而且也没有必要取得我们的许可。但是，如果你要复制的是核心代码，则需要和我们打个招呼。例如，你可以在无需获取我们许可的情况下，在程序中使用本书中的多个代码块。但是，销售或分发 O'Reilly 图书中的代码光盘则需要取得我们的许可。通过引用本书中的示例代码来回答问题时，不需要事先获得我们的许可。但是，如果你的产品文档中融合了本书中的大量示例代码，则需要取得我们的许可。

在引用本书中的代码示例时，如果能列出本书的属性信息是最好不过。一个属性信息通常包括书名、作者、出版社和 ISBN。例如："*Python for Finance* by Yves Hilpisch (O'Reilly). Copyright 2015Yves Hilpisch, 978-1-491-94528-5."

在使用书中的代码时，如果不确定是否属于正常使用，或是否超出了我们的许可，请通过 permissions@oreilly.com 与我们联系。

联系方式

如果你想就本书发表评论或有任何疑问，敬请联系出版社：

美国：

O'Reilly Media Inc.

1005 Gravenstein Highway North

Sebastopol, CA 95472

中国：

北京市西城区西直门南大街 2 号成铭大厦 C 座 807 室（100035）

奥莱利技术咨询（北京）有限公司

我们还为本书建立了一个网页，其中包含了勘误表、示例和其他额外的信息。你可以通过如下地址访问该网页：

http://bit.ly/python-finance

关于本书的技术性问题或建议，请发邮件到：

bookquestions@oreilly.com

欢迎登录我们的网站（http://www.oreilly.com），查看更多我们的书籍、课程、会议和最新动态等信息。

Facebook: http://facebook.com/oreilly

Twitter: http://twitter.com/oreillymedia

YouTube: http://www.youtube.com/oreillymedia

Safari® 在线图书

Safari 在线图书是一个按需订阅的数字图书馆。它有不少于 7500 本技术和创意相关的书籍和视频供你参考和搜索。

通过订阅，你可以在线阅读任何页面或任何视频，甚至可以从手机或移动设备上在线阅读。你可以在书籍出版前访问到它们，并给读者发送反馈。其他功能还包括：复制和粘贴代码、组织收藏夹、下载和标记章节、做笔记、打印等。

O'Reilly Media 已经将本书英文版上传到 Safari 在线图书服务。在 http://my.safaribooksonline.com 上免费注册，你就可以访问本书所有章节以及类似主题的书籍。

致谢

我要感谢所有为本书的实现提供帮助的人，特别是认真反馈甚至完全理解书中例子的人，例如 Ben Lerner、James Powell、Michael Schwed、Thomas Wiecki 或者 Felix

Zumstein。同样，我要感谢审稿人 Hugh Brown、Jennifer Pierce、Kevin Sheppard 和 Galen Wilkerson。本书从他们宝贵的反馈和许多建议中受益良多。

本书还从我在 2013 年和 2014 年出席的会议和研讨活动的参与者们那里得到很多有益的反馈，这些会议包括：PyData、For Python Quants、Big Data in Quant Finance、EuroPython、EuroScipy、PyCon DE、PyCon Ireland、Parallel Data Analysis、Budapest BI Forum 和 CodeJam。我还在柏林、伦敦、纽约的 Python 讨论组上的演讲期间得到了宝贵的反馈。

最后，我要感谢家人，他们完全接受我集中地做自己最热爱的事。完成这样篇幅的书籍需要在一年之中、在我已经安排得很满的工作负荷和行程基础上投入大量的时间——我也必须花费更多的时间，独自坐在计算机前。因此，感谢 Sandra、Lilli 和 Henry 的理解与支持。我将本书献给可爱的妻子 Sandra，她是我们全家的核心。

Yves

德国萨尔州，2014 年 11 月

目录

第 1 部分　Python 与金融

第 1 章　为什么将 Python 用于金融 ····· 3
1.1　Python 是什么 ····· 3
1.1.1　Python 简史 ····· 5
1.1.2　Python 生态系统 ····· 5
1.1.3　Python 用户谱系 ····· 7
1.1.4　科学栈 ····· 7
1.2　金融中的科技 ····· 8
1.2.1　科技开销 ····· 9
1.2.2　作为业务引擎的科技 ····· 9
1.2.3　作为进入门槛的科技和人才 ····· 9
1.2.4　不断提高的速度、频率、数据量 ····· 10
1.2.5　实时分析的兴起 ····· 11
1.3　用于金融的 Python ····· 12
1.3.1　金融和 Python 语法 ····· 12
1.3.2　Python 的效率和生产率 ····· 15
1.3.3　从原型化到生产 ····· 19
1.4　结语 ····· 20
1.5　延伸阅读 ····· 20
第 2 章　基础架构和工具 ····· 21
2.1　Python 部署 ····· 22
2.1.1　Anaconda ····· 22
2.1.2　Python Quant Platform ····· 27
2.1.3　工具 ····· 30
2.1.4　Python ····· 30
2.1.5　IPython ····· 30
2.1.6　Spyder ····· 40
2.2　结语 ····· 42
2.3　延伸阅读 ····· 43
第 3 章　入门示例 ····· 45
3.1　隐含波动率 ····· 46

3.2 蒙特卡洛模拟……54
3.2.1 纯 Python……56
3.2.2 用 NumPy 向量化……57
3.2.3 利用对数欧拉方法实现全向量化……59
3.2.4 图形化分析……60
3.2.5 技术分析……62
3.3 结语……67
3.4 延伸阅读……68

第 2 部分 金融分析和开发

第 4 章 数据类型和结构……71
4.1 基本数据类型……72
4.1.1 整数……72
4.1.2 浮点数……73
4.1.3 字符串……75
4.2 基本数据结构……77
4.2.1 元组……77
4.2.2 列表……78
4.2.3 离题：控制结构……80
4.2.4 离题：函数式编程……81
4.2.5 字典……82
4.2.6 集合……84
4.3 NumPy 数据结构……85
4.3.1 用 Python 列表形成数组……85
4.3.2 常规 NumPy 数组……87
4.3.3 结构数组……90
4.4 代码向量化……91
4.5 内存布局……93
4.6 结语……95
4.7 延伸阅读……95
第 5 章 数据可视化……97
5.1 二维绘图……97
5.1.1 一维数据集……98
5.1.2 二维数据集……103
5.1.3 其他绘图样式……109
5.2 金融学图表……116
5.3 3D 绘图……119

5.4 结语 ····· 122
5.5 延伸阅读 ····· 122
第 6 章 金融时间序列 ····· 123
6.1 pandas 基础 ····· 124
6.1.1 使用 DataFrame 类的第一步 ····· 124
6.1.2 使用 DataFrame 类的第二步 ····· 127
6.1.3 基本分析 ····· 131
6.1.4 Series 类 ····· 134
6.1.5 GroupBy 操作 ····· 135
6.2 金融数据 ····· 136
6.3 回归分析 ····· 142
6.4 高频数据 ····· 150
6.5 结语 ····· 154
6.6 延伸阅读 ····· 154
第 7 章 输入/输出操作 ····· 155
7.1 Python 基本 I/O ····· 156
7.1.1 将对象写入磁盘 ····· 156
7.1.2 读写文本文件 ····· 159
7.1.3 SQL 数据库 ····· 160
7.1.4 读写 NumPy 数组 ····· 162
7.2 Pandas 的 I/O ····· 164
7.2.1 SQL 数据库 ····· 165
7.2.2 从 SQL 到 pandas ····· 166
7.2.3 CSV 文件数据 ····· 168
7.2.4 Excel 文件数据 ····· 169
7.3 PyTables 的快速 I/O ····· 170
7.3.1 使用表 ····· 170
7.3.2 使用压缩表 ····· 175
7.3.3 使用数组 ····· 176
7.3.4 内存外计算 ····· 177
7.4 结语 ····· 179
7.5 延伸阅读 ····· 180
第 8 章 高性能的 Python ····· 181
8.1 Python 范型与性能 ····· 182
8.2 内存布局与性能 ····· 184
8.3 并行计算 ····· 186
8.3.1 蒙特卡洛算法 ····· 186

8.3.2 顺序化计算 ····· 187
8.3.3 并行计算 ····· 188
8.3.4 性能比较 ····· 191
8.4 多处理 ····· 191
8.5 动态编译 ····· 193
8.5.1 介绍性示例 ····· 193
8.5.2 二项式期权定价方法 ····· 195
8.6 用 Cython 进行静态编译 ····· 199
8.7 在 GPU 上生成随机数 ····· 201
8.8 结语 ····· 205
8.9 延伸阅读 ····· 205
第 9 章 数学工具 ····· 207
9.1 逼近法 ····· 208
9.1.1 回归 ····· 208
9.1.2 插值 ····· 218
9.2 凸优化 ····· 221
9.2.1 全局优化 ····· 222
9.2.2 局部优化 ····· 223
9.2.3 有约束优化 ····· 224
9.3 积分 ····· 226
9.3.1 数值积分 ····· 228
9.3.2 通过模拟求取积分 ····· 228
9.4 符号计算 ····· 229
9.4.1 基本知识 ····· 229
9.4.2 方程式 ····· 230
9.4.3 积分 ····· 231
9.4.4 微分 ····· 232
9.5 结语 ····· 233
9.6 延伸阅读 ····· 233
第 10 章 推断统计学 ····· 235
10.1 随机数 ····· 236
10.2 模拟 ····· 241
10.2.1 随机变量 ····· 241
10.2.2 随机过程 ····· 244
10.2.3 方差缩减 ····· 256
10.3 估值 ····· 259

10.3.1 欧式期权……259
10.3.2 美式期权……263
10.4 风险测度……266
10.4.1 风险价值……266
10.4.2 信用价值调整……270
10.5 结语……272
10.6 延伸阅读……273
第 11 章 统计学……275
11.1 正态性检验……276
11.1.1 基准案例……277
11.1.2 现实世界的数据……284
11.2 投资组合优化……289
11.2.1 数据……290
11.2.2 基本理论……291
11.2.3 投资组合优化……294
11.2.4 有效边界……296
11.2.5 资本市场线……297
11.3 主成分分析……300
11.3.1 DAX 指数和 30 种成分股……301
11.3.2 应用 PCA……301
11.3.3 构造 PCA 指数……302
11.4 贝叶斯回归……305
11.4.1 贝叶斯公式……305
11.4.2 PyMC3……306
11.4.3 介绍性示例……307
11.4.4 真实数据……310
11.5 结语……318
11.6 延伸阅读……318
第 12 章 Excel 集成……321
12.1 基本电子表格交互……322
12.1.1 生成工作簿（.xls）……323
12.1.2 生成工作簿（.xslx）……324
12.1.3 从工作簿中读取……326
12.1.4 使用 OpenPyxl……328
12.1.5 使用 pandas 读写……329
12.2 用 Python 编写 Excel 脚本……332

12.2.1 安装 DataNitro ····· 333
12.2.2 使用 DataNitro ····· 333
12.3 xlwings ····· 342
12.4 结语 ····· 342
12.5 延伸阅读 ····· 343
第 13 章 面向对象和图形用户界面 ····· 345
13.1 面向对象 ····· 345
13.1.1 Python 类基础知识 ····· 346
13.1.2 简单的短期利率类 ····· 350
13.1.3 现金流序列类 ····· 354
13.2 图形用户界面 ····· 356
13.2.1 带 GUI 的短期利率类 ····· 356
13.2.2 值的更新 ····· 358
13.2.3 带 GUI 的现金流序列类 ····· 360
13.3 结语 ····· 362
13.4 延伸阅读 ····· 362
第 14 章 Web 集成 ····· 365
14.1 Web 基础知识 ····· 366
14.1.1 ftplib ····· 366
14.1.2 httplib ····· 368
14.1.3 urllib ····· 369
14.2 Web 图表绘制 ····· 372
14.2.1 静态图表绘制 ····· 372
14.2.2 交互式图表绘制 ····· 374
14.2.3 实时图表绘制 ····· 375
14.3 快速 Web 应用 ····· 383
14.3.1 交易者的聊天室 ····· 384
14.3.2 数据建模 ····· 384
14.3.3 Python 代码 ····· 385
14.3.4 模板 ····· 391
14.3.5 样式化 ····· 396
14.4 Web 服务 ····· 397
14.4.1 金融模型 ····· 399
14.4.2 实现 ····· 400
14.5 结语 ····· 406
14.6 延伸阅读 ····· 406

第3部分　衍生品分析库

第 15 章　估值框架……409
15.1　资产定价基本定理……409
15.1.1　简单示例……409
15.1.2　一般结果……410
15.2　风险中立折现……412
15.2.1　日期建模和处理……412
15.2.2　固定短期利率……413
15.3　市场环境……415
15.4　结语……418
15.5　延伸阅读……419
第 16 章　金融模型的模拟……421
16.1　随机数生成……422
16.2　泛型模拟类……423
16.3　几何布朗运动……427
16.3.1　模拟类……427
16.3.2　用例……429
16.4　跳跃扩散……431
16.4.1　模拟类……431
16.4.2　用例……434
16.5　平方根扩散……435
16.5.1　模拟类……435
16.5.2　用例……437
16.6　结语……438
16.7　延伸阅读……440
第 17 章　衍生品估值……441
17.1　泛型估值类……441
17.2　欧式行权……445
17.3　估值类……445
17.4　美式行权……451
17.4.1　最小二乘蒙特卡洛方法……451
17.4.2　估值类……453
17.4.3　用例……454
17.5　结语……457
17.6　延伸阅读……458

第 18 章　投资组合估值……459
18.1　衍生品头寸……460
18.1.1　类……460
18.1.2　用例……462
18.2　衍生品投资组合……463
18.2.1　类……463
18.2.2　用例……467
18.3　结语……472
18.4　延伸阅读……474
第 19 章　波动率期权……475
19.1　VSTOXX 数据……476
19.1.1　VSTOXX 指数数据……476
19.1.2　VSTOXX 期货数据……477
19.1.3　VSTOXX 期权数据……479
19.2　模型检验……480
19.2.1　相关市场数据……480
19.2.2　期权建模……481
19.2.3　检验过程……483
19.3　基于 VSTOXX 的美式期权……487
19.3.1　期权头寸建模……487
19.3.2　期权投资组合……488
19.4　结语……489
19.5　延伸阅读……490
附录 A　精选的最佳实践……491
附录 B　看涨期权类……499
附录 C　日期和时间……503

第 1 部分

Python 与金融

本部分介绍 Python 的金融学应用，包括 3 章。

- 第 1 章简短地讨论 Python 的总体情况，证明 Python 确实适合于处理金融行业和财务（数据）分析中遇到的技术难题。
- 第 2 章介绍 Python 基础架构和工具，目的是简洁地概述用 Python 开始交互式分析和应用程序开发所需要了解的最重要知识；相关的附录 A 纵览一些精选的 Python 开发最佳方法。
- 第 3 章立即进入 3 个具体的金融实例：说明如何用 Python 计算期权的隐含波动率、如何用 Python 和数组库 Numpy 模拟金融模型，以及如何实现基于趋势投资策略的事后检验。本章为读者提供使用 Python 进行金融分析的感性认识——在这一阶段，细节并不重要，在第 2 部分会对所有细节进行解释。

第 1 章

为什么将 Python 用于金融

银行本质上是技术公司。

——Hugo Banziger

1.1 Python 是什么

Python 是一种高级的多用途编程语言，广泛用于各种非技术和技术领域。在 Python 网站上，你可以找到如下行动纲领（https://www.python.org/doc/essays/blurb）：

> Python 是一种具备动态语义、面向对象的解释型高级编程语言。它的高级内建数据结构和动态类型及动态绑定相结合，使其在快速应用开发上极具吸引力，也适合于作为脚本或者“粘合剂”语言，将现有组件连接起来。Python 简单、易学的语法强调可读性，因此可以降低程序维护成本。Python 支持模块和软件包，鼓励模块化和代码重用。Python 解释程序和大量标准库可以源代码或者二进制形式免费取得，用于所有主要平台，并且可以随意分发。

上述纲领很好地描述了 Python 成为当今主要编程语言之一的原因。当前，在学校、Web 公司、大型企业和金融机构以及任何科学领域，都有初学者和熟练的专业开发人员在使用 Python。

Python 有如下特征。

开放源码

> Python 和大部分可用的支持库及工具都是开源的，通常使用相当灵活和开放的许可证。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

解释型

Cpython 参考实现是该语言的一个解释程序，在运行时将 Python 代码翻译为可执行字节代码。

多重范型

Python 支持不同的编程和实现范型，例如面向对象和命令式、函数式或者过程式编程。

多用途

Python 可以用于快速、交互式代码开发，也可以用于构建大型应用程序；它可以用于低级系统操作，也可以承担高级分析任务。

跨平台

Python 可用于大部分重要的操作系统，如 Windows、Linux 和 Mac OS；它用于构建桌面应用和 Web 应用；可以在最大的群集和最强大的服务器上使用，也可以在树莓派（http://www.raspberrypi.org）这样的小设备上运行。

动态类型

Python 中的类型通常在运行时推知，而不像大部分编译语言那样静态声明。

缩进感知

和大部分其他编程语言不同，Python 使用缩进标记代码块，代替圆括号、方括号或者分号。

垃圾收集

Python 具有自动垃圾收集机制，避免程序员管理内存。

关于 Python 语法及其意义，Python 增强提案 20——即所谓的“Python 之禅”——提供了重要的指导方针。每个交互 shell 都可以用命令 import this 访问它：

```
$ ipython
Python 2.7.6 |Anaconda 1.9.1 (x86_64)| (default, Jan 10 2014, 11:23:15)
Type "copyright", "credits" or "license" for more information.

IPython 2.0.0—An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.
In [1]: import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-and preferably only one-obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea-let's do more of those!
```

1.1.1 Python 简史

Python 对于某些人来说可能还是个新事物，但是它已经出现了很长时间。实际上，早在 20 世纪 80 年代，荷兰人 Guido van Rossum 就开始了开发工作。他现在仍然活跃于 Python 开发中，被 Python 社区授予“仁慈独裁者”的称号（http://en.wikipedia.org/wiki/History_of_Python）。下面是 Python 开发的里程碑：

- 1991 年发行的 Python 0.9.0（第一个发行版本）；
- 1994 年发行的 Python 1.0；
- 2000 年发行的 Python 2.0；
- 2008 年发行的 Python 2.6；
- 2010 年发行的 Python 2.7；
- 2008 年发行的 Python 3.0；
- 2010 年发行的 Python 3.3；
- 2014 年发行的 Python 3.4。

值得注意的是，有两个可用的主要版本仍然在开发之中，更重要的是，它们从 2008 年起并行使用，这有时候令 Python 初学者感到困惑。到本书编著之时，这种情况已经持续了一段时间，因为两个版本之间没有 100%的代码兼容性，而且不是所有流行程序库都可以用于 Python 3.x。大部分可用和生产代码仍然是用 Python 2.6/2.7 编写的，本书基于 2.7.x 版本，但是大部分代码示例应该可以在 3.x 版本上运行。

1.1.2 Python 生态系统

Python 作为一个生态系统，而不仅仅是一种编程语言，其主要特征是有大量可用的库

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

和工具。这些库和工具通常必须在需要（例如，绘图库）时导入或者作为单独的系统进程（例如，Python 开发环境）启动。导入意味着使某个库可用于当前命名空间和当前 Python 解释程序进程。

Python 本身自带了一组大型的程序库，在不同方面增强了基本解释程序。例如，基本数学计算可以在不做任何导入的情况下完成，而更复杂的数学函数必须通过数学库导入：

```
In [2]: 100 * 2.5 + 50
Out[2]: 300.0
In [3]: log(1)
...

NameError: name 'log' is not defined
In [4]: from math import *

In [5]: log(1)
Out[5]: 0.0
```

虽然所谓的“星号导入”（也就是通过 from library import *指令导入库中的所有内容）有时候很方便，但是通常应该使用替代方法，避免命名空间和函数与库的关系方面的歧义。这采用如下方式实现：

```
In [6]: import math

In [7]: math.log(1)
Out[7]: 0.0
```

任何安装中 math 都是可用的标准 Python 库，但是还有许多库是可选安装的，可以和标准库相同的方式使用。这些库来自不同的（Web）来源。然而，通常建议使用某种 Python 分发版本，确保所有库的相互一致（这个主题的更多内容参见第 2 章）。

目前为止介绍的代码示例都使用 IPython（http://www.ipython.org），这可能是最流行的 Python 交互式开发环境（IDE），虽然它出现的时候只是一个增强的 shell，但是现在已经有了许多典型的 IDE 特性（例如，支持性能分析和调试）。它缺乏由 Sublime Text（http://www.sublimetext.com）等高级文本/代码编辑器所提供的功能，因此，将 IPython 与人们选择的某种文本/代码编辑器组合使用、组成 Python 开发过程基本工具集的情况也不少见。

IPyhon 有时候也被称作 Python 生态系统的“杀手级应用”。它从许多方面增强了标准交互式 Shell。例如，它提供了改进的命令行历史功能，并且能够进行简单的对象检查。例如，在函数名称后添加一个？就可以打印函数的帮助文本（添加？？将提供更多信息）：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [8]: math.log?
Type:       builtin_function_or_method
String Form:<built-in function log>
Docstring:
log(x[, base])

Return the logarithm of x to the given base.
If the base not specified, returns the natural logarithm (base e) of x.
In [9]:
```

IPython 有 3 个不同版本：Shell 版本、基于 QT 图形用户界面（QT console）的版本和基于浏览器的版本（Notebook）。这里只是一个摘要的说明，现在还无需操心这些版本的细节，第 2 章将更详细介绍 IPython。

1.1.3 Python 用户谱系

Python 不仅对专业软件开发人员有吸引力，临时开发人员和领域专家、科研开发人员也使用它。

专业软件开发人员寻求高效构建大型应用程序所需的一切工具。Python 支持几乎所有编程范式；有强大的开发工具；从理论上说，Python 可以对付任何工作。这些类型的用户通常构建自己的框架和类，也依靠基础的 Python 和科学栈进行工作，并且尽最大的努力利用生态系统。

科研开发人员和领域专家通常频繁使用某些库和框架，构建他们长年改进和优化的应用程序，并且根据特定的需求调整生态系统。这组用户通常参与较长的交互式会话，快速建立新代码原型，并探索和可视化其研究及领域数据集。

临时开发人员喜欢在已知 Python 具有优势的特定问题上使用 Python。例如，访问 Matplotlib 的展示页面，复制那里提供的某一段可视化代码，根据特殊需求调整这些代码，可能是对这些人有益的一个用例。

Python 用户还有另一个重要的群体：**编程入门者**，也就是刚刚开始编程的人。现在，Python 在大学、专业院校甚至中小学校中已经成为向学生介绍编程的流行语言[1]。这种现象的主要原因之一是其基本语法即使对于非开发人员也很容易学习和理解。此外，Python 支持几乎所有编程风格[2]。

1.1.4 科学栈

某些库的集合被统称为**科学栈**（Scientific Stack），其中包括如下库。

---

1 例如，Python 已经成为纽约城市大学金融工程研究生课程中使用的主要语言之一（http://mfe.baruch.cuny.edu）。——原注

2 http://wiki.python.org/moin/BeginnersGuide 上可以找到许多开发人员和非开发人员学习 Python 入门知识的宝贵资源。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

NumPy（http://www.numpy.org）

NumPy 提供多维数组对象，以存储同构或者异构数据；它还提供操作这一数组对象的优化函数/方法。

SciPy（http://www.scipy.org）

SciPy 是一组子库和函数，实现科学或者金融中常常需要的重要标准功能；例如，你可以找到三次样条插值和数值积分的函数。

Matplotlib（http://www.matplotlib.org）

这是最流行的 Python 绘图和可视化库，提供 2D 和 3D 可视化功能。

PyTables（http://www.pytables.org）

PyTables 是流行的 HDF5 数据存储库封装器（http://www.hdfgroup.org/HDF5/）；这个库实现基于层次数据库/文件格式的优化磁盘 I/O 操作。

pandas（http://pandas.pydata.org）

pandas 在 NumPy 基础上构建，提供更丰富的时间序列和表格数据管理及分析类；它与 Matplotlib 在绘图上、与 PyTables 在数据存储和读取上紧密集成。

根据特定的领域或者问题，科学栈可以通过更多的库进行扩展，这些库多半在一个或者多个上述的基本库基础上构建。但是，**最小公分母（或称基本组成部分）**通常是 NumPy ndarray 类（参见第 4 章）。

仅从编程语言来讲，有许多其他语言在语法和简洁性上可与 Python 比肩。例如，Ruby 也是相当流行的语言，可与 Python 相提并论。在该语言的网站上（http://www.ruby-lang.org），你可以找到如下的描述：

> 一种动态的开放源码编程语言，重视简洁性和效率。它具备简洁的语法，阅读自然、易于编写。

大部分 Python 使用者可能也赞成用相同的陈述描述 Python 本身。但是，对于许多 Python 用户而言，它与 Ruby 等同样具有吸引力的语言之间的区别在于科学栈。这使 Python 不仅是优秀、简洁的语言，还可以代替 Matlab 或者 R 等领域专用语言和工具集。此外，它默认提供各种人员（例如熟练的 Web 开发人员或者系统管理员）所需要的任何功能。

1.2 金融中的科技

现在，我们对 Python 已经有了大致的认识，回头简短地介绍一下科技在金融中的作用就很有意义了。这将使我们更好地评判 Python 在金融行业中已经承担的任务，更重要的是，还可以评判未来承担的任务。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

在某种意义上，科技对于金融机构（例如与工业企业相比）或者财务部门（与其他企业职能部门相比，如后勤）**没有什么特别的**作用。然而，近年来，在创新和监管的刺激下，银行和其他金融机构（如对冲基金）越来越多地发展成为技术公司而不仅仅是金融中介机构。科技成为了全球几乎所有金融机构的重要资产，具备导致竞争优势和劣势的潜力。某些背景信息可以解释这种发展的原因。

1.2.1 科技开销

银行和金融机构共同组成了每年在科技上投入最多的行业。因此，下面的陈述不仅说明科技对金融行业的重要性，也说明了金融行业对科技的重要性：

> 据 IDC 分析师称，银行在 2014 年的科技投入比 2013 年多 4.2%。这位分析师说，全球金融服务的总体 IT 花费在 2014 年将超过 4300 亿美元，2020 年将会超过 5000 亿美元。
>
> ——Crosman 2013

当今的大型跨国银行通常雇佣数千名开发人员，以维护现有系统、构建新系统。具有大量科技需求的大型投资银行每年的科技预算往往达到数十亿美元。

1.2.2 作为业务引擎的科技

科技发展对金融行业的创新和效率增进也有贡献：

> 科技创新已经为更高效的衍生品市场做出了显著的贡献。通过交易技术的创新，尽管交易量和商品报价数量猛增，欧洲期货交易所（Eurex）的交易速度仍然远快于 10 年前...这些显著改进只有在衍生品交易所和票据交换所不断地在 IT 上投入巨额资金的情况下才可能实现。
>
> ——德意志交易所集团 2008

效率提高的副作用之一就是，金融机构往往必须在更为复杂的产品或者交易中寻求竞争优势。这当然会使风险增大，并使风险管理和监控、监管越来越困难。2007 年和 2008 年的金融危机说明了这些发展带来的潜在危险。同样，“算法和计算机失控”也给金融市场带来潜在的风险；2010 年 5 月的所谓“闪电崩盘”事件戏剧性地展现了上述风险，自动化卖出导致某些股票和股票指数在当日大幅度下跌（http://en.wikipedia.org/wiki/2010_Flash_Crash）。

1.2.3 作为进入门槛的科技和人才

一方面，在其他条件不变的情况下，随着时间的推移，科技的进步会降低成本。另一方面，金融机构持续在科技上投入巨资，以增大市场份额、保持自身地位。在今天的金融市场上取得一席之地往往需要在科技和熟练人员上大规模投资。考虑衍生品分析

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

领域的一个例子（也可参见本书第 3 部分的案例研究）：

> 在整个软件生命期中，采用内部 OTC[衍生品]定价策略的公司仅在一个完整的衍生品库的构建、维护和改进上就需要投入 2500～3600 万美元。
>
> ——Ding 2010

构建一个完善的衍生品分析库不仅成本高昂和费时，而且需要有足够的专业人士来进行这项工作。这些专业人士必须有正确的工具和技术，才能完成他们的任务。

我们以下引用的另一段有关美国长期资本管理公司（LTCM）的陈述进一步支持关于科技和人才的观点，LTCM 曾是最受尊敬的计量对冲基金，但是在 20 世纪 90 年代末破产：

> Meriwether 在最新型计算机系统上花费了 2000 万美元，并雇用一个由一流金融工程师组成的团队在 LTCM 操纵该系统，他们在康涅狄格州的格林威治开始工作。这是行业级别的风险管理。
>
> ——Patterson 2010

Meriwether 花费数百万美元才能得到的计算能力在今天可能只需要几千美元就能实现。另一方面，大型金融机构的交易、定价和风险管理已经变得非常复杂，以致于现在必须部署具有数万个计算核心的 IT 基础架构。

1.2.4 不断提高的速度、频率、数据量

金融行业有一个方面最受科技进步的影响：金融交易决策和执行的**速度**及**频率**。Lewis 最近的著作（2014）生动而详细地描述了所谓的“闪电交易”——也就是以可能的最高速度进行的交易。

一方面，可用数据的尺度越来越小使实时反应成为必需的能力。另一方面，交易的速度和频率的提高使数据量进一步增大。这两方面相互补充，推动了金融交易平均时间标度的系统性下降：

> 复兴资本公司的“大奖章”基金在 2008 年获得了 80%的惊人增长率，它以闪电般速度的计算机抓住了市场极端活跃的机会。Jim Simons 是当年世界盈利最高的对冲基金管理人，收入达到 25 亿美元。
>
> ——Patterson 2010

单只股票 30 年的每日股价数据大致包含 7500 个报价。这类数据是大部分现代金融理论的基础。例如，现代投资组合理论（MPT）、资本定价模型（CAPM）和风险价值（VaR）等理论都以每日股价数据为基础。

相比之下，苹果公司（AAPL）典型交易日报价次数大约为 15000——两倍于 30 年间日终报价的数量。这带来了许多挑战。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

数据处理

考虑和处理股票日终报价或者其他金融计量是不够的；在每周的 7 天、每天的 24 个小时内，某些计量措施上发生的事情“太多了”。

分析速度

决策往往必须在几毫秒甚至更短的时间内作出，有必要构建个别分析能力，实时分析大量数据。

理论基础

虽然传统金融理论和概念远称不上完美，但是它们经受了时间的考验（有些时候受到排斥）；对于毫秒级计量能力很重要的今天，仍然缺乏在很长时间内证明是稳定的一致性概念和理论。

原则上，上述挑战都只能由现代科技应对。令人有些惊讶的是，缺乏一致性理论的问题也常常通过技术方法处理，在这种情况下，高速算法利用的是市场微观结构要素（例如，订单流、买卖价差），而不依赖于某种金融推理方法。

1.2.5 实时分析的兴起

金融行业中有一个学科的重要性正在强劲增长：**金融和数据分析**。这种现象与行业中速度、频率和数据率飞速增长有紧密的关系。实际上，实时分析可以视为该行业对这种趋势的反应。

粗略地讲，“金融和数据分析”指的是应用软件和科技，与（可能是先进的）算法和数据收集、处理及分析方法相结合，以获得深刻理解、作出决策或者满足监管需求的学科。这类分析的例子包括银行零售部门中某个金融产品定价结构的变化对销售情况影响的估算。另一个例子是投资银行衍生品复杂投资组合信用价值调整（CVA）的大规模隔夜计算。

金融机构在这种环境下面对两种主要挑战。

大数据

甚至在“大数据”这一术语出现之前，银行和其他金融机构就必须处理海量数据；然而，单一分析任务所处理的数据量随着时间的推移而有了很大的增长，要求计算能力的提高和更大的内存与存储能力。

实时经济

过去，决策者只能依赖结构化的定期计划、决策和（风险）管理过程，而今天面对的是实时完成这些任务的需求；过去在后台通过隔夜批量运行可以完成的任务，现在已经转向前台实时执行。

同样，人们可以观察到科技和金融/商业方法发展之间的相互作用。一方面，有通过现

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

代科技的应用不断改进分析方法速度和能力的需求。另一方面，科技方面的进步使几年甚至几个月之前认为不可能（或者由于预算约束而不可行）的新分析方法成为可能。

分析领域的一个重要趋势是利用 CPU（中央处理单元）的并行结构和 GPGPU（通用图形处理单元）的大规模并行结构。现在的 GPGPU 往往有超过 1000 个计算核心，有时候有必要彻底反思并行性对不同算法的意义。用户通常必须学习新的范型和技术才能利用这种硬件的能力[1]，仍是这方面的障碍之一。

1.3 用于金融的 Python

前一小节介绍了科技在金融中发挥作用的一些领域：

- 金融行业中的科技成本；
- 作为新业务和创新业务引擎的科技；
- 作为金融行业进入门槛的科技；
- 不断提升的速度、频率和数据量；
- 实时分析的兴起。

本节，我们分析 Python 如何帮助你应对这些方面的多种挑战。不过首先让我从更为基础的方面——语言和语法——介绍用于金融的 Python。

1.3.1 金融和 Python 语法

在金融环境中迈出使用 Python 第一步的大部分人都可能要攻克某个算法问题。这和想要解出微分方程、求取积分或者可视化某些数据的科学工作者类似。一般来说，在这一阶段，对正规开发过程、测试、文档或者部署没有太多的思考。然而，这一阶段似乎是人们特别容易爱上 Python 的时候，主要原因是 Python 的语法总体上和用于描述科学问题或者金融算法的数学语法相当接近。

我们可以通过一个简单的金融算法来说明这一现象——通过蒙特卡洛模拟方法估计欧式看涨期权的价值。我们将考虑 Black-Scholes-Merton（BSM）模型（参见第 3 章），在这种模型中期权潜在风险遵循几何布朗运动。

假定我们使用如下数值化**参数**进行估值：

- 初始股票指数水平 S_0=100；
- 欧式看涨期权的行权价格 K=105；
- 到期时间 T=1 年；

1 第 8 章提供了在随机数生成中利用现代 GPGPU 的一个例子。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- 固定无风险短期利率 r=5%；
- 固定波动率 σ =20%。

在 BSM 模型中，到期指数水平是一个随机变量，由公式 1-1 给出，其中 z 是一个标准正态分布随机变量。

公式 1-1　Black-Scholes-Merton（1973）到期指数水平

$$S_T = S_0 \exp\left(\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}z\right) \qquad (1\text{-}1)$$

下面是蒙特卡洛估值过程的**算法描述**。

1. 从标准正态分布中取得 I 个（伪）随机数 $z(i)$，$i \in \{1, 2, \cdots, I\}$。
2. 为给定的 $z(i)$和公式 1-1 计算所有到期指数水平 $S_T(i)$。
3. 计算到期时期权的所有内在价值 $h_T(i)=\max(S_T(i)-K,0)$。
4. 通过公式 1-2 中给出的蒙特卡罗估算函数估计期权现值。

公式 1-2　欧式期权的蒙特卡洛估算函数

$$C_0 \approx e^{-rT}\frac{1}{I}\sum_i h_T(i) \qquad (1\text{-}2)$$

现在，我们将这个问题和算法翻译为 Python 代码。读者可以使用 IPython 跟踪单独的步骤——但是在这一阶段实际上并不是必需的。

首先，我们从参数值开始。这真的很简单。

```
S0 = 100.
K = 105.
T = 1.0
r = 0.05
sigma = 0.2
```

接下来是估值算法。这里，我们第一次使用 NumPy，它使我们的第二项任务变得相当轻松。

```
from numpy import *

I = 100000

z = random.standard_normal(I)
ST = S0 * exp((r - 0.5 * sigma ** 2) * T + sigma * sqrt(T) * z)
hT = maximum(ST - K, 0)
C0 = exp(-r * T) * sum(hT) / I
```

第三步是打印结果。

```
print "Value of the European Call Option %5.3f" % C0
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

输出可能是如下所示[1]：

```
Value of the European Call Option 8.019
```

有下面 3 个方面值得注意。

语法

Python 语法与数学语法相当接近，例如，在参数赋值的方面。

翻译

每条数学或者算法语句一般都可以翻译为单行 Python 代码。

向量化

NumPy 的强项之一是紧凑的向量化语法，例如，允许在单一代码行中进行 10 万次计算。

这段代码可以用于 IPython 等交互式环境。但是，需要频繁重用的代码一般组织为所谓的**模块**（或者**脚本**），也就是带有**.py** 后缀的 Python（文本）文件。本例的这种模块如例 1-1 所示，可以保存为名为 **bsm_msc_euro.py** 的文件。

例 1-1　欧式看涨期权的蒙特卡洛估值

```
#
# Monte Carlo valuation of European call option
# in Black-Scholes-Merton model
# bsm_mcs_euro.py
#
import numpy as np

# Parameter Values
S0 = 100.  # initial index level
K = 105.  # strike price
T = 1.0  # time-to-maturity
r = 0.05  # riskless short rate
sigma = 0.2  # volatility

I = 100000  # number of simulations

# Valuation Algorithm
z = np.random.standard_normal(I)  # pseudorandom numbers
ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T + sigma * np.sqrt(T) * z)
  # index values at maturity
hT = np.maximum(ST - K, 0)  # inner values at maturity
C0 = np.exp(-r * T) * np.sum(hT) / I  # Monte Carlo estimator

# Result Output
print "Value of the European Call Option %5.3f" % C0
```

1　这种数值估算的输出取决于使用的伪随机数。因此，结果可能不同。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

这一小节中的简单算法示例说明Python的基本语法很适合为经典的科学语言二重奏——英语和数学——提供补充。在科学语言组合中添加Python似乎使其更加全面。我们现在拥有：

- 用于*写作和谈论*科学和金融等问题的**英语**；
- 用于*简洁、精确地描述抽象特征、算法、复数等并为其建模*的**数学**；
- *从技术上建立抽象特征、算法、复数等的模型并加以实现*的**Python**。

数学和Python语法

几乎没有任何编程语言像Python这样接近数学语法。因此，数值算法很容易从数学表示翻译为Python实现。使用Python，可以在这些领域中高效地进行原型化、开发和代码维护。

在某些领域中，使用**伪代码**是常见的做法，从而引入了第4个语言家族成员。举个例子，伪代码的任务是以更技术性的方式表示金融算法，不但仍然接近于数学表示，而且和技术实现已经相当接近。除了算法本身，伪代码还考虑了计算机的工作原理。

采用这种方法一般是因为使用大部分编程语言时，技术实现和正式的数学表现形式距离相当“遥远”。大部分编程语言都必须包含许多只在技术上必要的元素，在数学和代码之间很难看到等价的元素。

时下，Python常常以伪代码方式使用，因为它的语法和数学很类似，而且技术“开销”可以控制到最低。这一点是通过该语言所体现的一些高层概念实现的，这些概念不仅有其优势，也带来了风险和其他代价。不过可以肯定，可以在需求出现的时候使用Python，从一开始就遵循其他语言可能需要的严格实现和编码方法。从这个意义上说，Python可以在两个世界中提供最佳的平衡：*高层次的抽象*和*严格的实现*。

1.3.2 Python的效率和生产率

从较高的层次看，使用Python的好处可以从3个维度衡量。

效率

Python如何更快地获得结果、节约成本、节约时间？

生产率

Python如何在相同的资源（人员、资产等）下完成更多的工作？

质量

Python能够让我们做哪些替代技术所不能做到的事情？

对这些特性的讨论当然不可能很全面。然而，可以将某些参数作为出发点。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

在更短的时间里得到成果

Python 效率较为明显的领域之一是交互式的数据分析。这些领域从 IPython 等有力工具和 pandas 之类的程序库获益良多。

考虑一位正在撰写硕士论文的金融专业学生，她对 Google 的股价感兴趣，想要分析 5 年的历史股价信息，以了解股价变动在这段时间内的波动性。她希望找到证据证明这种变动性与某些典型的模型假设相反，是随时间变动而决非固定。而且，结果应该进行可视化，主要的工作如下：

- 从网络上下载 Google 的股价数据；
- 计算收益率的滚动标准差（波动率）；
- 绘制股价数据和结果图表。

这些任务很复杂，在不久之前还被认为是专业金融分析师才能完成的。而在今天，即使是金融专业的学生也可以轻松地对付这类问题。我们来看看具体的做法——此时还不用操心语法的细节（后续的章节中将对所有细节进行解释）。

首先，确保所有必要的库可用。

```
In [1]: import numpy as np
        import pandas as pd
        import pandas.io.data as web
```

其次，读取数据（例如从 Google 网站）。

```
In [2]: goog = web.DataReader('GOOG', data_source='google',
                              start='3/14/2009', end='4/14/2014')
        goog.tail()
Out[2]:             Open    High    Low     Close   Volume
        Date
        2014-04-08  542.60  555.00  541.61  554.90  3152406
        2014-04-09  559.62  565.37  552.95  564.14  3324742
        2014-04-10  565.00  565.00  539.90  540.95  4027743
        2014-04-11  532.55  540.00  526.53  530.60  3916171
        2014-04-14  538.25  544.10  529.56  532.52  2568020

        5 rows × 5 columns
```

第三步，实现对波动率的必要分析。

```
In [3]: goog['Log_Ret'] = np.log(goog['Close'] / goog['Close'].shift(1))
        goog['Volatility'] = pd.rolling_std(goog['Log_Ret'],
                                            window=252) * np.sqrt(252)
```

最后一步，绘制结果图表。为了生成内联图表，我们使用 IPython 的“魔法”命令 **%matplotlib**，加上 **inline** 选项。

```
In [4]: %matplotlib inline
        goog[['Close', 'Volatility']].plot(subplots=True, color='blue',
                                           figsize=(8, 6))
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 1-1 展示了在 IPython 进行的这一简短交互会话所得到的图形化结果。用 4 行代码就足以完成金融分析中遇到的典型复杂任务：数据收集、复杂和重复的数学计算以及结果的可视化，几乎令人觉得不可思议。这个例子中可以看到，**pandas** 使整个时间系列的处理变得就像浮点数上的数学运算那样容易。

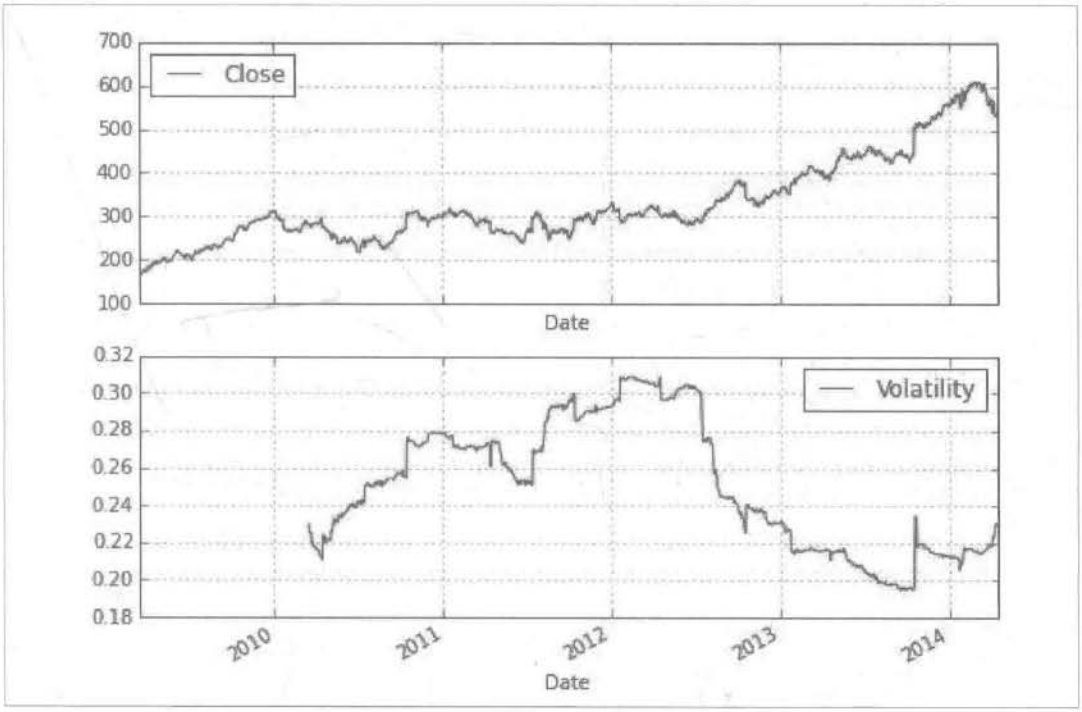


图 1-1　Google 收盘价格和年度波动率

将这个例子转换到专业的金融环境中，可以看出金融分析师们在应用提供高层次抽象的合适 Python 工具和库的时候，能够将焦点放在自身的领域上，而不用关心复杂的技术细节。分析师们可以快速反应，几乎实时提供宝贵的深刻见解，确保自己比竞争对手先行一步。这种效率的提高很容易转换为可度量的财务效果。

确保高性能

一般来说，Python 的语法相当简洁，编码效率相对高是为人们所接受的说法。但是，由于 Python 本质上是解释型语言，因此存在一种偏见，认为 Python 对于金融学中的计算密集任务来说过于缓慢。确实，在某些特定的实现方法下，Python 可能确实很慢，但是，它*并不一定都那么缓慢*——它可以在几乎任何应用领域中表现出高性能。理论上，人们至少可以找到 3 种提高性能的策略。

范型

一般来说，Python 可以用许多不同的方式得出相同的结果，但是这些方式的性能

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

特性有相当大的区别；只要选择合适的方式（例如，特定的库），就可以显著地改善效果。

编译

现在，有许多高性能库提供重要函数的编译版本，或者将 Python 代码静态或者动态地（在运行时或者调用时）编译为机器代码，这种代码的速度要快好几个数量级；比较流行的高性能库有 Cython 和 Numba 等。

并行化

许多计算任务（特别是金融学中的计算任务）可以从并行执行中得到很大好处；这对 Python 来说不足为奇，可以轻松地实现。

使用 Python 实现高性能计算

Python 本身不是一种高性能计算技术。但是，Python 已经发展成为一种访问当前高性能技术的理想平台。在这个意义上，Python 已经成为高性能计算的“黏合剂”语言。

后续的章节将详细介绍这 3 种技术。现在，我们打算坚持用简单而实际的例子介绍这 3 种技术。

金融分析中相当常见的任务之一是在大量数字上计算复杂的数学表达式。在这方面，Python 本身就提供了所有必须的功能：

```
In [1]: loops = 25000000
        from math import *
        a = range(1, loops)
        def f(x):
            return 3 * log(x) + cos(x) ** 2
        %timeit r = [f(x) for x in a]
Out[1]: 1 loops, best of 3: 15 s per loop
```

Python 解释程序在本例中需要 15 秒时间来完成 2500 万次函数 f 的计算。

使用 Numpy 可以完成相同的任务，它提供了优化（也就是预先编译）的函数，处理这种基于数组的运算：

```
In [2]: import numpy as np
        a = np.arange(1, loops)
        %timeit r = 3 * np.log(a) + np.cos(a) ** 2
Out[2]: 1 loops, best of 3: 1.69 s per loop
```

使用 Numpy 将执行时间大幅缩减到 1.7 秒。

甚至有一个库是专门用于此类任务的，这个库称作 **numexpr**，得名于“数值表达式”（Numerical Expressions）。它编译表达式，改善 NumPy 通用功能的性能，例如，在执行期间避免数组在内存中复制：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [3]: import numexpr as ne
        ne.set_num_threads(1)
        f = '3 * log(a) + cos(a) ** 2'
        %timeit r = ne.evaluate(f)
Out[3]: 1 loops, best of 3: 1.18 s per loop
```

使用这种更特殊的方法进一步将执行时间降低到 1.2 秒。而且，numexpr 还内建并行执行单独运算的功能。这使我们能够使用一个 CPU 的所有可用线程：

```
In [4]: ne.set_num_threads(4)
        %timeit r = ne.evaluate(f)
Out[4]: 1 loops, best of 3: 523 ms per loop
```

本例中，利用 2 个核心、4 个线程，执行时间进一步下降到 0.5 秒。总体的性能提升了 30 倍。特别要注意的是，这类改善不需要修改基本问题/算法，也不用了解任何有关编译和并行化问题的知识，即使是非专业人士也可以从较高的层次上去利用这种功能。当然，人们必须知道存在这些功能。

这个例子说明，Python 提供了一些选项，更好地利用现有资源——也就是说，提高生产率。采用串行化技术，每秒可以完成 2100 万次计算，而并行化方法每秒可以计算将近 4800 万次——这只需要告诉 Python 使用所有可用 CPU 线程（而非仅使用一个线程）即可实现。

1.3.3 从原型化到生产

从执行速度的角度看，交互式分析的效率和性能当然是 Python 值得考虑的两个好处。在金融学中使用 Python 的另一个好处初看似乎比较细微；但是细看之下就会发现它本身是一个重要的战略因素。这就是以端到端（*从原型化到生产*）的方式使用 Python 的可能性。

当今全球金融机构的金融开发过程往往是一个分离的两步式过程。一方面，*量化分析师*（“quants”也称“宽客”）负责模型开发和技术原型化。他们喜欢使用 MatLab 和 R 等工具和环境实现快速、交互式的应用程序开发。在开发的这一阶段，性能、稳定性、异常管理、数据访问分离和分析等问题都不重要。人们的主要目标是一种概念验证或者原型，展现某种算法或者整个应用程序主要的必需功能。

一旦原型完成，IT 部门中的*开发人员*接管工作，他们负责将现有*原型代码*翻译为可靠、易于维护和高性能的生产代码。这一阶段通常在用于满足生产性能的 C++或者 Java 语言中有一个范型转换的过程。而且，正规的开发过程还要应用专业工具、版本控制等技术。

这种两步式方法通常会产生一些意外的结果。

效率低下

原型代码不能重用；算法必须实现两次；多余的工作消耗时间和资源。

多种技能集

不同部门展现不同的技能集合，使用不同的语言实现“相同的工作”。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

代码必须以不同的语言存在和维护，往往使用不同的实现风格（例如，从架构的观点上看）。

另一方面，使用 Python 可以实现从最初的交互式原型化步骤到高可靠性、易于维护的生产代码的*合理化*端到端过程。不同部门之间的沟通变得更加简单，工作人员的培训也更为合理，只有一种主要语言覆盖金融应用构建的所有领域，还避免了在开发过程不同步骤中使用不同技术的低效和冗余性。总而言之，Python 可以为金融应用开发和算法实现提供*一致性的技术框架*。

1.4 结语

Python 作为一种语言——但是更多的是作为生态系统——是金融业理想的技术框架。它的特性中有许多好处，比如简洁的语法、高效的开发方法和原型化及生产的易用性等。利用 Python 大量的可用库和工具，似乎能够应付当今金融业中分析、数据量和频率、依从性及监管所引发的大部分问题。即使在较大型的金融机构中，它也具备提供*单一、强大、一致性的框架*，简化端到端开发和生产工作的潜力。

1.5 延伸阅读

下面两本书介绍 Python 在金融中的应用：

- Fletcher, Shayne and Christopher Gardner (2009): *Financial Modelling in Python*.John Wiley & Sons, Chichester, England.
- Hilpisch, Yves (2015): *Derivatives Analytics with Python*. Wiley Finance, Chichester, England. http://derivatives-analytics-with-python.com.

本章中引用了如下资源：

- Crosman, Penny (2013): "Top 8 Ways Banks Will Spend Their 2014 IT Budgets." *Bank Technology News*.
- Deutsche Börse Group (2008): "The Global Derivatives Market—An Introduction." White paper.
- Ding, Cubillas (2010): "Optimizing the OTC Pricing and Valuation Infrastructure." *Celent study*.
- Lewis, Michael (2014): *Flash Boys*. W. W. Norton & Company, New York.
- Patterson, Scott (2010): *The Quants*. Crown Business, New York.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第2章

基础架构和工具

基础架构比体系结构还要重要得多。

——Rem Koolhaas

你可能会说基础架构不是一切，但是没有基础架构，什么东西都可能毫无意义——在现实世界或者技术中都是如此。那么，我们所说的基础架构是指什么呢？理论上，它是使简单 Python 脚本或者更复杂的 Python 应用程序得以执行的硬件和软件组件。

但是，本章并不打算详细介绍硬件基础架构，因为所有 Python 代码和示例应该可以在几乎所有硬件上执行[1]。我们在此也不打算讨论不同的操作系统，因为 Python 代码理论上可以在具备 Python 可用版本的任何操作系统上执行。本章讨论的是如下的主题。

部署

如何以一致性的方式确保部署 Python 和应用程序所需的一切组件可用？本章介绍 Anaconda，这种 Python 分发版本可以相当高效地部署，还要介绍 Python Quant Platform，该平台可以进行基于 Web 和浏览器的部署。

工具

应该使用什么工具进行（交互式）Python 开发和数据分析？本章介绍 2 种最为流行的 Python 开发环境——IPython 和 Spyder。

附录 A 中还将介绍：

1 例如，一般来说，Python 代码和示例甚至可以在大约 30 美元就能买到的树莓派（http://www.raspberrypi.org）上运行，但是某些应用程序很快会发生内存问题。不过，从硬件来说，Python 的要求是相当低的。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

最佳实践

开发 Python 代码时应该遵循哪些最优方法？这个附录简短地回顾了 Python 代码语法和文档的基础。

2.1 Python 部署

本节说明如何在本地（或者服务器上）以及通过 Web 浏览器部署 Python。

2.1.1 Anaconda

有些操作系统自带某个版本的 Python 并且已经安装了一些附加的库。例如，Linux 操作系统常常依靠 Python 作为其主要语言（用于打包、管理等）。但是，下面我们假定 Python 尚未安装，或者打算使用 Anaconda 分发版本安装另一个版本的 Python（与现有版本并行）。

可以从 http://continuum.io/downloads 上下载适合你的操作系统的 Anaconda。考虑使用 Anaconda 进行 Python 部署有几个原因。

库/软件包

可以在一个安装步骤中得到 100 多个最重要的 Python 库和软件包；特别是，可以版本一致的方式安装所有软件包（也就是说，所有库和软件包可以相互配合）[1]。

开放源码

Anaconda 分发版本一般是免费的[2]，包含在分发版本中的所有库和软件包也是如此。

跨平台

Anaconda 可以用于 Windows、Mac OS 和 Linux 平台。

分离式安装

Anaconda 安装到一个单独的目录，不会干扰任何现有安装；不需要根/管理用户权限。

自动更新

包含在 Anaconda 中的库和软件包可以通过免费在线存储库（半）自动更新。

---

1 对于想要控制所安装的库和软件包的人们，可以使用 Miniconda，它仅自带最小化的 Python 安装。http://conda.pydata.org/miniconda.html。——原注

2 也有包含来自 ContinuumAnalytics 的专利库 Accelerate 的 Anaconda 版本。这种商业化版本的目标是改进典型 Python 的操作性能，必须在许可证下使用。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

这个软件包管理程序可以并行使用多个 Python 版本和多个版本的库（用于试验或者开发/测试目的）；它对虚拟环境的支持也很出色。

在下载 Anaconda 安装程序之后，安装总体来说很简单：在 Windows 平台上，只需双击安装程序文件并按照提示进行。在 Linux 下，打开 shell 程序，转到安装程序所在的目录，输入：

```
$ bash Anaconda -1.x.x-Linux-x86[_64].sh
```

用你的安装程序文件名称代替上面的文件名。然后按照提示进行。在 Apple 计算机上也相同，只要输入：

```
$ bash Anaconda -1.x.x-MacOSX-x86_64.sh
```

确保用正确的文件名代替上述文件名，也可以使用图形化的安装程序。

安装之后，你就拥有了 100 多个库和软件包，立即就可以使用。其中的科学与数据分析软件包在表 2-1 中列出。

表 2-1　Anaconda 中包含的一些库和软件包

名称	描述
BitArray	布尔数组所用的对象类型
Cubes OLAP	在线分析处理（OLAP）应用程序框架
Disco	分布式计算所用的 mapreduce 实现
Gdata	Google 数据协议实现
Hspy	Python HDF5 文件格式封装器
HDF5	快速 I/O 操作所用的文件格式
IPython	交互式开发环境（IDE）
lxml	用 Python 处理 XML 和 HTML
matplotlib	标准 2D 和 3D 绘图库
MPI4Py	消息解析接口（MPI）的并行计算实现
MPICH2	另一个 MPI 实现
NetworkX	构建和分析网络模型及算法
numexpr	优化的数值表达式执行
NumPy	强大的数组类及该类上的优化函数

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

名称	描述
Pandas	高效处理时间序列数据
PyTables	使用 HDF5 的层次化数据库
SciPy	科学函数集合
Scikit-Learn	机器学习算法
Spyder	Python IDE，具有语法检查、调试和检查功能
statsmodels	统计模型
SymPy	符号计算和数学
Theano	数学表达式编译器

如果安装过程成功，应该打开一个新的终端窗口，然后可以在 shell 中输入如下命令启动 Spyder IDE：

```
$ spyder
```

也可以输入如下命令，开始 Python 会话：

```
$ python
Python 2.7.6 |Anaconda 1.9.2 (x86_64)| (default, Feb 10 2014, 17:56:29)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
$
```

到本书编著时，Anaconda 默认安装 Python 2.7.x。它总是自带开源软件包管理程序 conda。关于该工具的有用信息可以通过下列命令获得：

```
$ conda info
Current conda install:

             platform : osx-64
        conda version : 3.4.1
       python version : 2.7.6.final.0
     root environment : /Library/anaconda  (writable)
  default environment : /Library/anaconda
     envs directories : /Library/anaconda/envs
        package cache : /Library/anaconda/pkgs
         channel URLs : http://repo.continuum.io/pkgs/free/osx-64/
                        http://repo.continuum.io/pkgs/pro/osx-64/
          config file : None
    is foreign system : False

$
```

conda 允许在本地和可用的在线存储库中搜索库和软件包：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
$ conda search pytables
Fetching package metadata: ..
pytables                     .  2.4.0                np17py27_0  defaults
                                2.4.0                np17py26_0  defaults
                                2.4.0                np16py27_0  defaults
                                2.4.0                np16py26_0  defaults
                             .  3.0.0                np17py27_0  defaults
                                3.0.0                np17py26_0  defaults
                                3.0.0                np16py27_0  defaults
                                3.0.0                np16py26_0  defaults
                             .  3.0.0                np17py33_1  defaults
                             .  3.0.0                np17py27_1  defaults
                                3.0.0                np17py26_1  defaults
                             .  3.0.0                np16py27_1  defaults
                                3.0.0                np16py26_1  defaults
                                3.1.0                np18py33_0  defaults
                             *  3.1.0                np18py27_0  defaults
                                3.1.0                np18py26_0  defaults
                                3.1.1                np18py34_0  defaults
                                3.1.1                np18py33_0  defaults
                                3.1.1                np18py27_0  defaults
                                3.1.1                np18py26_0  defaults
```

本例的结果包含可以下载和安装以及已经安装（由*号标出）的 Pytables 版本。类似地，**list** 命令显示匹配某个模式的所有本地安装软件包。下面命令列出以“**pyt**”开始的所有软件包：

```
$ conda list ^pyt
# packages in environment at /Library/anaconda:
#
pytables                  3.1.0                np18py27_0
pytest                    2.5.2                    py27_0
python                    2.7.6                         1
python-dateutil           1.5                      <pip>
python.app                1.2                      py27_1
pytz                      2014.2                   py27_0
```

还可以使用基于正则表达式的更复杂模式，例如：

```
$ conda list ^p.*les$
# packages in environment at /Library/anaconda:
#
pytables                  3.1.0                np18py27_0
$
```

假定我们想要在 2.7.x 版本之外使用 Python 3.x。软件包管理程序 **conda** 可以创建一个环境实现这一目标。下面的输出展示了工作原理：

```
The following packages will be downloaded:

    package                    |            build
    ---------|------
    anaconda-1.9.2             |       np18py33_0           2 KB
    …
    xlsxwriter-0.5.2           |          py33_0         168 KB
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
The following packages will be linked:

    package                    |            build
    ---------|------
    anaconda-1.9.2             |       np18py33_0   hard-link
    …
    zlib-1.2.7                 |                1   hard-link

Proceed ([y]/n)?
```

当你输入 **y** 确认创建时，**conda** 将按照计划进行（下载、提取和链接软件包）：

```
*******UPDATE**********

Fetching packages …
anaconda-1.9.2-np18py33_0.tar.bz2 100% |###########| Time: 0:00:00 173.62 kB/s
…
xlsxwriter-0.5.2-py33_0.tar.bz2 100% |#############| Time: 0:00:01 131.32 kB/s
Extracting packages …
[      COMPLETE      ] |#########################| 100%
Linking packages …
[      COMPLETE      ] |#########################| 100%
#
# To activate this environment, use:
# $ source activate py33test
#
# To deactivate this environment, use:
# $ source deactivate
#
```

现在，按照 **conda** 的建议激活新环境：

```
$ source activate py33test
discarding /Library/anaconda/bin from PATH
prepending /Library/anaconda/envs/py33test/bin to PATH
(py33test)$ python
Python 3.3.4 |Anaconda 1.9.2 (x86_64)| (default, Feb 10 2014, 17:56:29)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print "Hello Python 3.3"  # this shouldn't work with Python 3.3
  File "<stdin>", line 1
    print "Hello Python 3.3"  # this shouldn't work with Python 3.3
                            ^
SyntaxError: invalid syntax
>>> print ("Hello Python 3.3")  # this syntax should work
Hello Python 3.3
>>> exit()
$
```

很明显，我们现在确实进入了 Python 3.3 的世界，可以从显示的 Python 版本和需要为 **Print** 语句加入圆括号才能正确工作[1]中看出这一点。

1 这只是从 Python 2.7.x 到 3.x 的一个细微无害的语法更改，但可能使某些 Python 新手感到困惑。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

多个 Python 环境

使用 conda 软件包管理程序，可以在一台机器上安装和使用多个单独的 Python 环境。这和其他功能一起，简化了 Python 代码与不同 Python 版本兼容性的测试。

单独的库和软件包可以使用 **conda install** 命令或在 **Anaconda** 总体安装过程中安装：

```
$ conda install scipy
```

或者安装到特定的环境，如：

```
$ conda install -n py33test scipy
```

这里，**py33test** 是我们之前创建的环境。类似地，你可以轻松地更新单个软件包：

```
$ conda update pandas
```

下载和链接的包取决于安装包的版本。这可能很少，也可能很多，例如，当软件包有许多相关软件包尚未下载最新版本时。对于我们新创建的环境，更新采用如下方式：

```
$ conda update -n py33test pandas
```

最后，**conda** 可以用 **remove** 命令简单地从主安装或者特定环境中删除软件包，基本用法如下：

```
$ conda remove scipy
```

对于某个环境则是：

```
$ conda remove -n py33test scipy
```

由于删除是“最终”操作，你可能想要“预演”该命令：

```
$ conda remove –dry-run -n py33test scipy
```

如果你很确定，可以继续进行实际的删除。要回到最初的 Python 和 Anaconda 版本，可以使该环境不活动：

```
$ source deactivate
```

最后，我们可以使用 **remove** 和**--all** 选项，清理整个环境：

```
$ conda remove –all -n py33test
```

软件包管理程序 conda 使 Python 部署变得相当便利。除了本节展示的基本功能之外，还有许多更高级的功能。详细文档可以在 http://conda/pydata.org/docs/上找到。

2.1.2 Python Quant Platform

有许多原因促使我们希望通过 Web 浏览器部署 Python，举例如下。

无需安装

完整 Python 环境的本地安装可能很复杂（例如，在具有许多计算机的大型组织中），

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

支持和维护的成本很高；通过 Web 浏览器使用 Python 使部署在某些情况下更为高效。

使用（更好的）远程硬件

对于复杂、计算和内存密集的分析任务，本地计算机可能无法执行；使用（多台）具备多核心、更大内存、可能还有 GPGPU 的共享服务器可以更高效地完成这些任务。

协作

例如，在单一或者多台服务器上与某个团队合作，可以使协作更简单，而且能够增进效率：数据不用移动到每台本地机器上，在分析任务完成之后，结果也不用移回某个中央存储单元，在团队成员之间分发。

Python Quant Platform 是一个基于 Web 和浏览器的金融分析和协作平台，由 Python Quants 股份有限公司（http://www.pythonquants.com）开发和维护。你可以在 http://quantplatform .com 注册该平台。它有如下基本组件：

文件管理器

通过 Web GUI 管理文件上传/下载等功能的工具。

Linux 终端

与服务器（例如云中的虚拟服务器实例，或者公司场内运行的专用服务器）协同工作的 Linux 终端；可以使用 Vim、Nano 等进行代码编辑，使用 Git 存储库进行版本控制。

Anaconda

Anaconda 安装提供前面讨论的所有功能，默认情况下可以选择 Python2.7-3.4 之间的版本。

Python Shell

标准 Python Shell。

IPython Shell

增强的 IPython shell。

IPython Notebook

IPython 的浏览器版本，通常将它作为核心工具使用。

聊天室/论坛

用于协作、交换想法和上传/下载研究文档等。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

高级分析

除了 Linux 服务器和 Python 环境之外，该平台提供分析功能，例如投资组合分析、风险分析和衍生品分析，以及交易策略的事后检验（特别是 DX 分析；第 3 部分可以看到该库的一个简化而全功能的版本）；其中还有一个 R 语言栈，可以从 IPython Notebook 中调用 R 函数。

标准 API

供领先数据提供商的数据交付服务使用、基于 Python 的标准 API。

至于协作，Python Quant Platform 允许用户在一个“公司”下定义某些“用户组”，每个组对不同 Python 项目（即目录和文件）有不同的权限。该平台很容易伸缩，并可通过 Docker 容器（http://docker.com）部署。图 2-1 展示了 Python Quant Platform 主屏幕的截图。

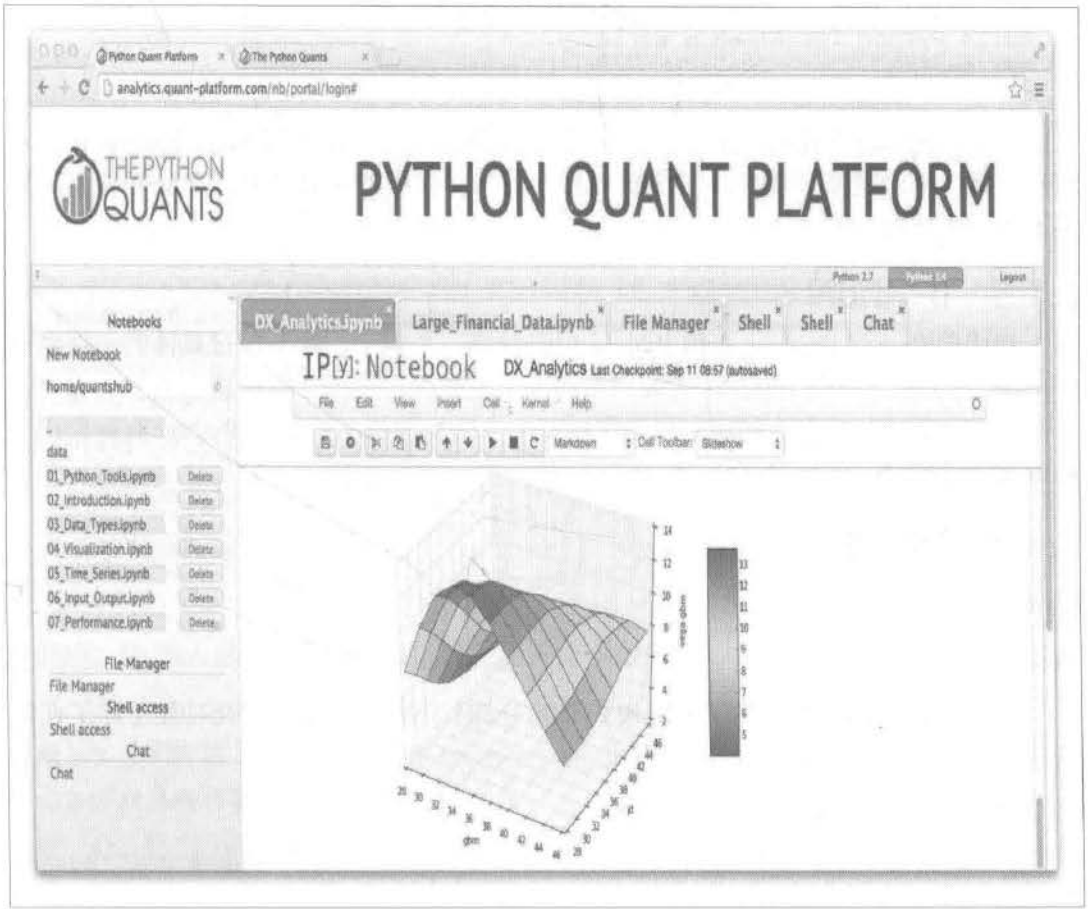


图 2-1　Python Quant Platfrom 屏幕截图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

2.1.3 工具

编程语言的成功和流行，在某种程度上归功于该语言可用的工具。Python 长时间以来都被认为是一种精细、易学、易用的语言，但是缺乏用于交互分析或者开发的一组具有吸引力的工具。这一点已经有了改变，现在有大量工具可以帮助分析人员和开发人员在 Python 上取得很高的效率。对这些工具不可能做全面的概述，但是，我们可以重点介绍当今最流行的两种工具：IPython 和 Spyder[1]。

2.1.4 Python

从完整性出发，我们首先考虑使用标准的 Python 解释程序。从系统 shell/命令行接口，输入 python 就可以调用 Python：

```
$ python
Python 2.7.6 |Anaconda 1.9.2 (x86_64)| (default, Feb 10 2014, 17:56:29)
[GCC 4.0.1 (Apple Inc. build 5493)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print "Hello Python for Finance World."
Hello Python for Finance World.
>>> exit()
$
```

可以在标准提示符下进行很多 Python 工作，但是大部分人都更愿意使用 IPython 作为默认工具，因为这个环境提供标准解释程序提示符提供的*一切*功能，在此基础上还提供了*更多的*功能。

2.1.5 IPython

第 1 章中曾经使用 IPython 介绍 Python 代码的第一个例子。本节通过具体的示例概述 IPython 的功能。围绕 IPython 发展出来的一个完整生态系统非常成功和具有吸引力，以至于其他语言的用户也利用它所提供的基本方法和架构。例如，IPython 有一个用于 **Julia** 语言的版本（http://julialang.org）。

从 shell 到浏览器

IPython 有 3 种风格：

Shell

顾名思义，Shell 版本基于系统和 Python shell；不包含任何图形化功能（除了在单独窗口中显示图表以外）。

QT 控制台

这一版本基于 QT 图形用户界面框架（http://qt-project.org），功能更丰富，例如，

1 对于 Windows 用户和开发人员，Python 在 Visual Studio 中的全面集成是很有吸引力的替代环境。甚至可以找到一整组用于 Visual Studio 的 Python 工具（http://pytools.codeplex.com）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

允许在线图形。

Notebook

这是基于 JavaScript 的 Web 浏览器版本，它已经成为 Python 社区中用于交互分析最受欢迎的版本，也可以用于教学、演示等。

Shell 版本可以简单地在 shell 中输入 **ipython** 调用：

```
$ ipython
Python 2.7.6 |Anaconda 1.9.2 (x86_64)| (default, Feb 10 2014, 17:56:29)
Type "copyright", "credits" or "license" for more information.

IPython 2.0.0 - An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.
In [1]: 3 + 4 * 2
Out[1]: 11
In [2]:
```

使用**--pylab** 选项在命名空间中导入一组科学和数据分析库，例如 **NumPy**：

```
$ ipython -pylab
Python 2.7.6 |Anaconda 1.9.2 (x86_64)| (default, Feb 10 2014, 17:56:29)
Type "copyright", "credits" or "license" for more information.

IPython 2.0.0 - An enhanced Interactive Python.
?         -> Introduction and overview of IPython's features.
%quickref -> Quick reference.
help      -> Python's own help system.
object?   -> Details about 'object', use 'object??' for extra details.
Using matplotlib backend: MacOSX
In [1]: a = linspace(0, 20, 5)  # linspace from NumPy

In [2]: a

Out[2]: array([  0.,   5.,  10.,  15.,  20.])

In [3]:
```

类似地，IPython 的 QT 控制台可以通过如下命令调用：

```
$ ipython qtconsole -pylab inline
```

使用 inline 参数和--pylab 选项，使 IPython 在线绘制图形，图 2-2 展示了带有在线图表的 QT 控制台屏幕截图。

最后，Notebook 版本可以这样调用：

```
$ ipython notebook -pylab inline
```

图 2-3 展示了 IPython Notebook 会话的屏幕截图。**inline** 选项的效果同样是使图表显示

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

在 IPython Notebook 中，而不是显示在单独的窗口中。

总而言之，IPython 核心的调用有许多选项，你可以输入如下命令，获得所有选项的列表：

```
$ ipython –h
```

选项的详细解释参考 IPython 文档（http://www.ipython.org/documentation.html）。

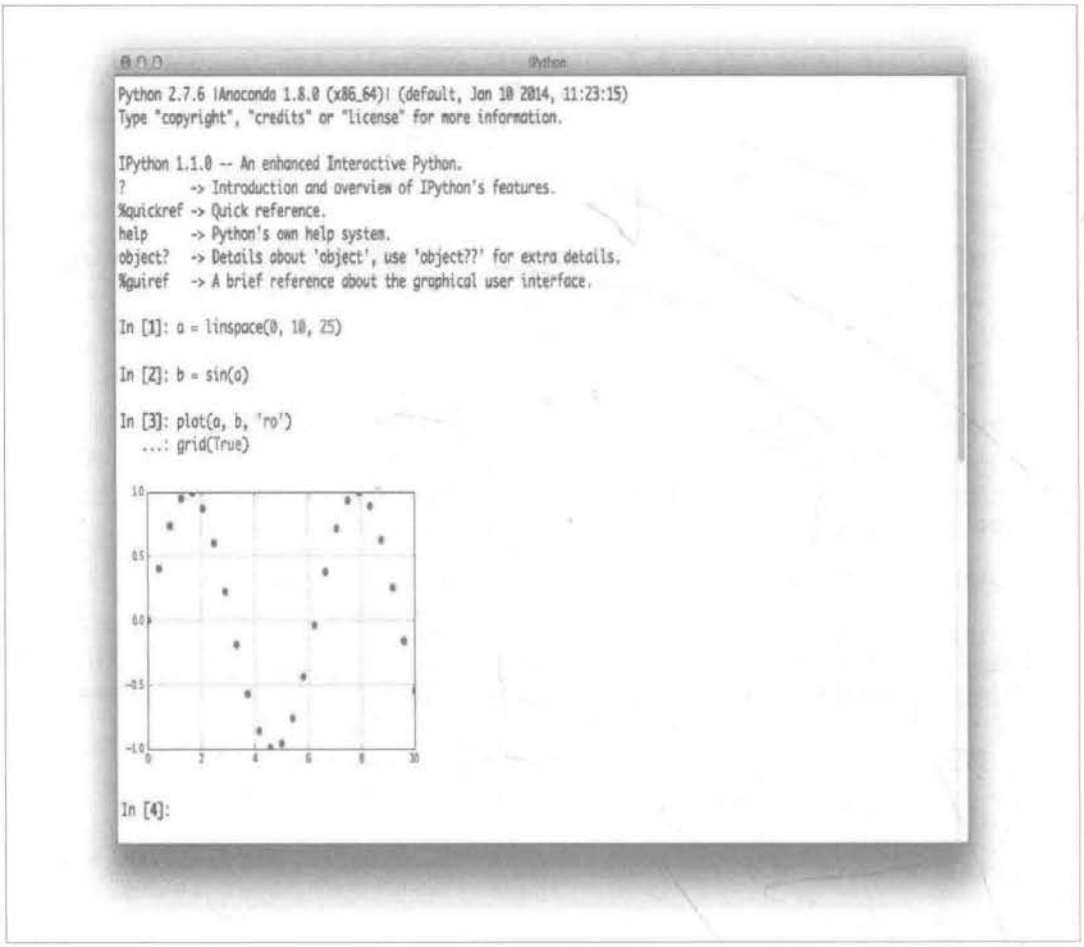


图 2-2　IPython 的 QT 控制台

基本用法

下面，我们描述 IPython Notebook 的基本用法。Notebook 的基本概念是不同类型单元格的使用。这些类型如下所示。

代码

包含可执行的 Python 代码。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

Markdown

包含以 Markdown 语言和 HTML 编写的文本。

原始文本

包含没有格式的文本[1]。

标题（1-6）

用于文本结构的标题，例如节标题。

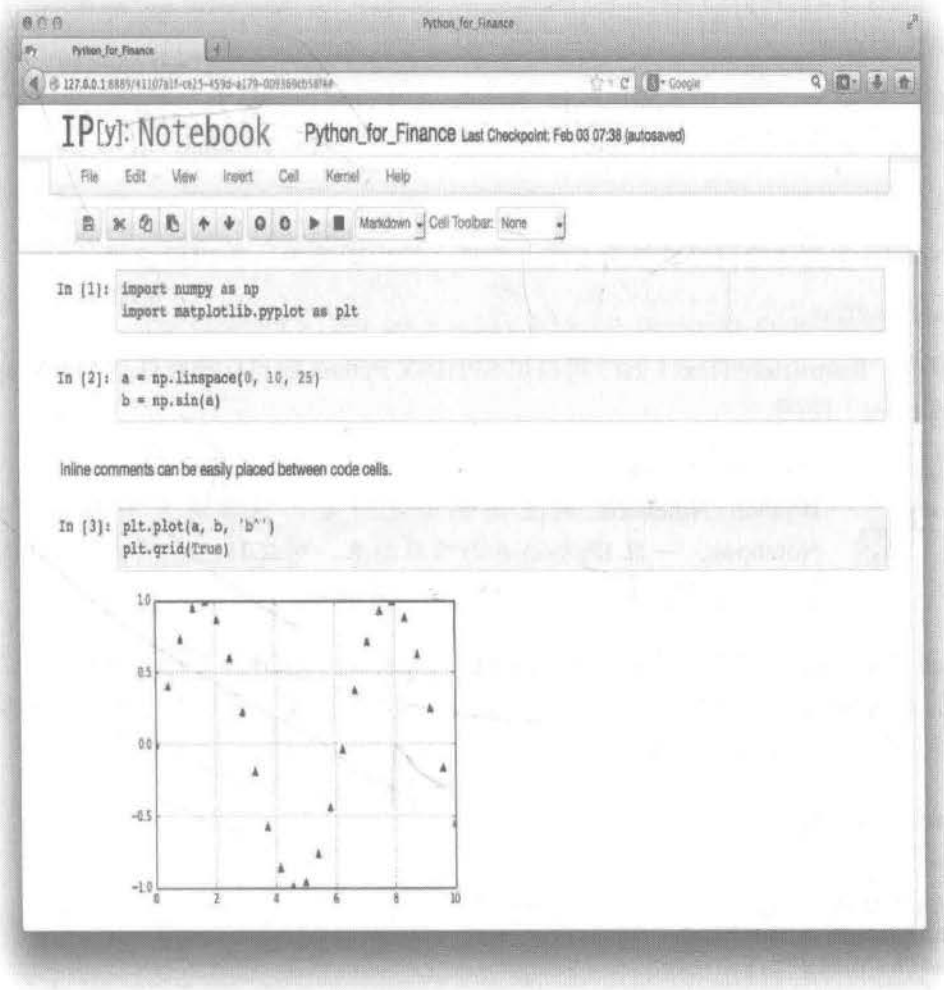


图 2-3　基于浏览器的 IPython Notebook

1　从 IPython 2.0 起，这种单元格被称作原始 MBConvert。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

不同的单元格类型已经表明，**Notebook** 不仅是改进的 Python shell。它的意图是满足多种文档和演示情境的需求。例如，IPython Notebook 文件（后缀为**.ipynb**）可以转换为如下格式：

Python 文件

从 IPython Notebook 文件中生成一个 Python 代码文件（.py），非代码单元格被注释掉。

HTML 页面

从单一 IPython Notebook 文件生成一个 HTML 页面。

HTML 5 幻灯片

使用标记为幻灯片的不同单元格，Notebook 文件可以转换为一个具有多张 HTML5 幻灯片的演示（使用 reveal.js 框架（http://lab.hakim.se/reveal-js/）。

LaTeX/PDF

Notebook 文件也可以转换为 LaTex 文件，然后转换为一个 PDF 文档。

RestructuredText

例如，RestructuredText（.rst）可以供 SPHINX Python 项目文档软件包（http://sphinx-doc.org）使用。

分析和发布平台

IPython Notebook 的主要优势之一是，很容易发布和分享整个 Notebook。一旦 IPython 分析项目结束，可以 HTML 页面或者 PDF 的方式发布它，或者将内容用于幻灯片演示。

IPython Notebook 文件的格式基于 JavaScript 对象标记（JSON）标准。下面是图 2.3 中显示的 Notebook 的文本版本——你会注意到，有些元数据、不同类型的单元格及其内容甚至图形都被翻译成 ASCII 字符：

```
{
"metadata": {
  "name": ""
},
"nbformat": 3,
"nbformat_minor": 0,
"worksheets": [
  {
   "cells": [
    {
     "cell_type": "code",
     "collapsed": false,
     "input": [
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
      "import numpy as np\n",
      "import matplotlib.pyplot as plt"
     ],
     "language": "python",
     "metadata": {},
     "outputs": [],
     "prompt_number": 1
    },
    {
     "cell_type": "code",
     "collapsed": false,
     "input": [
      "a = np.linspace(0, 10, 25)\n",
      "b = np.sin(a)"
     ],
     "language": "python",
     "metadata": {},
     "outputs": [],
     "prompt_number": 2
    },
    {
     "cell_type": "markdown",
     "metadata": {},
     "source": [
      "Inline comments can be easily placed between code cells."
     ]
    },
    {
     "cell_type": "code",
     "collapsed": false,
     "input": [
      "plt.plot(a, b, 'b^')\n",
      "plt.grid(True)"
     ],
     "language": "python",
     "metadata": {},
     "outputs": [
      {
       "metadata": {},
       "output_type": "display_data",
       "png": "iVBORw0KGgoAAAAN…SuQmCC\n",
       "text": [
        "<matplotlib.figure.Figure at 0x105812a10>"
       ]
      }
     ],
     "prompt_number": 3
    }
   ],
   "metadata": {}
  }
 ]
}
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例如，将这种文件转换为 LaTex 时，原始文本单元格可能包含 LaTex 代码，因为这些单元格的内容被转换程序简单地传送。这就是 IPython Notebook 现在常常被用于较大、较复杂文档（如科学研究论文）写作的原因。在单一文件中的代码和文档文本可以转换为许多不同的输出格式。

在金融环境中，这也使 IPython 成为有价值的工具，例如，算法的数学描述和可执行 Python 版本可以存在于同一个文档。根据应用场景，可以生成一个网页（例如企业内联网），PDF 文件（例如客户邮件）或者演示（例如董事会）。利用演示选项也可以实现很多功能，例如，可以跳过包含对演示来说过于冗长的文本段落的单元格。

Notebook 的基本用法相当直观。主要用箭头键浏览它，使用 Shift-Return 或者 Ctrl-Return "执行"单元格。不同之处在于第一个选项在执行之后自动转移到下一个单元格，而第二个选项留在同一个单元格。"执行"单元格的效果取决于单元格类型。如果是代码单元格，则执行代码并显示输出（如果有的话）。如果是 Markdown 单元格，则渲染内容，显示结果。

Markdown 和 Latex

下面展示几个 Markdown 命令的精选示例：

```
**bold** prints the text in bold

*italic* prints the text in italic

_italic_ also prints it in italic

**_italic_** bold and italic

bullet point lists:

* first_bullet
* second_bullet

&ndash; renders to a dash

<br> inserts a line break
```

图 2-4 展示了原始文本单元格（看上去和前面的文本一样）和在 Markdown 单元格中渲染的相同代码。这样，你可以轻松地在一个文档中结合 Python 代码和漂亮的格式化文本。

IPython Notebook 所用 Markdown 语言的详细描述可以在 http://daringfireball.net/projects/markdown 上找到。

如前所述，IPython 的渲染能力不限于 Markdown 语言。IPython 默认还可以渲染由 LaTeX 排版语言描述的数学公式，LaTeX 语言是科学出版的事实标准。例如，考虑第 1 章中 Black-Scholes-Merton（1973）模型中指数水平的公式（公式 1-1）。为了方便起见，我

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

们在公式 2-1 中重复了这个公式。

公式 2-1　Black-Scholes-Merton（1973）到期指数水平

$$S_T = S_0 \exp\left(\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}z\right) \qquad (2\text{-}1)$$

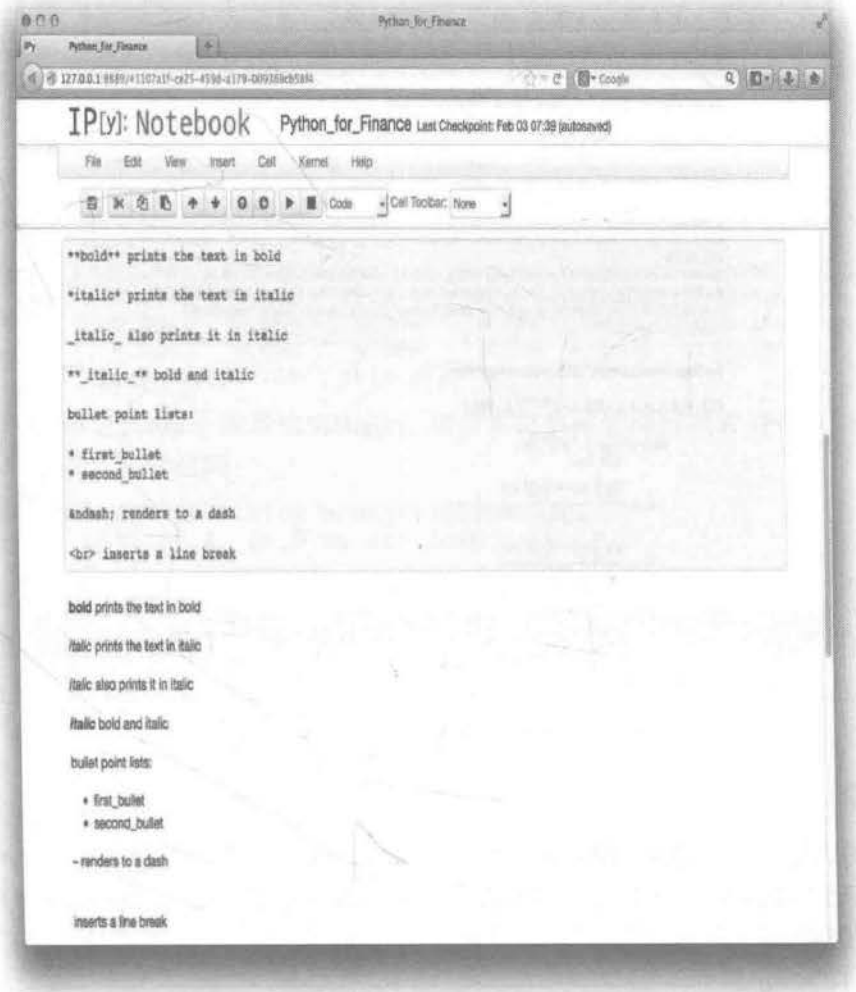


图 2-4　IPython Notebook Markdown 渲染的屏幕截图

描述公式 2-1 的 LaTeX 代码大约如下：

```
S_T = S_0 \exp((r - 0.5\sigma^2) T + \sigma \sqrt{T} z)
```

图 2-5 展示了一个包含 Markdown 文本和 LaTeX 文本的原始文本单元格，以及

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

MarkDown 单元格中渲染的结果，图中还展示了一个更复杂的公式：欧式看涨期权的 Black-Scholes-Merton 期权估价公式——第 3 章中的公式 3-1。

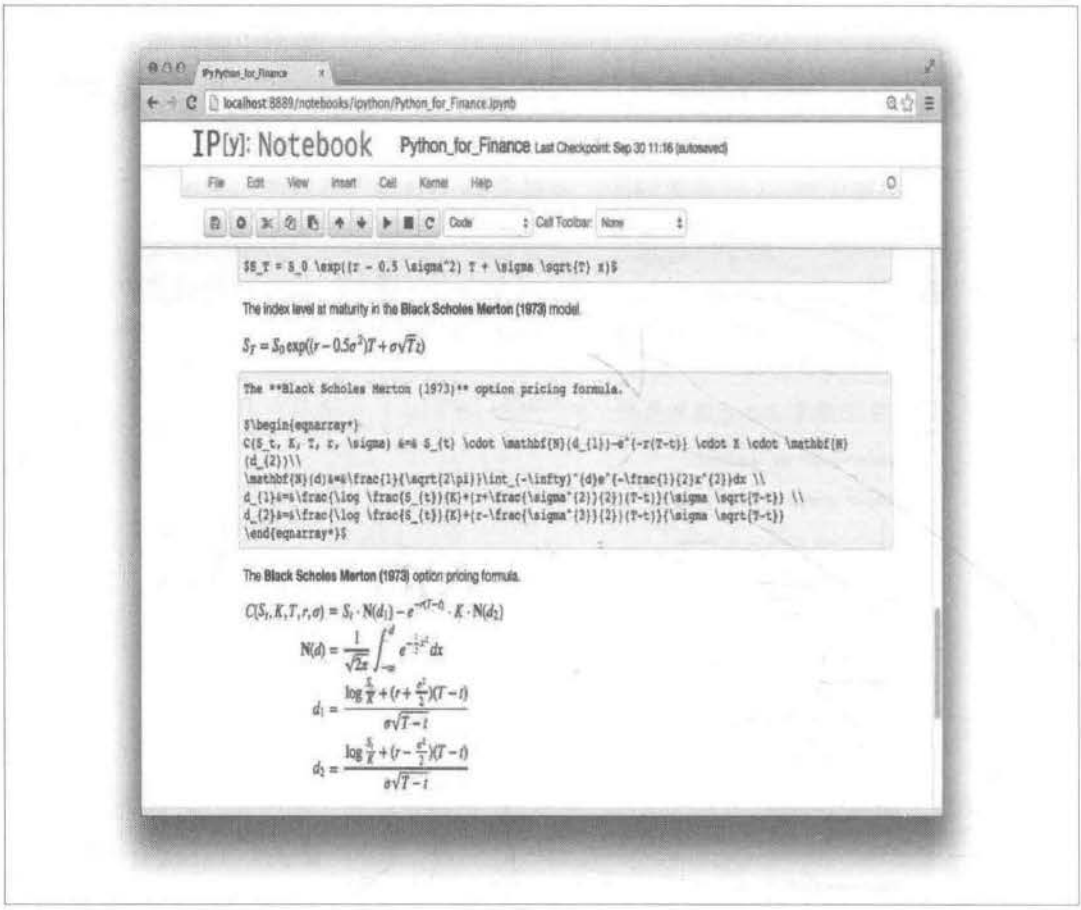


图 2-5　Markdown 和 LaTeX 金融公式

魔法命令

IPython 的优势之一是魔法（Magic）命令。它们的“魔法”表现为在标准的 Python shell 功能性基础上添加某些真正有益和强大的功能。基本信息和这些功能的帮助可以通过如下命令访问：

```
In [1]: %magic


IPython's 'magic' functions
===========================

The magic function system provides a series of functions which allow you to
control the behavior of IPython itself, plus a lot of system-type
features. There are two kinds of magics, line-oriented and cell-oriented.

...
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

如下命令可以在 IPython 会话中生成所有可用魔法命令的列表：

```
In [2]: %lsmagic
```

例如，在交互式计算中，魔法命令可以用于简单的剖析任务。对于这种用例，你可以使用%time 或者%prun：

```
In [3]: import numpy as np

In [4]: %time np.sin(np.arange(1000000))
CPU times: user 31.8 ms, sys: 7.87 ms, total: 39.7 ms
Wall time: 39 ms
Out[5]:
array([ 0.        ,  0.84147098,  0.90929743, ...,  0.21429647,
       -0.70613761, -0.97735203])
In [6]: %prun np.sin(np.arange(1000000))
         3 function calls in 0.043 seconds

   Ordered by: internal time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.041    0.041    0.043    0.043 <string>:1(<module>)
        1    0.002    0.002    0.002    0.002 {numpy.core.multiarray.arange}
        1    0.000    0.000    0.000    0.000 {method 'disable'
        of '_lsprof.Profiler' objects}
```

还有另外一条命令**%timer** 或者**%%timeit**，用于求得单一代码行或者 IPython Notebook 中整个单元格内代码的时间：

```
In [6]: %timeit np.sin(np.arange(1000000))
10 loops, best of 3: 27.5 ms per loop
```

上述函数执行多次循环，获得函数调用或者代码片段持续时间的可靠估算。

无法详细解释 IPython 提供的所有魔法函数。但是，IPython 本身坚持尽可能简化 IPython 及其命令相关信息的查找。其中最有用的方法在表 2-2 中列出（http://bit.ly/ipython_tutorial）。

表 2-2 精选的 IPython 帮助函数

名称	描述
?	IPython 功能的简介和概述
%quickref	快速参考
help	Python 自身的帮助系统
object?	关于“object”的细节；使用 object??可获得更多细节

IPython 的另一个特征是高度可配置。关于配置能力的信息也可以在文档中找到（http://ipython.org/ipython-doc/stable/config/）。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

对 IPython 自定义有帮助的魔法命令之一是**%bookmark**。这个命令可以使用自定义名称将任意目录标记为书签，以便在以后——不管从哪里调用 IPython 核心，也不管当前目录是什么——立刻导航到你标记书签的目录（也就是说，你不用使用 cd）。下面说明如何设置一个书签，如何列出所有书签：

```
In [6]: %bookmark py4fi

In [7]: %bookmark -l
Current bookmarks:
py4fi -> /Users/yhilpisch/Documents/Work/Python4Finance/
```

系统 shell 命令

另一个真正有益的特性是可以直接从 IPython 提示符或者 Notebook 单元格中执行命令行/系统 shell 函数。这时，需要使用！表示后面的命令应该转义为系统 shell（或者在整个单元格应该这样处理时使用%%!）。下面是一个简单的演示：创建一个目录，转到该目录，回到上一级目录，删除该目录：

```
In [7]: !mkdir python4finance

In [8]: cd python4finance/
/Users/yhilpisch/python4finance

In [9]: cd ..
/Users/yhilpisch

In [10]: !rm -rf python4finance/
```

IPython 提供了强大的交互式开发环境应该有的所有功能。初学者和专家们往往都会通过 IPython 进入 Python 世界。在本书中，有大量说明 IPython 在交互式数据和金融分析中应用的示例。你还应该参考 McKinney 的著作（2012）——特别是第 3 章，以了解如何高效使用 IPython 的更多信息。

2.1.6 Spyder

虽然 IPython 可以满足大部分用户交互分和原型化的所有需求，但是较大的项目通常有更高的要求。特别是，IPython 本身没有直接内建的编辑器[1]。对于寻求更传统开发环境的人，Spyder 可能是更好的选择。

和 IPython 类似，Spyder 设计用于支持快速的交互式 Python 开发。不过，它还有更多的特性，例如一个完善的编辑器，更强大的项目管理和调试能力，以及对象与变量检查器和完全继承的 IPython shell 版本。在 Spyder 中，还可以启动标准的 Python 提示符会话。

Spyder 内建的编辑器提供进行 Python 开发所需的一切功能。其中包括（http://code.google.com/p/spyderlib/wiki/Features）下面这些功能。

1 不过，可以为 IPython 配置自己喜欢的编辑器，用魔法命令%editor FILENAME 调用。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

高亮显示

Python、C/C++和 Fortran 代码的语法颜色表示；出现高亮显示。

内省

强大的动态代码内省功能（例如，用一次鼠标点击进行代码完成、调用提示、对象定义）。

代码浏览器

浏览类和函数。

项目管理

定义和管理项目；生成待办事项列表。

即时代码检查

实时获得错误和警告（通过 pyflakes，http://pypi.python.org/pypi/pyflakes/）

调试

设置要与 Python 调式器 pdb（http://dols.python.org/2/librany/pdb.html/）一起使用的断点和条件断点。

此外，Spyder 还提供另一些有益的功能。

控制台

为每个单独的进程打开多个 Python 和 IPython 控制台；从控制台中的活动编辑器选项卡（或者它的各个部分）运行代码。

变量管理器

编辑和比较变量及数组；实时生成数组的 2D 图表；在调试时检查变量。

对象检查器

交互式显示文档字符串；自动渲染富文本格式化。

其他特性

历史日志；与电子表格类似的数组编辑器；直接访问在线帮助；整个项目的管理和浏览；通过 Pylint（http://www.pylint.org）。

图 2-6 提供 Spyder 的屏幕截图，显示文本编辑器（在左侧）、变量检查器（右上）和活动的 Python 控制台（右下）。Spyder 是开始进行 Python 编程的好选择，特别是对于使用过 Matlab 或者 R 提供的环境的人们来说。不过，高级编程人员也可以在单一环境内找到许多有益的开发功能。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

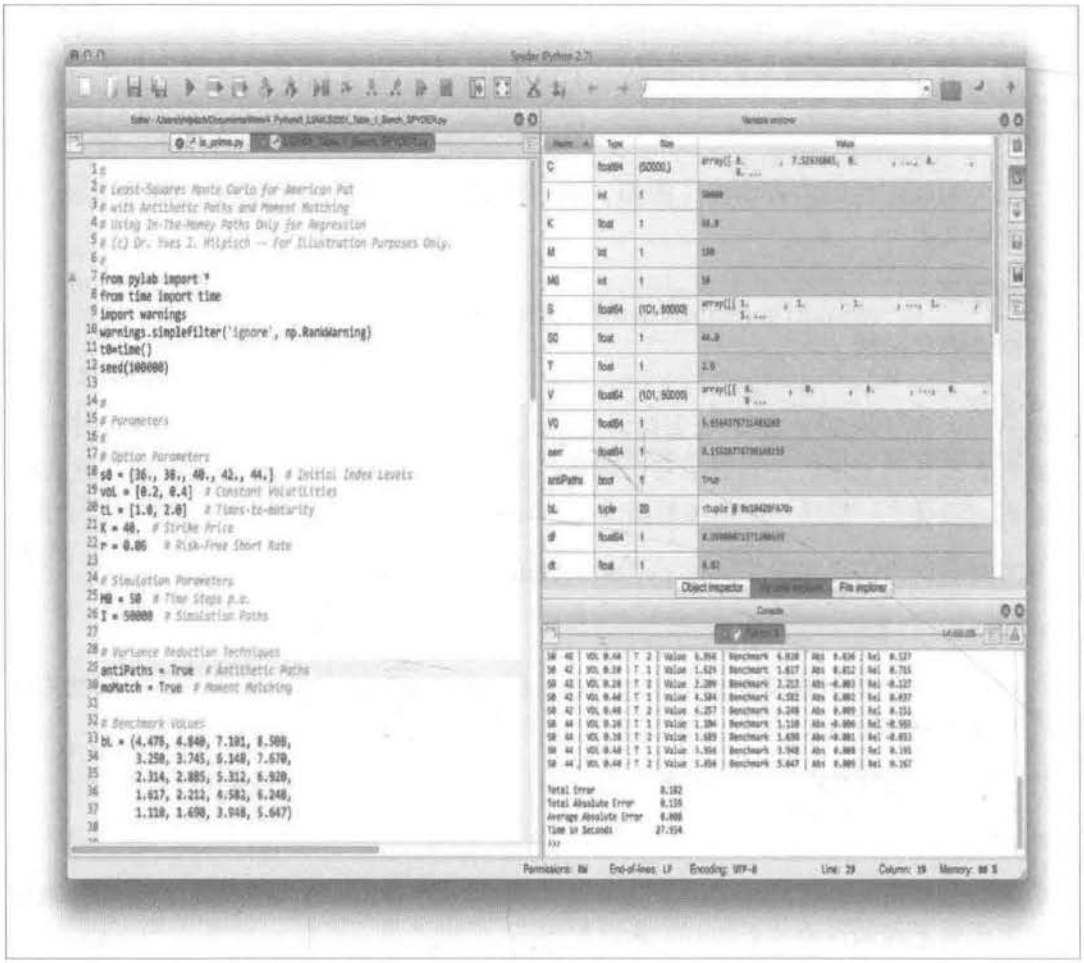


图 2-6　Spyder 屏幕截图

2.2　结语

如果你是初学者、临时 Python 开发人员或者来自不同编程背景的专业人士，Python 的入门一般相当简单，只需要几个简单的步骤。首先，应该安装合适的 Python 分发版本（如 Anaconda），以获得一致的 Python 环境，也可以简化定期更新的过程。

有了 Anaconda 之类的分发版本，就拥有了交互式实践数据和金融分析所需的最重要工具（例如 IPython），或者以更传统的实现-测试-调试风格开发更大应用程序的工具（例如 Spyder）。当然，可以加入自己喜欢的编辑器，这些编辑器可能已经包含了 Python 语法高亮显示功能。如果你还寻求语法和代码检查功能，可以考虑 Spyder 内建的编辑器或者任何以 Python 为焦点的编辑器。

附录 A 介绍了一些*语法*、*文档*和*单元测试*领域中的最优方法。在语法中，空格和空行

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

以及代码块缩进起到了重要的作用。至于文档，应该考虑在任何函数或者类中包含文档字符串，提供输入参数、输出和可能错误等背景和帮助信息以及使用示例。最后，应该从一开始就在开发过程中包含单元测试（至少对于较大的项目或者与更广泛拥护基础共享的项目）并使用专门工具简化测试过程。

2.3 延伸阅读

下面的网络资源有助于本章介绍的主题。

- Anaconda 文档——http://docs.continuum.io/anaconda/
- conda 文档——http://conda.pydata.org/docs/
- IPython 文档——http://ipython.org/ipython-doc/stable/
- IPython Notebook 使用的 Markdown 语言——http://daringfireball.net/projects/markdown/
- 有关 Spyder 的信息——http://code.google.com/p/spyderlib

下面的书籍很好地介绍了 Python 部署和 IPython 开发环境使用：

- Wes McKinney (2012): *Python for Data Analysis*. O'Reilly, Sebastopol, CA

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第3章

入门示例

> 在我们的定义中，定量分析是数学和/或统计学方法在市场数据上的应用。
>
> ——John Forman

本章深入**计量金融学**中的一些具体示例，说明使用 Python 及其金融分析库有多么方便。本章的重点是阐述流程，并没有接触到现实世界中可能很重要的一些细节。而且，我们跳过了 Python 使用方法的细节，因为后面的章节会更进一步做出解释。

具体地说，本章提供如下示例。

隐含波动率

以某些到期日的期权报价倒推出这些期权的隐含波动率，并绘出图表——这是期权交易者和风险管理者每天都要面对的任务。

蒙特卡洛模拟

通过蒙特卡洛技术，模拟股票指数在一段时间中的变化，对选中的结果进行可视化，并计算欧式期权价值。蒙特卡洛模拟是期权数字化定价的基石，也是涉及风险价值计算或者信用值调整的风险管理工作的基础。

技术分析

实施历史时间序列数据的分析，对根据趋势信号的投资策略进行事后验证；专业投资者和有抱负的业余投资者常常进行这类投资分析。

所有例子都以某种方式处理日期-时间信息。附录 C 介绍了用 Python、NumPy 和 pandas 处理这种信息的方法。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

3.1 隐含波动率

在像 Black-Scholes-Merton（1973）这样有深远影响的期权定价公式中，**隐含波动率**是在其他条件不变的情况下输入公式，得出不同期权行权价格和到期日测得市场报价的那些波动率值。在这种情况下，隐含波动率不是模型/公式的输入参数，而是对该公式进行某项（数字化）优化过程的结果。

在下面的讨论中，我们所考虑的例子与某种新一代期权（也就是 VSTOXX 波动率指数期权）有关。提供这些 VSTOXX 期权和相关期货合约的衍生品交易所 Eurex 于 2013 年 1 月建立了一个基于 Python、关于该指数和衍生品合约的全面教程，称作“VSTOXX 高级服务”（http://www.eurexchange.com/advanced-services/）[1]。

然而，在继续研究 VSTOXX 期权之前，让我们首先重现在没有分红的情况下，著名的 Black-Scholes-Merton 欧式看涨期权定价公式（公式 3-1）。

公式 3-1　Black-Scholes-Merton（1973）期权定价公式

$$C(S_t,K,t,T,r,\sigma)=S_t \cdot \mathrm{N}(d_1)-e^{-r(T-t)} \cdot K \cdot \mathrm{N}(d_2) \qquad (3\text{-}1)$$

$$\mathrm{N}(d)=\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{d} \mathrm{e}^{-\frac{1}{2}x^2}\,\mathrm{d}x$$

$$d_1=\frac{\log\frac{S_t}{K}+\left(r+\frac{\sigma^2}{2}\right)(T-t)}{\sigma\sqrt{T-t}}$$

$$d_2=\frac{\log\frac{S_t}{K}+\left(r-\frac{\sigma^2}{2}\right)(T-t)}{\sigma\sqrt{T-t}}$$

不同参数有如下含义：

S_t　在时点 t 的标的物价格水平；

σ　标的物固定波动率（也就是收益的标准差）；

K　期权行权价格；

T　期权到期日；

r　固定无风险短期利率。

现在考虑欧式看涨期权的一个报价 C^*已知的情况。隐含波动率 σ^{imp} 是公式 3-2 中的隐

1　第 19 章也讨论基于 VSTOXX 波动性指数的期权；根据市场报价校验一个期权定价模型，并根据校验后的模型估算美式非交易期权的价值。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

式方程的解。

公式 3-2　给定期权报价的隐含波动率

$$C(S_t, K, t, T, r, \sigma^{imp}) = C^* \tag{3-2}$$

对这个方程没有闭合解，人们必须使用牛顿迭代法等数值求根过程估算正确的解。这种方法是用相关函数的第一个结果进行迭代，直到一定的迭代数量，或者达到某种精度。对于起始值 σ_0^{imp} 和 $0<n<\infty$，我们有正式的公式，如公式 3-3 所示。

公式 3-3　方程式数值化求根的牛顿迭代法

$$\sigma_{n+1}^{imp} = \sigma_n^{imp} - \frac{C(\sigma_n^{imp}) - C^*}{\partial C(\sigma_n^{imp}) / \partial \sigma_n^{imp}} \tag{3-3}$$

期权定价公式对于波动率的偏微分称作 Vega，公式 3-4 给出 Vega 的闭合方式。

公式 3-4　BSM 模型中欧式期权的 Vega

$$\frac{\partial C}{\partial \sigma} = S_t \mathrm{N}(d_1)\sqrt{T-t} \tag{3-4}$$

现在，所需要的金融和数值化工具已经完整——尽管只是粗略地做了描述——我们可以看看假定特例 t=0 时相应的 Python 代码（例 3-1）。

例 3-1　Black-Scholes-Merton（1973）函数

```
#
# Valuation of European call options in Black-Scholes-Merton model
# incl. Vega function and implied volatility estimation
# bsm_functions.py
#

# Analytical Black-Scholes-Merton (BSM) Formula


def bsm_call_value(S0, K, T, r, sigma):
    ''' Valuation of European call option in BSM model.
    Analytical formula.

    Parameters
    ==========
    S0 : float
        initial stock/index level
    K : float
        strike price
    T : float
        maturity date (in year fractions)
    r : float
        constant risk-free short rate
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    sigma : float
        volatility factor in diffusion term

    Returns
    =======
    value : float
        present value of the European call option
    '''
    from math import log, sqrt, exp
    from scipy import stats

    S0 = float(S0)
    d1 = (log(S0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * sqrt(T))
    d2 = (log(S0 / K) + (r - 0.5 * sigma ** 2) * T) / (sigma * sqrt(T))
    value = (S0 * stats.norm.cdf(d1, 0.0, 1.0)
             - K * exp(-r * T) * stats.norm.cdf(d2, 0.0, 1.0))
      # stats.norm.cdf -> cumulative distribution function
      #                   for normal distribution
    return value

# Vega function


def bsm_vega(S0, K, T, r, sigma):
    ''' Vega of European option in BSM model.

    Parameters
    ==========
    S0 : float
        initial stock/index level
    K : float
        strike price
    T : float
        maturity date (in year fractions)
    r : float
        constant risk-free short rate
    sigma : float
        volatility factor in diffusion term

    Returns
    =======
    vega : float
        partial derivative of BSM formula with respect
        to sigma, i.e. Vega
    '''
    from math import log, sqrt
    from scipy import stats

    S0 = float(S0)
    d1 = (log(S0 / K) + (r + 0.5 * sigma ** 2) * T / (sigma * sqrt(T))
    vega = S0 * stats.norm.cdf(d1, 0.0, 1.0) * sqrt(T)
    return vega
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
# Implied volatility function


def bsm_call_imp_vol(S0, K, T, r, C0, sigma_est, it=100):
    ''' Implied volatility of European call option in BSM model.

    Parameters
    ==========
    S0 : float
        initial stock/index level
    K : float
        strike price
    T : float
        maturity date (in year fractions)
    r : float
        constant risk-free short rate
    sigma_est : float
        estimate of impl. volatility
    it : integer
        number of iterations

    Returns
    =======
    simga_est : float
        numerically estimated implied volatility
    '''
    for i in range(it):
        sigma_est -= ((bsm_call_value(S0, K, T, r, sigma_est) - C0)
                      / bsm_vega(S0, K, T, r, sigma_est))
    return sigma_est
```

上述函数只是计算隐含波动率所需的基本函数。当然，我们还需要单独的期权报价，也就是基于 VSTOXX 指数的欧式看涨期权，以及生成隐含波动率的代码。我们将看到如何在交互式 IPython 会话基础上完成这项工作。

我们首先看看所取的报价是哪一天的，也就是参考日期 $t=0$。这一天是 2014 年 3 月 31 日，在这一天，指数的收盘价是 $V0$=17.6639（我们将 S 改为 V，表示现在处理的是波动率指数）：

```
In [1]: V0 = 17.6639
```

对于无风险短期利率，我们假定年率 r=0.01：

```
In [2]: r = 0.01
```

其他所有输入参数都由期权数据（也就是 T 和 K）给出，或者必须计算（σ^{imp}），这些数据存储在一个 pandas DataFrame 对象（参见第 6 章）中，并保存到一个 Pytables 数据库文件（参见第 7 章），必须从磁盘中将其读入内存：

```
In [3]: import pandas as pd
        h5 = pd.HDFStore('./source/vstoxx_data_31032014.h5', 'r')
        futures_data = h5['futures_data']  # VSTOXX futures data
        options_data = h5['options_data']  # VSTOXX call option data
        h5.close()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们需要期货数据，以根据（远期）价值状况选择 VSTOXX 期权的子集。任何时候都有 8 种可交易的 VSTOXX 期货。它们的到期日是下 8 个月的“第三个周五”。在 3 月底，期货的到期日从 4 月的第三个周五到 11 月的第三个周五。下面的 **pandas** 表中的 TTM 代表距离到期日的年份数：

```
In [4]: futures_data
Out[4]:          DATE  EXP_YEAR  EXP_MONTH  PRICE   MATURITY    TTM
        496 2014-03-31      2014          4  17.85 2014-04-18  0.049
        497 2014-03-31      2014          5  19.55 2014-05-16  0.126
        498 2014-03-31      2014          6  19.95 2014-06-20  0.222
        499 2014-03-31      2014          7  20.40 2014-07-18  0.299
        500 2014-03-31      2014          8  20.70 2014-08-15  0.375
        501 2014-03-31      2014          9  20.95 2014-09-19  0.471
        502 2014-03-31      2014         10  21.05 2014-10-17  0.548
        503 2014-03-31      2014         11  21.25 2014-11-21  0.644
```

期权数据集较大，因为在任何交易日，都有各种到期日的看涨和看跌期权参与交易。期权到期日与期货相同，2014 年 3 月 31 日共有 395 种看涨期权报价：

```
In [5]: options_data.info()
Out[5]: <class 'pandas.core.frame.DataFrame'>
        Int64Index: 395 entries, 46170 to 46564
        Data columns (total 8 columns):
        DATE          395 non-null datetime64[ns]
        EXP_YEAR      395 non-null int64
        EXP_MONTH     395 non-null int64
        TYPE          395 non-null object
        STRIKE        395 non-null float64
        PRICE         395 non-null float64
        MATURITY      395 non-null datetime64[ns]
        TTM           395 non-null float64
        dtypes: datetime64[ns](2), float64(3), int64(2), object(1)
In [6]: options_data[['DATE', 'MATURITY', 'TTM', 'STRIKE', 'PRICE']].head()
Out[6]:          DATE   MATURITY    TTM  STRIKE  PRICE
        46170 2014-03-31 2014-04-18  0.049       1  16.85
        46171 2014-03-31 2014-04-18  0.049       2  15.85
        46172 2014-03-31 2014-04-18  0.049       3  14.85
        46173 2014-03-31 2014-04-18  0.049       4  13.85
        46174 2014-03-31 2014-04-18  0.049       5  12.85
```

从 **pandas** 表中很明显看出，有些交易和报价的看涨期权是很便宜的价内期权（指数水平远高于期权行权价格）。也有一些期权是很昂贵的价外期权（指数水平远低于期权行权价格）。因此，我们应该根据对应到期日的期货价值，分析具有一定（远期）价值的看涨期权。我们允许的与期货水平之间最大偏差为 50%。

在开始之前，我们必须在 **options_data DataFrame** 对象中定义一个新列，以存储结果，还需要从例 3-1 的脚本中导入函数：

```
In [7]: options_data['IMP_VOL'] = 0.0
          # new column for implied volatilities
In [8]: from bsm_functions import *
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

下面的代码计算上述所有看涨期权的隐含波动率：

```
In [9]: tol = 0.5  # tolerance level for moneyness
        for option in options_data.index:
            # iterating over all option quotes
            forward = futures_data[futures_data['MATURITY'] == \
                      options_data.loc[option]['MATURITY']]['PRICE'].values[0]
              # picking the right futures value
            if (forward * (1 - tol) < options_data.loc[option]['STRIKE']
                                    < forward * (1 + tol)):
                # only for options with moneyness within tolerance
                imp_vol = bsm_call_imp_vol(
                        V0,  # VSTOXX value
                        options_data.loc[option]['STRIKE'],
                        options_data.loc[option]['TTM'],
                        r,   # short rate
                        options_data.loc[option]['PRICE'],
                        sigma_est=2.,  # estimate for implied volatility
                        it=100)
                options_data['IMP_VOL'].loc[option] = imp_vol
```

在这段代码中，有一些 **pandas** 语法并非一目了然。第 6 章将详细解释 **pandas** 以及在这些操作中的使用方法。在这一阶段，理解如下功能就足够了：

```
In [10]: futures_data['MATURITY']
           # select the column with name MATURITY
Out[10]: 496   2014-04-18
         497   2014-05-16
         498   2014-06-20
         499   2014-07-18
         500   2014-08-15
         501   2014-09-19
         502   2014-10-17
         503   2014-11-21
         Name: MATURITY, dtype: datetime64[ns]

In [11]: options_data.loc[46170]
           # select data row for index 46170
Out[11]: DATE         2014-03-31 00:00:00
         EXP_YEAR                    2014
         EXP_MONTH                      4
         TYPE                           C
         STRIKE                         1
         PRICE                      16.85
         MATURITY     2014-04-18 00:00:00
         TTM                        0.049
         IMP_VOL                        0
         Name: 46170, dtype: object

In [12]: options_data.loc[46170]['STRIKE']
           # select only the value in column STRIKE
           # for index 46170
Out[12]: 1.0
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

现在应该可视化选中期权的隐含波动率。在此，我们只使用 **options_data** 对象的子集来计算隐含波动率：

```
In [13]: plot_data = options_data[options_data['IMP_VOL'] > 0]
```

为了可视化数据，我们循环读取数据集中的所有到期日，将隐含波动率绘制成线图和单点图。因为所有到期日都出现多次，我们必须使用一个小技巧，以获得到期日的无冗余排序列表。set 操作删除所有重复数据，但是提交的可能是一个未经排序的到期日集合。因此，我们对 set 对象进行排序（参见第 4 章）[1]：

```
In [14]: maturities = sorted(set(options_data['MATURITY']))
         maturities
Out[14]: [Timestamp('2014-04-18 00:00:00'),
          Timestamp('2014-05-16 00:00:00'),
          Timestamp('2014-06-20 00:00:00'),
          Timestamp('2014-07-18 00:00:00'),
          Timestamp('2014-08-15 00:00:00'),
          Timestamp('2014-09-19 00:00:00'),
          Timestamp('2014-10-17 00:00:00'),
          Timestamp('2014-11-21 00:00:00')]
```

下面的代码将迭代所有的到期日并绘制图形，最终结果如图 3-1 所示。在股票或外汇市场中，你将注意到所谓的隐含波动率微笑，而且到期日越短，隐含波动率微笑越明显；到期日越长，越不明显。

```
In [15]: import matplotlib.pyplot as plt
         %matplotlib inline
         plt.figure(figsize=(8, 6))
         for maturity in maturities:
             data = plot_data[options_data.MATURITY == maturity]
               # select data for this maturity
             plt.plot(data['STRIKE'], data['IMP_VOL'],
                      label=maturity.date(), lw=1.5)
             plt.plot(data['STRIKE'], data['IMP_VOL'], 'r.')
         plt.grid(True)
         plt.xlabel('strike')
         plt.ylabel('implied volatility of volatility')
         plt.legend()
         plt.show()
```

为完成这个例子，我们要展示 **pandas** 的另一个优势：也就是层次化索引数据集的处理。DateFrame 对象 **options_data** 有一个*整数索引*，我们已经在多处使用过它。但是，这个索引并没有实际意义——它“只”是个数字。2014 年 3 月 31 日的期权报价由到期日和行权价格唯一描述（“标识”）——也就是说，每个到期日和行权价只有一种对应的看涨期权。

1 因为我们只考虑单一日期的期货价值和期权报价，不存在重复数据，所以 futures_data 对象的 MATURITY 列交付信息会更容易些。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

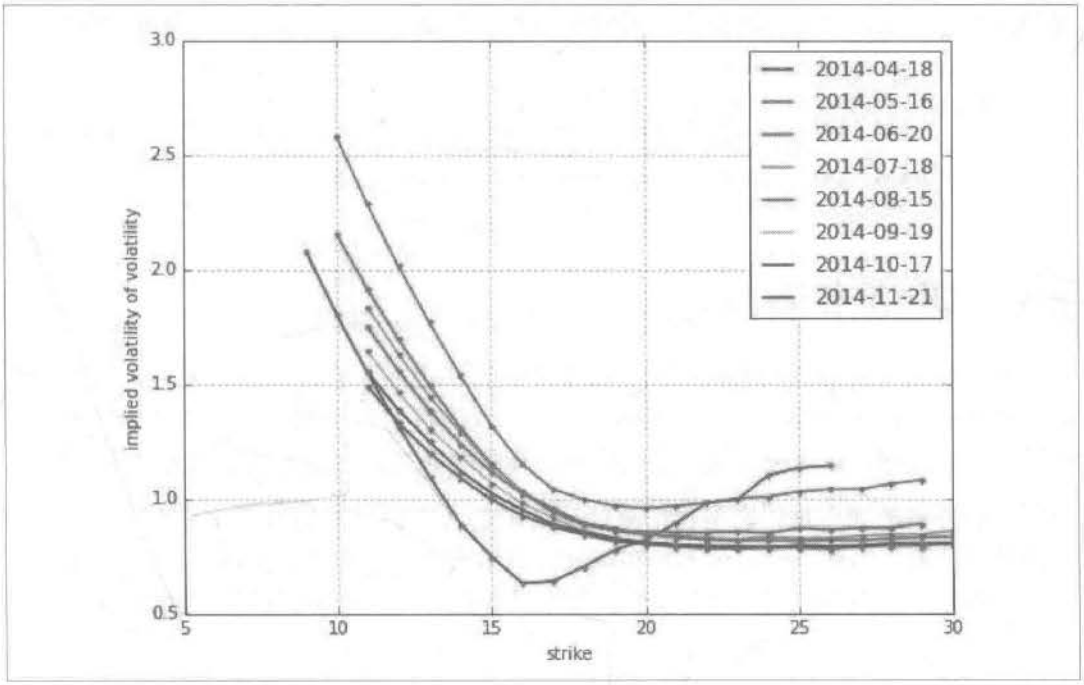


图 3-1　基于 2014 年 3 月 31 日 VSTOXX 指数的欧式看涨期权（波动率的）隐含波动率

groupby 方法可以用于利用这些认识，获得更有意义的索引。在此，我们首先按照 MATURITY（到期日）、然后根据 STRIKE（行权价）分组。我们只想保留 PRICE 和 IMP_VOL 列：

```
In [16]: keep = ['PRICE', 'IMP_VOL']
         group_data = plot_data.groupby(['MATURITY', 'STRIKE'])[keep]
         group_data
Out[16]: <pandas.core.groupby.DataFrameGroupBy object at 0x7faf483d5710>
```

上述操作返回一个 DataFrameGroupBy 对象[1]。为了得到这一数据，我们必须在该对象上应用聚合操作，比如求取总和。求取总和获得单一数据点，因为每个组中只有一个数据元素：

```
In [17]: group_data = group_data.sum()
         group_data.head()
Out[17]:                          PRICE    IMP_VOL
         MATURITY   STRIKE
         2014-04-18 9          8.85  2.083386
                    10         7.85  1.804194
                    11         6.85  1.550283
                    12         5.85  1.316103
                    13         4.85  1.097184
```

1　注意，你总是可以使用 Python 内建函数 dir 查找未知对象的属性和方法，例如 dir(group_data)。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

得到的 DataFrame 对象有两种索引级别和两列。下面展示两个索引的所有可能取值：

```
In [18]: group_data.index.levels
Out[18]: FrozenList([[2014-04-18 00:00:00, 2014-05-16 00:00:00, 2014-06-20 00:00
         :00, 2014-07-18 00:00:00, 2014-08-15 00:00:00, 2014-09-19 00:00:00, 201
         4-10-17 00:00:00, 2014-11-21 00:00:00], [9.0, 10.0, 11.0, 12.0, 13.0, 1
         4.0, 15.0, 16.0, 17.0, 18.0, 19.0, 20.0, 21.0, 22.0, 23.0, 24.0, 25.0,
         26.0, 27.0, 28.0, 29.0, 30.0]])
```

3.2 蒙特卡洛模拟

蒙特卡洛模拟是金融学和数值科学中最重要的算法之一。它之所以重要，是因为在期权定价或者风险管理问题上有很强的能力。和其他数值方法相比，蒙特卡洛方法很容易处理高维问题，在这种问题上复杂度和计算需求通常以线性方式增大。

蒙特卡洛方法的缺点是：它本身是**高计算需求**的，即使对于相当简单的问题也往往需要海量的计算。因此，必须高效地实现蒙特卡洛算法。下面的例子阐述了 Python 的不同实现策略，并提供 3 种不同的基于蒙特卡洛模拟的欧式期权估值方法[1]。这三种方法是[2]：

纯 Python

这个例子坚持使用标准库——也就是标准 Python 安装中自带的库和软件包——只使用内建的 Python 功能实现蒙特卡洛估值。

向量化 NumPy

这种实现使用 NumPy 的功能，使实现更紧凑、更快速。

全向量化 NumPy

最后一个例子结合不同数学公式和 NumPy 的向量化能力，获得相同算法的更紧凑版本。

这些例子仍然是基于 Black-Scholes-Merton（1973）的模型经济，其中的高风险标识物（例如股价或者指数水平）在风险中立的情况下遵循以随机微分方程（SDE，如公式 3-5）表示的布朗运动。

公式 3-5　Black-Scholes-Merton（1973）随机微分方程

$$dS_t = rS_t dt + \sigma S_t dZ_t \qquad (3\text{-}5)$$

1　尽管在这里并不是必需的，但是所有方法都将整个模拟路径保存在内存中。正如第 1 章中对应的例子所说明的那样，对于标准欧式期权的估值这并不是必要的。不过，对于美式期权或者某些风险管理的目的，需要完整的路径。——原注

2　这些蒙特卡洛示例和实现方法在 Hilpisch 的文章（2013）中也出现过。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

方程中参数的定义和公式 3-1 相同，Z 是一个布朗运动。公式 3-5 中 SDE 的离散化由公式 3-6 中的微分方程给出。

公式 3-6　SDE 的欧拉离散

$$S_t = S_{t-\Delta t} \exp\left(\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, z_t\right) \tag{3-6}$$

变量 Z 是标准正态分布随机变量，$0<\Delta t<T$，是一个（足够小的）时间间隔。最后一个时间范围 T 满足 $0<5\leqslant T$[1]。

我们以 $S0 = 100$、$K = 105$、$T = 1.0$、$r = 0.05$、$\sigma = 0.2$ 参数化上述模型。使用公式 3-1 中的 Black-Scholes-Merton 公式和前面的例 3-1，可以计算精确的期权价值：

```
In [19]: from bsm_functions import bsm_call_value
         S0 = 100.
         K = 105.
         T = 1.0
         r = 0.05
         sigma = 0.2
         bsm_call_value(S0, K, T, r, sigma)
Out[19]: 8.0213522351431763
```

这是我们的蒙特卡洛估值程序遵循的基准值。要实现欧式看涨期权的蒙特卡洛估值，可以应用如下诀窍。

1. 将时间间隔[0，T]分为等距的、长度为 Δt 的子时段。
2. 开始循环 i=1，2，…，I。
   1) 对于每个时间步 $t\in\{\Delta t, 2\Delta t, \cdots, T\}$，取伪随机数 $Z_t(i)$。
   2) 逐个时间步应用伪随机数，确定指数水平 $S_T(i)$的 T 值，以离散化公式 3-6 中的方案。
   3) 确定 T 时点欧式看涨期权的内在价值 ht：$ht(ST(i)) = \max(ST(i)-K,0)$。
   4) 循环到 $i = I$。
3. 根据公式 3-7，加总内在价值，求平均值，并扣除无风险短期利率。

公式 3-7 提供欧式看涨期权价值的数值化蒙特卡洛估算函数。

公式 3-7　欧式看涨期权价值的蒙特卡洛估算函数

$$C_0 \approx \mathrm{e}^{-rT}\frac{1}{I}\sum_I h_T(S_T(i)) \tag{3-7}$$

1　详见 Hilpisch 的著作（2015）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

3.2.1 纯 Python

例 3-2 将参数化和蒙特卡洛方法翻译为纯 Python。代码模拟 50 个时间步上的 25 万条路径。

例 3-2 用纯 Python 进行欧式看涨期权的蒙特卡洛估值

```
#
# Monte Carlo valuation of European call options with pure Python
# mcs_pure_python.py
#

from time import time
from math import exp, sqrt, log
from random import gauss, seed

seed(20000)
t0 = time()

# Parameters
S0 = 100.  # initial value
K = 105.  # strike price
T = 1.0  # maturity
r = 0.05  # riskless short rate
sigma = 0.2  # volatility
M = 50  # number of time steps
dt = T / M  # length of time interval
I = 250000  # number of paths

# Simulating I paths with M time steps
S = []
for i in range(I):
    path = []
    for t in range(M + 1):
        if t == 0:
            path.append(S0)
        else:
            z = gauss(0.0, 1.0)
            St = path[t - 1] * exp((r - 0.5 * sigma ** 2) * dt
                                   + sigma * sqrt(dt) * z)
            path.append(St)
    S.append(path)

# Calculating the Monte Carlo estimator
C0 = exp(-r * T) * sum([max(path[-1] - K, 0) for path in S]) / I

# Results output
tpy = time() - t0
print "European Option Value %7.3f" % C0
print "Duration in Seconds   %7.3f" % tpy
```

运行以上脚本获得如下输出：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [20]: %run mcs_pure_python.py

Out[20]: European Option Value   7.999
         Duration in Seconds    34.258
```

注意，估算的期权价值取决于生成的伪随机数，所需时间受到执行脚本的硬件的影响。

例 3-2 中代码的大部分由一个嵌套循环组成，内循环为每个时间步生成指数水平路径的值，在外循环中将完整的路径加到一个 list 对象中。蒙特卡洛估算函数用 Python 的“列表推导语法”（https://docs.python.org/2/tutorial/datastructures.html）计算。估算函数也可以通过一个 for 循环计算：

```
In [21]: sum_val = 0.0
         for path in S:
             # C-like iteration for comparison
             sum_val += max(path[-1] - K, 0)
         C0 = exp(-r * T) * sum_val / I
         round(C0, 3)
Out[21]: 7.999
```

这个循环得到相同的结果，但是列表推导语法更紧凑，更接近于蒙特卡洛估算函数的数学表示。

3.2.2 用 NumPy 向量化

NumPy 提供了强大的多维数组类 **ndarray**，以及一组全面的函数和方法，可以操纵数组、实现这些对象上的（复杂）运算。总体来看，使用 **NumPy** 有 2 个主要的好处：

语法

NumPy 一般可以实现比纯 Python 更紧凑的实现，往往更容易理解和维护。

速度

大部分 NumPy 代码是用 C 或者 Fortran 实现的，正确使用时，比纯 Python 更快。

更为紧凑的语法是由于 NumPy 为 Python 带来了强大的向量化和预报能力。这和数学中用于大型向量或者矩阵的向量标记法类似。例如，假定我们有一个包含前 100 个自然数（1，…，100）的向量：

$$\vec{v} = \begin{pmatrix} 1 \\ 2 \\ \vdots \\ 100 \end{pmatrix}$$

这个向量的标量乘法可以紧凑地写为：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

$$\vec{u} = 2 \bullet \vec{v} \begin{pmatrix} 2 \\ 4 \\ \vdots \\ 200 \end{pmatrix}$$

下面我们来看看使用 Python 的 **list** 对象如何完成上述运算：

```
In [22]: v = range(1, 6)
         print v
Out[22]: [1, 2, 3, 4, 5]
In [23]: 2 * v
Out[23]: [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
```

简单标量乘法不会返回标量乘积，在例子中，它返回的是重复两遍的对象（向量）。但是，使用 NumPy 可以得到我们希望的结果：

```
In [24]: import numpy as np
         v = np.arange(1, 6)
         v
Out[24]: array([1, 2, 3, 4, 5])
In [25]: 2 * v
Out[25]: array([ 2,  4,  6,  8, 10])
```

这种方法应用到蒙特卡洛算法上会带来好处。例 3-3 提供了对应的代码，这次使用了 NumPy 的向量化能力。

例 3-3　使用 NumPy 的欧式看涨期权蒙特卡洛估值（第一个版本）

```
#
# Monte Carlo valuation of European call options with NumPy
# mcs_vector_numpy.py
#
import math
import numpy as np
from time import time

np.random.seed(20000)
t0 = time()

# Parameters
S0 = 100.; K = 105.; T = 1.0; r = 0.05; sigma = 0.2
M = 50; dt = T / M; I = 250000

# Simulating I paths with M time steps
S = np.zeros((M + 1, I))
S[0] = S0
for t in range(1, M + 1):
    z = np.random.standard_normal(I)  # pseudorandom numbers
    S[t] = S[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                                    + sigma * math.sqrt(dt) * z)
        # vectorized operation per time step over all paths

    # Calculating the Monte Carlo estimator
    C0 = math.exp(-r * T) * np.sum(np.maximum(S[-1] - K, 0)) / I

    # Results output
    tnp1 = time() - t0
    print "European Option Value %7.3f" % C0
    print "Duration in Seconds   %7.3f" % tnp1
```

运行以上脚本：

```
In [26]: %run mcs_vector_numpy.py
Out[26]: European Option Value   8.037
         Duration in Seconds     1.215
In [27]: round(tpy / tnp1, 2)
Out[27]: 28.2
```

向量化和纯 Python 相比，*速度有 30 倍以上的提升*。估算出来的蒙特卡洛值仍然和基准值很接近。

在伪随机数生成时，向量化的作用显而易见。在问题中的代码行里，一个步骤就完成了 25 万个数字的生成，也就是说，这只需要一个代码行：

```
z = np.random.standard_normal(I)
```

类似地，这个伪随机数向量以向量化的方式，在每个时间步中应用到离散化方案中。在这个意义上，例 3-2 中外循环完成的任务被委托给 NumPy，完全避免了 Python 层面上的外循环。

向量化

利用 NumPy 的向量化通常可以得到更紧凑、更容易理解（和维护）、执行更快的代码。这些特征对金融应用程序都很重要。

3.2.3 利用对数欧拉方法实现全向量化

对于公式 3-5 中的 SDE，使用不同的离散化方案可能得到蒙特卡洛算法的更紧凑实现。为此，考虑公式 3-6 中离散化方案的对数版本，其形式如公式 3-8。

公式 3-8　SDE 的欧拉离散化方法（对数版本）

$$\log S_t = \log S_{t-\Delta t} + \left(r - \frac{1}{2}\sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} z_t \qquad (3\text{-}8)$$

这个版本完全采用递增法，可以在 Python 层面上不使用任何循环的情况下实现蒙特卡洛算法。例 3-4 展示了结果代码。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例 3-4　用 NumPy 进行欧式看涨期权的蒙特卡洛估值（第二个版本）

```
#
# Monte Carlo valuation of European call options with NumPy (log version)
# mcs_full_vector_numpy.py
#
import math
from numpy import *
from time import time
  # star import for shorter code

random.seed(20000)
t0 = time()

# Parameters
S0 = 100.; K = 105.; T = 1.0; r = 0.05; sigma = 0.2
M = 50; dt = T / M; I = 250000

# Simulating I paths with M time steps
S = S0 * exp(cumsum((r - 0.5 * sigma ** 2) * dt
            + sigma * math.sqrt(dt)
                    * random.standard_normal((M + 1, I)), axis=0))
  # sum instead of cumsum would also do
  # if only the final values are of interest
S[0] = S0

# Calculating the Monte Carlo estimator
C0 = math.exp(-r * T) * sum(maximum(S[-1] - K, 0)) / I

# Results output
tnp2 = time() - t0
print "European Option Value %7.3f" % C0
print "Duration in Seconds   %7.3f" % tnp2
```

下面我们运行上述脚本。

```
In [28]: %run mcs_full_vector_numpy.py

Out[28]: European Option Value   8.166
         Duration in Seconds     1.439
```

执行速度和第一种 NumPy 实现相比稍慢一些，这种实现方法也可能需要在紧凑性和易读性上求得某种平衡，使得 NumPy 层面上真正发生的操作难以掌握。不过，上述方法说明，使用 NumPy 向量化能够提供什么样的可能性。

3.2.4　图形化分析

最后，我们以图形化的方式查看底层机制（matplotlib 绘图库的解释参见第 5 章）。首先，我们绘制所有时间步上的前 10 条模拟路径。图 3-2 展示了输出：

```
In [29]: import matplotlib.pyplot as plt
         plt.plot(S[:, :10])
         plt.grid(True)
         plt.xlabel('time step')
         plt.ylabel('index level')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

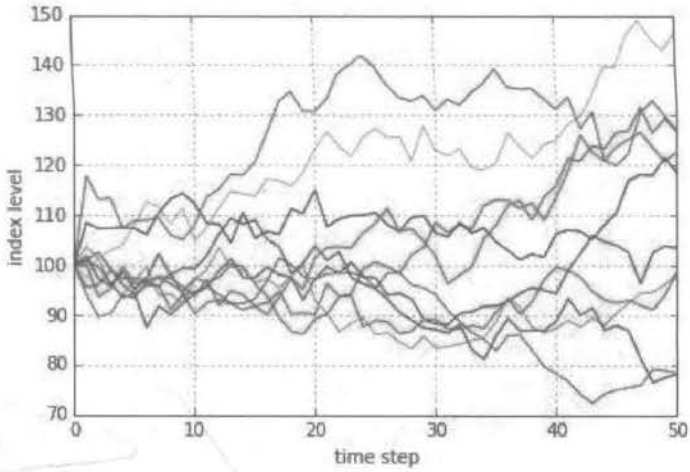


图 3-2　前 10 个模拟指数水平路径

其次，我们想要查看模拟期结束时模拟指数水平的频率。图 3-3 展示了输出，这次显示的是期末指数水平值的（近似）对数正态分布：

```
In [30]: plt.hist(S[-1], bins=50)
         plt.grid(True)
         plt.xlabel('index level')
         plt.ylabel('frequency')
```

期权期末（到期日）内在价值的同类图形看起来完全不同，如图 3-4 所示：

```
In [31]: plt.hist(np.maximum(S[-1] - K, 0), bins=50)
         plt.grid(True)
         plt.xlabel('option inner value')
         plt.ylabel('frequency')
         plt.ylim(0, 50000)
```

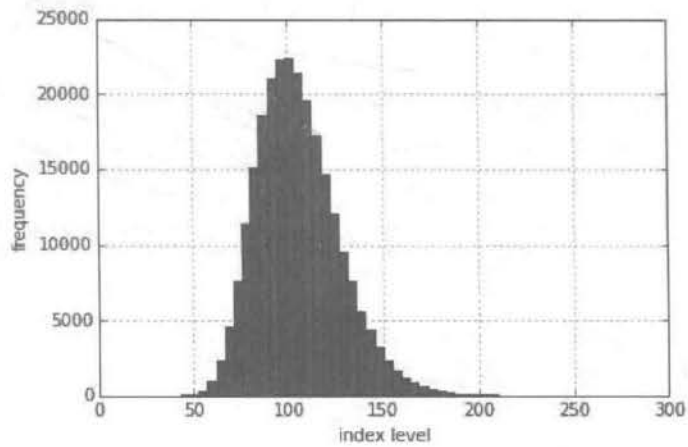


图 3-3　所有模拟期末指数水平值的直方图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

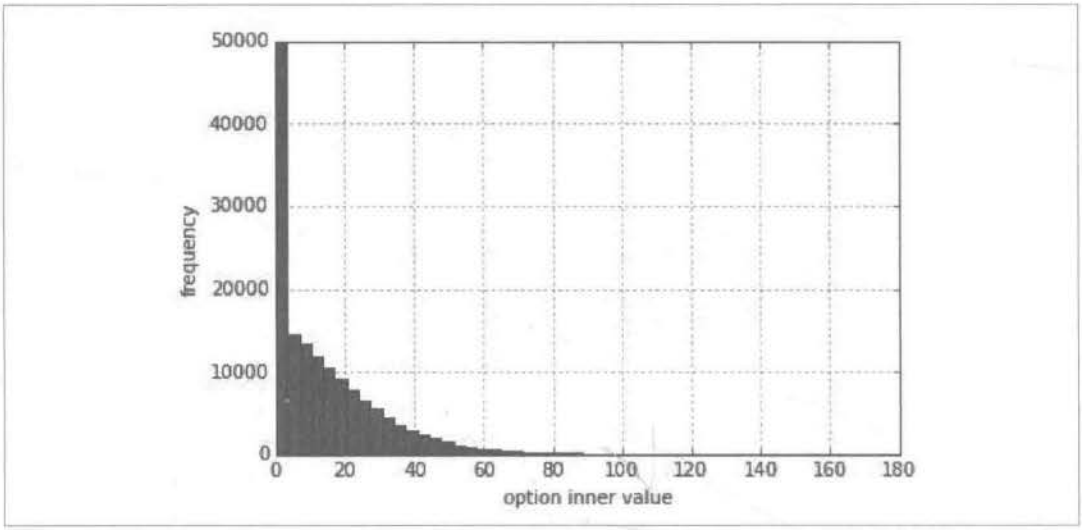


图 3-4　所有模拟期末期权内在价值的直方图

在这个例子中，大部分模拟值为 0，说明在大量情况下，欧式看涨期权到期时没有价值。准确数字通过如下计算生成：

```
In [32]: sum(S[-1] < K)
Out[32]: 133533
```

当然，在不同模拟中，这个数字有可能变化。

3.2.5　技术分析

基于历史价格信息的技术分析是金融专业人士和感兴趣的业余人士感兴趣的典型任务。在维基百科上（http://en.wikipedia.org/wiki/Technical_analysis）可以找到如下定义：

> 在金融学中，技术分析是通过对过去市场数据（主要是价格和成交量）的研究预测价格方向的证券分析方法。

在下面的段落中，我们将重点放在用于事后验证的过去市场数据的研究，而不是过多地关注使用我们的认识预测未来股价变动。我们的研究目标是标准普尔（S&P）500 基准指数，这通常被认为是整个美国股票市场很有代表性的指标，因为指数中包含许多著名公司的股票，代表着很高额的市场资本。而且，该指数还具有高流动性的期货和期权市场。

我们将从 Web 数据来源读取历史指数水平信息，并为一个基于趋势信号的交易系统实现简单的事后验证。但是，首先需要数据才能开始工作，为此，我们主要依靠 **pandas** 库，这简化了一些相关的技术问题。因为几乎总是会用到 **NumPy**，所以还应该默认导入它：

```
In [33]: import numpy as np
         import pandas as pd
         import pandas.io.data as web
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

Python 科学和金融栈

除了 NumPy 和 SciPy，Python 科学和金融栈仅由几个重要的库组成。其中之一是 pandas，一定要始终安装这些库的当前（稳定）版本（还要注意可能的语法和 API 更改）

pandas.io.data 子库包含 DataReader 函数，这个函数能够帮助我们从不同来源，特别是流行的雅虎财经网站（http://finance.yahoo.com）上获取金融事件序列数据。我们从 2000 年 1 月 1 日起读取所需数据：

```
In [34]: sp500 = web.DataReader('^GSPC', data_source='yahoo',
                                 start='1/1/2000', end='4/14/2014')
         sp500.info()
Out[34]: <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 3592 entries, 2000-01-03 00:00:00 to 2014-04-14 00:00:00
         Data columns (total 6 columns):
         Open          3592 non-null float64
         High          3592 non-null float64
         Low           3592 non-null float64
         Close         3592 non-null float64
         Volume        3592 non-null int64
         Adj Close     3592 non-null float64
         dtypes: float64(5), int64(1)
```

DataReader 已经通过互联网线路连接到数据源，并读取了从 2000 年的第一个交易日到结束日期的 S&P 500 指数事件序列数据，而且自动地用 TimeStamp 对象生成一个时间索引。

为了得到第一印象，可以绘制收盘价的时间序列图。输出类似图 3-5：

```
In [35]: sp500['Close'].plot(grid=True, figsize=(8, 5))
```

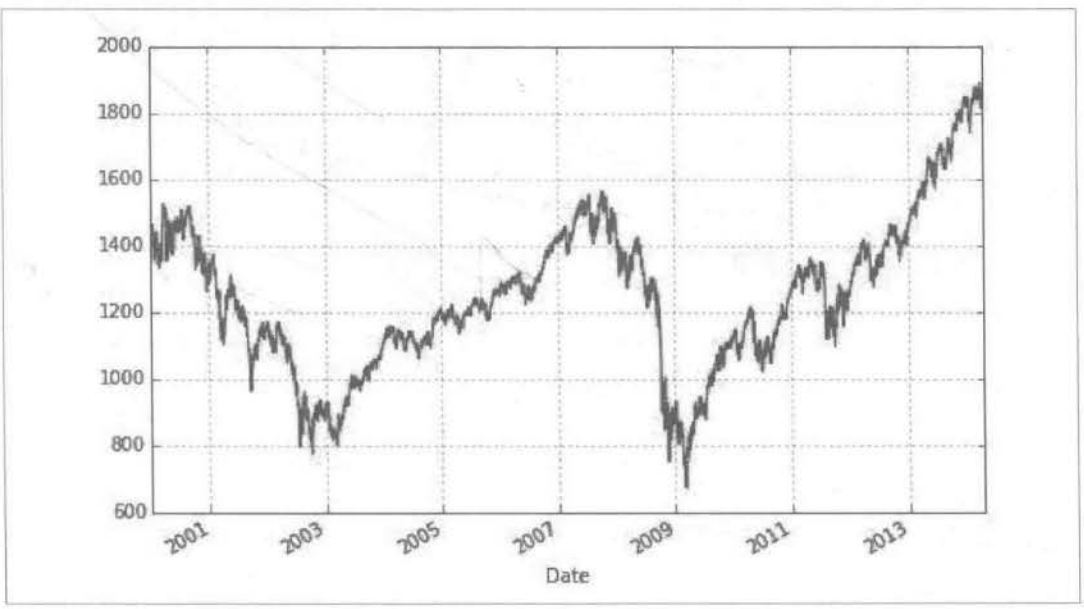


图 3-5　S&P 500 指数历史水平

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们要实现的趋势策略基于*两个月*（42 个交易日）和*一年*（252 个交易日）的*趋势*（也就是两种期间指数水平的移动平均数）。同样，**pandas** 可以高效地生成各个时间序列，并在一张图上绘制 3 个相关的事件序列。首先是生成趋势数据：

```
In [36]: sp500['42d'] = np.round(pd.rolling_mean(sp500['Close'], window=42), 2)
         sp500['252d'] = np.round(pd.rolling_mean(sp500['Close'], window=252), 2)
```

在这个例子中，第一行代码同时在 **pandas DataFrame** 对象中*添加一个新列*，并在其中*放入 42 天趋势值*。第二行代码对 252 天的趋势进行同样的操作。现在，我们有了两个新列。因为这些列所生成数据的性质，这些列的项目数较少——也就是说，到了第

```
In [37]: sp500[['Close', '42d', '252d']].tail()
Out[37]:                Close      42d     252d
         Date
         2014-04-08  1851.96  1853.88  1728.66
         2014-04-09  1872.18  1855.66  1729.79
         2014-04-10  1833.08  1856.46  1730.74
         2014-04-11  1815.69  1856.36  1731.64
         2014-04-14  1830.61  1856.63  1732.74
```

其次，我们绘制新数据的图表，结果如图 3-6 所示，已经提供了对过去涨跌趋势的一些认识：

```
In [38]: sp500[['Close', '42d', '252d']].plot(grid=True, figsize=(8, 5))
```

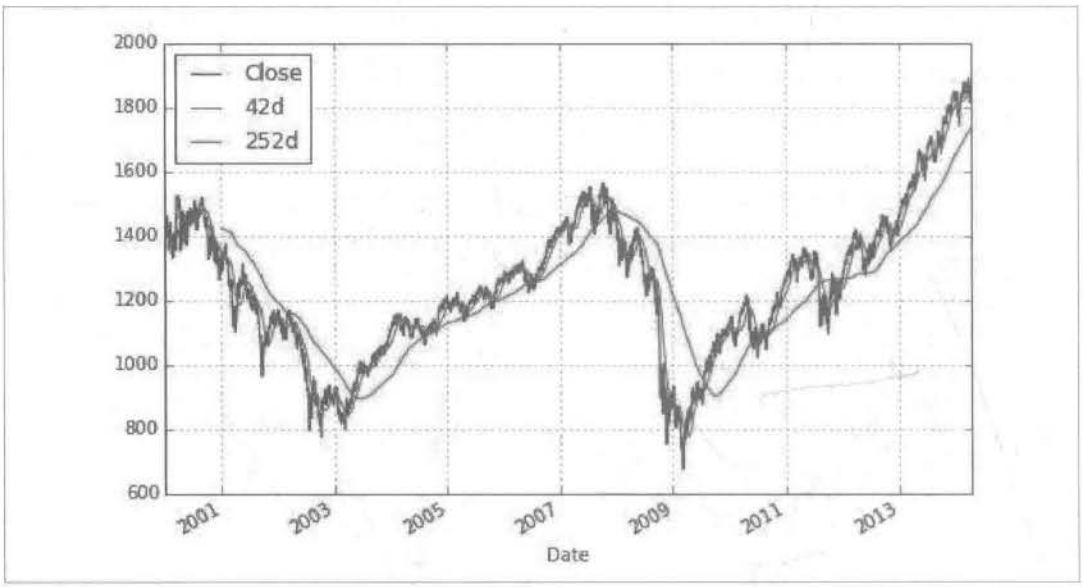


图 3-6　S&P 500 指数及 42 天和 252 天趋势线

我们的基本数据集很完整，因此现在可以设计一条规则来生成趋势信号。规则如下：

买入信号（多头）

42 天趋势第一次高于 252 天趋势 SD 点。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

等待（持币）

42 天趋势在 252 天趋势的+/- SD 点范围内。

卖出信号（空头）

42 天趋势第一次低于 252 天趋势 SD 点。

为此，我们在 **pandas DataFrame** 对象上添加一个新列，用于两个趋势之间的差值。正如你所看到的，**pandas** 数值运算通常以向量方式进行，这样可以取得两列的全部差值：

```
In [39]: sp500['42-252'] = sp500['42d'] - sp500['252d']
         sp500['42-252'].tail()

Out[39]: Date
         2014-04-08    125.22
         2014-04-09    125.87
         2014-04-10    125.72
         2014-04-11    124.72
         2014-04-14    123.89
         Name: 42-252, dtype: float64
```

在最后一个可用交易日上，42 日趋势线远远高于 252 日趋势线。尽管两个趋势列中的项目数量不相等，**pandas** 通过在相应的指数位置放入 NaN 值，处理这种情况：

```
In [40]: sp500['42-252'].head()

Out[40]: Date
         2000-01-03   NaN
         2000-01-04   NaN
         2000-01-05   NaN
         2000-01-06   NaN
         2000-01-07   NaN
         Name: 42-252, dtype: float64
```

为了让过程更为正式，我们再生成一个新列，用于所谓的*投资体制*（**Regime**）。我们假定信号阈值为 50：

```
In [41]: SD = 50
         sp500['Regime'] = np.where(sp500['42-252'] > SD, 1, 0)
         sp500['Regime'] = np.where(sp500['42-252'] < -SD, -1, sp500['Regime'])
         sp500['Regime'].value_counts()

Out[41]:  1    1489
          0    1232
         -1     871
         dtype: int64
```

用语言描述就是：在 1489 个交易日中，42 日趋势线比 252 日趋势线高 SD 个点以上，在 1232 个交易日中，42 日趋势线比 252 日趋势线低 SD 个点以上。很明显，如果短期趋势线与长期趋势线交叉，它很可能在这一带保持（较长）一段时间。这就是所谓的*投资体制*（**Regime**），如图 3-7 所示，该图由如下两行代码生成：

```
In [42]: sp500['Regime'].plot(lw=1.5)
         plt.ylim([-1.1, 1.1])
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

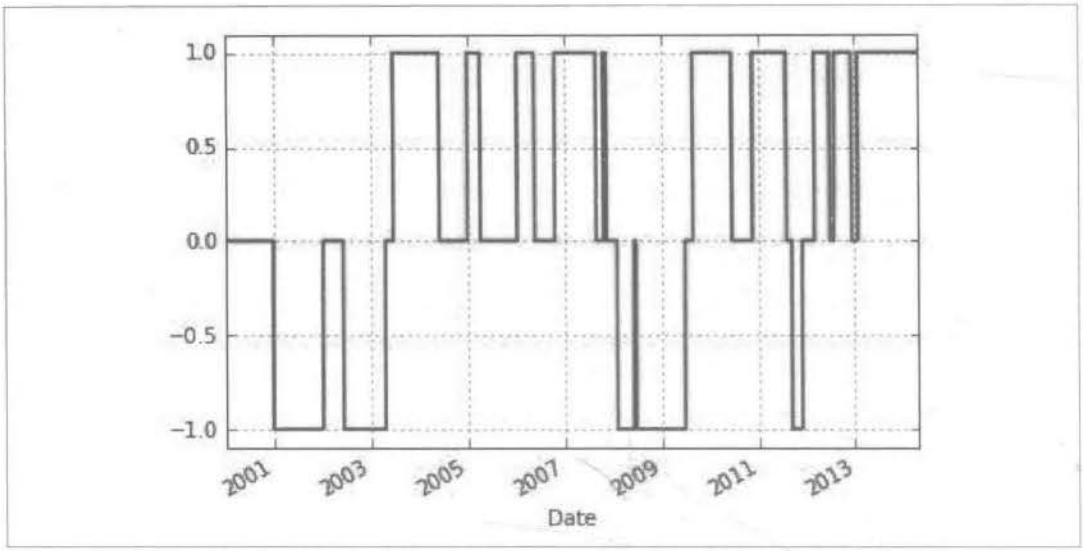


图 3-7　在一段时间内的投资体制信号

现在，测试基于信号投资策略所需的数据都已经准备就绪。为了简单起见，我们假定投资者可以直接投资于指数或者可以直接做空指数，这在现实世界中必须通过使用指数基金、交易所交易基金或者指数期货完成。这种交易不可避免地会造成我们现在忽略的交易成本。我们这样做似乎是合理的，因为并不打算“频繁”交易。

根据各自的投资体制，投资者可能做空、做多市场（指数）或者持币观望（这不是我们感兴趣的）。这种简化的策略使我们可以只注意市场收益。投资者做多时形成市场收益（1），做空时形成负的市场收益（-1），持币时不形成任何市场收益（0）。因此，我们首先需要这些收益数据，在 Python 中，采用如下的向量化 pandas 运算，计算对数收益率。注意，shift 方法按照所需指数输入项数量移动时间序列——在我们的例子中每次移动一个交易日，这样就能得到每日的对数收益率：

```
In [43]: sp500['Market'] = np.log(sp500['Close'] / sp500['Close'].shift(1))
```

回忆一下我们是如何建立投资体制的，现在，求得基于趋势的投资策略的收益很简单——只需要将 Regime 列乘以下一天的 Returns 列（用“昨天”的头寸得出“今天”的收益）：

```
In [44]: sp500['Strategy'] = sp500['Regime'].shift(1) * sp500['Market']
```

这种策略的收益很好；投资者可以在对应期间中得到比简单做多投资方式高得多的收益。图 3-8 比较了指数累计持续收益和我们所用策略的累积持续收益：

```
In [45]: sp500[['Market', 'Strategy']].cumsum().apply(np.exp).plot(grid=True,
                                                                   figsize=(8, 5))
```

图 3-8 说明，特别在市场低迷（2003 年和 2008/2009 年）时期，做空市场获得相当高的收益。尽管这种策略不能捕捉牛市中的整个上升段，但是整体表现明显好于市场。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

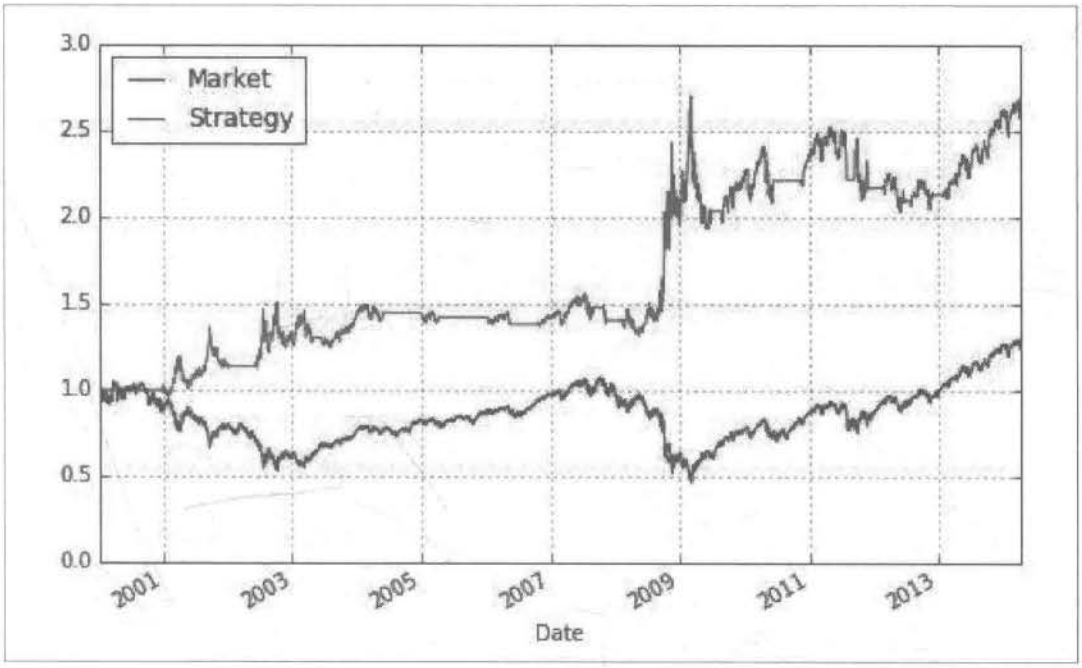


图 3-8　S&P 500 指数与投资者财富的对比

不过，我们必须记住，在研究中完全忽略了操作问题（比如交易执行）和相关的市场微观因素（例如交易成本）。例如，我们所使用的是每日的收盘价。问题是何时从市场退出（从看多变为中性/持币）：以同一天的收盘价还是次日的开盘价。这些考虑因素当然对表现有所影响，但是总体结果可能不变。还有，交易成本通常会稀释收益，但交易规则不会产生太多的信号。

金融时间序列

每当分析金融时间序列时，都可以考虑使用 pandas。几乎任何时间序列相关的问题都可以用这个强大的库解决。

3.3　结语

本章没有过多地深入细节，而是通过具体和典型的金融示例阐述了 Python 的使用。

隐含波动率的计算

使用现实世界的数据（以给定日期期权数据剖面的形式），我们以数值方式计算了基于 VSTOXX 波动率指数的欧式看涨期权的隐含波动率。这个例子介绍了一些自定义的 Python 函数（例如，用于分析性期权估值），并使用来自 **NumPy**、**SciPy** 和 **Pandas** 的功能。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

蒙特卡洛模拟

使用不同的实现方法，我们模拟一段时间指数水平的变化，并使用模拟期末价值得出欧式看涨期权的蒙特卡洛估算函数。使用 NumPy，阐述了 Python 代码向量化的好处：也就是，*代码的紧凑性和执行速度*。

趋势信号策略的事后验证

使用 S&P 500 的真实历史时间序列数据，我们对基于 42 天和 252 天趋势（移动平均）的一种交易策略表现进行了事后验证。这个例子说明了 pandas 在时间序列分析上的能力和方便性。

在 Python 的使用上，本章介绍了交互式金融分析（使用 IPython 交互式 shell）、模块中存储的更复杂函数的使用，以及使用向量化面向性能的算法实现。这里没有介绍一个重要的主题：Python 中的面向对象和类特性。对此感到好奇的读者可以参见附录 B，其中包含了欧式看涨期权的一个类定义，类的方法基于本章例 3-1 中的代码。

3.4 延伸阅读

本章使用的主要参考书如下所示。

- Black, Fischer and Myron Scholes (1973): "The Pricing of Options and Corporate Liabilities." *Journal of Political Economy*, Vol. 81, No. 3, pp. 638-659.
- Hilpisch, Yves (2015): *Derivatives Analytics with Python*. Wiley Finance, Chichester, England. http://www.derivatives-analytics-with-python.com.
- Hilpisch, Yves (2013): "Efficient Data and Financial Analytics with Python." *Software Developer's Journal*, No. 13, pp. 56-65.http://hilpisch.com/YH_Efficient_Analytics_Article.pdf.
- Merton, Robert (1973): "Theory of Rational Option Pricing." *Bell Journal of Economics and Management Science*, Vol. 4, pp. 141-183.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 2 部分

金融分析和开发

这一部分是本书的核心，介绍金融分析和应用程序开发中最重要的 Python 库、技术和方法。本部分涵盖大量主题，因此必须将焦点主要放在精选、相当特殊的示例和用例上。

本部分的各个章节根据特定主题组织，因此这一部分可以作为读者查找与感兴趣的主题相关示例的参考。本书的这个核心部分包含如下章节。

- 第 4 章，Python 数据类型和结构。
- 第 5 章，用 matplotlib 进行 2D 和 3D 可视化。
- 第 6 章，金融时间序列数据处理。
- 第 7 章，（高性能）输入/输出操作。
- 第 8 章，高性能技术和库。
- 第 9 章，金融学中需要的多种数学工具。
- 第 10 章，随机数生成和随机过程模拟。
- 第 11 章，Python 统计学应用。
- 第 12 章，Python 和 Excel 的集成。
- 第 13 章，Python 面向对象编程和（简单）图形用户界面（GUI）的开发。
- 第 14 章，Python 与 Web 技术的集成，以及基于 Web 应用及 Web 服务的开发。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第4章

数据类型和结构

拙劣的程序员担心代码。好的程序员担心数据结构及它们的关系。

——Linus Torvalds

本章介绍 Python 的基本数据类型和数据结构。Python 解释程序本身已经自带丰富的数据结构，**NumPy** 和其他库又为之增添了许多有价值的数据结构。

本章按照如下主题组织：

基本数据类型

第一个小节介绍 **int**、**float** 和 **string** 等基本数据类型。

基本数据结构

下一个小节介绍 Python 的基本数据结构（例如 **List** 对象）并阐述控制结构、函数式编程范型以及匿名函数。

NumPy 数据结构

接下来的一个小节专门介绍 **NumPy ndarray** 类的特性和功能，并阐述这个类在科学和金融应用上的好处。

代码向量化

正如最后一个小结所阐述的那样，由于 **NumPy** 的数组类向量化代码很容易实现，我们可以得到更加紧凑、性能更好的代码。

本章的主旨是提供 Python 数据类型和结构细节的一般介绍。如果你有其他编程语言（如 **C** 或者 **Matlab**）的背景，应该能够轻松地掌握 Python 使用方法带来的差异。本章介绍的主题是后续章节的重要基础。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

4.1 基本数据类型

Python 是一种*动态类型*语言，这意味着，Python 解释程序在运行时推知对象的类型。C 等编译语言通常是*静态类型*语言，在这类语言中，对象类型必须在编译之前与对象绑定[1]。

4.1.1 整数

最基本的数据类型是整数-**int**：

```
In [1]: a = 10
        type(a)
Out[1]: int
```

内建函数 **type** 为所有使用标准和内建类型的对象和新创建的类及对象提供类型信息。在后一种情况下，提供的信息取决于程序员存储在类中的描述。有一种说法："Python 中，一切都是对象"。这意味着，即使我们刚刚定义的简单对象 **int** 也有内建方法。例如，你可以调用 **bit_length** 方法，获得表现 **int** 对象所需的位数：

```
In [2]: a.bit_length()
Out[2]: 4
```

你将会看到，我们为该对象所赋的整数值越大，需要的位数越多：

```
In [3]: a = 100000
        a.bit_length()
Out[3]: 17
```

一般来说，这类方法很多，难以记住所有类和对象的所有方法。高级 Python 环境（如 IPython）提供 Tab 键完成功能，显示对象连接的所有方法。你只需要输入对象名和一个点（例如 a.）然后按下 Tab 键，就会提供一组可以在该对象上调用的方法。也可以使用 Python 内建函数 dir 显示任何对象属性和方法的完整列表。

Python 的特殊性之一是整数可以任意大。例如，考虑天文数字 10^{100}。Python 处理这样大的数毫无问题，这从技术上说是一个很"长"的对象：

```
In [4]: googol = 10 ** 100
        googol
Out[4]: 10000000000
```

大整数

Python 整数可以为任意大。解释程序简单地使用所需的位/字节数表现数值。

重要的是，要注意 **int** 对象上进行的数学运算返回 **int** 对象。这有时候可能在数学例程中造成混乱或者难以检测的错误。下面的表达式得到预期的结果：

```
In [6]: 1 + 4
Out[6]: 5
```

然而，下面这种情况可能返回有些令人吃惊的结果：

```
In [7]: 1 / 4
Out[7]: 0
In [8]: type(1 / 4)
Out[8]: int
```

4.1.2 浮点数

为了让上一个表达式返回所需的值——0.25，我们必须在*浮点*（**float**）对象上操作，这很自然地把我们带到下一种基本数据类型。在整数值上加一个点比如 **1.**或者 **1.0**，会导致 Python 将该对象解释为浮点数。涉及浮点数的表达式通常也必须返回一个浮点对象[1]：

```
In [9]: 1. / 4
Out[9]: 0.25
In [10]: type (1. / 4)
Out[10]: float
```

浮点数与通常不精确的实数计算机表现形式关系更大，取决于所采用的具体技术方法。为了说明其中的含义，我们定义另一个浮点对象：

```
In [11]: b = 0.35
         type(b)
Out[11]: float
```

这种浮点对象在内部总是只表现为某种精度。将 b 加上 0.1 时，这一点变得很明显：

```
In [12]: b + 0.1
Out[12]: 0.44999999999999996
```

出现以上结果的原因是浮点数在内部以二进制形式表示；也就是说，十进制数 n（$0<n<1$）表现为如下形式：$n=\frac{x}{2}+\frac{y}{4}+\frac{z}{8}+\cdots\cdots$ 对于某些浮点数，二进制表现形式可能包含大量

1 这里和后面的讨论中，浮点数、浮点对象等属于互换使用，承认每个浮点数都是对象。对于其他队象类型也是如此。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

元素甚至是一个无限系列。但是，考虑到用于表达这种数值的位数是固定的——也就是表现系列中的项目固定——结果是不精确的。其他数值可以完整表现，因而在可用位数有限时也可以精确地存储。考虑如下例子：

```
In [13]: c = 0.5
         c.as_integer_ratio()
Out[13]: (1, 2)
```

0.5 可以精确保存，因为它具备精确（有限）的二进制表示：0.5=1/2。但是，b=0.35 和预期的实数 0.35=7/20 不同：

```
In [14]: b.as_integer_ratio()
Out[14]: (3152519739159347, 9007199254740992)
```

精度取决于表示数值所用的位数。一般来说，Python 运行的所有平台使用 IEEE 754 双精度标准（64 位）作为内部表示[1]。这相当于 15 位数字的相对精度。

由于这个主题在多个金融应用领域中都很重要，有时候必须确保数值的精确（至少尽可能达到最佳）。例如，在加总一组数量很多的数值时，这个问题就可能很重要。在这种情况下，某个种类或者幅度的表示误差可能汇聚起来，造成和基准值的显著偏差。

decimal 模块为浮点数提供了任意精确度的对象，以及使用这些数值时处理精度问题的多个选项：

```
In [15]: import decimal
         from decimal import Decimal
In [16]: decimal.getcontext()
Out[16]: Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999999, Emax=999999
         999, capitals=1, flags=[], traps=[Overflow, InvalidOperation, DivisionB
         yZero])
In [17]: d = Decimal(1) / Decimal (11)
         d
Out[17]: Decimal('0.090909090909090909090909090091')
```

可以改变 Context 对象的各个属性值，从而改变表示精度：

```
In [18]: decimal.getcontext().prec = 4  # lower precision than default
In [19]: e = Decimal(1) / Decimal (11)
         e
Out[19]: Decimal('0.09091')
In [20]: decimal.getcontext().prec = 50  # higher precision than default
In [21]: f = Decimal(1) / Decimal (11)
         f
Out[21]: Decimal('0.09090909090909090909090909090909090909090909090909091')
```

如果有必要，精度可以这样按照具体的问题进行调整，人们可以使用不同精度的浮点对象进行运算：

1 参见 http://en.wikipedia.org/wiki/Double-precision_floating-point_format。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [22]: g = d + e + f
         g
Out[22]: Decimal('0.27272818181818181818181818181909090909090909090909')
```

任意精度浮点数

decimal 模块提供一个任意精度浮点数对象。在金融中，确保高精度、超出 64 位双精度度标准有时是必要的。

4.1.3 字符串

既然我们已经可以表示自然和浮点数，就可以转向文本了。Python 中表示文本的基本数据类型是字符串（String），字符串对象有一些真正实用的内建方法。实际上，Python 通常被视为处理任何类型、任何尺寸文本文件的很好选择。字符串对象通常由单引号或者双引号定义或者使用 str 函数转换而来（也就是使用对象的标准，或者用户定义的字符串表示）：

```
In [23]: t = 'this is a string object'
```

举个内建方法的例子，你可以将对象中的第一个词改为首字大写：

```
In [24]: t.capitalize()
Out[24]: 'This is a string object'
```

也可以将字符串的拆分成单个词，获得包含所有单词的列表对象：

```
In [25]: t.split()
Out[25]: ['this', 'is', 'a', 'string', 'object']
```

你还可以搜索一个单词，如果成功，可以得到该词第一个字母的位置（即索引值）：

```
In [26]: t.find('string')
Out[26]: 10
```

如果这个单词不在字符串对象中，该方法返回-1：

```
In [27]: t.find('Python')
Out[27]: -1
```

替换字符串中的字符是典型的任务，可以用 **replace** 方法简单地完成：

```
In [28]: t.replace(' ', '|')
Out[28]: 'this|is|a|string|object'
```

字符串“剥离”（Stripping）操作——也就是删除某些前导或者后缀字符——也常常是必要的：

```
In [29]: 'http://www.python.org'.strip('htp:/')
Out[29]: 'www.python.org'
```

表 4-1 列出了字符串对象的一些实用方法：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

表 4-1　精选字符串方法

方法	参数	返回/结果
capitalize	()	复制字符串，将第一个字符改成大写
count	sub[, start[, end]])	计算子字符串出现的次数
decode	([encoding[, errors]])	用 encoding 指定的编码方式（例如 UTF-8）解码字符串
encode	([encoding[, errors]])	字符串编码形式
find	(sub[, start[, end]])	找到的子字符串（最低）索引
join	(seq)	连接 seq 序列中的字符串
replace	(old, new[, count])	用 new 替换前 count 个 old
split	([sep[, maxsplit]])	字符串中的单词列表，以 sep 作为分隔符
splitlines	([keepends])	如果 keepends 为真，以行结束符/换行符分隔的行
strip	(chars)	从字符串首/尾删除 chars 中的字符
upper	()	复制字符串，所有字母改为大写

使用字符串对象时，*正则表达式*是一个强大的工具。Python 在 re 模块中提供这种功能：

```
In [30]: import re
```

假定你面对一个大的文本文件，例如逗号分隔值（CSV）文件，其中包含了某些事件序列和相应的日期-时间信息。日期-时间信息多半以 Python 无法直接解释的格式提供。然而，日期-时间信息通常可以通过正则表达式描述。考虑如下的字符串对象，它包含 3 个日期-时间元素、3 个整数和 3 个字符串。注意，3 个引号可以在多行上定义字符串：

```
In [31]: series = """
         '01/18/2014 13:00:00', 100, '1st';
         '01/18/2014 13:30:00', 110, '2nd';
         '01/18/2014 14:00:00', 120, '3rd'
         """
```

下面的正则表达式描述上述字符串对象中提供的日期-时间信息格式[1]：

```
In [32]: dt = re.compile("'[0-9/:\s]+'")  # datetime
```

使用上述正则表达式，我们可以查找所有日期-时间元素，一般来说，对字符串对象应用正则表达式还有带来典型解析任务性能的提升：

```
In [33]: result = dt.findall(series)
         result
Out[33]: ["'01/18/2014 13:00:00'", "'01/18/2014 13:30:00'", "'01/18/2014 14:00:0
         0'"]
```

1　在此不可能深入细节，但是互联网上能够找到很多正则表达式以及在 Python 中特殊应用的相关信息。这个主题的简介可以参见 Fitzgerald，Michael（2012）：《Introducing Regular Expressions.》，O'Reilly，Sebastopol，CA。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

正则表达式

解析字符串对象时，考虑使用正则表达式，可以为这些操作带来便利和高性能。

然后，结果字符串对象可以进行解析，生成 Python 的日期时间（datetime）对象（用 Python 处理日期和时间数据的概述参见附录 C）。要解析包含日期时间信息的字符串对象，我们必须提供解析方法的信息——同样是一个字符串对象：

```
In [34]: from datetime import datetime
         pydt = datetime.strptime(result[0].replace("'", ""),
                                  '%m/%d/%Y %H:%M:%S')
         pydt
Out[34]: datetime.datetime(2014, 1, 18, 13, 0)
In [35]: print pydt
Out[35]: 2014-01-18 13:00:00
In [36]: print type(pydt)
Out[36]: <type 'datetime.datetime'>
```

以后的章节会提供关于日期时间数据、这些数据的处理以及 **datetime** 对象及其方法的更多信息。现在我们只是预览一下这个金融中的重要主题。

4.2 基本数据结构

一般的原则是，数据结构是包含其他对象（可能很多）的对象。Python 提供如下内建结构：

元组（tuple）

任意对象的集合；只有少数可用方法

列表（list）

任意对象的集合；有许多可用方法

字典（dict）

键-值存储对象

集合（set）

其他*独特*对象的无序集合对象

4.2.1 元组

元组（tuple）是一种高级的数据结构，但是其应用仍然相当简单有限，通过在圆括号

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

中提供对象定义：

```
In [37]: t = (1, 2.5, 'data')
         type(t)
Out[37]: tuple
```

甚至可以去掉括号，提供以逗号分隔的多个对象：

```
In [38]: t = 1, 2.5, 'data'
         type(t)
Out[38]: tuple
```

和 Python 中几乎所有数据结构相似，元组有内建的索引，利用它可以读取元组的单个或者多个元素。重要的是记住 Python 使用*零起点编号*，元组的第 3 个元素在索引位置 2 上：

```
In [39]: t[2]
Out[39]: 'data'
In [40]: type(t[2])
Out[40]: str
```

零起点编号

与 Matlab 等其他编程语言不同，Python 使用零起点编号方案。例如，元组对象的第一个元素索引值为 0。

这种对象类型只有两个特殊方法：**count** 和 **index**。第一个方法计算某个对象的出现次数，第二个方法给出对象第一次出现位置的索引值：

```
In [41]: t.count('data')
Out[41]: 1
In [42]: t.index(1)
Out[42]: 0
```

元组对象不是非常灵活，因为一旦定义，它们不容易更改。

4.2.2 列表

与元组对象相比，列表（**list**）类型的对象更灵活、更强大。从金融角度看，许多工作只能用列表对象完成，例如存储股票报价和附加新数据。列表对象通过方括号定义，基本功能和行为与元组类似：

```
In [43]: l = [1, 2.5, 'data']
         l[2]
Out[43]: 'data'
```

列表对象也可以使用 **list** 函数定义或者转换。下面的代码通过转换前一个例子中的远足对象生成新的列表对象：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [44]: l = list(t)
         l
Out[44]: [1, 2.5, 'data']
In [45]: type(l)
Out[45]: list
```

除了元组对象的特性之外，列表对象还可以通过不同方法扩展和缩小。换句话说，字符串和元组对象是*不可变*序列对象（具有索引），一经创建不能更改，而列表对象是*可变*的，可以通过不同操作更改。可以在现有列表对象中附加一个或者多个列表对象：

```
In [46]: l.append([4, 3])  # append list at the end
         l
Out[46]: [1, 2.5, 'data', [4, 3]]
In [47]: l.extend([1.0, 1.5, 2.0])  # append elements of list
         l
Out[47]: [1, 2.5, 'data', [4, 3], 1.0, 1.5, 2.0]
In [48]: l.insert(1, 'insert')  # insert object before index position
         l
Out[48]: [1, 'insert', 2.5, 'data', [4, 3], 1.0, 1.5, 2.0]
In [49]: l.remove('data')  # remove first occurrence of object
         l
Out[49]: [1, 'insert', 2.5, [4, 3], 1.0, 1.5, 2.0]
In [50]: p = l.pop(3)  # removes and returns object at index
         print l, p
Out[50]: [1, 'insert', 2.5, 1.0, 1.5, 2.0] [4, 3]
```

切片操作也很容易实现。这里的*切片*（Slicing）指的是将数据集分解为（感兴趣的）较小部分：

```
In [51]: l[2:5]  # 3rd to 5th elements
Out[51]: [2.5, 1.0, 1.5]
```

表 4-2 提供列表对象精选操作和方法的摘要。

表 4-2　精选的列表对象操作和方法

方法	参数	返回/结果
l[i] = x	[i]	用 x 替代第 i 个元素
l[i:j:k] = s	[i:j:k]	用 s 代替从 i 到 j-1 号元素中的每第 k 个元素
append	(x)	在对象后附加 x
count	(x)	对象 x 的出现次数
del l[i:j:k]	[i:j:k]	删除索引值从 i 到 j-1 的元素
extend	(s)	将 s 的所有元素附加到对象

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

方法	参数	返回/结果
index	(x[, i[, j]])	元素 i 和 j-1 之间第一个 x 的索引
insert	(i, x)++	在索引 i 之前插入 x
remove	(i)	删除索引为 i 的元素
pop	(i)	删除索引为 i 的元素并返回之
reverse	()	颠倒所有项目的顺序
sort	([cmp[, key[, reverse]]])	排序所有项目

4.2.3 离题：控制结构

控制结构（如 for 循环）本身就是一个主题，但是在 Python 中，最好在列表对象基础上介绍。这是因为，循环一般来说都在列表对象上进行，这与其他语言中的常用标准有很大不同。看看下面的例子，for 循环在列表对象 l 索引值为 2-4 的元素上进行，打印各个元素的平方。注意第二行中缩进（空白）的重要性：

```
In [52]: for element in l[2:5]:
             print element ** 2
Out[52]: 6.25
         1.0
         2.25
```

和典型的基于计数器循环相比，这提供了很高的灵活性。基于计数器的循环也是 Python 的一个可选项，但是也是在（标准）列表对象 range 的基础上完成的：

```
In [53]: r = range(0, 8, 1)  # start, end, step width
         r
Out[53]: [0, 1, 2, 3, 4, 5, 6, 7]
In [54]: type(r)
Out[54]: list
```

为了比较，下面我们使用 range 实现同一个循环：

```
In [55]: for i in range(2, 5):
             print l[i] ** 2
Out[55]: 6.25
         1.0
         2.25
```

在列表上循环

在 Python 中，可以在任意列表对象上循环，不管这些对象的内容是什么，往往可以避免引入计数器。

Python 还提供典型（条件）控制元素 **if**、**elif** 和 **else**。它们的用途与其他语言相同：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [56]: for i in range(1, 10):
             if i % 2 == 0:  # % is for modulo
                 print "%d is even" % i
             elif i % 3 == 0:
                 print "%d is multiple of 3" % i
             else:
                 print "%d is odd" % i
Out[56]: 1 is odd
         2 is even
         3 is multiple of 3
         4 is even
         5 is odd
         6 is even
         7 is odd
         8 is even
         9 is multiple of 3
```

同样，while 提供另一种流程控制手段：

```
In [57]: total = 0
         while total < 100:
             total += 1
         print total
Out[57]: 100
```

Python 的特殊性之一是所谓的**列表推导（List Comprehensions）**。这种方法不在现有列表对象上循环，而是以紧凑的方式通过循环生成列表对象：

```
In [58]: m = [i ** 2 for i in range(5)]
         m
Out[58]: [0, 1, 4, 9, 16]
```

在某种意义上，这已经提供了以较为隐含（而非显式）的循环生成某种向量化代码的手段（代码向量化将在本章后面更详细地讨论）。

4.2.4 离题：函数式编程

Python 也提供一些用于函数式编程支持的工具——即在一整组输入（在我们的例子中是列表对象）上应用某个函数。这些工具是**过滤（filter）**、**映射（map）**和**归纳（reduce）**。但是，我们首先需要一个函数定义。从简单的功能出发，考虑返回输入 x 的平方数的函数 f：

```
In [59]: def f(x):
             return x ** 2
         f(2)
Out[59]: 4
```

当然，函数可能很复杂，有多个输入/参数对象甚至多个输出（返回对象）。考虑如下函数：

```
In [60]: def even(x):
             return x % 2 == 0
         even(3)
Out[60]: False
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

返回对象是一个布尔值。这样的函数可以用 **map** 应用到整个列表对象：

```
In [61]: map(even, range(10))
Out[61]: [True, False, True, False, True, False, True, False, True, False]
```

为此，我们还可以使用 **lambda** 或者匿名函数，直接提供一个函数定义作为 **map** 的参数：

```
In [62]: map(lambda x: x ** 2, range(10))
Out[62]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

函数也可以用于过滤列表对象。在下面的例子中，过滤器返回列表对象中匹配 **even** 函数定义的布尔条件的元素：

```
In [63]: filter(even, range(15))
Out[63]: [0, 2, 4, 6, 8, 10, 12, 14]
```

最后，归纳（reduce）在我们想要对列表对象的所有元素应用一个函数、返回单个值的时候很有用。列表对象中所有元素的总和就是一个例子（假定总和是为列表中包含的对象定义的）：

```
In [64]: reduce(lambda x, y: x + y, range(10))
Out[64]: 45
```

下面是替代的非函数式实现：

```
In [65]: def cumsum(l):
             total = 0
             for elem in l:
                 total += elem
             return total
         cumsum(range(10))
Out[65]: 45
```

列表推导，函数式编程，匿名函数

在 Python 级别上尽可能避免循环被视为“好的习惯”。列表推导和函数式编程工具（如 **map**、**filter** 和 **reduce**）提供了编写紧凑（一般来说也更易于理解）的无循环代码的手段。lambda 或者匿名函数也是这方面的强大工具。

4.2.5 字典

字典（**dict**）对象就是可以按照键码（可能是字符串对象）读取的数据字典，也是一种可变序列，是所谓的*键-值存储*。**列表**对象是有序且可排序的，而**字典**对象是无序、不可排序的。示例最能够说明和**列表**对象的不同之处。花括号是**字典**对象定义：

```
In [66]: d = {
             'Name' : 'Angela Merkel',
             'Country' : 'Germany',
             'Profession' : 'Chancelor',
             'Age' : 60
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            }
        type(d)
Out[66]: dict
In [67]: print d['Name'], d['Age']
Out[67]: Angela Merkel 60
```

同样，这类对象也有一些内建方法：

```
In [68]: d.keys()
Out[68]: ['Country', 'Age', 'Profession', 'Name']
In [69]: d.values()
Out[69]: ['Germany', 60, 'Chancelor', 'Angela Merkel']
In [70]: d.items()
Out[70]: [('Country', 'Germany'),
          ('Age', 60),
          ('Profession', 'Chancelor'),
          ('Name', 'Angela Merkel')]
In [71]: birthday = True
         if birthday is True:
             d['Age'] += 1
         print d['Age']
Out[71]: 61
```

从字典对象中获得迭代器（**Iterator**）对象有多种方法。这种对象在循环时的表现与列表对象类似：

```
In [72]: for item in d.iteritems():
             print item
Out[72]: ('Country', 'Germany')
         ('Age', 61)
         ('Profession', 'Chancelor')
         ('Name', 'Angela Merkel')
In [73]: for value in d.itervalues():
             print type(value)
Out[73]: <type 'str'>
         <type 'int'>
         <type 'str'>
         <type 'str'>
```

表 4-3 提供了字典对象上精选的操作和方法摘要。

表 4-3　字典对象精选操作和方法

方法	参数	返回/结果
d[k]	[k]	d 中键码为 k 的项目
d[k] = x	[k]	将键码为 k 的项目设置为 x
del d[k]	[k]	删除键码为 k 的项目

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

方法	参数	返回/结果
clear	()	删除所有项目
copy	()	建立一个拷贝
has_key	(k)	如果 k 是一个键码，为真
items	()	所有键-值对的拷贝
iteritems	()	所有项目的迭代器
iterkeys	()	所有键码的迭代器
itervalues	()	所有值的迭代器
keys	()	所有键码的拷贝
poptiem	(k)	返回并删除键码为 k 的项目
update	([e])	用来自 e 的项目更新字典项目
values	()	所有值的拷贝

4.2.6 集合

我们所要考虑的最后一种数据结构是**集合**（**set**）对象。虽然集合论是数学和金融理论的基石，但是**集合**对象并没有太多实际应用。这种对象是其他对象的无序集合，每个元素只包含一次：

```
In [74]: s = set(['u', 'd', 'ud', 'du', 'd', 'du'])
         s
Out[74]: {'d', 'du', 'u', 'ud'}
In [75]: t = set(['d', 'dd', 'uu', 'u'])
```

使用**集合**对象，可以实现数学集合论中进行的运算。例如，可以生成并集、交集和差集：

```
In [76]: s.union(t)  # all of s and t
Out[76]: {'d', 'dd', 'du', 'u', 'ud', 'uu'}
In [77]: s.intersection(t)  # both in s and t
Out[77]: {'d', 'u'}
In [78]: s.difference(t)  # in s but not t
Out[78]: {'du', 'ud'}
In [79]: t.difference(s)  # in t but not s
Out[79]: {'dd', 'uu'}
In [80]: s.symmetric_difference(t)  # in either one but not both
Out[80]: {'dd', 'du', 'ud', 'uu'}
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

集合对象的应用之一是去掉**列表**对象中的重复。例如：

```
In [81]: from random import randint
         l = [randint(0, 10) for i in range(1000)]
             # 1,000 random integers between 0 and 10
         len(l)  # number of elements in l
Out[81]: 1000
In [82]: l[:20]
Out[82]: [8, 3, 4, 9, 1, 7, 5, 5, 6, 7, 4, 4, 7, 1, 8, 5, 0, 7, 1, 9]
In [83]: s = set(l)
         s
Out[83]: {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

4.3 NumPy 数据结构

前一小节说明了 Python 提供的一些相当强大而灵活的通用数据结构，特别是**列表**对象，可以视为真正的主力干将，具有许多方便的特性和应用领域。然而，科学和金融应用通常有在特殊数据结构上进行高性能操作的需求。这方面最重要的数据结构之一是*数组*。数组通常在行和列中存放其他（基本）对象。

现在假定我们只使用数值，但是这些概念可以推广到其他类型的数据。在最简单的情况下，一维数组从数学上讲，一般可以表示一个实数向量，这些实数的内部表示是**浮点**对象。一维数组只表示单行或者单列元素。在更常见的情况下，数组表示一个 $i \times j$ *个元素的矩阵*。这一概念可以推广到 $i \times j \times k$ *个元素的立方矩阵（3 维数组）*，直到包含 $i \times j \times k \times l \cdots$ 个元素的 n 维数组。

线性代数和向量空间理论等数学门类阐述了这种数学结构在许多学科和领域中的重要性，也就证明了提供专门为方便高效处理数组设计的数据结构是卓有成效的。这就是 **Python** 库 **NumPy** 及其 **ndarray** 类大展身手的领域。

4.3.1 用 Python 列表形成数组

在我们转向 **NumPy** 之前，首先用前一节介绍的内建数据结构构造数组。**列表**对象特别适合于完成这项任务。简单的**列表**已经可以视为一维数组：

```
In [84]: v = [0.5, 0.75, 1.0, 1.5, 2.0]  # vector of numbers
```

由于**列表**对象可能包含任意的其他对象，它们也可以包含其他**列表**对象。这样，二维和更高维的数组很容易通过嵌套列表对象构造：

```
In [85]: m = [v, v, v]  # matrix of numbers
         m
Out[85]: [[0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0]]
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们也可以很轻松地通过简单的索引选择行，通过双重索引访问单个元素（但是，选择整列不容易）：

```
In [86]: m[1]
Out[86]: [0.5, 0.75, 1.0, 1.5, 2.0]
In [87]: m[1][0]
Out[87]: 0.5
```

嵌套可以推进一步，获得更通用的结构：

```
In [88]: v1 = [0.5, 1.5]
         v2 = [1, 2]
         m = [v1, v2]
         c = [m, m]  # cube of numbers
         c
Out[88]: [[[0.5, 1.5], [1, 2]], [[0.5, 1.5], [1, 2]]]
In [89]: c[1][1][0]
Out[89]: 1
```

注意，按照刚才介绍的方式组合对象通常使用原始对象的引用指针。这在实践中有何意义？我们来看看如下的操作：

```
In [90]: v = [0.5, 0.75, 1.0, 1.5, 2.0]
         m = [v, v, v]
         m
Out[90]: [[0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0]]
```

现在，改变 v 对象的第一个元素，看看 m 对象上发生了什么：

```
In [91]: v[0] = 'Python'
         m
Out[91]: [['Python', 0.75, 1.0, 1.5, 2.0],
          ['Python', 0.75, 1.0, 1.5, 2.0],
          ['Python', 0.75, 1.0, 1.5, 2.0]]
```

使用 copy 模块的 deepcopy 函数可以避免这一现象：

```
In [92]: from copy import deepcopy
         v = [0.5, 0.75, 1.0, 1.5, 2.0]
         m = 3 * [deepcopy(v), ]
         m
Out[92]: [[0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0]]
In [93]: v[0] = 'Python'
         m
Out[93]: [[0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0],
          [0.5, 0.75, 1.0, 1.5, 2.0]]
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

4.3.2 常规 NumPy 数组

显然，用列表对象组成数组结构是可行的。但是这实际上并不方便，**list** 类本身不是为这一特殊目标设计的，它是为了更广泛、更通用的目的而构建的。从这一点出发，某种专用类能够真正地为处理数组类型结构带来好处。

这一专用类就是 **numpy.ndarray**，它的具体目标就是方便高效（也就是高性能）地处理 n 维数组。同样，最好是用示例来说明这种类实例的处理方式：

```
In [94]: import numpy as np
In [95]: a = np.array([0, 0.5, 1.0, 1.5, 2.0])
         type(a)
Out[95]: numpy.ndarray
In [96]: a[:2]  # indexing as with list objects in 1 dimension
Out[96]: array([ 0. ,  0.5])
```

numpy.ndarray 类的主要特征是*多种内建方法*。例如：

```
In [97]: a.sum()  # sum of all elements
Out[97]: 5.0
In [98]: a.std()  # standard deviation
Out[98]: 0.70710678118654757
In [99]: a.cumsum()  # running cumulative sum
Out[99]: array([ 0. ,  0.5,  1.5,  3. ,  5. ])
```

另一个重要特征是 **ndarray** 对象上定义的（向量化）*数学运算*：

```
In [100]: a * 2
Out[100]: array([ 0.,  1.,  2.,  3.,  4.])
In [101]: a ** 2
Out[101]: array([ 0.  ,  0.25,  1.  ,  2.25,  4.  ])
In [102]: np.sqrt(a)
Out[102]: array([ 0.        ,  0.70710678,  1.        ,  1.22474487,  1.41421356
          ])
```

一维以上数组可以无缝过渡，迄今为止介绍的功能都可以更通用的方式延续。特别是，所有维度上都有一致的索引系统：

```
In [103]: b = np.array([a, a * 2])
          b
Out[103]: array([[ 0. ,  0.5,  1. ,  1.5,  2. ],
                 [ 0. ,  1. ,  2. ,  3. ,  4. ]])
In [104]: b[0]  # first row
Out[104]: array([ 0. ,  0.5,  1. ,  1.5,  2. ])
In [105]: b[0, 2]  # third element of first row
Out[105]: 1.0
In [106]: b.sum()
Out[106]: 15.0
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

和基于列表对象的数组构造方法相比，**numpy.ndarray** 类明确地知道数组的轴，从一个矩阵上选择行或者列本质上都是一样的：

```
In [107]: b.sum(axis=0)
          # sum along axis 0, i.e. column-wise sum
Out[107]: array([ 0. ,  1.5,  3. ,  4.5,  6. ])
In [108]: b.sum(axis=1)
          # sum along axis 1, i.e. row-wise sum
Out[108]: array([  5.,  10.])
```

初始化（实例化）**numpy.ndarray** 对象有许多种方式。前面已经介绍了一种——通过 **np.array**。但是，这种方法假设所有数组元素已经可用。相反，有人可能喜欢先实例化 **numpy.ndarray** 对象，然后再用代码执行期间生成的结果填充数组。为此，我们可以使用如下函数：

```
In [109]: c = np.zeros((2, 3, 4), dtype='i', order='C')  # also: np.ones()
          c
Out[109]: array([[[0, 0, 0, 0],
                  [0, 0, 0, 0],
                  [0, 0, 0, 0]],

                 [[0, 0, 0, 0],
                  [0, 0, 0, 0],
                  [0, 0, 0, 0]]], dtype=int32)
In [110]: d = np.ones_like(c, dtype='f16', order='C')  # also: np.zeros_like()
          d
Out[110]: array([[[ 1.0,  1.0,  1.0,  1.0],
                  [ 1.0,  1.0,  1.0,  1.0],
                  [ 1.0,  1.0,  1.0,  1.0]],

                 [[ 1.0,  1.0,  1.0,  1.0],
                  [ 1.0,  1.0,  1.0,  1.0],
                  [ 1.0,  1.0,  1.0,  1.0]]], dtype=float128)
```

利用这些函数，我们提供了如下信息：

组成

整数（int），整数序列或者应用另一个 numpy.ndarray

dtype（可选）

numpy.dtype——用于 **numpy.ndarray** 的 **Numpy** 特定数据类型

order（顺序，可选）

在内存中存储元素的顺序：**C** 指和 **C** 语言相似（即行优先），**F** 指和 **Fortran** 类似（列优先）

现在，和基于**列表**的方法相比，**NumPy** 用 **Numpy.ndarray** 类构造数组方式的特殊之处已经很明显：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- 数组的组成/长度/大小在任何维度内都是*同质*的。
- 整个数组只允许一*种数据类型（numpy.dtype）*。

order 参数的作用将在本章后面讨论。表 4-4 提供了 **numpy.dtype** 对象的概述（即 **NumPy** 可以使用的基本数据类型）。

表 4-4　NumPy dtype 对象

dtype	描述	示例
t	位域	t4（4 位）
b	布尔值	b（true 或者 false）
I	整数	i8（64 位）
u	无符号整数	u8（64 位）
f	浮点数	f8（64 位）
c	浮点复数	c16（128 位）
o	对象	0（指向对象的指针）
S, a	字符串	S24（24 个字符）
U	Unicode	U24（24 个 Unicode 字符）
V	其他	V12（12 字节数据块）

NumPy 提供了常规数组（至少缺乏 **dtype** 限制）的广义形式，但是我们暂时坚持使用常规数组，以观察这种专门化措施对性能的影响。

做个简单的练习，假定我们要生成一个 5000 × 5000 个元素组成的矩阵/数组，填入标准正态分布的（伪）随机数，然后计算所有元素的总和。首先采用纯 **Python** 方法，大量使用**列表**推导和函数式编程方法以及 **lambda** 函数：

```
In [111]: import random
          I = 5000
In [112]: %time mat = [[random.gauss(0, 1) for j in range(I)] for i in range(I)]
            # a nested list comprehension
Out[112]: CPU times: user 36.5 s, sys: 408 ms, total: 36.9 s
          Wall time: 36.4 s
In [113]: %time reduce(lambda x, y: x + y,      \
               [reduce(lambda x, y: x + y, row) \
                       for row in mat])
Out[113]: CPU times: user 4.3 s, sys: 52 ms, total: 4.35 s
          Wall time: 4.07 s

          678.5908519876674
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

现在转向 **NumPy**，看看同一个问题如何解决。为方便起见，**NumPy** 子库 **random** 提供了多种函数，以初始化 **numpy.ndarray** 对象，同时填入（伪）随机数：

```
In [114]: %time mat = np.random.standard_normal((I, I))
Out[114]: CPU times: user 1.83 s, sys: 40 ms, total: 1.87 s
          Wall time: 1.87 s
In [115]: %time mat.sum()
Out[115]: CPU times: user 36 ms, sys: 0 ns, total: 36 ms
          Wall time: 34.6 ms

          349.49777911439384
```

我们观察到如下情况：

语法

尽管我们使用了许多方法来压缩纯 **Python** 代码，**NumPy** 版本仍然更为紧凑、更易于理解。

性能

和纯 **Python** 的对应操作相比，**numpy.ndarray** 对象的生成大约快 20 倍，总和的计算大约快 100 倍。

使用 Numpy 数组

将 NumPy 用于基于数组的操作和算法，通常可以生成比纯 Python 更紧凑、容易理解的代码，并能显著地改进性能。

4.3.3 结构数组

numpy.ndarray 类的专门化带来了许多真正有价值的好处。但是，过分狭窄的专门化措施可能给大部分基于数组的算法和应用程序造成过大的负担。因此，**NumPy** 提供结构数组，允许我们至少在*每列*上使用不同的 **NumPy** 数据类型。“每列”是什么含义？考虑如下结构数组对象的初始化：

```
In [116]: dt = np.dtype([('Name', 'S10'), ('Age', 'i4'),
                         ('Height', 'f'), ('Children/Pets', 'i4', 2)])
          s = np.array([('Smith', 45, 1.83, (0, 1)),
                        ('Jones', 53, 1.72, (2, 2))], dtype=dt)
          s
Out[116]: array([('Smith', 45, 1.8300000429153442, [0, 1]),
                 ('Jones', 53, 1.7200000286102295, [2, 2])],
                dtype=[('Name', 'S10'), ('Age', '<i4'), ('Height', '<f4'), ('Chi
          ldren/Pets', '<i4', (2,))])
```

在某种意义上，这种构造方法很接近于 **SQL** 数据库中表的初始化。我们使用列名和列数据类型，以及某些附加信息（例如，每个字符串对象的最大字符数）。现在，各个列可以简单地用它们的名称访问：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [117]: s['Name']
Out[117]: array(['Smith', 'Jones'],
                dtype='|S10')
In [118]: s['Height'].mean()
Out[118]: 1.7750001
```

分别选择特定行和记录，结果对象的行为类似**字典**对象，可以通过键码检索值：

```
In [119]: s[1]['Age']
Out[119]: 53
```

总而言之，结构数组是常规 **numpy.ndarray** 对象类型的推广，其中*每列*的数据类型必须相等，就像 **SQL** 数据库中的表一样。结构数组的优势之一是某列中的元素可以使另一个多维对象，不一定要遵循基本的 **NumPy** 数据类型。

结构数组

除了常规数组之外，**NumPy** 还提供结构数组，可以用不同数据类型甚至结构，按照（命名）列描述和处理相当复杂的面向数组数据结构。它们为 **Python** 带来了类似于 SQL 表的数据结构，具有常规 **numpy.ndarray** 对象的所有好处（语法、方法、性能）。

4.4 代码向量化

代码的向量化是获得执行速度更快、更紧凑代码的一种策略。基本思路是“一次”在一个复杂对象上进行操作，或者向其应用某个函数，而不是通过在对象的单个元素上循环来进行。在 **Python** 中，函数式编程工具 **map**、**filter** 和 **reduce** 提供了向量化的手段。在某种意义上，**NumPy** 的向量化深植于其核心之中。

4.4.1 基本向量化

正如在前一小节中所学到的，简单的数学运算可以直接在 **numpy.ndarray** 对象上进行。例如，我们可以将两个 **NumPy** 数组按元素相加：

```
In [120]: r = np.random.standard_normal((4, 3))
          s = np.random.standard_normal((4, 3))
In [121]: r + s
Out[121]: array([[-1.94801686, -0.6855251 ,  2.28954806],
                 [ 0.33847593, -1.97109602,  1.30071653],
                 [-1.12066585,  0.22234207, -2.73940339],
                 [ 0.43787363,  0.52938941, -1.38467623]])
```

NumPy 还支持所谓的*广播*，可以在一个操作内组合不同组成的对象。之前我们已经使用过这种方法，考虑如下的示例：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [122]: 2 * r + 3
Out[122]: array([[ 2.54691692,  1.65823523,  8.14636725],
                 [ 4.94758114,  0.25648128,  1.89566919],
                 [ 0.41775907,  0.58038395,  2.06567484],
                 [ 0.67600205,  3.41004636,  1.07282384]])
```

在这个例子中，r 对象按元素乘以 2，然后按元素加 3——3 被*广播（或者延伸）*到 r 对象的组成之中。在某种程度上，这种方法也适合于不同组成的数组：

```
In [123]: s = np.random.standard_normal(3)
          r + s
Out[123]: array([[ 0.23324118, -1.09764268,  1.90412565],
                 [ 1.43357329, -1.79851966, -1.22122338],
                 [-0.83133775, -1.63656832, -1.13622055],
                 [-0.70221625, -0.22173711, -1.63264605]])
```

上述代码将大小为 3 的一维数组广播到（4，3）数组中。对于不同大小（例如 4）的一维数组，相同的代码无法正常工作：

```
In [124]: s = np.random.standard_normal(4)
          r + s
Out[124]: ValueError
          operands could not be broadcast together with shapes (4,3) (4,)
```

不过，将 r 对象转置，这一操作再次变为有效。在下面的代码中，**transpose** 方法将 **ndarray** 对象的组成从（4，3）转换为组成为（3，4）的同类型对象：

```
In [125]: r.transpose() + s
Out[125]: array([[-0.63380522,  0.5964174 ,  0.88641996, -0.86931849],
                 [-1.07814606, -1.74913253,  0.9677324 ,  0.49770367],
                 [ 2.16591995, -0.92953858,  1.71037785, -0.67090759]])
In [126]: np.shape(r.T)
Out[126]: (3, 4)
```

一般的原则是，自定义的 **Python** 函数也可以使用 **numpy.ndarrays**。如果实现允许，数组可以像整数或者浮点数对象一样用于函数。考虑如下的函数：

```
In [127]: def f(x):
              return 3 * x + 5
```

我们可以传递标准的 **Python** 对象，也可以传递 **numpy.ndarray** 对象（当然，函数中的操作必须定义）：

```
In [128]: f(0.5)  # float object
Out[128]: 6.5
In [129]: f(r)  # NumPy array
Out[129]: array([[  4.32037538,   2.98735285,  12.71955087],
                 [  7.9213717 ,   0.88472192,   3.34350378],
                 [  1.1266386 ,   1.37057593,   3.59851226],
                 [  1.51400308,   5.61506954,   2.10923576]])
```

NumPy 所做的是简单地为对象的各个元素应用函数 **f**。从这个意义上，使用此类操作

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

并没有避免循环，只是在 Python 级别上避免它们，并将循环委托给 NumPy。在 NumPy 级别上，在 **numpy.ndarray** 对象上进行的循环由经过高度优化的代码负责，大部分代码用 C 语言编写，因而一般远快于纯 **Python**。这就可以解释使用 NumPy 解决基于数组用例性能优势的“秘密”。

使用数组时，程序员必须正确调用对应对象上的函数。例如，来自 Python 标准模块 **math** 的 **sin** 函数不能用于 NumPy 数组：

```
In [130]: import math
          math.sin(r)
Out[130]: TypeError
          only length-1 arrays can be converted to Python scalars
```

该函数设计用于处理**浮点**对象——即单个数值而非数组。NumPy 提供了对应的函数，即所谓的*通用函数（ufuncs）*：

```
In [131]: np.sin(r)  # array as input
Out[131]: array([[-0.22460878, -0.62167738,  0.53829193],
                 [ 0.82702259, -0.98025745, -0.52453206],
                 [-0.96114497, -0.93554821, -0.45035471],
                 [-0.91759955,  0.20358986, -0.82124413]])
In [132]: np.sin(np.pi)  # float as input
Out[132]: 1.2246467991473532e-16
```

NumPy 提供大量 ufunc，将典型的数学函数推广到 **numpy.ndarray** 对象上[1]。

通用函数

使用 **from library import** *方法导入时要小心。这种方法可能造成 **NumPy** 对 **numpy.sin** 通用函数的应用被 **math** 函数 **math.sin** 的引用所代替。原则是，应该按名导入两个库以避免混乱：**import numpy as np**; **import math**。此后，就可以同时使用 **math.sin** 和 **np.sin**。

4.5 内存布局

第一次使用 **numpy.zero** 初始化 **numpy.ndarray** 对象时，我们为内存布局提供可选的参数。粗略地讲，这个参数指定数组中数组的哪些元素在内存中相邻的地方存储。使用小型数组时，这对数组操作性能没有任何可测的影响。但是，当数组变得很大时情况就有所不同，这取决于在数组上实施的操作。

为了说明根据内存布局进行的数组处理在科学和金融应用中的重要性，考虑如下多维 **numpy.ndarray** 对象的构造：

1 这个主题的概述参见 http://docs.scipy.org/doc/numpy/reference/ufuncs.html。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [133]: x = np.random.standard_normal((5, 10000000))
          y = 2 * x + 3  # linear equation y = a * x + b
          C = np.array((x, y), order='C')
          F = np.array((x, y), order='F')
          x = 0.0; y = 0.0  # memory cleanup
In [134]: C[:2].round(2)
Out[134]: array([[[-0.51, -1.14, -1.07, …,  0.2 , -0.18,  0.1 ],
                  [-1.22,  0.68,  1.83, …,  1.23, -0.27, -0.16],
                  [ 0.45,  0.15,  0.01, …, -0.75,  0.91, -1.12],
                  [-0.16,  1.4 , -0.79, …, -0.33,  0.54,  1.81],
                  [ 1.07, -1.07, -0.37, …, -0.76,  0.71,  0.34]],

                 [[ 1.98,  0.72,  0.86, …,  3.4 ,  2.64,  3.21],
                  [ 0.55,  4.37,  6.66, …,  5.47,  2.47,  2.68],
                  [ 3.9 ,  3.29,  3.03, …,  1.5 ,  4.82,  0.76],
                  [ 2.67,  5.8 ,  1.42, …,  2.34,  4.09,  6.63],
                  [ 5.14,  0.87,  2.27, …,  1.48,  4.43,  3.67]]])
```

我们来看看两种 **ndarray** 对象的基本示例和用例：

```
In [135]: %timeit C.sum()
Out[135]: 10 loops, best of 3: 123 ms per loop
In [136]: %timeit F.sum()
Out[136]: 10 loops, best of 3: 123 ms per loop
```

加总数组所有元素时，两种内存布局没有任何性能差异。但是，考虑如下类 C 语言内存布局的示例：

```
In [137]: %timeit C[0].sum(axis=0)
Out[137]: 10 loops, best of 3: 102 ms per loop
In [138]: %timeit C[0].sum(axis=1)
Out[138]: 10 loops, best of 3: 61.9 ms per loop
```

加总 5 个大型向量并返回单个大型结果向量，明显慢于加总 10 万个小型向量和返回相同数量的结果。这是因为小型向量（即行）的单个元素存储位置相邻。使用类 Fortran 内存布局，相对性能变化很可观：

```
In [139]: %timeit F.sum(axis=0)
Out[139]: 1 loops, best of 3: 801 ms per loop
In [140]: %timeit F.sum(axis=1)
Out[140]: 1 loops, best of 3: 2.23 s per loop
In [141]: F = 0.0; C = 0.0  # memory cleanup
```

在这个例子中，在少数大型向量上操作必在大量小型向量上性能更好。少数大型向量的元素保存在相邻的内存位置上，这可以解释相对的性能优势。但是，与类 C 语言变种相比，整体操作绝对要慢得多。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

4.6 结语

Python 结合 NumPy，提供了一组丰富灵活的数据结构，从金融的角度看，下面这些类型是最重要的：

基本数据类型

在金融中，**整数**、**浮点数**和**字符串**提供了原子数据类型。

标准数据结构

元组、**列表**、**字典**和**集合**类在金融中有许多应用领域，**列表**通常是最灵活的主力。

数组

金融相关问题和算法中的一大类可以转换为数组；NumPy 提供了专门的 **numpy.ndarray** 类，该类提供了代码方便性、简洁性和高性能。

本章展示了基本数据结构和 NumPy 提供的可以用高度向量化方式实现算法的数据结构。根据数据结构的特定组成，必须小心考虑数组的内存布局。选择合适的方法可能将代码执行速度提高 2 个以上的数量级。

4.7 延伸阅读

本章聚焦于对金融算法和应用特别重要的问题。然而，这只是 Python 中数据结构和数据建模探索的出发点。有许多宝贵的资源深入地探讨了这些问题。

下面是一些值得参考的互联网资源。

- Python 文档总是很好的起点：http://www.python.org/doc/。
- NumPy 数组及相关方法及函数的细节参见 http://docs.scipy.org/doc/。
- SciPy 讲义也是很好的入门资源：http://scipylectures.github.io/。

下面是一些很好的参考书：

- Goodrich, Michael et al. (2013): Data Structures and Algorithms in Python. JohnWiley & Sons, Hoboken, NJ.
- Langtangen, Hans Petter (2009): A Primer on Scientific Programming with Python.Springer Verlag, Berlin, Heidelberg.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第5章

数据可视化

一图胜千言。

——Arthur Brisbane（1911）

本章介绍的是 **matplotlib** 库（http://www.matplotlib.org）的基本可视化功能。尽管有许多其他的可视化库，但是 **matplotlib** 已经确立了一个标杆，在许多情况下，它都是健壮、可靠的可视化工具。对于标准的绘图工作，它很容易理解，进行更复杂的绘图和自定义时，它又很灵活。此外，它与 Numpy 及其提供的数据结构紧密集成。

本章主要介绍如下主题：

2D 绘图

从最简单的图形到具有两种刻度或者不同子图的较高级图形；还介绍典型的金融图表，例如烛柱图。

3D 绘图

介绍一些对金融应用有用的 3D 图形。

本章无法全面地介绍 **Python** 和 **matplotlib** 所能进行的数据可视化，但是提供了对金融学最基本和最重要的功能示例。后续的章节中还可以找到其他示例。例如，第 6 章说明如何用 **pandas** 库可视化时间序列数据。

5.1 二维绘图

首先，我们必须导入对应的库。主要的绘图函数在子库 **matplotlib.pyplot** 中：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [1]: import numpy as np
        import matplotlib as mpl
        import matplotlib.pyplot as plt
        %matplotlib inline
```

5.1.1 一维数据集

在下面的所有例子中，我们将按照存储在 **NumPy ndarray** 对象中的数据绘制图表。当然，**matplotlib** 也能够按照以不同的 **Python** 格式存储的数据（如列表对象）绘制图表。首先，我们需要用于绘制图表的数据。为此，我们生成 20 个标准正态分布（伪）随机数，保存在一个 **NumPy ndarray** 中：

```
In [2]: np.random.seed(1000)
        y = np.random.standard_normal(20)
```

pyplot 子库中的 **plot** 函数是最基础的绘图函数，但是也相当强大。原则上，它需要两组数值。

- **x 值**：包含 x 坐标（横坐标）的列表或者数组。
- **y 值**：包含 y 坐标（纵坐标）的列表或者数组。

当然，x 和 y 值的数量必须相等，考虑下面两行代码，其输出如图 5-1 所示：

```
In [3]: x = range(len(y))
        plt.plot(x, y)
```

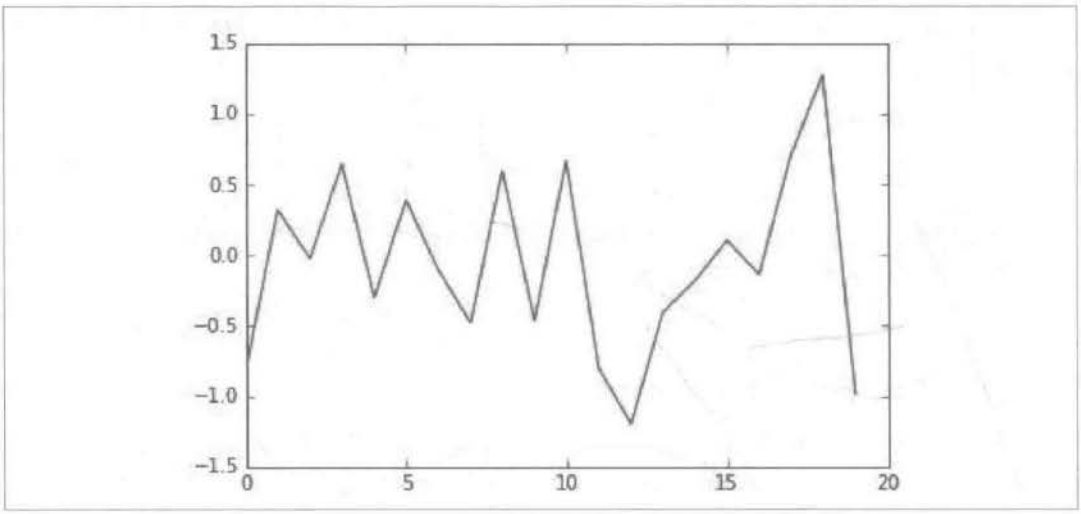


图 5-1　按照给定的 x 和 y 值绘图

plot 会注意到何时传递了 **ndarray** 对象。在这种情况下，没有必要提供 **x** 值的“额外”信息。如果你只提供 **y** 值，plot 以索引值作为对应的 **x** 值。因此，下面一行代码会生成完全一样的输出。（参见图 5-2）：

```
In [4]: plt.plot(y)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

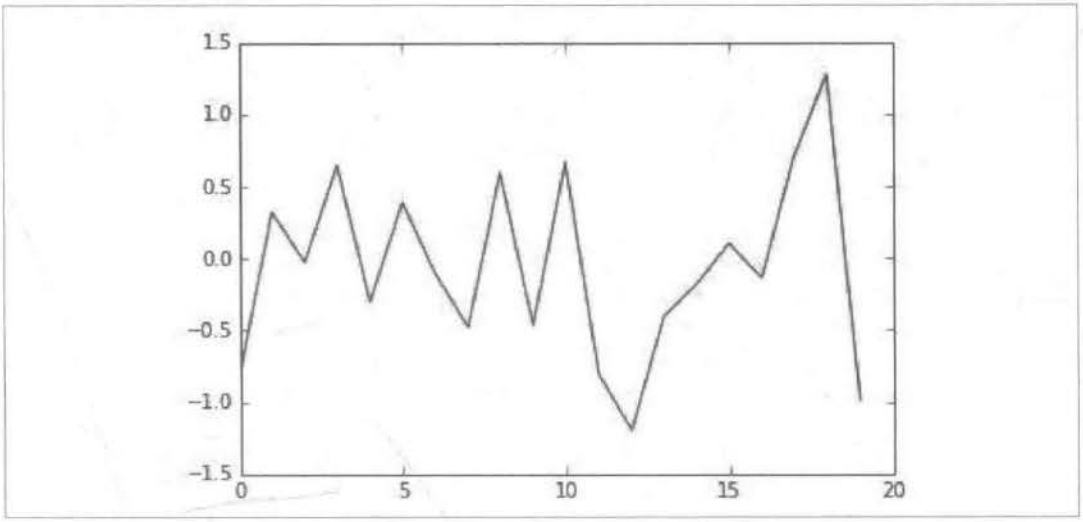


图 5-2　按照一维数组给出的数据绘图

NumPy 数组和 **matplotlib**

可以简单地向 **matplotlib** 函数传递 NumPy ndarray 对象。函数能够解释数据结构以简化绘图工作。但是要注意，不要传递太大或者太复杂的数组。

由于大部分 **ndarray** 方法返回的仍然是一个 **ndarray** 对象，也可以附加一个方法（有些时候甚至可以附加多个方法）传递对象。我们用样板数据调用 **ndarray** 对象上的 **cumsum** 方法，可以获得这些数据的总和，并且和预想的一样得到不同的输出（见图 5-3）：

```
In [5]: plt.plot(y.cumsum())
```

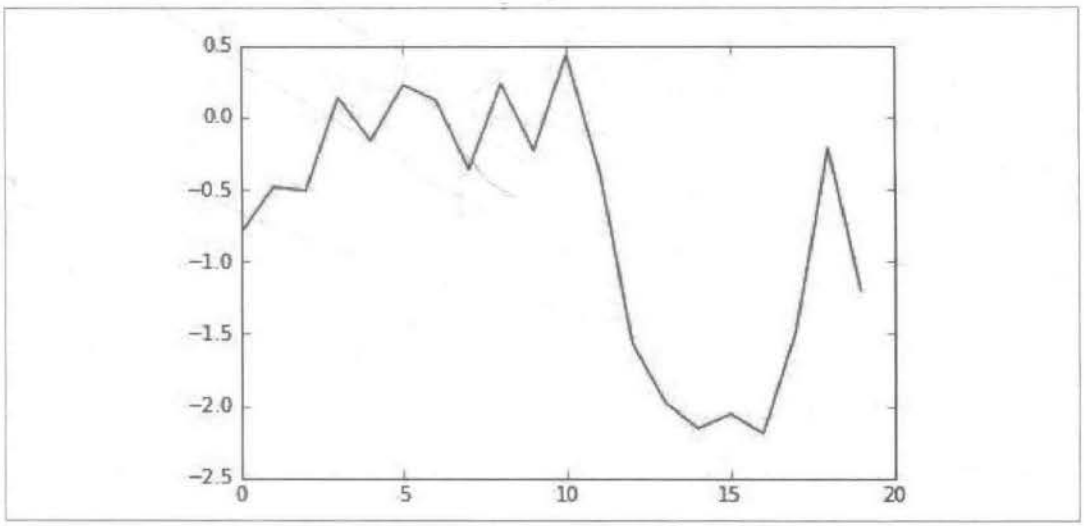


图 5-3　按照给定的一维数组和附加方法绘图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

一般来说，默认绘图样式不能满足报表、出版等的典型要求。例如，你可能希望自定义所使用的字体（例如，为了 LaTeX 字体兼容）、在坐标轴上有标签或者为了更好的可辨认性而绘制网格。因此，**matplotlib** 提供了大量函数以自定义绘图样式。有些函数容易理解，其他的则需要更深入一步研究。例如，操纵坐标轴和增加网格及标签的函数很容易理解（参见图 5-4）：

```
In [6]: plt.plot(y.cumsum())
        plt.grid(True)  # adds a grid
        plt.axis('tight')  # adjusts the axis ranges
```

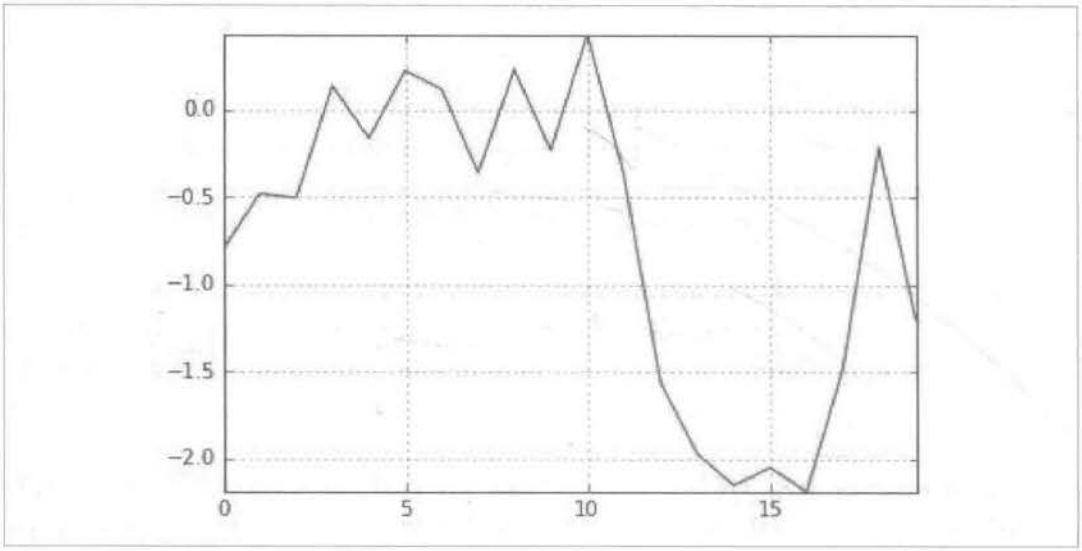


图 5-4　带有网格和紧凑坐标轴的图表

表 5-1 列出了 plt.axis 的其他选项，大部分都以字符串对象的形式传递。

表 5-1　plt.axis 选项

参数	描述
Empty	返回当前坐标轴限值
off	关闭坐标轴线和标签
equal	使用等刻度
scaled	通过尺寸变化平衡刻度
tight	使所有数据可见（缩小限值）
image	使所有数据可见（使用数据限值）
[xmin, xmax, ymin, ymax]	将设置限制为给定的（一组）值

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

此外，可以使用 **plt.xlim** 和 **plt.ylim** 设置每个坐标轴的最小值和最大值。下面的代码提供了一个示例，输出如图 5-5 所示：

```
In [7]: plt.plot(y.cumsum())
        plt.grid(True)
        plt.xlim(-1, 20)
        plt.ylim(np.min(y.cumsum()) - 1,
                 np.max(y.cumsum()) + 1)
```

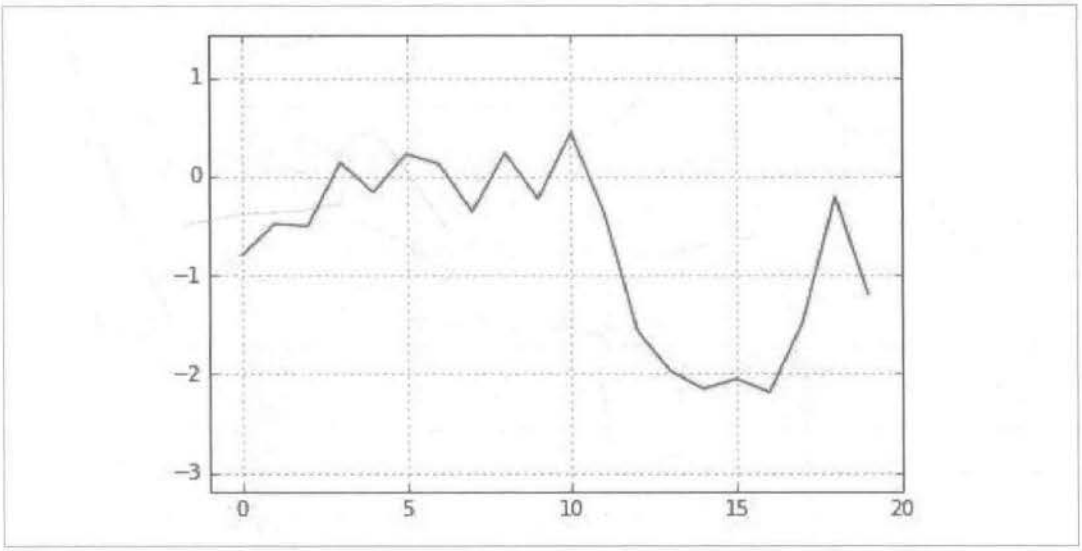


图 5-5　使用自定义坐标轴限值绘制图表

为了得到更好的易理解性，图表通常包含一些标签——例如描述 **x** 和 **y** 值性质的标题和标签。这些标签分别通过 **plt.title, plt.xlabe** 和 **plt.ylabel** 添加。默认情况下，即使提供的数据点是离散的，**plot** 也绘制连续线条。离散点的绘制通过选择不同的样式选项实现。图 5-6 覆盖（红色）点和线宽为 1.5 个点的（蓝色）线条：

```
In [8]: plt.figure(figsize=(7, 4))
          # the figsize parameter defines the
          # size of the figure in (width, height)
        plt.plot(y.cumsum(), 'b', lw=1.5)
        plt.plot(y.cumsum(), 'ro')
        plt.grid(True)
        plt.axis('tight')
        plt.xlabel('index')
        plt.ylabel('value')
        plt.title('A Simple Plot')
```

默认情况下，**plt.plot** 支持表 5-2 中的颜色缩写。

对于线和点的样式，**plt.plot** 支持表 5-3 中列出的字符。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

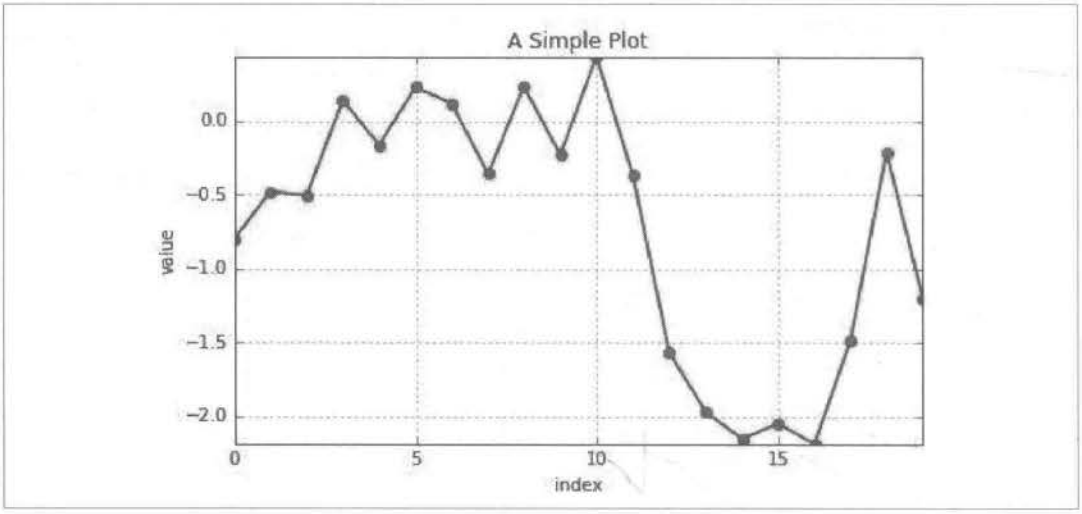


图 5-6　带有典型标签的图表

表 5-2　标准颜色缩写

字符	颜色
b	蓝
g	绿
r	红
c	青
m	品红
y	黄
k	黑
w	白

表 5-3　标准样式字符

字符	象征
-	实线样式
--	短划线样式
-.	点实线样式
:	虚线样式
.	点标记
,	像素标记
o	圆标记
v	向下三角形标记

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

字符	象征
^	向上三角形标记
<	向左三角形标记
>	向右三角形标记
1	Tri_down 标记
2	Tri_up 标记
3	Tri_left 标记
4	Tri_right 标记
s	方形标记
p	五边形标记
*	星号
h	六角形标记 1
H	六角形标记 2
+	加号
x	X 标记
D	菱形标记
d	细菱形标记
|	垂直标记

任何颜色缩写都可以与任何样式字符组合，这样，你可以确保不同的数据集能够轻松区分。我们将会看到，绘图样式也会反映到图例中。

5.1.2 二维数据集

按照一维数据绘图可以看做一种特例。一般来说，数据集包含多个单独的子集。这种数据的处理遵循 **matplotlib** 处理一维数据时的原则。但是，这种情况会出现其他一些问题，例如，两个数据集可能有不同的刻度，无法用相同的 y 或 x 轴刻度绘制。另一个问题是，你可能希望以不同的方式可视化两组不同数据，例如，一组数据使用线图，另一组使用柱状图。

首先，我们生成一个二维样本数据集。下面的代码生成包含标准正态分布（伪）随机数的 20 × 2 NumPy ndarray。在这个数组上调用 **cumsum** 计算样本数据在 0 轴（即第一维）上的总和：

```
In [9]: np.random.seed(2000)
        y = np.random.standard_normal((20, 2)).cumsum(axis=0)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

一般来说，也可以将这样的二维数组传递给 plt.plot。它将自动把包含的数据解释为单独的数据集（沿着 1 轴，即第二维）。对应的图表如图 5-7 所示：

```
In [10]: plt.figure(figsize=(7, 4))
         plt.plot(y, lw=1.5)
           # plots two lines
         plt.plot(y, 'ro')
           # plots two dotted lines
         plt.grid(True)
         plt.axis('tight')
         plt.xlabel('index')
         plt.ylabel('value')
         plt.title('A Simple Plot')
```

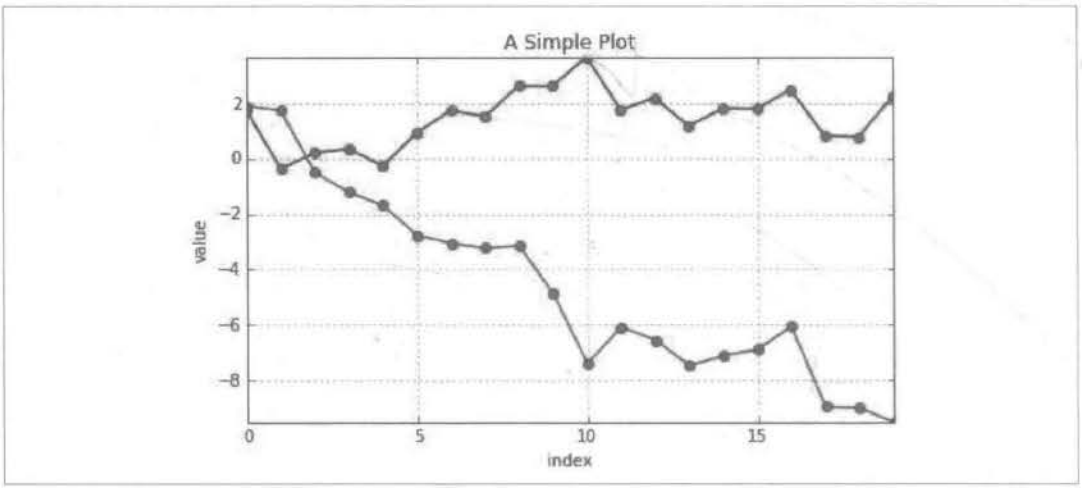


图 5-7　用两个数据集绘制图表

在这种情况下，进一步的注释有助于更好地理解图表，可以为每个数据集添加单独的标签并在图例中列出。**plt.legend** 接受不同的位置参数。0 表示 "*最佳位置*"，也就是图例尽可能少地遮盖数据。图 5-8 展示了包含两个数据集的图表，这一次带有图例。在生成代码中，我们没有传递整个 **ndarray** 对象，而是分别访问两个数据子集（y[:, 0]和 y[:, 0]），可以为它们附加单独的标签：

```
In [11]: plt.figure(figsize=(7, 4))
         plt.plot(y[:, 0], lw=1.5, label='1st')
         plt.plot(y[:, 1], lw=1.5, label='2nd')
         plt.plot(y, 'ro')
         plt.grid(True)
         plt.legend(loc=0)
         plt.axis('tight')
         plt.xlabel('index')
         plt.ylabel('value')
         plt.title('A Simple Plot')
```

plt.legend 的其他位置选项在表 5-4 中列出。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

表 5-4　plt.legend 选项

位置选项	描述
空白	自动
0	最佳
1	右上
2	左上
3	左下
4	右下
5	右
6	中左
7	中右
8	中下
9	中上
10	中

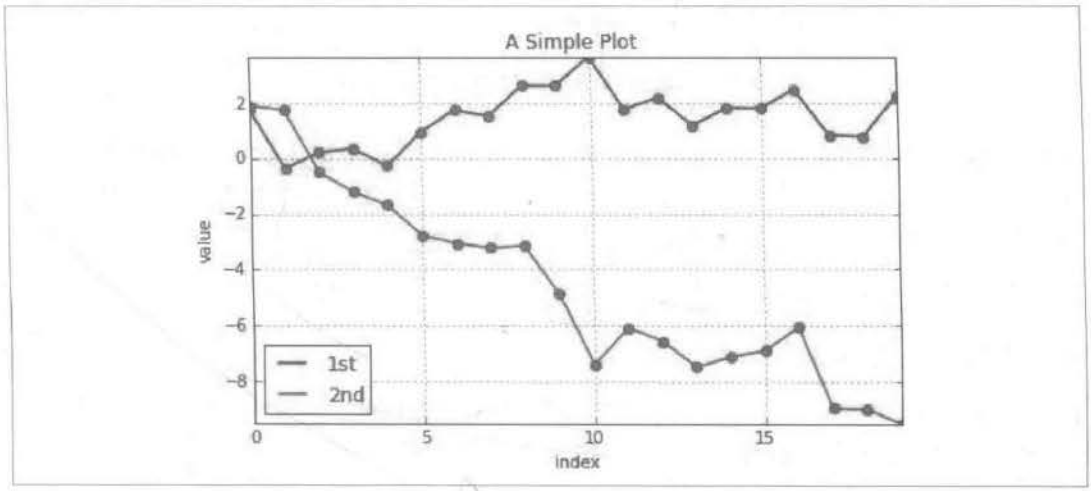


图 5-8　带有数据集的图表

多个具有类似刻度的数据集（如同一金融风险因素的模拟路径）可以用单一的 y 轴绘制。但是，数据集常常有不同的刻度，用单一 y 轴刻度绘制这种数据的图表通常会导致可视化信息的显著丢失。为了说明这种效果，我们将两个数据子集中的第一个扩大 100 倍，再次绘制该图（参见图 5-9）：

```
In [12]: y[:, 0] = y[:, 0] * 100
         plt.figure(figsize=(7, 4))
         plt.plot(y[:, 0], lw=1.5, label='1st')
         plt.plot(y[:, 1], lw=1.5, label='2nd')
         plt.plot(y, 'ro')
         plt.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
plt.legend(loc=0)
plt.axis('tight')
plt.xlabel('index')
plt.ylabel('value')
plt.title('A Simple Plot')
```

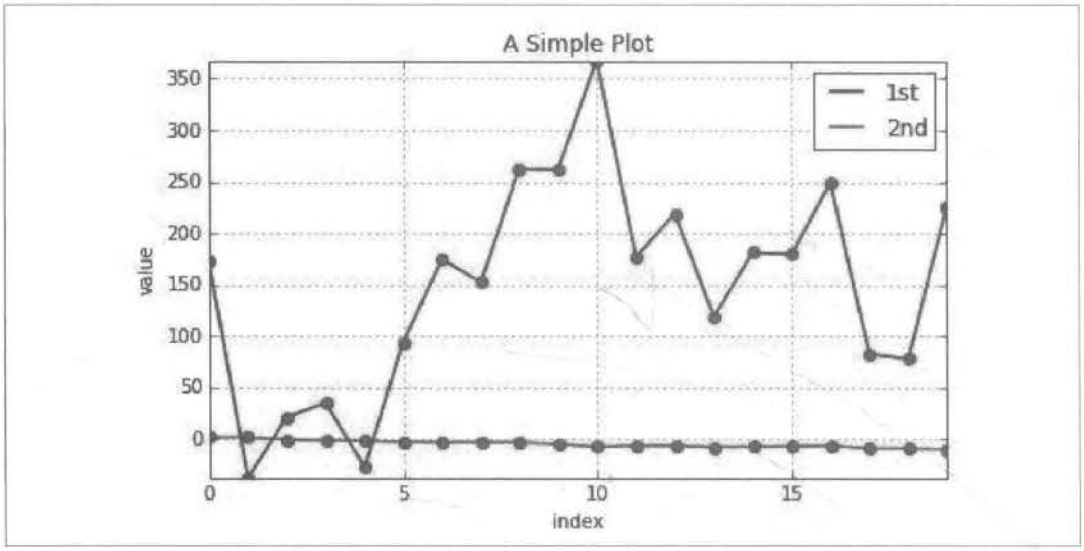


图 5-9 包含两个不同刻度数据集的图表

观察图 5-9 可以看出，第一个数据集仍然是“在视觉上易于辨认的”，而第二个数据集在新的 Y 轴刻度上看起来像一条直线。在某种程度上，第二个数据集的有关信息现在“在视觉上已经丢失”。解决这个问题有两种基本方法：

- 使用 2 个 y 轴（左/右）；
- 使用两个子图（上/下，左/右）。

我们首先在图表中引入第二个 y 轴。图 5-10 中有两个不同的 y 轴，左侧的 y 轴用于第一个数据集，右侧的 y 轴用于第二个数据集，因此，有两个图例：

```
In [13]: fig, ax1 = plt.subplots()
         plt.plot(y[:, 0], 'b', lw=1.5, label='1st')
         plt.plot(y[:, 0], 'ro')
         plt.grid(True)
         plt.legend(loc=8)
         plt.axis('tight')
         plt.xlabel('index')
         plt.ylabel('value 1st')
         plt.title('A Simple Plot')
         ax2 = ax1.twinx()
         plt.plot(y[:, 1], 'g', lw=1.5, label='2nd')
         plt.plot(y[:, 1], 'ro')
         plt.legend(loc=0)
         plt.ylabel('value 2nd')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

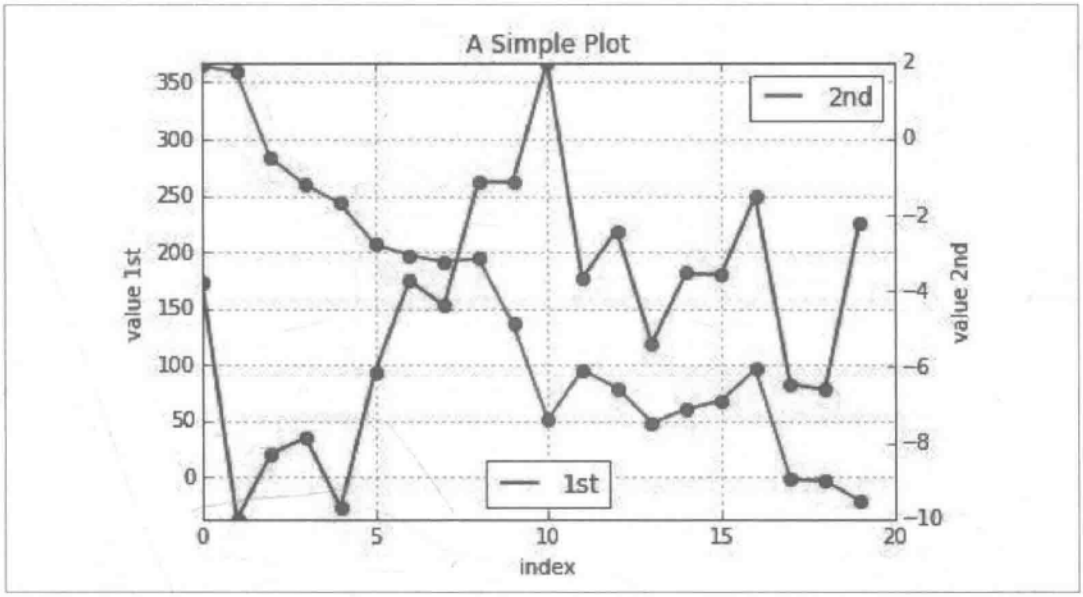


图 5-10　包含两个数据集、两个 y 轴的图表

管理坐标轴的代码行是关键，如：

```
fig, ax1 = plt.subplots()
  # plot first data set using first (left) axis
ax2 = ax1.twinx()
  # plot second data set using second (right) axis
```

通过使用 **plt.subplots** 函数，可以直接访问底层绘图对象（图、子图等）。例如，可以用它生成和第一个子图共享 x 轴的第二个子图。图 5-10 中有两个相互重叠的子图。

接下来，考虑两个单独子图的情况。这个选项提供了处理两个数据集的更大自由度，如图 5-11 所示：

```
In [14]: plt.figure(figsize=(7, 5))
         plt.subplot(211)
         plt.plot(y[:, 0], lw=1.5, label='1st')
         plt.plot(y[:, 0], 'ro')
         plt.grid(True)
         plt.legend(loc=0)
         plt.axis('tight')
         plt.ylabel('value')
         plt.title('A Simple Plot')
         plt.subplot(212)
         plt.plot(y[:, 1], 'g', lw=1.5, label='2nd')
         plt.plot(y[:, 1], 'ro')
         plt.grid(True)
         plt.legend(loc=0)
         plt.axis('tight')
         plt.xlabel('index')
         plt.ylabel('value')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

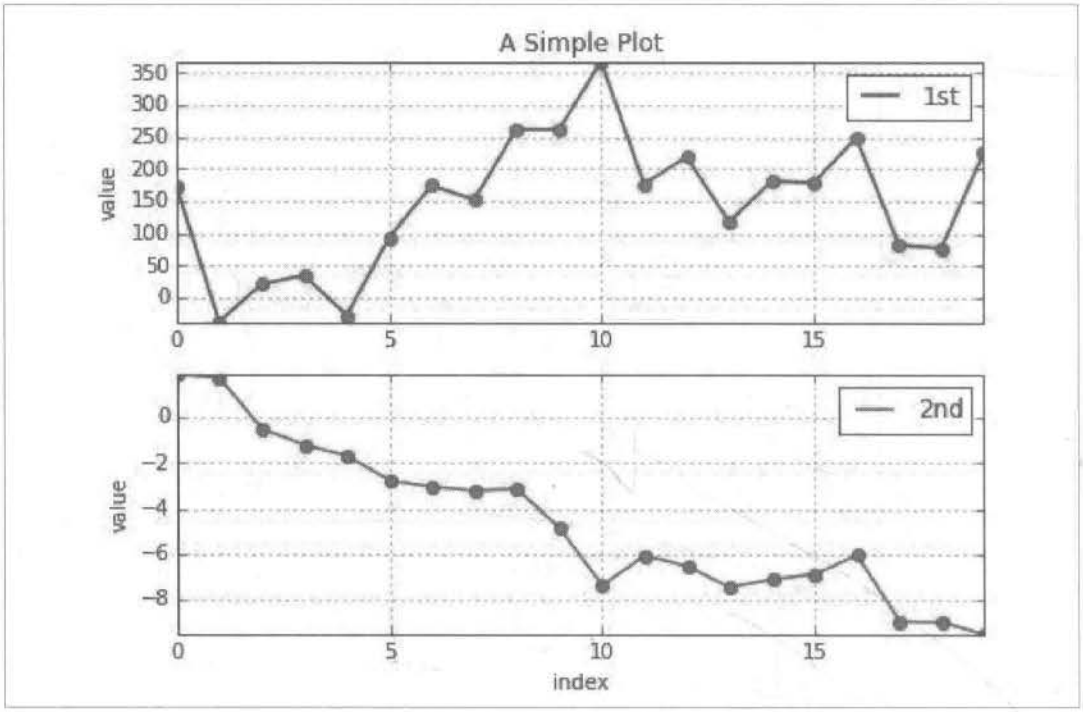


图 5-11　带有两个子图的图表

matplotlib figure 对象中子图的定位通过使用一种特殊的坐标系来实现。**plt.subplot** 有 3 个整数参数，即 **numrows**、**numcols** 和 **fignum**（可能由逗号分隔，也可能没有）。**numrows** 指定*行数*，**numcols** 指定*列数*，**fignum** 指定*子图编号*（从 1 到 **numrows** × **numcols**）。例如，有 9 个大小相同子图的图表有 **numrows=3**，**numcols=3**，**fignum=1**，**2**，…，**9**。左下角的子图“坐标”如下：**plt.subplot(3,3,9)** 。

有时候，选择两个不同的图表类型来可视化数据可能是必要的或者是理想的。利用子图方法，就可以自由地组合 **matplotlib** 提供的任意图表类型[1]。图 5-12 组合了线图/点图和柱状图：

```
In [15]: plt.figure(figsize=(9, 4))
         plt.subplot(121)
         plt.plot(y[:, 0], lw=1.5, label='1st')
         plt.plot(y[:, 0], 'ro')
         plt.grid(True)
         plt.legend(loc=0)
         plt.axis('tight')
         plt.xlabel('index')
         plt.ylabel('value')
         plt.title('1st Data Set')
         plt.subplot(122)
```

1　可用图表类型的概述可以访问 matplotlib 展厅（http://mtplotlib.org/gallery.html）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
plt.bar(np.arange(len(y)), y[:, 1], width=0.5,
        color='g', label='2nd')
plt.grid(True)
plt.legend(loc=0)
plt.axis('tight')
plt.xlabel('index')
plt.title('2nd Data Set')
```

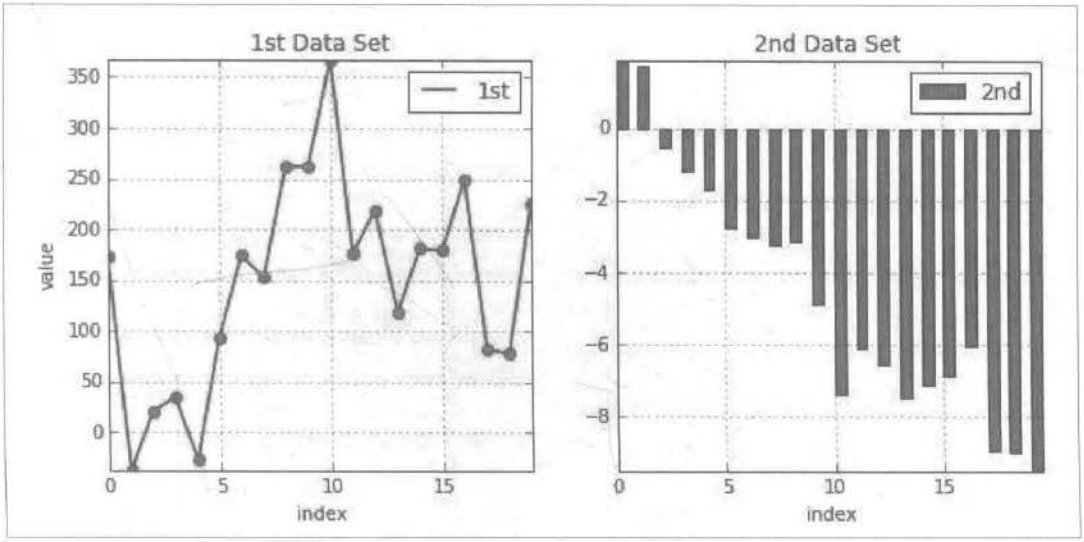


图 5-12　组合线/点子图和柱状子图

5.1.3　其他绘图样式

对于二维绘图，线图和点图可能是金融学中最重要的；这是因为许多数据集用于表示时间序列数据，此类数据通常可以由这些图表进行可视化。第 6 章详细探讨金融时间序列数据。但是，现在我们坚持使用二维数据集并说明一些对金融应用程序有用的替代可视化方法。

我们要介绍的第一种图表是*散点图*，这种图表中一个数据集的值作为其他数据集的 x 值。图 5-13 展示了一个这种图表。例如，这种图表类型可用于绘制一个金融时间序列的收益和另一个时间序列收益的对比。在下面的例子中，我们将使用二维数据集和其他一些数据：

```
In [16]: y = np.random.standard_normal((1000, 2))
In [17]: plt.figure(figsize=(7, 5))
         plt.plot(y[:, 0], y[:, 1], 'ro')
         plt.grid(True)
         plt.xlabel('1st')
         plt.ylabel('2nd')
         plt.title('Scatter Plot')
```

matplotlib 还提供了生成散点图的一个特殊函数。它的工作方式本质上相同，但是提供

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

了一些额外的功能。图 5-14 展示了与图 5-13 对应的散点图，这次使用的是 **scatter** 函数：

```
In [18]: plt.figure(figsize=(7, 5))
         plt.scatter(y[:, 0], y[:, 1], marker='o')
         plt.grid(True)
         plt.xlabel('1st')
         plt.ylabel('2nd')
         plt.title('Scatter Plot')
```

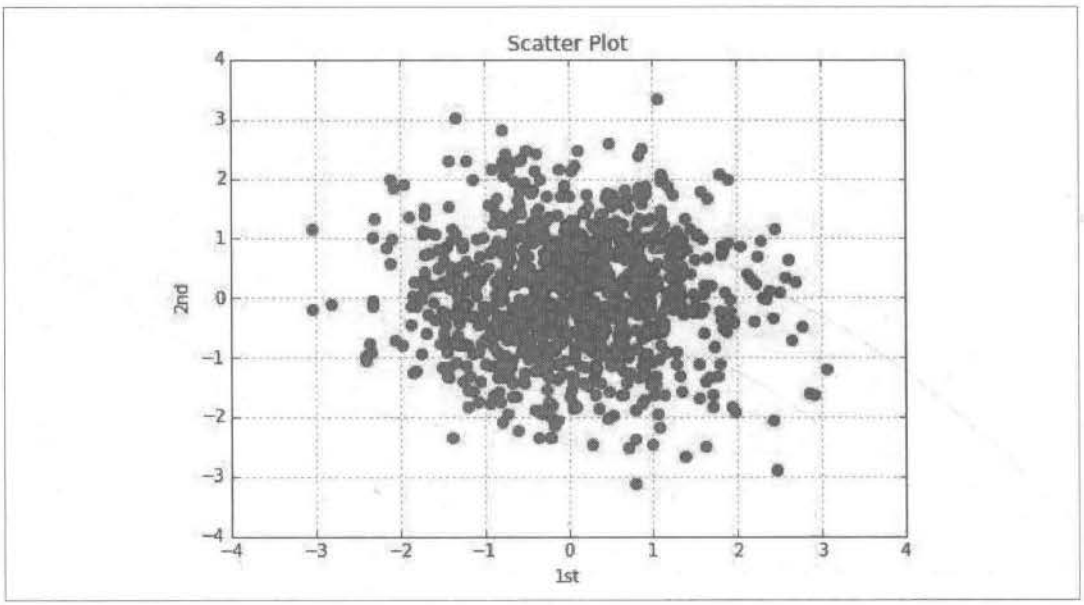


图 5-13　用 plot 函数绘制散点图

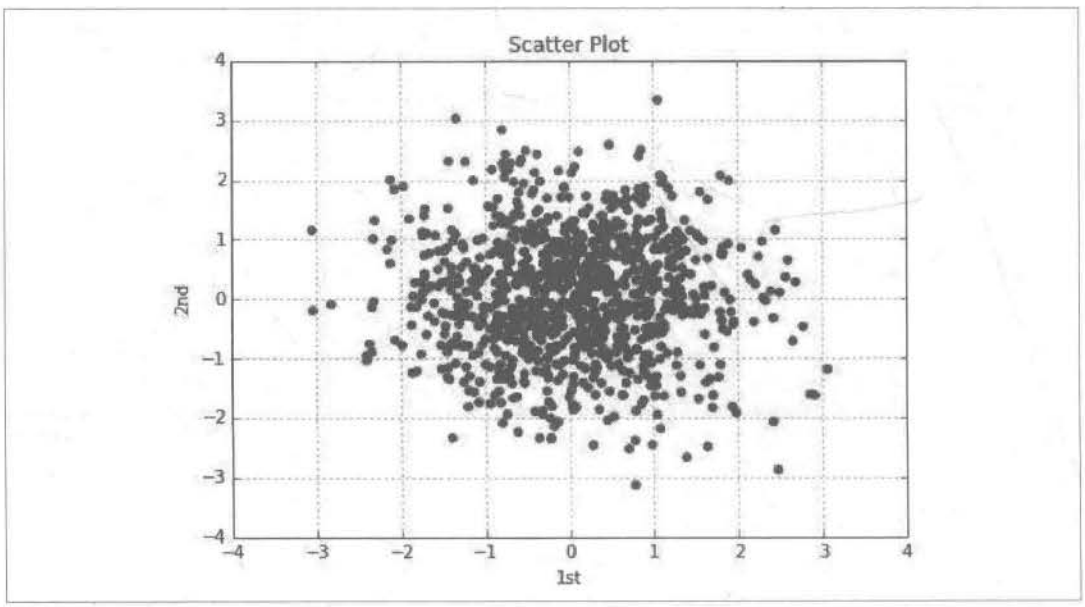


图 5-14　通过 scatter 函数绘制散点图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例如，**scatter** 绘图函数可以加入第三维，通过不同的颜色进行可视化，并使用彩条加以描述。为此，我们用随机数据生成第三个数据集，这次使用的是 0 和 10 之间的整数：

```
In [19]: c = np.random.randint(0, 10, len(y))
```

图 5-15 展示的散点图有不同颜色小点表示的第三维，还有作为颜色图例的彩条：

```
In [20]: plt.figure(figsize=(7, 5))
         plt.scatter(y[:, 0], y[:, 1], c=c, marker='o')
         plt.colorbar()
         plt.grid(True)
         plt.xlabel('1st')
         plt.ylabel('2nd')
         plt.title('Scatter Plot')
```

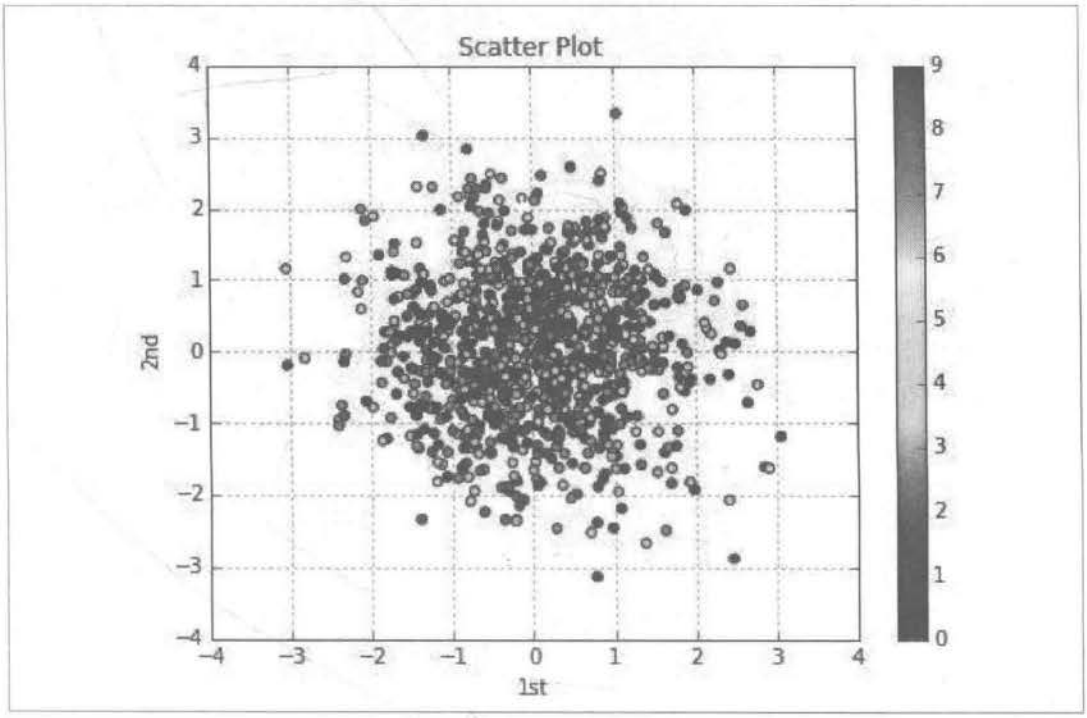


图 5-15　具备第三维的散点图

另一种图表类型*直方图*也常常用于金融收益中。图 5-16 在同一个图表中放置两个数据集的频率值：

```
In [21]: plt.figure(figsize=(7, 4))
         plt.hist(y, label=['1st', '2nd'], bins=25)
         plt.grid(True)
         plt.legend(loc=0)
         plt.xlabel('value')
         plt.ylabel('frequency')
         plt.title('Histogram')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

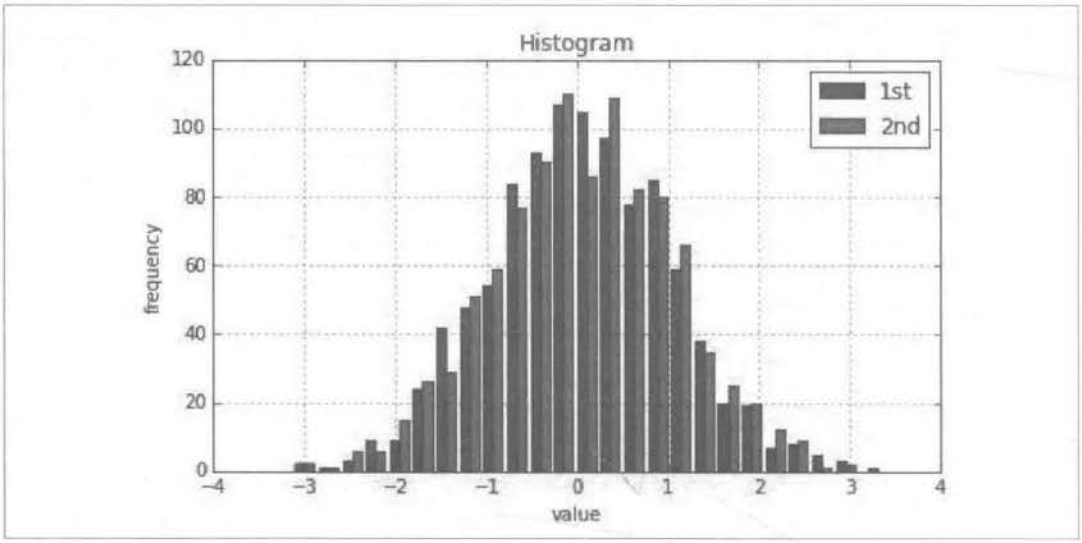


图 5-16　两个数据集的直方图

由于直方图是金融应用中的重要图表类型，我们要更认真地观察 **plt.hist** 的使用方法。下面的例子说明了所支持的参数：

```
plt.hist(x, bins=10, range=None, normed=False, weights=None, cumulative=False,
bottom=None, histtype='bar', align='mid', orientation='vertical', rwidth=None,
log=False, color=None, label=None, stacked=False, hold=None, **kwargs)
```

表 5-5 提供了 plt.hist 函数主要参数的描述。

表 5-5　plt.hist 参数

参数	描述
x	列表对象，ndarray 对象
bins	数据组（bin）数
range	数据组的下界和上界
normed	规范化为整数 1
weights	x 轴上每个值的权重
cumulative	每个数据组包含较低组别的计数
histtype	选项（字符串）：bar, barstacked, step, stepfilled
align	选项（字符串）：left, mid, right
orientation	选项（字符串）：horizontal, vertical
rwidth	条块的相对宽度
log	对数刻度
color	每个数据集的颜色（类似数组）
label	标签所用的字符串或者字符串序列
stacked	堆叠多个数据集

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 5-17 展示了一个类似的图表；这次，两个数据集的数据在直方图中堆叠：

```
In [22]: plt.figure(figsize=(7, 4))
         plt.hist(y, label=['1st', '2nd'], color=['b', 'g'],
                  stacked=True, bins=20)
         plt.grid(True)
         plt.legend(loc=0)
         plt.xlabel('value')
         plt.ylabel('frequency')
         plt.title('Histogram')
```

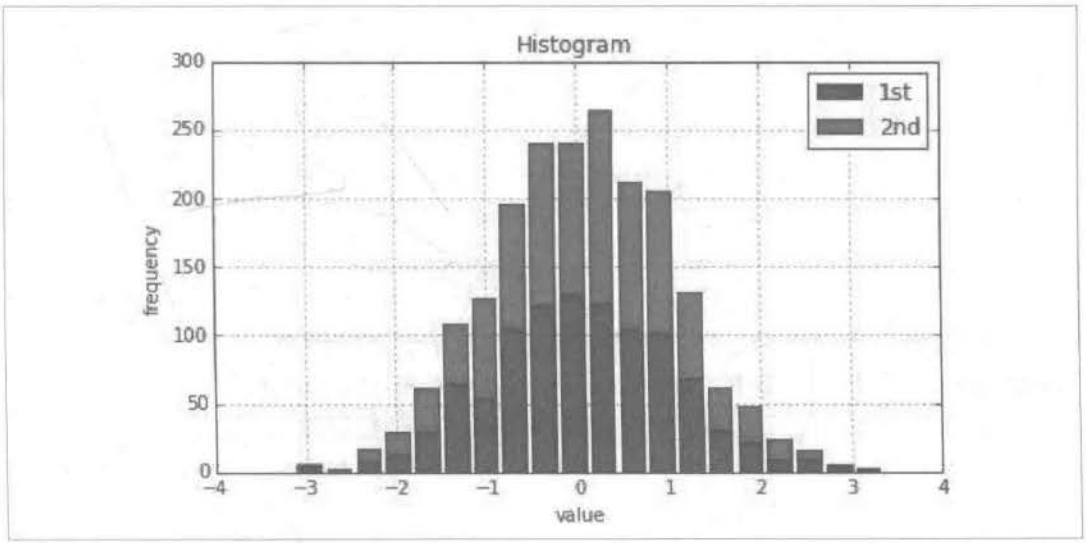


图 5-17　两个数据集堆叠的直方图

另一种实用的图表类型是*箱形图*。和直方图类似，箱形图可以简洁地概述数据集的特性，很容易比较多个数据集。图 5-18 展示了按照我们的数据集绘制的这类图表：

```
In [23]: fig, ax = plt.subplots(figsize=(7, 4))
         plt.boxplot(y)
         plt.grid(True)
         plt.setp(ax, xticklabels=['1st', '2nd'])
         plt.xlabel('data set')
         plt.ylabel('value')
         plt.title('Boxplot')
```

最后一个例子使用 **plt.setp** 函数，该函数设置（一组）图表实例的属性。例如，考虑下列代码生成的线图：

```
line = plt.plot(data, 'r')
```

使用如下的代码：

```
plt.setp(line, linestyle='-')
```

将线的样式更改为“短划线”。这样，可以轻松地在图表实例（“艺术家对象”）生成之后改变参数。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

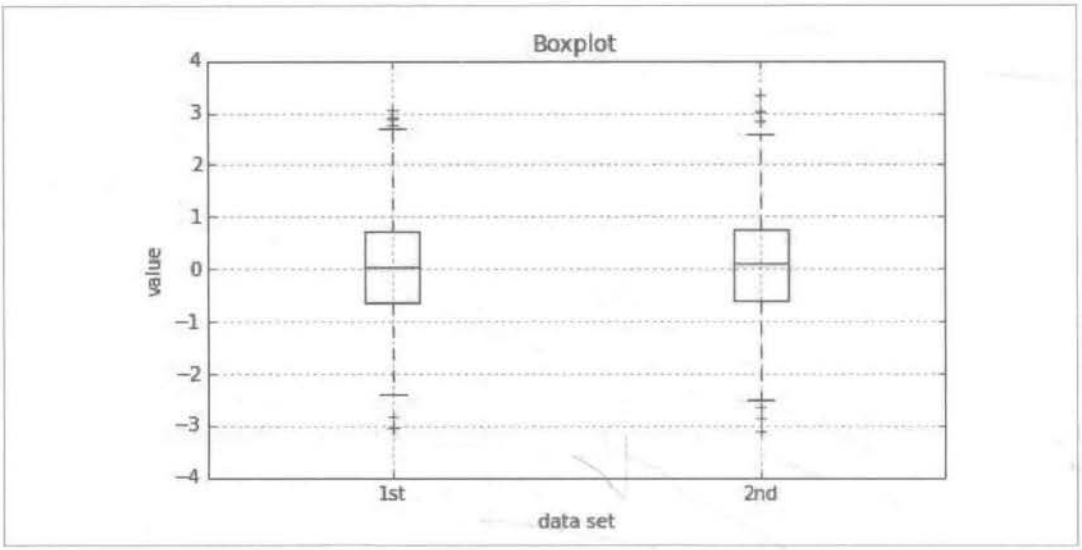


图 5-18　两个数据集的箱形图

在本节的最后一个例证中，我们考虑一个受到数学启迪的图表，这个例子也可以在 **matplotlib** 的“展厅”中找到（http://www/matplotlib.org/gallery.html）。它绘制一个函数的图像，并以图形的方式说明了某个下限和上限之间函数图像下方区域的面积——换言之，从下限到上限之间的函数积分值。图 5-19 展示了结果图表，说明 **matplotlib** 能够无缝地处理 LaTeX 字体设置，在图表中加入数学公式：

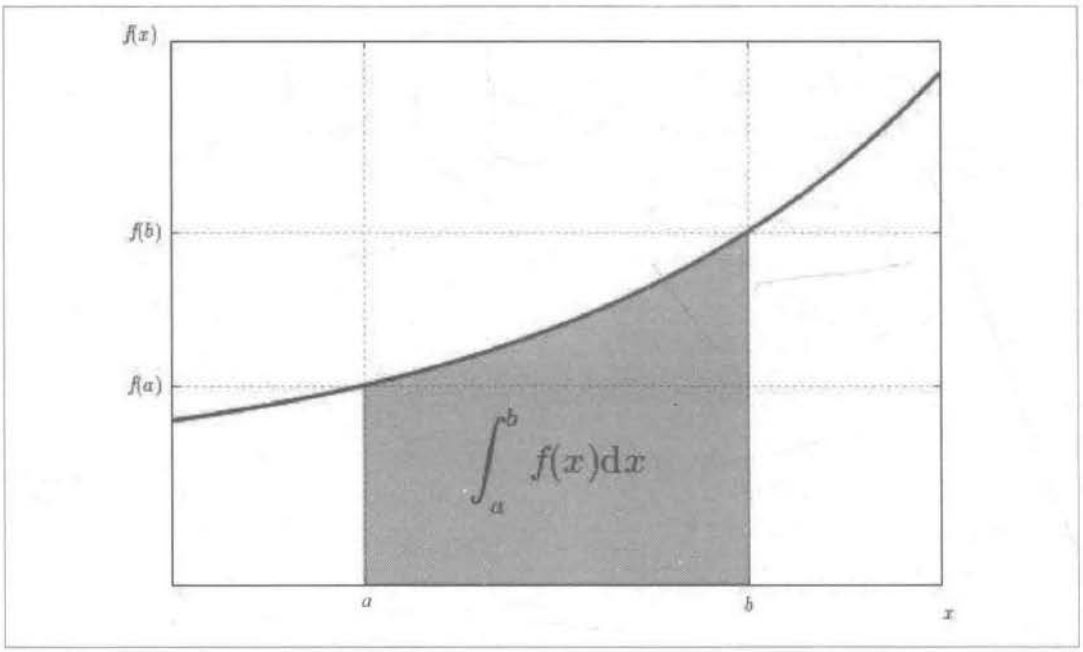


图 5-19　指数函数、积分面积和 LaTeX 标签

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [24]: from matplotlib.patches import Polygon
         def func(x):
             return 0.5 * np.exp(x) + 1

         a, b = 0.5, 1.5  # integral limits
         x = np.linspace(0, 2)
         y = func(x)

         fig, ax = plt.subplots(figsize=(7, 5))
         plt.plot(x, y, 'b', linewidth=2)
         plt.ylim(ymin=0)

         # Illustrate the integral value, i.e. the area under the function
         # between the lower and upper limits
         Ix = np.linspace(a, b)
         Iy = func(Ix)
         verts = [(a, 0)] + list(zip(Ix, Iy)) + [(b, 0)]
         poly = Polygon(verts, facecolor='0.7', edgecolor='0.5')
         ax.add_patch(poly)

         plt.text(0.5 * (a + b), 1, r"$\int_a^b f(x)\mathrm{d}x$",
                  horizontalalignment='center', fontsize=20)

         plt.figtext(0.9, 0.075, '$x$')
         plt.figtext(0.075, 0.9, '$f(x)$')

         ax.set_xticks((a, b))
         ax.set_xticklabels(('$a$', '$b$'))
         ax.set_yticks([func(a), func(b)])
         ax.set_yticklabels(('$f(a)$', '$f(b)$'))
         plt.grid(True)
```

下面我们一步一步地完成这个图表的生成。第一步是定义需要求取积分的函数：

```
def func(x):
    return 0.5 * np.exp(x) + 1
```

第二步是定义积分区间，生成必需的数值：

```
a, b = 0.5, 1.5  # integral limits
x = np.linspace(0, 2)
y = func(x)
```

第三，绘制函数图形：

```
fig, ax = plt.subplots(figsize=(7, 5))
plt.plot(x, y, 'b', linewidth=2)
plt.ylim(ymin=0)
```

第四步是核心，我们使用 Polygon 函数生成阴影部分（“补丁”），表示积分面积：

```
Ix = np.linspace(a, b)
Iy = func(Ix)
verts = [(a, 0)] + list(zip(Ix, Iy)) + [(b, 0)]
poly = Polygon(verts, facecolor='0.7', edgecolor='0.5')
ax.add_patch(poly)
```

第五步是用 **plt.text** 和 **plt.figtext** 在图表上添加数学公式和一些坐标轴标签。LaTeX 代

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

码在两个美元符号之间传递（$ … $）。两个函数的前两个参数都是放置对应文本的坐标值：

```
plt.text(0.5 * (a + b), 1, r"$\int_a^b f(x)\mathrm{d}x$",
         horizontalalignment='center', fontsize=20)

plt.figtext(0.9, 0.075, '$x$')
plt.figtext(0.075, 0.9, '$f(x)$')
```

最后，我们分别设置 x 和 y 刻度标签的位置。注意，尽管我们以 LaTeX 渲染变量名称，但是用于定位的是正确的数字值。我们还添加了网格，在这个特殊例子中，只是为了强调选中的刻度：

```
ax.set_xticks((a, b))
ax.set_xticklabels(('$a$', '$b$'))
ax.set_yticks([func(a), func(b)])
ax.set_yticklabels(('$f(a)$', '$f(b)$'))
plt.grid(True)
```

5.2 金融学图表

matplotlib 还提供了少数精选的特殊金融图表。这些图表（*如烛柱图*）主要用于可视化历史股价数据或者类似的金融时间序列数据，可以在 **matplotlib.finance** 子库中找到：

```
In [25]: import matplotlib.finance as mpf
```

作为一个方便的函数，该子库可以简单地从雅虎财经网站（http://finance.yahoo.com）读取历史股价数据。你所需要的只是起始和结束日期，以及对应的股票代码。下面的代码读取德国 DAX 指数的数据，其股票代码为^GDAXI：

```
In [26]: start = (2014, 5, 1)
         end = (2014, 6, 30)

         quotes = mpf.quotes_historical_yahoo('^GDAXI', start, end)
```

Web 来源的数据质量

现在，有几个 Python 库都提供了方便的函数，可以从雅虎财经读取数据。但是要注意，尽管这是可视化财经数据集的方便手段，但是数据质量不足以作为任何重要投资决策的基础。例如，拆股会造成“价格大跌”，雅虎财经中的数据往往没有正确地考虑这一点。其他免费的数据源也有类似的情况。

quotes 现在包含了 DAX 指数的时间序列数据，包括**日期**（采用纪元时间格式）、**开盘价**、**最高价**、**最低价**、**收盘价**和**成交量**：

```
In [27]: quotes[:2]
Out[27]: [(735355.0,
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
 9611.7900000000009,
 9556.0200000000004,
 9627.3799999999992,
 9533.2999999999993,
 88062300.0),
(735358.0,
 9536.3799999999992,
 9529.5,
 9548.1700000000001,
 9407.0900000000001,
 61911600.0)]
```

matplotlib.finance 的绘图函数准确地理解可能传递的格式和数据集，例如，传递给 **candlestick** 函数的数据。图 5-20 展示了结果。每日的*正收益*由*蓝色*的矩形表示，*负收益*由*红色*矩形表示。你可能会注意到，**matplotlib** 根据数据集中的日期信息，为 x 轴设置正确的标签：

```
In [28]: fig, ax = plt.subplots(figsize=(8, 5))
         fig.subplots_adjust(bottom=0.2)
         mpf.candlestick(ax, quotes, width=0.6, colorup='b', colordown='r')
         plt.grid(True)
         ax.xaxis_date()
           # dates on the x-axis
         ax.autoscale_view()
         plt.setp(plt.gca().get_xticklabels(), rotation=30)
```

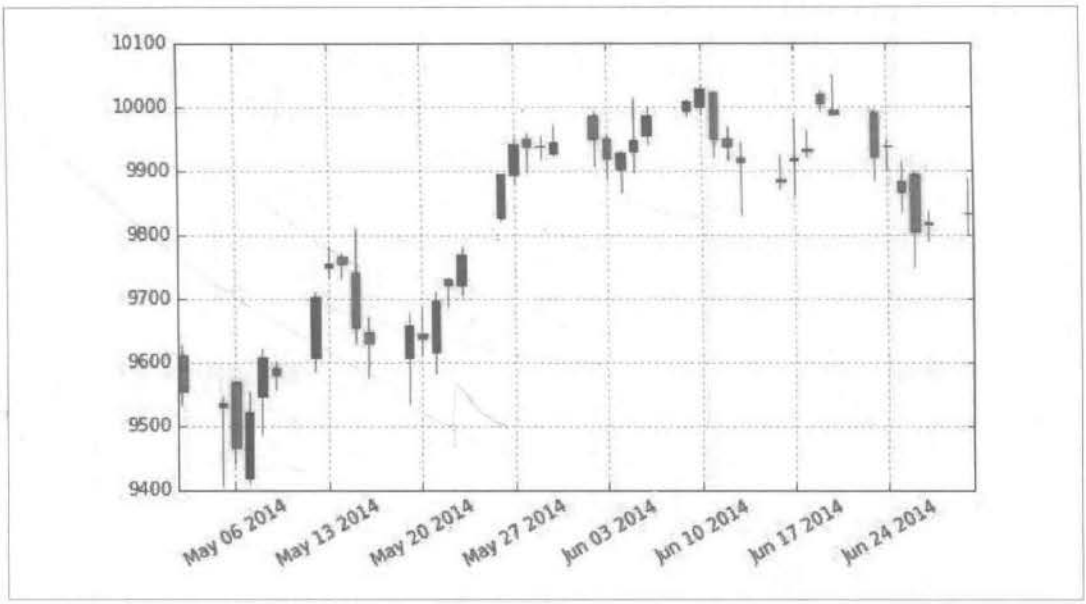


图 5-20　金融数据的烛柱图

在上述代码中，**plt.setp(plt.gca().get_xticklabels(), rotation=30)**将 x 轴标签旋转 30 度。为此使用了 **plt.gca** 函数，该函数返回当前 **figure** 对象。然后，调用 **get_xticklabels** 方法提供图形的 x 轴刻度标签。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

表 5-6 提供了 **mpf.candlestick** 函数不同参数的说明。

表 5-6　mpf.candlestick 参数

参数	描述
ax	绘图使用的 Axes 实例
quotes	绘图使用的金融数据（时间、开盘价、收盘价、最高价、最低价序列）
width	矩形宽度代表的天数
colorup	收盘价≥开盘价时的矩形颜色
colordown	收盘价＜开盘价时的矩形颜色
alpha	矩形的 Alpha 级别

plot_day_summary 函数提供了一个相当类似的图表类型，使用方法和 **candlestick** 函数相同，使用类似的参数。这里，开盘价和收盘价不是由彩色矩形表示，而是由两条短水平线表示，如图 5-21 所示：

```
In [29]: fig, ax = plt.subplots(figsize=(8, 5))
         mpf.plot_day_summary(ax, quotes, colorup='b', colordown='r')
         plt.grid(True)
         ax.xaxis_date()
         plt.title('DAX Index')
         plt.ylabel('index level')
         plt.setp(plt.gca().get_xticklabels(), rotation=30)
```

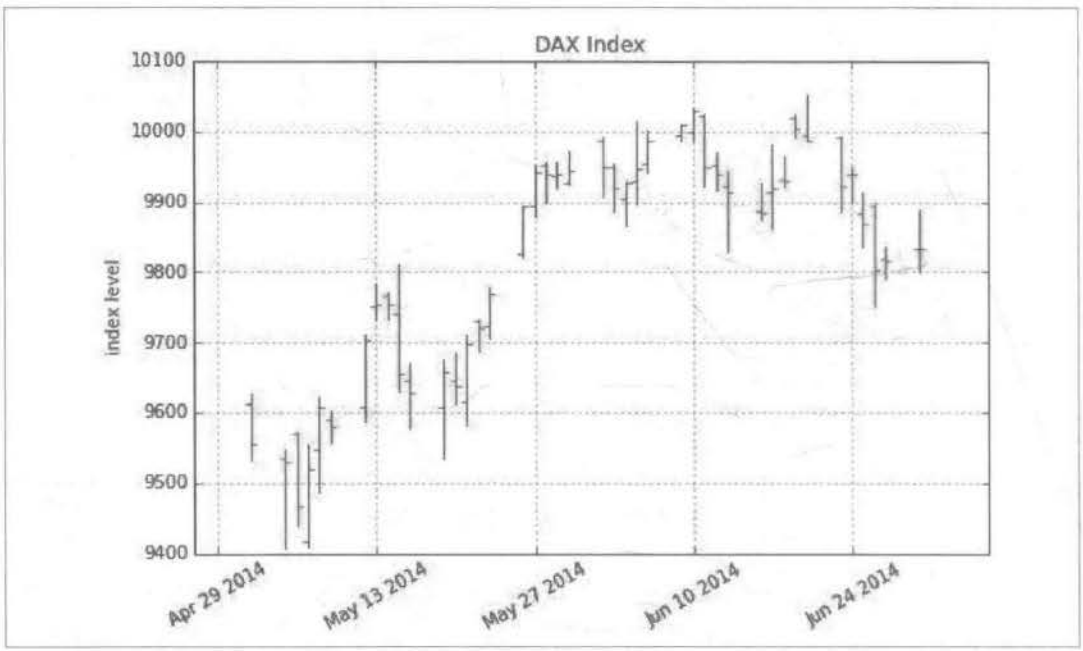


图 5-21　金融数据每日摘要图表

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

股价数据往往和成交量数据结合，在一张图表中同时提供市场活动的相关信息。下面的代码说明这种基于雅虎公司股价历史数据的用例，结果如图 5-22 所示：

```
In [30]: quotes = np.array(mpf.quotes_historical_yahoo('YHOO', start, end))
In [31]: fig, (ax1, ax2) = plt.subplots(2, sharex=True, figsize=(8, 6))
         mpf.candlestick(ax1, quotes, width=0.6, colorup='b', colordown='r')
         ax1.set_title('Yahoo Inc.')
         ax1.set_ylabel('index level')
         ax1.grid(True)
         ax1.xaxis_date()
         plt.bar(quotes[:, 0] - 0.25, quotes[:, 5], width=0.5)
         ax2.set_ylabel('volume')
         ax2.grid(True)
         ax2.autoscale_view()
         plt.setp(plt.gca().get_xticklabels(), rotation=30)
```

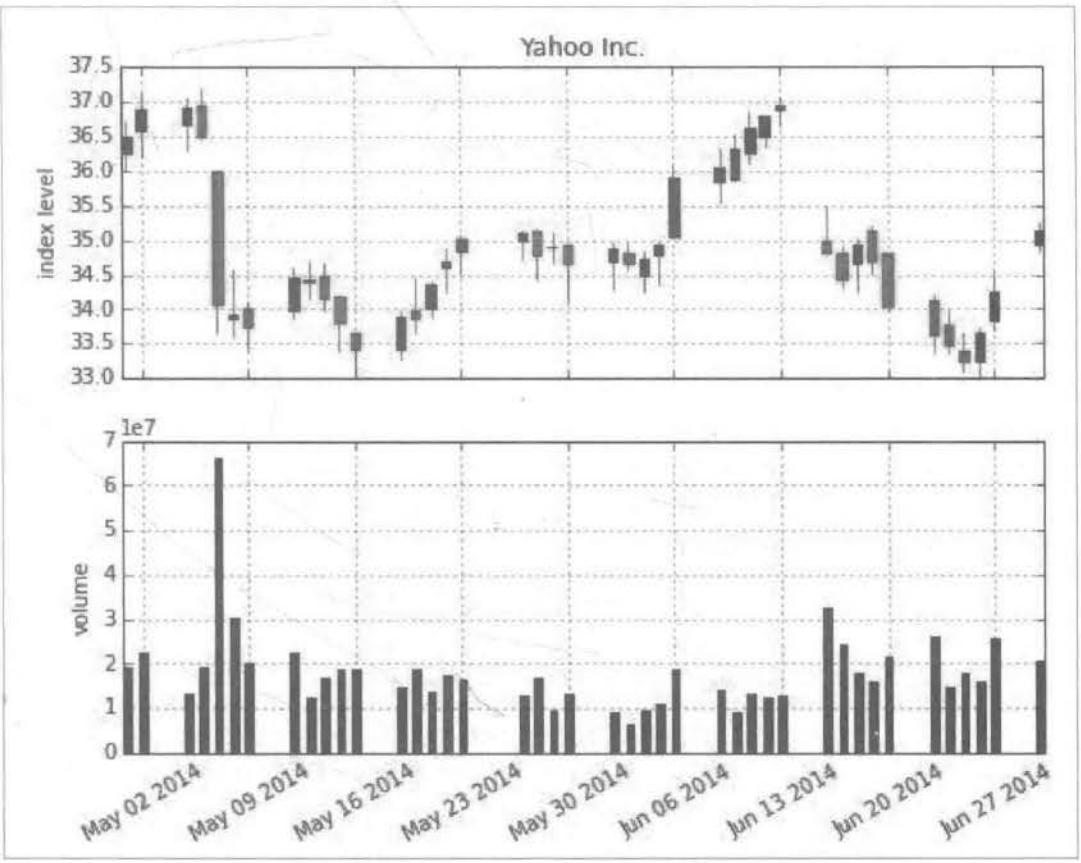


图 5-22　烛柱图和成交量柱状图组合而成的图表

5.3　3D 绘图

金融中从 3 维可视化中获益的领域不是太多。但是，波动率平面是一个应用领域，它

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

可以同时展示许多到期日和行权价的隐含波动率。在下面的例子中，我们人为生成一个类似波动率平面的图表。为此，我们考虑如下因素：

1. 行权价格在 50 ~ 150 元之间；
2. 到期日在 0.5 ~ 2.5 年之间。

这为我们提供了一个 2 维坐标系。我们可以使用 **NumPy** 的 **meshgrid** 函数，根据两个 1 维 **ndarray** 对象生成这样的坐标系：

```
In [32]: strike = np.linspace(50, 150, 24)
         ttm = np.linspace(0.5, 2.5, 24)
         strike, ttm = np.meshgrid(strike, ttm)
```

上述代码将两个 1 维数组转换为 2 维数组，在必要时重复原始坐标轴值：

```
In [33]: strike[:2]
Out[33]: array([[  50.        ,   54.34782609,   58.69565217,   63.04347826,
                   67.39130435,   71.73913043,   76.08695652,   80.43478261,
                   84.7826087 ,   89.13043478,   93.47826087,   97.82608696,
                  102.17391304,  106.52173913,  110.86956522,  115.2173913 ,
                  119.56521739,  123.91304348,  128.26086957,  132.60869565,
                  136.95652174,  141.30434783,  145.65217391,  150.        ],
                [  50.        ,   54.34782609,   58.69565217,   63.04347826,
                   67.39130435,   71.73913043,   76.08695652,   80.43478261,
                   84.7826087 ,   89.13043478,   93.47826087,   97.82608696,
                  102.17391304,  106.52173913,  110.86956522,  115.2173913 ,
                  119.56521739,  123.91304348,  128.26086957,  132.60869565,
                  136.95652174,  141.30434783,  145.65217391,  150.        ]])
```

现在，根据新的 **ndarray** 对象，我们通过简单的比例调整二次函数生成模拟的*隐含波动率*：

```
In [34]: iv = (strike - 100) ** 2 / (100 * strike) / ttm
             # generate fake implied volatilities
```

下面的代码得出图 5-23 中的图表：

```
In [35]: from mpl_toolkits.mplot3d import Axes3D

         fig = plt.figure(figsize=(9, 6))
         ax = fig.gca(projection='3d')

         surf = ax.plot_surface(strike, ttm, iv, rstride=2, cstride=2,
                                cmap=plt.cm.coolwarm, linewidth=0.5,
                                antialiased=True)

         ax.set_xlabel('strike')
         ax.set_ylabel('time-to-maturity')
         ax.set_zlabel('implied volatility')

         fig.colorbar(surf, shrink=0.5, aspect=5)
```

表 5-7 提供了 **plot_surface** 函数使用的不同参数的描述。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

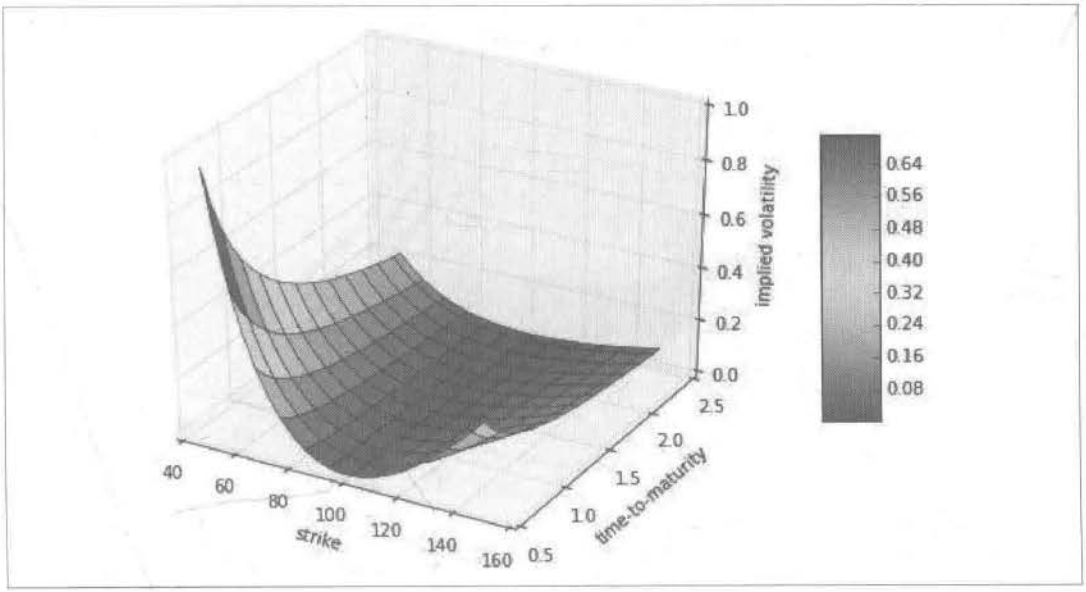


图 5-23 （模拟）隐含波动率的 3D 曲面图

表 5-7 plot_surface 参数

参数	描述
X, Y, Z	2D 数组形式的数据值
rstride	数组行距（步长大小）
cstride	数组列距（步长大小）
color	曲面块颜色
cmap	曲面块颜色映射
facecolors	单独曲面块表面颜色
norm	将值映射为颜色的 Normalize 实例
vmin	映射的最小值
vmax	映射的最大值

和 2 维图表一样，线样式可以由单个点或者下例中的单个三角形表示。图 5-24 用相同的数据绘制 3D 散点图，但是现在用 **view_init** 函数设置不同的视角：

```
In [36]: fig = plt.figure(figsize=(8, 5))
         ax = fig.add_subplot(111, projection='3d')
         ax.view_init(30, 60)

         ax.scatter(strike, ttm, iv, zdir='z', s=25,
                    c='b', marker='^')

         ax.set_xlabel('strike')
         ax.set_ylabel('time-to-maturity')
         ax.set_zlabel('implied volatility')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

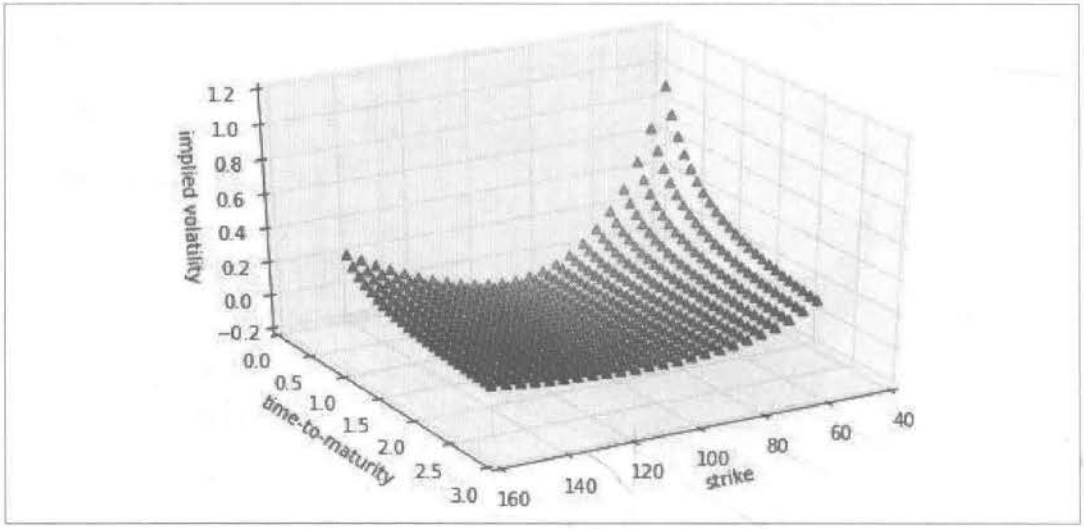


图 5-24 （模拟）隐含波动率的 3D 散点图

5.4 结语

matplotlib 可以视为 **Python** 数据可视化的基准和主力，它与 **NumPy** 紧密集成，基本功能可以轻松而方便地访问。但是，另一方面，**matplotlib** 是相当强大的库，具有复杂的 API，在本章中无法广泛地概述 **matplotlib** 的所有功能。

本章介绍了 **matplotlib** 可用于大部分金融环境下的基本 2D 和 3D 绘图函数。其他章节提供使用这个可视化基本库的进一步示例。

5.5 延伸阅读

matplotlib 的主要资源可以在下面的网站上找到。

- **matplotlib** 主页当然是最好的起点：http://matplotlib.org。
- **matplotlib** 网站上有一个展厅，其中有许多实用示例：http://matplotlib.org/gallery.html。
- 这里可以找到一个 2D 绘图的教程：http://matplotlib.org/users/pyplot_tutorial.html。
- 这里是 3D 绘图教程：http://matplotlib.org/mpl_toolkits/mplot3d/tutorial.html。

访问展厅已经成为某种标准规程，在那里可以看到相应的可视化示例，从对应的示例代码开始工作。例如，使用 IPython Notebook，当你找到合适的示例，只需要一个命令就可以开始了。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第6章

金融时间序列

时间存在的唯一价值就是证明所有事情都不曾发生过。

——阿尔伯特•爱因斯坦

在金融学中遇到的最重要数据类型之一是*金融时间序列*。这是以日期和/或时间作为索引的数据。例如，股价就表现为金融时间序列数据。类似地，美元-欧元汇率也是金融时间序列；汇率在短暂的时间间隔内报价，一组此类报价就是汇率的时间序列。

没有一种金融学科不将时间作为重要因素考虑，这和物理学及其他科学相同。**Python** 中处理时间序列的主要工具是 **pandas** 库。**pandas** 的主要作者 Wes McKinney 在大型对冲基金 AQR 资本管理公司任分析师时开始开发这个库。正如本章所阐述的，**DataFrame** 和 **Series** 等基本类的灵感来自于统计分析语言 R，该语言无疑长于这类建模和分析工作。

本章主要基于几个来自金融环境的例子。这些例子按照如下的步骤推进：

第一和第二步

我们使用非常简单和小型的数据集开始探索 **pandas** 的功能；然后使用 **NumPy ndarray** 对象并将其转换为 **DataFame** 对象。在此过程中，介绍基本的分析和可视化功能。

来自 Web 的数据

pandas 允许我们方便地从 Web 上读取数据——例如从雅虎财经（http://finance.yahoo.com）——并以许多种方式分析这些数据。

使用来自 CSV 文件的数据

逗号分隔值（CSV）文件是交换金融时间序列数据的全球标准之一；**pandas** 可以

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

高效地从这些文件中读取数据。使用两种指数的数据，我们利用 **pandas** 实施一次回归分析。

高频数据

今年来，可用的金融时间序列数据越来越多地从每日报价转向分时数据。每日分时股价数据量通常超过 30 年间收集的每日报价数据量[1]。

按照定义，所有金融时间序列数据都包含日期和/或时间信息。附录 C 提供了用 **Python**、**NumPy** 和 **pandas** 处理这种数据的概述，以及如何将典型的日期-时间对象类型转换为另一种类型。

6.1 pandas 基础

在某种意义上，**pandas** 是在 **NumPy** 基础上构建的。例如，**NumPy** 通用函数通常也可以在 **pandas** 对象上工作，因此，我们首先导入这两个库：

```
In [1]: import numpy as np
        import pandas as pd
```

6.1.1 使用 DataFrame 类的第一步

从最根本的层面上看，**DataFrame** 类设计用来管理具有索引和标签的数据，这些数据与来自 **SQL** 数据表或者电子表格应用中的工作表内的数据没有太多的不同。考虑如下代码创建的 **DataFrame** 对象：

```
In [2]: df = pd.DataFrame([10, 20, 30, 40], columns=['numbers'],
                          index=['a', 'b', 'c', 'd'])
        df
Out[2]:    numbers
        a       10
        b       20
        c       30
        d       40
```

这个简单的例子已经说明了 **DataFrame** 在存储数据上的主要特性：

数据

数据本身可以用不同组成及类型提供（**列表**、**元组**、**ndarray** 和**字典对象**都是候选者）。

标签

数据组织为列，可以自定义列名。

1 仅考虑每日收盘价，单支股票 30 年间的收盘价数量大约为 30×252=7560。单支股票每日分时报价（买入价/卖出价）超过 10000 个的情况并不少见。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

索引

索引可以采用不同的格式（例如数值、字符串、时间信息）。

使用这种 **DataFrame** 对象总体上相当方便和高效，例如，当你想要进行扩大现有对象等工作时，相比之下，常规的 **ndarray** 对象更专门化，也更受限制。下面是简单的例子，说明 **DataFrame** 对象上典型操作的工作方式：

```
In [3]: df.index  # the index values
Out[3]: Index([u'a', u'b', u'c', u'd'], dtype='object')
In [4]: df.columns  # the column names
Out[4]: Index([u'numbers'], dtype='object')
In [5]: df.ix['c']  # selection via index
Out[5]: numbers    30
        Name: c, dtype: int64
In [6]: df.ix[['a', 'd']]  # selection of multiple indices
Out[6]:    numbers
        a       10
        d       40
In [7]: df.ix[df.index[1:3]]  # selection via Index object
Out[7]:    numbers
        b       20
        c       30
In [8]: df.sum()  # sum per column
Out[8]: numbers    100
        dtype: int64
In [9]: df.apply(lambda x: x ** 2)  # square of every element
Out[9]:    numbers
        a      100
        b      400
        c      900
        d     1600
```

一般来说，可以在 DataFrame 对象上进行和 NumPy ndarray 对象相同的向量化操作：

```
In [10]: df ** 2  # again square, this time NumPy-like
Out[10]:    numbers
         a      100
         b      400
         c      900
         d     1600
```

在两个维度上同时扩增 DataFrame 对象是可能的：

```
In [11]: df['floats'] = (1.5, 2.5, 3.5, 4.5)
           # new column is generated
         df
Out[11]:    numbers  floats
         a       10     1.5
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
b       20     2.5
c       30     3.5
d       40     4.5
```

```
In [12]: df['floats']  # selection of column
```

```
Out[12]: a    1.5
         b    2.5
         c    3.5
         d    4.5
         Name: floats, dtype: float64
```

也可以取整个 **DataFrame** 对象来定义一个新列。在这种情况下，索引自动分配：

```
In [13]: df['names'] = pd.DataFrame(['Yves', 'Guido', 'Felix', 'Francesc'],
                                    index=['d', 'a', 'b', 'c'])
         df
```

```
Out[13]:    numbers  floats     names
         a       10     1.5     Guido
         b       20     2.5     Felix
         c       30     3.5  Francesc
         d       40     4.5      Yves
```

附加数据的方法也类似。但是，在下面的例子中，我们会看到通常必须避免的一个副作用——索引被简单的编号索引代替：

```
In [14]: df.append({'numbers': 100, 'floats': 5.75, 'names': 'Henry'},
                   ignore_index=True)
           # temporary object; df not changed
```

```
Out[14]:    numbers  floats     names
         0       10    1.50     Guido
         1       20    2.50     Felix
         2       30    3.50  Francesc
         3       40    4.50      Yves
         4      100    5.75     Henry
```

附加一个 DataFrame 对象以提供正确的索引信息往往是更好的做法。下面的例子保留这种索引：

```
In [15]: df = df.append(pd.DataFrame({'numbers': 100, 'floats': 5.75,
                                      'names': 'Henry'}, index=['z',]))
         df
```

```
Out[15]:    floats     names  numbers
         a    1.50     Guido       10
         b    2.50     Felix       20
         c    3.50  Francesc       30
         d    4.50      Yves       40
         z    5.75     Henry      100
```

pandas 的优势之一是处理缺漏的信息。为此，考虑如下代码，这段代码添加一个新列，但是使用稍微不同的索引。我们在此使用相当灵活的 **join** 方法：

```
In [16]: df.join(pd.DataFrame([1, 4, 9, 16, 25],
                              index=['a', 'b', 'c', 'd', 'y'],
                              columns=['squares',]))
           # temporary object
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[16]:    floats     names  numbers  squares
         a    1.50     Guido       10        1
         b    2.50     Felix       20        4
         c    3.50  Francesc       30        9
         d    4.50      Yves       40       16
         z    5.75     Henry      100      NaN
```

例中可以看到，**pandas** 默认只接受索引已经存在的值。我们丢失了索引为 y 的值，在索引位置 z 可以看到 **NaN**（也就是“不是一个数字”）值。为了保留这两个索引，我们可以提供一个附加参数，告诉 **pandas** 如何连接。例子中的 **how="outer"**表示使用两个索引中所有值的并集：

```
In [17]: df = df.join(pd.DataFrame([1, 4, 9, 16, 25],
                                   index=['a', 'b', 'c', 'd', 'y'],
                                   columns=['squares',]),
                                   how='outer')
         df
Out[17]:    floats     names  numbers  squares
         a    1.50     Guido       10        1
         b    2.50     Felix       20        4
         c    3.50  Francesc       30        9
         d    4.50      Yves       40       16
         y     NaN       NaN      NaN       25
         z    5.75     Henry      100      NaN
```

现在，索引确实是两个原始索引的并集。在扩增的新索引中，丢失的数据点都由 **NaN** 值代替。连接操作的其他选项包括 **inner**（索引的交集）、**left**（默认值，使用调用方法的对象中的索引值）以及 **right**（使用被连接对象的索引值）。

尽管有丢失的值，但是大部分方法调用仍然有效。例如：

```
In [18]: df[['numbers', 'squares']].mean()
           # column-wise mean
Out[18]: numbers    40
         squares    11
         dtype: float64
In [19]: df[['numbers', 'squares']].std()
           # column-wise standard deviation
Out[19]: numbers    35.355339
         squares     9.669540
         dtype: float64
```

6.1.2 使用 DataFrame 类的第二步

从现在开始，我们将处理数值数据。我们将随着进程增加 **DatetimeIndex** 等功能，以管理时间序列数据。为了生成一个模拟数据集，按照下面的例子，生成一个包含 9 行 4 列标准正态分布伪随机数的 **numpy.ndarray**：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [20]: a = np.random.standard_normal((9, 4))
         a.round(6)
Out[20]: array([[-0.737304,  1.065173,  0.073406,  1.301174],
                [-0.788818, -0.985819,  0.403796, -1.753784],
                [-0.155881, -1.752672,  1.037444, -0.400793],
                [-0.777546,  1.730278,  0.417114,  0.184079],
                [-1.76366 , -0.375469,  0.098678, -1.553824],
                [-1.134258,  1.401821,  1.227124,  0.979389],
                [ 0.458838, -0.143187,  1.565701, -2.085863],
                [-0.103058, -0.36617 , -0.478036, -0.03281 ],
                [ 1.040318, -0.128799,  0.786187,  0.414084]])
```

你可以更直接地构造 **DataFrame** 对象（正如前面所见），但是使用 **ndarray** 对象通常是一个好选择，因为 **pandas** 将保留基本结构，“只”添加元信息（例如索引值）。这也代表着金融应用和科学研究的一种典型用例。例如：

```
In [21]: df = pd.DataFrame(a)
         df
Out[21]:          0         1         2         3
         0 -0.737304  1.065173  0.073406  1.301174
         1 -0.788818 -0.985819  0.403796 -1.753784
         2 -0.155881 -1.752672  1.037444 -0.400793
         3 -0.777546  1.730278  0.417114  0.184079
         4 -1.763660 -0.375469  0.098678 -1.553824
         5 -1.134258  1.401821  1.227124  0.979389
         6  0.458838 -0.143187  1.565701 -2.085863
         7 -0.103058 -0.366170 -0.478036 -0.032810
         8  1.040318 -0.128799  0.786187  0.414084
```

表 6-1 列出了 DataFrame 函数使用的参数。表中，“类似数组”意味着和 **ndarray** 对象类似的数据结构——例如**列表对象**。“索引”是 **pandas Index** 类的一个实例。

表 6-1　DataFrame 函数参数

参数	格式	描述
data	ndarray/字典/DataFrame	DataFrame 数据；字典可以包含序列、ndarray 和列表
index	索引/类似数组	使用的索引；默认为 range(n)
columns	索引/类似数组	使用的列标题；默认为 range(n)
dtype	dtype，默认 None	使用/强制的数据类型；否则通过推导得出
copy	布尔值，默认 None	从输入拷贝数据

和结构数组一样，我们已经看到，**DataFrame** 对象可以通过指定一个具有合适数量元素的**列表**，直接定义列名。下面的例子说明，可以随时定义/更改 **DataFrame** 对象的属性：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [22]: df.columns = [['No1', 'No2', 'No3', 'No4']]
         df
Out[22]:        No1       No2       No3       No4 '
         0 -0.737304  1.065173  0.073406  1.301174
         1 -0.788818 -0.985819  0.403796 -1.753784
         2 -0.155881 -1.752672  1.037444 -0.400793
         3 -0.777546  1.730278  0.417114  0.184079
         4 -1.763660 -0.375469  0.098678 -1.553824
         5 -1.134258  1.401821  1.227124  0.979389
         6  0.458838 -0.143187  1.565701 -2.085863
         7 -0.103058 -0.366170 -0.478036 -0.032810
         8  1.040318 -0.128799  0.786187  0.414084
```

列名提供访问 DataFrame 对象中数据的高效机制，同样和结构数组类似：

```
In [23]: df['No2'][3]  # value in column No2 at index position 3
Out[23]: 1.7302783624820191
```

为了高效处理金融事件序列数据，还必须很好地处理时间索引。这也可以视为 **pandas** 的一个重要优势。例如，假定我们分为 4 列的 9 个数据项对应于 2015 年 1 月开始的月底数据。然后，用 **date_range** 生成一个 **DatetimeIndex** 对象：

```
In [24]: dates = pd.date_range('2015-1-1', periods=9, freq='M')
         dates
Out[24]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2015-01-31, …, 2015-09-30]
         Length: 9, Freq: M, Timezone: None
```

表 6-2 列出 **date_range** 函数的参数。

表 6-2　date_range 函数参数

参数	格式	描述
start	字符串/日期时间	生成日期的左界
end	字符串/日期时间	生成日期的右界
periods	整数/None	期数（如果 start 或者 end 空缺）
freq	字符串/日期偏移	频率字符串，例如 5D（5 天）
tz	字符串/None	本地化索引的时区名称
normalize	布尔值，默认 None	将 start 和 end 规范化为午夜
name	字符串，默认 None	结果索引名称

目前，我们只遇到过由**字符串**和**整数对象**组成的索引，但是，对于时间序列数据，当然需要由 **date_range** 函数生成的 **DatetimeIndex** 对象。

和列一样，我们将新生成的 **DatetimeIndex** 作为新的 **Index** 对象，赋给 **DataFrame** 对象：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [25]: df.index = dates
         df
Out[25]:                  No1       No2       No3       No4
         2015-01-31 -0.737304  1.065173  0.073406  1.301174
         2015-02-28 -0.788818 -0.985819  0.403796 -1.753784
         2015-03-31 -0.155881 -1.752672  1.037444 -0.400793
         2015-04-30 -0.777546  1.730278  0.417114  0.184079
         2015-05-31 -1.763660 -0.375469  0.098678 -1.553824
         2015-06-30 -1.134258  1.401821  1.227124  0.979389
         2015-07-31  0.458838 -0.143187  1.565701 -2.085863
         2015-08-31 -0.103058 -0.366170 -0.478036 -0.032810
         2015-09-30  1.040318 -0.128799  0.786187  0.414084
```

至于在 **date_range** 函数帮助下生成 **DatetimeIndex** 对象，频率参数 **freq** 有多种选择。表 6-3 列出了所有选项。

表 6-3　date_range 函数频率参数值

别名	描述
B	交易日
C	自定义交易日（试验性）
D	日历日
W	每周
M	每月底
BM	每月最后一个交易日
MS	月初
BMS	每月第一个交易日
Q	季度末
BQ	每季度最后一个交易日
QS	季度初
BQS	每季度第一个交易日
A	每年底
BA	每年最后一个交易日
AS	每年初
BAS	每年第一个交易日
H	每小时
T	每分钟
S	每秒
L	毫秒
U	微秒

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

在本小节中，我们从 **NumPy ndarray** 对象开始，以形式上更丰富的 **pandas DateFrame** 对象结束。但是这一过程有其他变通方案吗？答案是“有”：

```
In [26]: np.array(df).round(6)
Out[26]: array([[-0.737304,  1.065173,  0.073406,  1.301174],
                [-0.788818, -0.985819,  0.403796, -1.753784],
                [-0.155881, -1.752672,  1.037444, -0.400793],
                [-0.777546,  1.730278,  0.417114,  0.184079],
                [-1.76366 , -0.375469,  0.098678, -1.553824],
                [-1.134258,  1.401821,  1.227124,  0.979389],
                [ 0.458838, -0.143187,  1.565701, -2.085863],
                [-0.103058, -0.36617 , -0.478036, -0.03281 ],
                [ 1.040318, -0.128799,  0.786187,  0.414084]])
```

数组和 DataFrame

通常可以从一个 **ndarray** 对象生成 **DataFrame** 对象。但是也可以简单地使用 **NumPy** 的 **array** 函数从 **DataFrame** 生成一个 **ndarray**。

6.1.3 基本分析

和 **NumPy** 数组一样，**pandas DataFrame** 类有多个便利的内建方法。例如，可以轻松地获得按列总和、平均值和累计总和：

```
In [27]: df.sum()
Out[27]: No1   -3.961370
         No2    0.445156
         No3    5.131414
         No4   -2.948346
         dtype: float64
In [28]: df.mean()
Out[28]: No1   -0.440152
         No2    0.049462
         No3    0.570157
         No4   -0.327594
         dtype: float64
In [29]: df.cumsum()
Out[29]:                  No1       No2       No3       No4
         2015-01-31 -0.737304  1.065173  0.073406  1.301174
         2015-02-28 -1.526122  0.079354  0.477201 -0.452609
         2015-03-31 -1.682003 -1.673318  1.514645 -0.853403
         2015-04-30 -2.459549  0.056960  1.931759 -0.669323
         2015-05-31 -4.223209 -0.318508  2.030438 -2.223147
         2015-06-30 -5.357467  1.083313  3.257562 -1.243758
         2015-07-31 -4.898629  0.940126  4.823263 -3.329621
         2015-08-31 -5.001687  0.573956  4.345227 -3.362430
         2015-09-30 -3.961370  0.445156  5.131414 -2.948346
```

还有一个通往多种常用数值数据集统计数字的捷径——**describe** 方法：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [30]: df.describe()
Out[30]:              No1       No2       No3       No4
         count  9.000000  9.000000  9.000000  9.000000
         mean  -0.440152  0.049462  0.570157 -0.327594
         std    0.847907  1.141676  0.642904  1.219345
         min   -1.763660 -1.752672 -0.478036 -2.085863
         25%   -0.788818 -0.375469  0.098678 -1.553824
         50%   -0.737304 -0.143187  0.417114 -0.032810
         75%   -0.103058  1.065173  1.037444  0.414084
         max    1.040318  1.730278  1.565701  1.301174
```

也可以对 **DataFrame** 对象应用大部分 **NumPy** 通用函数：

```
In [31]: np.sqrt(df)
Out[31]:                 No1       No2       No3       No4
         2015-01-31      NaN  1.032072  0.270935  1.140690
         2015-02-28      NaN       NaN  0.635449       NaN
         2015-03-31      NaN       NaN  1.018550       NaN
         2015-04-30      NaN  1.315400  0.645844  0.429045
         2015-05-31      NaN       NaN  0.314131       NaN
         2015-06-30      NaN  1.183985  1.107756  0.989641
         2015-07-31 0.677376       NaN  1.251280       NaN
         2015-08-31      NaN       NaN       NaN       NaN
         2015-09-30 1.019960       NaN  0.886672  0.643494
```

NumPy 通用函数

一般来说，在 **NumPy** 通用函数适用于 **ndarray** 对象的任何情况下，都可以将这些函数应用到包含相同数据的 **pandas DataFrame** 对象。

pandas 有相当强的容错能力，它可以捕捉错误，在对应数学运算失败时放入 **NaN** 值。不仅如此，正如前面已经展示的，在许多情况下，还可以将这些不完整的数据集当成完整数据集来使用：

```
In [32]: np.sqrt(df).sum()
Out[32]: No1    1.697335
         No2    3.531458
         No3    6.130617
         No4    3.202870
         dtype: float64
```

在这些情况下，**pandas** 将不考虑 **NaN** 值，只使用其他可用值。一般情况下，绘制数据图表也只需要一行代码（参见图 6-1）：

```
In [33]: %matplotlib inline
         df.cumsum().plot(lw=2.0)
```

本质上，**pandas** 提供 **matplotplib**（参见第 5 章）的一个封装器，专门为 **DataFrame** 对象所设计。表 6-4 列出了 plot 方法的参数。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

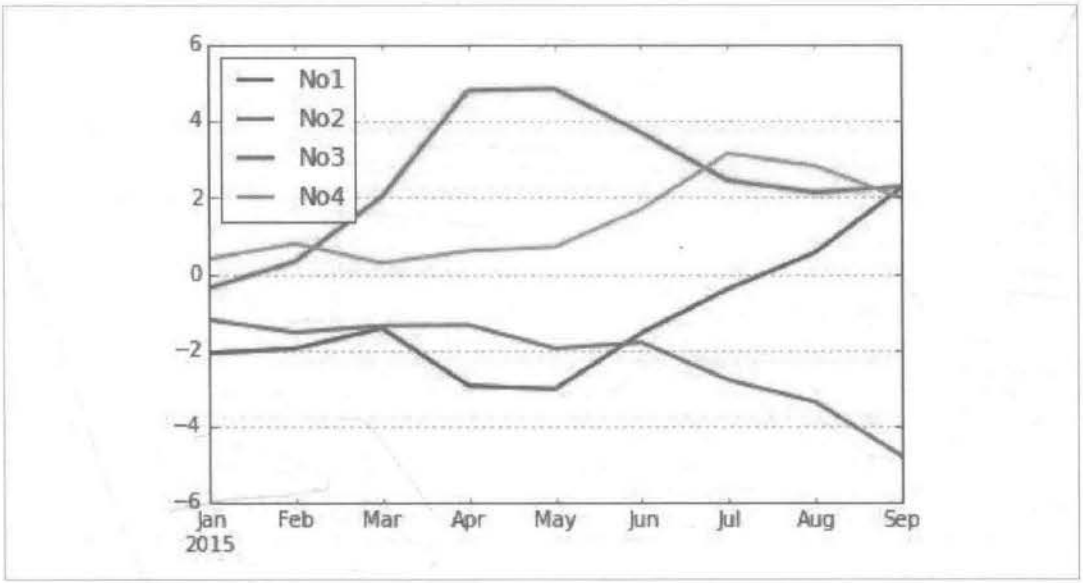


图 6-1　DataFrame 对象的线图

表 6-4　plot 方法参数

参数	格式	描述
x	标签/位置，默认 None	只在列值为 x 刻度时使用
y	标签/位置，默认 None	只在列值为 x 刻度时使用
subplots	布尔值，默认 False	子图中的绘图列
sharex	布尔值，默认 True	共用 x 轴
sharey	布尔值，默认 False	共用 y 轴
use_index	布尔值，默认 True	使用 DataFrame.index 作为 x 轴刻度
stacked	布尔值，默认 False	堆叠（只用于柱状图）
sort_columns	布尔值，默认 False	在绘图之前将列按字母顺序排列
title	字符串，默认 None	图表标题
grid	布尔值，默认 False	水平和垂直网格线
legend	布尔值，默认 True	标签图例
ax	matplotlib 轴对象	绘图使用的 matplotlib 轴对象
style	字符串或者列表/字典	绘图线形（每列）
kind	“line “/” bar “/” barh “/” kde “/” density”	图表类型
logx	布尔值，默认 False	x 轴的对数刻度
logy	布尔值，默认 False	y 轴的对数刻度

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

参数	格式	描述
xticks	序列，默认 Index	x 轴刻度
yticks	序列，默认 Values	y 轴刻度
xlim	二元组，列表	x 轴界限
ylim	二元组，列表	y 轴界限
rot	整数，默认 None	旋转 x 刻度
secondary_y	布尔值/序列，默认 False	第二个 y 轴
mark_right	布尔值，默认 True	第二个 y 轴自动设置标签
colormap	字符串/颜色映射对象，默认 None	用于绘图的颜色映射
kwds	关键字	传递给 matplotlib 选项

6.1.4 Series 类

目前，我们主要使用 **pandas DataFrame** 类：

```
In [34]: type(df)
Out[34]: pandas.core.frame.DataFrame
```

但是还有一个专用的 **Series** 类。例如，在从 **DataFrame** 对象中选择单一列时，我们就得到一个 **Series** 对象：

```
In [35]: df['No1']
Out[35]: 2015-01-31   -0.737304
         2015-02-28   -0.788818
         2015-03-31   -0.155881
         2015-04-30   -0.777546
         2015-05-31   -1.763660
         2015-06-30   -1.134258
         2015-07-31    0.458838
         2015-08-31   -0.103058
         2015-09-30    1.040318
         Freq: M, Name: No1, dtype: float64
In [36]: type(df['No1'])
Out[36]: pandas.core.series.Series
```

DataFrame 的主要方法也可用于 **Series** 对象，例如，我们可以和以前一样绘制结果图表（参见图 6-2）：

```
In [37]: import matplotlib.pyplot as plt
         df['No1'].cumsum().plot(style='r', lw=2.)
         plt.xlabel('date')
         plt.ylabel('value')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

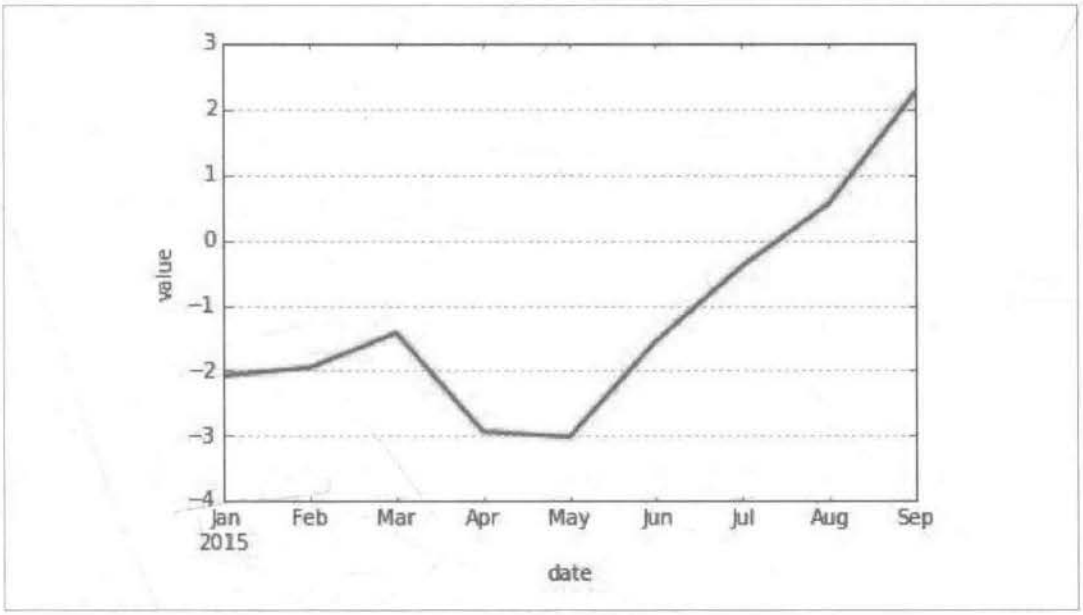


图 6-2　Series 对象的线图

6.1.5　GroupBy 操作

pandas 具备强大而灵活的分组功能，工作方式类似于 **SQL** 中的分组和 Microsoft **Excel** 中的透视表。为了进行分组，我们添加一列，表示对应索引数据所属的季度：

```
In [38]: df['Quarter'] = ['Q1', 'Q1', 'Q1', 'Q2', 'Q2', 'Q2', 'Q3', 'Q3', 'Q3']
         df
Out[38]:                 No1       No2       No3       No4 Quarter
         2015-01-31 -0.737304  1.065173  0.073406  1.301174      Q1
         2015-02-28 -0.788818 -0.985819  0.403796 -1.753784      Q1
         2015-03-31 -0.155881 -1.752672  1.037444 -0.400793      Q1
         2015-04-30 -0.777546  1.730278  0.417114  0.184079      Q2
         2015-05-31 -1.763660 -0.375469  0.098678 -1.553824      Q2
         2015-06-30 -1.134258  1.401821  1.227124  0.979389      Q2
         2015-07-31  0.458838 -0.143187  1.565701 -2.085863      Q3
         2015-08-31 -0.103058 -0.366170 -0.478036 -0.032810      Q3
         2015-09-30  1.040318 -0.128799  0.786187  0.414084      Q3
```

现在，我们可以根据“Quarter”列分组，输出单独组的统计数字：

```
In [39]: groups = df.groupby('Quarter')
```

例如，我们很容易用如下代码获得每个组的平均值（**mean**）、最大值（**max**）和组的大小（**size**）：

```
In [40]: groups.mean()
Out[40]:                 No1       No2       No3       No4
         Quarter
         Q1      -0.560668 -0.557773  0.504882 -0.284468
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Q2      -1.225155  0.918877  0.580972 -0.130118
Q3       0.465366 -0.212719  0.624617 -0.568196

In [41]: groups.max()

Out[41]:                No1       No2       No3       No4
         Quarter
         Q1       -0.155881  1.065173  1.037444  1.301174
         Q2       -0.777546  1.730278  1.227124  0.979389
         Q3        1.040318 -0.128799  1.565701  0.414084

In [42]: groups.size()

Out[42]: Quarter
         Q1         3
         Q2         3
         Q3         3
         dtype: int64
```

分组可以在多列上进行。为此，我们添加另一列，表示索引日期的月份是奇数还是偶数：

```
In [43]: df['Odd_Even'] = ['Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even',
                           'Odd', 'Even', 'Odd']
```

现在，可以使用上述附加信息，同时根据两列分组：

```
In [44]: groups = df.groupby(['Quarter', 'Odd_Even'])

In [45]: groups.size()

Out[45]: Quarter  Odd_Even
         Q1       Even        1
                  Odd         2
         Q2       Even        2
                  Odd         1
         Q3       Even        1
                  Odd         2
         dtype: int64

In [46]: groups.mean()

Out[46]:                        No1       No2       No3       No4
         Quarter Odd_Even
         Q1      Even      -0.788818 -0.985819  0.403796 -1.753784
                 Odd       -0.446592 -0.343749  0.555425  0.450190
         Q2      Even      -0.955902  1.566050  0.822119  0.581734
                 Odd       -1.763660 -0.375469  0.098678 -1.553824
         Q3      Even      -0.103058 -0.366170 -0.478036 -0.032810
                 Odd        0.749578 -0.135993  1.175944 -0.835890
```

对 **pandas** 和 **DataFrame** 对象使用的介绍到此结束，下面的小节将这组工具应用到现实世界的金融数据。

6.2 金融数据

当今的 Web 本身提供了大量的免费金融信息。谷歌或者雅虎等 Web 巨人提供全面的金

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

融数据。尽管这些数据有些时候不能满足专业需求，例如拆股的处理，但是适合于阐述 **pandas** 的“金融能力”。

为此，我们将使用 **pandas** 内建函数 **DataReader**，从雅虎财经网站（http://finance.yahoo.com）读取股价数据，分析该数据并生成不同的图表。[1]所需的函数保存在 **pandas** 的子模块中：

```
In [47]: import pandas.io.data as web
```

在本书编著的时候，pandas 支持如下数据源：

1. 雅虎财经（**yahoo**）
2. 谷歌财经（**google**）
3. 圣路易斯储备银行（**fred**）
4. 肯尼斯·弗伦奇的数据库（**famafrench**）
5. 世界银行（通过 **pandas.io.wb**）

例如，我们可以用一行代码，从雅虎财经读取德国 DAX 指数股价信息：

```
In [48]: DAX = web.DataReader(name='^GDAXI', data_source='yahoo',
                              start='2000-1-1')
         DAX.info()
Out[48]: <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 3760 entries, 2000-01-03 00:00:00 to 2014-09-26 00:00:00
         Data columns (total 6 columns):
         Open         3760 non-null float64
         High         3760 non-null float64
         Low          3760 non-null float64
         Close        3760 non-null float64
         Volume       3760 non-null int64
         Adj Close    3760 non-null float64
         dtypes: float64(5), int64(1)
```

表 6-5 介绍 DataReader 函数的参数。

表 6-5　DataReader 函数参数

参数	格式	描述
name	字符串	数据集名称——通常是股票代码
data_source	如“yahoo”	数据源
start	字符串/日期时间/None	范围左界（默认“2010/1/1”）
end	字符串/日期时间/None	范围右界（默认为当天）

tail 方法提供数据集的最后 5 行：

1　仅使用 matplotlib 的类似示例参见第 5 章。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [49]: DAX.tail()
Out[49]:                Open     High      Low    Close    Volume  Adj Close
         Date
         2014-09-22  9748.53  9812.77  9735.69  9749.54  73981000    9749.54
         2014-09-23  9713.40  9719.66  9589.03  9595.03  88196000    9595.03
         2014-09-24  9598.77  9669.45  9534.77  9661.97  85850600    9661.97
         2014-09-25  9644.36  9718.11  9482.54  9510.01  97697000    9510.01
         2014-09-26  9500.55  9545.34  9454.88  9490.55  83499600    9490.55
```

为了更好地概览指数的历史，可以再次用 **plot** 方法生成一个图表（参见图 6-3）：

```
In [50]: DAX['Close'].plot(figsize=(8, 5))
```

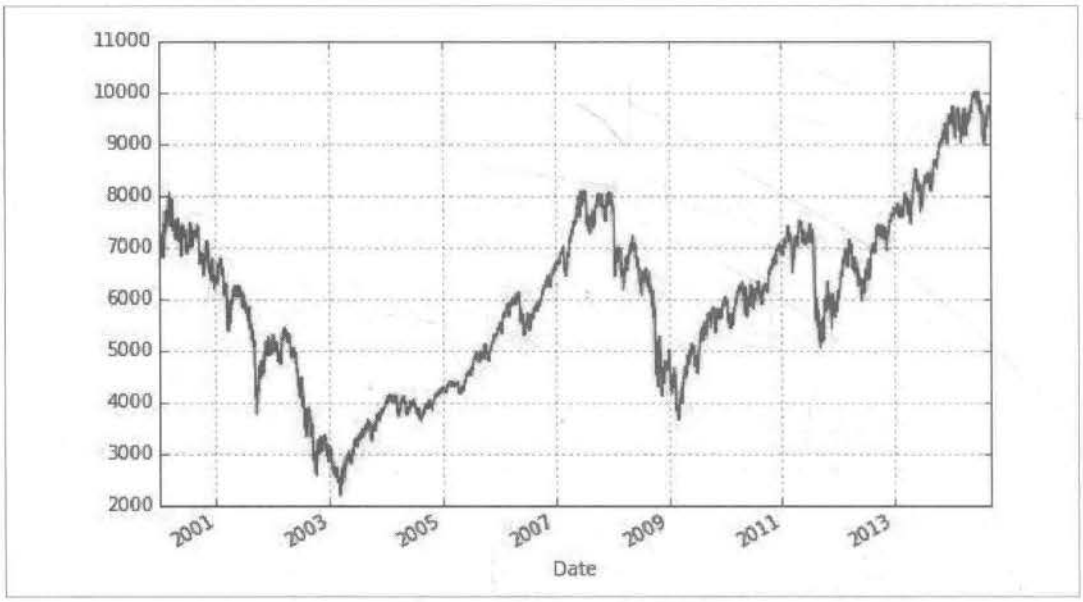


图 6-3　DAX 指数历史水平

读取数据并进行可视化是一回事，实现更复杂的分析任务是另一回事。和 **NumPy ndarray** 类似，**pandas** 允许在整个甚至复杂的 **DataFrame** 对象上进行向量化数学运算。根据每天的收盘价返回对数收益率就是一个例子。用下面的代码可以添加包含对应信息的列，首先生成一个新的空列，然后在所有指数上进行循环，逐步计算单个对数收益率值：

```
In [51]: %%time
         DAX['Ret_Loop'] = 0.0
         for i in range(1, len(DAX)):
             DAX['Ret_Loop'][i] = np.log(DAX['Close'][i] /
                                         DAX['Close'][i - 1])
Out[51]: CPU times: user 452 ms, sys: 12 ms, total: 464 ms
         Wall time: 449 ms
In [52]: DAX[['Close', 'Ret_Loop']].tail()
Out[52]:                Close  Ret_Loop
         Date
         2014-09-22  9749.54 -0.005087
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
2014-09-23  9595.03 -0.015975
2014-09-24  9661.97  0.006952
2014-09-25  9510.01 -0.015853
2014-09-26  9490.55 -0.002048
```

也可以使用向量化代码，在不使用循环的情况下得到相同的结果。在这方面，**shift** 方法很有用；它相对于索引向前或者向后移动 **Series** 或者整个 **DataFrame** 对象。为了实现我们的目标，必须将 **Close** 列移动一天，或者按照更一般的说法——一个索引位置：

```
In [53]: %time DAX['Return'] = np.log(DAX['Close'] / DAX['Close'].shift(1))
Out[53]: CPU times: user 4 ms, sys: 0 ns, total: 4 ms
         Wall time: 1.52 ms
In [54]: DAX[['Close', 'Ret_Loop', 'Return']].tail()
Out[54]:                Close  Ret_Loop    Return
         Date
         2014-09-22  9749.54 -0.005087 -0.005087
         2014-09-23  9595.03 -0.015975 -0.015975
         2014-09-24  9661.97  0.006952  0.006952
         2014-09-25  9510.01 -0.015853 -0.015853
         2014-09-26  9490.55 -0.002048 -0.002048
```

这不仅用更紧凑和更容易理解的代码提供了相同结果，而且是更快速的替代方案。

用 DataFrames 实现向量化

一般来说，在可以用两个包含相同数据的 **NumPy ndarray** 对象进行操作时，都可以用 **pandas DataFrame** 实现相同的向量化方法。

对于我们的目的，一个对数收益率数据列就足够了，所以我们可以删除另一列：

```
In [55]: del DAX['Ret_Loop']
```

现在，我们来看看新生成的收益率数据。图 6-4 展示了权益收益率的两种典型事实：

波动率聚集

波动率不是长期恒定的；既有*高波动率时期*（*正收益*和*负收益*都很高），也*有低波动率时期*。

杠杆效应

一般来说，波动性和股票市场收益是*负相关*的；当市场下跌的时候波动性升高，反之亦然。

下面是生成图表的代码：

```
In [56]: DAX[['Close', 'Return']].plot(subplots=True, style='b',
                                       figsize=(8, 5))
```

波动率对期权交易者特别重要，而（技术型）股票交易者可能对移动平均值（所谓**趋势**）更感兴趣。移动平均值很容易用 **pandas** 的 **rolling_mean** 函数计算（还有其他一些

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

“滚动”函数，如 **rolling_max**、**rolling_min** 和 **rolling_corr**）：

```
In [57]: DAX['42d'] = pd.rolling_mean(DAX['Close'], window=42)
         DAX['252d'] = pd.rolling_mean(DAX['Close'], window=252)
In [58]: DAX[['Close', '42d', '252d']].tail()
Out[58]:                Close          42d         252d
         Date
         2014-09-22  9749.54  9464.947143  9429.476468
         2014-09-23  9595.03  9463.780952  9433.168651
         2014-09-24  9661.97  9465.300000  9437.122381
         2014-09-25  9510.01  9461.880476  9440.479167
         2014-09-26  9490.55  9459.425000  9443.769008
```

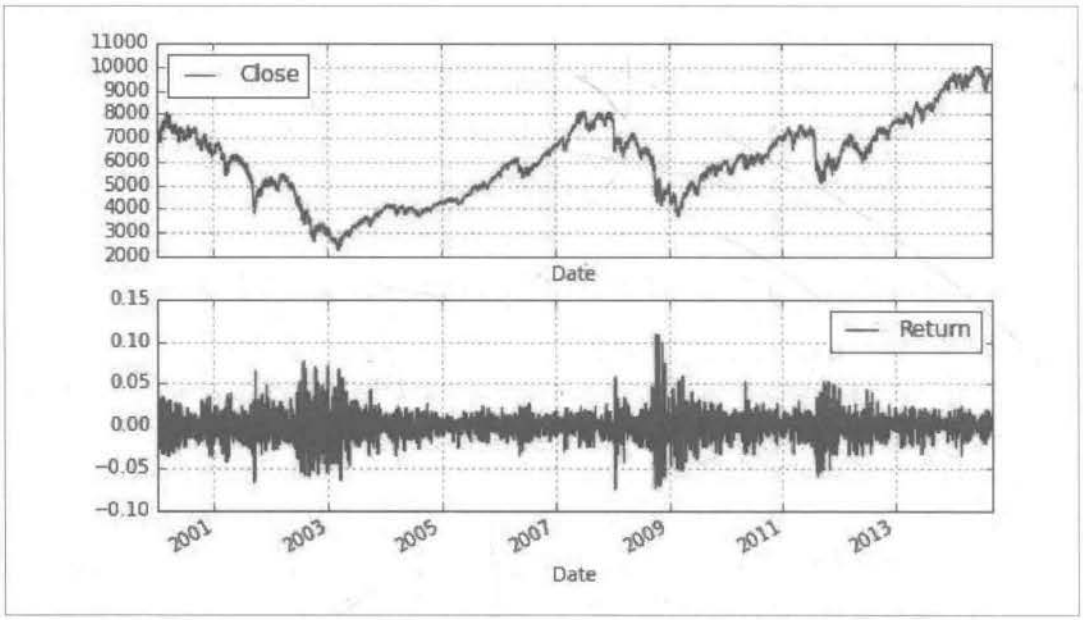


图 6-4　DAX 指数和每日指数收益

包含两种趋势的典型股价图表如图 6-5 所示：

```
In [59]: DAX[['Close', '42d', '252d']].plot(figsize=(8, 5))
```

回到期权交易者更喜欢的话题，对数收益率的移动历史标准差——即移动历史波动率——可能更有趣：

```
In [60]: import math
         DAX['Mov_Vol'] = pd.rolling_std(DAX['Return'],
                                         window=252) * math.sqrt(252)
           # moving annual volatility
```

图 6-6 进一步支持杠杆效应假设，说明市场下跌时历史移动波动率倾向于升高，而在市场上涨时波动率下降：

```
In [61]: DAX[['Close', 'Mov_Vol', 'Return']].plot(subplots=True, style='b',
                                                  figsize=(8, 7))
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

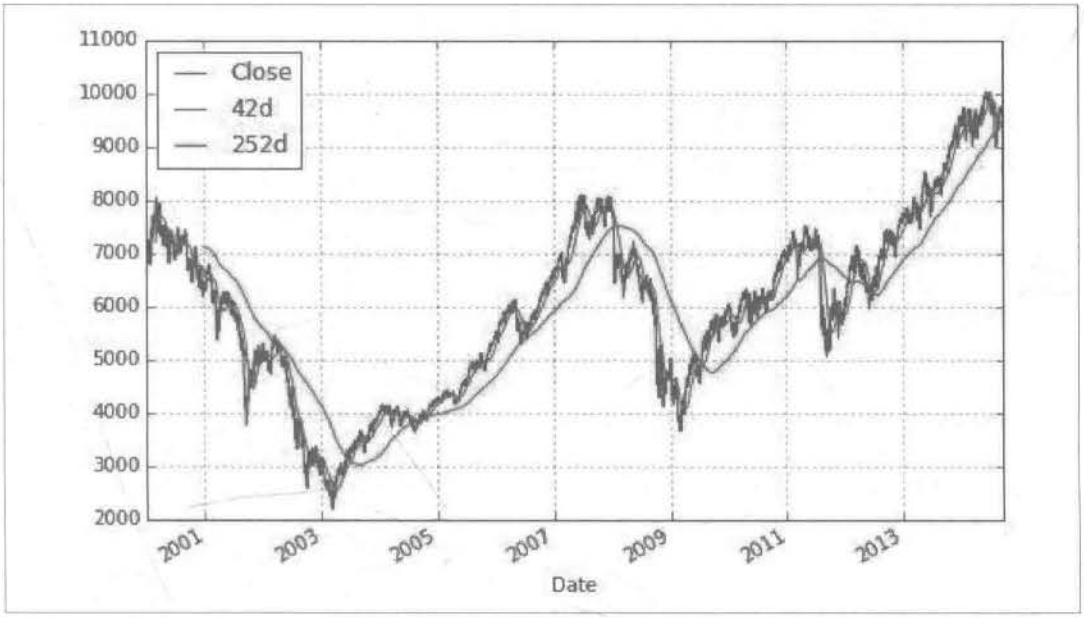


图 6-5　DAX 指数及移动平均值

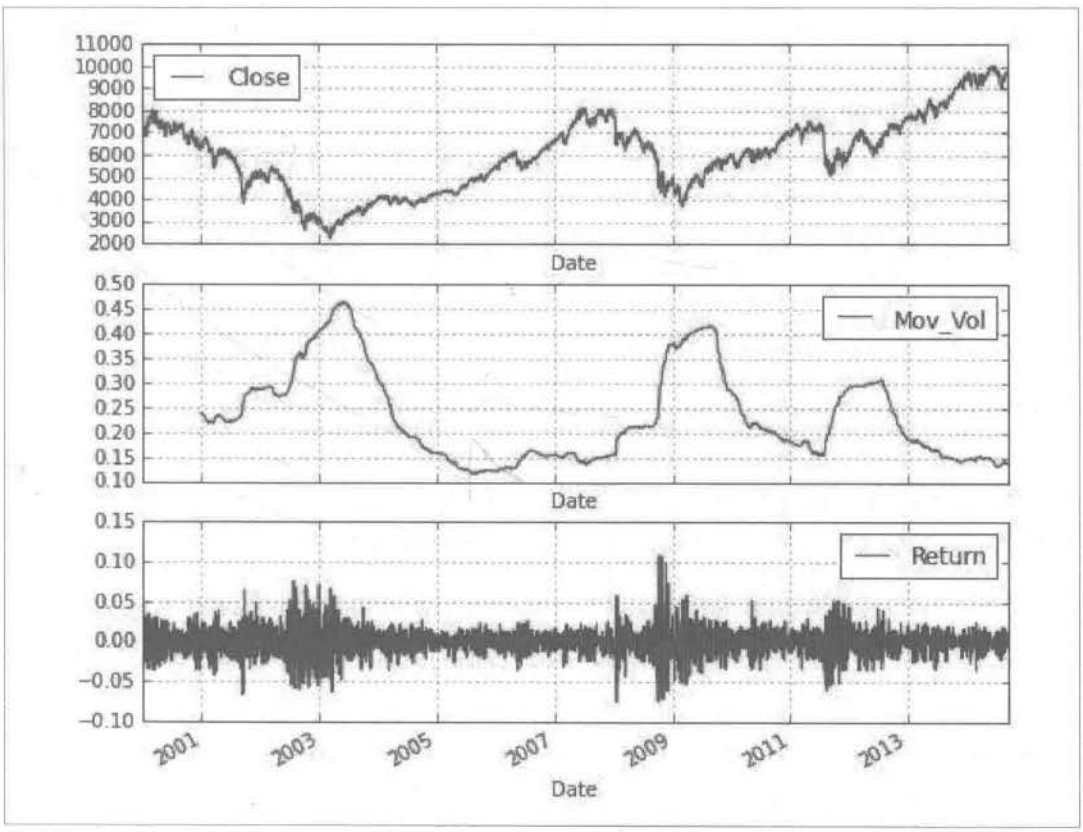


图 6-6　DAX 指数和移动年化波动率

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

6.3 回归分析

前一小节介绍过，杠杆效应是权益市场收益率的典型事实。目前为止，我们提供的支持仅基于对金融数据图表的检验。使用 **pandas**，还可以在更正式的统计学基础上进行这类分析。最简单的方法是使用（线性）*普通最小二乘回归（OLS）方法*。

下面，使用网络上可以取得的两个不同数据集进行分析：

EURO STOXX 50

EURO STOXX 50 指数的历史每日收盘价，该指数由欧洲蓝筹股价组成。

VSTOXX

VSTOXX 波动率指数历史每日收盘价，在 EURO STOXX 50 指数期权隐含波动率基础上计算。

值得注意的是，我们现在（间接）使用*隐含波动率*，这与未来波动率发展的预期相关，而之前的 DAX 分析使用的是历史波动率计量。这方面的细节，可以参见 Eurex 提供的“VSTOXX 高级服务”教程页面(http://www.eurexchange.com/advanced-services/vstoxx/)。

我们从几个导入操作开始：

```
In [62]: import pandas as pd
         from urllib import urlretrieve
```

为了分析，我们从 Web 读取文件，将其保存到 **data** 文件夹。如果没有这一文件夹，应该首先用 **mkdir data** 创建一个。接着，我们读取这两个指数的最新信息：

```
In [63]: es_url = 'http://www.stoxx.com/download/historical_values/hbrbcpe.txt'
         vs_url = 'http://www.stoxx.com/download/historical_values/h_vstoxx.txt'
         urlretrieve(es_url, './data/es.txt')
         urlretrieve(vs_url, './data/vs.txt')
         !ls -o ./data/*.txt
         # Windows: use dir
Out[63]: -rw--- 1 yhilpisch      0 Sep 28 11:14 ./data/es50.txt
         -rw--- 1 yhilpisch 641180 Sep 28 11:14 ./data/es.txt
         -rw--- 1 yhilpisch 330564 Sep 28 11:14 ./data/vs.txt
```

用 **pandas** 直接读取 EURO STOXX 50 数据不是最好的途径。事先进行一些数据清理可以得到更好的导入数据结构。必须解决和标题及结构相关的两个问题：

- 有两个额外的标题行无需导入；
- 从 2001 年 12 月 27 日开始，数据集中每行数据的最后“突然”出现一个额外的分号。

下列代码读取整个数据集并删除所有空白[1]：

```
In [64]: lines = open('./data/es.txt', 'r').readlines()
         lines = [line.replace(' ', '') for line in lines]
```

1 关于 Python 输入-输出操作的更多信息参见第 7 章。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

至于标题，我们很容易打印下载数据集的前几行进行检查：

```
In [65]: lines[:6]
Out[65]: ['PriceIndices-EUROCurrency\n',
          'Date;Blue-Chip;Blue-Chip;Broad;Broad;ExUK;ExEuroZone;Blue-Chip;Broad\
         n',
          ';Europe;Euro-Zone;Europe;Euro-Zone;;;Nordic;Nordic\n',
          ';SX5P;SX5E;SXXP;SXXE;SXXF;SXXA;DK5F;DKXF\n',
          '31.12.1986;775.00;900.82;82.76;98.58;98.06;69.06;645.26;65.56\n',
          '01.01.1987;775.00;900.82;82.76;98.58;98.06;69.06;645.26;65.56\n']
```

上面提到的格式更改可以在文件的第 3883 和 3990 行之间看到。从 12 月 27 日起，每个数据行的最后突然出现了一个额外的分号：

```
In [66]: for line in lines[3883:3890]:
             print line[41:],
Out[66]: 317.10;267.23;5268.36;363.19
         322.55;272.18;5360.52;370.94
         322.69;272.95;5360.52;370.94
         327.57;277.68;5479.59;378.69;
         329.94;278.87;5585.35;386.99;
         326.77;272.38;5522.25;380.09;
         332.62;277.08;5722.57;396.12;
```

为了使数据集更容易导入，我们进行如下处理。

1. 生成一个新的文本文件。
2. 删除不需要的标题行。
3. 在新文件中写入对应的新标题行。
4. 添加一个辅助列 DEL（捕捉最后的分号）。
5. 将所有数据行写入新文件。

经过这些调整，可以导入数据集，并在导入后删除辅助列。首先是清理代码：

```
In [67]: new_file = open('./data/es50.txt', 'w')
             # opens a new file
         new_file.writelines('date' + lines[3][:-1]
                             + ';DEL' + lines[3][-1])
             # writes the corrected third line of the original file
             # as first line of new file
         new_file.writelines(lines[4:])
             # writes the remaining lines of the orignial file
         new_file.close()
```

我们来看看新标题的外观：

```
In [68]: new_lines = open('./data/es50.txt', 'r').readlines()
         new_lines[:5]
Out[68]: ['date;SX5P;SX5E;SXXP;SXXE;SXXF;SXXA;DK5F;DKXF;DEL\n',
          '31.12.1986;775.00;900.82;82.76;98.58;98.06;69.06;645.26;65.56\n',
          '01.01.1987;775.00;900.82;82.76;98.58;98.06;69.06;645.26;65.56\n',
          '02.01.1987;770.89;891.78;82.57;97.80;97.43;69.37;647.62;65.81\n',
          '05.01.1987;771.89;898.33;82.82;98.60;98.19;69.16;649.94;65.82\n']
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

看起来，用 **pandas** 的 **read_csv** 函数进行导入是合适的，下面继续：

```
In [69]: es = pd.read_csv('./data/es50.txt', index_col=0,
                          parse_dates=True, sep=';', dayfirst=True)
In [70]: np.round(es.tail())
Out[70]:              SX5P  SX5E  SXXP  SXXE  SXXF  SXXA  DK5F  DKXF  DEL
         date
         2014-09-22  3096  3257   347   326   403   357  9703   565  NaN
         2014-09-23  3058  3206   342   321   398   353  9602   558  NaN
         2014-09-24  3086  3244   344   323   401   355  9629   560  NaN
         2014-09-25  3059  3202   341   320   397   353  9538   556  NaN
         2014-09-26  3064  3220   342   321   398   353  9559   557  NaN
```

辅助列已经完成了使命，可以删除：

```
In [71]: del es['DEL']
         es.info()
Out[71]: <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 7153 entries, 1986-12-31 00:00:00 to 2014-09-26 00:00:00
         Data columns (total 8 columns):
         SX5P    7153 non-null float64
         SX5E    7153 non-null float64
         SXXP    7153 non-null float64
         SXXE    7153 non-null float64
         SXXF    7153 non-null float64
         SXXA    7153 non-null float64
         DK5F    7153 non-null float64
         DKXF    7153 non-null float64
         dtypes: float64(8)
```

有了关于 EURO STOXX 50 数据集的知识，我们就可以使用 **read_csv** 函数的高级功能，使导入更加紧凑和高效：

```
In [72]: cols = ['SX5P', 'SX5E', 'SXXP', 'SXXE', 'SXXF',
                 'SXXA', 'DK5F', 'DKXF']
         es = pd.read_csv(es_url, index_col=0, parse_dates=True,
                          sep=';', dayfirst=True, header=None,
                          skiprows=4, names=cols)
In [73]: es.tail()
Out[73]:                SX5P     SX5E    SXXP    SXXE    SXXF    SXXA     DK5F
           DKXF
         2014-09-22  3096.02  3257.48  346.69  325.68  403.16  357.08  9703.33
         564.81
         2014-09-23  3057.89  3205.93  341.89  320.72  397.96  352.56  9602.32
         558.35
         2014-09-24  3086.12  3244.01  344.35  323.42  400.58  354.72  9628.84
         559.83
         2014-09-25  3059.01  3202.31  341.44  319.77  396.90  352.58  9537.95
         555.51
         2014-09-26  3063.71  3219.58  342.30  321.39  398.33  352.71  9558.51
         556.57
```

很幸运，VSTOXX 数据集的形式更容易导入 **DataFrame** 对象：

```
In [74]: vs = pd.read_csv('./data/vs.txt', index_col=0, header=2,
                          parse_dates=True, sep=',', dayfirst=True)
         vs.info()
Out[74]: <class 'pandas.core.frame.DataFrame'>
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
DatetimeIndex: 4010 entries, 1999-01-04 00:00:00 to 2014-09-26 00:00:00
Data columns (total 9 columns):
V2TX    4010 non-null float64
V6I1    3591 non-null float64
V6I2    4010 non-null float64
V6I3    3960 non-null float64
V6I4    4010 non-null float64
V6I5    4010 non-null float64
V6I6    3995 non-null float64
V6I7    4010 non-null float64
V6I8    3999 non-null float64
dtypes: float64(9)
```

表 6-6 列出了这个重要导入函数的参数。参数很多，其中大部分默认值为 **None**；当然，**object** 参数没有默认值，在任何情况下都必须指定。

表 6-6 read_csv 函数参数

参数	格式	描述
object	字符串	文件路径，URL 或者其他源
sep	字符串，默认 “,”	所使用的分隔符
lineterminator	字符串（一个字符）	换行符
quotechar	字符串	引号字符
quoting	整数	控制引号的识别
escapechar	字符串	转义字符串
dtpye	dtype/字典	列 dtype 字典
compression	“gzip” / “bz2”	用于数据解压缩
dialect	字符串/ csv.Dialect	CSV 风格，默认为 Excel
header	整数	标题行数量
skiprows	整数	跳过行数量
index_col	整数	索引列编号（多个索引的序列）
names	类数组	没有标题行时的列名
prefix	字符串	没有标题名时加到列编号上的字符串
na_values	列表/字典	识别为 NA、NaN 的其他字符串
true_values	列表	视为 True 的值
false_values	列表	视为 False 的值
keep_default_na	布尔值，默认 True	如果为 True，在 na_values 上加入 NaN
parse_dates	布尔值/列表，默认 False	是否解析索引列或者多列中的日期
keep_date_col	布尔值，默认 False	保持原始的日期列
dayfirst	布尔值，默认 False	用于欧洲日期惯例 DD/MM
Thousands	字符串	表示千的运算符
comment	字符串	行的剩余部分为注释（不解析）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

参数	格式	描述
Decimal	字符串	表示小数点的字符串，例如“.”或“,”
nrows	整数	读取的文件行数
iterator	布尔值，默认 False	返回 TextFileReader 对象
chunksize	整数	返回用于循环的 TextFileReader
skipfooter	整数	文件尾部跳过的行数
converters	字典	转换/翻译列数据的函数
verbose	布尔值，默认 False	报告非数值列中 NA 值的数量
delimiter	字符串	sep 的替代，可以包含正则表达式
encoding	字符串	使用的编码，如“UTF-8”
squeeze	布尔值，默认 False	以 Series 的形式返回单列数据集
na_filter	布尔值，默认 False	自动检测遗漏的值标记
usecols	类数组	选择使用的列
mangle_dupe_cols	布尔值，默认 False	为重复的列取不同的名称
tupleize_cols	布尔值，默认 False	按原样保留列上的元组列表

为了实施回归分析，我们只需要每个数据集中的一列。因此，我们生成一个新的 **DataFrame** 对象，在其中组合两个感兴趣的列，也就是主要的指数列。因为 VSTOXX 只有从 1999 年 1 月初开始的数据，我们只取该日期以后的数据：

```
In [75]: import datetime as dt
         data = pd.DataFrame({'EUROSTOXX' :
                              es['SX5E'][es.index > dt.datetime(1999, 1, 1)]})
         data = data.join(pd.DataFrame({'VSTOXX' :
                              vs['V2TX'][vs.index > dt.datetime(1999, 1, 1)]}))
```

我们还用时间序列中最后的可用值填补遗漏的数值，调用 **fillna** 方法，以 **ffill**（用于*向前填充*）作为 **method** 参数的值。另一个选则是 **bfill**（用于*向后填充*），但是这会造成“预见”问题：

```
In [76]: data = data.fillna(method='ffill')
         data.info()
Out[76]: <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 4034 entries, 1999-01-04 00:00:00 to 2014-09-26 00:00:00
         Data columns (total 2 columns):
         EUROSTOXX    4034 non-null float64
         VSTOXX       4034 non-null float64
         dtypes: float64(2)
In [77]: data.tail()
Out[77]:             EUROSTOXX   VSTOXX
         2014-09-22    3257.48  15.8303
         2014-09-23    3205.93  17.7684
         2014-09-24    3244.01  15.9504
         2014-09-25    3202.31  17.5658
         2014-09-26    3219.58  17.6012
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

同样，新数据集的图形表现可能提供一些深刻的认识。确实，图 6-7 表明两个指数之间似乎有某种负相关：

```
In [78]: data.plot(subplots=True, grid=True, style='b', figsize=(8, 6))
```

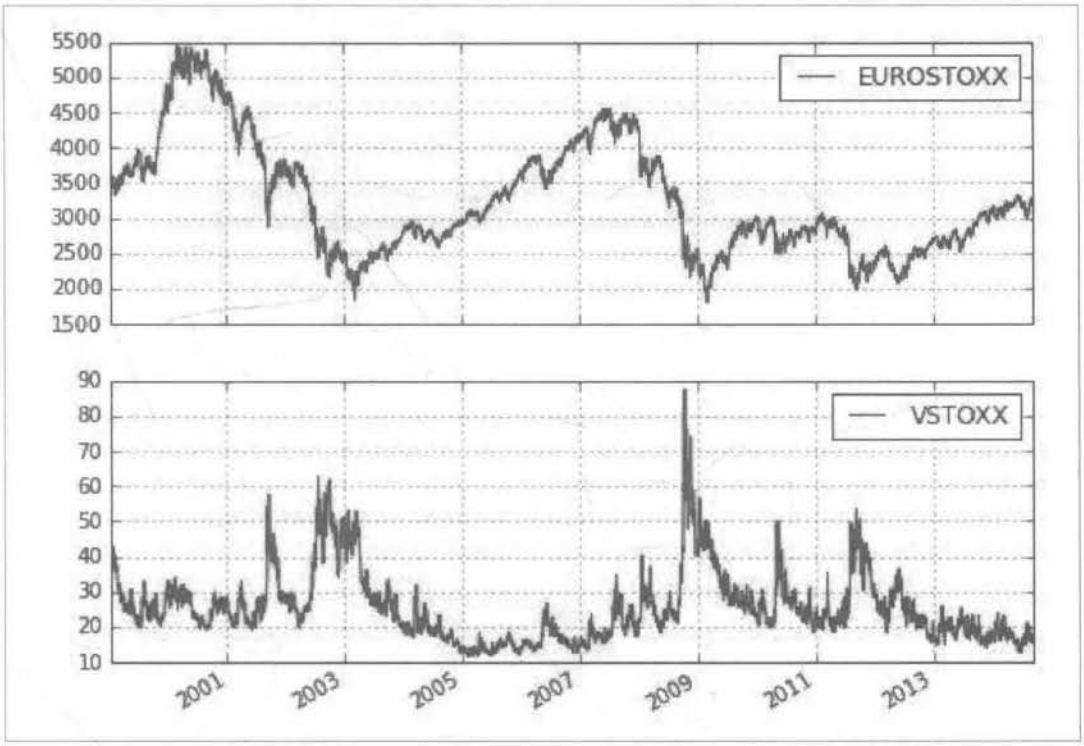


图 6-7　EURO STOXX 50 指数和 VSTOXX 波动率指数

不过，为了得到更为正式的论据，我们应该再次使用两个金融时间序列的对数收益率。图 6-8 以图形方式展示了这一数据：

```
In [79]: rets = np.log(data / data.shift(1))
         rets.head()
Out[79]:              EUROSTOXX     VSTOXX
         1999-01-04         NaN        NaN
         1999-01-05    0.017228   0.489248
         1999-01-06    0.022138  -0.165317
         1999-01-07   -0.015723   0.256337
         1999-01-08   -0.003120   0.021570
In [80]: rets.plot(subplots=True, grid=True, style='b', figsize=(8, 6))
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

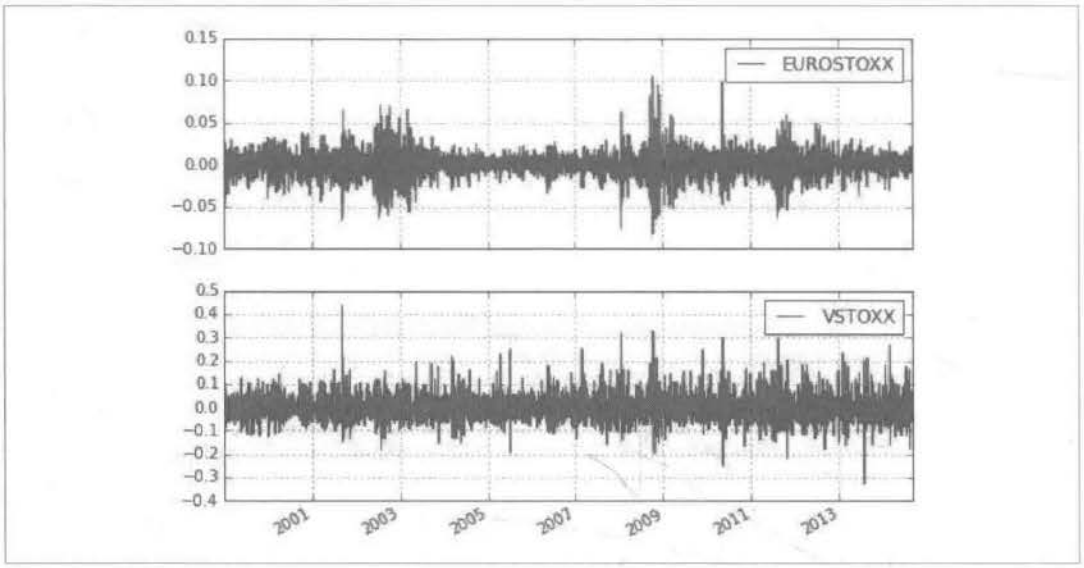


图 6-8　EURO STOXX 50 和 VSTOXX 对数收益率

我们已经有了实施回归分析的所有数据。接下来，将 EURO STOXX 50 收益率作为自变量，而将 VSTOXX 收益率作为因变量：

```
In [81]: xdat = rets['EUROSTOXX']
         ydat = rets['VSTOXX']
         model = pd.ols(y=ydat, x=xdat)
         model

Out[81]: ---------Summary of Regression Analysis------
         ---

         Formula: Y ~ <x> + <intercept>

         Number of Observations:         4033
         Number of Degrees of Freedom:   2

         R-squared:         0.5322
         Adj R-squared:     0.5321

         Rmse:              0.0389

         F-stat (1, 4031):  4586.3942, p-value:     0.0000

         Degrees of Freedom: model 1, resid 4031

         --------Summary of Estimated Coefficients-----
         ---
              Variable       Coef    Std Err     t-stat    p-value    CI 2.5%
          CI 97.5%
         ----------------------------------------------------------------------
         ---
                     x    -2.7529     0.0406     -67.72     0.0000    -2.8326
           -2.6732
              intercept    -0.0001     0.0006      -0.12     0.9043    -0.0013
            0.0011
         -----------End of Summary-------------------------------------------
         ---
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

显然，的确存在强负相关。我们可以这样访问结果：

```
In [82]: model.beta
Out[82]: x           -2.752894
         intercept   -0.000074
         dtype: float64
```

这个输入与原始对数收益率数据相结合，用于生成图 6-9 中的图表，该图表强烈支持杠杆效应：

```
In [83]: plt.plot(xdat, ydat, 'r.')
         ax = plt.axis()  # grab axis values
         x = np.linspace(ax[0], ax[1] + 0.01)
         plt.plot(x, model.beta[1] + model.beta[0] * x, 'b', lw=2)
         plt.grid(True)
         plt.axis('tight')
         plt.xlabel('EURO STOXX 50 returns')
         plt.ylabel('VSTOXX returns')
```

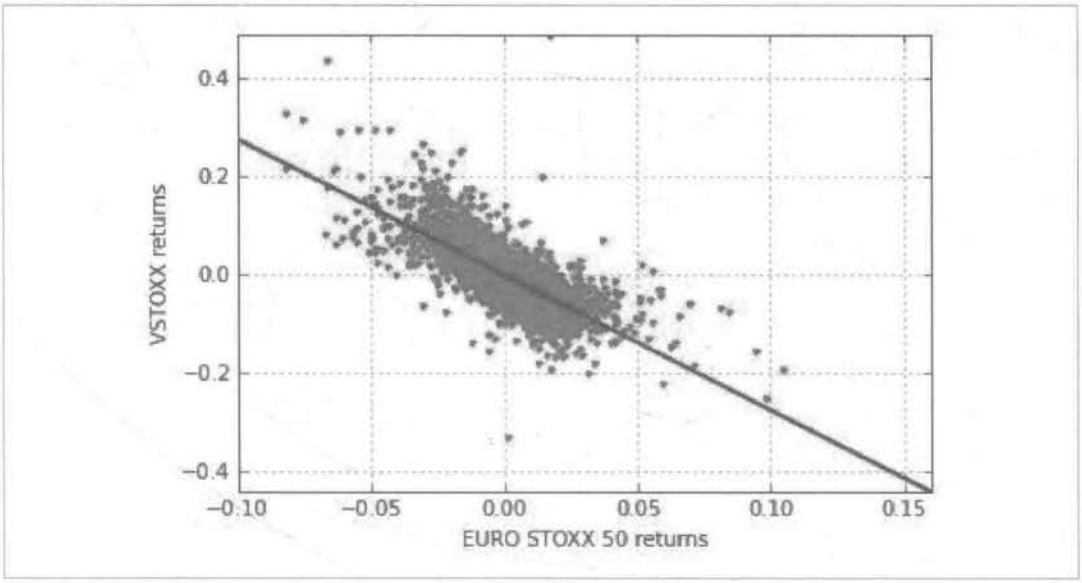


图 6-9　对数收益率散点图和回归线

作为最后的交叉检查，我们可以直接计算两个金融时间序列之间的相关性：

```
In [84]: rets.corr()
Out[84]:              EUROSTOXX     VSTOXX
         EUROSTOXX     1.000000 -0.729538
         VSTOXX       -0.729538  1.000000
```

尽管在整个数据集上有强烈的负相关，但是不同时期之间有显著的不同，如图 6-10 所示。该图使用了按年统计（252 个交易日）的相关性：

```
In [85]: pd.rolling_corr(rets['EUROSTOXX'], rets['VSTOXX'],
                         window=252).plot(grid=True, style='b')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

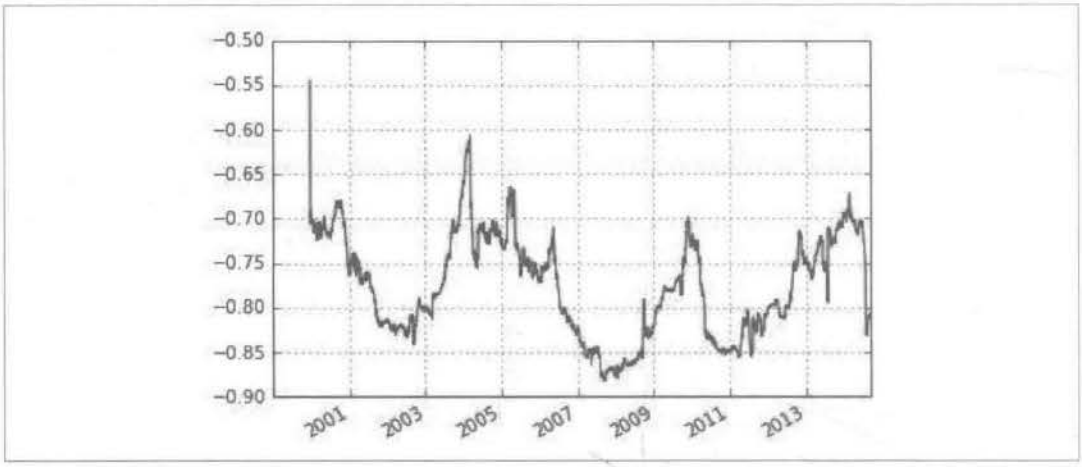


图 6-10　EURO STOXX 50 和 VSTOXX 之间的滚动相关

6.4　高频数据

到目前为止，你应该对 **pandas** 处理金融时间序列数据的优势有了感性认识。这种方法的一个特征在金融分析圈中已经十分普遍，对于某些市场参与者来说是很大的负担：*高频数据*。本小节简短地阐述如何处理分时数据（而非每日金融数据）。首先导入几个库：

```
In [86]: import numpy as np
         import pandas as pd
         import datetime as dt
         from urllib import urlretrieve
         %matplotlib inline
```

挪威在线经纪商 Netfonds（http://www.netfonds.no）提供多种股票的分时数据，特别是美国市场的股票。基于 Web 的 API 使用如下格式：

```
In [87]: url1 = 'http://hopey.netfonds.no/posdump.php?'
         url2 = 'date=%s%s%s&paper=AAPL.O&csv_format=csv'
         url = url1 + url2
```

我们想要下载、组合和分析苹果公司一周的分时股价数据，这是一支相当活跃的股票。我们首先确定感兴趣的日期[1]：

```
In [88]: year = '2014'
         month = '09'
         days = ['22', '23', '24', '25']
           # dates might need to be updated
In [89]: AAPL = pd.DataFrame()
         for day in days:
             AAPL = AAPL.append(pd.read_csv(url % (year, month, day),
                                index_col=0, header=0, parse_dates=True))
```

1　注意，这个数据提供者只提供当前日期前几天的此类数据。因此，你可能需要使用不同（更接近）的日期实现相同的示例。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
AAPL.columns = ['bid', 'bdepth', 'bdeptht',
                'offer', 'odepth', 'odeptht']
  # shorter columnn names
```

数据集现在包含近 10 万行：

```
In [90]: AAPL.info()
Out[90]: <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 95871 entries, 2014-09-22 10:00:01 to 2014-09-25 22:19:25
         Data columns (total 6 columns):
         bid        95871 non-null float64
         bdepth     95871 non-null float64
         bdeptht    95871 non-null float64
         offer      95871 non-null float64
         odepth     95871 non-null float64
         odeptht    95871 non-null float64
         dtypes: float64(6)
```

图 6-11 以图形形式展示了报价列数据。可以看到一些时期内没有任何交易活动——也就是休市期间：

```
In [91]: AAPL['bid'].plot()
```

当然，在开市期间的单个交易日内，通常有很高的活跃度。图 6-12 展示了样本中第一天和第三天的 3 个小时内的交易活动。这里使用的挪威的时区，很容易看到何时开始盘前交易，美国股市何时开市，何时闭市：

```
In [92]: to_plot = AAPL[['bid', 'bdeptht']][
             (AAPL.index > dt.datetime(2014, 9, 22, 0, 0))
           & (AAPL.index < dt.datetime(2014, 9, 23, 2, 59))]
            # adjust dates to given data set
         to_plot.plot(subplots=True, style='b', figsize=(8, 5))
```

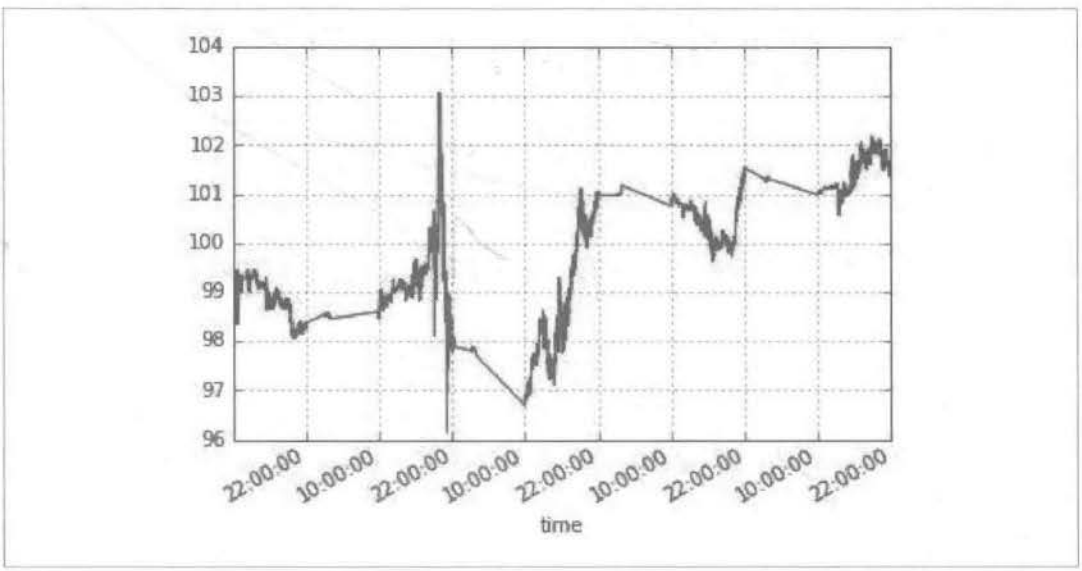


图 6-11　一周的苹果公司股价分时数据

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

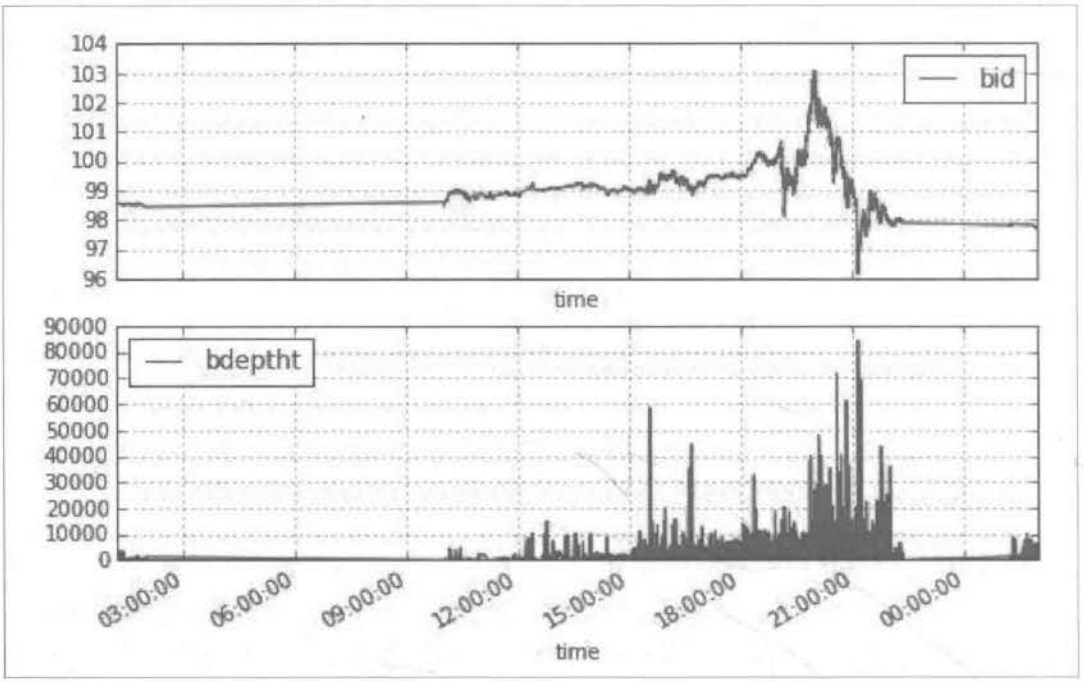


图 6-12　一个交易日的苹果公司股价分时数据和交易量

通常，金融分时数据会导致无规律的 **DateTimeIndex**。换句话说，两个观测点之间的时间间隔可能高度异质。鉴于这种背景，*重新采样*这种数据集有时可能很有用，甚至需要根据手上的任务重新排列。下面，我们简单地采用重采样过程；这在某些列（例如“bid”列）上是一致的，而在其他列（例如“bdepth”列上不一致）：

```
In [93]: AAPL_resam = AAPL.resample(rule='5min', how='mean')
         np.round(AAPL_resam.head(), 2)
Out[93]:                        bid  bdepth  bdeptht   offer  odepth  odeptht
         2014-09-22 10:00:00  100.49  366.67   366.67  100.95     200      200
         2014-09-22 10:05:00  100.49  100.00   100.00  100.84     200      200
         2014-09-22 10:10:00  100.54  150.00   150.00  100.74     100      100
         2014-09-22 10:15:00  100.59  200.00   200.00  100.75    1500     1500
         2014-09-22 10:20:00  100.50  100.00   100.00  100.75    1500     1500
```

结果如图 6-13，看上去较为平滑。这里，我们还用最近（空白时间间隔之前）的可用数值填充空白时间间隔：

```
In [94]: AAPL_resam['bid'].fillna(method='ffill').plot()
```

在本小节的最后，我们在新的数据集上应用自定义的 Python 函数。我们选择的是一个任意的函数，没有任何经济学意义；它只是镜像了某个股价水平上的股票表现（比较图 6-14 和图 6-13）：

```
In [95]: def reversal(x):
             return 2 * 95 - x
In [96]: AAPL_resam['bid'].fillna(method='ffill').apply(reversal).plot()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

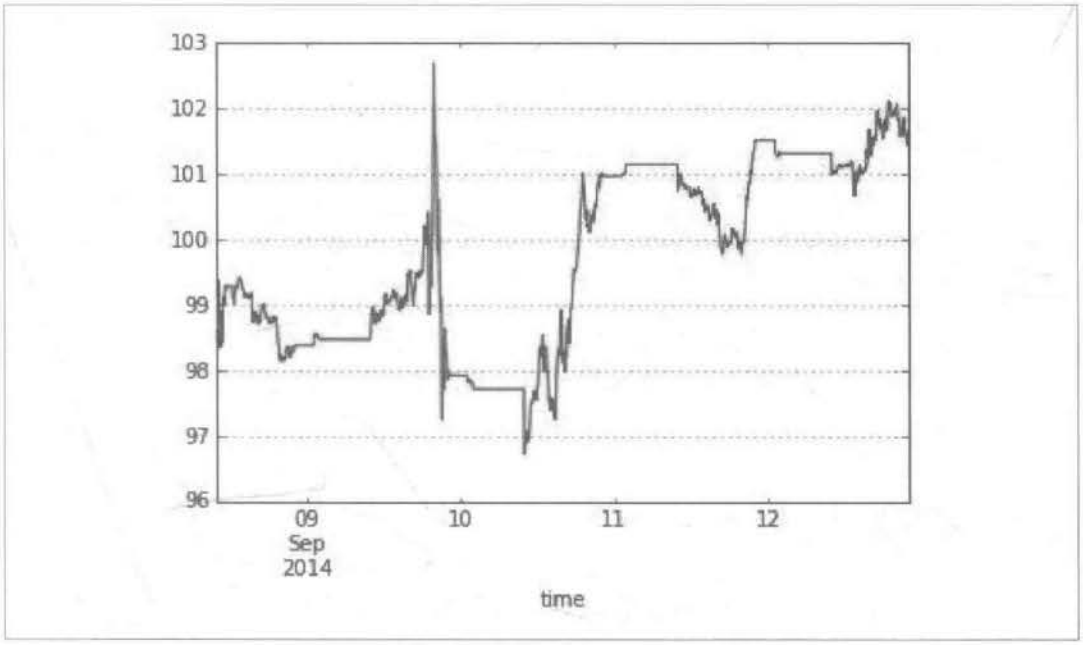


图 6-13　重新采样的苹果股价分时数据

最后，我们删除所有保存到磁盘的数据集，清理磁盘空间：

```
In [97]: !rm ./data/*
           # Windows: del /data/*
```

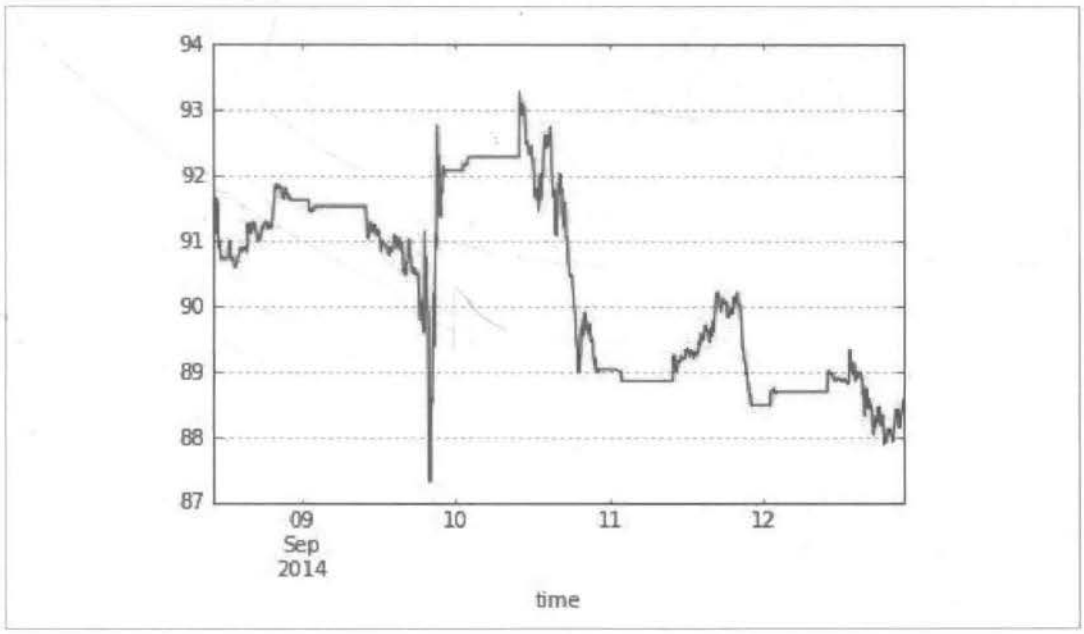


图 6-14　应用函数的重采样评估股价分时数据

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

6.5 结语

金融时间序列是金融数据最常见和重要的形式。在处理这些数据集时通常会选择 **pandas** 库。pandas DataFrame 类的模型来自 R 语言的 **data.frame** 类，提供了丰富的属性和方法，可以解决你所面对的几乎任何（金融）分析问题。*方便性*是 **pandas** 的另一个优点：尽管你可以使用 **NumPy** 和/或 **matplotlib** 产生相同的结果，但是 **pandas** 在强大和灵活的 API 基础上，通常能够提供很好的捷径。

此外，**pandas** 可以轻松地从各种 Web 数据源（如雅虎财经或者谷歌）读取数据。和“纯” **NumPy** 或者 **matplotlib** 相比，它在许多方面自动化了金融时间序列数据的管理，而且在数据集组合和扩充方面提供了更高的灵活性。

6.6 延伸阅读

在本书写作期间，印刷形式的 **pandas** 权威资源是该库主要作者的著作。

- McKinney, Wes (2012): *Data Analysis with Python*. O'Reilly, Sebastopol, CA.

当然，Web——特别是 **pandas** 网站本身——也提供了大量信息。

- 同样，**pandas** 库的主页是很好的起点：http://pandas.pydata.org。
- 在 http://pandas.pydata.org/pandas-docs/stable/上有一个相当全面的在线文档。
- http://pandas.pydata.org/pandasdocs/stable/pandas.pdf 上有一个 1500 多页的 PDF 格式文档，阐述了 **pandas** 所提供的丰富功能。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第7章

输入/输出操作

在未掌握数据之前就做出推测，是最大的错误。

——歇洛克•福尔摩斯

一般的原则是，不管是在金融环境还是其他任何应用领域，大部分数据都保存在硬盘驱动器（HDD）或者某种形式的固定存储设备上（如固态硬盘SSD或者混合型磁盘驱动器）。多年以来，存储容量稳步增长，而每个存储单位（例如兆字节）的成本不断下降。

与此同时，存储数据量的增长速度远远快于典型的随机访问存储器（RAM）的增长速度，即使在最大型的机器上也是如此。这就使得，将数据存储到磁盘不仅为了持久存储，也为了通过将数据从RAM交换到磁盘，弥补RAM不足的问题。

因此，输入/输出（I/O）操作通常是金融应用和数据密集型应用当中非常重要的任务。它们往往代表着性能关键计算的瓶颈，因为I/O操作一般无法以足够快的速度将数据写入RAM[1]和从RAM写入磁盘。在某种意义上，CPU常常因为缓慢的I/O操作而“挨饿”。

尽管大部分当今的金融和企业分析工作面对的是“大数据”（例如PB级别），单一分析任务使用的数据（子）集通常应该归入“中”数据的类别。最近的研究得出以下结论：

> 我们的测算和最近的其他工作表明，现实世界中大部分分析工作处理的输入小于100GB，但是Hadoop/MapReduce等流行基础架构最初是为PB级处理设计的。
>
> ——Appuswamy等人（2013）

至于频率，单一金融分析任务通常处理的数据不超过几个GB——这是**Python**及其科

1 在此，我们没有区分不同级别的RAM和处理器缓存。当前内存架构的最优使用方法本身就是一个主题。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

学栈中的库（如 **NumPy**、**pandas** 和 **PyTables**）最有效的区域。这样大小的数据集也可以在内存中分析，利用现在的 CPU 和 GPU 通常可以获得很高的速度。然而，数据必须读入 RAM、结果要写入磁盘，同时要确保满足当今的性能要求。

本章介绍如下领域：

基本 I/O

> **Python** 有内建的函数，序列化对象、将其存储到磁盘、从磁盘将其读入 RAM；除此之外，**Python** 很擅长处理文本文件和 **SQL** 数据库。**NumPy** 也提供了专用于快速存储和检索 **ndarray** 对象的函数。

pandas 的 I/O

> **pandas** 库提供了丰富的便利函数及方法，读取以不同格式存储的数据（例如 **CSV**、**JSON**），将数据写入不同格式的文件中。

PyTables 的 I/O

> **PyTables** 使用 HDF5 标准（http://www.hdfgroup.org）实现大数据集的快速 I/O 操作；其速度往往只受到所使用硬件的限制。

7.1 Python 基本 I/O

Python 本身自带多种 I/O 功能，有些为性能进行了优化，其他则更注重灵活性。不过，这些功能既可以方便地交互性使用，也可以在大规模部署下使用。

7.1.1 将对象写入磁盘

为了以后使用、文档或者与其他人共享，人们可能想将 **Python** 对象存储到磁盘上。选项之一是使用 **pickle** 模块。这个模块可以序列化大部分 **Python** 对象。*序列化*指的是将对象（层次结构）转换为一个字节流；*反序列化*是相反的操作。在下面的例子中，我们再次使用（伪）随机数据，这次保存在一个**列表**对象中：

```
In [1]: path = '/flash/data/'
In [2]: import numpy as np
        from random import gauss
In [3]: a = [gauss(1.5, 2) for i in range(1000000)]
          # generation of normally distributed randoms
```

现在的任务是将列表对象写入磁盘供以后检索。**pickle** 可以完成这项任务：

```
In [4]: import pickle
In [5]: pkl_file = open(path + 'data.pkl', 'w')
          # open file for writing
          # Note: existing file might be overwritten
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们需要的两个重要函数是写入对象的 **dump** 和将对象加载到内存的 **load**：

```
In [6]: %time pickle.dump(a, pkl_file)
Out[6]: CPU times: user 4.3 s, sys: 43 ms, total: 4.35 s
        Wall time: 4.36 s
In [7]: pkl_file
Out[7]: <open file '/flash/data/data.pkl', mode 'w' at 0x3df0540>
In [8]: pkl_file.close()
```

现在我们可以检查磁盘文件的大小。包含 100 万个浮点数的**列表**对象大约占据 20 兆字节（MB）磁盘空间：

```
In [9]: ll $path*
Out[9]: -rw-r—r— 1 root 20970325 28. Sep 15:16 /flash/data/data.pkl
```

有了磁盘数据，就可以通过 **pickle.load** 将其读入内存：

```
In [10]: pkl_file = open(path + 'data.pkl', 'r')  # open file for reading
In [11]: %time b = pickle.load(pkl_file)
Out[11]: CPU times: user 3.37 s, sys: 18 ms, total: 3.38 s
         Wall time: 3.39 s
In [12]: b[:5]
Out[12]: [-3.6459230447943165,
          1.4637510875573307,
          2.5483218463404067,
          0.9822259685028746,
          3.594915396586916]
```

将此与原始对象的前 5 个**浮点**对象比较：

```
In [13]: a[:5]
Out[13]: [-3.6459230447943165,
          1.4637510875573307,
          2.5483218463404067,
          0.9822259685028746,
          3.594915396586916]
```

为了确保对象 **a** 和 **b** 相同，**NumPy** 提供了 **allclose** 函数：

```
In [14]: np.allclose(np.array(a), np.array(b))
Out[14]: True
```

原理上，这和计算两个 **ndarray** 对象的差，检查是否为 0 一样：

```
In [15]: np.sum(np.array(a) - np.array(b))
Out[15]: 0.0
```

然而，**allclose** 有一个容错级别参数，默认设置为 **1e-5**。

用 **pickle** 存储和读取单个对象明显相当简单，那么，两个对象呢？

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [16]: pkl_file = open(path + 'data.pkl', 'w')  # open file for writing
In [17]: %time pickle.dump(np.array(a), pkl_file)
Out[17]: CPU times: user 799 ms, sys: 47 ms, total: 846 ms
         Wall time: 846 ms
In [18]: %time pickle.dump(np.array(a) ** 2, pkl_file)
Out[18]: CPU times: user 742 ms, sys: 41 ms, total: 783 ms
         Wall time: 784 ms
In [19]: pkl_file.close()
In [20]: ll $path*
Out[20]: -rw-r--r-- 1 root 44098737 28. Sep 15:16 /flash/data/data.pkl
```

发生了什么？主要有：

1. 我们将原始对象的 **ndarray** 版本写入磁盘。
2. 我们还将 **ndarray** 的平方版本写入磁盘上的同一个文件。
3. 两个操作都比原来的操作快（因为使用 **ndarray** 对象）。
4. 文件的大小大致为以前的两倍，因为我们保存了两倍的数据量。

下面我们将两个 **ndarray** 对象读回内存：

```
In [21]: pkl_file = open(path + 'data.pkl', 'r')  # open file for reading
```

pickle.load 可以完成这项工作。但是请注意，它只返回一个 **ndarray** 对象：

```
In [22]: x = pickle.load(pkl_file)
         x
Out[22]: array([-3.64592304,  1.46375109,  2.54832185, ...,  2.87048515,
                 0.66186994, -1.38532837])
```

第二次调用 **pickle.load** 返回第二个对象：

```
In [23]: y = pickle.load(pkl_file)
         y
Out[23]: array([ 13.29275485,   2.14256725,   6.49394423, ...,   8.23968501,
                  0.43807181,   1.9191347 ])
In [24]: pkl_file.close()
```

显然，**pickle** 按照*先进先出（FIFO）*原则保存对象。这种方法有一个重大问题：没有任何可用的元信息，让用户事先知道保存在 **pickle** 文件中的是什么。有时候可以采用一种变通的方法：不存储单个对象，而是存储包含所有其他对象的**字典（dict）**对象：

```
In [25]: pkl_file = open(path + 'data.pkl', 'w')  # open file for writing
         pickle.dump({'x' : x, 'y' : y}, pkl_file)
         pkl_file.close()
```

使用这种方法可以一次读取整组对象，还有一些其他好处，例如，可以在字典对象的关键值上循环：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [26]: pkl_file = open(path + 'data.pkl', 'r')  # open file for writing
         data = pickle.load(pkl_file)
         pkl_file.close()
         for key in data.keys():
             print key, data[key][:4]
Out[26]: y [ 13.29275485   2.14256725   6.49394423   0.96476785]
         x [-3.64592304  1.46375109  2.54832185  0.98222597]
In [27]: !rm -f $path*
```

不过，这种方法要求我们一次写入和读取所有对象，在许多情况下，人们可能需要为了更便利而承受这个问题。

7.1.2 读写文本文件

文本处理是 **Python** 的优势之一。实际上，许多企业和科学用户就将 **Python** 用于这项任务。利用 **Python** 处理字符串对象和文本文件有许多选项。

假定我们已经生成了一组相当大的数据，打算保存为逗号分隔值（**CSV**）文件并共享。尽管 **CSV** 文件有特殊的结构，但是这种文件本质上是普通文本文件：

```
In [28]: rows = 5000
         a = np.random.standard_normal((rows, 5))  # dummy data
In [29]: a.round(4)
Out[29]: array([[ 1.381 , -1.1236,  1.0622, -1.3997, -0.7374],
                [ 0.15  ,  0.967 ,  1.8391,  0.5633,  0.0569],
                [-0.9504,  0.4779,  1.8636, -1.9152, -0.3005],
                ...,
                [ 0.8843, -1.3932, -0.0506,  0.2717, -1.4921],
                [-1.0352,  1.0368,  0.4562, -0.0667, -1.3391],
                [ 0.9952, -0.6398,  0.8467, -1.6951,  1.122 ]])
```

为了使这个例子更逼真，我们添加日期-时间信息，并使用 **pandas date_range** 函数生成一系列以小时为间隔的日期-时间点（详见第 6 章和附录 C）：

```
In [30]: import pandas as pd
         t = pd.date_range(start='2014/1/1', periods=rows, freq='H')
             # set of hourly datetime objects
In [31]: t
Out[31]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2014-01-01 00:00:00, ..., 2014-07-28 07:00:00]
         Length: 5000, Freq: H, Timezone: None
```

我们需要在磁盘上打开一个新的文件对象，以写入数据：

```
In [32]: csv_file = open(path + 'data.csv', 'w')  # open file for writing
```

CSV 文件的第一行通常包含保存在文件中的每个数据列的名称，所以我们首先写入它：

```
In [33]: header = 'date,no1,no2,no3,no4,no5\n'
         csv_file.write(header)
```

然后，逐行写入实际数据，将日期-时间信息与（伪）随机数合并：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [34]: for t_, (no1, no2, no3, no4, no5) in zip(t, a):
             s = '%s,%f,%f,%f,%f,%f\n' % (t_, no1, no2, no3, no4, no5)
             csv_file.write(s)
         csv_file.close()
In [35]: ll $path*
Out[35]: -rw-r--r-- 1 root 337664 28. Sep 15:16 /flash/data/data.csv
```

另一种方法也很类似。首先，打开现存的 **CSV** 文件。其次，用**文件对象**的 **readline** 方法逐行读入其内容：

```
In [36]: csv_file = open(path + 'data.csv', 'r')  # open file for reading
In [37]: for i in range(5):
             print csv_file.readline(),
Out[37]: date,no1,no2,no3,no4,no5
         2014-01-01 00:00:00,1.381035,-1.123613,1.062245,-1.399746,-0.737369
         2014-01-01 01:00:00,0.149965,0.966987,1.839130,0.563322,0.056906
         2014-01-01 02:00:00,-0.950360,0.477881,1.863646,-1.915203,-0.300522
         2014-01-01 03:00:00,-0.503429,-0.895489,-0.240227,-0.327176,0.123498
```

也可以使用 **readlines** 方法一次读入所有内容：

```
In [38]: csv_file = open(path + 'data.csv', 'r')
         content = csv_file.readlines()
         for line in content[:5]:
             print line,
Out[38]: date,no1,no2,no3,no4,no5
         2014-01-01 00:00:00,1.381035,-1.123613,1.062245,-1.399746,-0.737369
         2014-01-01 01:00:00,0.149965,0.966987,1.839130,0.563322,0.056906
         2014-01-01 02:00:00,-0.950360,0.477881,1.863646,-1.915203,-0.300522
         2014-01-01 03:00:00,-0.503429,-0.895489,-0.240227,-0.327176,0.123498
```

最后，以某些关闭操作完成本例：

```
In [39]: csv_file.close()
         !rm -f $path*
```

7.1.3 SQL 数据库

Python 可以使用任何类型的 **SQL** 数据库，通常也可以使用 **NoSQL** 数据库。**Python** 默认自带 **SQLite3** 数据库（http://www.sqlite.org）。利用这个数据库，很容易说明 **Python** 处理 **SQL** 数据库的方式[1]：

```
In [40]: import sqlite3 as sq3
```

SQL 查询用**字符串对象**表示。当然，语法、数据类型等取决于使用的数据库：

```
In [41]: query = 'CREATE TABLE numbs (Date date, No1 real, No2 real)'
```

打开一个数据库连接，本例中，我们在磁盘上生成一个新的数据库文件：

```
In [42]: con = sq3.connect(path + 'numbs.db')
```

1 数据库领域中另一个“一等公民”是 MySQL，Python 和它的集成也非常好。虽然许多 Web 项目在所谓 LAMP 栈（Linux、Apache Web 服务器、MySQL、PHP）基础上实现，但是在许多技术栈中用 Python 代替 PHP。可用数据库连接器的概述参见 https://wiki.python.org/moin/DatabaseInterfaces。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

然后，使用 **execute** 方法执行查询语句，创建一个表：

```
In [43]: con.execute(query)
Out[43]: <sqlite3.Cursor at 0xb8a4490>
```

调用 **commit** 方法使查询生效：

```
In [44]: con.commit()
```

现在我们有了一个数据库和一个表，可以在表中填入数据。每个数据行由日期-时间信息和两个**浮点**数组成：

```
In [45]: import datetime as dt
```

可以用单独的 **SQL** 语句写入单一数据行，如：

```
In [46]: con.execute('INSERT INTO numbs VALUES(?, ?, ?)',
                     (dt.datetime.now(), 0.12, 7.3))
Out[46]: <sqlite3.Cursor at 0xb8a4570>
```

但是，通常必须（或者应该）批量写入较大的数据集：

```
In [47]: data = np.random.standard_normal((10000, 2)).round(5)
In [48]: for row in data:
             con.execute('INSERT INTO numbs VALUES(?, ?, ?)',
                         (dt.datetime.now(), row[0], row[1]))
         con.commit()
```

还有一个 **executemany** 方法，因为我们已经将当前日期-时间信息与伪随机数据集组合，所以在此不能使用这个方法。但是可以使用 **fetchmany** 一次从数据库中读一定数量的行：

```
In [49]: con.execute('SELECT * FROM numbs').fetchmany(10)
Out[49]: [(u'2014-09-28 15:16:19.486021', 0.12, 7.3),
          (u'2014-09-28 15:16:19.762476', 0.30736, -0.21114),
          (u'2014-09-28 15:16:19.762640', 0.95078, 0.50106),
          (u'2014-09-28 15:16:19.762702', 0.95896, 0.15812),
          (u'2014-09-28 15:16:19.762774', -0.42919, -1.45132),
          (u'2014-09-28 15:16:19.762825', -0.99502, -0.91755),
          (u'2014-09-28 15:16:19.762862', 0.25416, -0.85317),
          (u'2014-09-28 15:16:19.762890', -0.55879, -0.36144),
          (u'2014-09-28 15:16:19.762918', -1.61041, -1.29589),
          (u'2014-09-28 15:16:19.762945', -2.04225, 0.43446)]
```

我们也可以一次只读入一个数据行：

```
In [50]: pointer = con.execute('SELECT * FROM numbs')
In [51]: for i in range(3):
             print pointer.fetchone()
Out[51]: (u'2014-09-28 15:16:19.486021', 0.12, 7.3)
         (u'2014-09-28 15:16:19.762476', 0.30736, -0.21114)
         (u'2014-09-28 15:16:19.762640', 0.95078, 0.50106)
In [52]: con.close()
         !rm -f $path*
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

SQL 数据库是相当广泛的主题；确实，本章无法涵盖其广度和复杂度，只能提供一些基本信息：

1. Python 和几乎所有数据库技术都能很好地集成。
2. 基本 SQL 语法主要由使用的数据库决定；剩下的都是真正的 Python 风格。

7.1.4 读写 NumPy 数组

NumPy 本身有以便利、高性能的方式写入和读取 ndarray 对象的函数。这在某些情况下节省了工作量，例如必须将 NumPy dtype 转换为特定数据库类型（例如 SQLite3）时。为了说明 NumPy 有时候是基于 SQL 方法的高效替代品，我们重复前面的例子，这次仅使用 NumPy：

```
In [53]: import numpy as np
```

我们使用 NumPy 的 arange 函数代替 pandas，生成保存日期时间（datetime）对象的数组对象[1]：

```
In [54]: dtimes = np.arange('2015-01-01 10:00:00', '2021-12-31 22:00:00',
                            dtype='datetime64[m]')  # minute intervals
         len(dtimes)
Out[54]: 3681360
```

SQL 数据库中的表是 NumPy 中的结构数组。我们使用特殊的 dtype 对象镜像前面的 SQL 表：

```
In [55]: dty = np.dtype([('Date', 'datetime64[m]'), ('No1', 'f'), ('No2', 'f')])
         data = np.zeros(len(dtimes), dtype=dty)
```

我们用日期（dates）对象填充 Date 列：

```
In [56]: data['Date'] = dtimes
```

其他两列和以前一样用伪随机数填充：

```
In [57]: a = np.random.standard_normal((len(dtimes), 2)).round(5)
         data['No1'] = a[:, 0]
         data['No2'] = a[:, 1]
```

ndarray 对象的保存经过了高度优化，因此相当快速。在磁盘（这里使用 SSD）保存近 60MB 的数据只花了不到 0.1 秒：

```
In [58]: %time np.save(path + 'array', data)  # suffix .npy is added
Out[58]: CPU times: user 0 ns, sys: 77 ms, total: 77 ms
         Wall time: 77.1 ms
In [59]: ll $path*
Out[59]: -rw-r--r-- 1 root 58901888 28. Sep 15:16 /flash/data/array.npy
```

读取数据更快：

1 参见 http://docs.scipy.org/doc/numpy/reference/arrays.datetime.html。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [60]: %time np.load(path + 'array.npy')
Out[60]: CPU times: user 10 ms, sys: 29 ms, total: 39 ms
         Wall time: 37.8 ms

         array([ (datetime.datetime(2015, 1, 1, 9, 0), -1.4985100030899048,
         0.9664400219917297),
                (datetime.datetime(2015, 1, 1, 9, 1), -0.2501699924468994,
         -0.9184499979019165),
                (datetime.datetime(2015, 1, 1, 9, 2), 1.2026900053024292,
         0.49570000171661377),
                ...,
                (datetime.datetime(2021, 12, 31, 20, 57), 0.8927800059318542,
         -1.0334899425506592),
                (datetime.datetime(2021, 12, 31, 20, 58), 1.0062999725341797,
         -1.3476499915122986),
                (datetime.datetime(2021, 12, 31, 20, 59), -0.08011999726295471,
         0.4992400109767914)],
               dtype=[('Date', '<M8[m]'), ('No1', '<f4'), ('No2', '<f4')])
```

60MB 的数据集不算大。因此，我们尝试一个更大的 **ndarray** 对象：

```
In [61]: data = np.random.standard_normal((10000, 6000))
In [62]: %time np.save(path + 'array', data)
Out[62]: CPU times: user 0 ns, sys: 631 ms, total: 631 ms
         Wall time: 633 ms
In [63]: ll $path*
Out[63]: -rw-r--r-- 1 root 480000080 28. Sep 15:16 /flash/data/array.npy
```

在这个例子中，磁盘上的文件大约有 480MB，写入的时间不到 1 秒。这说明写入磁盘主要受到硬件的限制，因为 480MB/秒大约相当于本书编著时较好的 SSD 广告中声称的写入速度（512MB/S）。从磁盘读取文件/对象更快（注意，此时缓存技术可能也起到一定的作用）：

```
In [64]: %time np.load(path + 'array.npy')
Out[64]: CPU times: user 2 ms, sys: 216 ms, total: 218 ms
         Wall time: 216 ms

         array([[ 0.10989742, -0.48626177, -0.60849881, ..., -0.99051776,
                  0.88124291, -1.34261656],
                [-0.42301145,  0.29831708,  1.29729826, ..., -0.73426192,
                 -0.13484905,  0.91787421],
                [ 0.12322789, -0.28728811,  0.85956891, ...,  1.47888978,
                 -1.12452641, -0.528133  ],
                ...,
                [ 0.06507559, -0.37130379,  1.35427048, ..., -1.4457718 ,
                  0.49509821,  0.0738847 ],
                [ 1.76525714, -0.07876135, -2.94133788, ..., -0.62581084,
                  0.0933164 ,  1.55788205],
                [-1.18439949, -0.73210571, -0.45845113, ...,  0.0528656 ,
                 -0.39526633, -0.5964333 ]])
In [65]: data = 0.0
         !rm -f $path*
```

在任何情况下都可以预期，这种形式的数据存储和检索远快于 **SQL** 数据库或者使用标

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

准 **pickle** 库的序列化。当然，使用这种方法没有 **SQL** 数据库的功能性，但是后面几个小节介绍的 **PyTables** 对此有所帮助。

7.2 Pandas 的 I/O

pandas 库的主要优势之一是可以原生读取和写入不同数据格式，包括：

1. CSV（逗号分隔值）
2. SQL（结构化查询语言）
3. XLS/XSLX（Microsoft Excel 文件）
4. JSON（JavaScript 对象标记法）
5. HTML（超文本标记语言）

表 7-1 列出了所有支持格式和对应的 **pandas** 导入和导出函数/方法。导入函数使用的参数在表 6-6 中列出和说明（根据不同函数，可能应用其他约定）。

表 7-1 DataFrame 函数参数

格式	输入	输出	备注
CSV	read_csv	to_csv	文本文件
XLS/XLSX	read_excel	to_excel	电子表格
HDF	read_hdf	to_hdf	HDF5 数据库
SQL	read_sql	to_sql	SQL 表
JSON	read_json	to_json	JavaScript 对象标记
MSGPACK	read_msgpack	to_msgpack	可移植二进制格式
HTML	read_html	to_html	HTML 代码
GBQ	read_gbq	to_gbq	Google Big Query 格式
DTA	read_stata	to_stata	格式 104, 105, 108, 113-115, 117
任意	read_clipboard	to_clipboard	例如，从 HTML 页面
任意	read_pickle	to_pickle	（结构化）Python 对象

我们的测试用例仍然是较大的一组浮点数：

```
In [66]: import numpy as np
         import pandas as pd
         data = np.random.standard_normal((1000000, 5)).round(5)
                # sample data set
In [67]: filename = path + 'numbs'
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

为此，我们还将再次访问 **SQLite3**，比较使用 **pandas** 的替代方法的性能。

7.2.1 SQL 数据库

现在，下面关于 **SQLite3** 的代码应该是很熟悉的了：

```
In [68]: import sqlite3 as sq3
In [69]: query = 'CREATE TABLE numbers (No1 real, No2 real,\
                  No3 real, No4 real, No5 real)'
In [70]: con = sq3.Connection(filename + '.db')
In [71]: con.execute(query)
Out[71]: <sqlite3.Cursor at 0x9d59c00>
```

这次，可以应用 **executemany**，因为我们从单个 **ndarray** 对象写入：

```
In [72]: %%time
         con.executemany('INSERT INTO numbers VALUES (?, ?, ?, ?, ?)', data)
         con.commit()
Out[72]: CPU times: user 13.9 s, sys: 229 ms, total: 14.2 s
         Wall time: 14.9 s
In [73]: ll $path*
Out[73]: -rw-r--r-- 1 root 54446080 28. Sep 15:16 /flash/data/numbs.db
```

写入 100 万行的整个数据集要花上一段时间。将整个表格读入一个**列表**对象则要快得多：

```
In [74]: %%time
         temp = con.execute('SELECT * FROM numbers').fetchall()
         print temp[:2]
         temp = 0.0
Out[74]: [(-1.67378, -0.58292, -1.10616, 1.14929, -0.0393), (1.38006, 0.82665, 0
         .34168, -1.1676, -0.53274)]
         CPU times: user 1.54 s, sys: 138 ms, total: 1.68 s
         Wall time: 1.68 s
```

直接将 **SQL** 查询结果读入 **NumPy ndarray** 很容易实现。相应地，你也很容易绘制这种查询结果的图表，代码如下，输出见图 7-1：

```
In [75]: %%time
         query = 'SELECT * FROM numbers WHERE No1 > 0 AND No2 < 0'
         res = np.array(con.execute(query).fetchall()).round(3)
Out[75]: CPU times: user 766 ms, sys: 34 ms, total: 800 ms
         Wall time: 799 ms
In [76]: res = res[::100]  # every 100th result
         import matplotlib.pyplot as plt
         %matplotlib inline
         plt.plot(res[:, 0], res[:, 1], 'ro')
         plt.grid(True); plt.xlim(-0.5, 4.5); plt.ylim(-4.5, 0.5)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

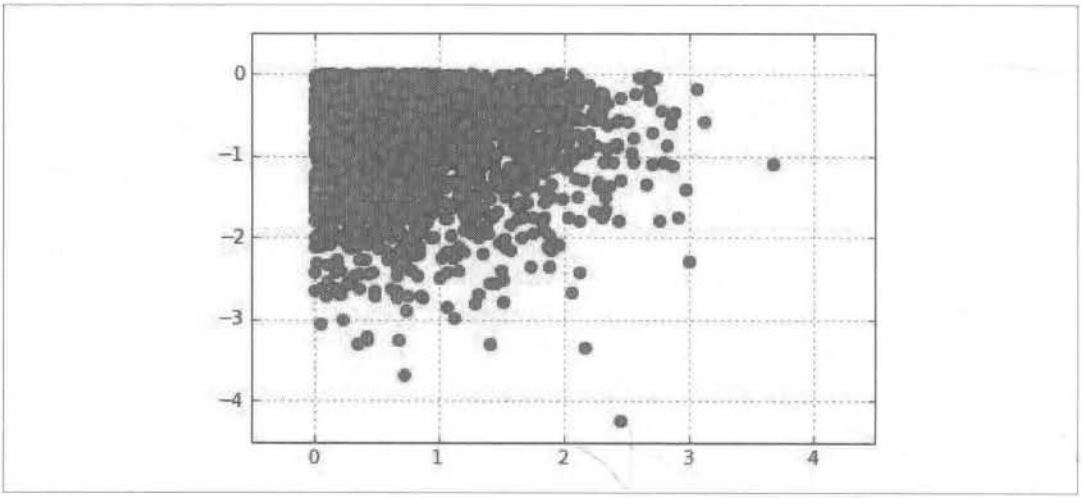


图 7-1　查询结果图表

7.2.2　从 SQL 到 pandas

用 **pandas** 读取整个表或者查询结果通常更为高效。在可以将整个表读入内存时，分析查询的执行通常比使用基于磁盘的 **SQL** 方法快得多。**pandas.io.sql** 子库包含处理 **SQL** 数据库中所保存数据的函数：

```
In [77]: import pandas.io.sql as pds
```

用 **pandas** 读取整个表花费的总时间和读入 **NumPy ndarray** 对象的时间大致相同。瓶颈同样在 **SQL** 数据库：

```
In [78]: %time data = pds.read_sql('SELECT * FROM numbers', con)
Out[78]: CPU times: user 2.16 s, sys: 60 ms, total: 2.22 s
         Wall time: 2.23 s
In [79]: data.head()
Out[79]:        No1      No2      No3      No4      No5
         0 -1.67378 -0.58292 -1.10616  1.14929 -0.03930
         1  1.38006  0.82665  0.34168 -1.16760 -0.53274
         2  0.79329  0.11947  2.06403 -0.36208  1.77442
         3 -0.33507 -0.00715 -1.01193  0.23157  1.30225
         4 -0.35292  0.67483  1.59507 -1.21263  0.14745

         [5 rows x 5 columns]
```

数据现在在内存中，可以进行更快的分析。在 **SQLite3** 中需要几秒钟才能完成的 **SQL** 查询，**pandas** 在内存中只需要不到 0.1 秒：

```
In [80]: %time data[(data['No1'] > 0) & (data['No2'] < 0)].head()
Out[80]: CPU times: user 50 ms, sys: 0 ns, total: 50 ms
         Wall time: 49.9 ms
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
          No1       No2       No3       No4       No5
6    1.17749 -1.13017 -0.24176 -0.64047  1.58002
8    0.18625 -0.99949  2.29854  0.91816 -0.92661
9    1.09481 -0.26301  1.11341  0.68716 -0.71524
18   0.31836 -0.33039 -1.50109  0.52961  0.96595
20   0.40261 -0.45917  0.37339 -1.09515  0.23972

[5 rows x 5 columns]
```

pandas 能够控制更为复杂的查询，但是不能替换结构复杂的关系型数据结构。下一个查询的结果如图 7-2 所示：

```
In [81]: %%time
         res = data[['No1', 'No2']][((data['No1'] > 0.5) | (data['No1'] < -0.5))
                                    & ((data['No2'] < -1) | (data['No2'] > 1))]
Out[81]: CPU times: user 49 ms, sys: 0 ns, total: 49 ms
         Wall time: 48.7 ms
In [82]: plt.plot(res.No1, res.No2, 'ro')
         plt.grid(True); plt.axis('tight')
```

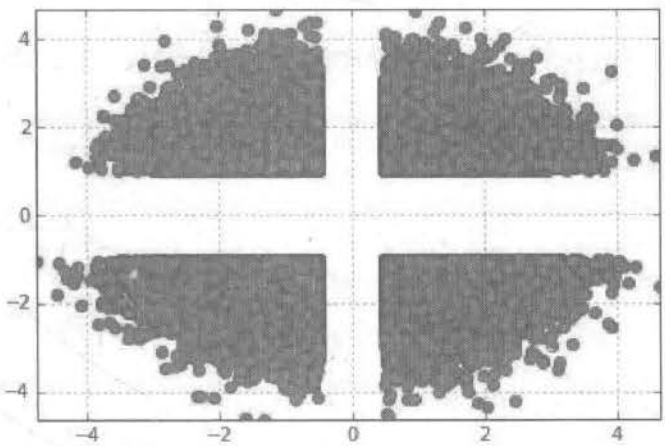


图 7-2　复杂查询结果的散点图

正如预期，只要 **pandas** 能够复制对应的 **SQL** 语句，使用 **pandas** 的内存分析能力可以大大加速。这不仅是使用 **pandas** 的优势，而是 **pandas** 与 **PyTables**（下一小节的主题）紧密集成实现的。在此，知道这种组合能够大大加速 I/O 操作就足够了。下面的例子就能说明这一点：

```
In [83]: h5s = pd.HDFStore(filename + '.h5s', 'w')
In [84]: %time h5s['data'] = data
Out[84]: CPU times: user 43 ms, sys: 60 ms, total: 103 ms
         Wall time: 161 ms
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [85]: h5s
Out[85]: <class 'pandas.io.pytables.HDFStore'>
         File path: /flash/data/numbs.h5s
         /data            frame        (shape->[1000000,5])
In [86]: h5s.close()
```

包含来自原始 SQL 表的整个 **DataFrame** 写入的时间不到 1 秒。正如预期的那样，读入更快：

```
In [87]: %%time
         h5s = pd.HDFStore(filename + '.h5s', 'r')
         temp = h5s['data']
         h5s.close()
Out[87]: CPU times: user 13 ms, sys: 22 ms, total: 35 ms
         Wall time: 32.7 ms
```

简单地检查两个数据集是否确实相同：

```
In [88]: np.allclose(np.array(temp), np.array(data))
Out[88]: True
In [89]: temp = 0.0
```

查看现在磁盘上的两个文件，可以看到 HDF5 格式消耗的磁盘空间较小：

```
In [90]: ll $path*
Out[90]: -rw-r--r-- 1 root 54446080 28. Sep 15:16 /flash/data/numbs.db
         -rw-r--r-- 1 root 48007368 28. Sep 15:16 /flash/data/numbs.h5s
```

总结起来，我们可以对模拟数据集做出如下陈述，该数据集的大小约为 50MB：

1. 写入 **SQLite3** 数据花费数秒，而使用 **pandas** 花费不到 1 秒。
2. 从 **SQL** 数据库读取数据花费数秒，而 **pandas** 花费不到 0.1 秒。

7.2.3 CSV 文件数据

CSV 格式是最广为使用的数据交换格式之一。虽然它没有真正标准化，但是可以由任何平台和大部分数据和金融分析应用程序处理。前一小节用标准 **Python** 功能循序渐进地说明如何写入和读取 CSV 文件中的数据（参见 7.1.2 小节）。**pandas** 使这整个规程变得更加方便，代码更加简洁，执行通常也更快：

```
In [91]: %time data.to_csv(filename + '.csv')
Out[91]: CPU times: user 5.55 s, sys: 137 ms, total: 5.69 s
         Wall time: 5.87 s
```

读取现在保存在 CSV 文件中的数据并绘制图表由 **read_csv** 函数实现（结果参见图 7-3）：

```
In [92]: %%time
         pd.read_csv(filename + '.csv')[['No1', 'No2',
                                         'No3', 'No4']].hist(bins=20)
Out[92]: CPU times: user 1.72 s, sys: 54 ms, total: 1.77 s
         Wall time: 1.78 s
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

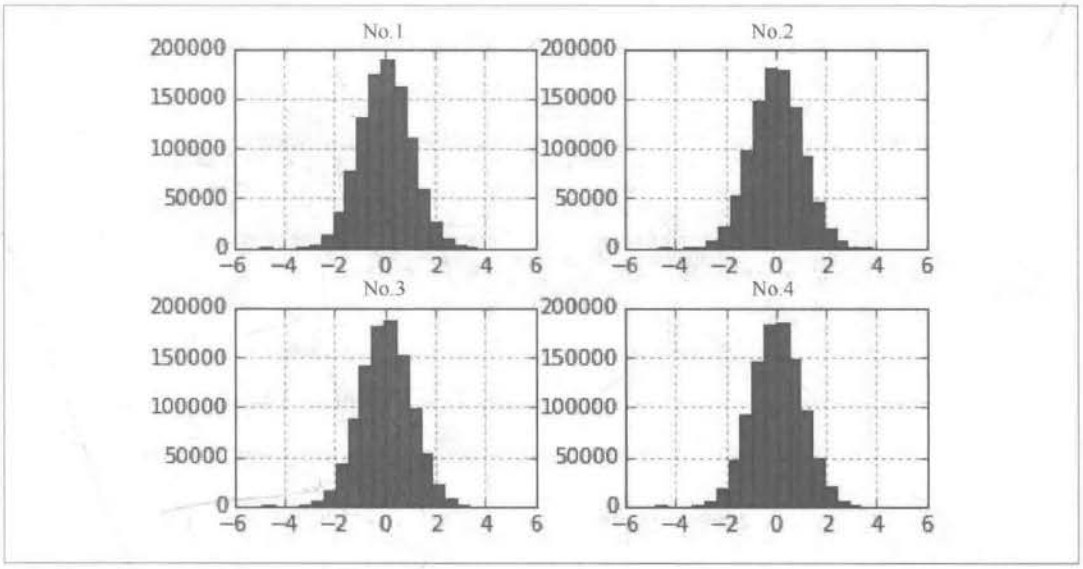


图 7-3　4 个数据集的直方图

7.2.4　Excel 文件数据

使用 Excel 数据表是后续章节的主题，但是我们要简短地阐述 **pandas** 如何以 Excel 格式写入数据，以及从 Excel 数据表读取数据。在例子中，我们将数据集限制在 10 万行：

```
In [93]: %time data[:100000].to_excel(filename + '.xlsx')
Out[93]: CPU times: user 27.5 s, sys: 131 ms, total: 27.6 s
         Wall time: 27.7 s
```

用这个小的数据子集生成 Excel 数据表需要花费一段时间。这表现了数据表结构带来的开销。读取数据（和绘制图表）是更快的过程（参见图 7-4）：

```
In [94]: %time pd.read_excel(filename + '.xlsx', 'Sheet1').cumsum().plot()
Out[94]: CPU times: user 12.9 s, sys: 6 ms, total: 12.9 s
         Wall time: 12.9 s
```

对生成的文件进行检查发现，**DataFrame** 与 HDFStore 相结合是最为紧凑的备选方法（使用本章后面介绍的压缩可以进一步加强这一优势）。同样的数据量如果用 CSV 文件（即文本文件）存储，文件尺寸会更大。这是使用 CSV 文件的性能较低的原因之一，另一个原因是它们“只是”一般的文本文件：

```
In [95]: ll $path*
Out[95]: -rw-r--r-- 1 root 48831681 28. Sep 15:17 /flash/data/numbs.csv
         -rw-r--r-- 1 root 54446080 28. Sep 15:16 /flash/data/numbs.db
         -rw-r--r-- 1 root 48007368 28. Sep 15:16 /flash/data/numbs.h5s
         -rw-r--r-- 1 root  4311424 28. Sep 15:17 /flash/data/numbs.xlsx
In [96]: rm -f $path*
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

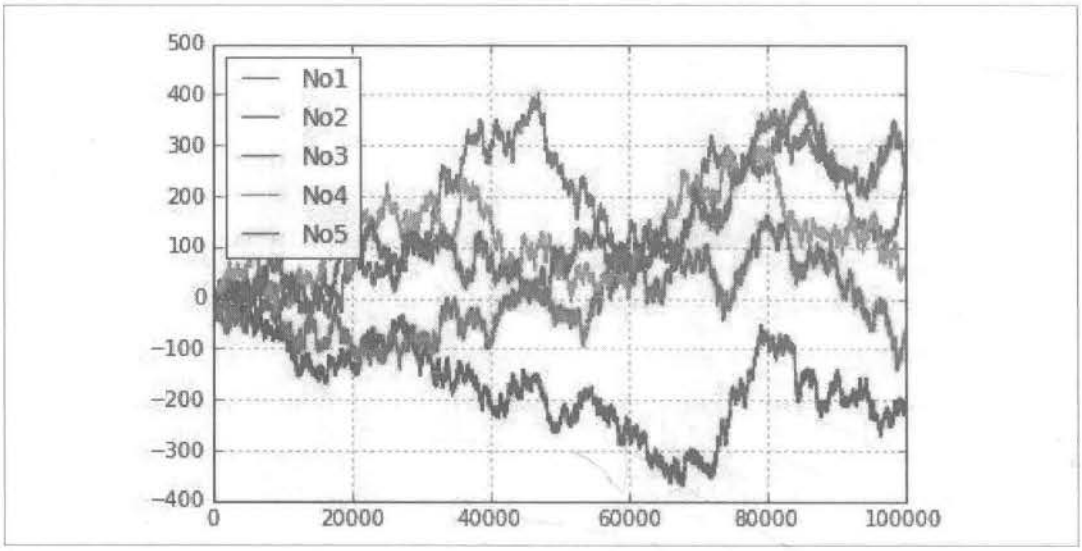


图 7-4　来自 Excel 文件的随机数据路径

7.3　PyTables 的快速 I/O

PyTables 是 **Python** 与 HDF5 数据库/文件标准的结合（http://www.hdfgroup.org）。它专门为优化 I/O 操作的性能、最大限度地利用可用硬件而设计。该库的导入名称为 **tables**。在内存中分析方面，**PyTables** 与 **pandas** 类似，并不是用于代替 **SQL** 数据库的，而是引入某些功能，进一步弥补不足。例如，**PyTables** 数据库可以有许多表，且支持压缩和索引，以及表的重要查询。此外，它可以高效地存储 **NumPy** 数组，并且具有自己独特的类数组数据结构。

我们首先导入库：

```
In [97]: import numpy as np
         import tables as tb
         import datetime as dt
         import matplotlib.pyplot as plt
         %matplotlib inline
```

7.3.1　使用表

PyTable 提供基于文件的数据库格式：

```
In [98]: filename = path + 'tab.h5'
         h5 = tb.open_file(filename, 'w')
```

为了举例，我们生成一个具有 200 万行数据的表：

```
In [99]: rows = 2000000
```

这个表本身有一个 **datetime** 列，两个 **int** 列和两个 **float** 列：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [100]: row_des = {
              'Date': tb.StringCol(26, pos=1),
              'No1': tb.IntCol(pos=2),
              'No2': tb.IntCol(pos=3),
              'No3': tb.Float64Col(pos=4),
              'No4': tb.Float64Col(pos=5)
              }
```

创建表格时，我们选择无压缩。后续的例子将加入压缩：

```
In [101]: filters = tb.Filters(complevel=0)  # no compression
          tab = h5.create_table('/', 'ints_floats', row_des,
                                title='Integers and Floats',
                                expectedrows=rows, filters=filters)

In [102]: tab

Out[102]: /ints_floats (Table(0,)) 'Integers and Floats'
            description := {
            "Date": StringCol(itemsize=26, shape=(), dflt=", pos=0),
            "No1": Int32Col(shape=(), dflt=0, pos=1),
            "No2": Int32Col(shape=(), dflt=0, pos=2),
            "No3": Float64Col(shape=(), dflt=0.0, pos=3),
            "No4": Float64Col(shape=(), dflt=0.0, pos=4)}
            byteorder := 'little'
            chunkshape := (2621,)

In [103]: pointer = tab.row
```

现在我们生成样本数据：

```
In [104]: ran_int = np.random.randint(0, 10000, size=(rows, 2))
          ran_flo = np.random.standard_normal((rows, 2)).round(5)
```

样本数据集逐行写入表：

```
In [105]: %%time
          for i in range(rows):
              pointer['Date'] = dt.datetime.now()
              pointer['No1'] = ran_int[i, 0]
              pointer['No2'] = ran_int[i, 1]
              pointer['No3'] = ran_flo[i, 0]
              pointer['No4'] = ran_flo[i, 1]
              pointer.append()
                # this appends the data and
                # moves the pointer one row forward
          tab.flush()

Out[105]: CPU times: user 15.7 s, sys: 3.53 s, total: 19.2 s
          Wall time: 19.4 s
```

一定要记住提交更改。**SQLite3** 数据库中 **commit** 命令的作用，在 PyTables 中由 **flush** 方法实现。我们现在可以检查磁盘上的数据，首先从逻辑上通过 **Table** 对象，然后从物理上通过文件信息进行：

```
In [106]: tab

Out[106]: /ints_floats (Table(2000000,)) 'Integers and Floats'
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            description := {
            "Date": StringCol(itemsize=26, shape=(), dflt=", pos=0),
            "No1": Int32Col(shape=(), dflt=0, pos=1),
            "No2": Int32Col(shape=(), dflt=0, pos=2),
            "No3": Float64Col(shape=(), dflt=0.0, pos=3),
            "No4": Float64Col(shape=(), dflt=0.0, pos=4)}
            byteorder := 'little'
            chunkshape := (2621,)
In [107]: ll $path*
Out[107]: -rw-r--r-- 1 root 100156256 28. Sep 15:18 /flash/data/tab.h5
```

使用 **NumPy** 结构数组，可以更高性能、更 **Python** 风格的方式实现相同结果：

```
In [108]: dty = np.dtype([('Date', 'S26'), ('No1', '<i4'), ('No2', '<i4'),
                                            ('No3', '<f8'), ('No4', '<f8')])
          sarray = np.zeros(len(ran_int), dtype=dty)
In [109]: sarray
Out[109]: array([(", 0, 0, 0.0, 0.0), (", 0, 0, 0.0, 0.0),
          (", 0, 0, 0.0, 0.0),
                ..., (", 0, 0, 0.0, 0.0), (", 0, 0, 0.0, 0.0),
                (", 0, 0, 0.0, 0.0)],
               dtype=[('Date', 'S26'), ('No1', '<i4'), ('No2', '<i4'), ('No3',
          '<f8'), ('No4', '<f8')])
In [110]: %%time
          sarray['Date'] = dt.datetime.now()
          sarray['No1'] = ran_int[:, 0]
          sarray['No2'] = ran_int[:, 1]
          sarray['No3'] = ran_flo[:, 0]
          sarray['No4'] = ran_flo[:, 1]
Out[110]: CPU times: user 113 ms, sys: 18 ms, total: 131 ms
          Wall time: 131 ms
```

整个数据集现在都保存在结构数组中，表的创建归结为如下几行代码。注意，行描述不再需要；**PyTables** 使用 **NumPy dtype** 代替：

```
In [111]: %%time
          h5.create_table('/', 'ints_floats_from_array', sarray,
                                  title='Integers and Floats',
                                  expectedrows=rows, filters=filters)
Out[111]: CPU times: user 38 ms, sys: 117 ms, total: 155 ms
          Wall time: 154 ms

          /ints_floats_from_array (Table(2000000,)) 'Integers and Floats'
            description := {
            "Date": StringCol(itemsize=26, shape=(), dflt=", pos=0),
            "No1": Int32Col(shape=(), dflt=0, pos=1),
            "No2": Int32Col(shape=(), dflt=0, pos=2),
            "No3": Float64Col(shape=(), dflt=0.0, pos=3),
            "No4": Float64Col(shape=(), dflt=0.0, pos=4)}
            byteorder := 'little'
            chunkshape := (2621,)
```

这种方法的速度比前一种方法快了一个数量级，以更少的代码实现了相同的结果：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [112]: h5
Out[112]: File(filename=/flash/data/tab.h5, title=u", mode='w', root_uep='/',
          filters=Filters(complevel=0, shuffle=False, fletcher32=False,
          least_significant_digit=None))
          / (RootGroup) u"
          /ints_floats (Table(2000000,)) 'Integers and Floats'
            description := {
            "Date": StringCol(itemsize=26, shape=(), dflt=", pos=0),
            "No1": Int32Col(shape=(), dflt=0, pos=1),
            "No2": Int32Col(shape=(), dflt=0, pos=2),
            "No3": Float64Col(shape=(), dflt=0.0, pos=3),
            "No4": Float64Col(shape=(), dflt=0.0, pos=4)}
            byteorder := 'little'
            chunkshape := (2621,)
          /ints_floats_from_array (Table(2000000,)) 'Integers and Floats'
            description := {
            "Date": StringCol(itemsize=26, shape=(), dflt=", pos=0),
            "No1": Int32Col(shape=(), dflt=0, pos=1),
            "No2": Int32Col(shape=(), dflt=0, pos=2),
            "No3": Float64Col(shape=(), dflt=0.0, pos=3),
            "No4": Float64Col(shape=(), dflt=0.0, pos=4)}
            byteorder := 'little'
            chunkshape := (2621,)
```

现在可以删除重复的表，因为已经不再需要：

```
In [113]: h5.remove_node('/', 'ints_floats_from_array')
```

表（**Table**）对象的表现在切片时的表现与典型的 **Python** 和 **NumPy** 对象类似，例如：

```
In [114]: tab[:3]
Out[114]: array([('2014-09-28 15:17:57.631234', 4342, 1672, -0.9293, 0.06343),
                 ('2014-09-28 15:17:57.631368', 3839, 1563, -2.02808, 0.3964),
                 ('2014-09-28 15:17:57.631383', 5100, 1326, 0.03401, 0.46742)],
                dtype=[('Date', 'S26'), ('No1', '<i4'), ('No2', '<i4'), ('No3',
          '<f8'), ('No4', '<f8')])
```

类似地，我们可以只选择单独列：

```
In [115]: tab[:4]['No4']
Out[115]: array([ 0.06343,  0.3964 ,  0.46742, -0.56959])
```

更方便和重要的是：我们可以对表或者表的子集应用 **NumPy** 通用函数：

```
In [116]: %time np.sum(tab[:]['No3'])
Out[116]: CPU times: user 31 ms, sys: 58 ms, total: 89 ms
          Wall time: 88.3 ms

          -115.34513999999896
In [117]: %time np.sum(np.sqrt(tab[:]['No1']))
Out[117]: CPU times: user 53 ms, sys: 48 ms, total: 101 ms
          Wall time: 101 ms

          133360523.08794475
```

至于绘制图表，表对象的表现也与 **ndarray** 对象很相似（参见图 7-5）：

```
In [118]: %%time
          plt.hist(tab[:]['No3'], bins=30)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
plt.grid(True)
print len(tab[:]['No3'])
```

```
Out[118]: 2000000
CPU times: user 396 ms, sys: 89 ms, total: 485 ms
Wall time: 485 ms
```

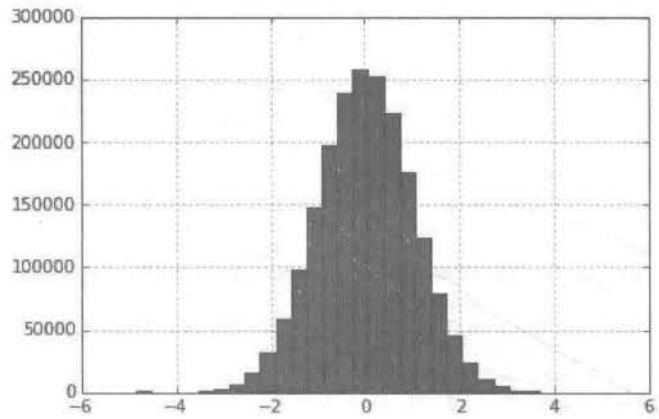


图 7-5　数据直方图

当然，我们有相当灵活的工具，通过类似典型 **SQL** 的语句查询数据，如以下的例子（结果在图 7-6 中显示，将其与基于 **pandas** 查询的图 7-2 比较）：

```
In [119]: %%time
res = np.array([(row['No3'], row['No4']) for row in
        tab.where('((No3 < -0.5) | (No3 > 0.5)) \
                & ((No4 < -1) | (No4 > 1))')][::100]
```

```
Out[119]: CPU times: user 530 ms, sys: 52 ms, total: 582 ms
Wall time: 469 ms
```

```
In [120]: plt.plot(res.T[0], res.T[1], 'ro')
plt.grid(True)
```

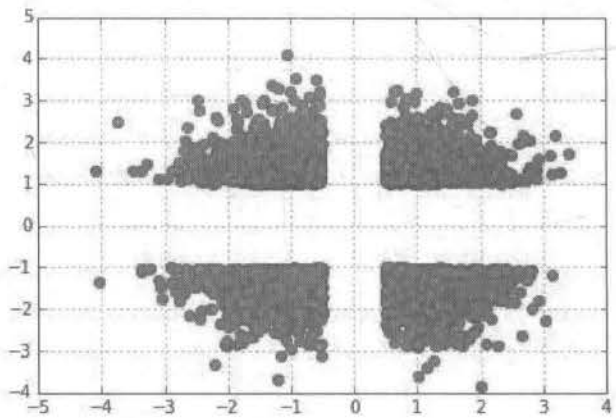


图 7-6　查询结果的散点图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

快速复杂查询

pandas 和 **PyTable** 都能处理复杂的类 SQL 查询及选择。它们都对这些操作的速度进行了优化。

正如下面的例子所示，从*语法*和*性能*的角度看，以表（**Table**）对象的形式使用 **PyTables** 中保存的数据都像使用 **NumPy** 在内存中工作一样：

```
In [121]: %%time
          values = tab.cols.No3[:]
          print "Max %18.3f" % values.max()
          print "Ave %18.3f" % values.mean()
          print "Min %18.3f" % values.min()
          print "Std %18.3f" % values.std()

Out[121]: Max              5.152
          Ave             -0.000
          Min             -5.537
          Std              1.000
          CPU times: user 44 ms, sys: 39 ms, total: 83 ms
          Wall time: 82.6 ms

In [122]: %%time
          results = [(row['No1'], row['No2']) for row in
                     tab.where('((No1 > 9800) | (No1 < 200)) \
                                & ((No2 > 4500) & (No2 < 5500))')]
          for res in results[:4]:
              print res

Out[122]: (9987, 4965)
          (9934, 5263)
          (9960, 4729)
          (130, 5023)
          CPU times: user 167 ms, sys: 37 ms, total: 204 ms
          Wall time: 118 ms

In [123]: %%time
          results = [(row['No1'], row['No2']) for row in
                     tab.where('(No1 == 1234) & (No2 > 9776)')]
          for res in results:
              print res

Out[123]: (1234, 9805)
          (1234, 9785)
          (1234, 9821)
          CPU times: user 93 ms, sys: 40 ms, total: 133 ms
          Wall time: 90.1 ms
```

7.3.2 使用压缩表

使用 **PyTables** 的主要优势之一是压缩方法。使用压缩不仅能节约磁盘空间，还能改善 I/O 操作的性能。这是如何实现的？当 I/O 成为瓶颈而 CPU 能够快速（解）压缩数据时，使用压缩对速度有正面的净效应。由于下面的例子基于最新型（在编写本书时）SSD 的 I/O 操作，没有观察到压缩的速度优势。但是，使用压缩几乎没有任何*劣势*：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [124]: filename = path + 'tab.h5c'
          h5c = tb.open_file(filename, 'w')
In [125]: filters = tb.Filters(complevel=4, complib='blosc')
In [126]: tabc = h5c.create_table('/', 'ints_floats', sarray,
                                  title='Integers and Floats',
                                 expectedrows=rows, filters=filters)
In [127]: %%time
          res = np.array([(row['No3'], row['No4']) for row in
                          tabc.where('((No3 < -0.5) | (No3 > 0.5)) \
                                     & ((No4 < -1) | (No4 > 1))')][::100]
Out[127]: CPU times: user 670 ms, sys: 41 ms, total: 711 ms
          Wall time: 602 ms
```

生成包含原始数据的表并在上面进行分析比未压缩的表稍慢。那么读入 **ndarray** 会如何呢？我们检查一下：

```
In [128]: %time arr_non = tab.read()
Out[128]: CPU times: user 13 ms, sys: 49 ms, total: 62 ms
          Wall time: 61.3 ms
In [129]: %time arr_com = tabc.read()
Out[129]: CPU times: user 161 ms, sys: 33 ms, total: 194 ms
          Wall time: 193 ms
```

这种方法花的时间确实比以前长。但是，压缩率大约为 20%，节约 80%磁盘空间，对于备份历程或者服务器之间甚至数据中心之间交换大数据集可能很重要：

```
In [130]: ll $path*
Out[130]: -rw-r--r-- 1 root 200313168 28. Sep 15:18 /flash/data/tab.h5
          -rw-r--r-- 1 root  41335178 28. Sep 15:18 /flash/data/tab.h5c
In [131]: h5c.close()
```

7.3.3 使用数组

我们已经看到，**NumPy** 内建 **ndarray** 对象的快速写入和读取能力。**PyTables** 在存储和检索 **ndarray** 对象时也相当快速高效：

```
In [132]: %%time
          arr_int = h5.create_array('/', 'integers', ran_int)
          arr_flo = h5.create_array('/', 'floats', ran_flo)
Out[132]: CPU times: user 2 ms, sys: 33 ms, total: 35 ms
          Wall time: 35 ms
```

将这些对象直接写入 **HDF5** 数据库当然比循环读取对象然后将数据逐行写入表对象快得多。对数据库的最终检查可以看到现在库中有 3 个对象——表和两个数组：

```
In [133]: h5
Out[133]: File(filename=/flash/data/tab.h5, title=u'', mode='w', root_uep='/', f
          ilters=Filters(complevel=0, shuffle=False, fletcher32=False, least_sig
          nificant_digit=None))
          / (RootGroup) u''
          /floats (Array(2000000, 2)) ''
            atom := Float64Atom(shape=(), dflt=0.0)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    maindim := 0
    flavor := 'numpy'
    byteorder := 'little'
    chunkshape := None
  /integers (Array(2000000, 2)) "
    atom := Int64Atom(shape=(), dflt=0)
    maindim := 0
    flavor := 'numpy'
    byteorder := 'little'
    chunkshape := None
  /ints_floats (Table(2000000,)) 'Integers and Floats'
    description := {
    "Date": StringCol(itemsize=26, shape=(), dflt='', pos=0),
    "No1": Int32Col(shape=(), dflt=0, pos=1),
    "No2": Int32Col(shape=(), dflt=0, pos=2),
    "No3": Float64Col(shape=(), dflt=0.0, pos=3),
    "No4": Float64Col(shape=(), dflt=0.0, pos=4)}
    byteorder := 'little'
    chunkshape := (2621,)
In [134]: ll $path*
Out[134]: -rw-r--r-- 1 root 200313168 28. Sep 15:18 /flash/data/tab.h5
          -rw-r--r-- 1 root  41335178 28. Sep 15:18 /flash/data/tab.h5c
In [135]: h5.close()
In [136]: !rm -f $path*
```

基于 HDF5 的数据存储

HDF5 数据库（文件）格式是结构化数值和金融数据的强大替代方案，例如可以用它代替关系数据库。在使用 **PyTables** 时单独访问或者结合 **pandas** 的能力，都可以得到硬件所能支持的最高 I/O 性能。

7.3.4 内存外计算

PyTables 支持内存外计算，因此可以实现不适合于内存的基于数组计算：

```
In [137]: filename = path + 'array.h5'
          h5 = tb.open_file(filename, 'w')
```

我们创建一个 **EArray** 对象，它的第一维可以扩展，而第二维固定宽度为 1000：

```
In [138]: n = 1000
          ear = h5.createEArray(h5.root, 'ear',
                                atom=tb.Float64Atom(),
                                shape=(0, n))
```

因为 EArray 对象可以扩展，所以可以块的形式填充：

```
In [139]: %%time
          rand = np.random.standard_normal((n, n))
          for i in range(750):
              ear.append(rand)
          ear.flush()
Out[139]: CPU times: user 2.42 s, sys: 7.29 s, total: 9.71 s
          Wall time: 20.6 s
```

为了从逻辑上和物理上检查生成数据的多少，我们可以检查为对象提供的元信息和磁

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

盘空间消耗：

```
In [140]: ear
Out[140]: /ear (EArray(750000, 1000)) ''
            atom := Float64Atom(shape=(), dflt=0.0)
            maindim := 0
            flavor := 'numpy'
            byteorder := 'little'
            chunkshape := (8, 1000)
In [141]: ear.size_on_disk
Out[141]: 6000000000L
```

EArray 对象有 6GB 大，为了进行内存外计算，我们需要在数据库中有一个目标 **EArray** 对象：

```
In [142]: out = h5.createEArray(h5.root, 'out',
                                atom=tb.Float64Atom(),
                                shape=(0, n))
```

PyTables 有一个特殊模块，可以高效地处理数值表达式，这个模块叫做 **Expr**，基于数值表达式库 **numexpr**（http://code.google.com/p/numexpr/）。我们希望用这个模块，在前面生成的整个 **EArray** 对象上计算公式 7-1 中的数学表达式：

公式 7-1　数学表达式示例

$$y = 3\sin(x) + \sqrt{|x|} \tag{7-1}$$

下面的代码展示了内存外计算的能力：

```
In [143]: expr = tb.Expr('3 * sin(ear) + sqrt(abs(ear))')
            # the numerical expression as a string object
          expr.setOutput(out, append_mode=True)
            # target to store results is disk-based array
In [144]: %time expr.eval()
            # evaluation of the numerical expression
            # and storage of results in disk-based array
Out[144]: CPU times: user 34.4 s, sys: 11.6 s, total: 45.9 s
          Wall time: 1min 41s
         /out (EArray(750000, 1000)) ''
           atom := Float64Atom(shape=(), dflt=0.0)
           maindim := 0
           flavor := 'numpy'
           byteorder := 'little'
           chunkshape := (8, 1000)
In [145]: out[0, :10]
Out[145]: array([-0.95979563, -1.21530335,  0.02687751,  2.88229293, -0.05596624,
                 -1.70266651, -0.58575264,  1.70317385,  3.54571202,  2.81602673
         ])
```

考虑到整个运算在内存外进行，这样的结果应该算相当快速了，尤其是在标准硬件上执行时。我们简单地将其与 **numexpr** 模块在内存中的性能做个比较（参见第 8 章）：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [146]: %time imarray = ear.read()
            # read whole array into memory
Out[146]: CPU times: user 1.26 s, sys: 4.11 s, total: 5.37 s
          Wall time: 5.39 s
In [147]: import numexpr as ne
          expr = '3 * sin(imarray) + sqrt(abs(imarray))'
In [148]: ne.set_num_threads(16)
          %time ne.evaluate(expr)[0, :10]
Out[148]: CPU times: user 24.2 s, sys: 29.1 s, total: 53.3 s
          Wall time: 3.81 s

          array([-0.95979563, -1.21530335,  0.02687751,  2.88229293, -0.05596624,
                 -1.70266651, -0.58575264,  1.70317385,  3.54571202,  2.81602673
          ])
In [149]: h5.close()
In [150]: !rm -f $path*
```

7.4 结语

基于 SQL（即关系）数据库在单个对象/表格之间存在大量关系的复杂数据结构上具有优势，这可以为某些情况下它们的性能劣于基于 **ndarray** 的纯 **NumPy** 方法或者基于 **DataFrame** 的 **pandas** 方法辩解。

不过，金融或者科学中的许多应用领域通常主要使用基于数组的数据建模方法。在这些情况下，使用原生 **NumPy I/O** 功能、结合 **NumPy** 和 **PyTables** 的能力或者通过基于 **HDF5** 存储的 **Pandas** 方法可能实现巨大的性能改进。

虽然近来的趋势是使用基于云的解决方案——云由大量基于商用硬件的计算节点组成——但是人们应该**小心考虑，特别是在金融领域**，哪一种硬件架构最适合于分析需求。Microsoft 最近的一项研究阐明了这一主题：

> 我们断言，单个“垂直扩容”服务器能够处理所有此类工作，并且在性能、成本、电源消耗和服务器密度上与群集相当或者更好。
>
> ——Appuswamy 等人（2013）

因此，参与数据分析的公司、研究机构和其他组织应该首先分析一般需要完成的特定任务，然后决定根据以下方式决定硬件/软件架构：

水平扩容

使用具有许多商用节点的群集，每个节点采用标准 CPU 和相对较少的内存。

垂直扩容

使用一个或者少数强大的服务器，具有多核 CPU、可能有一个 GPU 以及大量内存

本章介绍的内存外分析示例支持这种观点。在标准硬件上，用 **PyTables** 对数值表达式

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

进行内存外计算大约花费 1.5 分钟。相同任务在（使用 **numexpr** 库）内存中执行时花费大约 4 秒，而从磁盘读取整个数据集只需要 5 秒多的时间。这些观测值来自一台具有足够内存（在这个特例中有 64GB）和一个 SSD 硬盘的 8 核服务器。因此，垂直扩容硬件并应用不同的实现方法可能显著影响性能，下一章将更详细地介绍这个主题。

7.5 延伸阅读

本章开始和“结语”中引用的论文是很好的读物，也是思考金融分析硬件架构的很好起点。

- Appuswamy, Raja et al. (2013): “Nobody Ever Got Fired for Buying a Cluster.” Microsoft Research, Cambridge, England,http://research.microsoft.com/apps/pubs/ default.aspx?id=179615。

和往常一样，Web 提供与本章介绍的主题相关的许多有价值资源。

- 用 **pickle** 进行的 **Python** 对象序列化参见如下文档：http://docs.python.org/2/library/pickle.html。
- **SciPy** 网站上提供了 **NumPy** I/O 能力的概述：http://docs.scipy.org/doc/numpy/reference/routines.io.html。
- **pandas** 的 I/O 参见在线文档的相应章节：http://pandas.pydata.org/pandas-docs/stable/io.html。
- **PyTables** 首页提供教程和详细文档：http://www.pytables.org。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第8章

高性能的 Python

不要降低预期去屈就性能，而要提升性能满足预期。

——Ralph Marston

对于性能关键应用，总是应该检查两件事：我们使用的是正确的*实现范型*吗？使用的是正确的*高性能库*吗？许多高性能库可以用于加速 Python 代码的执行。其中，下面的库最为实用，本章将全部加以介绍（但是顺序不同）。

- Cython，用于合并 Python 和 C 语言静态编译范型。
- IPython.parallel，用于在本地或者在群集上并行执行代码/函数。
- numexpr，用于快速数值运算。
- multiprocessing，Python 内建的（本地）并行处理模块。
- Numba，用于为 CPU 动态编译 Python 代码。
- NumbaPro，用于为多核 CPU 和 GPU 动态编译 Python 代码。

本章自始至终都将比较相同算法的不同实现。为了使这一比较更容易进行，我们定义一个方便的函数，可以系统性地比较在相同或者不同数据集上执行不同函数的性能：

```
In [1]: def perf_comp_data(func_list, data_list, rep=3, number=1):
            ''' Function to compare the performance of different functions.

            Parameters
            ==========
            func_list : list
                list with function names as strings
            data_list : list
                list with data set names as strings
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    rep : int
        number of repetitions of the whole comparison
    number : int
        number of executions for every function
    '''
    from timeit import repeat
    res_list = {}
    for name in enumerate(func_list):
        stmt = name[1] + '(' + data_list[name[0]] + ')'
        setup = "from __main__ import " + name[1] + ', ' \
                                        + data_list[name[0]]
        results = repeat(stmt=stmt, setup=setup,
                         repeat=rep, number=number)
        res_list[name[1]] = sum(results) / rep
    res_sort = sorted(res_list.iteritems(),
                      key=lambda (k, v): (v, k))
    for item in res_sort:
        rel = item[1] / res_sort[0][1]
        print 'function: ' + item[0] + \
              ', av. time sec: %9.5f, ' % item[1] \
            + 'relative: %6.1f' % rel
```

8.1 Python 范型与性能

在金融学中与其他科学及数据密集学科一样，大数据集上的数值计算相当费时。举个例子，我们想要在包含 50 万个数值的数组上求取某个复杂数学表达式的值。我们选择公式 8-1 中的表达式，它的每次计算都会带来一定的计算负担。除此之外，该公式没有任何特殊的含义。

公式 8-1　数学表达式示例

$$y = \sqrt{|\cos(x)|} + \sin(2 + 3x) \tag{8-1}$$

公式 8-1 很容易转换为一个 Python 函数：

```
In [2]: from math import *
        def f(x):
            return abs(cos(x)) ** 0.5 + sin(2 + 3 * x)
```

使用 **range** 函数，我们可以高效地生成一个包含 50 万个数值的列表对象：

```
In [3]: I = 500000
        a_py = range(I)
```

下面是第一个实现，函数 f1 在整个数据集中循环，在结果列表对象中附加单独函数求值结果：

```
In [4]: def f1(a):
            res = []
            for x in a:
                res.append(f(x))
            return res
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

实现这一计算的方法不只一种，也可以使用不同的 **Python** 范型，例如*迭代子*或者 **eval** 函数，得到下面的 **f2** 和 **f3** 函数：

```
In [5]: def f2(a):
            return [f(x) for x in a]
In [6]: def f3(a):
            ex = 'abs(cos(x)) ** 0.5 + sin(2 + 3 * x)'
            return [eval(ex) for x in a]
```

当然，可以使用 NumPy 向量化技术实现同一个算法。在这种情况下，数据数组是一个 **ndarray** 对象而不是列表对象。函数实现 **f4** 没有任何循环；所有循环发生在 **NumPy** 级别而不是 **Python** 级别：

```
In [7]: import numpy as np
In [8]: a_np = np.arange(I)
In [9]: def f4(a):
            return (np.abs(np.cos(a)) ** 0.5 +
                    np.sin(2 + 3 * a))
```

然后，我们可以使用专用库 **numexpr** 求数值表达式的值。这个库内建了多线程执行支持。为了比较单线程和多线程方法的性能，我们定义两个不同函数 **f5**（单线程）和 **f6**（多线程）：

```
In [10]: import numexpr as ne
In [11]: def f5(a):
             ex = 'abs(cos(a)) ** 0.5 + sin(2 + 3 * a)'
             ne.set_num_threads(1)
             return ne.evaluate(ex)
In [12]: def f6(a):
             ex = 'abs(cos(a)) ** 0.5 + sin(2 + 3 * a)'
             ne.set_num_threads(16)
             return ne.evaluate(ex)
```

总而言之，相同的任务——在有 50 万个元素的数组上求公式 8-1 中数值表达式的值——以 6 种不同方式实现：

- 包含显式循环的标准 Python 函数；
- 包含隐含循环的迭代子方法；
- 包含隐含循环、使用 eval 的迭代子方法；
- NumPy 向量化实现；
- numexpr 单线程实现；
- numexpr 多线程实现。

我们首先检查各种实现是否交付相同的结果。我们使用 **IPython** 单元格魔法命令 **%%time** 记录总执行时间：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [13]: %%time
         r1 = f1(a_py)
         r2 = f2(a_py)
         r3 = f3(a_py)
         r4 = f4(a_np)
         r5 = f5(a_np)
         r6 = f6(a_np)
Out[13]: CPU times: user 16 s, sys: 125 ms, total: 16.1 s
         Wall time: 16 s
```

NumPy 函数 **allclose** 可以轻松地检查两个（类）**ndarray** 对象是否包含相同数据：

```
In [14]: np.allclose(r1, r2)
Out[14]: True
In [15]: np.allclose(r1, r3)
Out[15]: True
In [16]: np.allclose(r1, r4)
Out[16]: True
In [17]: np.allclose(r1, r5)
Out[17]: True
In [18]: np.allclose(r1, r6)
Out[18]: True
```

很明显，情况正是如此。当然，不同实现执行速度的对比更加有趣。为此，我们使用 **perf_comp_data** 函数，向其提供所有函数和数据集名称：

```
In [19]: func_list = ['f1', 'f2', 'f3', 'f4', 'f5', 'f6']
         data_list = ['a_py', 'a_py', 'a_py', 'a_np', 'a_np', 'a_np']
```

现在，我们已经为这场竞赛做好了准备：

```
In [20]: perf_comp_data(func_list, data_list)
Out[20]: function: f6, av. time sec:   0.00583, relative:    1.0
         function: f5, av. time sec:   0.02711, relative:    4.6
         function: f4, av. time sec:   0.06331, relative:   10.9
         function: f2, av. time sec:   0.46864, relative:   80.3
         function: f1, av. time sec:   0.59660, relative:  102.3
         function: f3, av. time sec:  15.15156, relative: 2597.2
```

胜者很明显：多线程 **numexpr** 实现 **f6**。当然，它的速度优势取决于可用核心数量。向量化 **NumPy** 版本 **f4** 慢于 **f5**。纯 **Python** 实现 **f1** 和 **f2** 比冠军慢 80 倍以上。**f3** 是最慢的版本，因为对这样大量的求值运算使用 **eval** 会造成巨大的负担。在 **numexpr** 的例子中，基于字符串的表达式计算一次之后被编译供以后使用；而使用 **Python eval** 函数，这一操作要进行 50 万次。

8.2 内存布局与性能

NumPy 可以为每个 **ndarray** 对象指定所谓的 **dtype**；例如，**np.int32** 或者 **f8**。**NumPy**

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

还允许在初始化 ndarray 对象时从两种不同的*内存布局中*选择。根据对象结构，某种布局可能比另一种布局更有优势。下面是一个例子：

```
In [21]: import numpy as np
In [22]: np.zeros((3, 3), dtype=np.float64, order='C')
Out[22]: array([[ 0.,  0.,  0.],
                [ 0.,  0.,  0.],
                [ 0.,  0.,  0.]])
```

初始化 **NumPy ndarray** 对象的方式可能（根据数组的大小）对这些数组的操作性能产生显著的影响。下面介绍 **ndarray** 对象的初始化（例如通过 **np.zeros** 或者 **np.array**）输入参数：

组成（shape）

整数（**int**）、**整数**序列或者引用另一个 **numpy.ndarray**

dtype（可选）

numpy.dtype——用于 **numpy.ndarray** 对象的 **NumPy** 特定数据类型

顺序（可选）

元素在内存中存储的顺序：**C** 表示类似 **C**（行优先），F 表示类似 **Fortran**（列优先）

考虑类似 **C**（行优先）的存储方式：

```
In [23]: c = np.array([[ 1.,  1.,  1.],
                       [ 2.,  2.,  2.],
                       [ 3.,  3.,  3.]], order='C')
```

在本例中，1，2，3 相邻存储。作为对比，考虑类似 **Fortran**（列优先）的存储方式：

```
In [24]: f = np.array([[ 1.,  1.,  1.],
                       [ 2.,  2.,  2.],
                       [ 3.,  3.,  3.]], order='F')
```

现在，数据的存储方式是 1，2，3 在每列中相邻存储。我们来看看内存布局在数组很大时可能带来什么差异：

```
In [25]: x = np.random.standard_normal((3, 1500000))
         C = np.array(x, order='C')
         F = np.array(x, order='F')
         x = 0.0
```

现在，我们在类 C 布局的数组上实施某些标准操作。首先，计算总和：

```
In [26]: %timeit C.sum(axis=0)
Out[26]: 100 loops, best of 3: 11.3 ms per loop
In [27]: %timeit C.sum(axis=1)
Out[27]: 100 loops, best of 3: 5.84 ms per loop
```

在第一个轴上计算总和比第二个轴慢了将近一倍。计算标准差得到的结果类似：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [28]: %timeit C.std(axis=0)
Out[28]: 10 loops, best of 3: 70.6 ms per loop
In [29]: %timeit C.std(axis=1)
Out[29]: 10 loops, best of 3: 32.6 ms per loop
```

为了比较，考虑类似 **Fortran** 的布局，首先是总和：

```
In [30]: %timeit F.sum(axis=0)
Out[30]: 10 loops, best of 3: 29.2 ms per loop
In [31]: %timeit F.sum(axis=1)
Out[31]: 10 loops, best of 3: 37 ms per loop
```

尽管和其他布局相比绝对更慢，但是两个轴的相对差值并不太大，现在计算标准差：

```
In [32]: %timeit F.std(axis=0)
Out[32]: 10 loops, best of 3: 107 ms per loop
In [33]: %timeit F.std(axis=1)
Out[33]: 10 loops, best of 3: 98.8 ms per loop
```

同样，与类似 **C** 的布局相比，这种布局的性能更差。两个轴之间的差距很小，但是同样不像其他布局那样显著。结果表明，一般来说，类似 **C** 的布局表现更好——这也是 **NumPy ndarray** 对象没有指定的时候默认使用这种内存布局的原因：

```
In [34]: C = 0.0; F = 0.0
```

8.3 并行计算

近年来，即使最紧凑的笔记本电脑也拥有多核处理器。而且，现代化的基于云计算服务（如 Amazon 的 EC2 或者 Microsoft 的 Azure）可以在相当低、各不相同的价格下提供高可伸缩性的并行架构。这为小企业、研究人员甚至有抱负的业余爱好者带来大规模计算能力。不过，为了利用这些服务的威力，需要合适的工具。IPython.parallel 库就是其中之一。

8.3.1 蒙特卡洛算法

期权的蒙特卡洛估值是导致高计算负担的金融算法之一。作为特例，我们选择 Black-Scholes-Merton 设置下的*欧式看涨期权价值蒙特卡洛估算函数*（参见第 3 章中的相同示例）。在这种设置下，所要估值的期权标的遵循随机微分方程式（SDE），如公式 8-2 所示。S_t 是时间 t 的标的价值；r 是一个常数——无风险短期利率；σ 是恒定瞬时波动率；Z 是布朗运动。

公式 8-2　Black-Scholes-Merton SDE

$$dS_t = rS_t dt + \sigma S_t dZ_t \qquad (8\text{-}2)$$

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

欧式看涨期权的蒙特卡洛估算函数在公式 8-3 中给出，其中 $S_T(i)$是到期日 T 时第 i 个标的的模拟价值。

公式 8-3　欧式看涨期权的蒙特卡洛估算函数

$$C_0 = e^{-rT}\frac{1}{I}\sum_I \max(S_T(i)-K,0) \qquad (8\text{-}3)$$

如果我们只允许欧式看涨期权的行权价可变，实现 Black-Scholes-Merton 设置下蒙特卡洛估值的函数如下所示：

```
In [35]: def bsm_mcs_valuation(strike):
             ''' Dynamic Black-Scholes-Merton Monte Carlo estimator
             for European calls.

             Parameters
             ==========
             strike : float
                 strike price of the option

             Results
             =======
             value : float
                 estimate for present value of call option
             '''
             import numpy as np
             S0 = 100.; T = 1.0; r = 0.05; vola = 0.2
             M = 50; I = 20000
             dt = T / M
             rand = np.random.standard_normal((M + 1, I))
             S = np.zeros((M + 1, I)); S[0] = S0
             for t in range(1, M + 1):
                 S[t] = S[t-1] * np.exp((r - 0.5 * vola ** 2) * dt
                                        + vola * np.sqrt(dt) * rand[t])
             value = (np.exp(-r * T)
                              * np.sum(np.maximum(S[-1] - strike, 0)) / I)
             return value
```

8.3.2　顺序化计算

作为基准用例，我们对不同行权价的 100 种期权进行估值。**seq_value** 函数计算蒙特卡洛估算函数，返回包含行权价和估值结果的**列表**对象：

```
In [36]: def seq_value(n):
             ''' Sequential option valuation.

             Parameters
             ==========
             n : int
                 number of option valuations/strikes
             '''
             strikes = np.linspace(80, 120, n)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            option_values = []
            for strike in strikes:
                option_values.append(bsm_mcs_valuation(strike))
            return strikes, option_values
```

```
In [37]: n = 100  # number of options to be valued
         %time strikes, option_values_seq = seq_value(n)
```

```
Out[37]: CPU times: user 11.7 s, sys: 1e+03 µs, total: 11.7 s
Wall time: 11.7 s
```

计算效率大约是每秒 8.5 种权证。图 8-1 展示了估值结果：

```
In [38]: import matplotlib.pyplot as plt
         %matplotlib inline
         plt.figure(figsize=(8, 4))
         plt.plot(strikes, option_values_seq, 'b')
         plt.plot(strikes, option_values_seq, 'r.')
         plt.grid(True)
         plt.xlabel('strikes')
         plt.ylabel('European call option values')
```

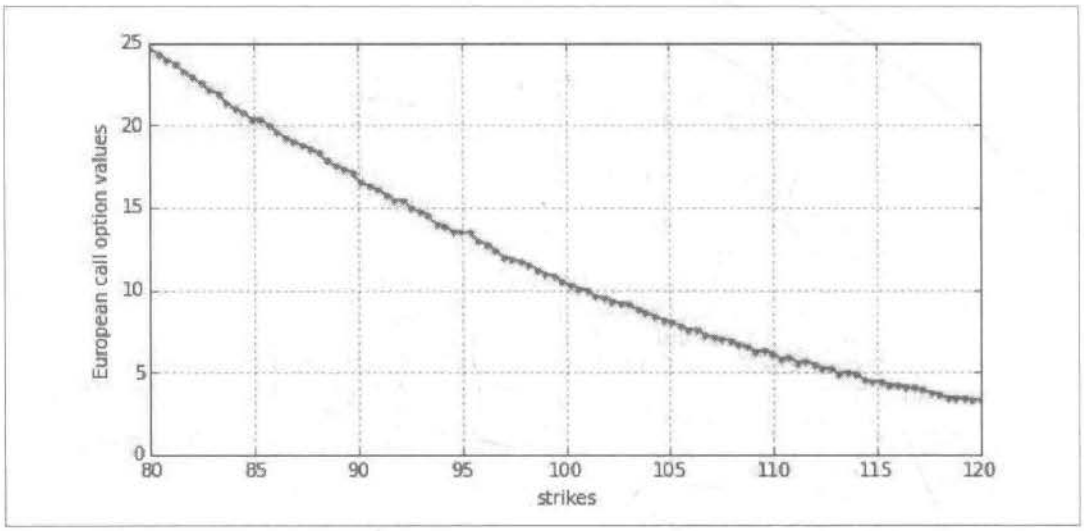


图 8-1　通过蒙特卡洛模拟估算的欧式看涨期权价值

8.3.3　并行计算

为了并行计算 100 个权证价值，我们使用 **IPython.parallel** 和一个本地“群集”。本地群集很容易通过 **IPython** Notebook 仪表盘中的 **Clusters** 选项卡启动。使用的线程数量当然取决于运行代码的机器和处理器。图 8-2 展示了启动群集的 **IPython** 页面。

IPython.parallel 需要哪个群集用于代码并行执行的相关信息。在本例中，群集配置文件保存在“default”（默认）配置文件中。此外，我们需要在群集上生成一个视图：

```
In [39]: from IPython.parallel import Client
         c = Client(profile="default")
         view = c.load_balanced_view()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 8-2 IPython 群集页面屏幕截图

实现并行期权估值的函数看起来类似于顺序化的实现：

```
In [40]: def par_value(n):
             ''' Parallel option valuation.

             Parameters
             ==========
             n : int
                 number of option valuations/strikes
             '''
             strikes = np.linspace(80, 120, n)
             option_values = []
             for strike in strikes:
                 value = view.apply_async(bsm_mcs_valuation, strike)
                 option_values.append(value)
             c.wait(option_values)
             return strikes, option_values
```

有两个主要的不同之处需要注意。首先是估值函数通过 **view.apply_sync** 异步应用到群

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

集视图，这实际上一次性初始化所有期权的并行估值。当然，不是所有期权都可以并行估值，因为（一般）没有足够的核心/线程可用。因此，我们必须等待，直到队列完全结束；这由 **Client** 对象 **c** 的 **wait** 方法实现。当所有结果可用时，该函数返回，和以前一样，**列表**对象分别包含行权价和估值结果。

并行估值函数的执行效率在理想的情况下和可用核心（线程）数量呈线性关系。例如，拥有 *8* 个可用核心（线程）最多可以将执行时间降低到顺序化计算所需时间的 *1/8*：

```
In [41]: %time strikes, option_values_obj = par_value(n)
Out[41]: CPU times: user 415 ms, sys: 30 ms, total: 445 ms
         Wall time: 1.88 s
```

并行执行不会直接返回期权价值；而是返回更复杂的结果对象：

```
In [42]: option_values_obj[0].metadata
Out[42]: {'after': [],
          'completed': datetime.datetime(2014, 9, 28, 16, 6, 54, 93979),
          'data': {},
          'engine_id': 5,
          'engine_uuid': u'6b64aebb-39d5-49aa-9466-e6ab37d3b2c9',
          'follow': [],
          'msg_id': u'c7a44c22-b4bd-46d7-ba5e-34690f178fa9',
          'outputs': [],
          'outputs_ready': True,
          'pyerr': None,
          'pyin': None,
          'pyout': None,
          'received': datetime.datetime(2014, 9, 28, 16, 6, 54, 97195),
          'started': datetime.datetime(2014, 9, 28, 16, 6, 53, 921633),
          'status': u'ok',
          'stderr': '',
          'stdout': '',
          'submitted': datetime.datetime(2014, 9, 28, 16, 6, 53, 917290)}
```

估值结果本身保存在对象的 **result** 属性中：

```
In [43]: option_values_obj[0].result
Out[43]: 24.436651486350289
```

为了生成和顺序化计算相同的结果列表，我们必须从返回对象中读出单个结果：

```
In [44]: option_values_par = []
         for res in option_values_obj:
             option_values_par.append(res.result)
```

当然，上述工作可以在并行估值循环中直接完成，图 8-3 比较了顺序化计算的估值结果和并行计算的结果。其中的差异是因为与蒙特卡洛估值相关的数值化问题：

```
In [45]: plt.figure(figsize=(8, 4))
         plt.plot(strikes, option_values_seq, 'b', label='Sequential')
         plt.plot(strikes, option_values_par, 'r.', label='Parallel')
         plt.grid(True); plt.legend(loc=0)
         plt.xlabel('strikes')
         plt.ylabel('European call option values')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

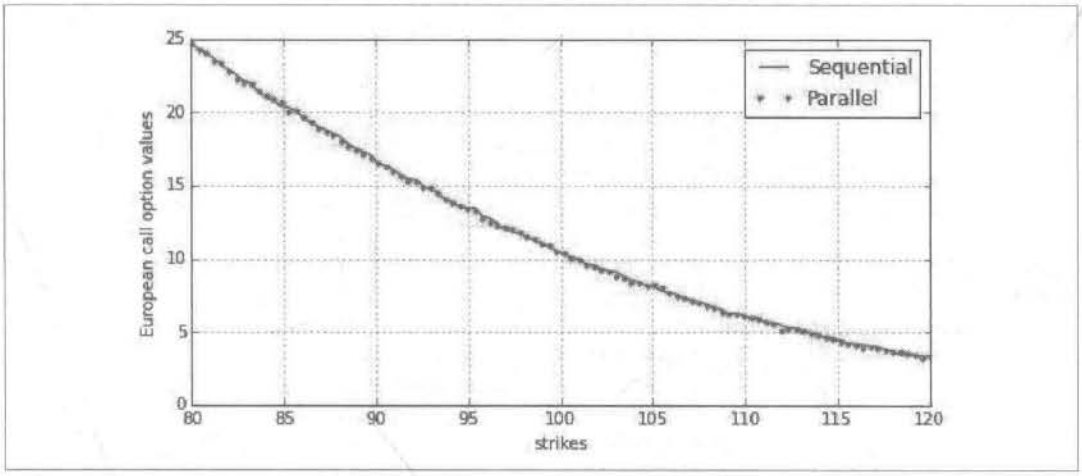


图 8-3　欧式看涨期权价值的比较

8.3.4　性能比较

在 perf_comp_func 函数的帮助下，我们可以更严格地比较性能：

```
In [46]: n = 50  # number of option valuations
         func_list = ['seq_value', 'par_value']
         data_list = 2 * ['n']

In [47]: perf_comp_data(func_list, data_list)

Out[47]: function: par_value, av. time sec:   0.90832, relative:    1.0
         function: seq_value, av. time sec:   5.75137, relative:    6.3
```

结果明显地说明，使用 **IPython.parallel** 并行执行函数，可以得到与可用核心数几乎成线性比例的性能。

8.4　多处理

IPython.parallel 的优势是可以在中小规模群集（例如 256 个节点的群集）上伸缩。但是有时候在本地并行执行代码是很有益的。这就是“标准”**Python multiprocessing** 模块的用武之地：

```
In [48]: import multiprocessing as mp
```

考虑如下模拟几何布朗运动的函数：

```
In [49]: import math
         def simulate_geometric_brownian_motion(p):
             M, I = p
               # time steps, paths
             S0 = 100; r = 0.05; sigma = 0.2; T = 1.0
               # model parameters
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    dt = T / M
    paths = np.zeros((M + 1, I))
    paths[0] = S0
    for t in range(1, M + 1):
        paths[t] = paths[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt +
                sigma * math.sqrt(dt) * np.random.standard_normal(I))
    return paths
```

这个函数返回以 M 和 I 为参数的模拟路径：

```
In [50]: paths = simulate_geometric_brownian_motion((5, 2))
         paths
Out[50]: array([[ 100.        ,  100.        ],
                [  93.65851581,   98.93916652],
                [  94.70157252,   93.44208625],
                [  96.73499004,   97.88294562],
                [ 110.64677908,   96.04515015],
                [ 124.09826521,  101.86087283]])
```

我们在一个具有 8 个核心的服务器上根据如下参数值实现一个测试系列。特别地，我们打算进行 100 次模拟：

```
In [51]: I = 10000  # number of paths
         M = 100  # number of time steps
         t = 100  # number of tasks/simulations
In [52]: # running on server with 8 cores/16 threads
         from time import time
         times = []
         for w in range(1, 17):
             t0 = time()
             pool = mp.Pool(processes=w)
               # the pool of workers
             result = pool.map(simulate_geometric_brownian_motion,
                               t * [(M, I), ])
               # the mapping of the function to the list of parameter tuples
             times.append(time() - t0)
```

我们再次得出性能和可用核心数量成正比的结论。不过，超线程在本例中不能带来太多好处（甚至更糟），如图 8-4 所示：

```
In [53]: plt.plot(range(1, 17), times)
         plt.plot(range(1, 17), times, 'ro')
         plt.grid(True)
         plt.xlabel('number of processes')
         plt.ylabel('time in seconds')
         plt.title('%d Monte Carlo simulations' % t)
```

简易的并行化

金融学中的许多问题可以应用简单的并行化技术，例如，在算法的不同实例之间没有共享数据时。**Python** 的 **multiprocessing** 模块可以高效地利用现代硬件架构的能力，一般不需要改变基本算法或者并行执行的 **Python** 函数。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

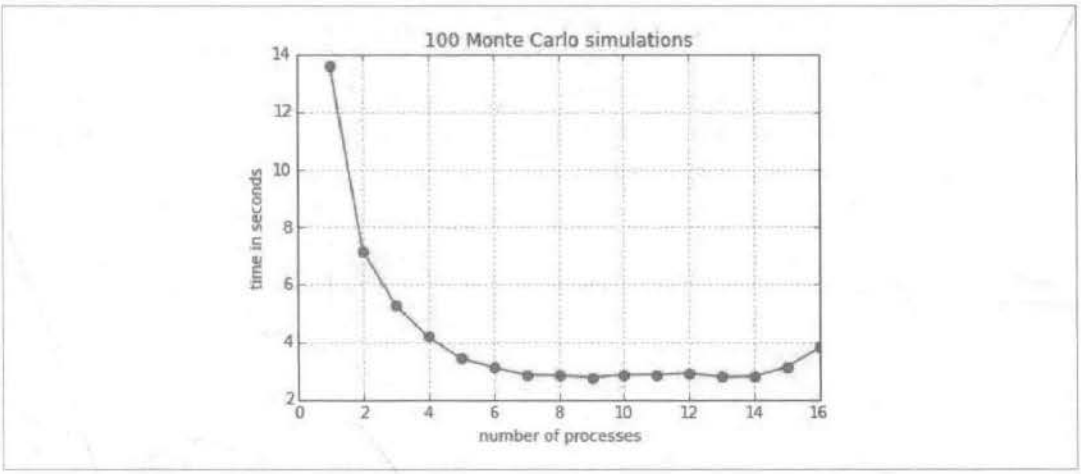


图 8-4　执行速度取决于所用线程数量（8 核机器）

8.5　动态编译

Numba（http://numba.pydata.org）是开源、**NumPy** 感知的优化 **Python** 代码编译器。它使用 LLVM 编译器基础架构[1]，将 Python 字节代码编译专门用于 **NumPy** 运行时和 **SciPy** 模块的机器代码。

8.5.1　介绍性示例

我们从一个在 Python 中通常会影响性能的问题开始：包含嵌套循环的算法。可以用一个沙箱变种来说明该问题：

```
In [54]: from math import cos, log
         def f_py(I, J):
             res = 0
             for i in range(I):
                 for j in range (J):
                     res += int(cos(log(1)))
             return res
```

这个函数采用计算密集的方式，以 I 和 J 为输入参数，返回总循环数量。将 I 和 J 都设置为 5000，导致 2500 万个循环：

```
In [55]: I, J = 5000, 5000
         %time f_py(I, J)
Out[55]: CPU times: user 17.4 s, sys: 2.3 s, total: 19.7 s
         Wall time: 15.2 s

         25000000
```

1　LLVM（http://www.llvm.org）原来是 Low Level Virtual Machine（低级虚拟机）的缩写；现在“它使该项目的全名”。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

原则上，这可以在 **NumPy ndarray** 对象的帮助下实现向量化：

```
In [56]: def f_np(I, J):
             a = np.ones((I, J), dtype=np.float64)
             return int(np.sum(np.cos(np.log(a)))), a

In [57]: %time res, a = f_np(I, J)

Out[57]: CPU times: user 1.41 s, sys: 285 ms, total: 1.69 s
         Wall time: 1.65 s
```

这种方法快得多（大约 8～10 倍），但是并不能真正高效地利用内存。**ndarray** 对象消耗 200MB 内存：

```
In [58]: a.nbytes

Out[58]: 200000000
```

考虑到 RAM 的数量，很容易选择 I 和 J 使 **NumPy** 方法变得不可行。**Numba** 提供了一种有吸引力的替代方法，可以解决这种循环结构的性能问题，同时保持纯 **Python** 方法的内存效率：

```
In [59]: import numba as nb
```

使用 Numba，只需要对 **Python** 函数应用 **jit** 函数，生成该函数的 **Python** 可调用编译版本：

```
In [60]: f_nb = nb.jit(f_py)
```

正如承诺的那样，这个新函数可以直接从 **Python** 解释程序中调用，实现比 **NumPy** 向量化版本更显著的加速效果：

```
In [61]: %time f_nb(I, J)

Out[61]: CPU times: user 143 ms, sys: 12 ms, total: 155 ms
         Wall time: 139 ms

         25000000L
```

我们再次用较为系统性的方法比较不同备选方案的性能：

```
In [62]: func_list = ['f_py', 'f_np', 'f_nb']
         data_list = 3 * ['I, J']

In [63]: perf_comp_data(func_list, data_list)

Out[63]: function: f_nb, av. time sec:   0.02022, relative:    1.0
         function: f_np, av. time sec:   1.67494, relative:   82.8
         function: f_py, av. time sec:  15.82375, relative:  782.4
```

嵌套循环实现的 **Numba** 版本是目前最快的方案，甚至远快于 **NumPy** 向量化版本。**Python** 版本比其他两种版本慢得多。

速效方法

改善（数值算法）性能的许多方法都需要花费可观的精力。利用 **Python** 和 **Numba**，就有了需要最少精力的一种方法——一般来说，只需要导入该库和一行附加代码。它不能用于所有类型算法，但是往往值得（简单地）一试，有时候确实能够快速取得效果。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

8.5.2 二项式期权定价方法

前一小节使用蒙特卡洛模拟方法、利用并行计算估计欧式看涨期权的价值。估算期权价值的另一种流行数值方法是由 Cox、Ross 和 Rubinstein（1979）率先提出的二项式期权定价模型。这种模型和 Black-Scholes-Merton 设置一样有风险资产（指数或者股票）以及无风险资产（债券）。和蒙特卡洛方法一样，从当天到期权到期日的时间间隔被分为通常等距的子间隔 Δt。如果时间 s 的指数水平为 S_s，则 $t=s+\Delta t$ 时的指数水平为 $S_t=S_s \bullet m$，其中 m 是从 $\{u, d\}$ 中随机选取（$0<d<e^{r\Delta t}<u=e^{\sigma\sqrt{\Delta t}}$，$u=1/d$）。$r$ 是一个常数——无风险利率。风险中立的上涨概率为 $q=\frac{e^{r\Delta t}-d}{u-d}$。

下面对该模型进行参数化：

```
In [64]: # model & option parameters
         S0 = 100.  # initial index level
         T = 1.  # call option maturity
         r = 0.05  # constant short rate
         vola = 0.20  # constant volatility factor of diffusion

         # time parameters
         M = 1000  # time steps
         dt = T / M  # length of time interval
         df = exp(-r * dt)  # discount factor per time interval

         # binomial parameters
         u = exp(vola * sqrt(dt))  # up-movement
         d = 1 / u  # down-movement
         q = (exp(r * dt) - d) / (u - d)  # martingale probability
```

欧式期权二项式算法的实现主要包含如下部分：

指数水平模拟

逐步模拟指数水平。

内在价值计算

计算到期日和每个时间步的内在价值。

风险中性折算

逐步折算（预期）内在价值直到达到现值。

在 Python 中，这可能采取函数 **binomial_py** 中的形式。该函数使用 **NumPy ndarray** 对象作为基本数据结构，并实现 3 个不同的嵌套循环，以实现上述的 3 个步骤：

```
In [65]: import numpy as np
         def binomial_py(strike):
             ''' Binomial option pricing via looping.

             Parameters
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    ==========
    strike : float
        strike price of the European call option
    '''
    # LOOP 1 - Index Levels
    S = np.zeros((M + 1, M + 1), dtype=np.float64)
      # index level array
    S[0, 0] = S0
    z1 = 0
    for j in xrange(1, M + 1, 1):
        z1 = z1 + 1
        for i in xrange(z1 + 1):
            S[i, j] = S[0, 0] * (u ** j) * (d ** (i * 2))

    # LOOP 2 - Inner Values
    iv = np.zeros((M + 1, M + 1), dtype=np.float64)
      # inner value array
    z2 = 0
    for j in xrange(0, M + 1, 1):
        for i in xrange(z2 + 1):
            iv[i, j] = max(S[i, j] - strike, 0)
        z2 = z2 + 1

    # LOOP 3 - Valuation
    pv = np.zeros((M + 1, M + 1), dtype=np.float64)
      # present value array
    pv[:, M] = iv[:, M]  # initialize last time point
    z3 = M + 1
    for j in xrange(M - 1, -1, -1):
        z3 = z3 - 1
        for i in xrange(z3):
            pv[i, j] = (q * pv[i, j + 1] +
                        (1 - q) * pv[i + 1, j + 1]) * df
    return pv[0, 0]
```

上述函数使用前面指定的参数，返回欧式看涨期权的现值：

```
In [66]: %time round(binomial_py(100), 3)
Out[66]: CPU times: user 4.18 s, sys: 312 ms, total: 4.49 s
         Wall time: 3.64 s

         10.449
```

我们可以将这个结果与蒙特卡洛函数 **bsm_mcs_valuation** 返回的估算结果比较：

```
In [67]: %time round(bsm_mcs_valuation(100), 3)
Out[67]: CPU times: user 133 ms, sys: 0 ns, total: 133 ms
         Wall time: 126 ms

         10.318
```

两个值很类似，它们只是“相似”而不是相同，是因为蒙特卡洛估值和 **bsm_mcs_valuation** 所实现的算法都不是很精确，不同的随机数会导致（稍有）不同的估算结果，对于健全的蒙特卡洛估算来说，每次模拟使用 20000 条路径也可能略少一些（但是可以得到

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

较高的估值速度）。相比之下，本例中的二项式期权估价使用 1000 个时间步已经相当精确，但是花费的时间也长得多。

同样，我们可以尝试 **NumPy** 向量化技术，从二项式方法中得到同样精确、但是速度更快的结果。**binomial_np** 函数初看有些神秘，但是，当你运行单独的构建步骤并检查结果，后台（NumPy）发生的操作就显而易见了：

```
In [68]: def binomial_np(strike):
             ''' Binomial option pricing with NumPy.

             Parameters
             ==========
             strike : float
                 strike price of the European call option
             '''
             # Index Levels with NumPy
             mu = np.arange(M + 1)
             mu = np.resize(mu, (M + 1, M + 1))
             md = np.transpose(mu)
             mu = u ** (mu - md)
             md = d ** md
             S = S0 * mu * md

             # Valuation Loop
             pv = np.maximum(S - strike, 0)
             z = 0
             for t in range(M - 1, -1, -1):  # backward iteration
                 pv[0:M - z, t] = (q * pv[0:M - z, t + 1]
                                   + (1 - q) * pv[1:M - z + 1, t + 1]) * df
                 z += 1
             return pv[0, 0]
```

下面我们简单地看看后台的情况。为了简洁和易于理解，只考虑 **M=4** 的时间步。第一步如下：

```
In [69]: M = 4  # four time steps only
         mu = np.arange(M + 1)
         mu
Out[69]: array([0, 1, 2, 3, 4])
```

下面是构造的第二步：

```
In [70]: mu = np.resize(mu, (M + 1, M + 1))
         mu
Out[70]: array([[0, 1, 2, 3, 4],
                [0, 1, 2, 3, 4],
                [0, 1, 2, 3, 4],
                [0, 1, 2, 3, 4],
                [0, 1, 2, 3, 4]])
```

第三步：

```
In [71]: md = np.transpose(mu)
         md
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[71]: array([[0, 0, 0, 0, 0],
                [1, 1, 1, 1, 1],
                [2, 2, 2, 2, 2],
                [3, 3, 3, 3, 3],
                [4, 4, 4, 4, 4]])
```

第四步和第五步：

```
In [72]: mu = u ** (mu - md)
         mu.round(3)
Out[72]: array([[ 1.   ,  1.006,  1.013,  1.019,  1.026],
                [ 0.994,  1.   ,  1.006,  1.013,  1.019],
                [ 0.987,  0.994,  1.   ,  1.006,  1.013],
                [ 0.981,  0.987,  0.994,  1.   ,  1.006],
                [ 0.975,  0.981,  0.987,  0.994,  1.   ]])
In [73]: md = d ** md
         md.round(3)
Out[73]: array([[ 1.   ,  1.   ,  1.   ,  1.   ,  1.   ],
                [ 0.994,  0.994,  0.994,  0.994,  0.994],
                [ 0.987,  0.987,  0.987,  0.987,  0.987],
                [ 0.981,  0.981,  0.981,  0.981,  0.981],
                [ 0.975,  0.975,  0.975,  0.975,  0.975]])
```

最后，将所有步骤结合起来：

```
In [74]: S = S0 * mu * md
         S.round(3)
Out[74]: array([[ 100.   ,  100.634,  101.273,  101.915,  102.562],
                [  98.743,   99.37 ,  100.   ,  100.634,  101.273],
                [  97.502,   98.121,   98.743,   99.37 ,  100.   ],
                [  96.276,   96.887,   97.502,   98.121,   98.743],
                [  95.066,   95.669,   96.276,   96.887,   97.502]])
```

在 **ndarray** 对象 S 中，只有上三角矩阵是重要的。虽然使用这种方法进行的计算多于原则上的需要，但是这种方法和预计的一样，比严重依赖 **Python** 级别嵌套循环的第一个版本快得多：

```
In [75]: M = 1000  # reset number of time steps
         %time round(binomial_np(100), 3)
Out[75]: CPU times: user 308 ms, sys: 6 ms, total: 314 ms
         Wall time: 304 ms

         10.449
```

在我们的沙箱示例中，**Numba** 已经证明是一种宝贵的性能增强工具。下面，它还能证明自己在非常重要的金融算法中也是很重要的：

```
In [76]: binomial_nb = nb.jit(binomial_py)
In [77]: %time round(binomial_nb(100), 3)
Out[77]: CPU times: user 1.71 s, sys: 137 ms, total: 1.84 s
         Wall time: 1.59 s

         10.449
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们还没有看出对 **NumPy** 向量化版本有多少加速效果，因为第一次调用编译后的函数涉及一些开销。因此，使用 **perf_comp_func** 函数应该更现实地揭示出不同实现的性能对比。显然，**Numba** 编译版本确实明显快于 **NumPy** 版本：

```
In [78]: func_list = ['binomial_py', 'binomial_np', 'binomial_nb']
         K = 100.
         data_list = 3 * ['K']
In [79]: perf_comp_data(func_list, data_list)
Out[79]: function: binomial_nb, av. time sec:   0.14800, relative:    1.0
         function: binomial_np, av. time sec:   0.31770, relative:    2.1
         function: binomial_py, av. time sec:   3.36707, relative:   22.8
```

总而言之，我们可以得出如下结论。

- **效率**：使用 **Numba** 只需要花费很少的额外精力。原始函数往往完全不需要改变；你所需要做的就是调用 **jit** 函数。
- **加速**：**Numba** 往往带来执行速度的显著提高，不仅和纯 **Python** 相比是如此，即使对向量化的 **NumPy** 实现也有明显优势。
- **内存**：使用 **Numba** 不需要初始化大型数组对象；编译器专门为手上的问题生成机器代码（和 **NumPy** 的“通用”函数相比）并维持和纯 **Python** 相同的内存效率。

8.6 用 Cython 进行静态编译

Numba 的优势是对任意函数应用该方法毫不费力。但是，Numba 只能为某些问题“毫不费力”地产生显著的性能改善。另一种方法更为灵活，但是也需要更多精力，这就是通过 Cython 的*静态编译*。实际上，Cython 是 Python 和 C 语言的混血儿。从 Python 的角度看，需要注意的主要不同是静态类型声明（和 C 语言相同）和一个单独的编译步骤（和任何编译语言相同）。

下面是一个简单的实例函数，嵌套循环再次简单地返回循环次数。和前一个嵌套循环示例相比，这次内循环次数由外循环次数放大。在这种情况下，当你试图用 NumPy 加速时会很快遇到内存问题：

```
In [80]: def f_py(I, J):
             res = 0.  # we work on a float object
             for i in range(I):
                 for j in range (J * I):
                     res += 1
             return res
```

检查一下 I=500 和 J=500 时的 Python 性能。本例中，使我们可以向量化 **f_py** 的 **NumPy ndarray** 对象组成必须为（500，250000）：

```
In [81]: I, J = 500, 500
         %time f_py(I, J)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[81]: CPU times: user 17 s, sys: 2.72 s, total: 19.7 s
         Wall time: 14.2 s

         125000000.0
```

接下来考虑例 8-1 所示的代码。它采用相同的函数并引入用于 Cython 的静态类型声明。注意，这个 Cython 文件的扩展名为**.pyx**。

例 8-1　使用 Cython 静态类型声明的嵌套循环

```
#
# Nested loop example with Cython
# nested_loop.pyx

def f_cy(int I, int J):
    cdef double res = 0
    # double float much slower than int or long
    for i in range(I):
        for j in range (J * I):
            res += 1
    return res
```

在这么简单的例子中不需要任何特殊的 C 模块，导入模块有一种简单的方法——也就是通过 **pyximport**：

```
In [82]: import pyximport
         pyximport.install()

Out[82]: (None, <pyximport.pyximport.PyxImporter at 0x92cfc10>)
```

现在我们可以直接从 Cython 模块中导入：

```
In [83]: import sys
         sys.path.append('data/')
           # path to the Cython script
           # not needed if in same directory

In [84]: from nested_loop import f_cy
```

下面检查 Cython 函数的性能：

```
In [85]: %time res = f_cy(I, J)
Out[85]: CPU times: user 154 ms, sys: 0 ns, total: 154 ms
         Wall time: 153 ms

In [86]: res
Out[86]: 125000000.0
```

在 **IPython Notebook** 中工作时，使用 Cython 有一个更便利的方法——**cythonmagic**：

```
In [87]: %load_ext cythonmagic
```

从 IPython Notebook 中加载这个扩展，可以从该工具中用 Cython 编译代码：

```
In [88]: %%cython
         #
         # Nested loop example with Cython
         #
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
def f_cy(int I, int J):
    cdef double res = 0
    # double float much slower than int or long
    for i in range(I):
        for j in range (J * I):
            res += 1
    return res
```

性能结果当然（几乎）相同：

```
In [89]: %time res = f_cy(I, J)
Out[89]: CPU times: user 156 ms, sys: 0 ns, total: 156 ms
         Wall time: 154 ms

In [90]: res
Out[90]: 125000000.0
```

我们来看看 **Numba** 在这种情况下能起什么作用。应用和前面一样简单：

```
In [91]: import numba as nb
In [92]: f_nb = nb.jit(f_py)
```

第一次调用函数时，性能比 **Cython** 版本差（前面介绍过，第一次调用 **Numba** 编译函数总是有某些开销）：

```
In [93]: %time res = f_nb(I, J)
Out[93]: CPU times: user 285 ms, sys: 9 ms, total: 294 ms
         Wall time: 273 ms

In [94]: res
Out[94]: 125000000.0
```

最后，更严格的比较表明，**Numba** 版本确实能比得上 **Cython** 版本：

```
In [95]: func_list = ['f_py', 'f_cy', 'f_nb']
         I, J = 500, 500
         data_list = 3 * ['I, J']
In [96]: perf_comp_data(func_list, data_list)
Out[96]: function: f_nb, av. time sec:   0.15162, relative:    1.0
         function: f_cy, av. time sec:   0.15275, relative:    1.0
         function: f_py, av. time sec:  14.08304, relative:   92.9
```

8.7 在 GPU 上生成随机数

本章的最后一个主题是使用设备进行大规模的并行操作——也就是*通用图形处理单元*（GPGPU 或者简称 GPU）。要使用 Nvidia GPU，就必须安装 CUDA（**统一计算设备架构**，https://developer.nvidia.com）。利用 Nvidia GPU 的简单方法之一是使用 NumbaPro，这个由 Continuum Analytics 开发的高性能库为 GPU（或者多核 CPU）动态编译 Python。

本章无法深入 Python 编程中使用 GPU 的细节，但是，有一个金融领域可以从 GPU 的

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

使用中得到很大的好处：蒙特卡洛模拟，特别是（伪）随机数生成[1]。下面，我们使用原生 CUDA 库，在 GPU 上生成随机数：

```
In [97]: from numbapro.cudalib import curand
```

和基准示例相同，我们使用 NumPy 定义一个函数，产生一个标准正态分布伪随机数组成的二维数组：

```
In [98]: def get_randoms(x, y):
             rand = np.random.standard_normal((x, y))
             return rand
```

首先检查它是否能够正常工作：

```
In [99]: get_randoms(2, 2)
Out[99]: array([[-0.30561007,  1.33124048],
                [-0.04382143,  2.31276888]])
```

下面是用于 Nvidia GPU 的函数：

```
In [100]: def get_cuda_randoms(x, y):
              rand = np.empty((x * y), np.float64)
                  # rand serves as a container for the randoms
                  # CUDA only fills 1-dimensional arrays
              prng = curand.PRNG(rndtype=curand.PRNG.XORWOW)
                  # the argument sets the random number algorithm
              prng.normal(rand, 0, 1)  # filling the container
              rand = rand.reshape((x, y))
                  # to be "fair", we reshape rand to 2 dimensions
              return rand
```

再次对功能性进行简单的检查：

```
In [101]: get_cuda_randoms(2, 2)
Out[101]: array([[ 1.07102161,  0.70846868],
                 [ 0.89437398, -0.86693007]])
```

第一次比较性能：

```
In [102]: %timeit a = get_randoms(1000, 1000)
Out[102]: 10 loops, best of 3: 72 ms per loop
In [103]: %timeit a = get_cuda_randoms(1000, 1000)
Out[103]: 100 loops, best of 3: 14.8 ms per loop
```

现在用更系统性的例程比较性能：

```
In [104]: import time as t
          step = 1000
          def time_comparsion(factor):
              cuda_times = list()
              cpu_times = list()
              for j in range(1, 10002, step):
                  i = j * factor
```

1 这些主题也可以参见第 10 章。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        t0 = t.time()
        a = get_randoms(i, 1)
        t1 = t.time()
        cpu_times.append(t1 - t0)
        t2 = t.time()
        a = get_cuda_randoms(i, 1)
        t3 = t.time()
        cuda_times.append(t3 - t2)
    print "Bytes of largest array %i" % a.nbytes
    return cuda_times, cpu_times
```

可视化性能结果的助手函数：

```
In [105]: def plot_results(cpu_times, cuda_times, factor):
              plt.plot(x * factor, cpu_times,'b', label='NUMPY')
              plt.plot(x * factor, cuda_times, 'r', label='CUDA')
              plt.legend(loc=0)
              plt.grid(True)
              plt.xlabel('size of random number array')
              plt.ylabel('time')
              plt.axis('tight')
```

我们观察具备中等工作负载的第一个测试系列：

```
In [106]: factor = 100
          cuda_times, cpu_times = time_comparsion(factor)
Out[106]: Bytes of largest array 8000800
```

在 GPU 上计算随机数的时间几乎与生成数的个数*无关*。相比之下，在 CPU 上花费的时间随着生成随机数数组的尺寸增大而*猛增*。这两种情况可以在图 8-5 中验证：

```
In [107]: x = np.arange(1, 10002, step)
In [108]: plot_results(cpu_times, cuda_times, factor)
```

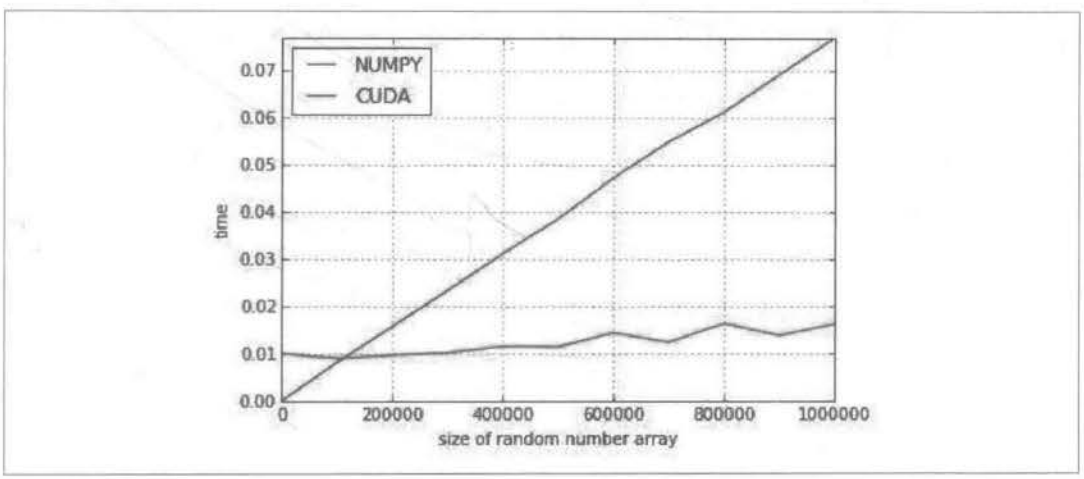


图 8-5　在 GPU 和 CPU 上生成随机数（因子=100）

现在，我们来看第二个测试系列，它的负载相当低：

```
In [109]: factor = 10
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        cuda_times, cpu_times = time_comparsion(factor)
Out[109]: Bytes of largest array 800080
```

使用 GPU 的开销对于低工作负载来说太大——从图 8-6 中可以很明显地看到这一点：

```
In [110]: plot_results(cpu_times, cuda_times, factor)
```

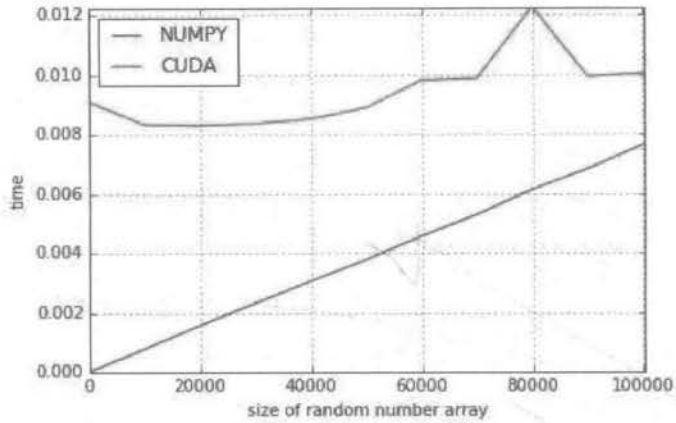


图 8-6　在 GPU 和 CPU 上生成随机数（因子=10）

现在我们来考虑一个相对*较重*的负载。最大随机数数组的尺寸为 400MB：

```
In [111]: %%time
          factor = 5000
          cuda_times, cpu_times = time_comparsion(factor)
Out[111]: Bytes of largest array 400040000
          CPU times: user 22 s, sys: 3.52 s, total: 25.5 s
          Wall time: 25.4 s
```

在重负载的情况下，GPU 明显表现出优势，图 8-7 生动地说明了这一点：

```
In [112]: plot_results(cpu_times, cuda_times, factor)
```

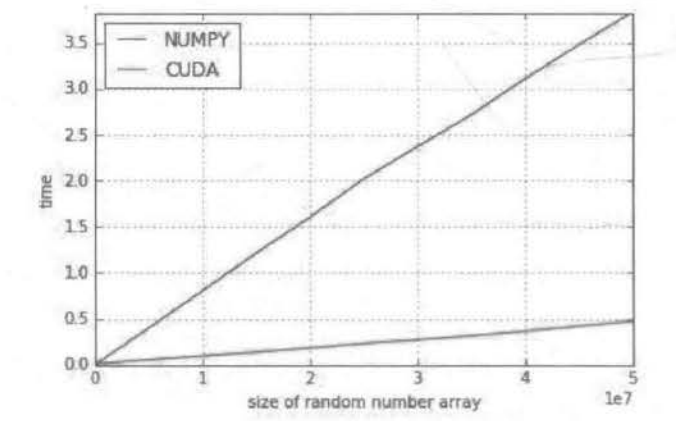


图 8-7　在 GPU 和 CPU 上生成随机数（因子=5000）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

8.8 结语

现在，**Python** 生态系统提供了一些改进代码性能的手段。

范型

在给定问题上，某些 **Python** 范型比其他范型性能更好

库

有用于不同类型问题的大量库，这些库（如 **numexpr**）往往导致适合于该库范围的问题得到性能更好的解决方案。

编译

有一些强大的编译解决方案，包括静态（如 **Cython**）和动态（例如 **Numba**）方案。

并行化

有些 **Python** 有内建的并行化功能（例如 **numexpr**），而其他一些库允许我们利用多核 CPU、整个群集（如 **IPython.parallel**）或者 GPU（例如 **NumbaPro**）的能力。

Python 生态系统的重大优势之一是所有方法通常都很容易实现，需要的额外精力通常很少（即使对非专业人士也是如此）。换言之，在有这么多高性能库的今天，性能的改善是唾手可得的。

8.9 延伸阅读

对于本章介绍的所有高性能库，都有宝贵的 Web 资源。

- **numexpr** 的细节参见 http://github.com/pydata/numexpr。
- **IPython.parallel** 在这里介绍：http://ipython.org/ipython-doc/stable/parallel。
- 这里可以找到 **multiprocessing** 模块的文档：https://docs.python.org/2/library/multiprocessing.html
- Numba 的信息可以在 http://github.com/numba/numba 找到。
- http://cython.org 是 Cython 编译器项目的首页。
- **NumbaPro** 的文档可以参见 http://docs.continuum.io/numbapro。

下面是一本参考书。

- Gorelick, Misha and Ian Ozsvald (2014): *High Performance Python*. O'Reilly, Sebastopol, CA.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第9章

数学工具

数学家是现代化世界的祭司。

——Bill Gaede

从20世纪80年代和90年代华尔街出现所谓“火箭科学家”以来，金融学已经发展成为一种应用数学学科。虽然早期金融学研究论文中出现的数学表达式和公式不多，但是这些表达式和公式已经成为当今金融学论文的主要组成部分，其他部分则是解释它们的文字。

本章介绍对金融学有用的一些数学工具，但是对每种工具的背景不做详细的介绍。这个主题有许多实用的书籍，因此，本章的焦点是如何在Python中使用这些工具和技术。这些工具如下所示。

逼近法

回归和插值是金融学中最常用的数学技术。

凸优化

一些金融学科需要凸优化工具（例如，期权定价模型检验）。

积分

金融（衍生）资产的估值往往归结为积分计算。

符号数学

Python提供**SymPy**，这是一种强大的符号数学工具，例如，它可以解方程（组）。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

9.1 逼近法

首先，我们导入现在需要的库——**NumPy** 和 **matplotlib.pyplot**：

```
In [1]: import numpy as np
        import matplotlib.pyplot as plt
        %matplotlib inline
```

在整个讨论中，我们使用的主要示例函数如下，该函数由一个三角函数项和一个线性项组成：

```
In [2]: def f(x):
            return np.sin(x) + 0.5 * x
```

主要的重点是在给定区间内通过*回归*和*插值*求取该函数的近似值。首先，我们生成该函数的图形，更好地观察逼近法所实现的结果。我们感兴趣的区间是[−2π，2π]。图 9-1 显示了该函数在通过 **linspace** 函数定义的固定区间上的图像。**np.linspace(start, stop, num)**返回从 **start** 开始，**stop** 结束的 **num** 个点，两个连续点之间的子区间均匀分布：

```
In [3]: x = np.linspace(-2 * np.pi, 2 * np.pi, 50)
In [4]: plt.plot(x, f(x), 'b')
        plt.grid(True)
        plt.xlabel('x')
        plt.ylabel('f(x)')
```

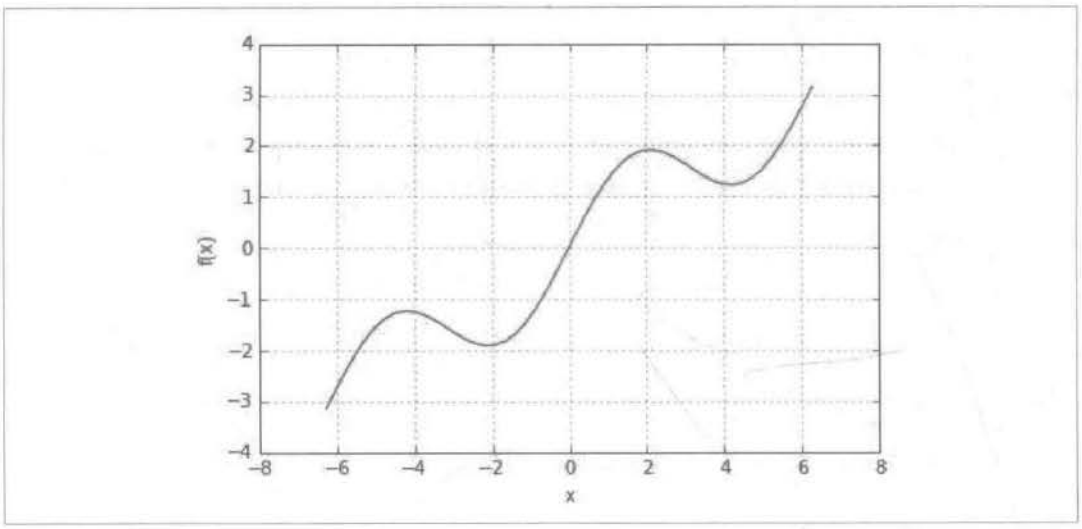


图 9-1 示例函数图形

9.1.1 回归

回归是相当高效的函数近似值计算工具。它不仅适合求取一维函数的近似值，在更高维度上也很有效。得出回归结果所需要的数值化方法很容易实现，执行也很快速。本质上，回归的任务是在给定一组所谓“基函数” b_d，$d \in \{1, \cdots, D\}$的情况下，根据公

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

式 9-1 找出最优参数 α_1^*，$\cdots \alpha_D^*$，其中对于 $i \in \{1,\cdots I\}$ 观察点，$y_i \equiv f(x_i)$。x_i 可以视为自变量观测值，y_i 可视为因变量观测值（从函数或者统计的意义上说）。

公式 9-1. 最小化回归问题

$$\min_{\alpha_1,\cdots,\alpha_D} \frac{1}{I}\sum_{i=1}^{I}\left(y_i - \sum_{d=1}^{D}\alpha_d \cdot b_d(x_i)\right)^2 \qquad (9\text{-}1)$$

1. 作为基函数的单项式

最简单的情况是以单项式作为基函数——也就是说，$b_1=1$，$b_2=x$，$b_3=x^2$，$b_4=x^3$，…在这种情况下，**NumPy** 有确定最优参数（**polyfit**）和以一组输入值求取近似值（**polyval**）的内建函数。

表 9-1 列出了 **polyfit** 函数的参数。在 **polyfit** 返回的最优回归相关系数 ρ 基础上，**np.polyval**（ρ，x)返回 x 坐标的回归值。

表 9-1 polyfit 函数参数

参数	描述
x	x 坐标（自变量值）
y	y 坐标（因变量值）
deg	多项式拟合度
full	如果为真，返回额外的诊断信息
w	应用到 y 坐标的权重
cov	如果为真，返回协方差矩阵

典型向量化风格的 **polyfit** 和 **polyval** 线性回归（**deg=1**）应用方式如下：

```
In [5]: reg = np.polyfit(x, f(x), deg=1)
        ry = np.polyval(reg, x)
```

由于回归估算值保存在 **ry** 数组中，我们可以如图 9-2 那样比较回归结果和原始函数。当然，线性回归无法处理示例函数的 **sin** 部分：

```
In [6]: plt.plot(x, f(x), 'b', label='f(x)')
        plt.plot(x, ry, 'r.', label='regression')
        plt.legend(loc=0)
        plt.grid(True)
        plt.xlabel('x')
        plt.ylabel('f(x)')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

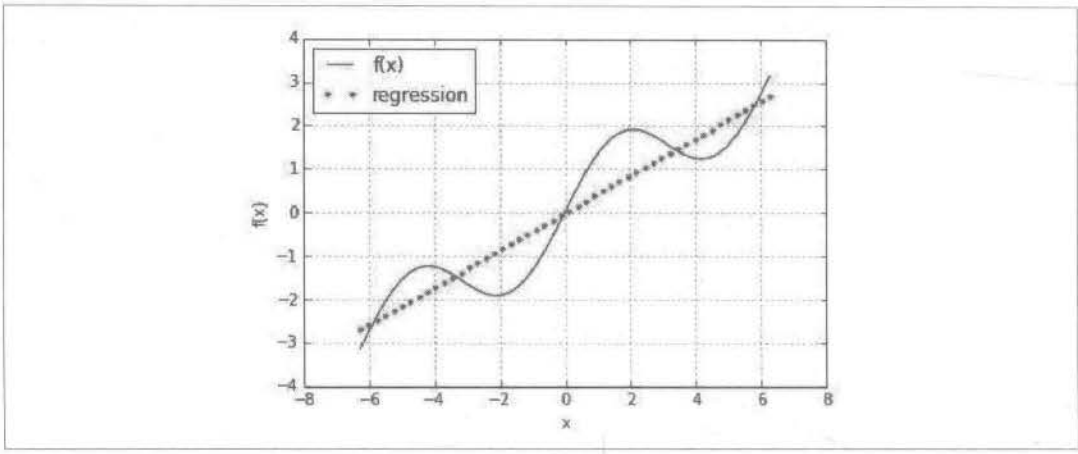


图 9-2　示例函数和线性回归

为了处理示例函数的 **sin** 部分，必须使用更高次的单项式。下一个回归试图使用 5 次单项式作为基函数。果不其然，回归结果（如图 9-3 所示）看上去更接近原始函数。但是，它还远称不上完美：

```
In [7]: reg = np.polyfit(x, f(x), deg=5)
        ry = np.polyval(reg, x)
In [8]: plt.plot(x, f(x), 'b', label='f(x)')
        plt.plot(x, ry, 'r.', label='regression')
        plt.legend(loc=0)
        plt.grid(True)
        plt.xlabel('x')
        plt.ylabel('f(x)')
```

最后一次尝试使用最高 7 次的单项式作为基函数以计算示例函数的近似值。这次的结果如图 9-4 所示，相当有说服力：

```
In [9]: reg = np.polyfit(x, f(x), 7)
        ry = np.polyval(reg, x)
```

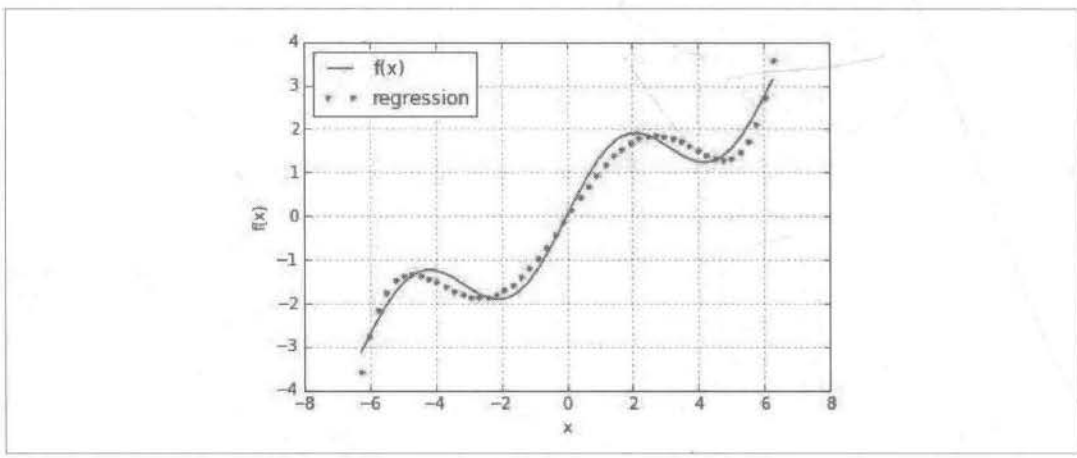


图 9-3　使用最高 5 次的单项式进行回归

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [10]: plt.plot(x, f(x), 'b', label='f(x)')
         plt.plot(x, ry, 'r.', label='regression')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```

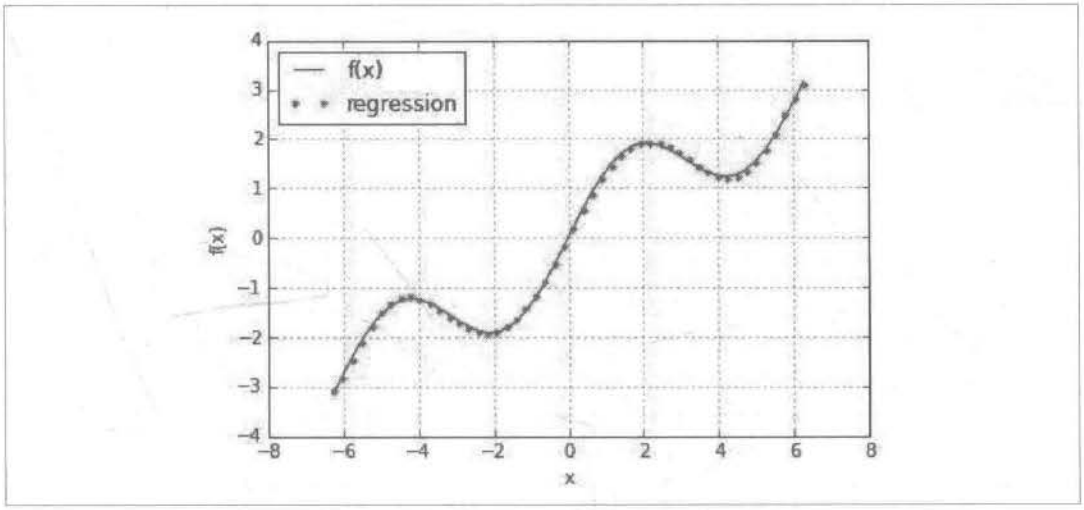


图 9-4　使用最高 7 次的单项式进行回归

简单的检查表明结果并不完美：

```
In [11]: np.allclose(f(x), ry)
Out[11]: False
```

但是，均方差（MSE）不是太大——至少在较窄的 x 值区间内：

```
In [12]: np.sum((f(x) - ry) ** 2) / len(x)
Out[12]: 0.0017769134759517413
```

2. 单独的基函数

一般来说，当你选择更好的基函数组时，可以得到更好的回归结果，例如利用对函数的认识进行近似值计算。在这种情况下，单独的基函数必须通过一个矩阵方法定义（也就是使用 **NumPy ndarray** 对象）。首先，例子中的多项式最高为 3 次：

```
In [13]: matrix = np.zeros((3 + 1, len(x)))
         matrix[3, :] = x ** 3
         matrix[2, :] = x ** 2
         matrix[1, :] = x
         matrix[0, :] = 1
```

numpy.linalg 子库提供 **lstsq** 函数，以解决和公式 9-1 类似的最小二乘优化问题：

```
In [14]: reg = np.linalg.lstsq(matrix.T, f(x))[0]
```

对于我们的问题，按照这种方式应用 **lstsq** 可以得到单个基函数的最优参数：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [15]: reg
Out[15]: array([  1.13968447e-14,   5.62777448e-01,  -8.88178420e-16,
                -5.43553615e-03])
```

为了获得回归估算结果，我们对 **reg** 和 **matrix** 数组应用 **dot** 函数。图 9-5 展示结果。**np.dot(a,b)**给出两个数组 **a** 和 **b** 的点乘：

```
In [16]: ry = np.dot(reg, matrix)
In [17]: plt.plot(x, f(x), 'b', label='f(x)')
         plt.plot(x, ry, 'r.', label='regression')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```

根据前面单项式的经验，图 9-5 中的结果并不真的如预期那么好。使用更通用的方法，可以利用对示例函数的认识。我们知道函数中有一个 **sin** 部分。因此，在基函数中包含一个正弦函数是有意义的。为了简单起见，我们替换最高次的单项式：

```
In [18]: matrix[3, :] = np.sin(x)
         reg = np.linalg.lstsq(matrix.T, f(x))[0]
         ry = np.dot(reg, matrix)
```

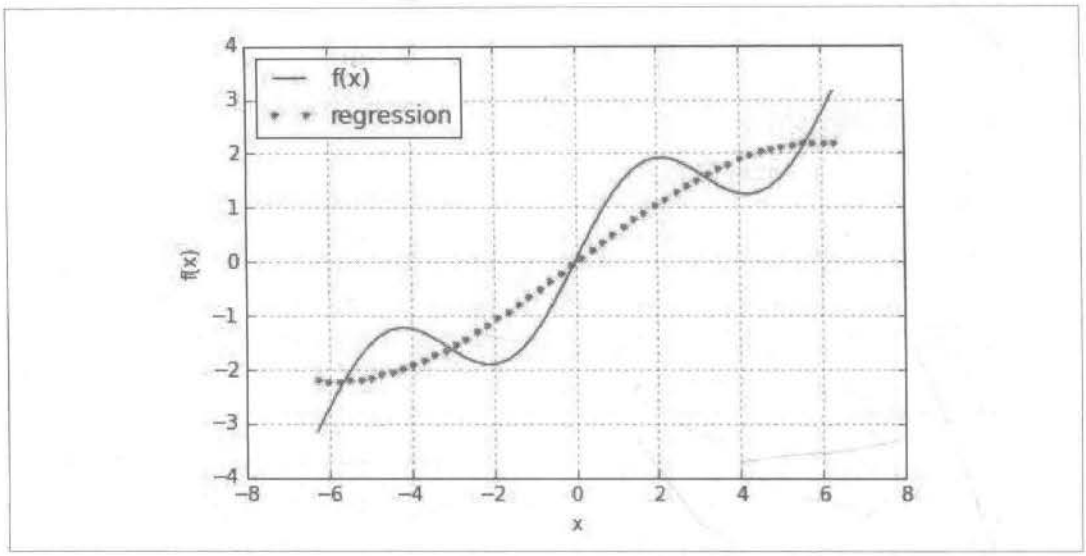


图 9-5　通过最小二乘函数的回归

图 9-6 表明回归现在相当接近原始函数：

```
In [19]: plt.plot(x, f(x), 'b', label='f(x)')
         plt.plot(x, ry, 'r.', label='regression')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

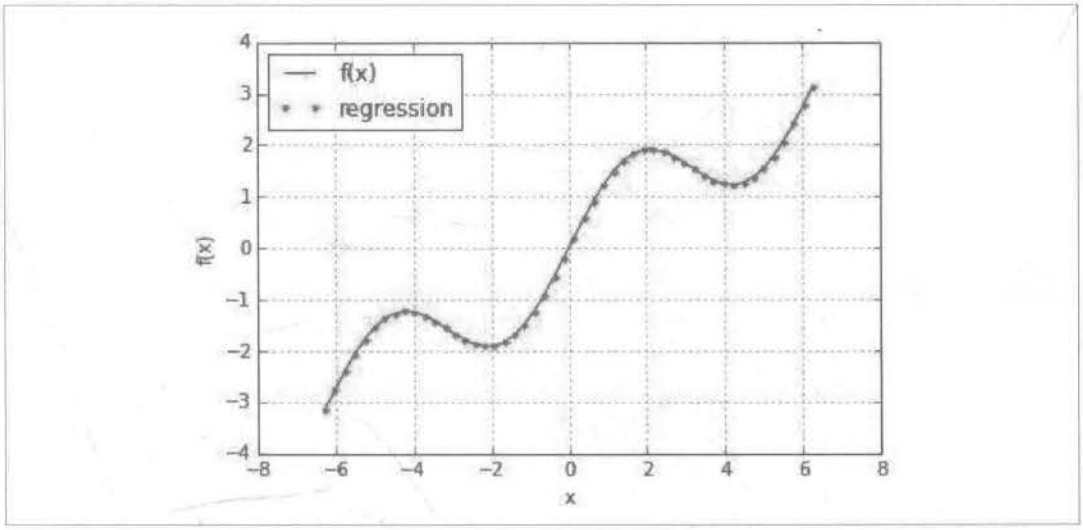


图 9-6　使用单独函数的回归

确实，回归现在从数值的意义上说是“完美”的：

```
In [20]: np.allclose(f(x), ry)
Out[20]: True
In [21]: np.sum((f(x) - ry) ** 2) / len(x)
Out[21]: 2.2749084503102031e-31
```

实际上，上述最小化例程恢复了正确的参数：**sin** 部分为 1，线性部分为 0.5：

```
In [22]: reg
Out[22]: array([  1.55428020e-16,   5.00000000e-01,   0.00000000e+00,
                  1.00000000e+00])
```

3. 有噪声的数据

回归对于有噪声的数据同样能够很好的处理，这种数据来自于模拟或者（不完善的）测量。为了阐述这个要点，我们生成同样具有噪声的自变量观测值和因变量观测值：

```
In [23]: xn = np.linspace(-2 * np.pi, 2 * np.pi, 50)
         xn = xn + 0.15 * np.random.standard_normal(len(xn))
         yn = f(xn) + 0.25 * np.random.standard_normal(len(xn))
```

基本的回归是一样的：

```
In [24]: reg = np.polyfit(xn, yn, 7)
         ry = np.polyval(reg, xn)
```

图 9-7 表明，回归结果比有噪声的数据点更接近原始函数。在某种意义上，回归在一定程度上平均了噪声：

```
In [25]: plt.plot(xn, yn, 'b^', label='f(x)')
         plt.plot(xn, ry, 'ro', label='regression')
         plt.legend(loc=0)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
plt.grid(True)
plt.xlabel('x')
plt.ylabel('f(x)')
```

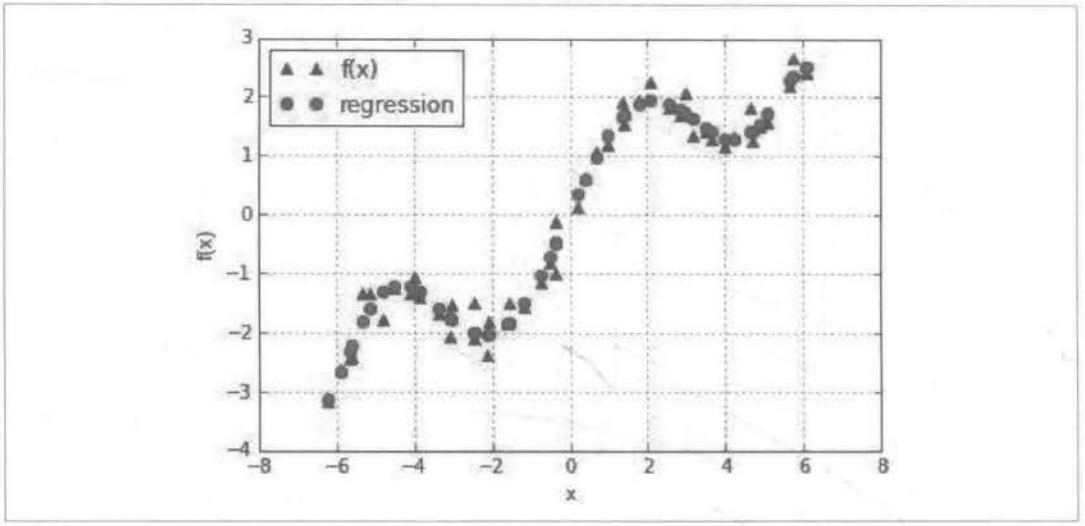


图 9-7　使用有噪声数据的回归

4. 未排序数据

回归的另一个重要特点是这种方法可以无缝地处理未排序数据。前面的例子都依赖于经过排序的 x 数据，情况并不总是这样的，为了说明这一点，我们随机产生自变量数据点：

```
In [26]: xu = np.random.rand(50) * 4 * np.pi - 2 * np.pi
         yu = f(xu)
```

在这种情况下，仅靠从视觉上检查原始数据很难识别出任何结构：

```
In [27]: print xu[:10].round(2)
         print yu[:10].round(2)
Out[27]: [ 4.09  0.5   1.48 -1.85  1.65  4.51 -5.7   1.83  4.42 -4.2 ]
         [ 1.23  0.72  1.74 -1.89  1.82  1.28 -2.3   1.88  1.25 -1.23]
```

和有噪声数据一样，回归方法不关心观测点的顺序。这在研究公式 9-1 中最小化问题的结构时很明显。从图 9-8 中显示的结果来看也很明显：

```
In [28]: reg = np.polyfit(xu, yu, 5)
         ry = np.polyval(reg, xu)
In [29]: plt.plot(xu, yu, 'b^', label='f(x)')
         plt.plot(xu, ry, 'ro', label='regression')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

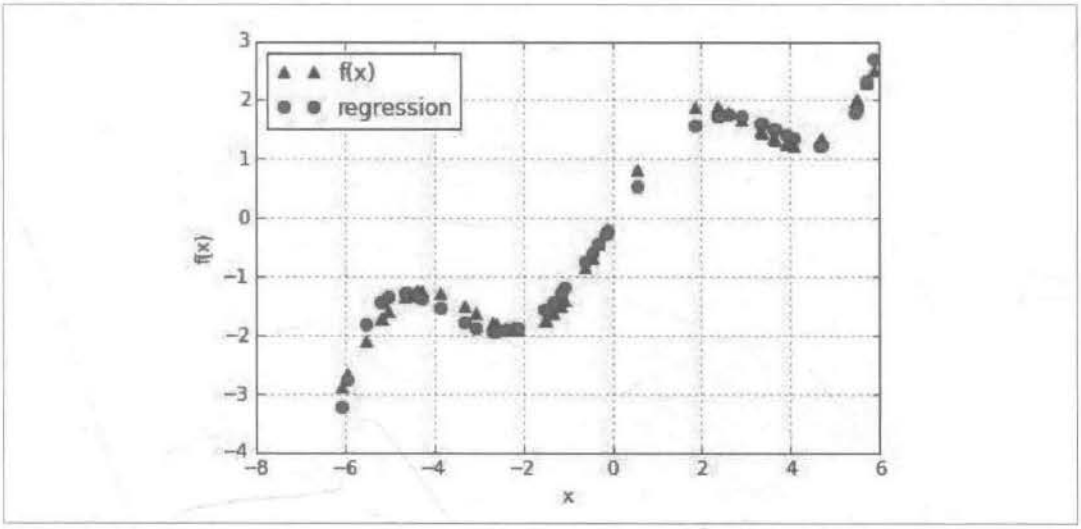


图 9-8　使用未排序数据的回归

5. 多维

最小二乘回归方法的另一个便利特性是不需要太多的修改就可以用于多维的情况。我们以下面介绍的 **fm** 函数为例：

```
In [30]: def fm((x, y)):
             return np.sin(x) + 0.25 * x + np.sqrt(y) + 0.05 * y ** 2
```

为了这个函数的可视化，我们需要（自变量）数据点的网格：

```
In [31]: x = np.linspace(0, 10, 20)
         y = np.linspace(0, 10, 20)
         X, Y = np.meshgrid(x, y)
           # generates 2-d grids out of the 1-d arrays
         Z = fm((X, Y))
         x = X.flatten()
         y = Y.flatten()
           # yields 1-d arrays from the 2-d grids
```

根据以 **x**，**y**，**z** 表示的自变量和因变量数据点网格，图 9-9 显示 **fm** 函数的形状：

```
In [32]: from mpl_toolkits.mplot3d import Axes3D
         import matplotlib as mpl

         fig = plt.figure(figsize=(9, 6))
         ax = fig.gca(projection='3d')
         surf = ax.plot_surface(X, Y, Z, rstride=2, cstride=2,
                 cmap=mpl.cm.coolwarm,
                 linewidth=0.5, antialiased=True)
         ax.set_xlabel('x')
         ax.set_ylabel('y')
         ax.set_zlabel('f(x, y)')
         fig.colorbar(surf, shrink=0.5, aspect=5)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

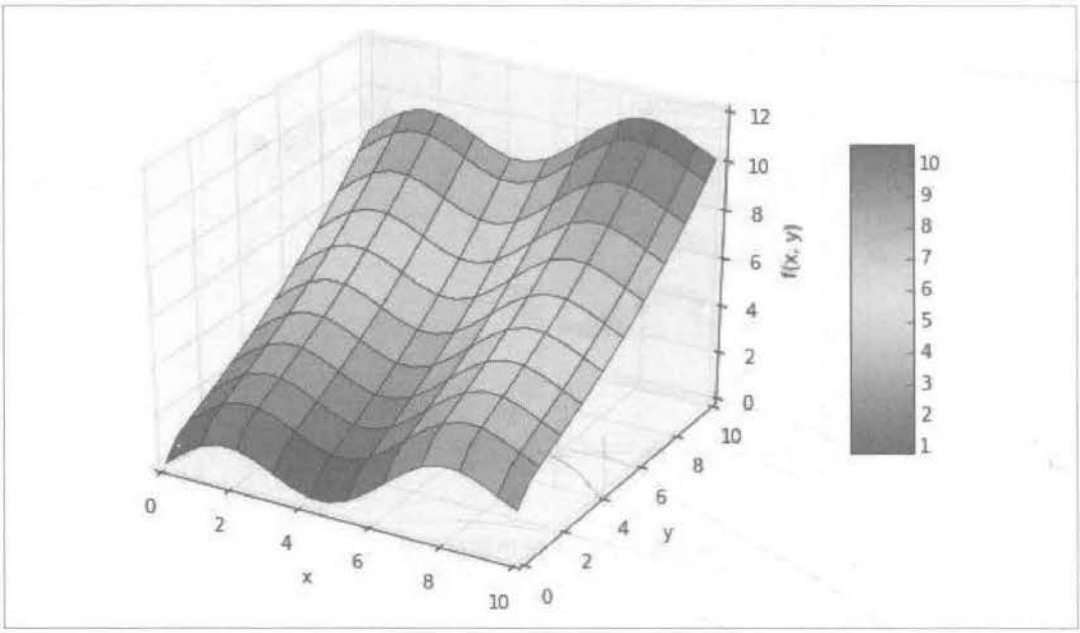


图 9-9　使用两个参数的函数

为了获得好的回归结果，我们编辑一组基函数，包括一个 **sin** 和一个 **sqrt** 函数，利用了我们对示例函数的认识：

```
In [33]: matrix = np.zeros((len(x), 6 + 1))
         matrix[:, 6] = np.sqrt(y)
         matrix[:, 5] = np.sin(x)
         matrix[:, 4] = y ** 2
         matrix[:, 3] = x ** 2
         matrix[:, 2] = y
         matrix[:, 1] = x
         matrix[:, 0] = 1
```

statsmodels 库提供相当通用和有益的函数 **OLS**，可以用于一维和多维最小二乘回归[1]：

```
In [34]: import statsmodels.api as sm
In [35]: model = sm.OLS(fm((x, y)), matrix).fit()
```

OLS 函数的好处之一是提供关于回归及其质量的大量附加信息。调用 model.summary 可以访问结果的一个摘要。单独统计数字（如*确定系数*）通常可以直接访问：

```
In [36]: model.rsquared
Out[36]: 1.0
```

对于我们的目的，当然需要最优回归参数，这些参数保存在 **model** 对象的 **params** 属性中：

1　OLS 使用的细节可以参考文档（http://bit.ly/using_ols）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [37]: a = model.params
         a
Out[37]: array([  7.14706072e-15,   2.50000000e-01,  -2.22044605e-16,
                 -1.02348685e-16,   5.00000000e-02,   1.00000000e+00,
                  1.00000000e+00])
```

reg_func 返回给定最优回归参数和自变量数据点的回归函数值：

```
In [38]: def reg_func(a, (x, y)):
             f6 = a[6] * np.sqrt(y)
             f5 = a[5] * np.sin(x)
             f4 = a[4] * y ** 2
             f3 = a[3] * x ** 2
             f2 = a[2] * y
             f1 = a[1] * x
             f0 = a[0] * 1
             return (f6 + f5 + f4 + f3 +
                     f2 + f1 + f0)
```

然后，可以将这些值与示例函数原始构成比较，如图 9-10 所示：

```
In [39]: RZ = reg_func(a, (X, Y))
In [40]: fig = plt.figure(figsize=(9, 6))
         ax = fig.gca(projection='3d')
         surf1 = ax.plot_surface(X, Y, Z, rstride=2, cstride=2,
                     cmap=mpl.cm.coolwarm, linewidth=0.5,
                     antialiased=True)
         surf2 = ax.plot_wireframe(X, Y, RZ, rstride=2, cstride=2,
                                   label='regression')
         ax.set_xlabel('x')
         ax.set_ylabel('y')
         ax.set_zlabel('f(x, y)')
         ax.legend()
         fig.colorbar(surf, shrink=0.5, aspect=5)
```

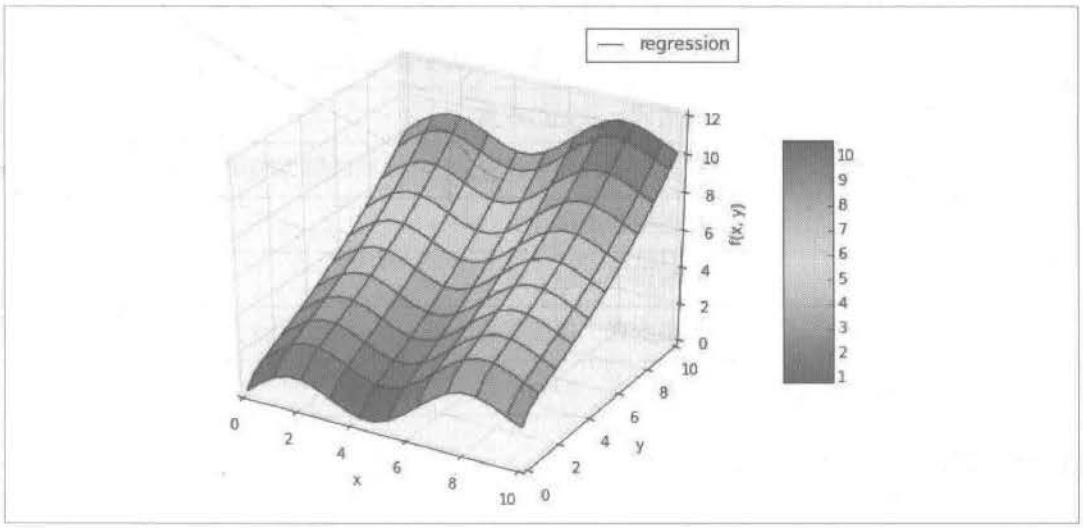


图 9-10　更高维的回归

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

回归

最小二乘回归方法有多种应用领域，包括简单的函数逼近和基于有噪声或者未排序数据的函数逼近。这些方法可以应用于一维问题，也可以应用于多维问题。由于这种方法的基础数学理论使然，在一维问题和多维问题上的应用总是“几乎相同”。

9.1.2 插值

与回归相比，插值（例如，三次样条插值）在数学上更为复杂。它还被限制在低维度问题上。给定一组有序的观测点（按照 x 维排序），基本的思路是在两个相邻数据点之间进行回归，不仅产生的分段插值函数完全匹配数据点，而且函数在数据点上连续可微分。连续可微分性需要至少三阶插值——也就是三次样条插值。然而，这种方法一般也适用于四次甚至线性样条插值。首先，导入对应的子库：

```
In [41]: import scipy.interpolate as spi
In [42]: x = np.linspace(-2 * np.pi, 2 * np.pi, 25)
```

为了说明的目的，我们再次使用原始示例函数：

```
In [43]: def f(x):
             return np.sin(x) + 0.5 * x
```

在给定以 x 排序的数据点集时，应用这种方法和 **polyfit**、**polyval** 一样简单，此时对应的函数是 **splrep** 和 **splev**。表 9-2 列出了 **splrep** 函数的主要参数。

表 9-2 splrep 函数参数

参数	描述
x	（有序）x 坐标（自变量值）
y	（按 x 排序）y 坐标（因变量值）
w	应用到 y 坐标的权重
xb, xe	拟合区间，如果设置为 None 则区间为[x[0]，[-1]]
k	样条拟合顺序（1<=k<=5）
s	平滑因子（越大越平滑）
full_output	如果为真，返回附加输出
quiet	如果为真，抑制消息

表 9-3 列出 splev 函数的参数。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

表 9-3　splev 函数参数

参数	描述
x	（有序）x 坐标（自变量值）
tck	splrep 返回的长度为 3 的序列（节点，系数，阶数）
der	导数的阶（0 为元函数，1 为一阶导数）
ext	如果 x 不在节点序列中时的行为（0 外推，1 返回 0，2 引发 ValueError 异常）

应用到当前示例，可以转化为如下代码：

```
In [44]: ipo = spi.splrep(x, f(x), k=1)
In [45]: iy = spi.splev(x, ipo)
```

如图 9-11 所示，一次样条插值（k=1）的结果似乎已经很好了：

```
In [46]: plt.plot(x, f(x), 'b', label='f(x)')
         plt.plot(x, iy, 'r.', label='interpolation')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```

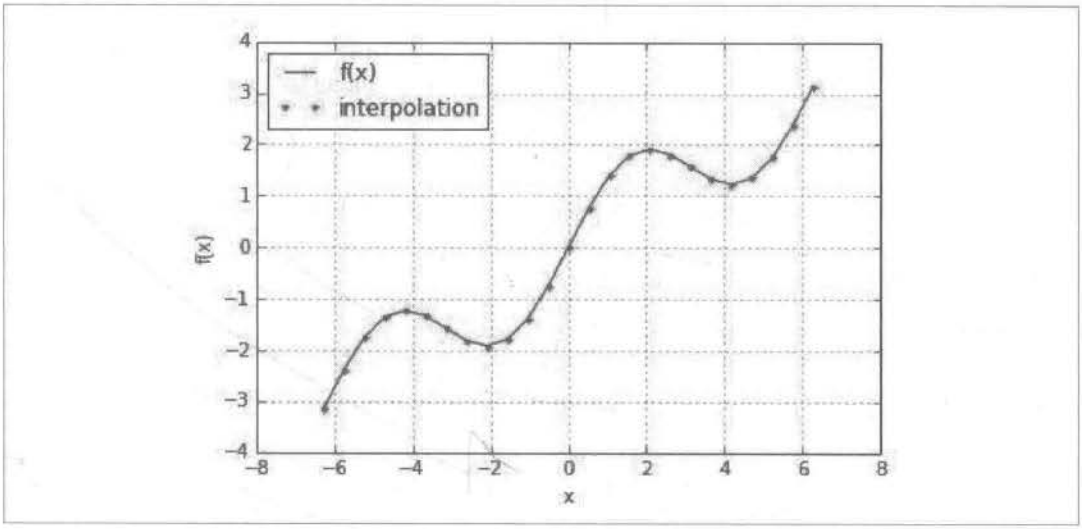


图 9-11　一次样条插值示例图表

这可以用数值化方式加以确认：

```
In [47]: np.allclose(f(x), iy)
Out[47]: True
```

样条插值在金融学中往往用于估算未包含在原始观测点中的自变量数据点的因变量值。为次，我们选择一个更小的区间，仔细观察一次样条插入的值：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [48]: xd = np.linspace(1.0, 3.0, 50)
         iyd = spi.splev(xd, ipo)
```

图 9-12 说明，插值函数确实*线性地*在两个观测点之间插值。对于某些应用，这可能不够精确。此外，很明显函数在原始数据点上不是连续可微分的——这是另一个不足：

```
In [49]: plt.plot(xd, f(xd), 'b', label='f(x)')
         plt.plot(xd, iyd, 'r.', label='interpolation')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```

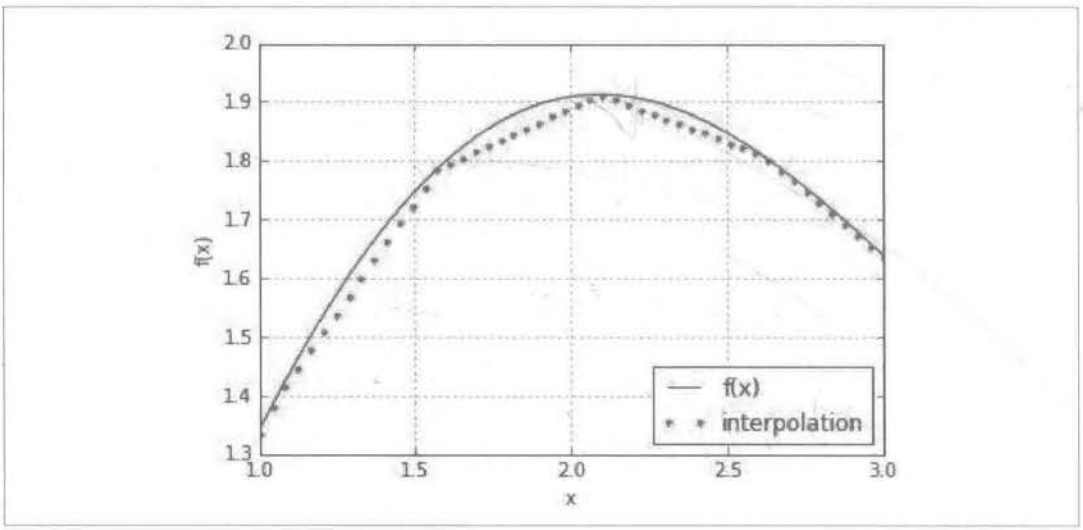


图 9-12　线性插值示例图表（细节）

因此，我们这次使用三次样条插值完成练习：

```
In [50]: ipo = spi.splrep(x, f(x), k=3)
         iyd = spi.splev(xd, ipo)
```

现在，图 9-13 中的详细子区间展示的插值从图形上看很完美：

```
In [51]: plt.plot(xd, f(xd), 'b', label='f(x)')
         plt.plot(xd, iyd, 'r.', label='interpolation')
         plt.legend(loc=0)
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('f(x)')
```

从数值上看，这种插值并不完美，但是 MSE 确实很小：

```
In [52]: np.allclose(f(xd), iyd)
Out[52]: False
In [53]: np.sum((f(xd) - iyd) ** 2) / len(xd)
Out[53]: 1.1349319851436252e-08
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

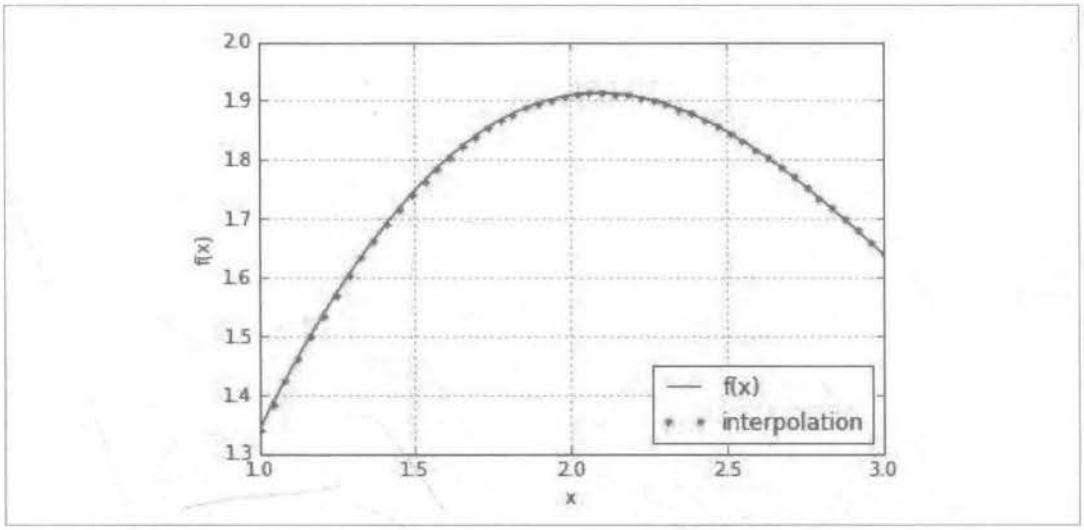


图 9-13　三次样条插值示例图表（细节）

插值

在可以应用样条插值的情况下，可以预期比最小二乘回归方法更好的近似结果。但是要记住，你必须有排序（且“无噪声”）的数据，该方法仅限于低维度问题。样条插值的计算要求也更高，在某些用例中可能导致花费的时间比回归方法长得多。

9.2　凸优化

在金融学和经济学中，*凸优化*起着重要的作用。这方面的例子包括根据市场数据校准期权定价模型，或者效用函数的优化。我们以下面定义的函数 **fm** 为例进行这种优化：

```
In [54]: def fm((x, y)):
             return (np.sin(x) + 0.05 * x ** 2
                   + np.sin(y) + 0.05 * y ** 2)

In [55]: x = np.linspace(-10, 10, 50)
         y = np.linspace(-10, 10, 50)
         X, Y = np.meshgrid(x, y)
         Z = fm((X, Y))
```

图 9-14 展示了上述函数在定义的 **x** 和 **y** 区间内的图形。从视觉上已经可以看出这个函数有多个局部极小值。从这个特定的图形表现中无法真正确认全局最小值是否存在：

```
In [56]: fig = plt.figure(figsize=(9, 6))
         ax = fig.gca(projection='3d')
         surf = ax.plot_surface(X, Y, Z, rstride=2, cstride=2,
                cmap=mpl.cm.coolwarm,
                linewidth=0.5, antialiased=True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('f(x, y)')
fig.colorbar(surf, shrink=0.5, aspect=5)
```

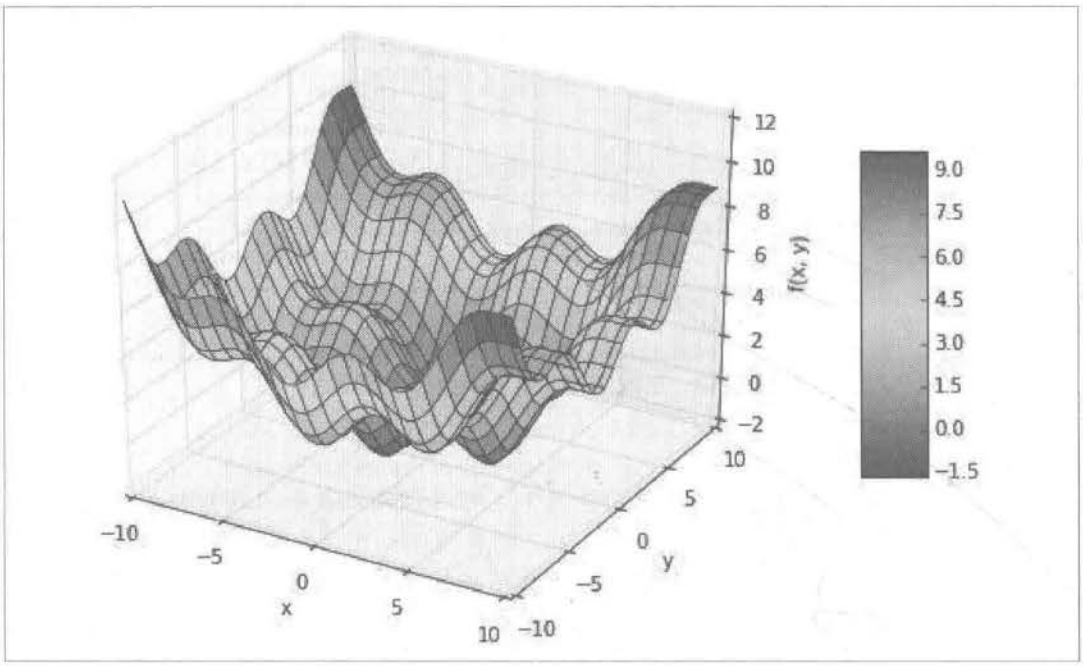


图 9-14　以两个参数最小化的函数

下面，我们打算实施全局最小化方法和局部最小化方法。所使用的 **brute** 和 **fmin** 函数可以在子库 **scipy.optimize** 中找到：

```
In [57]: import scipy.optimize as spo
```

9.2.1　全局优化

为了更仔细地观察初始化最小化过程时后台发生的情况，我们改善原函数，通过一个选项输出当前参数值和函数值：

```
In [58]: def fo((x, y)):
             z = np.sin(x) + 0.05 * x ** 2 + np.sin(y) + 0.05 * y ** 2
             if output == True:
                 print '%8.4f %8.4f %8.4f' % (x, y, z)
             return z
```

正如后面的代码和对应的输出所示，这使我们可以跟踪过程的所有相关信息。**brute** 以参数范围作为输入。例如，提供 x 值范围（-10，10.1，5）会得到-10、15、0、5、10 这 5 个“测试值”：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [59]: output = True
         spo.brute(fo, ((-10, 10.1, 5), (-10, 10.1, 5)), finish=None)
Out[59]: -10.0000 -10.0000  11.0880
         -10.0000 -10.0000  11.0880
         -10.0000  -5.0000   7.7529
         -10.0000   0.0000   5.5440
         -10.0000   5.0000   5.8351
         -10.0000  10.0000  10.0000
          -5.0000 -10.0000   7.7529
          -5.0000  -5.0000   4.4178
          -5.0000   0.0000   2.2089
          -5.0000   5.0000   2.5000
          -5.0000  10.0000   6.6649
           0.0000 -10.0000   5.5440
           0.0000  -5.0000   2.2089
           0.0000   0.0000   0.0000
           0.0000   5.0000   0.2911
           0.0000  10.0000   4.4560
           5.0000 -10.0000   5.8351
           5.0000  -5.0000   2.5000
           5.0000   0.0000   0.2911
           5.0000   5.0000   0.5822
           5.0000  10.0000   4.7471
          10.0000 -10.0000  10.0000
          10.0000  -5.0000   6.6649
          10.0000   0.0000   4.4560
          10.0000   5.0000   4.7471
          10.0000  10.0000   8.9120

         array([ 0.,  0.])
```

在给定的函数初始参数化条件下，最优化参数值是 **x=y=0**。回顾前面的输出可以看出，结果函数值也为 0。第一次参数化相当粗糙，我们对两个输入参数均使用 5 的步距。这当然有很大的微调空间，得到更好的结果：

```
In [60]: output = False
         opt1 = spo.brute(fo, ((-10, 10.1, 0.1), (-10, 10.1, 0.1)), finish=None)
         opt1
Out[60]: array([-1.4, -1.4])
In [61]: fm(opt1)
Out[61]: -1.7748994599769203
```

最优化参数值现在是 **x=y=−1.4**，全局最小化的最小函数值大约为**−1.7749**。

9.2.2 局部优化

对于局部凸优化，我们打算利用全局优化的结果。**fmin** 函数的输入是需要最小化的函数和起始参数值。此外，可以定义输入参数宽容度和函数值宽容度，以及最大迭代及函数调用次数：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [62]: output = True
         opt2 = spo.fmin(fo, opt1, xtol=0.001, ftol=0.001, maxiter=15, maxfun=20)
         opt2
Out[62]:  -1.4000  -1.4000  -1.7749
          -1.4700  -1.4000  -1.7743
          -1.4000  -1.4700  -1.7743
          -1.3300  -1.4700  -1.7696
          -1.4350  -1.4175  -1.7756
          -1.4350  -1.3475  -1.7722
          -1.4088  -1.4394  -1.7755
          -1.4438  -1.4569  -1.7751
          -1.4328  -1.4427  -1.7756
          -1.4591  -1.4208  -1.7752
          -1.4213  -1.4347  -1.7757
          -1.4235  -1.4096  -1.7755
          -1.4305  -1.4344  -1.7757
          -1.4168  -1.4516  -1.7753
          -1.4305  -1.4260  -1.7757
          -1.4396  -1.4257  -1.7756
          -1.4259  -1.4325  -1.7757
          -1.4259  -1.4241  -1.7757
          -1.4304  -1.4177  -1.7757
          -1.4270  -1.4288  -1.7757
         Warning: Maximum number of function evaluations has been exceeded.

         array([-1.42702972, -1.42876755])
```

同样，我们可以观察到解决方案的微调和更低的函数值：

```
In [63]: fm(opt2)
Out[63]: -1.7757246992239009
```

在许多凸优化问题中，建议在局部优化之前进行全局优化。主要原因是局部凸优化算法很容易陷入某个局部最小值（所谓的“盆地跳跃”（basin hopping）），而忽略“更好”的局部最小值和全局最小值。下面可以看到，将初始参数化设置为 **x=y=2** 得出高于 0 的“最小”值：

```
In [64]: output = False
         spo.fmin(fo, (2.0, 2.0), maxiter=250)
Out[64]: Optimization terminated successfully.
                  Current function value: 0.015826
                  Iterations: 46
                  Function evaluations: 86

         array([ 4.2710728 ,  4.27106945])
```

9.2.3 有约束优化

目前为止，我们只考虑了无约束优化问题，但是许多类型的经济学或者金融学优化问题都有一个或者多个约束条件。这些约束可能采取等式或者不等式等正规形式。

举个简单的例子，考虑可能投资于两种高风险证券（希望效用最大化）的投资者的效用最大化问题。两种证券今天的价格为 $q_a=q_b=10$。一年之后，状态 u 下它们的收益分别为 15 美元和 5 美元，而在状态 d 下收益分别为 5 美元和 12 美元。两种状态的出现

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

可能性相同。两种证券的向量收益分别记为 r_a 和 r_b。

投资者的投资预算为 w_0=100 美元，并根据效用函数 $u(w)=\sqrt{w}$ 得出未来的财富效率，其中 w 是可用的财富（以美元计算）。公式 9-2 是最优化问题的公式，其中 a，b 是投资者购买的证券数量。

公式 9-2　预期效用最大化问题

$$\begin{aligned}\max_{a,b} -\mathrm{E}(u(w_1)) &= p\sqrt{w_{1u}} + (1-p)\sqrt{w_{1d}} \\ w_1 &= ar_a + br_b \\ w_0 &\geqslant aq_a + bq_b \\ a,b &\geqslant 0\end{aligned} \tag{9-2}$$

代入所有数值化假设，就得到公式 9-3 中的问题。注意，我们还为负数的预期效用最小化做了更改。

公式 9-3　预期效用最大化问题

$$\begin{aligned}\min_{a,b} -\mathrm{E}(u(w_1)) &= -(0.5\cdot\sqrt{w_{1u}} + 0.5\cdot\sqrt{w_{1d}} \\ w_{1u} &= a\cdot 15 + b\cdot 5 \\ w_{1d} &= a\cdot 5 + b\cdot 12 \\ 100 &\geqslant a\cdot 10 + b\cdot 10 \\ a,b &\geqslant 0\end{aligned} \tag{9-3}$$

我们使用 **scipy.optimize.minimize** 函数解决上述问题。除了需要最小化的函数之外，这个函数还以公式和不等式（以字典对象列表的形式）和参数范围（以元组对象元组的形式）作为输入[1]。我们可以将公式 9-3 中的问题翻译为如下代码：

```
In [65]: # function to be minimized
         from math import sqrt
         def Eu((s, b)):
             return -(0.5 * sqrt(s * 15 + b * 5) + 0.5 * sqrt(s * 5 + b * 12))

         # constraints
         cons = ({'type': 'ineq', 'fun': lambda (s, b):  100 - s * 10 - b * 10})
           # budget constraint
         bnds = ((0, 1000), (0, 1000))  # uppper bounds large enough
```

现在，我们已经拥有了使用 **minimize** 函数所需的一切——只需要添加对最优参数的初始猜测：

1　使用 minimize 函数的细节和示例参见文档（http://bit.ly/using_minimize）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [66]: result = spo.minimize(Eu, [5, 5], method='SLSQP',
                               bounds=bnds, constraints=cons)
In [67]: result
Out[67]:   status: 0
         success: True
            njev: 5
            nfev: 21
             fun: -9.700883611487832
               x: array([ 8.02547122,  1.97452878])
         message: 'Optimization terminated successfully.'
             jac: array([-0.48508096, -0.48489535,  0.        ])
             nit: 5
```

该函数返回一个**字典**对象。最优参数可以这样读出：

```
In [68]: result['x']
Out[68]: array([ 8.02547122,  1.97452878])
```

最优函数值（同样要改变符号）如下：

```
In [69]: -result['fun']
Out[69]: 9.700883611487832
```

按照对上述简单模型的参数化，投资者的最优选择是购买大约 8 个单位的 a 证券和大约 2 个单位的 b 证券。预算是固定的，也就是说，投资者在证券中共投入 100 美元资金。这很容易通过取得最优参数向量和价格向量的点乘来验证：

```
In [70]: np.dot(result['x'], [10, 10])
Out[70]: 99.999999999999986
```

9.3 积分

在估值和期权定价时，积分是一个重要的数学工具。这来源于一个事实：衍生物的风险中立价值一般可以用风险中立（鞅）测度下的*预期*折现收益来表示。这一预期在离散情况下是个总和，在连续情况下是一个积分。**scipy.integrate** 子库提供了数值积分所用的不同函数：

```
In [71]: import scipy.integrate as sci
```

同样，我们继续使用由 sin 部分和线性部分组成的示例函数：

```
In [72]: def f(x):
             return np.sin(x) + 0.5 * x
```

我们感兴趣的是[0.5，9.5]区间内的积分，也就是公式 9-4 表示的积分。

公式 9-4　示例函数的积分

$$\int_{0.5}^{9.5} \sin(x) + 0.5x dx \tag{9-4}$$

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 9-15 提供了上述积分与函数 f(x)≡sin(x)+0.5x 的图形表示：

```
In [73]: a = 0.5  # left integral limit
         b = 9.5  # right integral limit
         x = np.linspace(0, 10)
         y = f(x)

In [74]: from matplotlib.patches import Polygon

         fig, ax = plt.subplots(figsize=(7, 5))
         plt.plot(x, y, 'b', linewidth=2)
         plt.ylim(ymin=0)

         # area under the function
         # between lower and upper limit
         Ix = np.linspace(a, b)
         Iy = f(Ix)
         verts = [(a, 0)] + list(zip(Ix, Iy)) + [(b, 0)]
         poly = Polygon(verts, facecolor='0.7', edgecolor='0.5')
         ax.add_patch(poly)

         # labels
         plt.text(0.75 * (a + b), 1.5, r"$\int_a^b f(x)dx$",
                  horizontalalignment='center', fontsize=20)

         plt.figtext(0.9, 0.075, '$x$')
         plt.figtext(0.075, 0.9, '$f(x)$')

         ax.set_xticks((a, b))
         ax.set_xticklabels(('$a$', '$b$'))
         ax.set_yticks([f(a), f(b)])
```

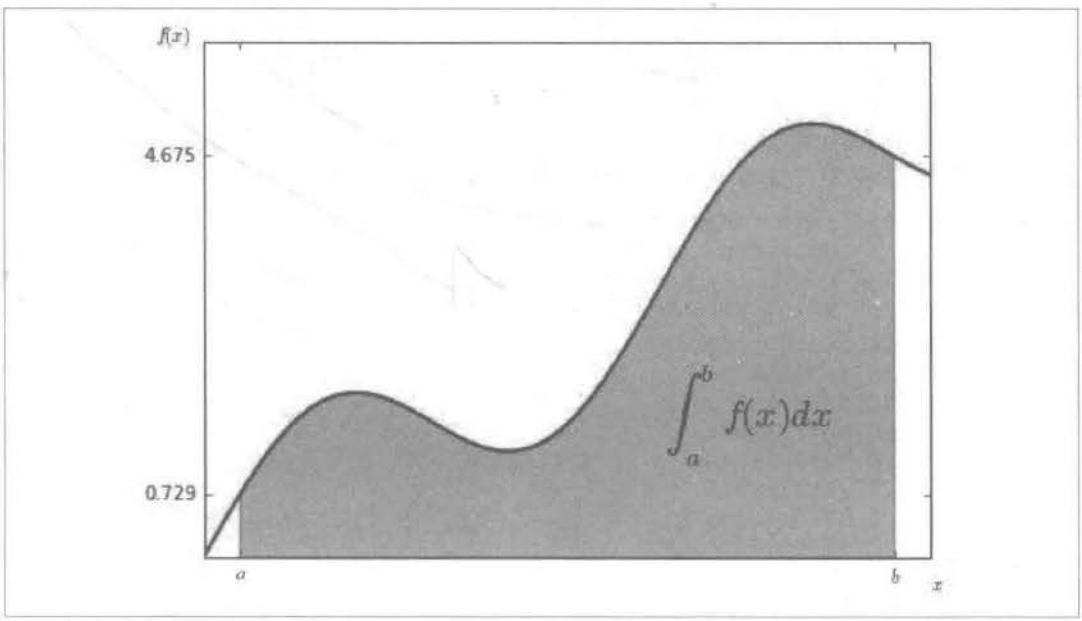


图 9-15　包含积分区域的示例函数

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

9.3.1 数值积分

integrate 子库包含一组精选的函数，可以计算给定上下限和数学函数下的数值积分。这些函数的例子包含用于固定高斯求积的 **fixed_quad**、用于自适应求积的 **quad** 和用于龙贝格积分的 **romberg**：

```
In [75]: sci.fixed_quad(f, a, b)[0]
Out[75]: 24.366995967084588
In [76]: sci.quad(f, a, b)[0]
Out[76]: 24.374754718086752
In [77]: sci.romberg(f, a, b)
Out[77]: 24.374754718086713
```

还有一些积分函数以输入**列表**或者包含函数值和输入值的 **ndarray** 对象作为输入。这种函数的例子包括使用梯形法则的 **trapz**，和实现辛普森法则的 **simps**：

```
In [78]: xi = np.linspace(0.5, 9.5, 25)
In [79]: sci.trapz(f(xi), xi)
Out[79]: 24.352733271544516
In [80]: sci.simps(f(xi), xi)
Out[80]: 24.374964184550748
```

9.3.2 通过模拟求取积分

通过蒙特卡洛模拟（参见第 10 章）的期权和衍生物估值依赖于通过模拟求取积分。为此，在积分区间内取 I 个随机的 x 值，并计算每个随机 x 值处的积分函数值。加总所有函数值并求其平均值，就可以得到积分区间中的平均函数值。将该值乘以积分区间长度，可以得出估算的积分值。

下面代码说明蒙特卡洛估算积分值如何随着提取的随机数个数的增加而收敛。即使提取的随机数个数较少，估算值也已经相当接近：

```
In [81]: for i in range(1, 20):
             np.random.seed(1000)
             x = np.random.random(i * 10) * (b - a) + a
             print np.sum(f(x)) / len(x) * (b - a)
Out[81]: 24.8047622793
         26.5229188983
         26.2655475192
         26.0277033994
         24.9995418144
         23.8818101416
         23.5279122748
         23.507857659
         23.6723674607
         23.6794104161
         24.4244017079
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
24.2390053468
24.115396925
24.4241919876
23.9249330805
24.1948421203
24.1173483782
24.1006909297
23.7690510985
```

9.4 符号计算

前几个小节主要关心的是数值计算。本届介绍*符号*计算，这种计算在许多金融领域中很有益。为此，我们导入专用于符号计算的 **SymPy** 库：

```
In [82]: import sympy as sy
```

9.4.1 基本知识

SymPy 引入了新的对象类。最基本的是 **Symbol** 类：

```
In [83]: x = sy.Symbol('x')
         y = sy.Symbol('y')
In [84]: type(x)
Out[84]: sympy.core.symbol.Symbol
```

和 **NumPy** 一样，**SymPy** 有一些（数学）函数定义。例如：

```
In [85]: sy.sqrt(x)
Out[85]: sqrt(x)
```

这已经说明了一项重大差异。尽管 x 没有数字值，但是 **SymPy** 已经定义了 x 的平方根，因为 x 是一个符号（**Symbol**）对象。在这个意义上，**sy.sqrt(x)**可以成为任何数学表达式的一部分。注意，**SymPy** 通常自动简化给定的数学表达式：

```
In [86]: 3 + sy.sqrt(x) - 4 ** 2
Out[86]: sqrt(x) - 13
```

类似地，可以用符号对象定义任何函数。它们不会和 *Python* 函数混淆：

```
In [87]: f = x ** 2 + 3 + 0.5 * x ** 2 + 3 / 2
In [88]: sy.simplify(f)
Out[88]: 1.5*x**2 + 4
```

SymPy 为数学表达式提供了 3 个基本的渲染器：

1. 基于 LaTex 的渲染器
2. 基于 unicode 的渲染器
3. 基于 ASCII 的渲染器

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例如，只使用 **IPython Notebook** 工作时，LaTex 渲染通常是好（在视觉上引人入胜）的选择。下面，我们将一直使用最简单的选项 ASCII，以表明不涉及任何人工制作的排版设定：

```
In [89]: sy.init_printing(pretty_print=False, use_unicode=False)
In [90]: print sy.pretty(f)
Out[90]:      2
         1.5*x  + 4
```

从输出可以看到，在必要时会使用多行。下面再看一个平方根函数直观表示的例子：

```
In [91]: print sy.pretty(sy.sqrt(x) + 0.5)
Out[91]:   ___
         \/ x  + 0.5
```

这里我们不过多地纠缠细节，**SymPy** 还提供了许多其他有用的数学函数——例如，用于求 π 值的函数。下面展示 π 的字符串表示的前 40 个字符（最多可达 40 万位）：

```
In [92]: pi_str = str(sy.N(sy.pi, 400000))
         pi_str[:40]
Out[92]: '3.1415926535897932384626433832795028841'
```

下面是前 40 万位中的最后 40 个数字：

```
In [93]: pi_str[-40:]
Out[93]: '8245672736856312185020980470362464176198'
```

如果你希望，也可以查找你的生日；不过，不保证能够找到：

```
In [94]: pi_str.find('111272')
Out[94]: 366713
```

9.4.2 方程式

SymPy 的长处之一是解方程，例如 $x^2-1=0$ 这样的形式：

```
In [95]: sy.solve(x ** 2 - 1)
Out[95]: [-1, 1]
```

一般来说，**SymPy** 假定你寻找的是使指定表达式为 0 的方程解。因此，$x^2-1=3$ 这样的方程可能必须改写，才能获得想要的结果：

```
In [96]: sy.solve(x ** 2 - 1 - 3)
Out[96]: [-2, 2]
```

当然，**SymPy** 可以处理更复杂的表达式，如 $x^3 + 0.5\,x^2 - 1 = 0$：

```
In [97]: sy.solve(x ** 3 + 0.5 * x ** 2 - 1)
Out[97]: [0.858094329496553, -0.679047164748276 - 0.839206763026694*I,
         -0.679047164748276 + 0.839206763026694*I]
```

然而，不管从数学的角度（解的存在性）还是从算法的角度（实现），都很明显不能保证有解。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

对于有超过一个输入参数的函数，**SymPy** 也同样有效，对于复数也是如此。举个简单的例子，求 $x^2+y^2=0$ 的解：

```
In [98]: sy.solve(x ** 2 + y ** 2)
Out[98]: [{x: -I*y}, {x: I*y}]
```

9.4.3 积分

SymPy 的另一个长处是积分和微分。下面，我们回到用于数值和模拟积分的示例函数，现在既要求出符号解，也要求出精确的数值解。我们需要积分上下限的符号：

```
In [99]: a, b = sy.symbols('a b')
```

定义新符号之后，可以“漂亮地打印”符号积分：

```
In [100]: print sy.pretty(sy.Integral(sy.sin(x) + 0.5 * x, (x, a, b)))
Out[100]:   b
            /
           |
           |  (0.5*x + sin(x)) dx
           |
          /
          a
```

使用 **integrate**，我们可以得出积分函数的*反导数*（不定积分）：

```
In [101]: int_func = sy.integrate(sy.sin(x) + 0.5 * x, x)
In [102]: print sy.pretty(int_func)
Out[102]:       2
          0.25*x  - cos(x)
```

有了反导数，求积分只需要三步。要求取 **SymPy** 表达式的值，用方法 **subs** 将数值代入对应的符号，在新表达式上调用方法 **evalf**：

```
In [103]: Fb = int_func.subs(x, 9.5).evalf()
          Fa = int_func.subs(x, 0.5).evalf()
```

Fb 和 Fa 的差就是积分的准确值：

```
In [104]: Fb - Fa  # exact value of integral
Out[104]: 24.3747547180867
```

积分也可以用符号积分上下限得到符号解：

```
In [105]: int_func_limits = sy.integrate(sy.sin(x) + 0.5 * x, (x, a, b))
          print sy.pretty(int_func_limits)
Out[105]:         2         2
          - 0.25*a  + 0.25*b  + cos(a) - cos(b)
```

和前面一样，代入数值——这次使用一个字典对象代表多个替代值——并求值可以得到积分值：

```
In [106]: int_func_limits.subs({a : 0.5, b : 9.5}).evalf()
Out[106]: 24.3747547180868
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

最后，提供量化的积分上下限，在一步中得出准确的值：

```
In [107]: sy.integrate(sy.sin(x) + 0.5 * x, (x, 0.5, 9.5))
Out[107]: 24.3747547180867
```

9.4.4 微分

对不定积分求导通常应该得出原函数。我们对前面的符号反导数应用 **diff** 函数，检查这一点：

```
In [108]: int_func.diff()
Out[108]: 0.5*x + sin(x)
```

和积分的示例一样，我们现在使用微分得出前面的凸最小化问题的解。为此，我们定义如下的符号函数：

```
In [109]: f = (sy.sin(x) + 0.05 * x ** 2
               + sy.sin(y) + 0.05 * y ** 2)
```

为了最小化，我们需要两个变量 x 和 y 的偏微分：

```
In [110]: del_x = sy.diff(f, x)
          del_x
Out[110]: 0.1*x + cos(x)
In [111]: del_y = sy.diff(f, y)
          del_y
Out[111]: 0.1*y + cos(y)
```

全局最小值的必要（但不充分）条件之一是两个偏微分都为 0。如前所述，这不能保证有符号解。算法和（多种）存在性问题在这里都起作用。然而，我们可以从数值上求出两个方程式的解，根据前面的全局和局部最小化工作提供“有根据”的推测：

```
In [112]: xo = sy.nsolve(del_x, -1.5)
          xo
Out[112]: mpf('-1.4275517787645941')
In [113]: yo = sy.nsolve(del_y, -1.5)
          yo
Out[113]: mpf('-1.4275517787645941')
In [114]: f.subs({x : xo, y : yo}).evalf()
            # global minimum
Out[114]: -1.77572565314742
```

提供没有根据/随机的猜测，同样可能使算法陷入某个局部最小值，而非全局最小值：

```
In [115]: xo = sy.nsolve(del_x, 1.5)
          xo
Out[115]: mpf('1.7463292822528528')
In [116]: yo = sy.nsolve(del_y, 1.5)
          yo
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[116]: mpf('1.7463292822528528')
In [117]: f.subs({x : xo, y : yo}).evalf()
            # local minimum
Out[117]: 2.27423381055640
```

这从数值上说明了偏微分为 0 是必要条件，但并非充分条件。

符号计算

用 **Python** 解决数学问题时，应该时刻考虑到 **SymPy** 和符号计算。特别是对于交互式的金融分析，这可能是比非符号方法更高效的方法。

9.5 结语

本章介绍了一些对金融学很重要的数学主题和工具。*函数逼近*在许多金融领域都很重要，例如基于曲线插值和回归的美式期权蒙特卡洛估值方法。*凸优化*技术在金融学中也常常需要用到；例如，用期权的市场报价或者隐含波动率检验参数化期权定价模型。

*数值积分*是很多问题的中心，例如期权和衍生品的定价问题。得出一组随机过程的风险中立概率测度之后，期权的定价就可以归结于求取风险中立测度下的期权预期收益，并折现为当前日期的价值。第 10 章介绍多种随机过程在风险中立测度下的模拟。

最后，本章介绍了使用 SymPy 进行*符号计算*的方法。对于一些数学运算（如积分、微分或者方程式的解），符号计算可以提供一种真正实用、高效的工具。

9.6 延伸阅读

本章使用的 **Python** 库的进一步信息可以参考如下 **Web** 资源。

- **NumPy** 使用的所有函数参见 http://docs.scipy.org/doc/numpy/reference/。
- **statsmodels** 库的文档：http://statsmodels.sourceforge.net。
- **scipy.optimize** 的细节可以访问 http://docs.scipy.org/doc/scipy/reference/optimize.html。
- **scipy.integrate** 的积分运算在这里可以找到解释：http://docs.scipy.org/doc/scipy/reference/integrate.html。

下面是本章所介绍的数学概念的一本很好的参考书。

- Brandimarte, Paolo (2006): Numerical Methods in Finance and Economics, 2nd ed.John Wiley & Sons, Hoboken, NJ

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 10 章

推断统计学

可预测性不是事物的走向，而是它们的可能趋势。

——Raheel Farooq

当今，推断统计学是金融学中最重要的数学和数值学科。在现代金融学发端之时（20 世纪 70 年代和 80 年代），金融研究的主要目标是提出封闭式的解决方案，例如，在特定金融模型下的期权定价。近年来需求有了根本的变化，不仅需要准确地估计对金融市场参与者很重要的单一金融工具的价值，还要一致性地估计整个衍生品谱系的价格。类似地，为了在整个金融机构中提出一致性的风险测度，例如风险价值和信用价值调整，需要将该机构及所有同行当成一个整体来考虑。这种令人畏缩的任务只能通过灵活高效的数值方法来处理。因此，推断统计学和作为特例的蒙特卡洛模拟被提升到了显要的地位。

本章从 **Python** 的角度介绍如下主题。

随机数生成

一切都从（伪）随机数开始，它是所有模拟工作的基础；尽管准随机数（例如，基于 Sobol 序列）在金融学中相当流行，但是伪随机数似乎仍是基准。

模拟

在金融学中，两种模拟工作特别重要：随机变量和随机过程的模拟。

估值

估值有两种主要方式：欧式行权（特定日期）的衍生品估值和美式行权（在特定时期内）估值；另外还有用于百慕大行权（一组有限的特定日期）的估值工具。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

模拟本身需要借助风险测度的计算，例如风险价值、信用风险价值和信用价值调整。

10.1 随机数

在本章中，我们都将用 **numpy.random** 子库提供的函数生成随机数[1]：

```
In [1]: import numpy as np
        import numpy.random as npr
        import matplotlib.pyplot as plt
        %matplotlib inline
```

例如，**rand** 函数返回开区间[0，1）内的随机数，随机数的个数由参数指定。返回对象是一个 **ndarray** 对象：

```
In [2]: npr.rand(10)
Out[2]: array([ 0.40628966,  0.43098644,  0.9435419 ,  0.26760198,  0.2729951 ,
                0.67519064,  0.41349754,  0.3585647 ,  0.07450132,  0.95130158])
In [3]: npr.rand(5, 5)
Out[3]: array([[ 0.87263851,  0.8143348 ,  0.34154499,  0.56695052,  0.60645041],
               [ 0.39398181,  0.71671577,  0.63568321,  0.61652708,  0.93526172],
               [ 0.12632038,  0.35793789,  0.04241014,  0.88085228,  0.54260211],
               [ 0.14503456,  0.32939077,  0.28834351,  0.4050322 ,  0.21120017],
               [ 0.45345805,  0.29771411,  0.67157606,  0.73563706,  0.48003387]
       ])
```

这些数值很容易转换，以覆盖其余实际区间。例如，如果想生成区间[5，10）内的随机数，可以这样转换 **rand** 的返回值：

```
In [4]: a = 5.
        b = 10.
        npr.rand(10) * (b - a) + a
Out[4]: array([ 7.27123881,  6.51309437,  7.51380629,  7.84258434,  7.62199611,
                8.86229349,  6.78202851,  6.33248656,  8.10776244,  9.48668419])
```

由于 **NumPy** 的广播特性，这也适合于多维数组：

```
In [5]: npr.rand(5, 5) * (b - a) + a
Out[5]: array([[ 6.65649828,  6.51657569,  9.7912274 ,  8.93721206,  6.66937996],
               [ 8.97919481,  8.27547365,  5.00975386,  8.99797249,  6.05374605],
               [ 7.50268777,  8.43810167,  9.33608096,  8.5513646 ,  5.53651748],
               [ 7.04179874,  6.98111966,  8.42677435,  6.22325043,  6.39226557],
               [ 9.88334499,  7.59597546,  5.93724861,  5.39285822,  5.28435207]
       ])
```

表 10-1 列出了生成简单随机数的函数[2]。

1 为了简单起见，我们将使用的所有伪随机数都称作随机数。——原注

2 参见 http://docs.scipy.org/doc/numpy/reference/routines.random.html。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

表 10-1　简单随机数生成函数

函数	参数	描述
rand	d0, d1, …, dn	指定组成的随机数
randn	d0, d1, …, dn	来自标准正态分布的一个（或者多个）样本
randint	low[, high, size]	从 low（含）到 high（不含）的随机整数
random_integers	low[, high, size]	low 和 high（含）之间的随机整数
random_sample	[size]	半开区间[0.0, 1.0)内的随机浮点数
random	[size]	半开区间[0.0, 1.0)内的随机浮点数
ranf	[size]	半开区间[0.0, 1.0)内的随机浮点数
sample	[size]	半开区间[0.0, 1.0)内的随机浮点数
choice	a[, size, replace, p]	给定一维数组中的随机样本
bytes	length	随机字节

下面我们将从表 10-1 中选择的函数生成的随机数可视化：

```
In [6]: sample_size = 500
        rn1 = npr.rand(sample_size, 3)
        rn2 = npr.randint(0, 10, sample_size)
        rn3 = npr.sample(size=sample_size)
        a = [0, 25, 50, 75, 100]
        rn4 = npr.choice(a, size=sample_size)
```

图 10-1 显示了两个连续分布和两个离散分布的结果：

```
In [7]: fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2,
                                                     figsize=(7, 7))
        ax1.hist(rn1, bins=25, stacked=True)
        ax1.set_title('rand')
        ax1.set_ylabel('frequency')
        ax1.grid(True)
        ax2.hist(rn2, bins=25)
        ax2.set_title('randint')
        ax2.grid(True)
        ax3.hist(rn3, bins=25)
        ax3.set_title('sample')
        ax3.set_ylabel('frequency')
        ax3.grid(True)
        ax4.hist(rn4, bins=25)
        ax4.set_title('choice')
        ax4.grid(True)
```

表 10-2 列出了根据不同分布生成随机数的函数[1]。

1　参见 http://docs.scipy.org/doc/numpy/reference/routines.random.html。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

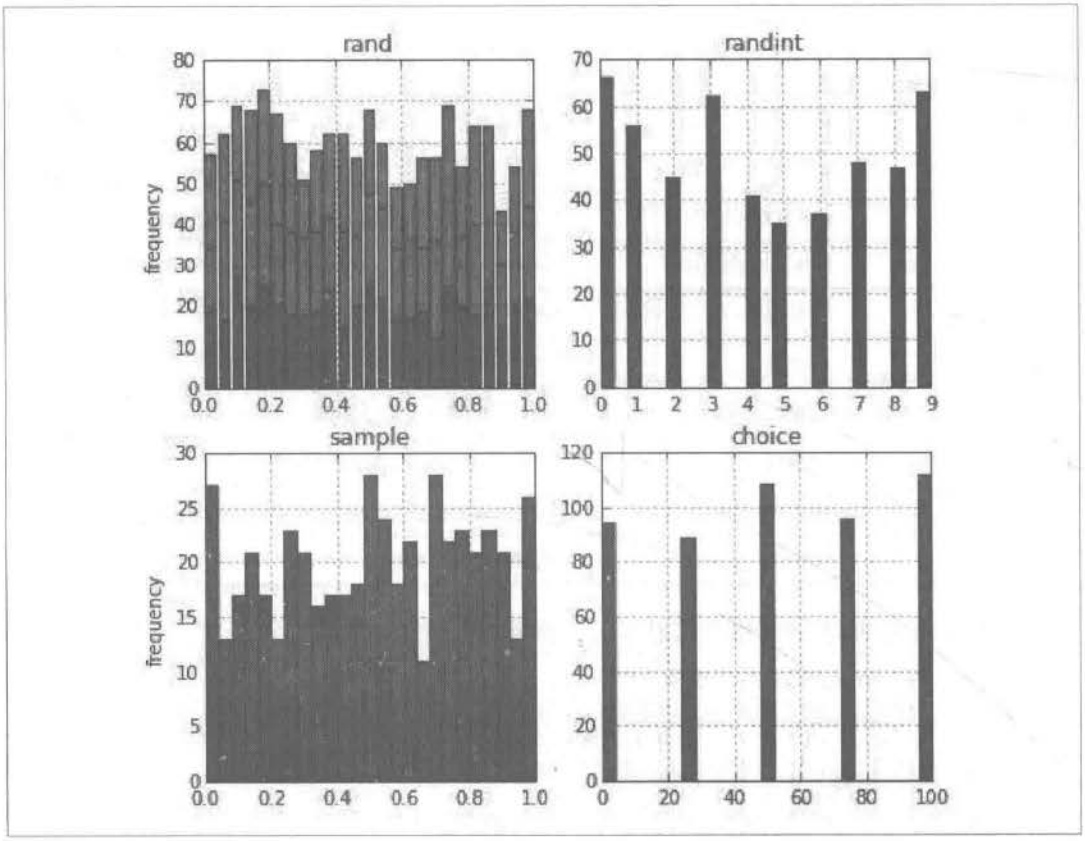


图 10-1　简单伪随机数

表 10-2　根据不同分布生成随机数的函数

函数	参数	描述
beta	a, b[, size]	[0，1]区间上的β分布样本
binomial	n, p[, size]	二项分布样本
chisquare	df[, size]	卡方分布样本
dirichlet	alpha[, size]	狄利克雷分布样本
exponential	[scale, size]	指数分布样本
f	dfnum, dfden[, size]	F 分布样本
gamma	shape[, scale, size]	γ分布样本
geometric	p[, size]	集合分布样本
gumbel	[loc, scale, size]	冈贝尔分布样本
hypergeometric	ngood, nbad, nsample[,size]	超几何分布样本
laplace	[loc, scale, size]	拉普拉斯分布或者双指数分布样本

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

续表

函数	参数	描述
Logistic	[loc, scale, size]	逻辑分布样本
lognormalv	[mean, sigma, size]	对数正态分布样本
logseries	p[, size]	对数序列分布样本
multinomial	n, pvals[, size]	多项分布样本
multivariate_normal	mean, cov[, size]	多变量正态分布样本
negative_binomial	n, p[, size]	负二项式分布样本
noncentral_chisquare	df, nonc[, size]	非中心卡方分布样本
noncentral_f	dfnum, dfden, nonc[, size]	非中心 F 分布样本
normal	[loc, scale, size]	正态（高斯）分布样本
pareto	a[, size]	特定组成的帕累托II或者洛马克思分布样本
poisson	[lam, size]	泊松分布样本
power	a[, size]	[0，1]区间内指数为正（a-1）的幂次分布样本
Rayleigh	[scale, size]	瑞利分布样本
standard_cauchy	[size]	标准柯西分布（模式 0）样本
standard_exponential	[size]	标准指数分布样本
standard_gamma	shape[, size]	标准 γ 分布样本
standard_normal	[size]	标准正态分布(均值为 0，标准差为 1)样本
standard_t	df[, size]	学生的 t 分布样本（自由度为 df）
triangular	left, mode, right[, size]	三角分布样本
uniform	[low, high, size]	均匀分布样本
vonmises	mu, kappa[, size]	冯•米塞斯分布样本
wald	mean, scale[, size]	瓦尔德（逆高斯）分布样本
weibull	a[, size]	威布尔分布样本
zipf	a[, size]	齐夫分布样本

虽然在金融学中使用（标准）正态分布遭到了许多批评，但是它们是不可或缺的工具，在分析和数值应用中仍然是最广泛使用的分布类型。原因之一是许多金融模型直接依赖于正态分布或者对数正态分布。另一个原因是许多不直接依赖（对数）正态假设的金融模型可以离散化，从而通过使用正态分布进行近似模拟。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

作为例子，我们将可视化来自如下分布的随机数：

1. 均值为 0，标准差为 1 的**标准正态分布**
2. 均值为 100，标准差为 20 的**正态分布**
3. 自由度为 0.5 的**卡方分布**
4. λ 值为 1 的**泊松分布**

产生上述分布的代码如下：

```
In [8]: sample_size = 500
        rn1 = npr.standard_normal(sample_size)
        rn2 = npr.normal(100, 20, sample_size)
        rn3 = npr.chisquare(df=0.5, size=sample_size)
        rn4 = npr.poisson(lam=1.0, size=sample_size)
```

图 10-2 展示了 3 个连续分布和 1 个离散分布（泊松分布）的结果。泊松分布用于模拟（罕见）外部事件的发生，例如某种金融工具价格暴涨或者外部冲击。下面是生成图形的代码：

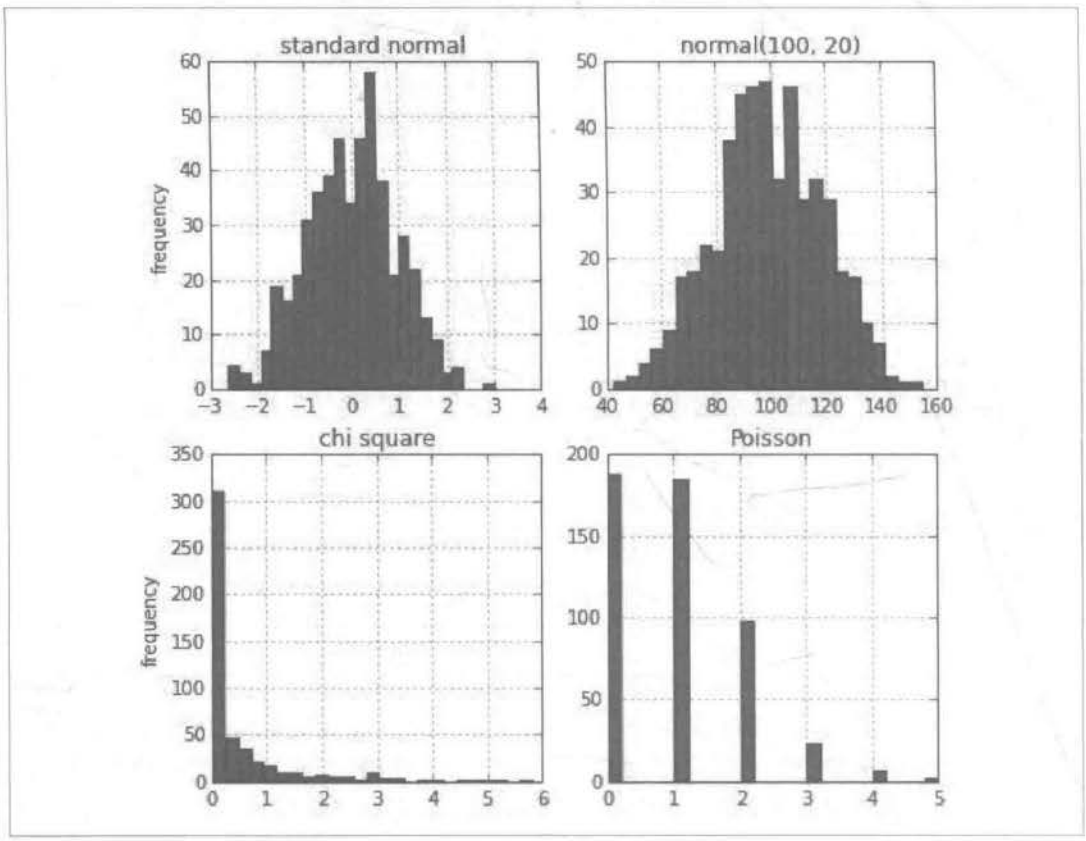


图 10-2　来自不同分布的伪随机数

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [9]: fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(nrows=2, ncols=2,
                                                     figsize=(7, 7))
        ax1.hist(rn1, bins=25)
        ax1.set_title('standard normal')
        ax1.set_ylabel('frequency')
        ax1.grid(True)
        ax2.hist(rn2, bins=25)
        ax2.set_title('normal(100, 20)')
        ax2.grid(True)
        ax3.hist(rn3, bins=25)
        ax3.set_title('chi square')
        ax3.set_ylabel('frequency')
        ax3.grid(True)
        ax4.hist(rn4, bins=25)
        ax4.set_title('Poisson')
        ax4.grid(True)
```

10.2 模拟

蒙特卡洛模拟（MCS）是金融学中最重要的数值技术之一（在重要性和使用广泛程度上也许没有“之一”）。这主要是因为它是最灵活的数学表达式（如积分）求值方法，特别适合于金融衍生品的估值。但是，这种灵活性的代价是相对高的计算负担，估算一个值就可能需要数十万次甚至数百万次复杂计算。

10.2.1 随机变量

举个例子，我们考虑期权定价所用的 Black-Scholes-Merton 设置（参见第 3 章）。这种设置中，在今日股票指数水平 S_0 给定的情况下，未来某个日期 T 的股票指数水平 S_T 根据公式 10-1 得出。

公式 10-1　以 Black-Scholes-Merton 设置模拟未来指数水平

$$S_T = S_0 \exp\left(\left(r - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}z\right) \qquad (10\text{-}1)$$

变量和参数的含义如下：

S_T

T 日的指数水平

r

恒定无风险短期利率

σ

S 的恒定波动率（=收益的标准差）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

z

标准正态分布随机变量

这种简单的金融模型很容参数化和模拟：

```
In [10]: S0 = 100  # initial value
         r = 0.05  # constant short rate
         sigma = 0.25  # constant volatility
         T = 2.0  # in years
         I = 10000  # number of random draws
         ST1 = S0 * np.exp((r - 0.5 * sigma ** 2) * T
                     + sigma * np.sqrt(T) * npr.standard_normal(I))
```

模拟代码的输出如图 10-3 所示：

```
In [11]: plt.hist(ST1, bins=50)
         plt.xlabel('index level')
         plt.ylabel('frequency')
         plt.grid(True)
```

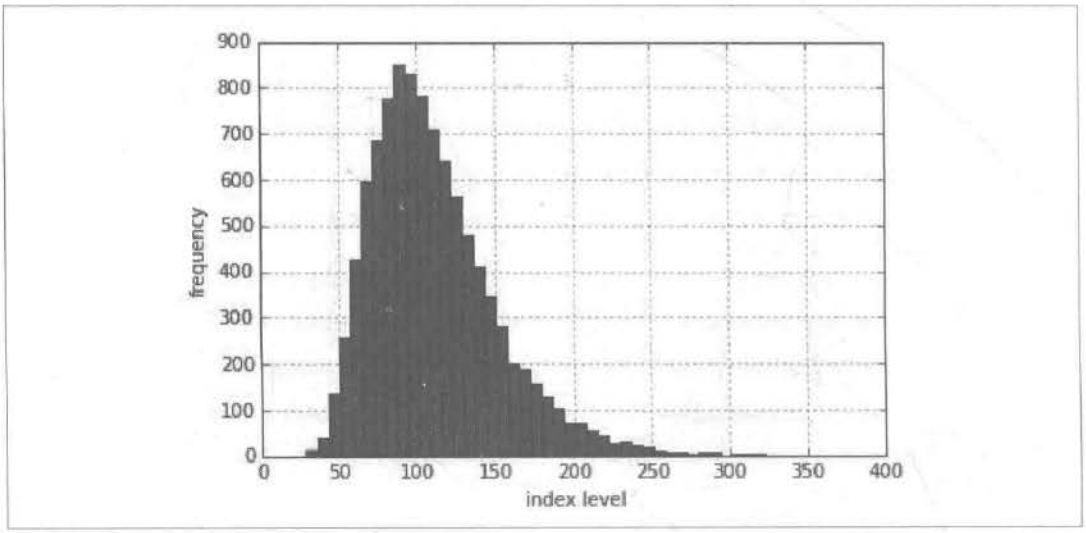


图 10-3 （通过 standard_normal）模拟的几何布朗运动

从图 10-3 中可以看出，公式 10-1 中定义的随机变量呈对数正态分布。因此，我们还可以尝试使用 **lognormal** 函数直接得出随机变量值。在这种情况下，必须向函数提供均值和标准差：

```
In [12]: ST2 = S0 * npr.lognormal((r - 0.5 * sigma ** 2) * T,
                                  sigma * np.sqrt(T), size=I)
```

Figure 10-4 shows the output of the following simulation code:

```
In [13]: plt.hist(ST2, bins=50)
         plt.xlabel('index level')
         plt.ylabel('frequency')
         plt.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

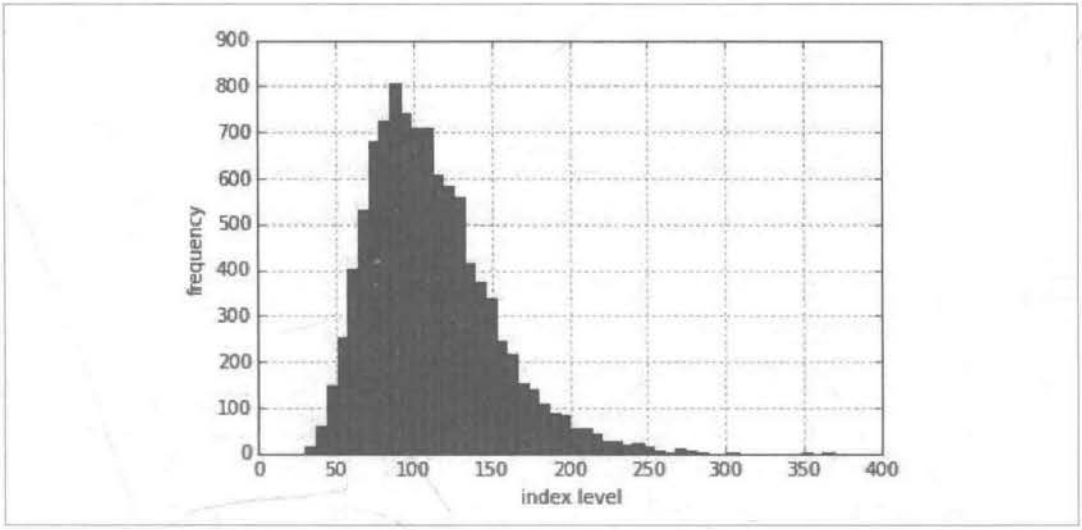


图 10-4 （通过 lognormal）模拟几何布朗运动

通过视觉检验，图 10-4 和图 10-3 确实看起来很相似。但是我们还将通过比较结果分布的统计矩，更严格地检验。

我们使用 **scipy.stats** 子库和下面定义的助手函数 **print_statistics** 比较模拟结果的分布特性：

```
In [14]: import scipy.stats as scs

In [15]: def print_statistics(a1, a2):
             ''' Prints selected statistics.

             Parameters
             ==========
             a1, a2 : ndarray objects
                 results object from simulation
             '''
             sta1 = scs.describe(a1)
             sta2 = scs.describe(a2)
             print "%14s %14s %14s" % \
                 ('statistic', 'data set 1', 'data set 2')
             print 45 * "-"
             print "%14s %14.3f %14.3f" % ('size', sta1[0], sta2[0])
             print "%14s %14.3f %14.3f" % ('min', sta1[1][0], sta2[1][0])
             print "%14s %14.3f %14.3f" % ('max', sta1[1][1], sta2[1][1])
             print "%14s %14.3f %14.3f" % ('mean', sta1[2], sta2[2])
             print "%14s %14.3f %14.3f" % ('std', np.sqrt(sta1[3]),
                                                  np.sqrt(sta2[3]))
             print "%14s %14.3f %14.3f" % ('skew', sta1[4], sta2[4])
             print "%14s %14.3f %14.3f" % ('kurtosis', sta1[5], sta2[5])

In [16]: print_statistics(ST1, ST2)

Out[16]:      statistic     data set 1     data set 2
         ---------------------------------------------------------------
                  size      10000.000      10000.000
                   min         27.936         27.266
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
              max      410.795      358.997
             mean      110.442      110.528
              std       39.932       40.894
             skew        1.082        1.150
         kurtosis        1.927        2.273
```

显然，两个模拟结果的特性很类似，差异主要是由于模拟中的所谓*采样误差*。在离散地模拟连续随机过程时会引入*离散化误差*，但是由于模拟方法的特性，这种误差在此不起任何作用。

10.2.2 随机过程

粗略地讲，随机过程是一个随机变量序列。在这个意义上，我们应该预期，在模拟一个过程时，对一个随机变量的一序列重复模拟应该有某种类似之处。这个结论大体上是正确的，但是随机数的选取一般不是独立的，而是依赖于前几次选取的结果。不过，金融学中使用的随机过程通常表现出*马尔科夫特性*——主要的含义是：明天的过程值只依赖于今天的过程状态，而不依赖其他任何“历史”状态，甚至不依赖整个路径历史。这种过程也被称做“*无记忆过程*”。

几何布朗运动

现在我们考虑 Black-Scholes-Merton 模型的动态形式，这种形式由公式 10-2 中的随机微分方程（SDE）描述。式中的 Z_t 是标准布朗运动，SDE 被称作几何布朗运动。S_t 的值呈对数正态分布，（边际）收益 dS_t/S_t 呈正态分布。

公式 10-2　Black-Scholes-Merton 设置中的随机微分方程

$$dS_t = rS_t dt + \sigma S_t dZ_t \qquad (10\text{-}2)$$

公式 10-2 中的 SDE 可以由一个欧拉格式精确地离散化，公式 10-3 中介绍了一个这样的格式，其中 Δt 是固定的离散化间隔，z_t 是标准正态分布随机变量。

公式 10-3　在 Black-Scholes-Merton 设置中动态模拟指数水平

$$S_t = S_{t-\Delta t} \exp\left(\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t} z_t\right) \qquad (10\text{-}3)$$

和以前一样，转换为 **Python** 和 **NumPy** 代码很简单：

```
In [17]: I = 10000
         M = 50
         dt = T / M
         S = np.zeros((M + 1, I))
         S[0] = S0
         for t in range(1, M + 1):
             S[t] = S[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt
                    + sigma * np.sqrt(dt) * npr.standard_normal(I))
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

得出的指数水平终值仍然呈对数正态分布，如图 10-5 所示：

```
In [18]: plt.hist(S[-1], bins=50)
         plt.xlabel('index level')
         plt.ylabel('frequency')
         plt.grid(True)
```

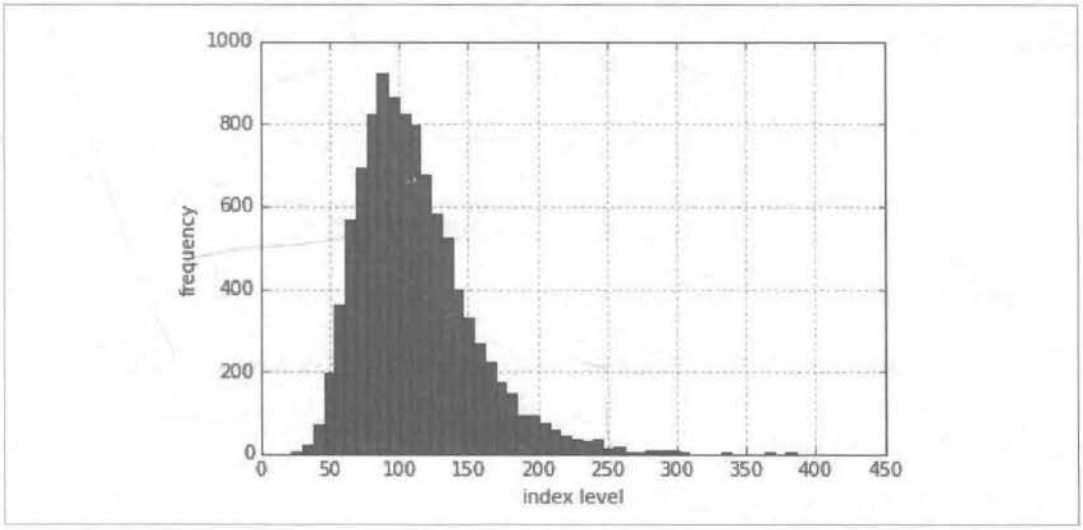


图 10-5　到期日的模拟几何布朗运动

前四个统计矩和静态模拟方法得出的结果相当接近：

```
In [19]: print_statistics(S[-1], ST2)
Out[19]:      statistic     data set 1     data set 2
         ---------------------------------------------
                  size      10000.000      10000.000
                   min         25.531         27.266
                   max        425.051        358.997
                  mean        110.900        110.528
                   std         40.135         40.894
                  skew          1.086          1.150
              kurtosis          2.224          2.273
```

图 10-6 展示了前 10 条模拟路径：

```
In [20]: plt.plot(S[:, :10], lw=1.5)
         plt.xlabel('time')
         plt.ylabel('index level')
         plt.grid(True)
```

使用动态模拟方法不仅可以像图 10-6 那样可视化路径，还可以估算美式/百慕大期权或者收益与路径相关的期权价值。可以这么说，你所得到的是全动态图像。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

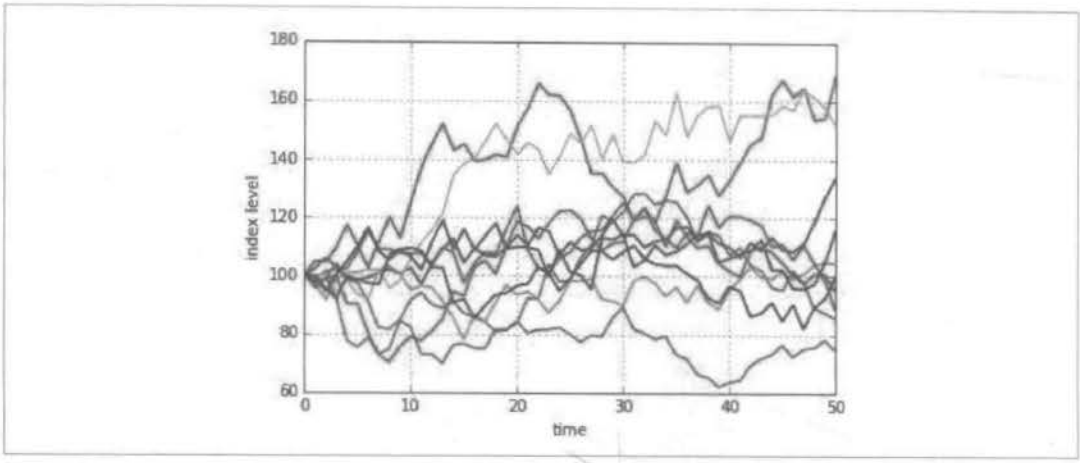


图 10-6　模拟几何布朗运动路径

平方根扩散

另一类重要的金融过程是*均值回归过程*，用于建立短期利率或者波动性过程的模型。流行和广泛使用的模型之一是平方根扩散，由 Cox、Ingersoll 和 Ross（1985）提出。公式 10-4 提供了对应的 SDE。

公式 10-4　平方根扩散的随机微分方程

$$dx_t = k(\theta - x_t)dt + \sigma\sqrt{x_t}dZ_t \qquad (10\text{-}4)$$

式中的变量和参数含义如下：

x_t

日期 t 的过程水平

k

均值回归因子

θ

长期过程均值

σ

恒定波动率参数

Z

标准布朗运动

众所周知，x_t 的值呈卡方分布。但是，如前所述，许多金融模型可以使用正态分布进行离散化和近似计算（即所谓的欧拉离散化格式）。虽然欧拉格式对几何布朗运动很准确，

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

但是对于大部分其他随机过程则会产生偏差。即使有精确的格式——很快就会介绍一个用于平方根扩散的格式——因为数值化和/或计算的原因，使用欧拉格式可能最合适。定义 $s \equiv t - \Delta t$ 和 $x^+ \equiv \max(x,0)$，公式 10-5 提出了一种欧拉格式。这种特殊格式在文献中通常称作*完全截断*（参见 Hilpisch（2015））。

公式 10-5　平方根扩散的欧拉离散化

$$\begin{aligned} \tilde{x}_t &= \tilde{x}_s + k(\theta - \tilde{x}_s^+)\Delta t + \sigma\sqrt{\tilde{x}_s^+}\sqrt{\Delta t}z_t \\ x_t &= \tilde{x}_t^+ \end{aligned} \qquad (10\text{-}5)$$

我们用可以表示短期利率模型的值参数化后续模拟所用的模型：

```
In [21]: x0 = 0.05
         kappa = 3.0
         theta = 0.02
         sigma = 0.1
```

平方根扩散有方便和实际的特性——x_t 的值严格为正。用欧拉格式离散化时，负值无法排除。这就是人们处理的总是原始模拟过程正数版本的原因。因此在模拟代码中，需要两个 **ndarray** 对象，而不是一个：

```
In [22]: I = 10000
         M = 50
         dt = T / M
         def srd_euler():
             xh = np.zeros((M + 1, I))
             x1 = np.zeros_like(xh)
             xh[0] = x0
             x1[0] = x0
             for t in range(1, M + 1):
                 xh[t] = (xh[t - 1]
                       + kappa * (theta - np.maximum(xh[t - 1], 0)) * dt
                       + sigma * np.sqrt(np.maximum(xh[t - 1], 0)) * np.sqrt(dt)
                       * npr.standard_normal(I))
             x1 = np.maximum(xh, 0)
             return x1
         x1 = srd_euler()
```

图 10-7 以直方图的形式展示了模拟结果：

```
In [23]: plt.hist(x1[-1], bins=50)
         plt.xlabel('value')
         plt.ylabel('frequency')
         plt.grid(True)
```

图 10-8 展示了前 10 条模拟路径，说明得出的平均偏离值为负值（因为 $x_0 > \theta$）并收敛于 $\theta = 0.02$：

```
In [24]: plt.plot(x1[:, :10], lw=1.5)
         plt.xlabel('time')
         plt.ylabel('index level')
         plt.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

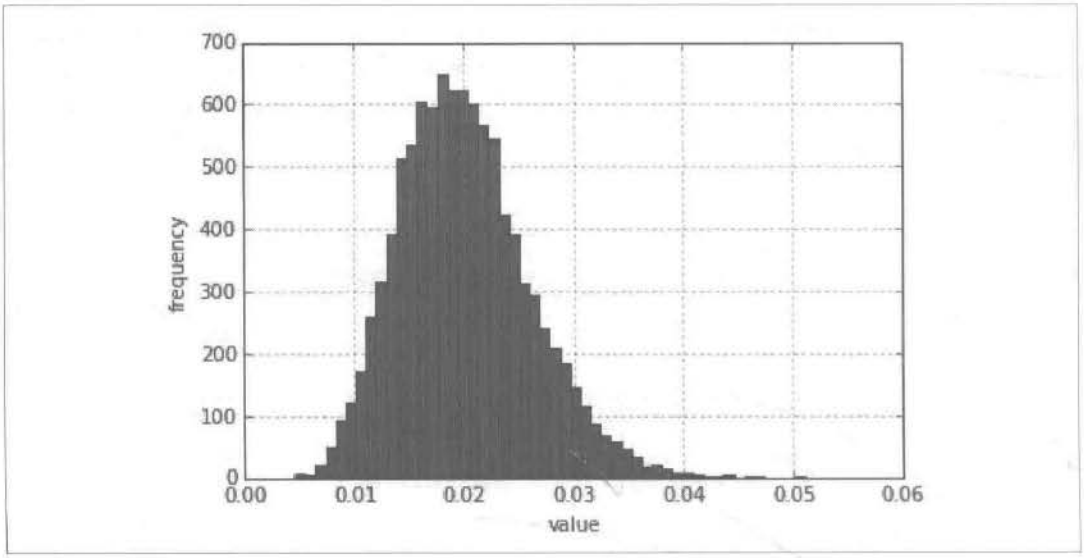


图 10-7　到期日的模拟平方根扩散（欧拉格式）

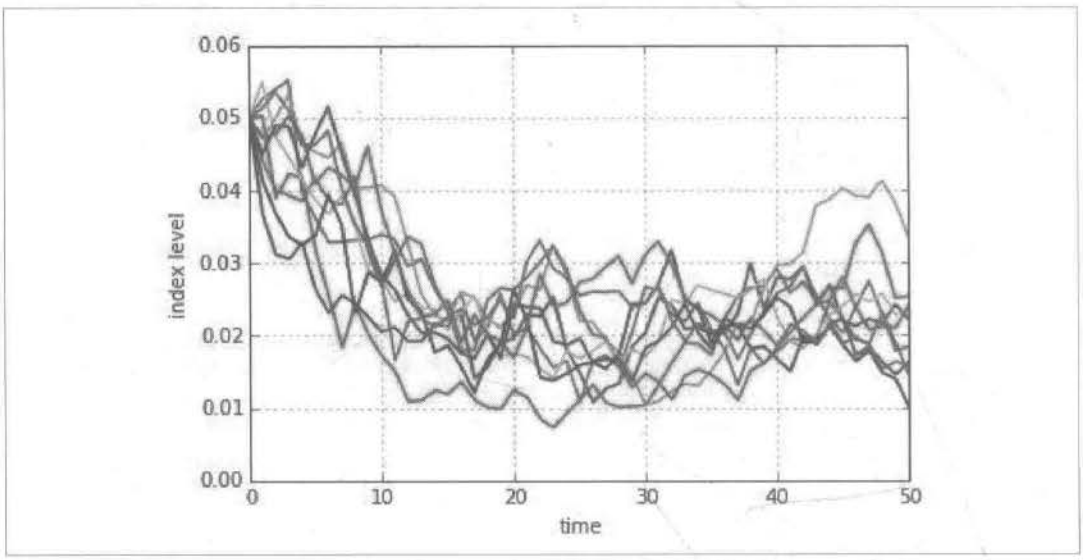


图 10-8　模拟平方根扩散路径（欧拉格式）

现在我们寻求更精确的结果。公式 10-6 提出了基于自由度 $df=\frac{4\theta\kappa}{\sigma^2}$、非中心参数 $nc=\frac{4\kappa e^{-\kappa\Delta t}}{\sigma^2(1-e^{-\kappa\Delta t})}x_s$ 的卡方分布 χ'^2_d 平方根扩散的精确离散化格式。

公式 10-6　平方根扩散的精确离散化

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

$$x_t = \frac{\sigma^2(1-e^{-k\Delta t})}{4k} x_d'^2\left(\frac{4ke^{-k\Delta t}}{\sigma^2(1-e^{-k\Delta t})} x_s\right) \tag{10-6}$$

上述离散化格式的 **Python** 实现较为复杂，但是仍然很简洁：

```
In [25]: def srd_exact():
             x2 = np.zeros((M + 1, I))
             x2[0] = x0
             for t in range(1, M + 1):
                 df = 4 * theta * kappa / sigma ** 2
                 c = (sigma ** 2 * (1 - np.exp(-kappa * dt))) / (4 * kappa)
                 nc = np.exp(-kappa * dt) / c * x2[t - 1]
                 x2[t] = c * npr.noncentral_chisquare(df, nc, size=I)
             return x2
         x2 = srd_exact()
```

图 10-9 以直方图形式展示了这个精确格式的模拟输出：

```
In [26]: plt.hist(x2[-1], bins=50)
         plt.xlabel('value')
         plt.ylabel('frequency')
         plt.grid(True)
```

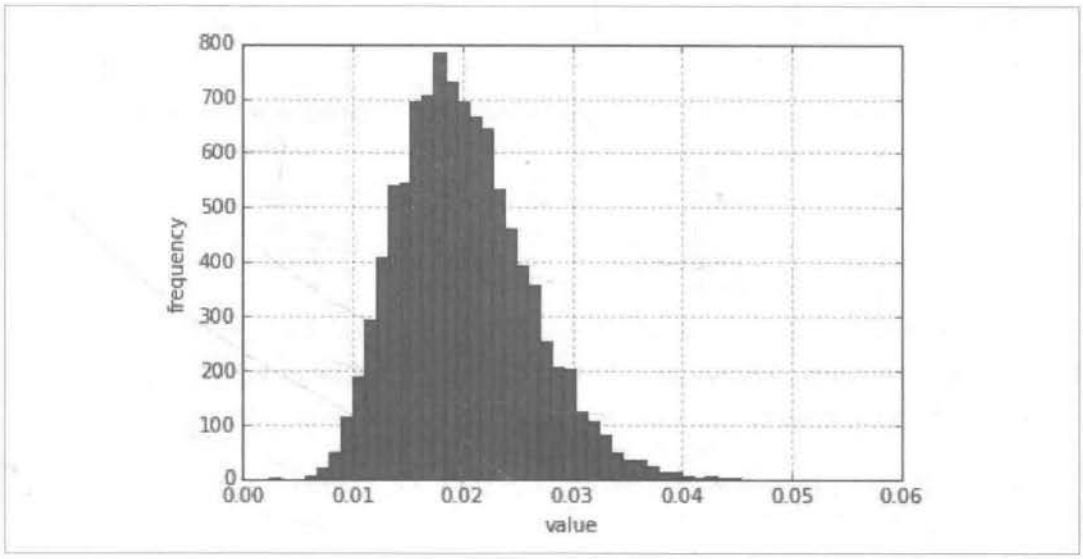


图 10-9　到期日的模拟平方根扩散（精确格式）

图 10-10 和以前一样提供了 10 条模拟路径，同样呈现负的平均漂移、收敛于 θ：

```
In [27]: plt.plot(x2[:, :10], lw=1.5)
         plt.xlabel('time')
         plt.ylabel('index level')
         plt.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

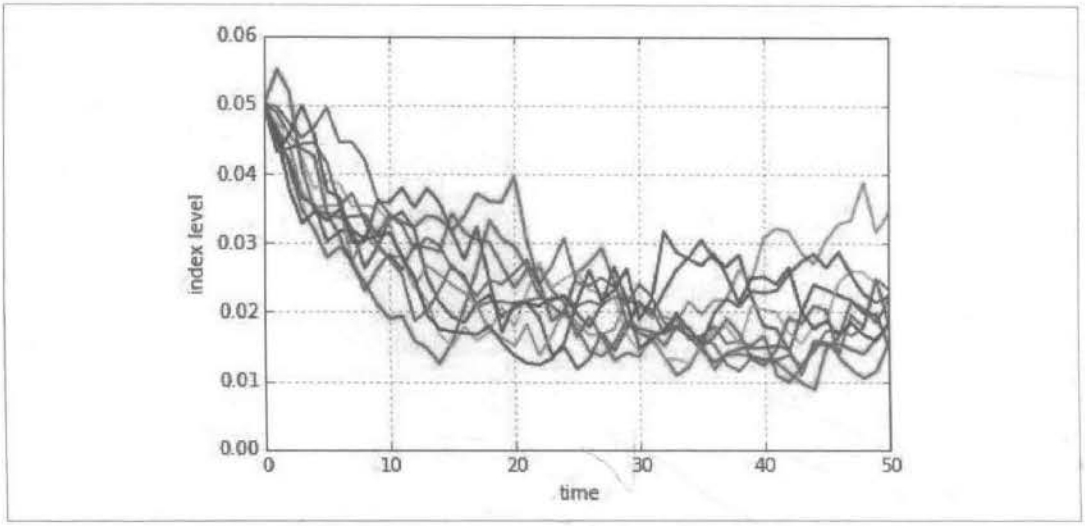


图 10-10　模拟平方根扩散路径（精确格式）

比较不同方法中的主要统计数字可以看出，偏置欧拉格式确实很好地表现出理想的统计属性：

```
In [28]: print_statistics(x1[-1], x2[-1])
Out[28]:      statistic     data set 1     data set 2
         ----------------------------------------------
                    size      10000.000      10000.000
                     min          0.004          0.005
                     max          0.049          0.050
                    mean          0.020          0.020
                     std          0.006          0.006
                    skew          0.529          0.572
                kurtosis          0.418          0.503
```

然而，在执行速度方面可以观察到重大的差异，这是因为从非中心卡方分布中采样的计算要求高于标准正态分布的采样。为了说明这一点，我们考虑更大的模拟路径数量：

```
In [29]: I = 250000
         %time x1 = srd_euler()
Out[29]: CPU times: user 1.02 s, sys: 84 ms, total: 1.11 s
         Wall time: 1.11 s
In [30]: %time x2 = srd_exact()
Out[30]: CPU times: user 2.26 s, sys: 32 ms, total: 2.3 s
         Wall time: 2.3 s
```

精确格式大约需要花费 2 倍的时间，而结果实际上和欧拉格式相同：

```
In [31]: print_statistics(x1[-1], x2[-1])
         x1 = 0.0; x2 = 0.0
Out[31]:      statistic     data set 1     data set 2
         ----------------------------------------------
                    size     250000.000     250000.000
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
             min          0.003          0.004
             max          0.069          0.060
            mean          0.020          0.020
             std          0.006          0.006
            skew          0.554          0.578
        kurtosis          0.488          0.502
```

随机波动率

Black-Scholes-Merton 模型中重要的简化假设之一是*恒定*波动率。但是，波动率一般来说既不是恒定的、也不具有确定性，而是*随机*的。因此，20 世纪 90 年代初金融模型的重大进步之一是所谓*随机波动率模型*的推出。这一类别中最为流行的模型之一是 Heston（1993）模型，如公式 10-7 所示。

公式 10-7　Heston 随机波动率模型的随机微分方程

$$
\begin{aligned}
dS_t &= rS_t dt + \sqrt{v_t} S_t dZ_t^1 \\
dv_t &= k_v(\theta_v - v_t)dt + \sigma_v \sqrt{v_t} dZ_t^2 \\
dZ_t^1 dZ_t^2 &= \rho
\end{aligned}
\tag{10-7}
$$

单一变量和参数的均值现在很容易从几何布朗运动和平方根扩散的讨论中得出，参数 ρ 代表两个标准布朗运动 Z_t^1，Z_t^2 之间的瞬时相关性。这使我们可以解释一个典型事实——*杠杆效应*，该效应本质上指的是波动性在困难时期（衰退市场）中上升，而在牛市（上升市场）时下降。

考虑上述模型的参数化：

```
In [32]: S0 = 100.
         r = 0.05
         v0 = 0.1
         kappa = 3.0
         theta = 0.25
         sigma = 0.1
         rho = 0.6
         T = 1.0
```

为了说明两个随机过程之间的相关性，我们需要确定相关矩阵的柯列斯基分解：

```
In [33]: corr_mat = np.zeros((2, 2))
         corr_mat[0, :] = [1.0, rho]
         corr_mat[1, :] = [rho, 1.0]
         cho_mat = np.linalg.cholesky(corr_mat)

In [34]: cho_mat

Out[34]: array([[ 1. ,  0. ],
                [ 0.6,  0.8]])
```

在开始模拟随机过程之前，我们为两个过程生成整组随机数，指数过程使用第 0 组，

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

波动性过程使用第 1 组：

```
In [35]: M = 50
         I = 10000
         ran_num = npr.standard_normal((2, M + 1, I))
```

对于以平方根扩散过程类型建模的波动性过程，我们使用欧拉格式，考虑相关性参数：

```
In [36]: dt = T / M
         v = np.zeros_like(ran_num[0])
         vh = np.zeros_like(v)
         v[0] = v0
         vh[0] = v0
         for t in range(1, M + 1):
             ran = np.dot(cho_mat, ran_num[:, t, :])
             vh[t] = (vh[t - 1] + kappa * (theta - np.maximum(vh[t - 1], 0)) * dt
                   + sigma * np.sqrt(np.maximum(vh[t - 1], 0)) * np.sqrt(dt)
                   * ran[1])
         v = np.maximum(vh, 0)
```

对于指数水平过程，我们也考虑相关性，使用几何布朗运动的精确欧拉格式：

```
In [37]: S = np.zeros_like(ran_num[0])
         S[0] = S0
         for t in range(1, M + 1):
             ran = np.dot(cho_mat, ran_num[:, t, :])
             S[t] = S[t - 1] * np.exp((r - 0.5 * v[t]) * dt +
                              np.sqrt(v[t]) * ran[0] * np.sqrt(dt))
```

这说明了平方根扩散中使用欧拉格式的另一项优势：相关性很容易一致性地处理，因为我们只提取标准正态随机数。没有一种混合方法（对指数使用欧拉格式，对波动性过程使用基于非中心卡方分布的精确方法）能够实现相同的效果。

图 10-11 用直方图展示了指数水平过程和波动性过程的模拟结果。

```
In [38]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 5))
         ax1.hist(S[-1], bins=50)
         ax1.set_xlabel('index level')
         ax1.set_ylabel('frequency')
         ax1.grid(True)
         ax2.hist(v[-1], bins=50)
         ax2.set_xlabel('volatility')
         ax2.grid(True)
```

对每个过程的前 10 条模拟路径（见图 10-12）的检查表明，波动性过程的平均漂移值为正数，和预期的一样收敛于 θ_v=0.25：

```
In [39]: fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(7, 6))
         ax1.plot(S[:, :10], lw=1.5)
         ax1.set_ylabel('index level')
         ax1.grid(True)
         ax2.plot(v[:, :10], lw=1.5)
         ax2.set_xlabel('time')
         ax2.set_ylabel('volatility')
         ax2.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

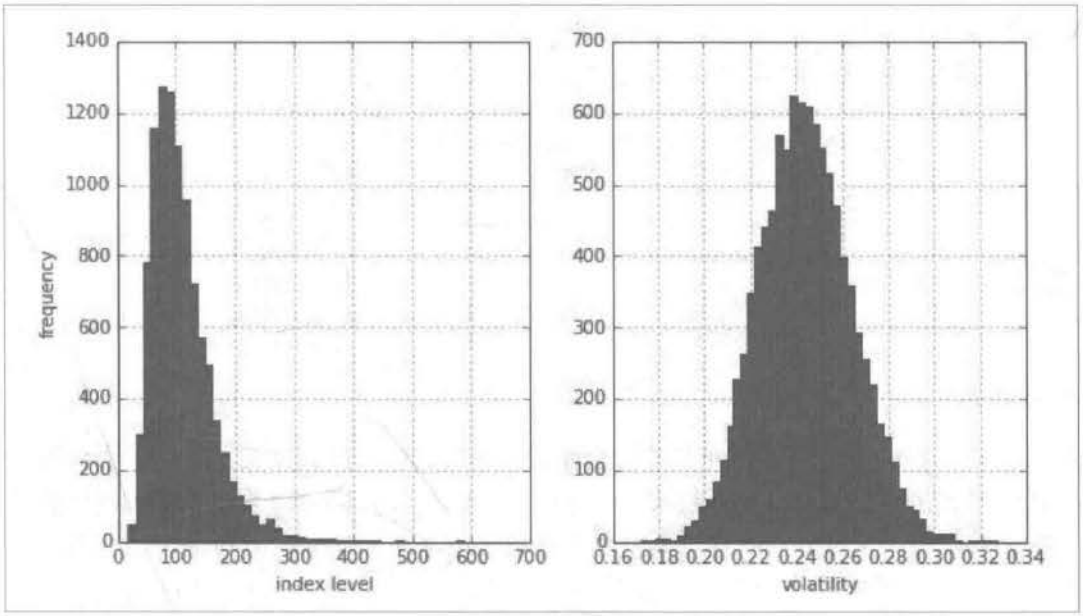


图 10-11　到期日的模拟随机波动率模型

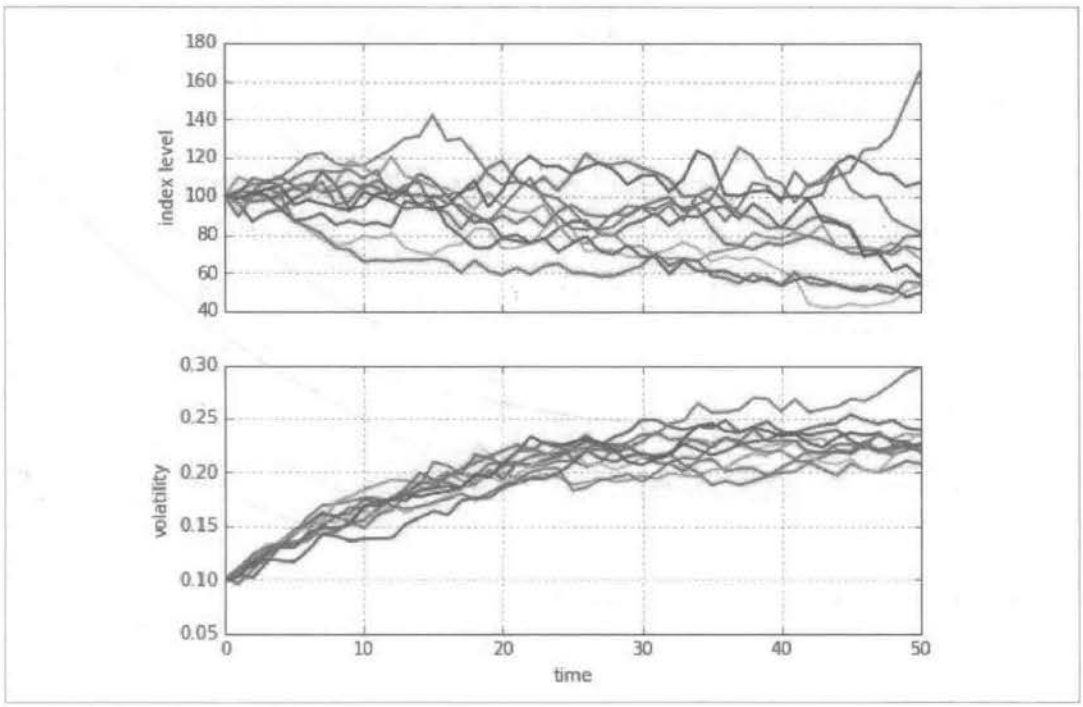


图 10-12　模拟随机波动率模型路径

最后，我们简短地看看两个数据集最后一个时间点的统计数字，该时点显示指数水平过程有一个相当高的最大值。实际上，在其他条件不变的情况下，这个最大值远大于

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

固定波动率下几何布朗运动所能达到的最大值：

```
In [40]: print_statistics(S[-1], v[-1])
Out[40]:      statistic     data set 1     data set 2
         ---------------------------------------------
                  size      10000.000      10000.000
                   min         19.814          0.174
                   max        600.080          0.322
                  mean        108.818          0.243
                   std         52.535          0.020
                  skew          1.702          0.151
              kurtosis          5.407          0.071
```

跳跃扩散

随机波动率和杠杆效应是在许多市场上都能发现的典型（经验主义）事实。另一种典型的经验主义事实是资产价格和波动率的*跳跃*。1976 年，Merton 发布了他的跳跃扩散模型，模型的一个部分以对数正态分布生成跳跃，改进了 Black-Scholes-Merton 设置。风险中立 SDE 如公式 10-8 所示。

公式 10-8　Merton 跳跃扩散模型的随机微分方程

$$dS_t = (r - r_J)S_t dt + \sigma S_t dZ_t + J_t S_t dN_t \qquad (10\text{-}8)$$

为完整性期间，下面列出变量和参数的含义：

S_t

t 日的指数水平

r

恒定无风险短期利率

$$r_J \equiv \lambda \cdot (e^{\mu_J + \delta^2/2} - 1)$$

维持风险中立性的跳跃漂移校正

σ

S 的恒定波动率

Z_t

标准布朗运动

J_t

t 日呈……分布的跳跃

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- $\cdots \log(1+J_t) \approx \mathrm{N}\left(\log(1+\mu_J)-\frac{\delta^2}{2},\delta^2\right)$
- …N 是标准正态随机变量的累积分布函数

N_t

密度为 λ 的泊松分布

公式 10-9 介绍一种用于跳跃扩散的欧拉离散化公式，其中 z_t'' 呈标准正态分布，yt 呈密度为 λ 的泊松分布。

公式 10-9　Merton 跳跃扩散模型的欧拉离散化

$$S_t = S_{t-\Delta t}(e^{(r-r_J-\sigma^2/2)\Delta t+\sigma\sqrt{\Delta t}z_t^1}+(e^{u_J+\delta z_t^2}-1)yt) \qquad (10\text{-}9)$$

有了上述的离散化格式，考虑下面的数值参数：

```
In [41]: S0 = 100.
         r = 0.05
         sigma = 0.2
         lamb = 0.75
         mu = -0.6
         delta = 0.25
         T = 1.0
```

为了模拟跳跃扩散，我们需要生成 3 组（独立）随机数：

```
In [42]: M = 50
         I = 10000
         dt = T / M
         rj = lamb * (np.exp(mu + 0.5 * delta ** 2) - 1)
         S = np.zeros((M + 1, I))
         S[0] = S0
         sn1 = npr.standard_normal((M + 1, I))
         sn2 = npr.standard_normal((M + 1, I))
         poi = npr.poisson(lamb * dt, (M + 1, I))
         for t in range(1, M + 1, 1):
             S[t] = S[t - 1] * (np.exp((r - rj - 0.5 * sigma ** 2) * dt
                                + sigma * np.sqrt(dt) * sn1[t])
                                + (np.exp(mu + delta * sn2[t]) - 1)
                                * poi[t])
             S[t] = np.maximum(S[t], 0)
```

由于我们已经假定跳跃的均值为很大的负数，最终的模拟指数水平（见图 10-13）与典型正态分布相比更向右倾斜也就不足为奇了：

```
In [43]: plt.hist(S[-1], bins=50)
         plt.xlabel('value')
         plt.ylabel('frequency')
         plt.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

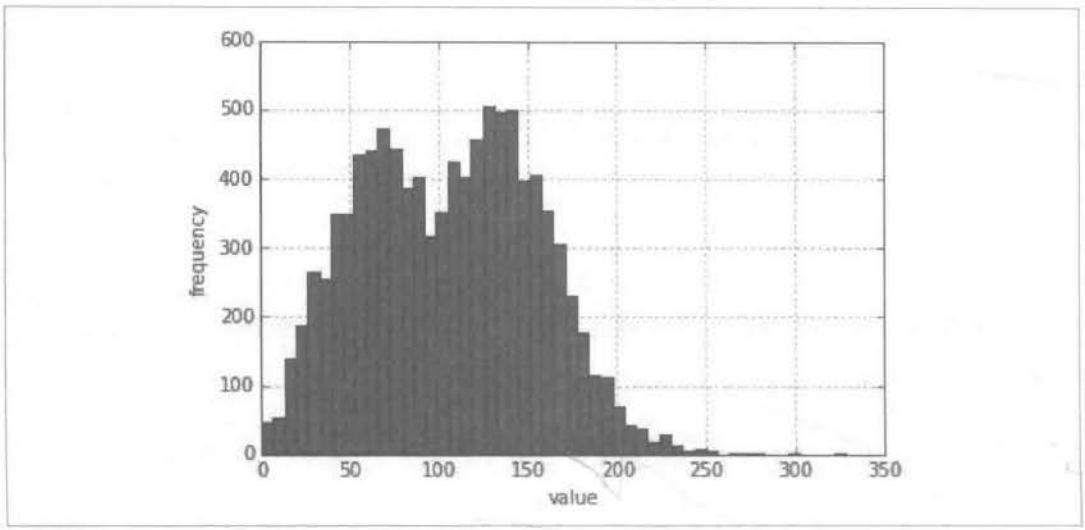


图 10-13　到期日的模拟跳跃扩散

在前 10 条模拟指数水平路径中也可以看到负数的跳跃均值，如图 10-14 所示：

```
In [44]: plt.plot(S[:, :10], lw=1.5)
         plt.xlabel('time')
         plt.ylabel('index level')
         plt.grid(True)
```

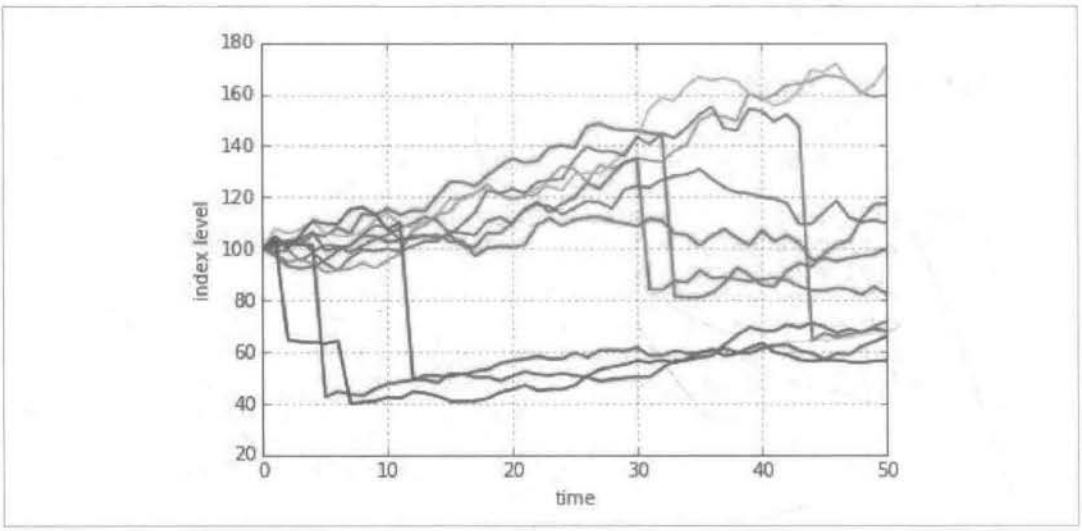


图 10-14　模拟跳跃扩散路径

10.2.3　方差缩减

目前为止，我们得到的结果表现出来的统计数值和预期/理想值并不足够接近，这不仅是因为使用的 **Python** 函数生成的是伪随机数，还因为提取的样本大小各不相同。例如，

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

你可能预期一组标准正态分布随机数的均值为0，标准差为1。我们来看看不同组随机数的统计数字。为了实现逼真的比较，我们修改随机数生成器的种子值：

```
In [45]: print "%15s %15s" % ('Mean', 'Std. Deviation')
         print 31 * "-"
         for i in range(1, 31, 2):
             npr.seed(1000)
             sn = npr.standard_normal(i ** 2 * 10000)
             print "%15.12f %15.12f" % (sn.mean(), sn.std())
Out[45]:            Mean  Std. Deviation
         ---------------------------------------------------
         -0.011870394558  1.008752430725
         -0.002815667298  1.002729536352
         -0.003847776704  1.000594044165
         -0.003058113374  1.001086345326
         -0.001685126538  1.001630849589
         -0.001175212007  1.001347684642
         -0.000803969036  1.000159081432
         -0.000601970954  0.999506522127
         -0.000147787693  0.999571756099
         -0.000313035581  0.999646153704
         -0.000178447061  0.999677277878
          0.000096501709  0.999684346792
         -0.000135677013  0.999823841902
         -0.000015726986  0.999906493379
         -0.000039368519  1.000063091949
In [46]: i ** 2 * 10000
Out[46]: 8410000
```

结果显示，提取随机数的个数越多，统计数值就“莫名其妙”地变得越好。但即使在最大的样本（超过800万个随机数）中，统计数值也不等于理想的数字。

幸运的是，可以使用易于实现的通用方差缩减方法，改善（标准）正态分布前两个统计矩的匹配。第一种技术是使用*对偶变量*。这种方法只提取理想数量一半的随机数，并在之后加入同一组随机数的相反数[1]。例如，如果随机数生成器（即对应的 **Python** 函数）提取0.5，则在数据集中加入另一个值-0.5。

用 **NumPy** 的函数 **concatenate** 可以简洁地实现上述方法：

```
In [47]: sn = npr.standard_normal(10000 / 2)
         sn = np.concatenate((sn, -sn))
         np.shape(sn)
Out[47]: (10000,)
```

下面用对偶变量重复之前的练习：

1 这里描述的方法只适用于对称中位数为0的随机变量，比如标准正态分布随机变量，我们在本书自始至终几乎都使用这种变量。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [48]: print "%15s %15s" % ('Mean', 'Std. Deviation')
         print 31 * "-"
         for i in range(1, 31, 2):
             npr.seed(1000)
             sn = npr.standard_normal(i ** 2 * 10000 / 2)
             sn = np.concatenate((sn, -sn))
             print "%15.12f %15.12f" % (sn.mean(), sn.std())
Out[48]:            Mean  Std. Deviation
         -------------------------------------------------
          0.000000000000  1.009653753942
         -0.000000000000  1.000413716783
          0.000000000000  1.002925061201
         -0.000000000000  1.000755212673
          0.000000000000  1.001636910076
         -0.000000000000  1.000726758438
         -0.000000000000  1.001621265149
          0.000000000000  1.001203722778
         -0.000000000000  1.000556669784
          0.000000000000  1.000113464185
         -0.000000000000  0.999435175324
          0.000000000000  0.999356961431
         -0.000000000000  0.999641436845
         -0.000000000000  0.999642768905
         -0.000000000000  0.999638303451
```

你立刻就会注意到，这种方法完美地更正了第一个统计矩——这并不令人惊讶，原因在于每当提取数 n 时，就加入-n。因为我们有了这样的配对，整组随机数的均值就会等于 0。然而，这种方法对第二个统计矩——标准差没有任何影响。

使用另一种方差缩减技术——*矩匹配*，有助于在一个步骤中更正第一个和第二个统计矩：

```
In [49]: sn = npr.standard_normal(10000)
In [50]: sn.mean()
Out[50]: -0.001165998295162494
In [51]: sn.std()
Out[51]: 0.99125592020460496
```

从每个随机数中减去均值并将每个随机数除以标准差，就可以得到一组匹配随机数，（几乎）完美地匹配理想的标准正态分布第一和第二统计矩：

```
In [52]: sn_new = (sn - sn.mean()) / sn.std()
In [53]: sn_new.mean()
Out[53]: -2.3803181647963357e-17
In [54]: sn_new.std()
Out[54]: 0.99999999999999989
```

下面的函数利用对方差缩减技术的认识，用两种、一种或者不用方差缩减技术生成用于过程模拟的标准正态随机数：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [55]: def gen_sn(M, I, anti_paths=True, mo_match=True):
             ''' Function to generate random numbers for simulation.

             Parameters
             ==========
             M : int
                 number of time intervals for discretization
             I : int
                 number of paths to be simulated
             anti_paths: Boolean
                 use of antithetic variates
             mo_math : Boolean
                 use of moment matching
             '''
             if anti_paths is True:
                 sn = npr.standard_normal((M + 1, I / 2))
                 sn = np.concatenate((sn, -sn), axis=1)
             else:
                 sn = npr.standard_normal((M + 1, I))
             if mo_match is True:
                 sn = (sn - sn.mean()) / sn.std()
             return sn
```

10.3 估值

蒙特卡洛模拟的最重要应用之一是*未定权益*（期权，衍生品，混合型工具等）*的估值*。简单地说，在风险中立的世界中，未定权益的价值是风险中立（鞅）测度下的折现后预期收益。这是所有风险因素（股票、指数等）偏离无风险短期利率的概率测度。根据资产定价基本定理，这种概率测度的存在等价于套利机会的缺失。

金融期权表示在规定（行权期）日期（*欧式期权*）或者规定时期（*美式期权*）内，以规定价格（所谓*行权价*）购买（*看涨期权*）或者出售（*看跌期权*）指定金融工具。我们首先考虑估值较为简单的情况——欧式期权。

10.3.1 欧式期权

基于某种指数的欧式看涨期权到期日收益通过公式 $h(S_T)\equiv\max(S_T-K,0)$得出，其中 S_T是到期日 T 的指数水平，K 是行权价格。给定相关随机过程（例如几何布朗运动）的风险中立测度，或者在一个完备市场中，这种权证的价格由公式 10-10 表示。

公式 10-10　风险中立预期定价

$$C_0 = e^{-rT}\mathrm{E}_0^Q(h(S_T)) = e^{-rT}\int_0^\infty h(s)q(s)\mathrm{d}s \qquad (10\text{-}10)$$

第 9 章概略介绍了如何通过蒙特卡洛模拟计算积分。下面使用这种方法并应用到公式 10-10 中。公式 10-11 提供了欧式期权的对应蒙特卡洛模拟公式，其中 $\tilde{S}_T^i$ 是到期日的第

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

i 个模拟指数水平。

公式 10-11　风险中立蒙特卡洛模拟公式

$$\widetilde{C_0} = e^{-rT} \frac{1}{I} \sum_{i=1}^{I} h(\tilde{S}_T^i) \tag{10-11}$$

现在考虑几何布朗运动的参数化和估值函数 *gbm_mcs_stat*，该函数仅以行权价格作为参数。这里只模拟到期日的指数水平：

```
In [56]: S0 = 100.
         r = 0.05
         sigma = 0.25
         T = 1.0
         I = 50000
         def gbm_mcs_stat(K):
             ''' Valuation of European call option in Black-Scholes-Merton
             by Monte Carlo simulation (of index level at maturity)

             Parameters
             ==========
             K : float
                 (positive) strike price of the option

             Returns
             =======
             C0 : float
                 estimated present value of European call option
             '''
             sn = gen_sn(1, I)
             # simulate index level at maturity
             ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T
                              + sigma * np.sqrt(T) * sn[1])
             # calculate payoff at maturity
             hT = np.maximum(ST - K, 0)
             # calculate MCS estimator
             C0 = np.exp(-r * T) * 1 / I * np.sum(hT)
             return C0
```

作为参考，考虑行权价 K=105 的情况：

```
In [57]: gbm_mcs_stat(K=105.)
Out[57]: 10.044221852841922
```

接下来，我们考虑动态模拟方法，除了看涨期权之外还可以模拟欧式看跌期权。函数 gbm_mcs_dyna 实现了这一算法：

```
In [58]: M = 50
         def gbm_mcs_dyna(K, option='call'):
             ''' Valuation of European options in Black-Scholes-Merton
             by Monte Carlo simulation (of index level paths)

             Parameters
             ==========
             K : float
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        (positive) strike price of the option
    option : string
        type of the option to be valued ('call', 'put')

    Returns
    =======
    C0 : float
        estimated present value of European call option
    '''
    dt = T / M
    # simulation of index level paths
    S = np.zeros((M + 1, I))
    S[0] = S0
    sn = gen_sn(M, I)
    for t in range(1, M + 1):
        S[t] = S[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt
                + sigma * np.sqrt(dt) * sn[t])
    # case-based calculation of payoff
    if option == 'call':
        hT = np.maximum(S[-1] - K, 0)
    else:
        hT = np.maximum(K - S[-1], 0)
    # calculation of MCS estimator
    C0 = np.exp(-r * T) * 1 / I * np.sum(hT)
    return C0
```

现在，可以比较相同行权价的看涨和看跌期权的价格估算：

```
In [59]: gbm_mcs_dyna(K=110., option='call')
Out[59]: 7.9500085250284336
In [60]: gbm_mcs_dyna(K=110., option='put')
Out[60]: 12.629934942682004
```

问题是，这些基于模拟的估值方法与 Black-Scholes-Merton 估值公式得出的基准值相比表现如何？为了找出这种差别，我们用 BSM_Functions.py 模块中的 Black-Scholes-Merton 分析性欧式看涨期权定价公式生成一定范围行权价的对应期权价值/估算：

```
In [61]: from bsm_functions import bsm_call_value
         stat_res = []
         dyna_res = []
         anal_res = []
         k_list = np.arange(80., 120.1, 5.)
         np.random.seed(200000)
         for K in k_list:
             stat_res.append(gbm_mcs_stat(K))
             dyna_res.append(gbm_mcs_dyna(K))
             anal_res.append(bsm_call_value(S0, K, T, r, sigma))
         stat_res = np.array(stat_res)
         dyna_res = np.array(dyna_res)
         anal_res = np.array(anal_res)
```

首先，我们将静态模拟方法的结果与精确的分析值相比：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [62]: fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(8, 6))
         ax1.plot(k_list, anal_res, 'b', label='analytical')
         ax1.plot(k_list, stat_res, 'ro', label='static')
         ax1.set_ylabel('European call option value')
         ax1.grid(True)
         ax1.legend(loc=0)
         ax1.set_ylim(ymin=0)
         wi = 1.0
         ax2.bar(k_list - wi / 2, (anal_res - stat_res) / anal_res * 100, wi)
         ax2.set_xlabel('strike')
         ax2.set_ylabel('difference in %')
         ax2.set_xlim(left=75, right=125)
         ax2.grid(True)
```

图 10-15 展示了比较的结果。所有估值的差异都小于 1%，正负的差异均有。

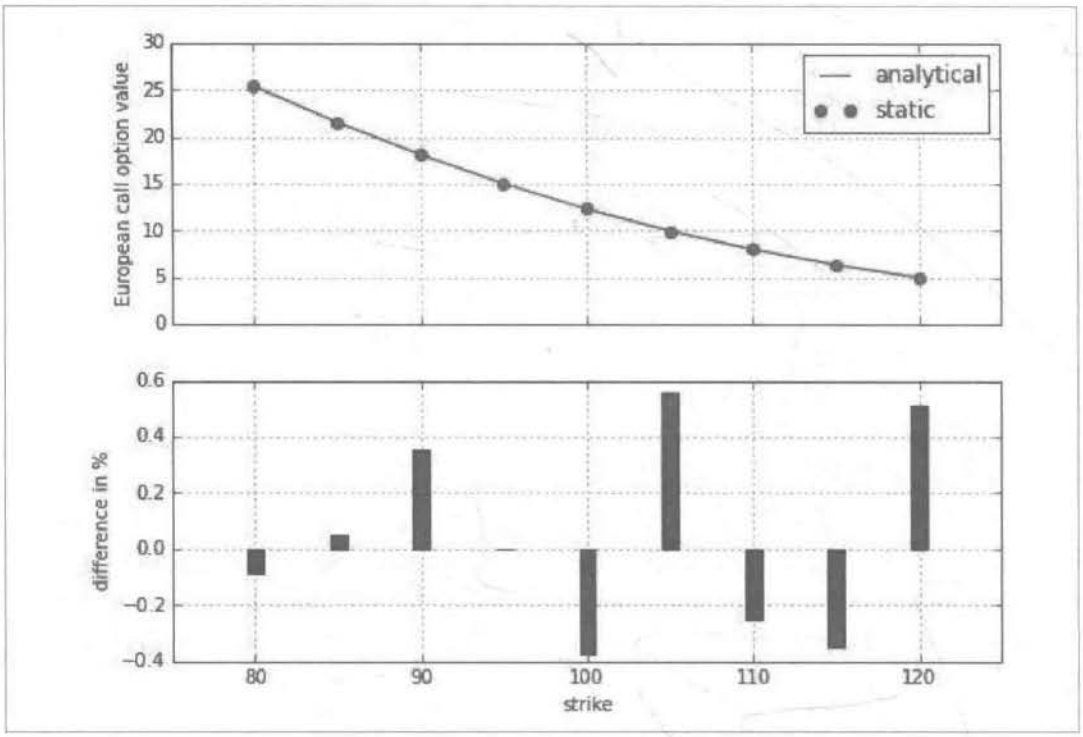


图 10-15　静态和动态蒙特卡洛估算值的对比

合并动态模拟和估值方法的类似图表，可以得到图 10-16 的结果。同样，所有估值差异小于 1%，标准差既有负数也有正数的情况。作为一般原则，蒙特卡洛估算函数的质量可以通过调整使用的时间间隔 M 和模拟路径数 I 控制：

```
In [63]: fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(8, 6))
         ax1.plot(k_list, anal_res, 'b', label='analytical')
         ax1.plot(k_list, dyna_res, 'ro', label='dynamic')
         ax1.set_ylabel('European call option value')
         ax1.grid(True)
         ax1.legend(loc=0)
         ax1.set_ylim(ymin=0)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
wi = 1.0
ax2.bar(k_list - wi / 2, (anal_res - dyna_res) / anal_res * 100, wi)
ax2.set_xlabel('strike')
ax2.set_ylabel('difference in %')
ax2.set_xlim(left=75, right=125)
ax2.grid(True)
```

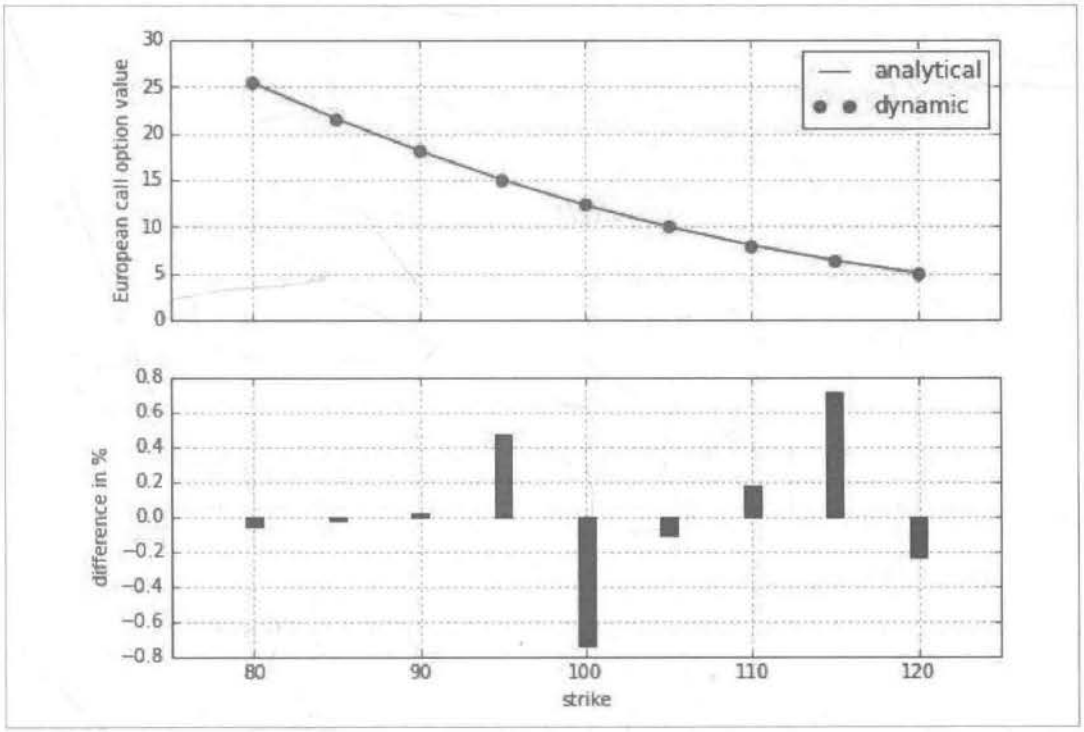


图 10-16　静态和动态蒙特卡洛估算值对比

10.3.2　美式期权

美式期权的估值比欧式期权更复杂。在这种情况下，必须解决*最优截止问题*，提出期权的公允价值。公式 10-12 是将美式期权作为最优截止问题时的估值公式。该问题的公式化已经基于离散的时间网格，以便用于数值化模拟。在某种意义上，更准确地说，这是百慕大式期权的估值公式。时间间隔收敛于 0 长度时，百慕大期权的价值收敛于美式期权的价值。

公式 10-12　以最优截止问题形式出现的美式期权价格

$$V_0 = \sup_{\tau \in \{0, \Delta t, 2\Delta t, \cdots, T\}} e^{-rT} E_0^Q(h_T(S_T)) \qquad (10\text{-}12)$$

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们下面描述的算法称为*最小二乘蒙特卡洛*（LSM）*方法*，来源于 Longstaff 和 Schwartz（2001）的论文。可以看出，由 $V_t(s)=\max(h_t(s),C_t(s))$（其中 $Ct(s)=\mathrm{E}_t^Q(e^{-r\Delta t}V_{t+\Delta t}(S_{t+\Delta t})\mid S_t=s)$ 给出的任何给定日期 t 的美式（百慕大）期权价值是给定指数水平 $S_t=s$ 下的期权*持续价值*。

现在考虑我们在 M 个等长（Δt）的时间间隔中模拟指数水平的 I 条路径。定义 $Y_{t,i}\equiv e^{-r\Delta t}V_{t+\Delta t,i}$ 为路径 i 在时间 t 时的模拟持续价值。我们不能直接使用这个数字，因为它意味着完美的预期。但是，我们可以使用所有模拟持续价值的截面，通过最小二乘回归估算（预期）持续价值。

给定一组基函数 b_d，d=1，…，D，然后由回归估算公式 $\hat{C}_{t,i}=\sum_{d=1}^{D}\alpha_{d,t}^{*}\cdot b_d(S_{t,i})$ 算出持续价值，其中最优回归参数 α^{*} 是公式 10-13 中最小二乘问题的解。

公式 10-13　美式期权估值的最小二乘回归

$$\min_{\alpha_{1,t},\cdots,\alpha_{D,t}}\frac{1}{L}\sum_{i=1}^{I}\left(Y_{t,i}-\sum_{d=1}^{D}\alpha_{d,t}\cdot b_d(S_{t,i})\right)^2 \qquad (10\text{-}13)$$

gbm_mcs_amer 函数实现美式看涨和看跌期权的 LSM 算法[1]：

```
In [64]: def gbm_mcs_amer(K, option='call'):
             ''' Valuation of American option in Black-Scholes-Merton
             by Monte Carlo simulation by LSM algorithm

             Parameters
             ==========
             K : float
                 (positive) strike price of the option
             option : string
                 type of the option to be valued ('call', 'put')

             Returns
             =======
             C0 : float
                 estimated present value of European call option
             '''
             dt = T / M
             df = np.exp(-r * dt)
             # simulation of index levels
             S = np.zeros((M + 1, I))
             S[0] = S0
             sn = gen_sn(M, I)
             for t in range(1, M + 1):
```

1　算法的细节详见 Hilpisch(2015)。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            S[t] = S[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt
                    + sigma * np.sqrt(dt) * sn[t])
        # case-based calculation of payoff
        if option == 'call':
            h = np.maximum(S - K, 0)
        else:
            h = np.maximum(K - S, 0)
        # LSM algorithm
        V = np.copy(h)
        for t in range(M - 1, 0, -1):
            reg = np.polyfit(S[t], V[t + 1] * df, 7)
            C = np.polyval(reg, S[t])
            V[t] = np.where(C > h[t], V[t + 1] * df, h[t])
        # MCS estimator
        C0 = df * 1 / I * np.sum(V[1])
        return C0
In [65]: gbm_mcs_amer(110., option='call')
Out[65]: 7.7789332794493156
In [66]: gbm_mcs_amer(110., option='put')
Out[66]: 13.614023206242445
```

欧式期权的价值处于美式期权价值的下界。两者的差异通常称作*提前行权溢价*。下面，我们比较和以前相同的行权价范围内的欧式和美式期权价值，以估算期权溢价。这次我们选择看跌期权：

```
In [67]: euro_res = []
         amer_res = []
         k_list = np.arange(80., 120.1, 5.)
         for K in k_list:
             euro_res.append(gbm_mcs_dyna(K, 'put'))
             amer_res.append(gbm_mcs_amer(K, 'put'))
         euro_res = np.array(euro_res)
         amer_res = np.array(amer_res)
```

图 10-17 说明对于所选择的行权价范围，溢价可能最高达到 10%：

```
In [68]: fig, (ax1, ax2) = plt.subplots(2, 1, sharex=True, figsize=(8, 6))
         ax1.plot(k_list, euro_res, 'b', label='European put')
         ax1.plot(k_list, amer_res, 'ro', label='American put')
         ax1.set_ylabel('call option value')
         ax1.grid(True)
         ax1.legend(loc=0)
         wi = 1.0
         ax2.bar(k_list - wi / 2, (amer_res - euro_res) / euro_res * 100, wi)
         ax2.set_xlabel('strike')
         ax2.set_ylabel('early exercise premium in %')
         ax2.set_xlim(left=75, right=125)
         ax2.grid(True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

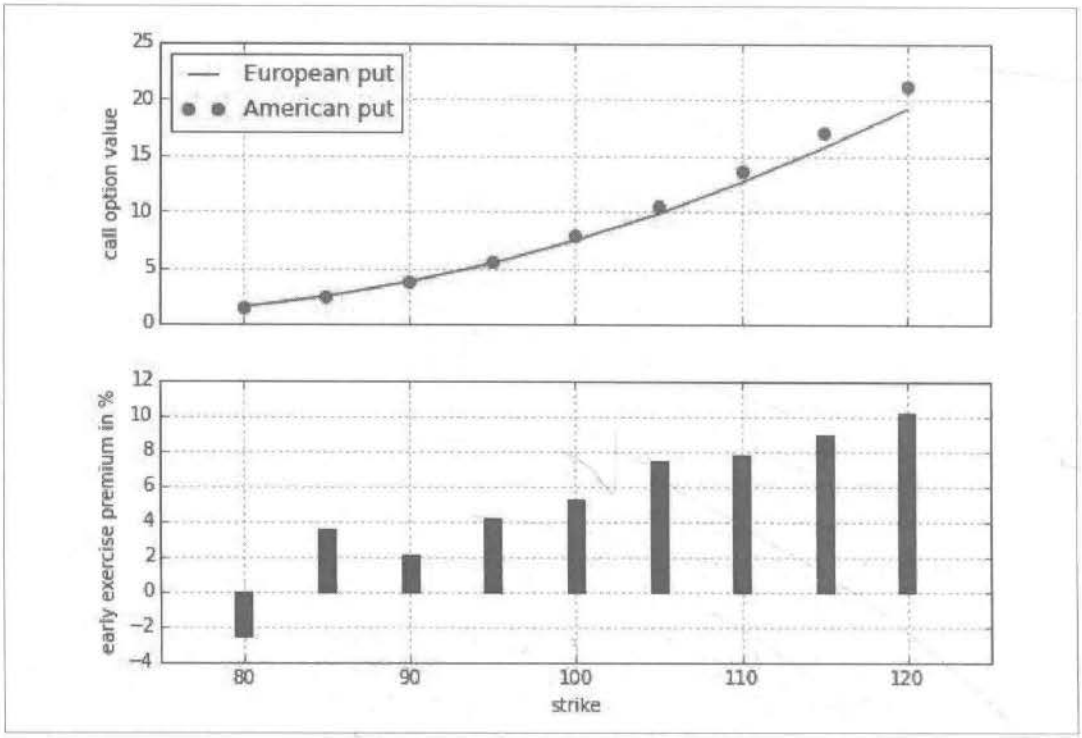


图 10-17　欧式期权和 LSM 蒙特卡洛估算值的对比

10.4　风险测度

除了估值之外，*风险管理*是随机方法和模拟的另一个重要应用领域。本节介绍当今金融行业最常用的两个风险测度的计算/估算。

10.4.1　风险价值

*风险价值（VaR）*是最广泛使用的风险测度之一，也是饱受争议的测度之一。从业人员喜欢其直观性，但是许多人对其有限的*尾部风险*（很快将会详加介绍）捕捉能力进行广泛的讨论和批评——主要是在理论依据上。从字面上讲，VaR 是一个以货币单位（如美元、欧元、日元）表示的数字，表示在给定时间周期中不超过某种置信度（概率）的损失（或者一个投资组合、股票头寸等）。

考虑一个当日价值为 100 万美元的股票头寸，在 30 天（1 个月）内、置信度为 99%的情况下 VaR 为 5 万美元。这个 VaR 数字说明，30 天内损失不超过 5 万美元的概率为 99%（100 个案例中有 99 个）。但是，它并不说明一旦损失超过 5 万美元，损失的规模会达到什么程度——也就是如果最大损失为 10 万或者 50 万美元时，这种特定的“高于 VaR 的损失”概率有多大。它所说明的只是，发生 5 万美元或者更大损失的概率为 1%。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们再次假定使用 Black-Scholes-Merton 设置，考虑如下的参数化和未来日期 T=30/365（即假定 30 天的一段时期）指数水平的模拟：

```
In [69]: S0 = 100
         r = 0.05
         sigma = 0.25
         T = 30 / 365.
         I = 10000
         ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T
                          + sigma * np.sqrt(T) * npr.standard_normal(I))
```

为了估算 VaR 数字，需要模拟的绝对利润和相对于近日持仓价值的亏损，并加以排序，即从最严重的亏损到最大的利润：

```
In [70]: R_gbm = np.sort(ST - S0)
```

图 10-18 展示了模拟绝对绩效的直方图：

```
In [71]: plt.hist(R_gbm, bins=50)
         plt.xlabel('absolute return')
         plt.ylabel('frequency')
         plt.grid(True)
```

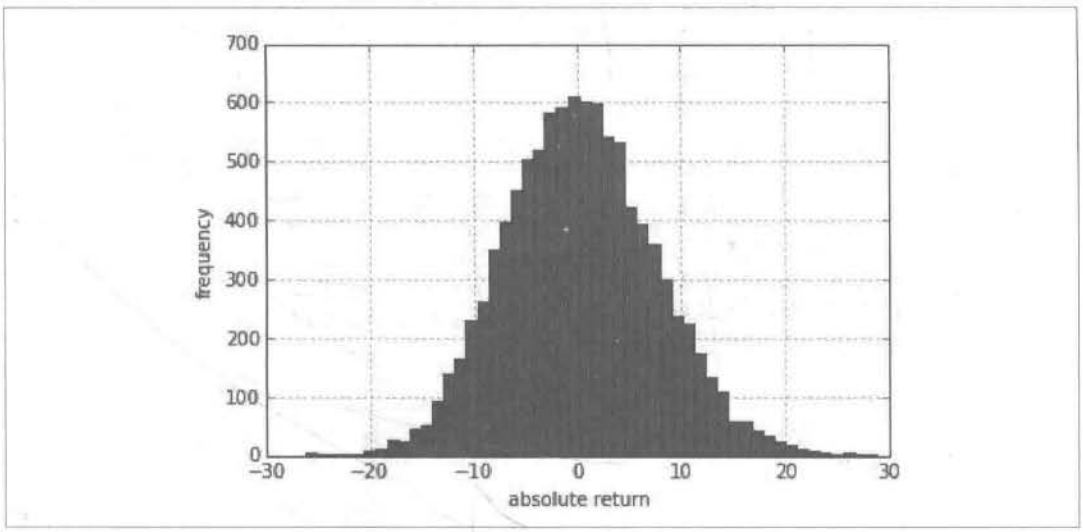


图 10-18　几何布朗运动的绝对收益（30 日）

有了包含排序结果的 **ndarray** 对象，**scoreatpercentile** 函数已经取得了成功。我们所需要做的就是定义感兴趣的百分比（以百分数表示），在列表对象 **percs** 中，0.1 转换为置信度 100%-0.1%=99.9%。在本例中，置信度为 99.9%的 30 日 VaR 为 20.2 货币单位，而 89%置信度下为 8.9 个货币单位：

```
In [72]: percs = [0.01, 0.1, 1., 2.5, 5.0, 10.0]
         var = scs.scoreatpercentile(R_gbm, percs)
         print "%16s %16s" % ('Confidence Level', 'Value-at-Risk')
         print 33 * "-"
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        for pair in zip(percs, var):
            print "%16.2f %16.3f" % (100 - pair[0], -pair[1])
Out[72]: Confidence Level    Value-at-Risk
        ------------------------------------------------------------
                   99.99           26.072
                   99.90           20.175
                   99.00           15.753
                   97.50           13.265
                   95.00           11.298
                   90.00            8.942
```

作为第 2 个例子，回忆一下 Merton 的跳跃扩散，我们打算动态模拟：

```
In [73]: dt = 30. / 365 / M
         rj = lamb * (np.exp(mu + 0.5 * delta ** 2) - 1)
         S = np.zeros((M + 1, I))
         S[0] = S0
         sn1 = npr.standard_normal((M + 1, I))
         sn2 = npr.standard_normal((M + 1, I))
         poi = npr.poisson(lamb * dt, (M + 1, I))
         for t in range(1, M + 1, 1):
             S[t] = S[t - 1] * (np.exp((r - rj - 0.5 * sigma ** 2) * dt
                                + sigma * np.sqrt(dt) * sn1[t])
                                + (np.exp(mu + delta * sn2[t]) - 1)
                                * poi[t])
             S[t] = np.maximum(S[t], 0)
In [74]: R_jd = np.sort(S[-1] - S0)
```

在这个例子中，利用均值为负数的跳跃成分，我们看到了图 10-19 中类似二项分布的模拟利润/亏损。从正态分布的角度看，在左侧有明显的“大尾巴”：

```
In [75]: plt.hist(R_jd, bins=50)
         plt.xlabel('absolute return')
         plt.ylabel('frequency')
         plt.grid(True)
```

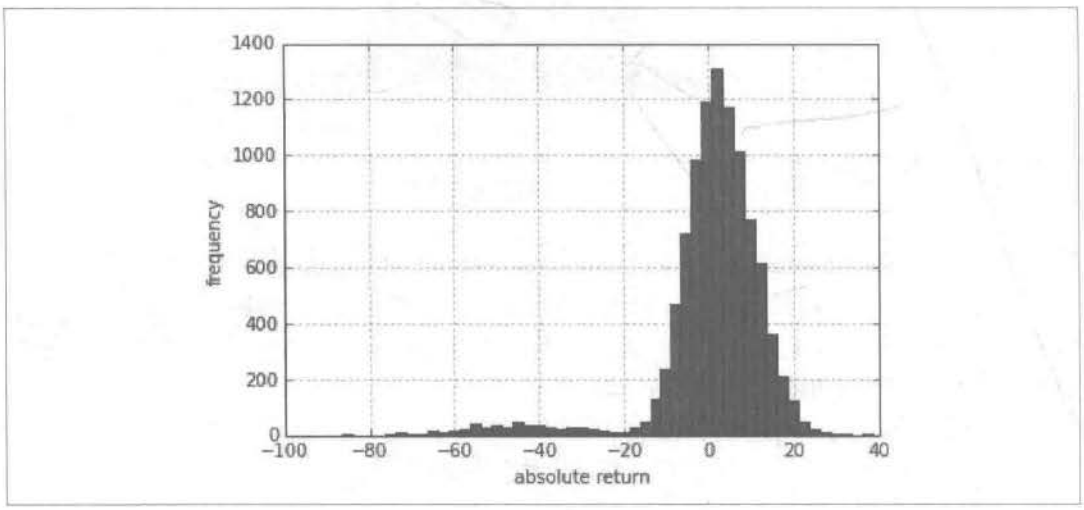


图 10-19　跳跃扩散的绝对收益（30 日）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

对于这种过程和参数化，置信度90%的30日VaR几乎完全相同，但是在99.9%置信度下与几何布朗运动相比高出*3倍多*（71.8对20.2货币单位）：

```
In [76]: percs = [0.01, 0.1, 1., 2.5, 5.0, 10.0]
         var = scs.scoreatpercentile(R_jd, percs)
         print "%16s %16s" % ('Confidence Level', 'Value-at-Risk')
         print 33 * "-"
         for pair in zip(percs, var):
             print "%16.2f %16.3f" % (100 - pair[0], -pair[1])
Out[76]: Confidence Level    Value-at-Risk
         -----------------------------------------------------
                   99.99           75.029
                   99.90           71.833
                   99.00           55.901
                   97.50           45.697
                   95.00           25.993
                   90.00            8.773
```

这说明标准VaR测度在捕捉金融市场经常遇到的尾部风险方面的问题。

为了进一步说明这一点，我们最后以图形方式展示两种情况的VaR测度以便比较。如图10-20所示，在典型置信度范围内的VaR测度表现完全不同：

```
In [77]: percs = list(np.arange(0.0, 10.1, 0.1))
         gbm_var = scs.scoreatpercentile(R_gbm, percs)
         jd_var = scs.scoreatpercentile(R_jd, percs)
In [78]: plt.plot(percs, gbm_var, 'b', lw=1.5, label='GBM')
         plt.plot(percs, jd_var, 'r', lw=1.5, label='JD')
         plt.legend(loc=4)
         plt.xlabel('100 - confidence level [%]')
         plt.ylabel('value-at-risk')
         plt.grid(True)
         plt.ylim(ymax=0.0)
```

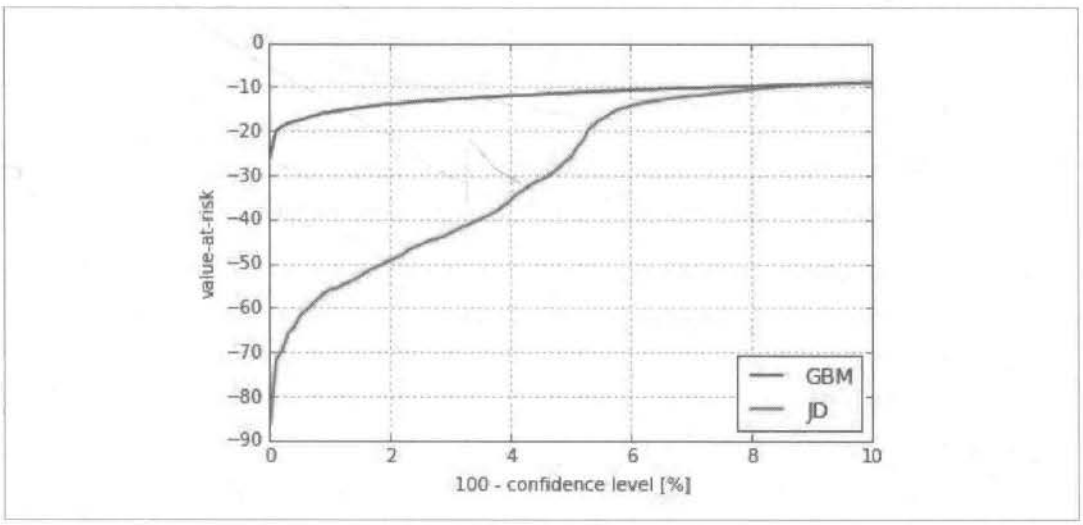


图10-20　几何布朗运动和跳跃扩散的风险价值

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

10.4.2 信用价值调整

其他重要的风险测度是信用风险价值（CVaR）和从 CVaR 中派生而来的信用价值调整（CVA）。粗略地讲，CVaR 是对手方可能无法履行其义务所引发风险（例如，对手方破产）的一个测度。在这种情况下，有两个主要的假设：*违约概率*和（平均）*损失水平*。

举个具体的例子，我们再次考虑 Black-Scholes-Merton 的基准设置，使用如下参数：

```
In [79]: S0 = 100.
         r = 0.05
         sigma = 0.2
         T = 1.
         I = 100000
         ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T
                     + sigma * np.sqrt(T) * npr.standard_normal(I))
```

在最简单的情况下，人们可以考虑固定（平均）损失水平 **L** 和对手方违约（每年）概率 **p**：

```
In [80]: L = 0.5
In [81]: p = 0.01
```

使用泊松分布，违约的方案可以用如下代码生成，考虑了违约只能发生一次的事实：

```
In [82]: D = npr.poisson(p * T, I)
         D = np.where(D > 1, 1, D)
```

如果没有违约，未来指数水平的风险中立价值应该等于资产当日现值（取决于数值误差造成的差异）：

```
In [83]: np.exp(-r * T) * 1 / I * np.sum(ST)
Out[83]: 99.981825216842921
```

在我们假定的条件下，CVaR 可以这样计算：

```
In [84]: CVaR = np.exp(-r * T) * 1 / I * np.sum(L * D * ST)
         CVaR
Out[84]: 0.5152011134161355
```

类似地，经过信用风险调整之后的资产现值可以这样计算：

```
In [85]: S0_CVA = np.exp(-r * T) * 1 / I * np.sum((1 - L * D) * ST)
         S0_CVA
Out[85]: 99.466624103426781
```

这应该（大约）等于 CVaR 价值减去当前资产价值：

```
In [86]: S0_adj = S0 - CVaR
         S0_adj
Out[86]: 99.48479888658386
```

在这个特殊的模拟示例中，我们观察到由于信用风险引起的大约 1000 次亏损，这是假定违约概率为 1%、10 万次模拟下预期的结果：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [87]: np.count_nonzero(L * D * ST)
Out[87]: 1031
```

图 10-21 展示了由于违约引起亏损的完整频率分布。当然，在大部分情况下（10 万例中的 99000 例），没有发现亏损：

```
In [88]: plt.hist(L * D * ST, bins=50)
         plt.xlabel('loss')
         plt.ylabel('frequency')
         plt.grid(True)
         plt.ylim(ymax=175)
```

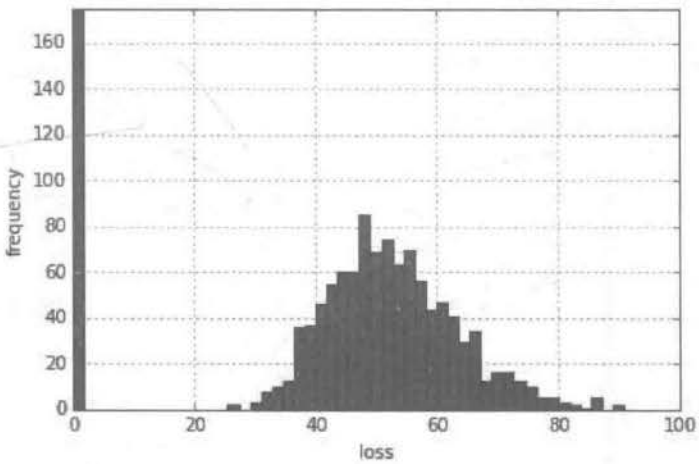


图 10-21　由于风险中立预期违约引起的亏损（股票）

现在考虑欧式看涨期权的情况。它在行权价 100 时的价值大约为 10.4 个货币单位：

```
In [89]: K = 100.
         hT = np.maximum(ST - K, 0)
         C0 = np.exp(-r * T) * 1 / I * np.sum(hT)
         C0
Out[89]: 10.427336109660052
```

在相同的违约概率和损失水平假设下，CVaR 大约为 5 分：

```
In [90]: CVaR = np.exp(-r * T) * 1 / I * np.sum(L * D * hT)
         CVaR
Out[90]: 0.053822578452208093
```

相应地，调整后的期权价值大约低了 5 分：

```
In [91]: C0_CVA = np.exp(-r * T) * 1 / I * np.sum((1 - L * D) * hT)
         C0_CVA
Out[91]: 10.373513531207843
```

和常规资产相比，期权有不同的特性。我们只看到略低于 500 次因违约引起的亏损，但是仍然有大约 1000 次违约。这一结果源于这样的事实：期权到期日时的收益为 0 的

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

概率很大：

```
In [92]: np.count_nonzero(L * D * hT)  # number of losses
Out[92]: 582
In [93]: np.count_nonzero(D)  # number of defaults
Out[93]: 1031
In [94]: I - np.count_nonzero(hT)  # zero payoff
Out[94]: 43995
```

图 10-22 说明，期权的 CVaR 和常规资产相比有着完全不同的频率分布：

```
In [95]: plt.hist(L * D * hT, bins=50)
         plt.xlabel('loss')
         plt.ylabel('frequency')
         plt.grid(True)
         plt.ylim(ymax=350)
```

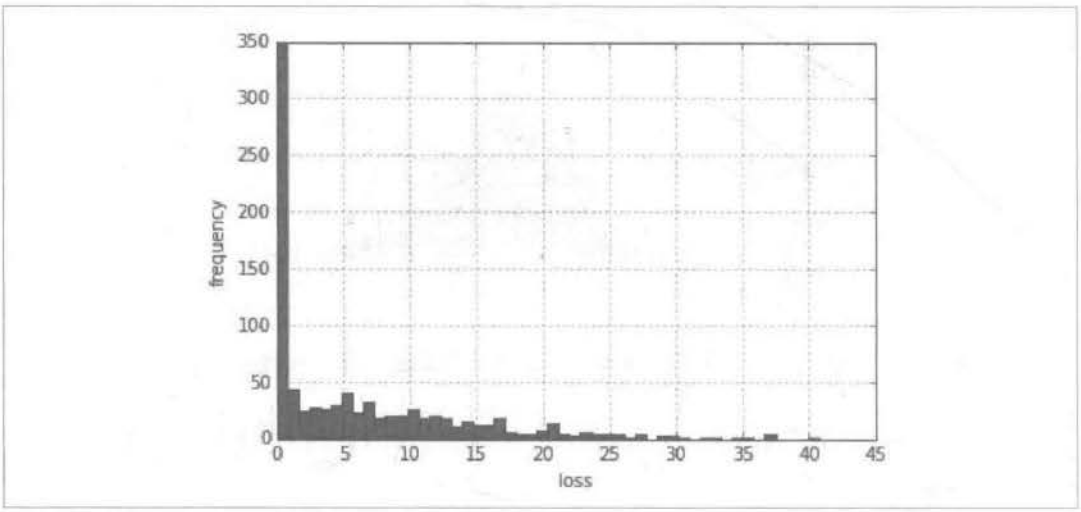


图 10-22　由于风险中立预期违约引起的亏损（看涨期权）

10.5　结语

本章介绍对蒙特卡洛模拟在金融学中的应用起重要作用的方法和技术，特别说明了如何根据不同的分布法则*生成（伪）随机数*。接下来，本章继续介绍了在许多金融领域中很重要的*随机变量及随机过程模拟*。本章深入地讨论了两个应用领域：欧式和美式*期权的估值*以及风险价值和信用价值调整等*风险测度的估算*。

本章说明 **Python** 和 **NumPy** 的组合很适合于实现计算要求很高的任务，如通过蒙特卡洛模拟进行的美式期权估值。这主要是因为 **NumPy** 的大部分函数和类用 **C** 语言实现，使其在一般情况下对纯 **Python** 代码有明显的速度优势。由于向量化操作，**NumPy** 还具有结果代码紧凑和易于理解的好处。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

10.6 延伸阅读

最早在金融学中引入蒙特卡洛模拟的论文是：

- Boyle, Phelim (1977): "Options: A Monte Carlo Approach." *Journal of Financial Economics*, Vol. 4, No. 4, pp. 322–338

本章引用的其他原创论文（参见第 16 章）：

- Black, Fischer and Myron Scholes (1973): "The Pricing of Options and CorporateLiabilities." *Journal of Political Economy*, Vol. 81, No. 3, pp. 638–659.
- Cox, John, Jonathan Ingersoll and Stephen Ross (1985): "A Theory of the TermStructure of Interest Rates." *Econometrica*, Vol. 53, No. 2, pp. 385–407.
- Heston, Steven (1993): "A Closed-From Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options." *The Review of Financial Studies*, Vol. 6, No. 2, 327–343.
- Merton, Robert (1973): "Theory of Rational Option Pricing." *Bell Journal of Economics and Management Science*, Vol. 4, pp. 141–183.
- Merton, Robert (1976): "Option Pricing When the Underlying Stock Returns AreDiscontinuous." *Journal of Financial Economics*, Vol. 3, No. 3, pp. 125–144.

Glassermann（2004）和 Hilpisch（2015）的著作深入介绍了本章的所有主题（但是，第一本书没有介绍任何技术实现的细节）：

- Glasserman, Paul (2004): *Monte Carlo Methods in Financial Engineering*. Springer, New York.
- Hilpisch, Yves (2015): *Derivatives Analytics with Python*. Wiley Finance, Chichester, England. http://www.derivatives-analytics-with-python.com.

直到世纪之交，通过蒙特卡洛模拟估算美式期权价值的有效方法才最终发布：

- Longstaff, Francis and Eduardo Schwartz (2001): "Valuing American Options bySimulation: A Simple Least Squares Approach." *Review of Financial Studies*, Vol.14, No. 1, pp. 113-147.

下面的书籍广泛深入地介绍了信用风险的处置方法：

- Duffie, Darrell and Kenneth Singleton (2003): *Credit Risk—Pricing, Measurement,and Management*. Princeton University Press, Princeton, NJ.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 11 章

统计学

我可以用统计学证明一切，除了真相。

——George Canning

统计学是一个广泛的领域。该领域提供的工具和结果已经成为金融学中不可或缺的一部分。这也就能够解释 R 等领域专用语言在金融行业中的流行。统计模型越精细、越复杂，易于使用和高性能的计算解决方案就越重要。

只用本书一章的篇幅无法充分说明统计领域的丰富和广博。因此，正如其他许多章节一样，我们将重点放在精选的主题上，这些主题极其重要，或者能够为使用 **Python** 完成特定任务提供好的起点。本章有 4 个重点：

正态性检验

许多重要金融模型，如均值-方差投资组合理论和资本性资产定价模型（CAPM），都依赖于证券收益呈正态分布这一假设；因此，本章介绍一些方法，测试给定时间序列的正态性。

投资组合理论

可以将现代投资组合理论（MPT）看做金融统计学的最大成功之一；从 20 世纪 50 年代初开始，由于先驱者 Harry Markowitz 的努力，这一理论开始用严格的数学和统计学方法，代替人们投资金融市场时对判断和经验的依赖。在这个意义上，它可能是金融学中第一种真正的计量方法。

主成分分析

主成分分析（PCA）是金融学中相当流行的工具，例如，在实施权益投资策略或

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

者分析解释利率变化的主要成分时。它的主要好处是“降低复杂度”，这是通过从一组（可能很大）高相关事件序列成分中提取一小组线性无关（无关联，正交）的成分；我们将根据德国 DAX 指数和指数中包含的 30 种股票阐述其应用。

贝叶斯回归

从基础层面上，贝叶斯统计在统计学中引入了主体*信念度*和*信念度更新*；例如，对于线性回归，可能采取回归参数统计分布的形式代替单点估算（例如，回归线截距和斜率）。如今，贝叶斯方法在金融学中相当流行和重要，这也是我们在本章中阐述一些（高级）应用的原因。

本章的许多方面与日期和时间信息相关。用 **Python**、**NumPy** 和 **pandas** 处理这类数据的概述参见附录 C。

11.1 正态性检验

可以说，正态分布是金融学中最重要的分布，也是金融理论的主要统计学基础之一。尤其是下面这些金融理论基础，在很大程度上依赖于股票市场收益的正态分布。

投资组合理论

当股票收益呈正态分布时，最优化投资组合可以在这样的环境中选择：只有平均收益和收益的方差（或者波动率）以及不同股票之间的协方差与投资决策（即最优化投资组合构成）相关。

资本性资产定价模型

同样，当股票收益呈正态分布时，单独证券的价格可以很好地以和某种大规模市场指数的关系表示；这种关系通常用单一股票与市场指数的联动指标（β）表示。

有效市场假设

有效市场指的是价格反映所有可用信息的市场，其中的“所有”可以是狭义的，也可以是广义的（例如“所有公开信息”或者同时包括“只为个人所有”的信息）；如果这个假设成立，股票价格波动将是随机的，而收益呈正态分布。

期权定价理论

布朗运动是随机股票（和其他证券）价格变动的标准、基准模型；著名的 Black-Scholes-Merton 期权定价公式使用几何布朗运动作为股票在一段时间内随机波动的模型，这种波动造成收益呈正态分布。

上述的理论只是支持金融学中正态性假设重要性的一部分原因。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

11.1.1 基准案例

为了给进一步的分析提供基础，我们从几何布朗运动开始，它是金融建模中使用的规范随机过程之一。关于几何布朗运动中的路径特性，有如下说明：

正态对数收益率

在两个时点之间的对数收益率 $\log\frac{S_t}{S_s} = \log S_t - \log S_s$（$0 < s < t$）。

对数-正态价值

在任何时点 $t > 0$，价值 S_t 呈正态分布。

为了下面的工作，我们需要一些 **Python** 库，包括 **scipy.stats**（http://doc.scipy.org/doc/scipy/reference/stats.html）和 **statsmodels.api**（http://statsmodels.sourceforge.net/stable/）：

```
In [1]: import numpy as np
        np.random.seed(1000)
        import scipy.stats as scs
        import statsmodels.api as sm
        import matplotlib as mpl
        import matplotlib.pyplot as plt
        %matplotlib inline
```

我们定义一个函数，为几何布朗运动生成蒙特卡洛路径（参见第 10 章）：

```
In [2]: def gen_paths(S0, r, sigma, T, M, I):
            ''' Generates Monte Carlo paths for geometric Brownian motion.

            Parameters
            ==========
            S0 : float
                initial stock/index value
            r : float
                constant short rate
            sigma : float
                constant volatility
            T : float
                final time horizon
            M : int
                number of time steps/intervals
            I : int
                number of paths to be simulated

            Returns
            =======
            paths : ndarray, shape (M + 1, I)
                simulated paths given the parameters
            '''
            dt = float(T) / M
            paths = np.zeros((M + 1, I), np.float64)
            paths[0] = S0
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    for t in range(1, M + 1):
        rand = np.random.standard_normal(I)
        rand = (rand - rand.mean()) / rand.std()
        paths[t] = paths[t - 1] * np.exp((r - 0.5 * sigma ** 2) * dt +
                                         sigma * np.sqrt(dt) * rand)
    return paths
```

下面是蒙特卡洛模拟的一种可能的参数化，和 **gen_paths** 函数相结合，生成 25 万条路径，每条有 50 个时间步：

```
In [3]: S0 = 100.
        r = 0.05
        sigma = 0.2
        T = 1.0
        M = 50
        I = 250000
In [4]: paths = gen_paths(S0, r, sigma, T, M, I)
```

图 11-1 展示了前 10 条模拟路径：

```
In [5]: plt.plot(paths[:, :10])
        plt.grid(True)
        plt.xlabel('time steps')
        plt.ylabel('index level')
```

我们主要感兴趣的是对数收益率的分布。下面的代码生成一个包含所有对数收益率的 **ndarray** 对象：

```
In [6]: log_returns = np.log(paths[1:] / paths[0:-1])
```

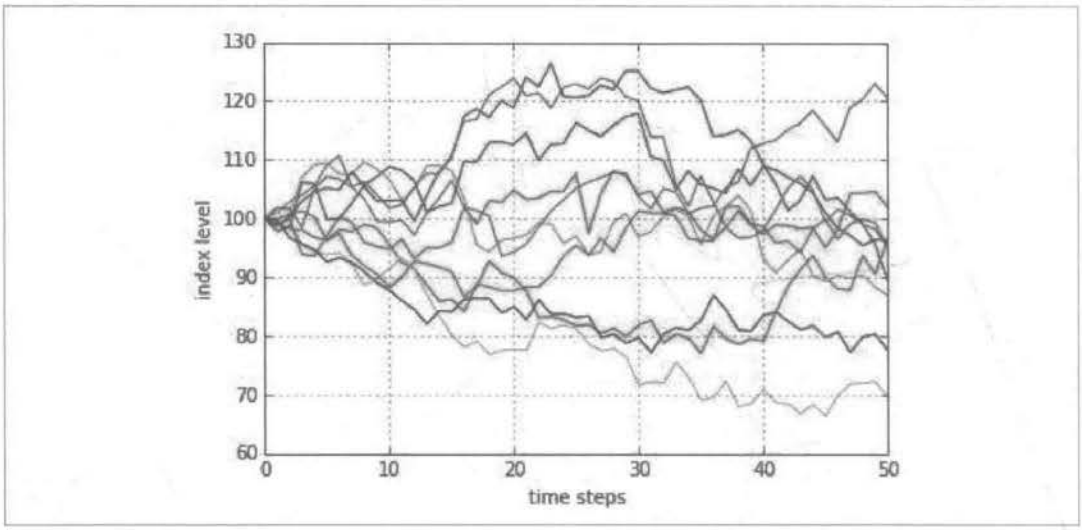


图 11-1　几何布朗运动的 10 条模拟路径

考虑 50 个时间步上的第一条模拟路径：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [7]: paths[:, 0].round(4)
Out[7]: array([ 100.    ,   97.821 ,   98.5573,  106.1546,  105.899 ,   99.8363,
                100.0145,  102.6589,  105.6643,  107.1107,  108.7943,  108.2449,
                106.4105,  101.0575,  102.0197,  102.6052,  109.6419,  109.5725,
                112.9766,  113.0225,  112.5476,  114.5585,  109.942 ,  112.6271,
                112.7502,  116.3453,  115.0443,  113.9586,  115.8831,  117.3705,
                117.9185,  110.5539,  109.9687,  104.9957,  108.0679,  105.7822,
                105.1585,  104.3304,  108.4387,  105.5963,  108.866 ,  108.3284,
                107.0077,  106.0034,  104.3964,  101.0637,   98.3776,   97.135 ,
                 95.4254,   96.4271,   96.3386])
```

模拟路径的对数收益率序列可能采取如下形式：

```
In [8]: log_returns[:, 0].round(4)
Out[8]: array([-0.022 ,  0.0075,  0.0743, -0.0024, -0.059 ,  0.0018,  0.0261,
                0.0289,  0.0136,  0.0156, -0.0051, -0.0171, -0.0516,  0.0095,
                0.0057,  0.0663, -0.0006,  0.0306,  0.0004, -0.0042,  0.0177,
               -0.0411,  0.0241,  0.0011,  0.0314, -0.0112, -0.0095,  0.0167,
                0.0128,  0.0047, -0.0645, -0.0053, -0.0463,  0.0288, -0.0214,
               -0.0059, -0.0079,  0.0386, -0.0266,  0.0305, -0.0049, -0.0123,
               -0.0094, -0.0153, -0.0324, -0.0269, -0.0127, -0.0178,  0.0104,
               -0.0009])
```

这是人们在金融市场上可能经历的：在一些日子里你的投资获得*正收益*，而在其他日子里，相对于最近的财富状况，你*损失了金钱*。

print_statistics 函数是 **scipy.stats** 子库中的 **describe** 函数的包装器函数。它主要以更易于（人类）理解的方式输出给定（历史或者模拟）数据集均值、偏斜度或者峰度等统计数字：

```
In [9]: def print_statistics(array):
            ''' Prints selected statistics.

            Parameters
            ==========
            array: ndarray
                object to generate statistics on
            '''
            sta = scs.describe(array)
            print "%14s %15s" % ('statistic', 'value')
            print 30 * "-"
            print "%14s %15.5f" % ('size', sta[0])
            print "%14s %15.5f" % ('min', sta[1][0])
            print "%14s %15.5f" % ('max', sta[1][1])
            print "%14s %15.5f" % ('mean', sta[2])
            print "%14s %15.5f" % ('std', np.sqrt(sta[3]))
            print "%14s %15.5f" % ('skew', sta[4])
            print "%14s %15.5f" % ('kurtosis', sta[5])
```

例如，下面展示函数的实际运行情况，使用包含对数收益率的 **ndarray** 对象平整版本。**flattern** 方法返回一个 1 维数组，包含多维数组中给出的所有数据：

```
In [10]: print_statistics(log_returns.flatten())
Out[10]:      statistic           value
         ----------------------------------
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
          size  12500000.00000
           min        -0.15664
           max         0.15371
          mean         0.00060
           std         0.02828
          skew         0.00055
      kurtosis         0.00085
```

本例中的数据集包含 1250 万个数据点，其值主要处于-0.15 和 0.15 之间。我们可以预期平均年化收益为 0.05，标准差（波动率）为 0.2。数据集中的年化值不完全等于上述值，但是很接近（均值乘以 50，标准差乘以 $\sqrt{50}$ ）。

图 11-2 比较模拟对数收益率的分布和参数化 r 和 **sigma** 之后的正态分布概率密度函数（pdf）。使用的函数是 **scipy.stats** 子库中的 **norm.pdf**。两者明显很相符：

```
In [11]: plt.hist(log_returns.flatten(), bins=70, normed=True, label='frequency')
         plt.grid(True)
         plt.xlabel('log-return')
         plt.ylabel('frequency')
         x = np.linspace(plt.axis()[0], plt.axis()[1])
         plt.plot(x, scs.norm.pdf(x, loc=r / M, scale=sigma / np.sqrt(M)),
                  'r', lw=2.0, label='pdf')
         plt.legend()
```

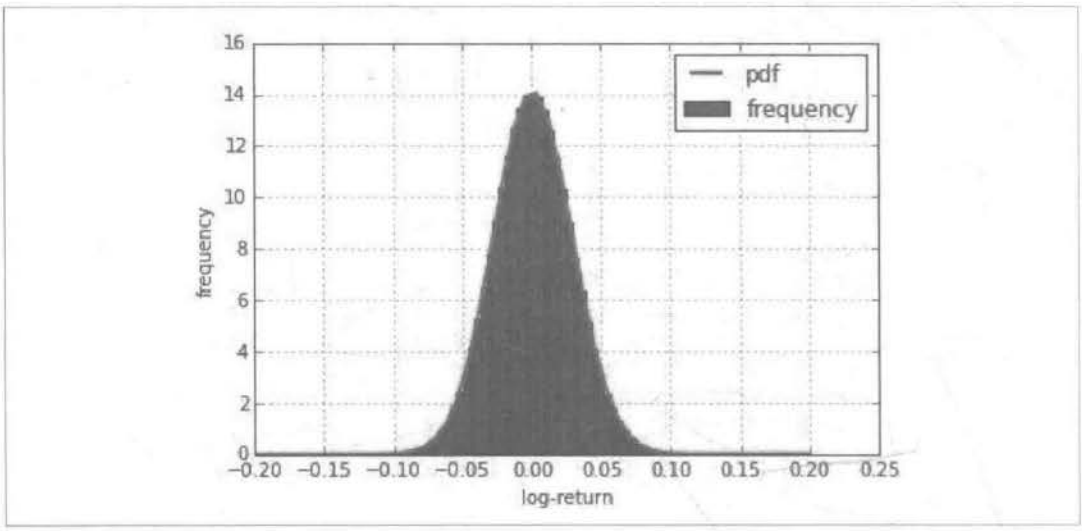


图 11-2　对数收益率和正态密度函数的直方图

对比频率分布（直方图）与理论化 pdf 不是图形化“检验”正态性的唯一方法。所谓的分位数-分位数图（qq 图）也很适合于这一任务。在这种图表中，样本分位数值与理论分位数值对比。对于正态分布的样本数据集，qq 图如图 11-3 所示，绝大多数分位数值（点）位于一条直线上：

```
In [12]: sm.qqplot(log_returns.flatten()[::500], line='s')
         plt.grid(True)
         plt.xlabel('theoretical quantiles')
         plt.ylabel('sample quantiles')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

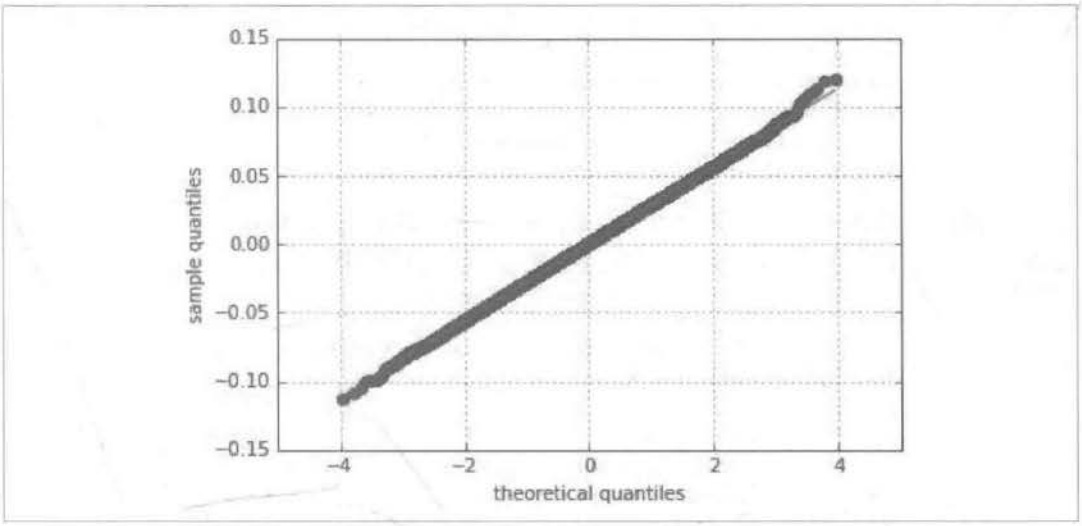


图 11-3　对数收益率的分位数-分位数图

尽管图形方法很有吸引力，但是它们通常无法代替更严格的测试过程。**normality_tests** 函数组合了 3 种不同的统计学测试：

偏斜度测试（skewtest）

测试样本数据的偏斜是否“正态”（也就是值足够接近 0）。

峰度测试（kurtosistest）

与上一种测试类似，测试样本数据的峰度是否“正态”（同样是值足够接近 0）。

正态性测试（normaltest）

组合其他两种测试方法，检验正态性。

我们定义如下函数：

```
In [13]: def normality_tests(arr):
             ''' Tests for normality distribution of given data set.

             Parameters
             ==========
             array: ndarray
                 object to generate statistics on
             '''
             print "Skew of data set  %14.3f" % scs.skew(arr)
             print "Skew test p-value %14.3f" % scs.skewtest(arr)[1]
             print "Kurt of data set  %14.3f" % scs.kurtosis(arr)
             print "Kurt test p-value %14.3f" % scs.kurtosistest(arr)[1]
             print "Norm test p-value %14.3f" % scs.normaltest(arr)[1]
```

测试值表明，对数收益率确实呈正态分布——也就是说，它们的 p 值为 0.05 或者更高：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [14]: normality_tests(log_returns.flatten())
Out[14]: Skew of data set            0.001
         Skew test p-value           0.430
         Kurt of data set            0.001
         Kurt test p-value           0.541
         Norm test p-value           0.607
```

最后，我们检查期末值是否确实呈正态分布。这也归结于正态性检验，因为我们只需要应用对数函数转换数据（得到正态分布数据——也可能得不到）。图 11-4 中绘出了正态分布的期末值和转换后的值（“对数指数水平”）：

```
In [15]: f, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 4))
         ax1.hist(paths[-1], bins=30)
         ax1.grid(True)
         ax1.set_xlabel('index level')
         ax1.set_ylabel('frequency')
         ax1.set_title('regular data')
         ax2.hist(np.log(paths[-1]), bins=30)
         ax2.grid(True)
         ax2.set_xlabel('log index level')
         ax2.set_title('log data')
```

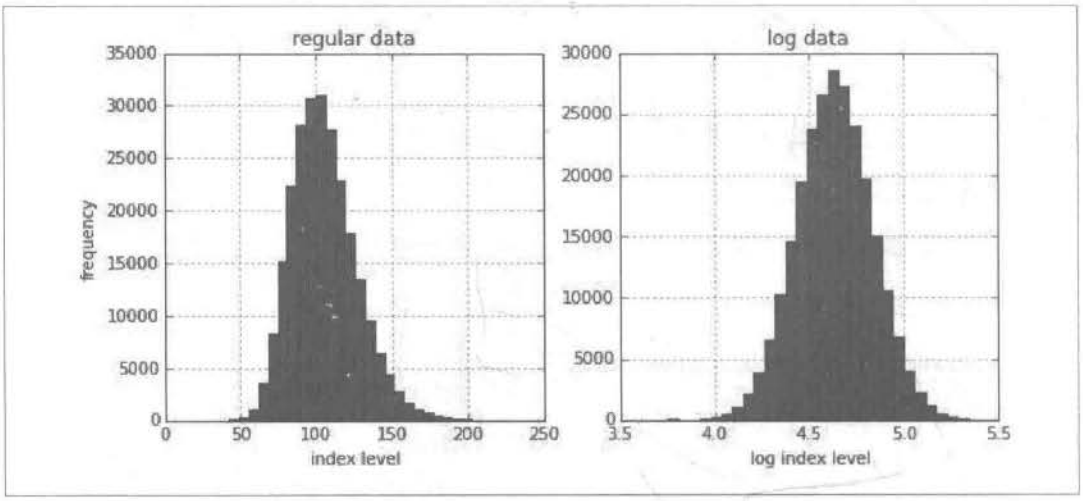


图 11-4　模拟期末指数水平直方图

数据集的统计数字和预期的表现一样——例如，均值接近 105，标准差（波动性）接近 20%：

```
In [16]: print_statistics(paths[-1])
Out[16]:      statistic           value
         ------------------------------
                  size     250000.00000
                   min         42.74870
                   max        233.58435
                  mean        105.12645
                   std         21.23174
                  skew          0.61116
              kurtosis          0.65182
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

对数指数水平值的偏斜度和峰度也接近于 0：

```
In [17]: print_statistics(np.log(paths[-1]))
Out[17]:      statistic           value
         ------------------------------------
                    size    250000.00000
                     min         3.75534
                     max         5.45354
                    mean         4.63517
                     std         0.19998
                    skew        -0.00092
                kurtosis        -0.00327
```

这个数据集也展现了高的 p 值，为正态分布假设提供了很强的支持：

```
In [18]: normality_tests(np.log(paths[-1]))
Out[18]: Skew of data set          -0.001
         Skew test p-value          0.851
         Kurt of data set          -0.003
         Kurt test p-value          0.744
         Norm test p-value          0.931
```

图 11-5 再次比较频率分布和正态分布的 pdf，两者相当匹配（当然，这在意料之中）：

```
In [19]: log_data = np.log(paths[-1])
         plt.hist(log_data, bins=70, normed=True, label='observed')
         plt.grid(True)
         plt.xlabel('index levels')
         plt.ylabel('frequency')
         x = np.linspace(plt.axis()[0], plt.axis()[1])
         plt.plot(x, scs.norm.pdf(x, log_data.mean(), log_data.std()),
                  'r', lw=2.0, label='pdf')
         plt.legend()
```

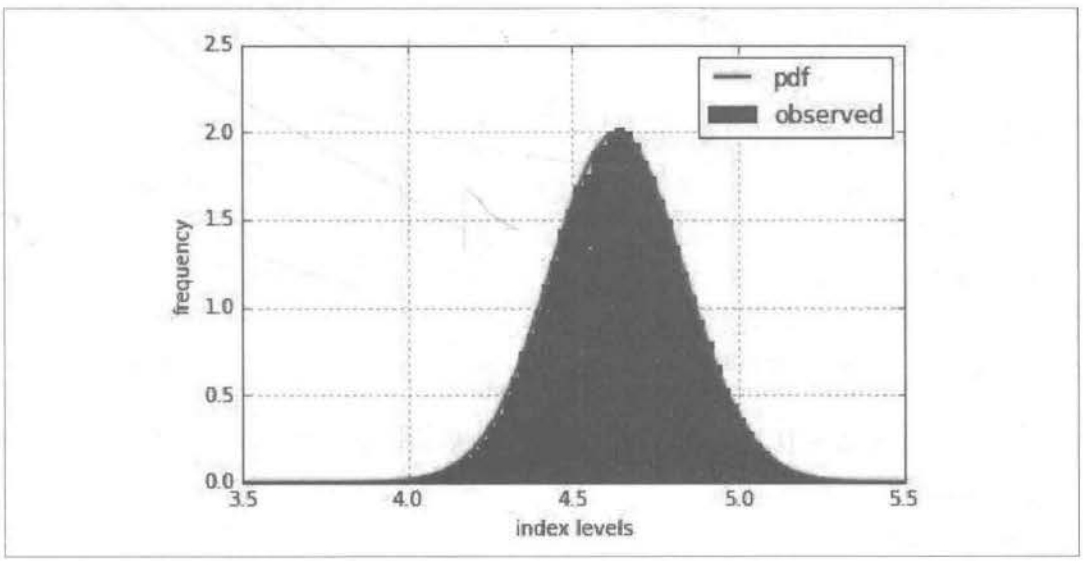


图 11-5　对数指数水平和正态密度函数的直方图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 11-6 也支持对数指数水平呈正态分布的假设：

```
In [20]: sm.qqplot(log_data, line='s')
         plt.grid(True)
         plt.xlabel('theoretical quantiles')
         plt.ylabel('sample quantiles')
```

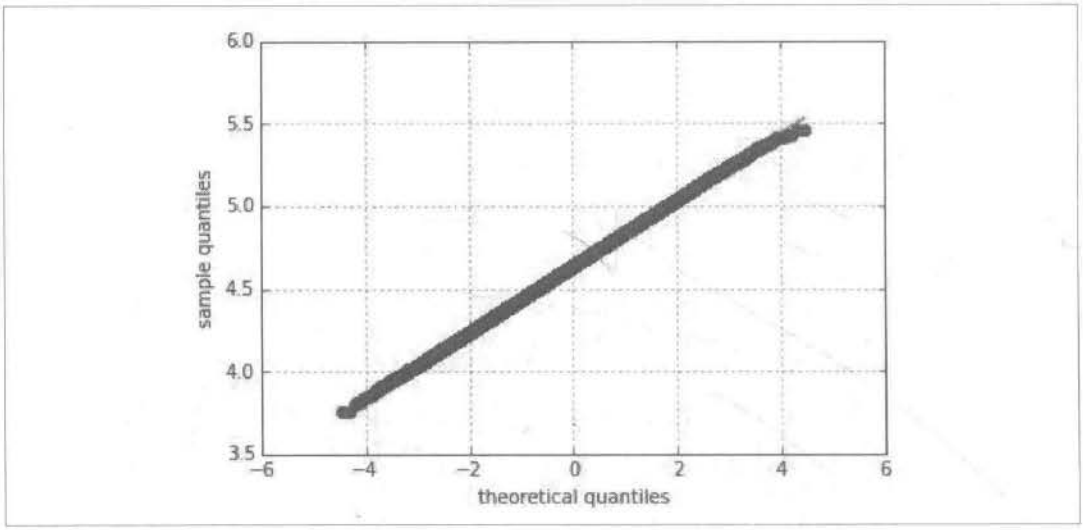


图 11-6　对数指数水平的分位数-分位数图

正态性

证券收益的正态性假设是许多重要金融理论的核心。Python 提供了高效的统计和图形手段，以测试时间序列数据是否正态分布。

11.1.2　现实世界的数据

现在，我们已经有了很好的工具来处理现实世界的数据，看看正态性的假设是否不仅在金融实验室里成立。我们将分析 4 个时间序列：2 种股票指数（德国 DAX 指数和美国 S&P 500 指数），2 种股票（Yahoo!和 Microsoft）。选择的数据管理工具是 **pandas**（参见第 6 章），所以从几个导入指令开始：

```
In [21]: import pandas as pd
         import pandas.io.data as web
```

下面是我们感兴趣的时间序列的股票代号。好奇的读者们当然可以用其他感兴趣的股票代号代替它们：

```
In [22]: symbols = ['^GDAXI', '^GSPC', 'YHOO', 'MSFT']
```

下面的代码只将 **Adj Close** 时间序列数据读入用于所有股票代码的 **DataFrame** 对象：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [23]: data = pd.DataFrame()
         for sym in symbols:
             data[sym] = web.DataReader(sym, data_source='yahoo',
                                        start='1/1/2006')['Adj Close']
         data = data.dropna()
In [24]: data.info()
Out[24]: <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 2179 entries, 2006-01-03 00:00:00 to 2014-09-26 00:00:00
         Data columns (total 4 columns):
         ^GDAXI     2179 non-null float64
         ^GSPC      2179 non-null float64
         YHOO       2179 non-null float64
         MSFT       2179 non-null float64
         dtypes: float64(4)
```

4 个时间序列开始的绝对值有相当大的差异：

```
In [25]: data.head()
Out[25]:                ^GDAXI    ^GSPC   YHOO   MSFT
         Date
         2006-01-03  5460.68  1268.80  40.91  22.09
         2006-01-04  5523.62  1273.46  40.97  22.20
         2006-01-05  5516.53  1273.48  41.53  22.22
         2006-01-06  5536.32  1285.45  43.21  22.15
         2006-01-09  5537.11  1290.15  43.42  22.11
```

图 11-7 直接比较 4 个时间序列，但是规范化为起始值 100：

```
In [26]: (data / data.ix[0] * 100).plot(figsize=(8, 6))
```

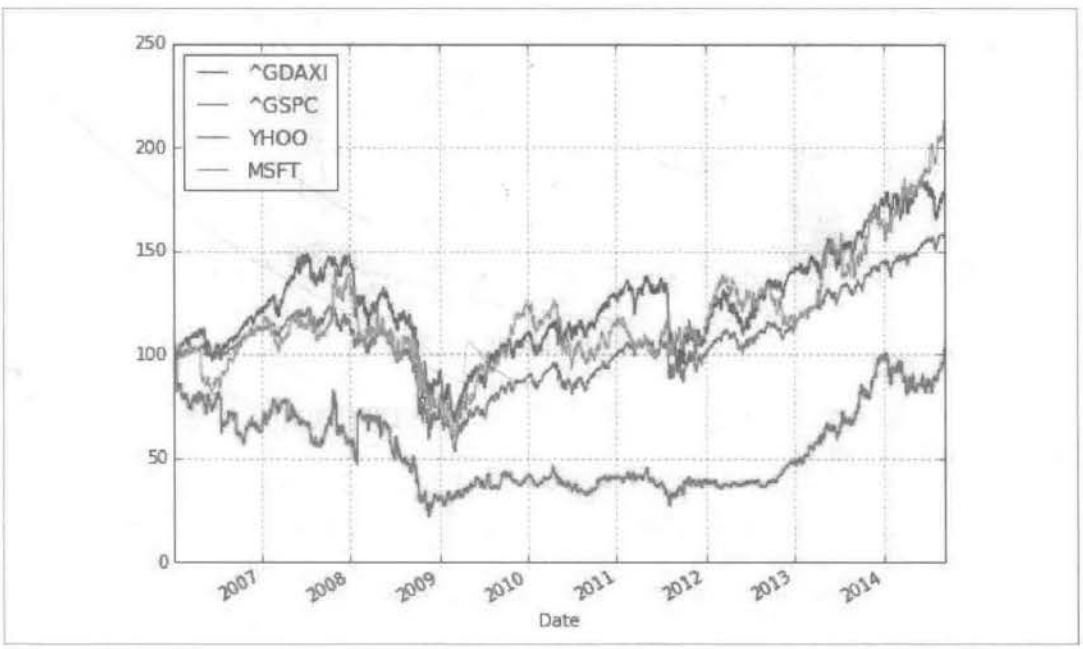


图 11-7　股票和指数水平在一段时间内的变动

用 **pandas** 计算对数收益率比 **NumPy** 更方便一些，因为我们可以使用 **shift** 方法：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [27]: log_returns = np.log(data / data.shift(1))
         log_returns.head()
Out[27]:                ^GDAXI     ^GSPC      YHOO      MSFT
         Date
         2006-01-03        NaN       NaN       NaN       NaN
         2006-01-04   0.011460  0.003666  0.001466  0.004967
         2006-01-05  -0.001284  0.000016  0.013576  0.000900
         2006-01-06   0.003581  0.009356  0.039656 -0.003155
         2006-01-09   0.000143  0.003650  0.004848 -0.001808
```

图 11-8 以直方图的形式提供所有对数收益率。尽管不容易判断，但是可以猜测这些频率分布可能不是正态分布：

```
In [28]: log_returns.hist(bins=50, figsize=(9, 6))
```

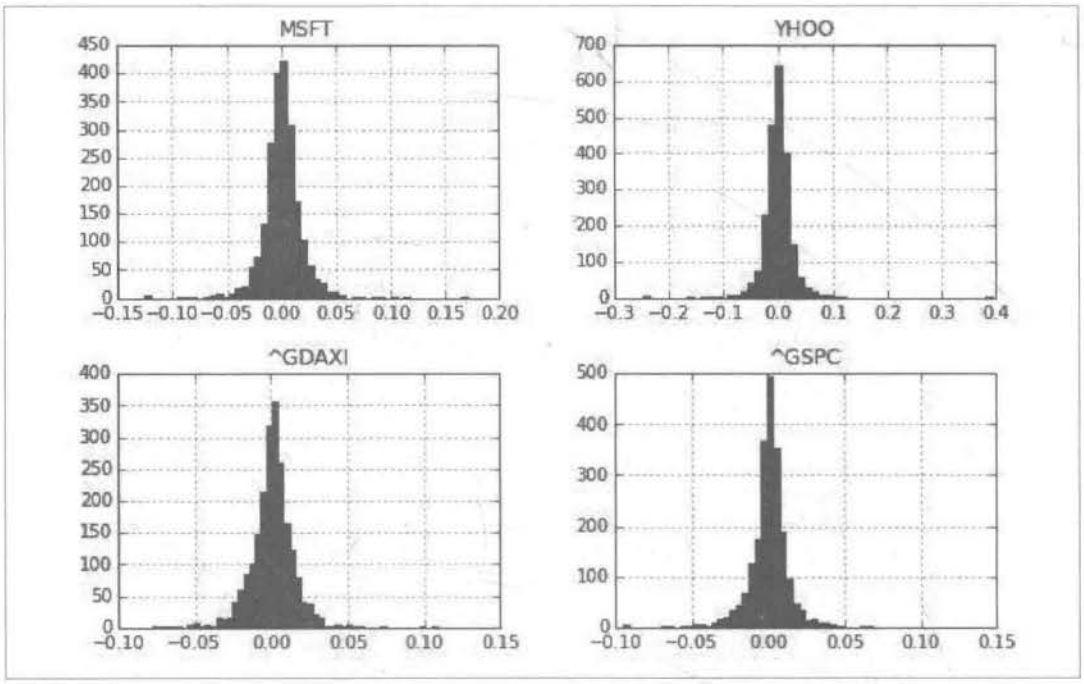


图 11-8 对数收益率直方图

下一步考虑时间序列数据集的不同统计数字。峰度值似乎在所有 4 个数据集上都与正态分布的要求相去甚远：

```
In [29]: for sym in symbols:
             print "\nResults for symbol %s" % sym
             print 30 * "-"
             log_data = np.array(log_returns[sym].dropna())
             print_statistics(log_data)
Out[29]: Results for symbol ^GDAXI
         ------------------------------------------------
              statistic           value
         ------------------------------------------------
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            size      2178.00000
             min        -0.07739
             max         0.10797
            mean         0.00025
             std         0.01462
            skew         0.02573
        kurtosis         6.52461

Results for symbol ^GSPC
------------------------------------------------
     statistic           value
------------------------------------------------
            size      2178.00000
             min        -0.09470
             max         0.10957
            mean         0.00020
             std         0.01360
            skew        -0.32017
        kurtosis        10.05425

Results for symbol YHOO
------------------------------------------------
     statistic           value
------------------------------------------------
            size      2178.00000
             min        -0.24636
             max         0.39182
            mean        -0.00000
             std         0.02620
            skew         0.56530
        kurtosis        31.98659

Results for symbol MSFT
------------------------------------------------
     statistic           value
------------------------------------------------
            size      2178.00000
             min        -0.12476
             max         0.17039
            mean         0.00034
             std         0.01792
            skew         0.04262
        kurtosis        10.18038
```

我们将通过 qq 图检查两个代码的数据。图 11-9 展示了 S&P 500 的 qq 图。很显然，样本的分位数值不在一条直线上，表明“非正态性”。在左侧和右侧分别有许多值远低于和远高于直线。换言之，这一时间序列信息展现出“*大尾巴*”（*Fat tails*）。大尾巴一词指的是（频率）分布中观察到的正负异常值远多于正态分布应有表现的情况。生成图表的代码如下：

```
In [30]: sm.qqplot(log_returns['^GSPC'].dropna(), line='s')
         plt.grid(True)
         plt.xlabel('theoretical quantiles')
         plt.ylabel('sample quantiles')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

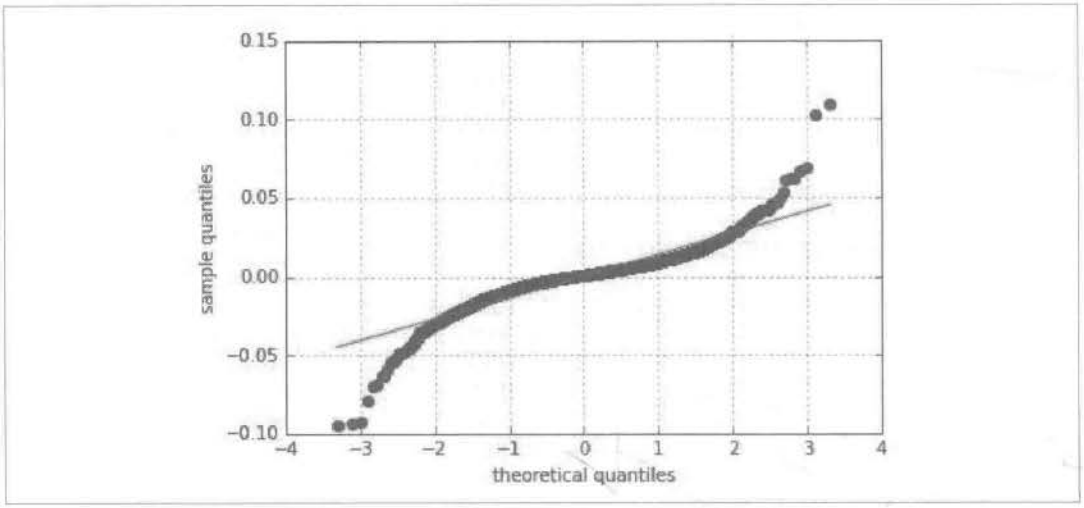


图 11-9　S&P 500 对数收益率分位数-分位数图

图 11-10 提供了 Microsoft 公司股票的数据，可以从中得出相同的结论，分布中也有明显的“大尾巴”现象：

```
In [31]: sm.qqplot(log_returns['MSFT'].dropna(), line='s')
         plt.grid(True)
         plt.xlabel('theoretical quantiles')
         plt.ylabel('sample quantiles')
```

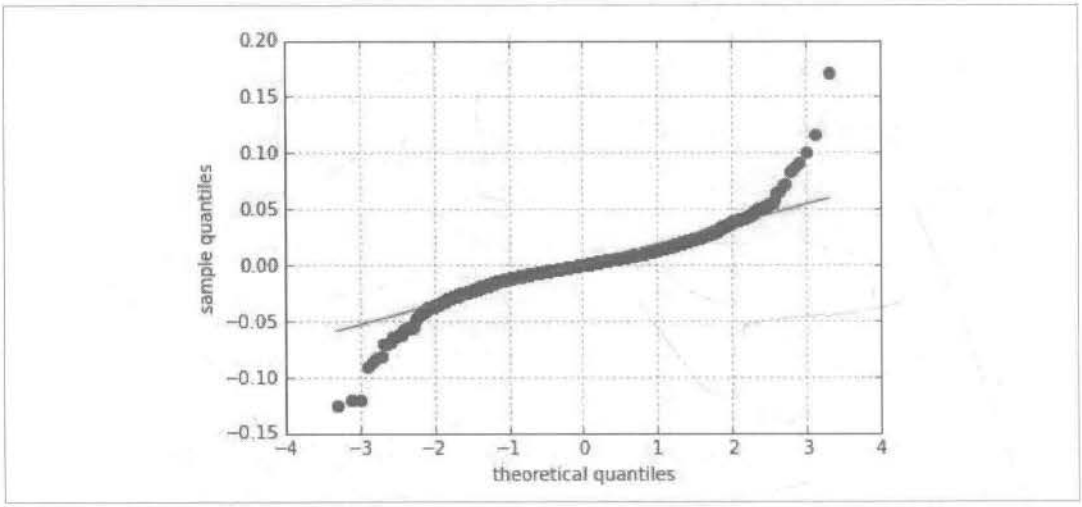


图 11-10　Microsoft 对数收益率分位数-分位数图

我们最后用上述结果进行正式的正态性检验：

```
In [32]: for sym in symbols:
             print "\nResults for symbol %s" % sym
             print 32 * "-"
             log_data = np.array(log_returns[sym].dropna())
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
normality_tests(log_data)
```

```
Out[32]: Results for symbol ^GDAXI
         ------------------------------------------------
         Skew of data set           0.026
         Skew test p-value          0.623
         Kurt of data set           6.525
         Kurt test p-value          0.000
         Norm test p-value          0.000

         Results for symbol ^GSPC
         ------------------------------------------------
         Skew of data set          -0.320
         Skew test p-value          0.000
         Kurt of data set          10.054
         Kurt test p-value          0.000
         Norm test p-value          0.000

         Results for symbol YHOO
         ------------------------------------------------
         Skew of data set           0.565
         Skew test p-value          0.000
         Kurt of data set          31.987
         Kurt test p-value          0.000
         Norm test p-value          0.000

         Results for symbol MSFT
         ------------------------------------------------
         Skew of data set           0.043
         Skew test p-value          0.415
         Kurt of data set          10.180
         Kurt test p-value          0.000
         Norm test p-value          0.000
```

自始至终，不同测试的 p 值都为 0，*强烈否决*不同样板数据集呈正态分布的*测试假设*。这说明，股票市场收益率的正态假设——例如几何布朗运动模型中的假设——通常无法证明是正确的，可能需要使用产生“大尾巴”的更丰富模型（例如，跳跃扩散模型或者具备随机波动率的模型）。

11.2 投资组合优化

现代/均值-方差投资组合理论（MPT）是金融理论的重要基础。因为这一理论突破，1990 年的诺贝尔经济学奖授予了其发明人 Harry Markowitz。尽管这种方法是 20 世纪 50 年代提出的[1]，但时至今日仍然是一种教授金融学学生并在实践中应用（往往有或大或小的改良）的理论。本节阐述这种理论的基本原理。

Copeland、Wetson 和 Shastri（2005）的著作第 5 章很好地介绍了与 MPT 相关联的正式

1 参见 Markowitz，Harry（1952）：“Portfolio Selection”《Journal of Finance》，第 7 卷，77-91 页。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

主题。正如前面所指出的，正态分布收益率假设是该理论的基础：

> 只观察均值和方差，我们必然假定没有必要用其他统计数字描述期末财富的分布。除非投资者有特殊的效用函数（二次效用函数），否则有必要假设收益率呈正态分布，用均值和方差就可以完整地描述。

11.2.1 数据

我们导入几个目前已经耳熟能详的库，开始 **Python** 会话：

```
In [33]: import numpy as np
         import pandas as pd
         import pandas.io.data as web
         import matplotlib.pyplot as plt
         %matplotlib inline
```

我们选择 5 种不同的资产以供分析：美国科技类股票 Apple、Yahoo!和 Micrsoft，以及德意志银行股份有限公司和通过一种交易所交易基金（ETF）进行的黄金交易。MPT 的基本思路是通过*分散投资*实现投资组合风险最小化或者在指定风险水平下的组合收益最大化。通过合理地组合足够大量的资产和资产的多样化，可以预期上述的结果。然而，为了表达基本思路并说明典型的效果，5 种资产应该足够了：

```
In [34]: symbols = ['AAPL', 'MSFT', 'YHOO', 'DB', 'GLD']
         noa = len(symbols)
```

使用 **pandas** 的 **DataReader** 函数（参见第 6 章）可以相当高效地获取这些时间序列数据。正如前一个例子，我们只对每种股票的**收盘价**（**Close**）感兴趣：

```
In [35]: data = pd.DataFrame()
         for sym in symbols:
             data[sym] = web.DataReader(sym, data_source='yahoo',
                                        end='2014-09-12')['Adj Close']
         data.columns = symbols
```

图 11-11 以图形方式展示了规范化之后的时间序列数据：

```
In [36]: (data / data.ix[0] * 100).plot(figsize=(8, 5))
```

均值-方差指的是不同证券（对数）收益的均值和方差，可以这样计算：

```
In [37]: rets = np.log(data / data.shift(1))
```

在这些时间序列数据中，可以看到*年化收益表现*的显著差异。我们使用 252 个交易日，从每日收益得出年化收益：

```
In [38]: rets.mean() * 252
Out[38]: AAPL    0.266036
         MSFT    0.114476
         YHOO    0.196165
         DB     -0.125170
         GLD     0.016054
         dtype: float64
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

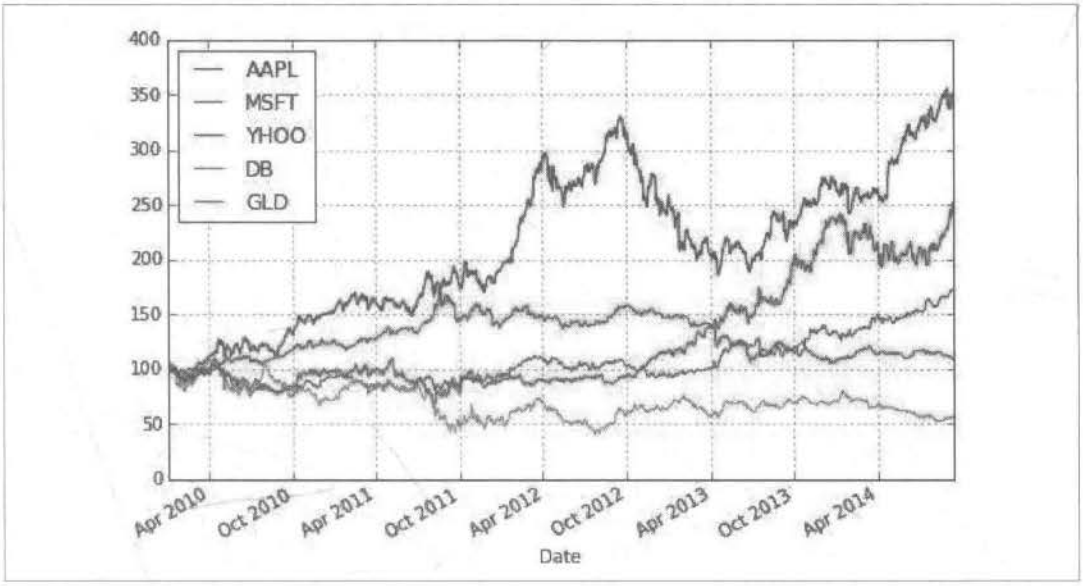


图 11-11　一段时间的股票价格走势

投资资产的协方差矩阵是整个投资组合选择过程的核心部分。**pandas** 有一个内建方法，可以生成协方差矩阵：

```
In [39]: rets.cov() * 252
Out[39]:
            AAPL      MSFT      YHOO        DB       GLD
AAPL    0.072813  0.020426  0.023254  0.041044  0.005234
MSFT    0.020426  0.049384  0.024247  0.046100  0.002105
YHOO    0.023254  0.024247  0.093349  0.051528 -0.000864
DB      0.041044  0.046100  0.051528  0.177477  0.008775
GLD     0.005234  0.002105 -0.000864  0.008775  0.032406
```

11.2.2　基本理论

下面，我们假定投资者不允许在某种证券上建立空头头寸。只允许多头头寸意味着投资者的财富将在可用资产中分配，所有头寸均为多头（正）头寸，且头寸的总和为 100%。例如，可以在 5 种证券中投入相同的资金量（每种 20%）。下面的代码生成 5 个 0 和 1 之间的随机数，然后对这些数值进行规范化，使所有值的总和为 1：

```
In [40]: weights = np.random.random(noa)
         weights /= np.sum(weights)
In [41]: weights
Out[41]: array([ 0.0346395 ,  0.02726489,  0.2868883 ,  0.10396806,  0.54723926])
```

现在可以检查资产权重综合确实为 1；也就是说 $\sum_i w_i = 1$，其中 I 是资产的数量，$w_i \geq 0$ 是资产的权重。公式 11-1 提供在给定单一证券权重情况下的*预期投资组合收益*。这是假定历史平均表现是未来（预期）表现的最佳预测因素时的*预期投资收益*公式。公式中，r_i 是状态相关未来收益（由假定为正态分布的收益值组成的向量），而 μ_i 是证券

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

i 的预期收益。最后，w^T 是权重向量的转置，μ 是预期证券收益的向量。

公式 11-1　预期收益一般公式

$$\begin{aligned}\mu_p &= \mathrm{E}\left(\sum_I w_i r_i\right) \\ &= \sum_I w_i \mathrm{E}(r_i) \\ &= \sum_I w_i \mu_i \\ &= w^T \mu\end{aligned} \tag{11-1}$$

将其转换为 **Python**，得到如下的代码行，其中再次乘以 252 得到年化收益值：

```
In [42]: np.sum(rets.mean() * weights) * 252
           # expected portfolio return
Out[42]: 0.064385749262353215
```

MPT 中选择的第二个对象是预期投资组合方差。两种证券之间的协方差定义为 $\sigma_{ij} = \sigma_{ji} = \mathrm{E}(r_i - \mu_i)(r_j - \mu_j)$。证券的方差是与自身的协方差的特例：$\sigma_i^2 = \mathrm{E}((r_i - \mu_i)^2)$。公式 11-2 提供了一个证券投资组合的协方差矩阵（假定每种证券的权重均为 1）。

公式 11-2　投资组合协方差矩阵

$$\sum = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1I} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2I} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{I1} & \sigma_{I2} & \cdots & \sigma_I^2 \end{bmatrix} \tag{11-2}$$

利用投资组合协方差矩阵，公式 11-3 提供了预期投资组合方差的公式。

公式 11-3　预期投资组合方差一般公式

$$\begin{aligned}\sigma_p^2 &= \mathrm{E}((r - \mu)^2) \\ &= \sum_{i \in I} \sum_{j \in I} w_i w_j \sigma_{ij} \\ &= w^T \sum w\end{aligned} \tag{11-3}$$

在 **Python** 中，这都可以利用 **NumPy** 的向量化功能归结为一行代码。**dot** 函数给出两个向量/矩阵的点乘。**T** 或者 **transpose** 方法给出向量或者矩阵的转置：

```
In [43]: np.dot(weights.T, np.dot(rets.cov() * 252, weights))
           # expected portfolio variance
Out[43]: 0.024929484097150213
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

（预期）投资组合标准差（波动率）只需要计算一次平方根即可得到（$\sigma_p = \sqrt{\sigma_p^2}$）：

```
In [44]: np.sqrt(np.dot(weights.T, np.dot(rets.cov() * 252, weights)))
           # expected portfolio standard deviation/volatility
Out[44]: 0.15789073467797346
```

语言

MPT 示例再次展示了 Python 将数学概念（如投资组合收益或者投资组合方差）转换为可执行向量化代码的效率（第 1 章的论点之一）。

这大体上完成了均值-方差投资组合选择所需的工具集。投资者最感兴趣的是给定证券组合的风险-收益均衡性及其统计学特性。为此，我们实施一次蒙特卡洛模拟（参见第 10 章），生成较大规模的随机投资组合权重向量。对于每一种模拟的分配，我们记录得出的预期投资组合收益和方差：

```
In [45]: prets = []
         pvols = []
         for p in range (2500):
             weights = np.random.random(noa)
             weights /= np.sum(weights)
             prets.append(np.sum(rets.mean() * weights) * 252)
             pvols.append(np.sqrt(np.dot(weights.T,
                                  np.dot(rets.cov() * 252, weights))))
         prets = np.array(prets)
         pvols = np.array(pvols)
```

图 11-12 展示了蒙特卡洛模拟的结果。除此之外，还提供定义为 $SR \equiv \frac{\mu_p - r_f}{\sigma_p}$ 的夏普指数（即预期投资组合超额收益），也就是投资收益率超过无风险短期利率 r_f 的部分除以预期投资组合标准差。为了简单起见，我们假定 $r_f = 0$：

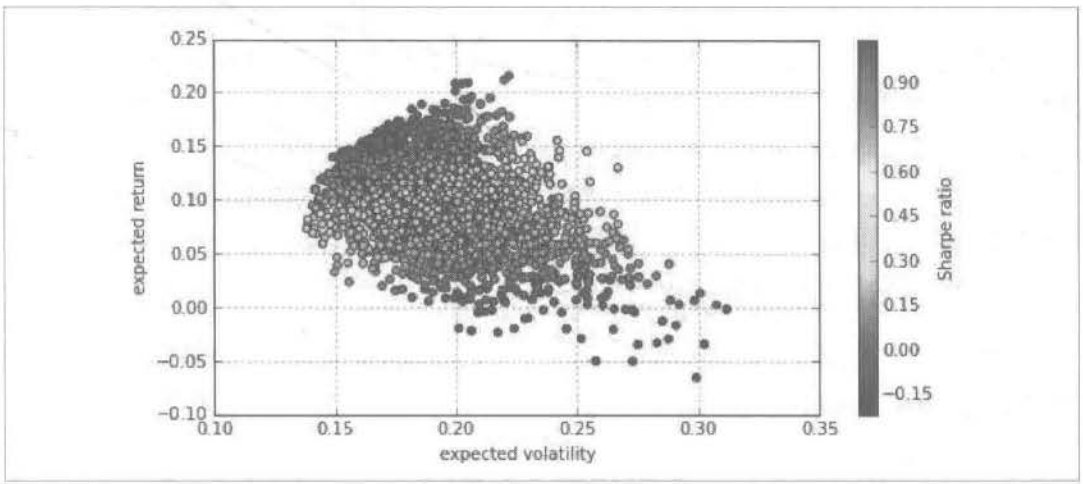


图 11-12　不同/随机投资组合权重的预期收益和波动率

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [46]: plt.figure(figsize=(8, 4))
         plt.scatter(pvols, prets, c=prets / pvols, marker='o')
         plt.grid(True)
         plt.xlabel('expected volatility')
         plt.ylabel('expected return')
         plt.colorbar(label='Sharpe ratio')
```

查看图 11-12 可以明显地看出，从均值和方差考量，并不是所有权重分布都表现良好。例如，对于固定风险水平（如 20%），多种组合都表现出不同的收益。作为投资者，人们通常对固定风险水平的最大收益率或者固定收益率预期下的最小风险感兴趣。这两种投资组合组成所谓的*有效边界*，这是本节后面所要得到的结果。

11.2.3 投资组合优化

为了让生活更轻松一些，我们首先建立一个方便的函数，为输入的权重向量/数组给出重要的投资组合统计数字：

```
In [47]: def statistics(weights):
             ''' Returns portfolio statistics.

             Parameters
             ==========
             weights : array-like
                 weights for different securities in portfolio

             Returns
             =======
             pret : float
                 expected portfolio return
             pvol : float
                 expected portfolio volatility
             pret / pvol : float
                 Sharpe ratio for rf=0
             '''
             weights = np.array(weights)
             pret = np.sum(rets.mean() * weights) * 252
             pvol = np.sqrt(np.dot(weights.T, np.dot(rets.cov() * 252, weights)))
             return np.array([pret, pvol, pret / pvol])
```

最优化投资组合的推导是一个约束最优化问题，我们用来自 **scipy.optimize** 子库的 **minimize** 函数求解（参见第 9 章）：

```
In [48]: import scipy.optimize as sco
```

最小化函数 minimize 很通用，考虑了参数的（不）等式约束和参数的范围。我们从*夏普指数的最大化*开始。正式地说，我们最小化夏普指数的负值：

```
In [49]: def min_func_sharpe(weights):
             return -statistics(weights)[2]
```

约束是所有参数（权重）的总和为 1。这可以用 **minimize** 函数的约定表达如下（参见函数文档（http://bit.ly/using_minimiaze））[1]。

1 np.sum(x)-1 的替代写法是 np.sum(x) == 1，这种写法考虑到 Python 的布尔值 True 等于 1，False 等于 0。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [50]: cons = ({'type': 'eq', 'fun': lambda x:  np.sum(x) - 1})
```

我们还将参数值（权重）限制在 0 和 1 之间。这些值以多个元组组成的一个元组形式提供给最小化函数：

```
In [51]: bnds = tuple((0, 1) for x in range(noa))
```

优化函数调用中忽略的唯一输入是起始参数列表（对权重的初始猜测）。我们简单地使用平均分布：

```
In [52]: noa * [1. / noa,]
Out[52]: [0.2, 0.2, 0.2, 0.2, 0.2]
```

调用该函数不仅返回最优化参数值，还返回其他很多信息。我们将结果存储在 opts 对象中：

```
In [53]: %%time
         opts = sco.minimize(min_func_sharpe, noa * [1. / noa,], method='SLSQP',
                             bounds=bnds, constraints=cons)
Out[53]: CPU times: user 52 ms, sys: 0 ns, total: 52 ms
         Wall time: 50.3 ms
```

结果如下：

```
In [54]: opts
Out[54]:   status: 0
          success: True
             njev: 6
             nfev: 42
              fun: -1.0597540702789927
                x: array([  6.59141408e-01,   8.82635668e-02,   2.52595026e-01,
                    8.34564622e-17,  -8.91214186e-17])
          message: 'Optimization terminated successfully.'
              jac: array([  3.27527523e-05,  -1.61930919e-04,  -2.88933516e-05,
                    1.51561590e+00,   1.24186277e-03,   0.00000000e+00])
              nit: 6
```

我们的主要兴趣点在于获得最优投资组合的构成。为此，我们提供所感兴趣的键值（例中是 x），访问结果对象。最优化工作得出一个投资组合，仅由 5 种资产中的 3 种组成：

```
In [55]: opts['x'].round(3)
Out[55]: array([ 0.659,  0.088,  0.253,  0.   , -0.   ])
```

使用优化中得到的投资组合权重，得出如下统计数字：

```
In [56]: statistics(opts['x']).round(3)
Out[56]: array([ 0.235,  0.222,  1.06 ])
```

预期收益率约为 23.5%，预期波动率约为 22.2%，得到的最优夏普指数为 1.06。

接下来，我们最小化投资组合的方差，这与波动率的最小化相同，但是我们定义一个函数对方差进行最小化：

```
In [57]: def min_func_variance(weights):
             return statistics(weights)[1] ** 2
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

调用 **minimize** 函数的其他细节都保持不变：

```
In [58]: optv = sco.minimize(min_func_variance, noa * [1. / noa,],
                             method='SLSQP', bounds=bnds,
                             constraints=cons)

In [59]: optv

Out[59]:   status: 0
          success: True
             njev: 9
             nfev: 64
              fun: 0.018286019968366075
                x: array([  1.07591814e-01,   2.49124471e-01,   1.09219925e-01,
                    1.01101853e-17,   5.34063791e-01])
          message: 'Optimization terminated successfully.'
              jac: array([ 0.03636634,  0.03643877,  0.03613905,  0.05222051,
                  0.03676446,  0.        ])
              nit: 9
```

这次，投资组合中加入了第 4 种资产。这种组合可以得到*绝对最小方差投资组合*：

```
In [60]: optv['x'].round(3)

Out[60]: array([ 0.108,  0.249,  0.109,  0.   ,  0.534])
```

得到的预期收益率、波动率和夏普指数如下：

```
In [61]: statistics(optv['x']).round(3)

Out[61]: array([ 0.087,  0.135,  0.644])
```

11.2.4 有效边界

所有最优化投资组合——即目标收益率水平下波动率最小的所有投资者（或者给定风险水平下收益率最大的所有投资组合）——的求取都和上述最优化过程类似。唯一的区别是我们必须循环使用多种起始条件。最优化采用两种条件：一是目标收益率水平 **tret**，另一个和以前一样是投资组合权重的总和。每个参数的边界值保持不变：

```
In [62]: cons = ({'type': 'eq', 'fun': lambda x:  statistics(x)[0] - tret},
                 {'type': 'eq', 'fun': lambda x:  np.sum(x) - 1})
         bnds = tuple((0, 1) for x in weights)
```

为了清晰起见，我们定义一个专用函数 min_func，在最小化过程中使用。它仅仅从 **statistics** 函数中返回波动率值：

```
In [63]: def min_func_port(weights):
             return statistics(weights)[1]
```

在不同目标**收益率水平**（**trets**）中循环时，最小化的一个条件会变化。这就是每次循环中更新条件字典对象的原因：

```
In [64]: %%time
         trets = np.linspace(0.0, 0.25, 50)
         tvols = []
         for tret in trets:
             cons = ({'type': 'eq', 'fun': lambda x:  statistics(x)[0] - tret},
                     {'type': 'eq', 'fun': lambda x:  np.sum(x) - 1})
             res = sco.minimize(min_func_port, noa * [1. / noa,], method='SLSQP',
                                bounds=bnds, constraints=cons)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            tvols.append(res['fun'])
        tvols = np.array(tvols)
Out[64]: CPU times: user 4.35 s, sys: 4 ms, total: 4.36 s
         Wall time: 4.36 s
```

图 11-13 展示了最优化的结果。交叉表示给定某个目标收益率的最优投资组合；小点和以前一样表示随机组合。此外，该图还展示了两个较大的星号：一个表示最小波动率/方差投资组合（最左侧的组合），另一个表示具有最大夏普指数的投资组合：

```
In [65]: plt.figure(figsize=(8, 4))
         plt.scatter(pvols, prets,
                     c=prets / pvols, marker='o')
                     # random portfolio composition
         plt.scatter(tvols, trets,
                     c=trets / tvols, marker='x')
                     # efficient frontier
         plt.plot(statistics(opts['x'])[1], statistics(opts['x'])[0],
                  'r*', markersize=15.0)
                     # portfolio with highest Sharpe ratio
         plt.plot(statistics(optv['x'])[1], statistics(optv['x'])[0],
                  'y*', markersize=15.0)
                     # minimum variance portfolio
         plt.grid(True)
         plt.xlabel('expected volatility')
         plt.ylabel('expected return')
         plt.colorbar(label='Sharpe ratio')
```

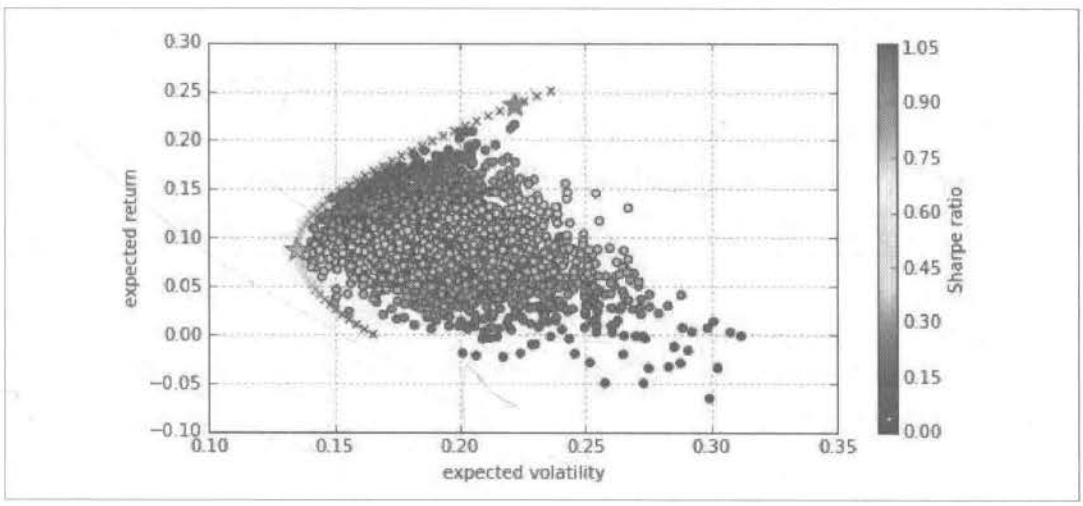


图 11-13　给定收益率水平的最小风险投资组合（交叉）

有效边界由所有收益率高于绝对最小方差投资组合的最优投资组合构成。这些投资组合在给定某一风险水平的预期收益率上优于其他所有投资组合。

11.2.5　资本市场线

除了股票等高风险证券或者商品（如黄金）之外，通常有一种普遍的无风险投资机会：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

现金或者**现金账户**。在理想化的世界中，保存在大银行现金账户中的资金可以认为是无风险的（例如，通过公共存款保险计划）。这种无风险投资的缺点是通常只能得到很少的收益，有时接近于0。

然而，考虑这些无风险资产，可以显著加强投资者的有效投资机会。基本思路是，投资者首先确定高风险资产的一个有效组合，然后在组合中加入无风险资产。通过调整投资于无风险资产中的财富比例，有可能实现任何风险-收益均衡性，这些配置位于（风险-收益空间中）无风险资产和有效投资组合之间的直线上。

（在许多选项中）使用哪一个有效投资组合进行最优化风格的投资？这是有效边界切线恰好通过无风险投资组合风险-收益点的投资组合。例如，考虑无风险利率 $r_f = 0.01$ 的情况。我们寻找有效边界上切线穿过风险-收益空间上的点（σ_f，r_f）=（0，1）的投资组合。

为了进行下面的计算，需要一个函数逼近和有效边界的一阶导数。为此，我们使用三次样条插值（参见第9章）：

```
In [66]: import scipy.interpolate as sci
```

对于样条插值，我们只使用有效边界中的投资组合。下面的代码从过去使用的 **tvols** 和 **trets** 集合中选择这些投资组合：

```
In [67]: ind = np.argmin(tvols)
         evols = tvols[ind:]
         erets = trets[ind:]
```

新的 ndarray 对象 evols 和 erets 用于插值：

```
In [68]: tck = sci.splrep(evols, erets)
```

通过这条数值化路径，最终可以为有效边界定义一个连续可微函数 **f(x)**和对应的一阶导数函数 **df(x)**：

```
In [69]: def f(x):
             ''' Efficient frontier function (splines approximation). '''
             return sci.splev(x, tck, der=0)
         def df(x):
             ''' First derivative of efficient frontier function. '''
             return sci.splev(x, tck, der=1)
```

我们所寻求的是函数 $t(x) = a+b \cdot x$，描述穿过风险-收益空间中无风险资产、与有效边界相切的一条直线。公式11-4描述了函数 $t(x)$满足的全部三个条件。

公式 11-4　资本市场线的数学条件

$$t(x) = a + b \cdot x$$

$$t(0) = r_f \quad \Leftrightarrow \quad a = r_f$$

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

$t(x)=f(x) \quad \Leftrightarrow \quad a+b \cdot x=f(x)$

$t'(x)=f'(x) \quad \Leftrightarrow \quad b=f'(x)$

由于我们没有有效边界的封闭公式或者它的一阶导数，所以必须以数值化方式求得公式 11-4 中方程组的解。为此，我们定义一个 **Python** 函数，返回给定参数集 p = (a, b, x) 下的 3 个方程式的值：

```
In [70]: def equations(p, rf=0.01):
             eq1 = rf - p[0]
             eq2 = rf + p[1] * p[2] - f(p[2])
             eq3 = p[1] - df(p[2])
             return eq1, eq2, eq3
```

scipy.optimize 中的 **fsolve** 函数能够解这类方程组。我们提供初始参数，以及 **equations** 函数。注意，优化的成败可能取决于初始参数化，因此必须小心选择这些参数——通常需要组合合理的猜测，并反复尝试：

```
In [71]: opt = sco.fsolve(equations, [0.01, 0.5, 0.15])
```

数值优化得到如下的值，正如预期，有 $a=r_f$=0.01：

```
In [72]: opt
Out[72]: array([ 0.01      ,  1.01498858,  0.22580367])
```

3 个方程也合乎预期，均为 0：

```
In [73]: np.round(equations(opt), 6)
Out[73]: array([ 0., -0., -0.])
```

图 11-14 以图形方式提供结果：星号代表有效边界中切线穿过无风险资产点(0 , r_f=0.01) 的最优化投资组合。这个最优化组合的预期波动率为 20.5%，预期收益率为 17.6%。生成图表的代码如下：

```
In [74]: plt.figure(figsize=(8, 4))
         plt.scatter(pvols, prets,
                     c=(prets - 0.01) / pvols, marker='o')
                     # random portfolio composition
         plt.plot(evols, erets, 'g', lw=4.0)
                     # efficient frontier
         cx = np.linspace(0.0, 0.3)
         plt.plot(cx, opt[0] + opt[1] * cx, lw=1.5)
                     # capital market line
         plt.plot(opt[2], f(opt[2]), 'r*', markersize=15.0)
         plt.grid(True)
         plt.axhline(0, color='k', ls='-', lw=2.0)
         plt.axvline(0, color='k', ls='-', lw=2.0)
         plt.xlabel('expected volatility')
         plt.ylabel('expected return')
         plt.colorbar(label='Sharpe ratio')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

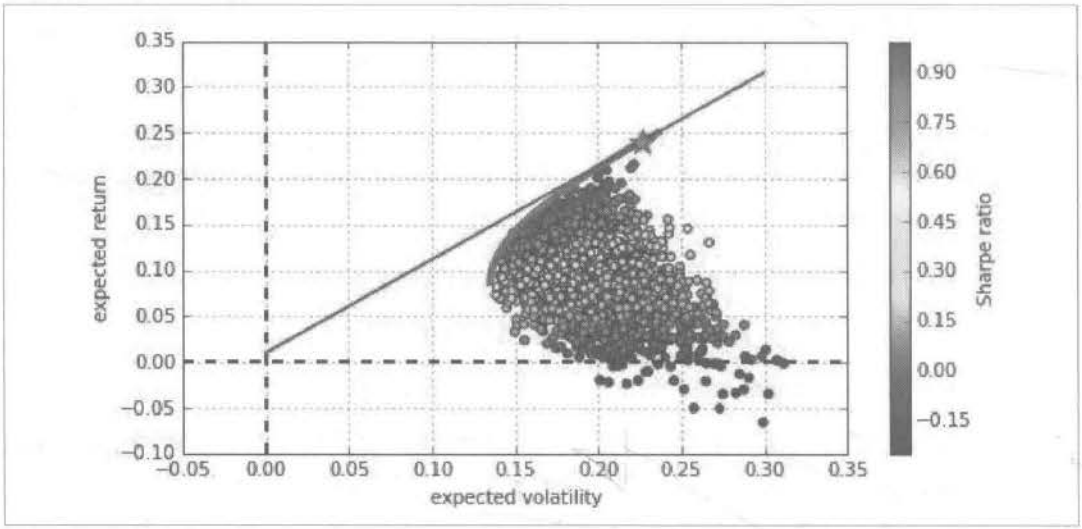


图 11-14　无风险利率为 1%时的资本市场线和相切的投资组合（星号）

最优投资组合的权重如下，组合只包含 5 种资产中的 3 种：

```
In [75]: cons = ({'type': 'eq', 'fun': lambda x:  statistics(x)[0] - f(opt[2])},
                 {'type': 'eq', 'fun': lambda x:  np.sum(x) - 1})
         res = sco.minimize(min_func_port, noa * [1. / noa,], method='SLSQP',
                            bounds=bnds, constraints=cons)
In [76]: res['x'].round(3)
Out[76]: array([ 0.684,  0.059,  0.257, -0.   ,  0.   ])
```

11.3　主成分分析

*主成分分析（PCA）*已经成为金融学中的流行工具。维基百科对这种技术的定义如下：

> 主成分分析（PCA）是一种统计过程，使用正交转换将一组可能相关的变量观测值转换为一组线性无关的变量（主成分）。主成分的数量少于或者等于原始变量数量。这种转换的定义方式是，第一个主成分有最大的可能方差（也就是，考虑数据中尽可能多的易变性），后续的每个成分在正交（即无关）于前面的成分的条件下，具备最大的可能方差。

例如，考虑一个股票指数，如 30 种不同股票构成的德国 DAX 指数。所有股票价格的变动共同决定指数的变动（通过某种有根据的公式）。此外，单独股票价格的变动通常是相关的，例如，由于总体经济状况或者某一领域的一些发展。

对于统计学应用，通常难以使用 30 种相关因素解释某种股票指数的变动。这就是 PCA 发挥作用的地方。它得出单独的不相关“*成分*”，这些成分“很适合”解释股票指数的变动。人们可以将这些成分视为指数中选择股票的线性组合（即投资组合），不用处理 30 种关联的指数成分，而可以处理 5 个、3 个甚至只有 1 个*主成分*。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

本节的例子阐述 PCA 在这种环境下的使用。我们读取德国 DAX 指数和所有成分股的数据，然后使用 PCA 得出主成分，用于构造一个 **pca_index** 对象。

首先是一些导入操作。我们特别要使用 **scikit-learn** 机器学习库的 **KernelPCA** 函数（参见 KernelPCA 的文档（http://bit.ly/kernelPca））：

```
In [1]: import numpy as np
        import pandas as pd
        import pandas.io.data as web
        from sklearn.decomposition import KernelPCA
```

11.3.1 DAX 指数和 30 种成分股

下面的列表对象包含了德国 DAX 指数 30 种成分股的代码，以及指数本身的代码：

```
In [2]: symbols = ['ADS.DE', 'ALV.DE', 'BAS.DE', 'BAYN.DE', 'BEI.DE',
                   'BMW.DE', 'CBK.DE', 'CON.DE', 'DAI.DE', 'DB1.DE',
                   'DBK.DE', 'DPW.DE', 'DTE.DE', 'EOAN.DE', 'FME.DE',
                   'FRE.DE', 'HEI.DE', 'HEN3.DE', 'IFX.DE', 'LHA.DE',
                   'LIN.DE', 'LXS.DE', 'MRK.DE', 'MUV2.DE', 'RWE.DE',
                   'SAP.DE', 'SDF.DE', 'SIE.DE', 'TKA.DE', 'VOW3.DE',
                   '^GDAXI']
```

我们只使用读取的每个数据集中的收盘价（用 **pandas** 读取股票数据的细节参见第 6 章）：

```
In [3]: %%time
        data = pd.DataFrame()
        for sym in symbols:
            data[sym] = web.DataReader(sym, data_source='yahoo')['Close']
        data = data.dropna()
Out[3]: CPU times: user 408 ms, sys: 68 ms, total: 476 ms
        Wall time: 5.61 s
```

因为经常需要指数数据，所以我们将其分离：

```
In [4]: dax = pd.DataFrame(data.pop('^GDAXI'))
```

DataFrame 对象 **data** 现在包含 30 种 DAX 成分股的对数收益率数据：

```
In [5]: data[data.columns[:6]].head()
Out[5]:             ADS.DE  ALV.DE  BAS.DE  BAYN.DE  BEI.DE  BMW.DE
        Date
        2010-01-04   38.51   88.54   44.85    56.40   46.44   32.05
        2010-01-05   39.72   88.81   44.17    55.37   46.20   32.31
        2010-01-06   39.40   89.50   44.45    55.02   46.17   32.81
        2010-01-07   39.74   88.47   44.15    54.30   45.70   33.10
        2010-01-08   39.60   87.99   44.02    53.82   44.38   32.65
```

11.3.2 应用 PCA

PCA 通常处理规范化的数据集，因此，下面这个便利的函数很有帮助：

```
In [6]: scale_function = lambda x: (x - x.mean()) / x.std()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

首先，考虑多个成分的 PCA（也就是说，我们不限制成分的数量）[1]：

```
In [7]: pca = KernelPCA().fit(data.apply(scale_function))
```

每个成分的重要性（或解释功效）由*特征值*表示。特征值可以在 KernelPCA 对象的属性中找到。这样的分析给出了太多的成分：

```
In [8]: len(pca.lambdas_)
Out[8]: 655
```

因此，我们只观察前 10 个成分。第 10 种成分的影响已经几乎可以忽略不计。

```
In [9]: pca.lambdas_[:10].round()
Out[9]: array([ 22816.,   6559.,   2535.,   1558.,    697.,    442.,    378.,
                  255.,    183.,    151.])
```

我们主要感兴趣的是每种成分的相对重要性，所以将对这些值进行规范化。我们同样适用一个方便的函数：

```
In [10]: get_we = lambda x: x / x.sum()
In [11]: get_we(pca.lambdas_)[:10]
Out[11]: array([ 0.6295725 ,  0.1809903 ,  0.06995609,  0.04300101,  0.01923256,
                 0.01218984,  0.01044098,  0.00704461,  0.00505794,  0.00416612])
```

有了这些信息，情况就清楚得多了。第一个成分可以解释 30 种时间序列中易变性的 60%。前五种成分解释易变性的大约 95%。

```
In [12]: get_we(pca.lambdas_)[:5].sum()
Out[12]: 0.94275246704834414
```

11.3.3 构造 PCA 指数

接下来，我们使用 PCA 构造一段时间内的 PCA（或者因素）指数，将其与原始指数对比。首先，我们建立一个只包含第一个成分的 PCA 指数：

```
In [13]: pca = KernelPCA(n_components=1).fit(data.apply(scale_function))
         dax['PCA_1'] = pca.transform(-data)
```

图 11-15 展示了规范化数据的结果——考虑到这种方法的应用相当简单，这已经不错了：

```
In [14]: import matplotlib.pyplot as plt
         %matplotlib inline
         dax.apply(scale_function).plot(figsize=(8, 4))
```

我们来看看添加**更多**的成分是否能改善结果。为此，需要计算单个结果成分的加权平均数：

```
In [15]: pca = KernelPCA(n_components=5).fit(data.apply(scale_function))
         pca_components = pca.transform(-data)
         weights = get_we(pca.lambdas_)
         dax['PCA_5'] = np.dot(pca_components, weights)
```

---

1 注意，我们在这里（以及随后关于贝叶斯统计的小节）使用绝对股价，而没有使用更有统计学意义的收益率数据，原因是这样做易于理解，使图表更容易解读。在现实世界的应用中，应该使用收益率数据。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

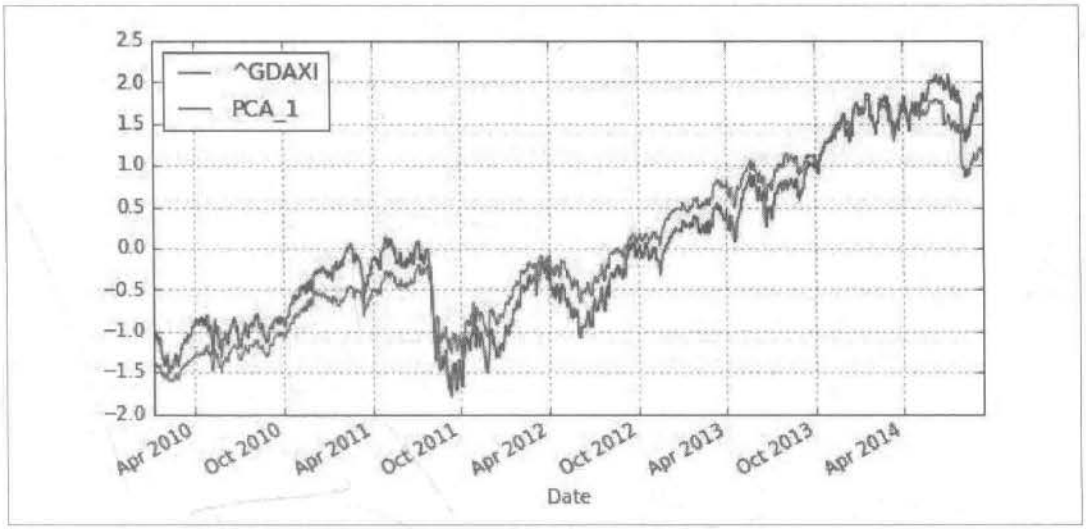


图 11-15　德国 DAX 指数和包含一个成分的 PCA 指数

结果如图 11-16 所示，仍然“良好”，但是比之前没有太大的改善——至少从视觉上是如此：

```
In [16]: import matplotlib.pyplot as plt
         %matplotlib inline
         dax.apply(scale_function).plot(figsize=(8, 4))
```

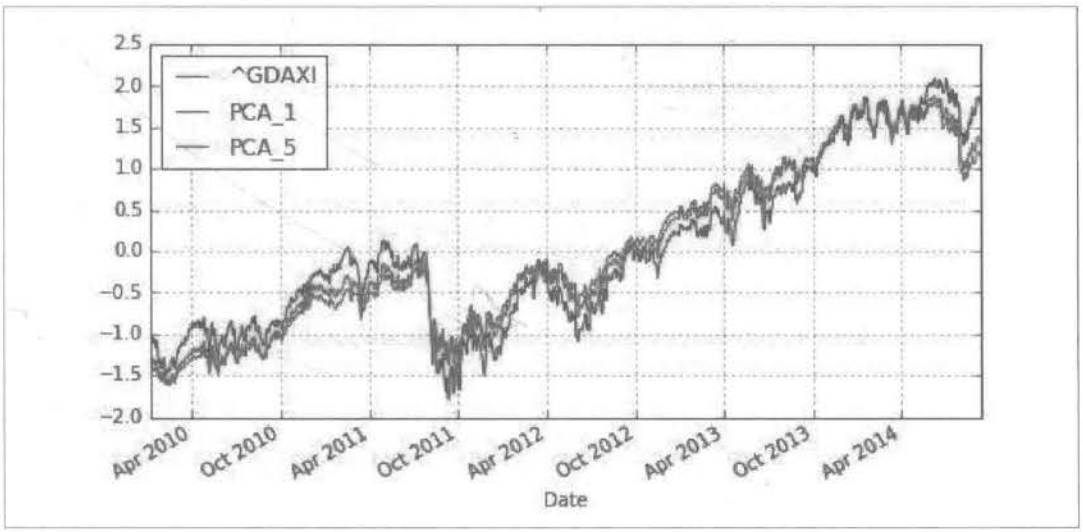


图 11-16　德国 DAX 指数和包含 1 种和 5 种成分的 PCA 指数

考虑到目前的结果，我们应该以不同的方式检查 DAX 指数和 PCA 指数之间的关系——通过散点图，在组合中加入日期信息。首先，我们将 **DataFrame** 对象的 **DatetimeIndex**

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

转换为 **matplotlib** 兼容的格式：

```
In [17]: import matplotlib as mpl
         mpl_dates = mpl.dates.date2num(data.index)
         mpl_dates
Out[17]: array([ 733776.,  733777.,  733778., …,  735500.,  735501.,  735502.])
```

这个新的日期列表可以用于散点图，通过不同颜色强调每个数据点的日期。图 11-17 亿这种方式显示数据：

```
In [18]: plt.figure(figsize=(8, 4))
         plt.scatter(dax['PCA_5'], dax['^GDAXI'], c=mpl_dates)
         lin_reg = np.polyval(np.polyfit(dax['PCA_5'],
                                         dax['^GDAXI'], 1),
                                         dax['PCA_5'])
         plt.plot(dax['PCA_5'], lin_reg, 'r', lw=3)
         plt.grid(True)
         plt.xlabel('PCA_5')
         plt.ylabel('^GDAXI')
         plt.colorbar(ticks=mpl.dates.DayLocator(interval=250),
                      format=mpl.dates.DateFormatter('%d %b %y'))
```

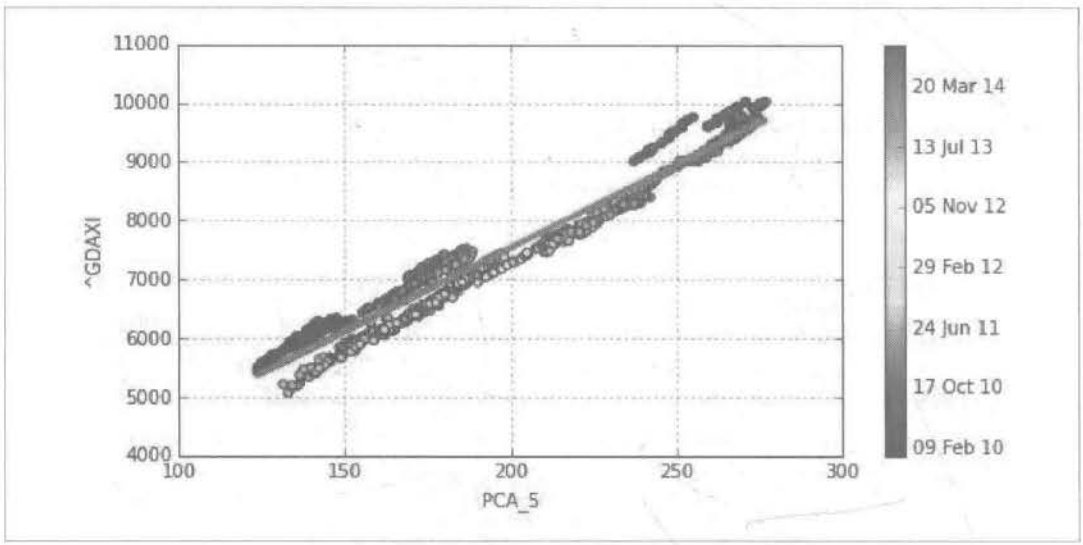


图 11-17　用线性回归对比 DAX 收益率值与 PCA 收益率值

图 11-17 揭示了 2011 年中期的某些时候存在某种结构断裂。如果 PCA 指数完美地复制 DAX 指数，我们就可以预期所有数据点在一条直线上，并且看到回归线穿越这些点。完美是难以达到的，但是我们可以做得更好。

为此，我们将整个时间轴分为两段子时期。然后，我们可以实施一次*早期*回归和一次*晚期*回归：

```
In [19]: cut_date = '2011/7/1'
         early_pca = dax[dax.index < cut_date]['PCA_5']
         early_reg = np.polyval(np.polyfit(early_pca,
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                        dax['^GDAXI'][dax.index < cut_date], 1),
                        early_pca)
In [20]: late_pca = dax[dax.index >= cut_date]['PCA_5']
         late_reg = np.polyval(np.polyfit(late_pca,
                        dax['^GDAXI'][dax.index >= cut_date], 1),
                        late_pca)
```

图 11-18 展示了新的回归线，它确实显示了很高的解释功效，在我们的截止日期前后都是如此。这种启发式方法将在关于贝叶斯统计的下一节中更正式地介绍：

```
In [21]: plt.figure(figsize=(8, 4))
         plt.scatter(dax['PCA_5'], dax['^GDAXI'], c=mpl_dates)
         plt.plot(early_pca, early_reg, 'r', lw=3)
         plt.plot(late_pca, late_reg, 'r', lw=3)
         plt.grid(True)
         plt.xlabel('PCA_5')
         plt.ylabel('^GDAXI')
         plt.colorbar(ticks=mpl.dates.DayLocator(interval=250),
                        format=mpl.dates.DateFormatter('%d %b %y'))
```

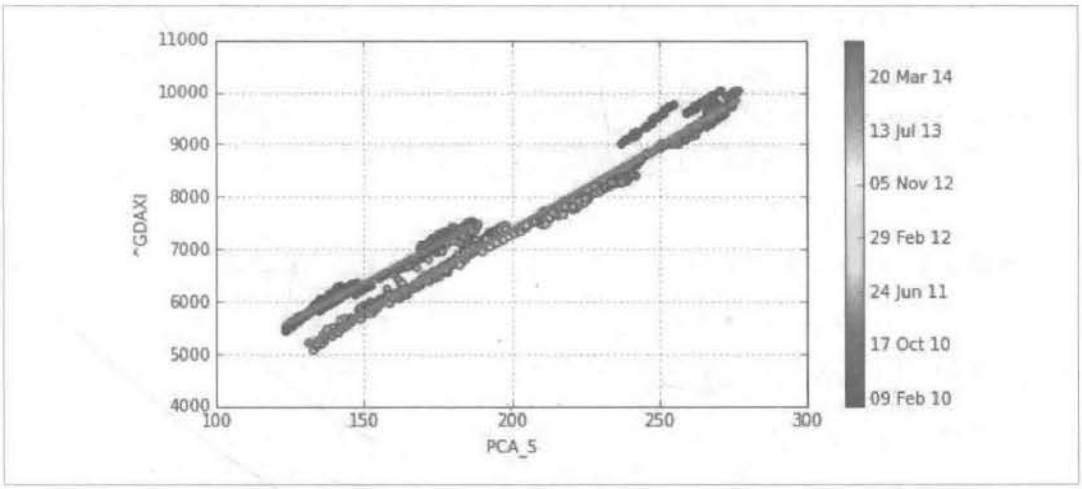


图 11-18　用早期回归和晚期回归（机制转换）对比 DAX 指数值和 PCA 指数值

11.4　贝叶斯回归

近年来，贝叶斯统计已经成为实证金融学的基石。本章无法为该领域的所有概念打下基础。因此，在必要时应该参考 Geweke（2005）的教科书了解一般的入门知识，利益驱动者应该参考 Rachev（2008）的教科书。

11.4.1　贝叶斯公式

贝叶斯公式在金融学中最常见的解读是*历时解读*。这主要说明，随着时间的推移，我们会了解关于感兴趣的变量或者参数的新信息，例如时间序列的平均收益率。公式 11-5

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

正式描述了这种理论，式中 H 表示某个事件（假设），D 代表实验或者真实世界可能提供的数据[1]。在这些定义的基础上，我们得到：

- p(H)称作**先验**概率。
- p(D)是任何假设下数据的概率，称作*标准化常数*。
- p($D|H$)是假设 H 下数据的*似然度*（即概率）。
- p($H|D$)是*后验*概率，即我们看到数据之后得出的概率。

公式 11-5　贝叶斯公式

$$p(H \mid D) = \frac{p(H)p(D \mid H)}{p(D)}$$

考虑一个简单的例子。我们有两个盒子 B_1 和 B_2，B_1 中有 20 个黑球和 70 个红球，而 B_2 中有 40 个黑球和 50 个红球。随机从两个盒子中取出一个球，假定这个球是黑色的。“H_1：球来自 B_1”和“H_2：球来自 B_2”这两个假设的概率分别是多少？

在我们随机取出这个球之前，两个假设的可能性是相同的。但是在球明显是黑色时，我们必须根据贝叶斯公式更新两个假设的概率，考虑假设 $H1$：

- **先验概率**：$p(H_1) = 0.5$
- **标准化常数**：$p(D)$=0.5 • 0.2+0.5 • 0.4=0.3
- **似然度**：$p(D|H_1)$=0.2

更新后的 H_1 概率 p($H_1|D$) = 0.5 • 0.2/0.3=1/3

这一结果也有直观的意义。从 B_2 取出一个黑球的概率两倍于同一件事情发生在 B_1 上的概率。因此，取出一个黑球之后，假设 H_2 的更新后概率 p($H_2|D$)=2/3 也是假设 H_1 更新后概率的 2 倍。

11.4.2　PyMC3

有了 **PyMC3**，**Python** 生态系统就能提供一个强大的高性能库，从技术上实施贝叶斯统计。**PyMC3**（在本书编著时）不是第 2 章推荐的 **Anaconda** 发行版本的一部分。在 **Linux** 或者 **Mac OS X** 操作系统上，安装主要由如下步骤组成。

首先，你必须安装 **PyMC3** 所需的 **Theano** 编译器包（参见 http://blt.ly/install_theano）。在 **shell** 中，执行如下命令：

```
$ git clone git://github.com/Theano/Theano.git
$ sudo python Theano/python.py install
```

1　基于 Python 的贝叶斯统计学基础概念介绍参见 Downey（2013）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

在 Mac OS X 系统上，可能需要在**.bash_profile**（可以在 home/user 目录中找到）文件中添加如下行：

```
export DYLD_FALLBACK_LIBRARY_PATH= \
$DYLD_FALLBACK_LIBRARY_PATH:/Library/anaconda/lib:
```

一旦安装了 **Theano**，**PyMC3** 的安装就很简单了：

```
$ git clone https://github.com/pymc-devs/pymc.git
$ cd pymc
$ sudo python setup.py install
```

如果成功，就可以和平常一样导入 pymc 库：

```
In [22]: import warnings
         warnings.simplefilter('ignore')
         import pymc as pm
         import numpy as np
         np.random.seed(1000)
         import matplotlib.pyplot as plt
         %matplotlib inline
```

PyMC3

PyMC3 在本书编著时已经是一个强大的库，但是，它仍然处于早期阶段，所以可以预期将有进一步的增强和 API 等更改。使用 PyMC3 时，应该定期检查网站（http://bit.ly/pymc3），了解最新信息。

11.4.3 介绍性示例

现在考虑一个例子，例中我们拥有围绕一条直线的有噪声数据[1]：

```
In [23]: x = np.linspace(0, 10, 500)
         y = 4 + 2 * x + np.random.standard_normal(len(x)) * 2
```

作为基准，首先考虑使用 **NumPy** 的 **polyfit** 函数（参见第 9 章）进行这些有噪声数据的普通最小二乘回归。回归的实施如下：

```
In [24]: reg = np.polyfit(x, y, 1)
           # linear regression
```

图 11-19 展示了数据和回归线的图形：

```
In [25]: plt.figure(figsize=(8, 4))
         plt.scatter(x, y, c=y, marker='v')
         plt.plot(x, reg[1] + reg[0] * x, lw=2.0)
         plt.colorbar()
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('y')
```

1 这个例子和下一小节的例子来自于 PyMC3 开发负责人之一 Thomas Wiecki 的演讲（http://bit.ly/bayesian-data-analysis-pymc3）；衷心感谢他允许我在本章中使用这些例子。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

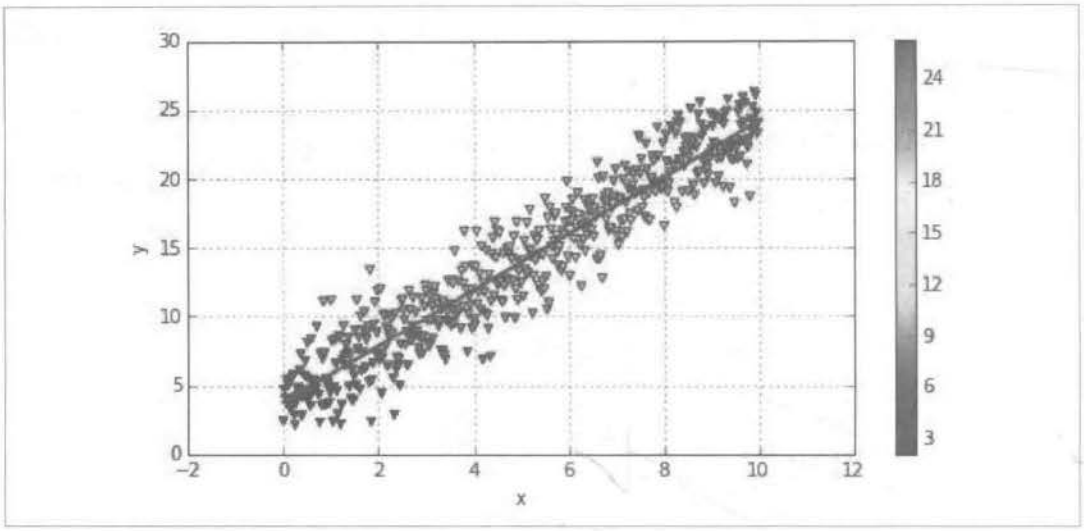


图 11-19　样本数据点和回归线

“标准”回归方法的结果是回归线参数的固定值：

```
In [26]: reg
Out[26]: array([ 2.03384161,  3.77649234])
```

注意，最高阶单项因子（本例中是回归线的斜率）在索引位置 0，截距在索引位置 1。原始参数 2 和 4 没有完好地恢复，当然这是因为数据中包含噪声。

接下来进行贝叶斯回归。在此，我们假定参数呈某种分布。例如，考虑描述回归线的公式 $\hat{y}(x) = \alpha + \beta \cdot x$。现在假定有如下的先验概率：

1. α 呈正态分布，均值为 0，标准差为 20。
2. β 呈正态分布，均值为 0，标准差为 20。

对于似然度，我们假定其呈正态分布，均值为 $\hat{y}(x)$，标准差均匀分布于 0 和 10 之间。

贝叶斯回归的一个要素是（马尔科夫链）蒙特卡洛（MCMC）采样[1]。原则上，这和前一个例子中多次从盒子取出球相同——只是采用更系统性、更自动化的方式。

技术性采样有 3 个不同的函数可以调用：

1. **find_MAP** 求取局部最大后验点，寻找采样算法的起点。
2. **NUTS** 为假定先验概率的 MCMC 采样实现所谓的“高效双平均无回转采样器”

1　参见 http://en.wikipedia.org/wiki/Markov_chain_Monte_Carlo。例如，本书自始至终使用并在第 10 章中详细分析的蒙特卡洛算法生成所谓的马尔科夫链，因为直接的下一步骤/值取决于过程的当前状态，而非其他历史状态或者值。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

（NUTS）算法。

3. **sample** 以给定起始值（来自 find_MAP）和最优化步长（来自 NUTS 算法）提取一定数量的样本。

上述函数都包装到一个 PyMC3Model 对象，在 **with** 语句中执行：

```
In [27]: with pm.Model() as model:
             # model specifications in PyMC3
             # are wrapped in a with statement
         # define priors
         alpha = pm.Normal('alpha', mu=0, sd=20)
         beta = pm.Normal('beta', mu=0, sd=20)
         sigma = pm.Uniform('sigma', lower=0, upper=10)

         # define linear regression
         y_est = alpha + beta * x

         # define likelihood
         likelihood = pm.Normal('y', mu=y_est, sd=sigma, observed=y)

         # inference
         start = pm.find_MAP()
           # find starting value by optimization
         step = pm.NUTS(state=start)
           # instantiate MCMC sampling algorithm
         trace = pm.sample(100, step, start=start, progressbar=False)
           # draw 100 posterior samples using NUTS sampling
```

观察来自第一个样本的估算：

```
In [28]: trace[0]
Out[28]: {'alpha': 3.8783781152509031,
          'beta': 2.0148472296530033,
          'sigma': 2.0078134493352975}
```

3 个值都相当接近原始值（4，2，2）。当然，整个过程会得到许多估算值。它们最好用轨迹图（如图 11-20 所示）描述——这种图表展示了不同参数得到的后验分布以及每个样本的单独估算值。后验分布帮助我们直观地了解估算值中的不确定性：

```
In [29]: fig = pm.traceplot(trace, lines={'alpha': 4, 'beta': 2, 'sigma': 2})
         plt.figure(figsize=(8, 8))
```

仅从回归中取得 **alpha** 和 **beta** 值，就可以绘制图 11-21 中的所有结果回归线：

```
In [30]: plt.figure(figsize=(8, 4))
         plt.scatter(x, y, c=y, marker='v')
         plt.colorbar()
         plt.grid(True)
         plt.xlabel('x')
         plt.ylabel('y')
         for i in range(len(trace)):
             plt.plot(x, trace['alpha'][i] + trace['beta'][i] * x)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

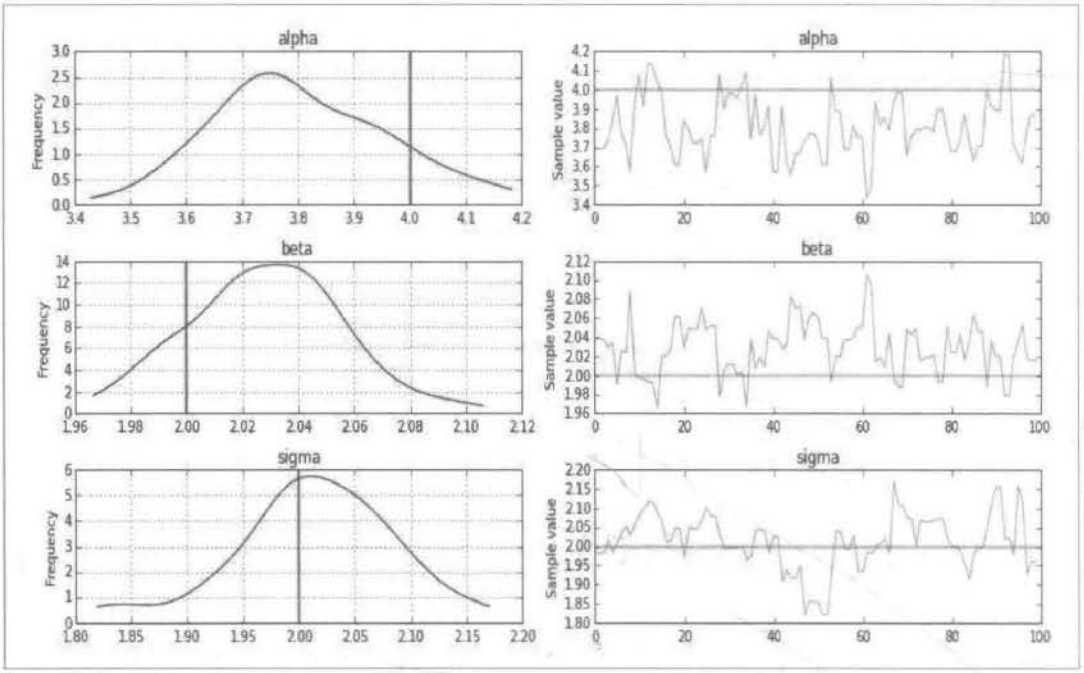


图 11-20　α、β、γ 值的轨迹图

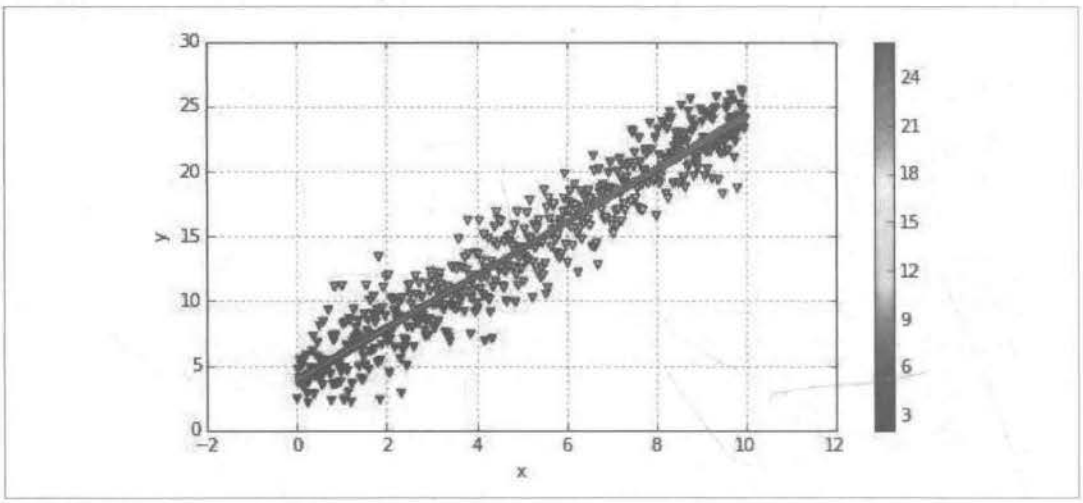


图 11-21　贝叶斯回归中的样本数据和回归线

11.4.4　真实数据

我们已经看到 PyMC3 使用虚拟数据进行贝叶斯回归的运行情况，现在转向真实的市场数据。在这种环境下，我们介绍另一个 **Python** 库：**zipline**（参见 https://github.com/quantopian/zipline 和 https://pypi.python.org/pypi/zipline）。**zipline** 是一个 **Python** 风格的开源算法交易库，驱动社区事后检验平台 Quantopian（http://www.quantopian.com）。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

该库也是单独安装的，例如使用 pip（https://pip.readthedocs.org/en/latest/）：

```
$ pip install zipline
```

安装之后，导入 **zipline**，以及 **pytz** 和 **datetime**：

```
In [31]: import warnings
         warnings.simplefilter('ignore')
         import zipline
         import pytz
         import datetime as dt
```

和 **pandas** 类似，**zipline** 提供一个方便的函数，从不同来源加载财经数据。在其内部，**zipline** 使用 **pandas**。

我们使用的例子是一个“经典”的配对交易策略，也就是说，投资黄金和黄金开采公司的股票。这些投资标的分别由如下代码的 ETF 代表：

- GLD（http://finance.yahoo.com/q/pr?s=GLD+Profile）
- GDX（http://finance.yahoo.com/q/pr?s=GDX+Profile）

我们可以用 **zipline** 加载数据：

```
In [32]: data = zipline.data.load_from_yahoo(stocks=['GLD', 'GDX'],
                    end=dt.datetime(2014, 3, 15, 0, 0, 0, 0, pytz.utc)).dropna()
         data.info()
Out[32]: GLD
         GDX
         <class 'pandas.core.frame.DataFrame'>
         DatetimeIndex: 1967 entries, 2006-05-22 00:00:00+00:00 to 2014-03-14 00
         :00:00+00:00
         Data columns (total 2 columns):
         GDX    1967 non-null float64
         GLD    1967 non-null float64
         dtypes: float64(2)
```

图 11-22 展示了两种 ETF 的历史数据：

```
In [33]: data.plot(figsize=(8, 4))
```

两个品种的绝对表现有明显的不同：

```
In [34]: data.ix[-1] / data.ix[0] - 1
Out[34]: GDX   -0.216002
         GLD    1.038285
         dtype: float64
```

然而，观察图 11-22 可以看出，两个时间序列似乎有相当强的正相关，这从相关数据中也反映出来：

```
In [35]: data.corr()
Out[35]:          GDX       GLD
         GDX  1.000000  0.466962
         GLD  0.466962  1.000000
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

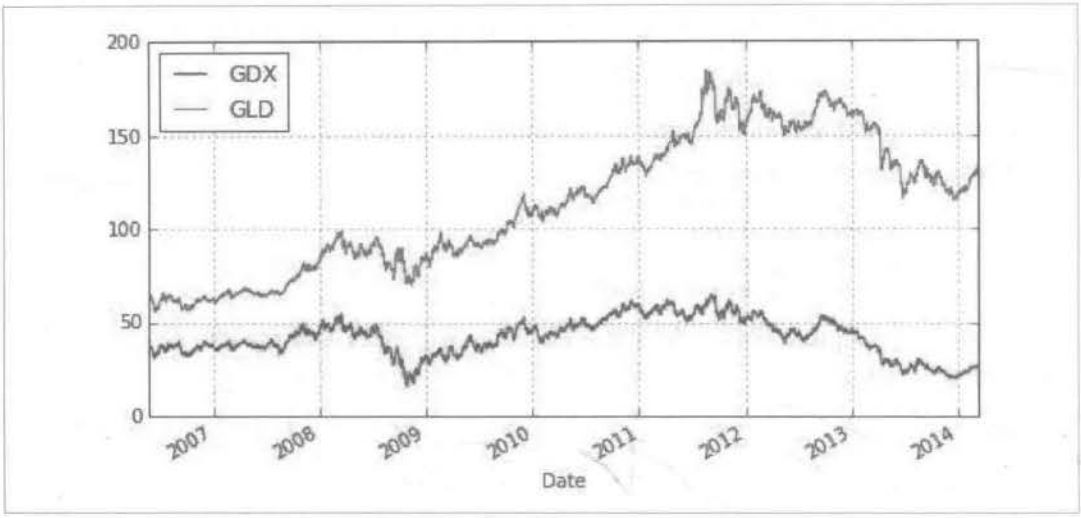


图 11-22　交易配对的联动

和往常一样，**DataFrame** 对象的 **DatetimeIndex** 对象由 **Timestamp** 对象组成：

```
In [36]: data.index
Out[36]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2006-05-22, …, 2014-03-14]
         Length: 1967, Freq: None, Timezone: UTC
```

我们必须首先将日期-时间信息转换为顺序日期表示，才能在后面以希望的方式用于 **matplotlib**：

```
In [37]: import matplotlib as mpl
         mpl_dates = mpl.dates.date2num(data.index)
         mpl_dates
Out[37]: array([ 732453.,  732454.,  732455., …,  735304.,  735305.,  735306.])
```

图 11-23 展示了时间序列数据的散点图，对比 GLD 和 GDX 的价值，以不同的颜色描绘每对数据的日期[1]：

```
In [38]: plt.figure(figsize=(8, 4))
         plt.scatter(data['GDX'], data['GLD'], c=mpl_dates, marker='o')
         plt.grid(True)
         plt.xlabel('GDX')
         plt.ylabel('GLD')
         plt.colorbar(ticks=mpl.dates.DayLocator(interval=250),
                      format=mpl.dates.DateFormatter('%d %b %y'))
```

下面我们在这两个时间序列基础上实施贝叶斯回归，参数化本质上和前一个虚拟数据示例相同，我们只是用现在拥有的真实数据代替虚拟数据：

1　还要注意，这里使用的是绝对价格水平而非更有统计意义的收益率数据。对于真实世界（交易）应用，你最好选择收益率数据实施此类分析。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [39]: with pm.Model() as model:
             alpha = pm.Normal('alpha', mu=0, sd=20)
             beta = pm.Normal('beta', mu=0, sd=20)
             sigma = pm.Uniform('sigma', lower=0, upper=50)

             y_est = alpha + beta * data['GDX'].values

             likelihood = pm.Normal('GLD', mu=y_est, sd=sigma,
                                    observed=data['GLD'].values)

             start = pm.find_MAP()
             step = pm.NUTS(state=start)
             trace = pm.sample(100, step, start=start, progressbar=False)
```

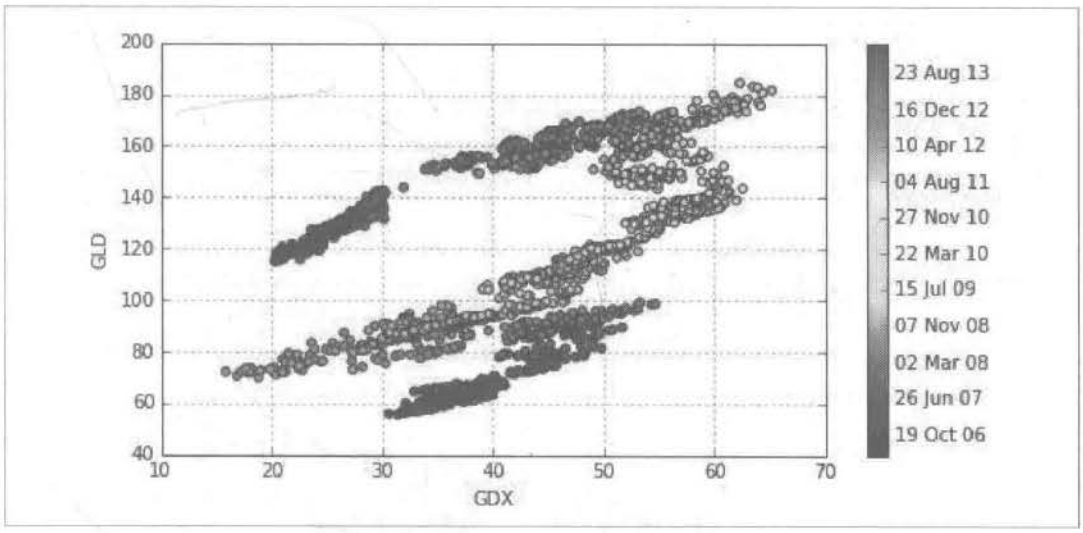


图 11-23　GLD 和 GDX 价格的散点图

图 11-24 展示了在关于 3 个参数先验概率分布的假设下，MCMC 采样过程的结果：

```
In [40]: fig = pm.traceplot(trace)
         plt.figure(figsize=(8, 8))
```

图 11-25 将得到的所有回归线添加到之前的散点图中。所有回归线相互很靠近：

```
In [41]: plt.figure(figsize=(8, 4))
         plt.scatter(data['GDX'], data['GLD'], c=mpl_dates, marker='o')
         plt.grid(True)
         plt.xlabel('GDX')
         plt.ylabel('GLD')
         for i in range(len(trace)):
             plt.plot(data['GDX'], trace['alpha'][i] + trace['beta'][i] * data
                      ['GDX'])
         plt.colorbar(ticks=mpl.dates.DayLocator(interval=250),
                      format=mpl.dates.DateFormatter('%d %b %y'))
```

上图揭示了所用回归方法的一个重大不足：该方法没有考虑随时间推移发生的变化。也就是说，最近的数据和最老的数据受到同等对待。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

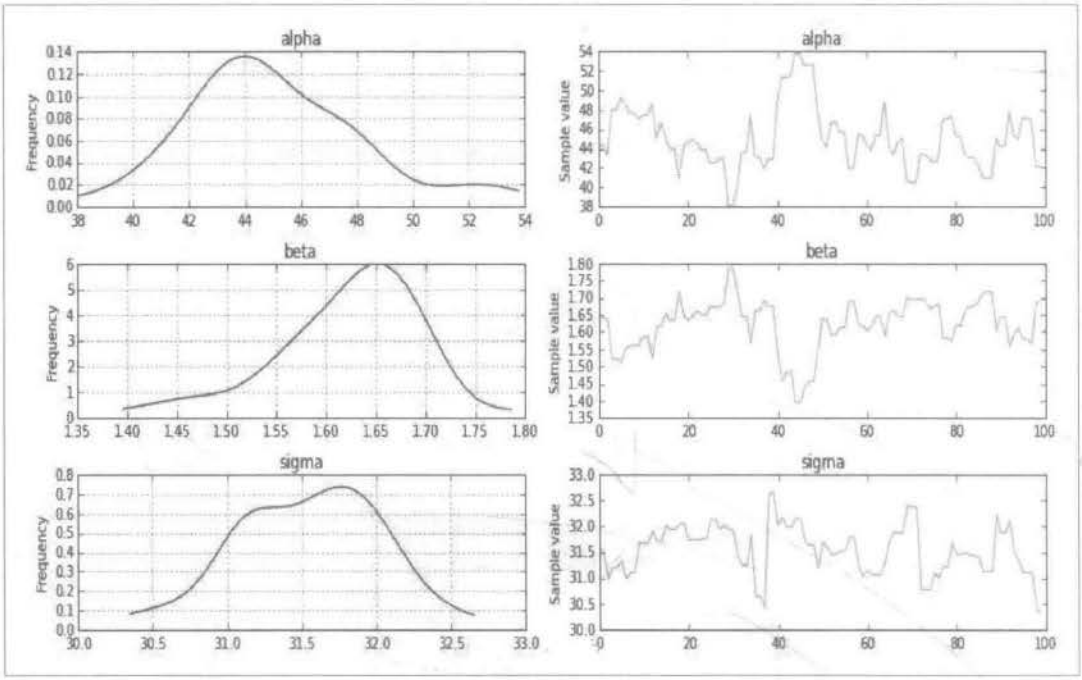


图 11-24　基于 GDX 和 GLD 数据的α、β、γ轨迹图

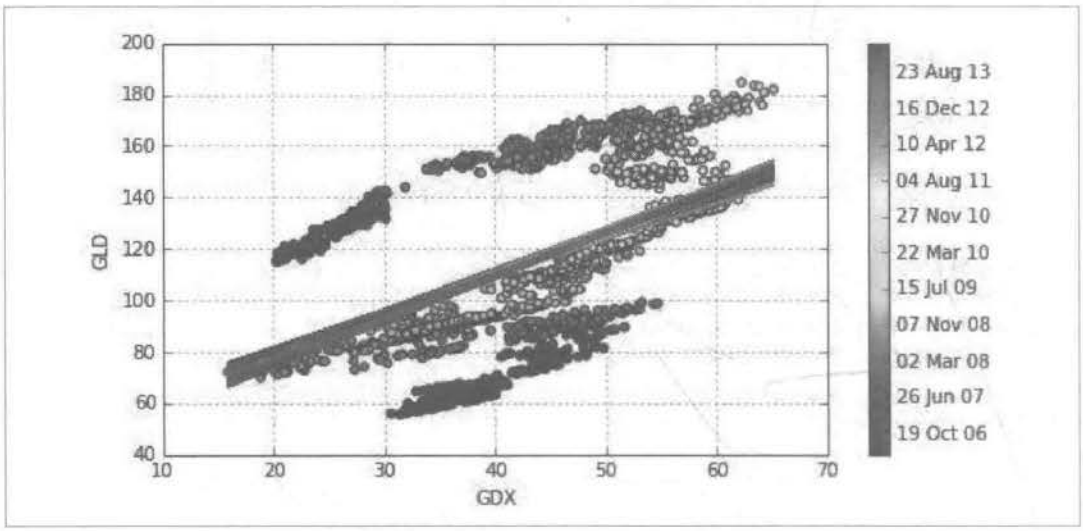


图 11-25　包含“简单”回归线的散点图

正如本节开始时指出的，金融学上的贝叶斯方法通常在历史分析时最有用——也就是说，随着时间推移揭示的新数据可以得出更好的回归和估算。

为了在当前的示例中加入上述概念，我们假定回归参数不只是随机和呈某种分布的，而且随着时间的推移随机“游走”。这和金融理论从随机变量推广到随机过程（本质上

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

是随机变量的有序序列）时采用相同的方式。

为此，我们定义一个新的 PyMC3 模型，这次指定参数值随机游走，方差参数值转换到对数空间（为了得到更好的采样特性）。

```
In [42]: model_randomwalk = pm.Model()
         with model_randomwalk:
             # std of random walk best sampled in log space
             sigma_alpha, log_sigma_alpha = \
                     model_randomwalk.TransformedVar('sigma_alpha',
                                     pm.Exponential.dist(1. / .02, testval=.1),
                                     pm.logtransform)
             sigma_beta, log_sigma_beta = \
                     model_randomwalk.TransformedVar('sigma_beta',
                                     pm.Exponential.dist(1. / .02, testval=.1),
                                     pm.logtransform)
```

在指定随机游走参数的分布之后，我们可以继续指定随机游走的 alpha 和 beta 值。为了使整个过程更加高效，50 个数据点使用相同的系数：

```
In [43]: from pymc.distributions.timeseries import GaussianRandomWalk
         # to make the model simpler, we will apply the same coefficients
         # to 50 data points at a time
         subsample_alpha = 50
         subsample_beta = 50

         with model_randomwalk:
             alpha = GaussianRandomWalk('alpha', sigma_alpha**-2,
                                        shape=len(data) / subsample_alpha)
             beta = GaussianRandomWalk('beta', sigma_beta**-2,
                                       shape=len(data) / subsample_beta)

             # make coefficients have the same length as prices
             alpha_r = np.repeat(alpha, subsample_alpha)
             beta_r = np.repeat(beta, subsample_beta)
```

时间序列数据集长度为 1967 个数据点：

```
In [44]: len(data.dropna().GDX.values)  # a bit longer than 1,950
Out[44]: 1967
```

为了之后的采样，数据点的数量必须能被 50 整除，因此，只有前 1950 个数据点用于回归：

```
In [45]: with model_randomwalk:
             # define regression
             regression = alpha_r + beta_r * data.GDX.values[:1950]

             # assume prices are normally distributed
             # the mean comes from the regression
             sd = pm.Uniform('sd', 0, 20)
             likelihood = pm.Normal('GLD',
                                    mu=regression,
                                    sd=sd,
                                    observed=data.GLD.values[:1950])
```

由于使用随机游走代替单一随机变量，所以这些定义都比以前更复杂一些。不过，

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

MCMC 的推导步骤本质上仍然没有变化。要注意的是，由于我们必须为每个随机游走样本计算一个参数对，共计 1950/50=39 个（以前只需要 1 个），所以计算负担明显增大：

```
In [46]: import scipy.optimize as sco
         with model_randomwalk:
             # first optimize random walk
             start = pm.find_MAP(vars=[alpha, beta], fmin=sco.fmin_l_bfgs_b)

             # sampling
             step = pm.NUTS(scaling=start)
             trace_rw = pm.sample(100, step, start=start, progressbar=False)
```

我们总共对 39 个时间间隔做 100 次估算：

```
In [47]: np.shape(trace_rw['alpha'])
Out[47]: (100, 39)
```

可以绘制估算子集合所有样本平均值的图表（见图 11-26），说明回归因子 **alpha** 和 **beta** 在一段时间内的变动：

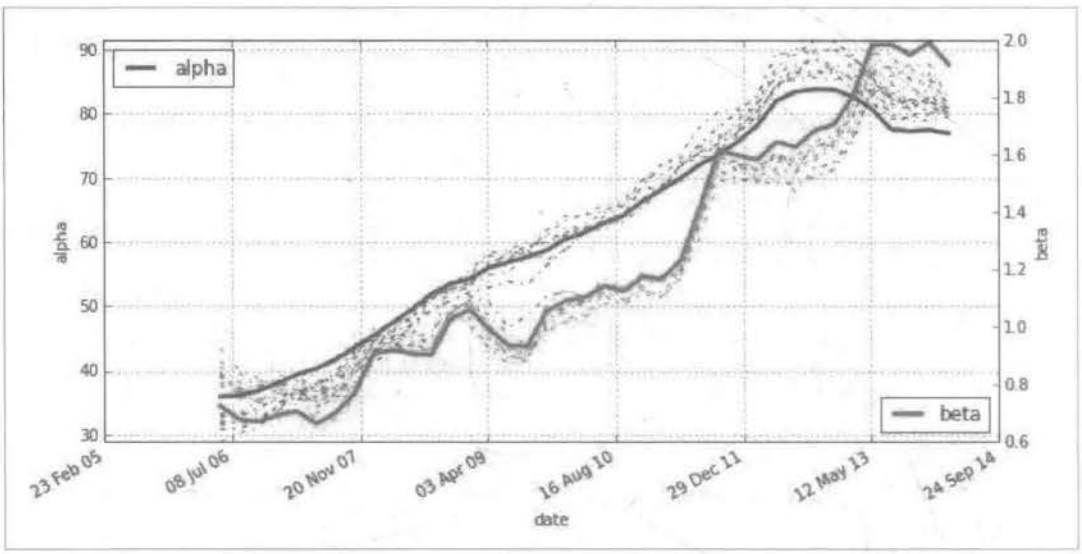


图 11-26　一段时间内（平均）alpha 和（平均）beta 值的变动（随时更新的估算）

```
In [48]: part_dates = np.linspace(min(mpl_dates), max(mpl_dates), 39)
In [49]: fig, ax1 = plt.subplots(figsize=(10, 5))
         plt.plot(part_dates, np.mean(trace_rw['alpha'], axis=0),
                  'b', lw=2.5, label='alpha')
         for i in range(45, 55):
             plt.plot(part_dates, trace_rw['alpha'][i], 'b-.', lw=0.75)
         plt.xlabel('date')
         plt.ylabel('alpha')
         plt.axis('tight')
         plt.grid(True)
         plt.legend(loc=2)
         ax1.xaxis.set_major_formatter(mpl.dates.DateFormatter('%d %b %y') )
         ax2 = ax1.twinx()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
plt.plot(part_dates, np.mean(trace_rw['beta'], axis=0),
         'r', lw=2.5, label='beta')
for i in range(45, 55):
    plt.plot(part_dates, trace_rw['beta'][i], 'r-.', lw=0.75)
plt.ylabel('beta')
plt.legend(loc=4)
fig.autofmt_xdate()
```

绝对价格数据和相对收益率数据

在介绍 PCA 分析的实施以及关于贝叶斯统计的示例时，我们都使用绝对价格水平而非相对（对数）收益率数据。这仅仅是为了说明目的，因为对应的图形化结果更容易理解和解读（它们在视觉上“更引人注目”）。但是，对于现实世界中的金融应用，应该依赖相对收益率数据。

使用 **alpha** 和 **beta** 均值，我们可以说明一段时间内回归的更新。图 11-27 再次以散点图的形式展示了这些数据点。此外，还显示了从 **alpha** 和 **beta** 均值得出的 39 条回归线。很明显，随时间推移进行更新大大提高了回归拟合（对于当前/最新的数据）——换言之，每个时间周期都需要自己的回归：

```
In [50]: plt.figure(figsize=(10, 5))
         plt.scatter(data['GDX'], data['GLD'], c=mpl_dates, marker='o')
         plt.colorbar(ticks=mpl.dates.DayLocator(interval=250),
                      format=mpl.dates.DateFormatter('%d %b %y'))
         plt.grid(True)
         plt.xlabel('GDX')
         plt.ylabel('GLD')
         x = np.linspace(min(data['GDX']), max(data['GDX']))
         for i in range(39):
             alpha_rw = np.mean(trace_rw['alpha'].T[i])
             beta_rw = np.mean(trace_rw['beta'].T[i])
             plt.plot(x, alpha_rw + beta_rw * x, color=plt.cm.jet(256 * i / 39))
```

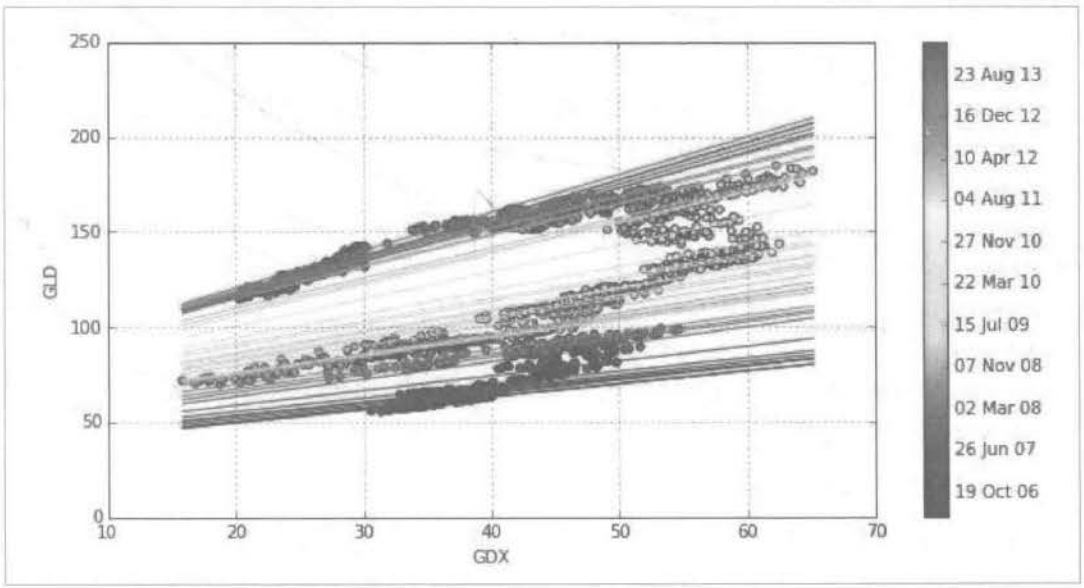


图 11-27　包含时间相关回归线（更新的估算）的散点图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

关于贝叶斯回归的这一节到此告一段落，**Python** 提供了强大的 **PyMC3** 库，实现贝叶斯统计中的不同方法，尤其是贝叶斯回归，它在计量金融学中已经成为相当流行而重要的工具。

11.5 结语

统计不仅自身就是一个重要的学科，而且为许多其他学科（如金融和社会科学）提供了不可或缺的工具。在一个章节中无法广泛地概述统计学。因此，本章集中于 4 个重要的主题，以实例为基础阐述 **Python** 和多个统计库。

正态性检验

关于金融市场收益的正态性假设对于许多金融理论和应用都很重要；因而，测试某些时间序列数据是否遵循这一假设也很重要。正如我们所看到的——通过图形和统计手段——现实世界的收益率数据通常*不是*正态分布的。

现代投资组合理论

MPT 的焦点是收益率的均值和方差/波动率，被视为是金融统计学概念和智能方面的重大成功。分散投资这一重要概念在这一背景中得到了很好的诠释。

主成分分析

PCA 提供了降低因素/成分分析任务复杂度的一种相当有益的方法；我们已经证明，从 DAX 指数中包含的 30 种股票构建的 5 个主成分足以解释指数易变性的 95%。

贝叶斯回归

贝叶斯统计（尤其是贝叶斯回归）已经成为金融学中的流行工具，因为这种方法克服了第 9 章中介绍的其他方法的缺点；尽管数学和形式体系上更加复杂，但是基本思路——如随时间推移更新概率/分布——很容易直观地掌握。

11.6 延伸阅读

下列在线资源很有益处：

- 关于 **SciPy** 统计函数的信息可以在这里找到：http://docs.scipy.org/doc/scipy/reference/stats.html。
- 也可以参考 **statsmodels** 库的文档：http://statsmodels.sourceforge.net/stable/。
- 对于本章中使用的优化函数，可以参考 http:// docs.scipy.org/doc/scipy/reference/optimize.html。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- PyMC3 有一个简短的教程（http://bit.ly/pymc3_tutorial）；在本书编写的时候，该库仍然在早期发行模式中，尚没有完整的文档。

下面是一些有用的参考书：

- Copeland, Thomas, Fred Weston, and Kuldeep Shastri (2005): *Financial Theory andCorporate Policy*, 4th ed. Pearson, Boston, MA.
- Downey, Allen (2013): *Think Bayes*. O'Reilly, Sebastopol, CA.
- Geweke, John (2005): *Contemporary Bayesian Econometrics and Statistics*. JohnWiley & Sons, Hoboken, NJ.
- Rachev, Svetlozar et al. (2008): *Bayesian Methods in Finance*. John Wiley & Sons, Hoboken, NJ.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 12 章

Excel 集成

Microsoft Excel 可能是史上最成功的数据分析平台。

——Kirat Singh

公平地说，Microsoft Excel——作为 Microsoft Office 生产率工具套件的一部分——是金融行业和企业及其他机构金融职能部分中最广泛应用的工具之一。最初作为纸质试算表计算机化版本的 Excel，现在已经成为金融分析和金融应用构建的多用途工具（此外，在其他领域和行业还有许多用例）。

Microsoft Excel 和 LibreOffice Calc 等电子表格应用有几个主要特征。

组织

*工作簿*是一种电子表格应用文件，组织为单独的*工作表*，工作表又组织为*单元格*。

数据

数据通常以表格形式存储在单独的单元格中；单元格包含数据本身（如浮点数或者字符串）、用于显示目的的格式信息（如字体、颜色），可能还有一些计算机代码（例如，单元格中的数据是某个数值运算的结果）。

功能性

数据存储在单独的单元格中，你就可以用这些数据进行计算和其他操作，例如整数加法或者乘法。

可视化

数据很容易可视化，例如绘制饼图。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

可编程性

现代电子表格应用有很灵活的可编程性，例如通过 Excel 电子表格中的 Visual Basic for Application（VBA）编程。

引用

实现功能性或者编写 VBA 代码的主要工具是*单元格*引用；每个单元格有唯一标识的坐标（工作簿、工作表名称、列和行）。

这些特征可以解释其流行性：实现金融分析或者应用所需的所有技术要素都可以在一个地方找到。想一想 **Python** 和前面几章，你需要组合几个库和工具（**Python**、**NumPy**、**matplotlib**、**PyTables** 等），才能拥有上述几种功能。

但是，这么方便、通用的方法一般需要付出代价。举个例子，电子表格不适合于存储大量数据或者具有复杂关系的数据。这就是金融学中 Microsoft Excel 更多地仅作为*通用图形用户界面（GUI）*的原因。在许多情况下，Excel 主要用于显示和可视化数据，汇聚信息实施临时性的分析。例如，有许多接口可以从 Bloomberg 和 Thomson Reuters 等领先的数据服务提供商取得数据，导入 Excel（可能还有其他方法）。

本章的基础是 Microsoft **Excel** 在几乎所有桌面或者笔记本电脑上都存在且用作通用 GUI 的假设。在这个意义上，**Python** 可以起到如下的作用：

操纵工具

使用 **Python**，可以与 **Excel** 电子表格交互并加以操纵。

数据处理器

Python 可以向电子表格提供数据，从电子表格读取数据。

分析引擎

Python 可以向电子表格提供完整的分析能力，成为 VBA 编程的成熟替代品。

12.1 基本电子表格交互

处理 **Excel** 电子表格文件的 **Python** 基本库是 **xlrd** 和 **xlwt**（参见 http://www.python-excel.org）。尽管很流行，但是 **xlwt** 的主要缺点是只能写入与 Microsoft Excel 97/2000/xp/2003、OpenOffice.org Calc 和 Gnumeric 兼容的电子表格文件，也就是扩展名为**.xls** 的文件。因此，我们还使用 **xlsxwriter**（http://xlsxwriter.readthedocs.org）和 **OpenPyxl**（http://openpyxl.readthedocs.org）库，它们可以生成最新**.xslx** 格式的电子表格文件，首先导入：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [1]: import numpy as np
        import pandas as pd
        import xlrd, xlwt
        import xlsxwriter
        path = 'data/'
```

12.1.1 生成工作簿（.xls）

我们首先生成一个包含两个工作表的工作簿[1]。首先是*工作簿（Workbook）*对象 **wb**。注意，这只是工作簿的一个*内存中*版本（迄今为止）：

```
In [2]: wb = xlwt.Workbook()
In [3]: wb
Out[3]: <xlwt.Workbook.Workbook at 0x7f7dcc49df10>
```

第二步是在工作簿对象中加入一个或者多个工作表：

```
In [4]: wb.add_sheet('first_sheet', cell_overwrite_ok=True)
Out[4]: <xlwt.Worksheet.Worksheet at 0x7f7dac9dde90>
```

现在有一个工作表（**Worksheet**）对象，其索引号为 0：

```
In [5]: wb.get_active_sheet()
Out[5]: 0
```

再进一步，为工作表定义一个别名：

```
In [6]: ws_1 = wb.get_sheet(0)
        ws_1
Out[6]: <xlwt.Worksheet.Worksheet at 0x7f7dac9dde90>
```

当然，这两步（实例化和别名定义）可以合并为一步：

```
In [7]: ws_2 = wb.add_sheet('second_sheet')
```

两个工作表对象仍然为空。因此，我们生成一个包含数字的 **NumPy ndarray** 对象：

```
In [8]: data = np.arange(1, 65).reshape((8, 8))
In [9]: data
Out[9]: array([[ 1,  2,  3,  4,  5,  6,  7,  8],
               [ 9, 10, 11, 12, 13, 14, 15, 16],
               [17, 18, 19, 20, 21, 22, 23, 24],
               [25, 26, 27, 28, 29, 30, 31, 32],
               [33, 34, 35, 36, 37, 38, 39, 40],
               [41, 42, 43, 44, 45, 46, 47, 48],
               [49, 50, 51, 52, 53, 54, 55, 56],
               [57, 58, 59, 60, 61, 62, 63, 64]])
```

1 注意，从 Python 生成 Excel 电子表格的简单机制是以逗号分隔值（CSV）文件的形式导出数据，然后用 Excel 导入这些数据。这在有些时候比本文下面讨论的方法更高效。——原注。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

使用 **write** 方法并提供行/列信息（使用零基索引），很容易将数据写入某个工作表的某个单元格：

```
In [10]: ws_1.write(0, 0, 100)
           # write 100 in cell "A1"
```

这样，样本数据可以“批量”写入两个工作簿对象：

```
In [11]: for c in range(data.shape[0]):
             for r in range(data.shape[1]):
                 ws_1.write(r, c, data[c, r])
                 ws_2.write(r, c, data[r, c])
```

Workbook 类的 **save** 方法可以将整个工作簿对象保存到磁盘：

```
In [12]: wb.save(path + 'workbook.xls')
```

在 **Windows** 系统上，路径可能类似 **“C:\path\data\workbook.xls”**。

12.1.2 生成工作簿（.xslx）

（新格式电子表格的创建方法本质上相同）。首先，创建一个工作簿对象：

```
In [13]: wb = xlsxwriter.Workbook(path + 'workbook.xlsx')
```

第二步，创建工作表对象：

```
In [14]: ws_1 = wb.add_worksheet('first_sheet')
         ws_2 = wb.add_worksheet('second_sheet')
```

第三步，将数据写入工作表对象：

```
In [15]: for c in range(data.shape[0]):
             for r in range(data.shape[1]):
                 ws_1.write(r, c, data[c, r])
                 ws_2.write(r, c, data[r, c])
```

最后，关闭工作簿对象：

```
In [16]: wb.close()
In [17]: ll $path*
Out[17]: -rw--- 1 yhilpisch 7375 Sep 28 18:18 data/chart.xlsx
         -rw--- 1 yhilpisch 5632 Sep 28 18:18 data/workbook.xls
         -rw--- 1 yhilpisch 6049 Sep 28 18:18 data/workbook.xlsx
```

如果一切正常，在 Microsoft Exce 中打开的文件应该如图 12-1 所示。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

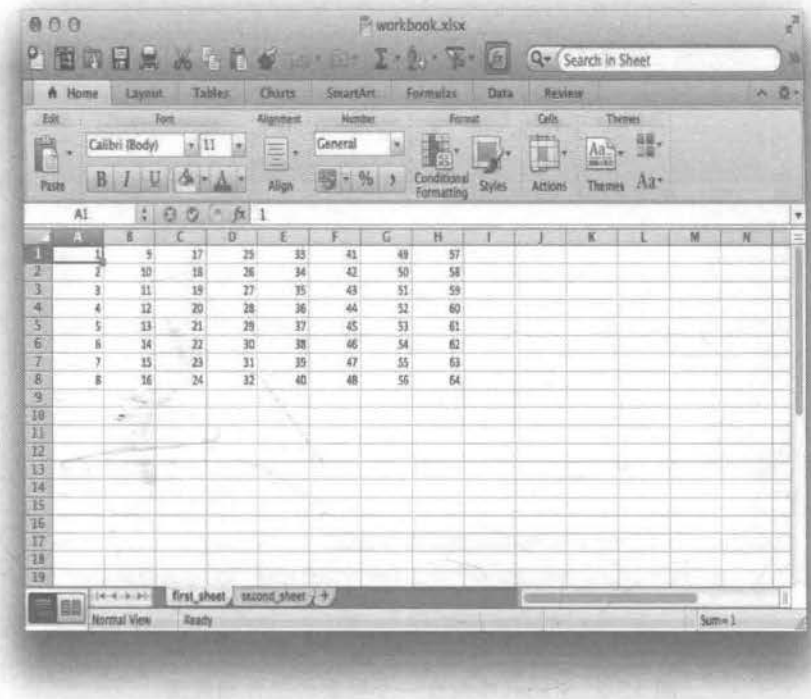


图 12-1　Excel 中的工作簿截图

xlsxwriter 还有许多生成工作簿对象的选项，例如图表。考虑如下的代码（参考 **xlsxwriter** 文档（http://bit.ly/xlsxwriter））：

```
In [18]: wb = xlsxwriter.Workbook(path + 'chart.xlsx')
         ws = wb.add_worksheet()

         # write cumsum of random values in first column
         values = np.random.standard_normal(15).cumsum()
         ws.write_column('A1', values)

         # create a new chart object
         chart = wb.add_chart({'type': 'line'})

         # add a series to the chart
         chart.add_series({'values': '=Sheet1!$A$1:$A$15',
                           'marker': {'type': 'diamond'},})
           # series with markers (here: diamond)

         # insert the chart
         ws.insert_chart('C1', chart)

         wb.close()
```

生成的电子表格文件屏幕截图如图 12-2 所示。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

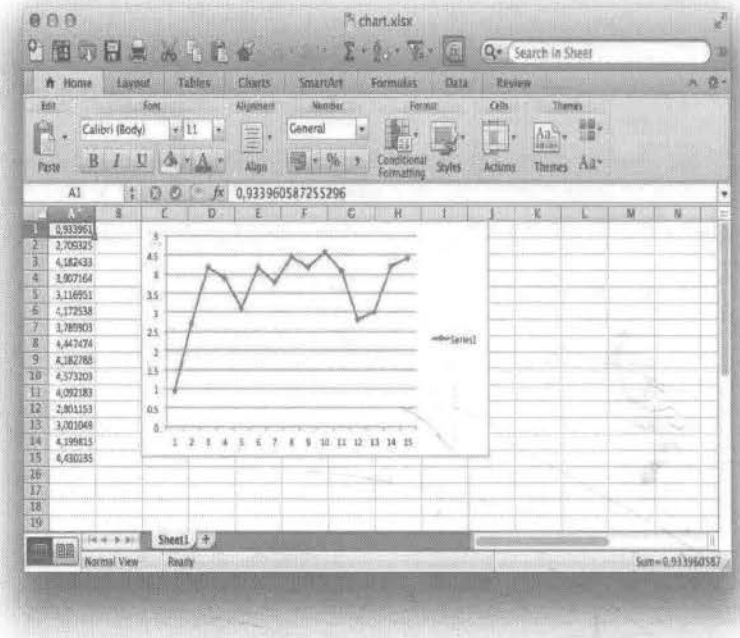


图 12-2　Excel 中包含一个图表的工作簿截图

12.1.3　从工作簿中读取

xlwt 的姊妹库 **xlrd** 负责从电子表格文件（工作簿）中读取数据：

```
In [19]: book = xlrd.open_workbook(path + 'workbook.xlsx')
In [20]: book
Out[20]: <xlrd.book.Book at 0x7f7dabec4890>
```

一旦打开工作簿，**sheet_names** 方法提供特定工作簿对象中的所有工作表对象名称：

```
In [21]: book.sheet_names()
Out[21]: [u'first_sheet', u'second_sheet']
```

工作表可以通过名称或者索引值访问：

```
In [22]: sheet_1 = book.sheet_by_name('first_sheet')
         sheet_2 = book.sheet_by_index(1)
         sheet_1
Out[22]: <xlrd.sheet.Sheet at 0x7f7dabec4a10>
In [23]: sheet_2.name
Out[23]: u'second_sheet'
```

工作表对象的重要属性是 **ncols** 和 **nrows**，分别表示包含数据的列数和行数：

```
In [24]: sheet_1.ncols, sheet_1.nrows
Out[24]: (8, 8)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

单独的单元格——即单元格（Cell）对象——通过 cell 方法访问，以行和列号作为参数（同样采用零基编号）。**value** 属性给出保存在特定单元格的数据：

```
In [25]: cl = sheet_1.cell(0, 0)
         cl.value
Out[25]: 1.0
```

ctype 属性给出单元格的类型：

```
In [26]: cl.ctype
Out[26]: 2
```

表 12-1 列出了所有 Excel 单元格类型。

表 12-1　Excel 单元格类型

类型	编号	Python 类型
XL_CELL_EMPTY	0	空字符串
XL_CELL_TEXT	1	Unicode 字符串
XL_CELL_NUMBER	2	float
XL_CELL_DATE	3	float
XL_CELL_BOOLEAN	4	int (1 = TRUE, 0 = FALSE)
XL_CELL_ERROR	5	int 表示 Excel 内部编码
XL_CELL_BLANK	6	空字符串，仅当 formatting_info=True

类似地，你可以为 **row** 方法提供行号，访问整行：

```
In [27]: sheet_2.row(3)
Out[27]: [number:25.0,
          number:26.0,
          number:27.0,
          number:28.0,
          number:29.0,
          number:30.0,
          number:31.0,
          number:32.0]
```

同理，可以访问整列：

```
In [28]: sheet_2.col(3)
Out[28]: [number:4.0,
          number:12.0,
          number:20.0,
          number:28.0,
          number:36.0,
          number:44.0,
          number:52.0,
          number:60.0]
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

row_values 和 **col_values** 只提供包含在对应行或者列的值：

```
In [29]: sheet_1.col_values(3, start_rowx=3, end_rowx=7)
Out[29]: [28.0, 29.0, 30.0, 31.0]
In [30]: sheet_1.row_values(3, start_colx=3, end_colx=7)
Out[30]: [28.0, 36.0, 44.0, 52.0]
```

要读出工作簿对象中的所有数据，只要在所有包含数据的行和列上循环：

```
In [31]: for c in range(sheet_1.ncols):
             for r in range(sheet_1.nrows):
                 print '%i' % sheet_1.cell(r, c).value,
             print
Out[31]: 1 2 3 4 5 6 7 8
         9 10 11 12 13 14 15 16
         17 18 19 20 21 22 23 24
         25 26 27 28 29 30 31 32
         33 34 35 36 37 38 39 40
         41 42 43 44 45 46 47 48
         49 50 51 52 53 54 55 56
         57 58 59 60 61 62 63 64
```

12.1.4 使用 OpenPyxl

还有另一个库可以在 **Python** 中生成和读取 Excel 电子表格文件：**OpenPyxl**（http://openpyxl.readthedocs.org）。这个库可以创建电子表格文件，也可以读取它们。此外，虽然基本用法和其他库类似，但是接口更像 **Python** 风格，因此值得一看。导入如下库：

```
In [32]: import openpyxl as oxl
```

和以前一样，首先生成一个工作簿对象：

```
In [33]: wb = oxl.Workbook()
```

第二步，创建一个工作表对象：

```
In [34]: ws = wb.create_sheet(index=0, title='oxl_sheet')
```

第三步，将数据写入工作表：

```
In [35]: for c in range(data.shape[0]):
             for r in range(data.shape[1]):
                 ws.cell(row=r, column=c).value = data[c, r]
                 # creates a Cell object and assigns a value
```

第四步，关闭文件对象：

```
In [36]: wb.save(path + 'oxl_book.xlsx')
```

利用 OpenPyxl 还可以读取工作簿：

```
In [37]: wb = oxl.load_workbook(path + 'oxl_book.xlsx')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

现在，单独的单元格可以通过名称轻松地访问：

```
In [38]: ws = wb.get_active_sheet()
In [39]: cell = ws['B4']
In [40]: cell.column
Out[40]: 'B'
In [41]: cell.row
Out[41]: 4
In [42]: cell.value
Out[42]: 12
```

可以像 Excel 中一样访问单元格区域：

```
In [43]: ws['B1':'B4']
Out[43]: ((<Cell oxl_sheet.B1>,),
          (<Cell oxl_sheet.B2>,),
          (<Cell oxl_sheet.B3>,),
          (<Cell oxl_sheet.B4>,))
In [44]: for cell in ws['B1':'B4']:
             print cell[0].value
Out[44]: 9
         10
         11
         12
```

还有一个 range 方法，可以字符串方式提供 Excel 语法的单元格区域：

```
In [45]: ws.range('B1:C4')
           # same as ws['B1':'C4']
Out[45]: ((<Cell oxl_sheet.B1>, <Cell oxl_sheet.C1>),
          (<Cell oxl_sheet.B2>, <Cell oxl_sheet.C2>),
          (<Cell oxl_sheet.B3>, <Cell oxl_sheet.C3>),
          (<Cell oxl_sheet.B4>, <Cell oxl_sheet.C4>))
In [46]: for row in ws.range('B1:C4'):
             for cell in row:
                 print cell.value,
             print
Out[46]: 9 17
         10 18
         11 19
         12 20
```

更多细节参考该库的网站（http://pythonhosted.org/openpyxl/）。

12.1.5 使用 pandas 读写

第 7 章展示了用 **pandas** 库与 **Excel** 电子表格文件交互的方法，我们用这些方法读取 **xlwt** 库写入的数据，这需要为每个工作表建立一个 **DataFrame** 对象。设置参数 **header=None**，**pandas** 就不会将第一行数据解释为数据集的表头：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [47]: df_1 = pd.read_excel(path + 'workbook.xlsx',
                              'first_sheet', header=None)
         df_2 = pd.read_excel(path + 'workbook.xlsx',
                              'second_sheet', header=None)
```

为了恢复电子表格文件的列名/值，我们为 **DataFrame** 对象生成一个列表，以大写字母作为列名。

```
In [48]: import string
         columns = []
         for c in range(data.shape[0]):
             columns.append(string.uppercase[c])
         columns
Out[48]: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
```

将列表作为新列的名称传递给两个对象：

```
In [49]: df_1.columns = columns
         df_2.columns = columns
```

确实，两个 **DataFrame** 的输出现在都和电子表格的风格很相近：

```
In [50]: df_1
Out[50]:
    A   B   C   D   E   F   G   H
0   1   9  17  25  33  41  49  57
1   2  10  18  26  34  42  50  58
2   3  11  19  27  35  43  51  59
3   4  12  20  28  36  44  52  60
4   5  13  21  29  37  45  53  61
5   6  14  22  30  38  46  54  62
6   7  15  23  31  39  47  55  63
7   8  16  24  32  40  48  56  64

In [51]: df_2
Out[51]:
    A   B   C   D   E   F   G   H
0   1   2   3   4   5   6   7   8
1   9  10  11  12  13  14  15  16
2  17  18  19  20  21  22  23  24
3  25  26  27  28  29  30  31  32
4  33  34  35  36  37  38  39  40
5  41  42  43  44  45  46  47  48
6  49  50  51  52  53  54  55  56
7  57  58  59  60  61  62  63  64
```

Similarly, pandas allows us to write the data to Excel spreadsheet files:

```
In [52]: df_1.to_excel(path + 'new_book_1.xlsx', 'my_sheet')
```

注意，在向电子表格文件写入 **DataFrame** 对象时，**pandas** 加入列名和索引值，如图 12-3 所示。

当然，**pandas** 写入的 Excel 工作簿可以像前面一样，供 **xlrd** 库读取：

```
In [53]: wbn = xlrd.open_workbook(path + 'new_book_1.xlsx')
In [54]: wbn.sheet_names()
Out[54]: [u'my_sheet']
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

要将多个 **DataFrame** 对象写入单个电子表格文件，需要一个 **ExcelWriter** 对象：

```
In [55]: wbw = pd.ExcelWriter(path + 'new_book_2.xlsx')
         df_1.to_excel(wbw, 'first_sheet')
         df_2.to_excel(wbw, 'second_sheet')
         wbw.save()
```

我们来检查一下，是否已经在单一电子表格文件中生成了两个工作表：

```
In [56]: wbn = xlrd.open_workbook(path + 'new_book_2.xlsx')
In [57]: wbn.sheet_names()
Out[57]: [u'first_sheet', u'second_sheet']
```

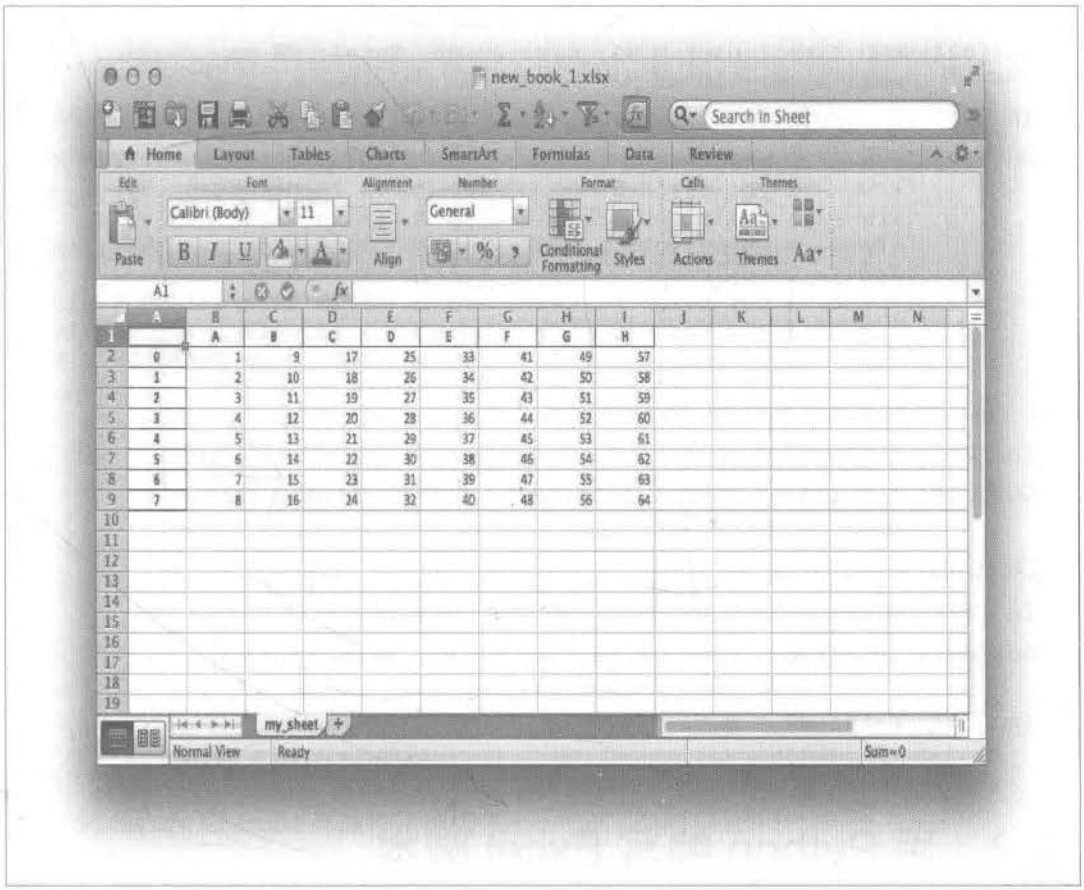


图 12-3　用 pandas 写入的 Excel 工作簿截图

我们所要考虑的最后一个 **pandas** 和 **Excel** 用例是更大量数据的读写。尽管这不是一种快速的操作，但是在某些情况下可能有用。首先，使用样本数据：

```
In [58]: data = np.random.rand(20, 100000)
In [59]: data.nbytes
Out[59]: 16000000
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

其次，由样本数据生成一个 **DataFrame** 对象：

```
In [60]: df = pd.DataFrame(data)
```

第三步，将其作为一个 Excel 文件写入磁盘：

```
In [61]: %time df.to_excel(path + 'data.xlsx', 'data_sheet')
Out[61]: CPU times: user 1min 25s, sys: 460 ms, total: 1min 26s
         Wall time: 1min 25s
```

这需要花费一些时间。为了对比，看看 **NumPy ndarray** 对象原生存储有多快（在 SSD 驱动器上）：

```
In [62]: %time np.save(path + 'data', data)
Out[62]: CPU times: user 8 ms, sys: 20 ms, total: 28 ms
         Wall time: 159 ms
In [63]: ll $path*
Out[63]: -rw--- 1 yhilpisch     7372 Sep 28 18:18 data/chart.xlsx
         -rw--- 1 yhilpisch 16000080 Sep 28 18:20 data/data.npy
         -rw--- 1 yhilpisch  3948600 Sep 28 18:20 data/data.xlsx
         -rw--- 1 yhilpisch     5828 Sep 28 18:18 data/new_book_1.xlsx
         -rw--- 1 yhilpisch     6688 Sep 28 18:18 data/new_book_2.xlsx
         -rw--- 1 yhilpisch     6079 Sep 28 18:18 data/oxl_book.xlsx
         -rw--- 1 yhilpisch     5632 Sep 28 18:18 data/workbook.xls
         -rw--- 1 yhilpisch     6049 Sep 28 18:18 data/workbook.xlsx
```

最后一步，从磁盘上读取数据，这明显比写入快：

```
In [64]: %time df = pd.read_excel(path + 'data.xlsx', 'data_sheet')
Out[64]: CPU times: user 6.53 s, sys: 44 ms, total: 6.58 s
         Wall time: 6.51 s
```

再次对比和原生存储的速度差异：

```
In [65]: %time data = np.load(path + 'data.npy')
Out[65]: CPU times: user 16 ms, sys: 8 ms, total: 24 ms
         Wall time: 40.5 ms
In [66]: data, df = 0.0, 0.0
         !rm $path*
```

12.2 用 Python 编写 Excel 脚本

前一小节说明了生成、读取和操纵 Excel 电子表格文件（即工作簿）的方法。尽管有一些很有益的用例，但是 **Python** 不是实现前述结果的唯一方法，有时候也不是最好的方法。

更有趣的方法是向 **Excel** 电子表格输出 **Python** 的分析能力。但是，这是技术要求更高的工作。例如，**Python** 库 **PyXLL**（http://www.pyxll.com）提供通过 **Excel** 加载项（**add-in**，Microsoft 增强 **Excel** 功能性的技术）输出 Python 的手段。此外，**DataNitro** 公司提供

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

一个解决方案，可以将 **Python** 和 **Excel** 完全集成，使 **Python** 成为 VBA 编程的替代品。不过，两种解决方案都是需要许可证的商用产品。

下面，我们概述使用 **DataNitro** 编写 **Excel** 脚本的方法，因为这是相当灵活的 **Python** 和 **Excel** 集成方法。

12.2.1 安装 DataNitro

DataNitro 只能工作于 Windows 操作系统和 **Excel** 安装之上。在 **Mac** OS 系统上，可以运行于 Windows 虚拟机环境中。它与 Office2007 和更高版本兼容。获得该解决方案（试用）许可证及安装的指南参见网站 http://www.datanitro.com。

安装 **DataNitro** 时有一个同时安装 **Python** 的选项。然而，如果已经安装了 **Python**，例如 **Anaconda**（参见第 2 章），就没有必要安装另一个 **Python** 版本。你只需要自定义 **DataNitro** 解决方案（通过 **Settings** 菜单）使用现有 **Anaconda** 安装。**DataNitro** 可以使用 2.6 和更高版本的 **Python**，也可以使用版本 3.x。

成功安装之后，可以在 Excel 中找到 DataNitro 功能区，如图 12-4 所示。

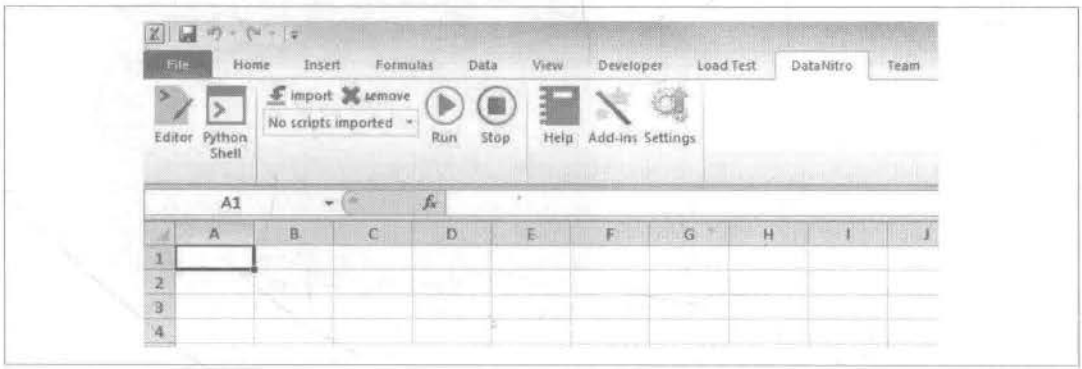


图 12-4 带有 DataNitro 功能区的 Excel 截图

12.2.2 使用 DataNitro

组合 **Datanitro** 和 **Excel** 有两种主要方法：

脚本

利用这种方法，可以通过 **Python** 脚本控制 **Excel** 电子表格，类似于前一小节介绍的方法。

用户定义函数

使用这种方法，可以将自己的 **Python** 函数输出到 **Excel**，从 **Excel** 中调用。

两种方法都需要安装 **DataNitro** 解决方案——也就是说，不能将已经制作好的功能分发

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

给没有安装 **DataNitro** 解决方案的人。

用 DataNitro 编写脚本

在 **Excel** 中打开前面生成的电子表格文件 **workbook.xlsx**。我们打算用 **DataNitro** 处理这个特殊文件。单击 **DataNitro** 功能区中的 **Python Shell** 符号，将会看到如图 12-5 的屏幕。

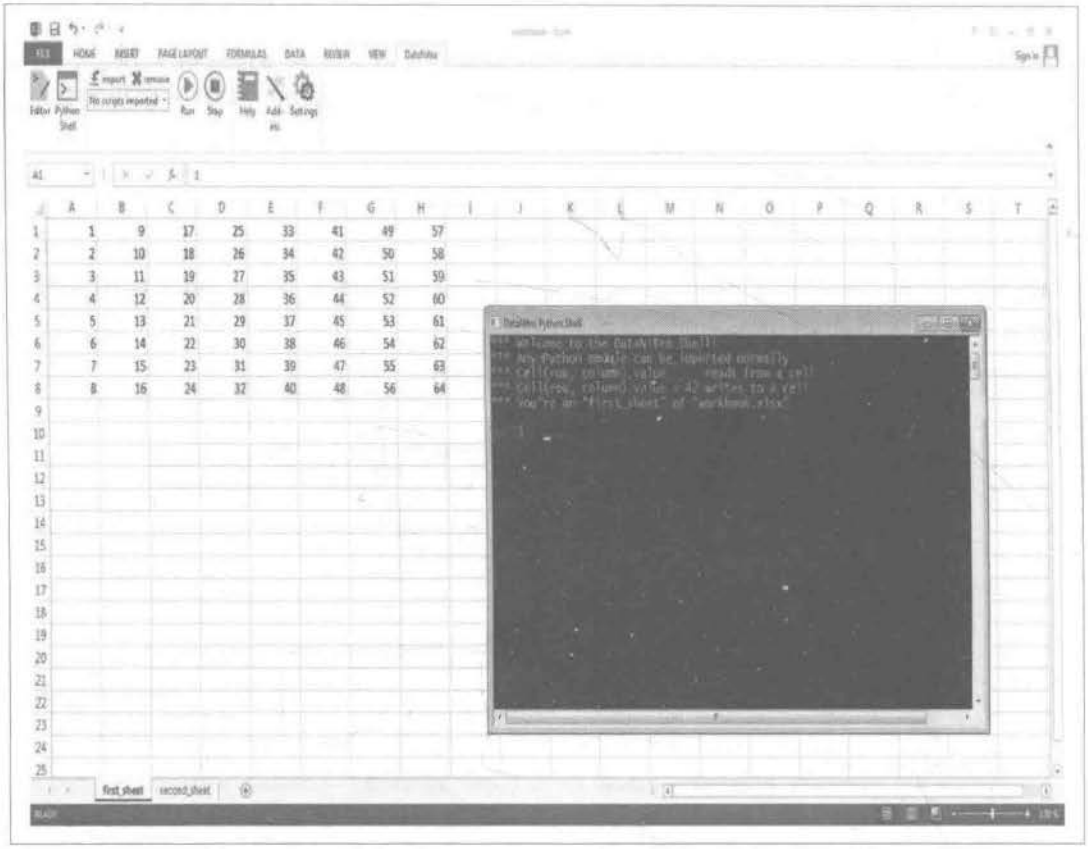


图 12-5　在 Excel 中使用 DataNitro IPython Shell 的截图

然后，可以进行如下的简单会话：

```
In [1]: Cell("B1")
Out[1]: B1

In [2]: Cell("B1").value
Out[2]: 9

In [3]: Cell("B1").value = 'Excel with Python'
  # this immediately changes the (displayed) value
  # in the spreadsheet

In [4]: Cell("B1").value
Out[4]: u'Excel with Python'
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

正如更改单元格对象中的 **value** 属性一样，可以为其指定一个公式：

```
In [5]: Cell("A9").formula = '=Sum(A1:A8)'

In [6]: Cell("A9").value
Out[6]: 36
```

表 12-2 列出了 DataNitro Cell 对象的属性。

表 12-2　DataNitro Cell 属性

属性	描述
row	单元格所在行
col	单元格所在列
position	（row，col）形式的位置
sheet	单元格所在工作表名称
name	Excel 风格的单元格名称
value	单元格值
vertical	单元格和所有下方单元格的值
vertical_range	单元格和所有下方单元格组成的 Excel 区域
horizontal	单元格和所有右方单元格的值
horizontal_range	单元格和所有右方单元格组成的 Excel 区域
table	单元格和所有右下方单元格的值，形式为嵌套的列表对象
table_range	table 对象的 Excel 区域
formula	Excel 公式
comment	单元格附加的注释
hyperlink	字符串对象形式的超链接或者电子邮件地址
alighment	文本/值显示时的对齐方式
color	单元格颜色
df	直接向电子表格写入 pandas DataFrame 对象

表 12-3 展示了 **Cell** 对象的字体选项。所有选项都是 **font** 对象的属性，而该对象是 **Cell** 对象的属性之一。例如：

```
Cell("A1").font.size = 15
```

设置（新）字体大小。

最后，还有几个用于 **Cell** 对象的方法，在表 12-4 中列出。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

表 12-3　DataNitro Cell 字体选项

属性	描述
size	字体大小
color	字体颜色
bold	通过 Cell（“A1”）.font.bold=True 加粗字体
italic	斜体字体
underline	带下划线的字体
strikethrough	加删除线的字体
subscript	下标文本
superscript	上标文本

表 12-4　DataNitro Cell 方法

属性	描述
clear	重置单元格的所有属性
copy_from	从另一个单元格复制所有属性
copy_format_from	从另一个单元格复制除了 value 和 formula 之外的所有属性
is_empty	如果单元格为空返回 True
offset	返回指定相对偏移（以(row,col)元组表示）的单元格对象
subtraction	相减得出偏移量；例如 Cell(“B4”) –Cell(“A2) 得出（2，1）
print	给出单元格名称和所在工作表
set_name	设置 Excel 中的命名区域；例如 Cell(“A1”).set_name(“upper_left”)

使用 **CellRange** 代替 **Cell** 对象往往很有益，可以将其看作多个 **Cell** 对象上的向量化操作方法。考虑如下的示例，仍然基于我们前面更改过的同一个电子表格文件 **workbook.xlsx**：

```
In [6]: CellRange("A1:A8").value
Out[6]: [1, 2, 3, 4, 5, 6, 7, 8]

In [7]: CellRange("A1:A8").value = 1
  # like broadcasting

In [8]: CellRange("A1:A8").value
Out[8]: [1, 1, 1, 1, 1, 1, 1, 1]

In [9]: CellRange("A1:A8").value = 2 * [1, 2, 3, 4]

In [10]: CellRange("A1:A8").value
Out[10]: [1, 2, 3, 4, 1, 2, 3, 4]

In [11]: Cell("A9").value
Out[11]: 20
  # value of Sum function is
  # automatically updated
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

当然，你也可以使用 **CellRange** 循环：

```
In [12]: for cell in CellRange("A1:B2"):
   ....:     print cell.name, cell.value
   ....:
A1 1
B1 Python with Excel
A2 2
B2 10
```

大部分 **Cell** 属性和方法也可以用于 **CellRange**。

在编写复杂的 **Python** 脚本与 **Excel** 电子表格交互时，*性能*可能成为问题。本质上，性能受限于 **Excel** 输入/输出（I/O）速度。应该尽可能遵循如下的规则：

读/写

不要交替进行读写操作，因为这可能明显降低性能。

向量化

使用 **CellRange** 对象或者 **Cell().table** 对象读写（大）块数据，代替循环方法[1]。

使用 Python

例如，在必须转换一个数据块时，用 **Python** 将其全部读出、操纵并以块形式写回电子表格，是更好的做法；逐个单元格操作实际上很慢。

将数据存储到 Python

尽可能在 **Python** 中存储数值而不是重新读取它们，特别是在性能关键循环或者类似操作中。

使用整个工作表（**Worksheet**）（http://datanitro.com/docs/sheets.html）和**工作簿**（**Workbook**）（http://datanitro.com/docs/workbook.html）对象的细节参见 DataNitro 文档的相应章节。

用 DataNitro 绘制图表

用 **DataNitro** 编写 **Excel** 脚本时的一个特殊主题是用 **Python** 代替 **Excel** 的绘图功能，绘制包含在电子表格中数据的图表。例 12-1 展示了一个 **Python** 脚本，只有在安装了 **DataNitro** 时才能执行。该脚本用 **pandas** 的 **DataReader** 函数（参见第 6 章）读取 **Apple** 公司股价数据，将数据写入新生成的工作簿对象中，然后用 Python 根据保存在对应工作表对象中的数据绘制图表——在 **DataNitro** 的 **maplotlib.pyplot** 包装器 **nitroplot** 帮助下——并将结果输出到电子表格。

例 12-1　用 DataNitro 绘制保存在电子表格中的数据图表，并在同一个电子表格中显

1　参考第 4 章中关于 NumPy ndarray 对象的类似讨论以及向量化的好处。第 4 章和本章中的经验法则都是避免 Python 级别上的循环。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

示 matplotlib 图表

```
#
# Plotting with DataNitro in Excel
# dn_plotting.py
#
import pandas.io.data as web
import nitroplot as nplt
  # wrapper for matplotlib.pyplot (plt)

# make a new workbook

wb = new_wkbk()
active_wkbk(wb)
rename_sheet("Sheet1", "Apple_Stock")

# read Apple Inc. stock data

aapl = web.DataReader('aapl', data_source='yahoo')[['Open', 'Close']]

# write the data to the new workbook

Cell("A1").df = aapl

# generate matplotlib plot
nplt.figure(figsize=(8, 4))
nplt.plot(Cell("A2").vertical, Cell("C2").vertical, label='AAPL')
nplt.legend(loc=0)
nplt.grid(True)
nplt.xticks(rotation=35)

# expose plot to Excel spreadsheet

nplt.graph()
  # as plt.show()

# save the new workbook with data and plot

save('dn_plot.xlsx')
```

从 **DataNitro IPython Shell** 中执行该脚本：

```
In [1]: %run dn_plotting.py
```

如果脚本成功执行，**Excel** 中的工作簿/工作表如图 12-6 所示。

用户定义函数

从金融的角度看，最有趣的似乎是通过 **DataNitro** 向 **Excel** 输出用户定义函数（UDF）。这一选项必须在 **DataNitro** 的 **Settings** 菜单中启用。启用这一功能之后，可以用 **DataNitro** 导入 **Python** 脚本 functions.py。包含在这个特殊文件中的所有 **Python** 函数将可以直接从 **Excel** 中调用，也只有包含在该文件中的函数才能被调用。考虑例 12-2 中的函数，

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

这个函数是现在已经很熟悉的 Black-Scholes-Merton 模型欧式看涨期权估值。

图 12-6　Excel 中 Apple 股价数据的 DataNitro 图表截图

例 12-2　用 DataNitro 导入 Excel 的 Python 脚本

```
#
# Valuation of European call options in BSM model
# for use with DataNitro and Excel spreadsheets
# functions.py
#

# analytical Black-Scholes-Merton (BSM) formula


def bsm_call_value(S0, K, T, r, sigma):
    ''' Valuation of European call option in BSM model.
    Analytical formula.

    Parameters
    ==========
    S0 : float
        initial stock/index level
    K : float
        strike price
    T : float
        time-to-maturity (for t=0)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    r : float
        constant risk-free short rate
    sigma : float
        volatility factor in diffusion term

    Returns
    =======
    value : float
        present value of the European call option
    '''
    from math import log, sqrt, exp
    from scipy import stats
    S0 = float(S0)
    d1 = (log(S0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * sqrt(T))
    d2 = (log(S0 / K) + (r - 0.5 * sigma ** 2) * T) / (sigma * sqrt(T))
    value = (S0 * stats.norm.cdf(d1, 0.0, 1.0)
            - K * exp(-r * T) * stats.norm.cdf(d2, 0.0, 1.0))
    return value
```

如果这个脚本通过 **DataNitro** 导入且启用 UDF，就可以从 **Excel** 中使用估值公式。在实践中，可以在 **Excel** 单元格中输入如下内容：

```
= bsm_call_value(B1, 100, 2.0, B4, B5)
```

这和任何其他 Excel 公式相同。例如，观察图 12-7，在左上角可以看到 Black-Scholes-Merton 模型的参数。在 Excel 的“公式”（FORMULAS）选项卡中单击“插入函数”

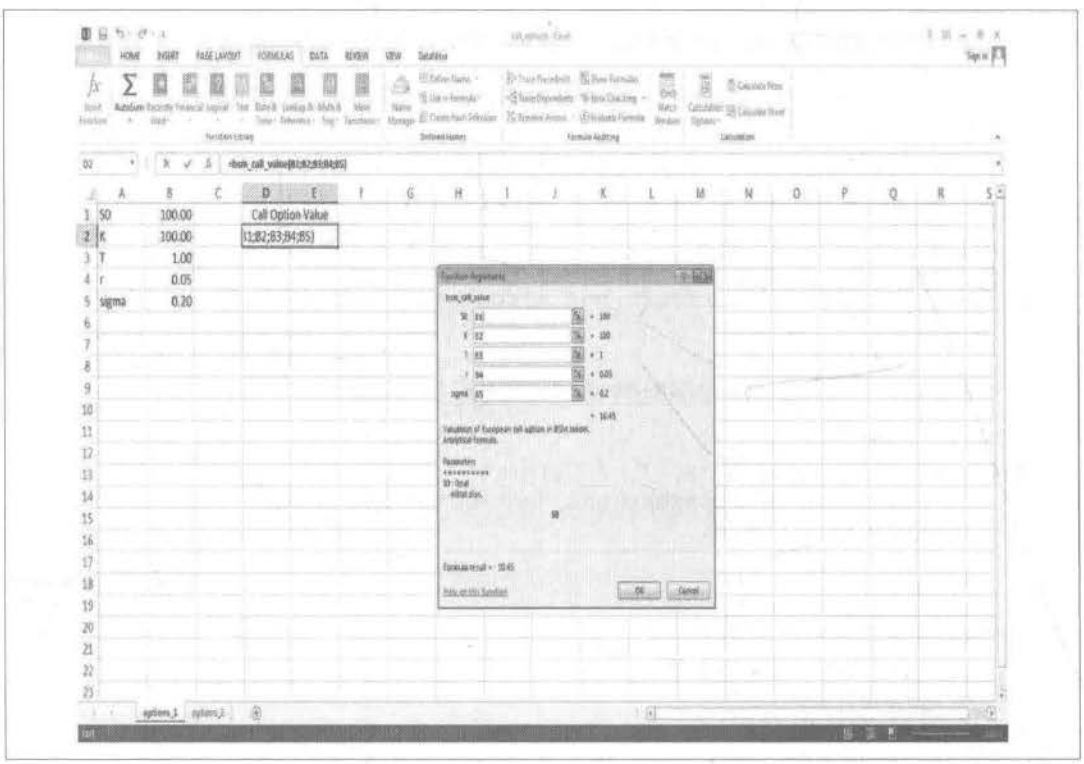


图 12-7　用于 Python 函数的 Excel 函数对话框截图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

（**Insert Function**），可以在函数对话框中输入来自 **Python** 的期权估值公式（在 **DataNitro functions** 下查找）。一旦提供了对包含单独参数值的单元格的引用，**Python** 就会计算期权价值并返回结果给 **Excel**。

尽管这个函数的计算要求不高，但是说明了如何在 **Excel** 中利用 **Python** 的分析能力，以及如何将结果直接输出到 **Excel**（单元格）。类似地，观察图 12-8，图中我们使用一个参数网格，一次性计算多个期权价值。D11 单元格的公式采用如下形式：

```
= bsm_call_value($B$1, $A11, D$8, $B$4, $B$5)
```

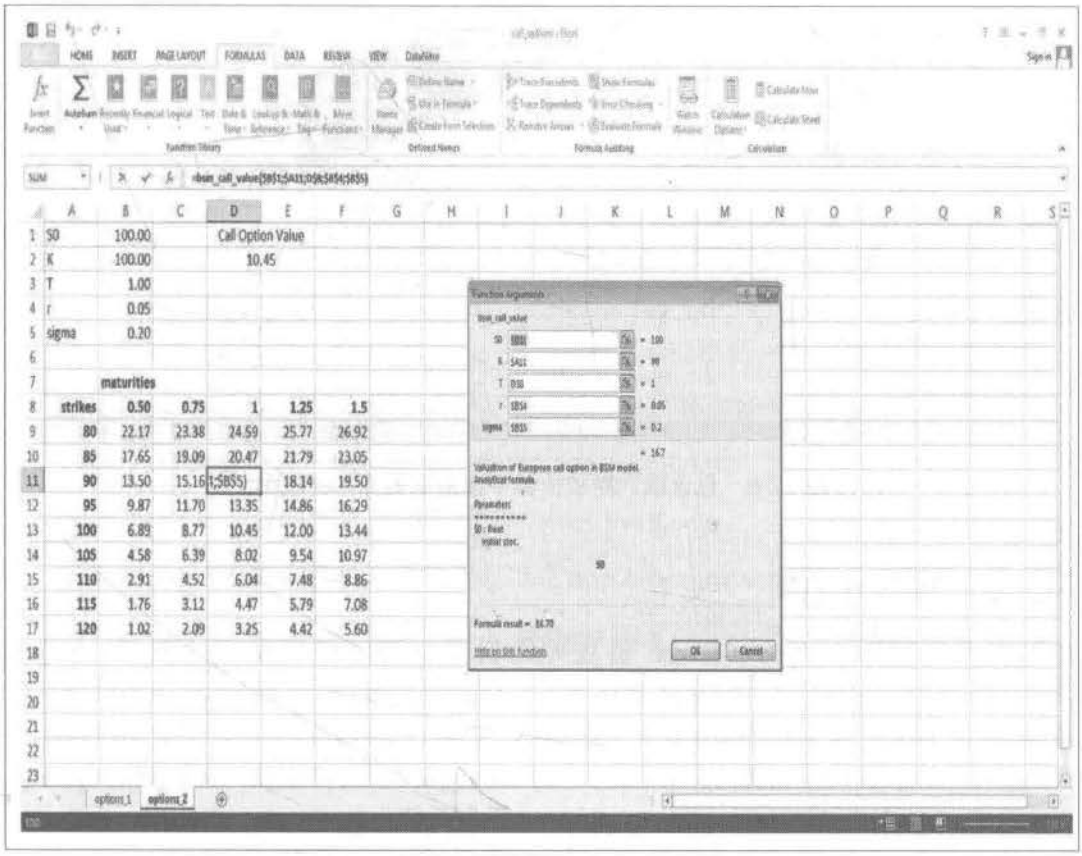


图 12-8　包含欧式期权股指参数网格的 Excel 截图

前一个例子我们绘制了包含在 **Excel** 电子表格中的数据图表，现在也要在电子表格中为 **Python** 生成的数据绘制表格。图 12-9 展示了用 **Excel** 为欧式期权价值平面绘制的 3D 图表。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

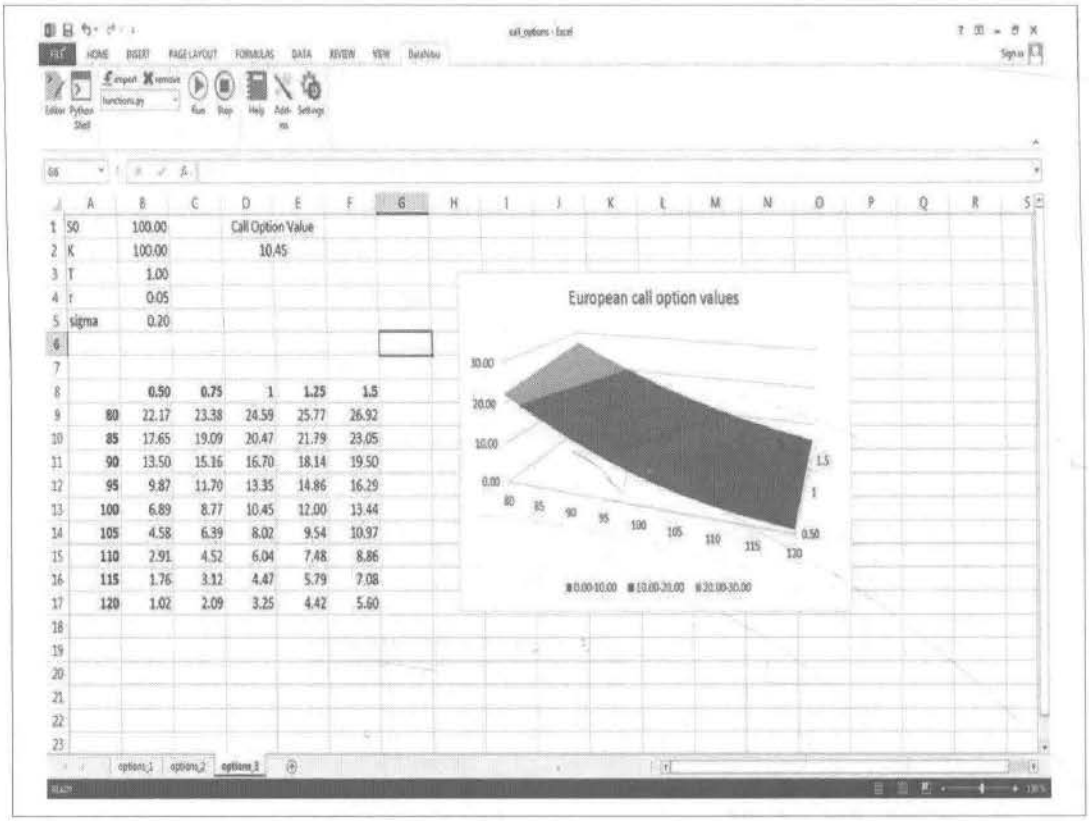


图 12-9　包含欧式期权价值平面图表的 Excel 截图

12.3　xlwings

在本书编写期间，**Python-Excel** 集成界的新竞争者已经出现，这就是 **xlwings**（http://www.xlwings.org）。**xlwings** 提供了几乎所有用 **Python** 与 **Excel** 电子表格交互和编写脚本的功能。和 **DataNitro** 解决方案相比，它是一个开源库，可以免费随任何电子表格发送。**xlwings** 的接收者只需要（最小）**Python** 安装就可以“驱动”电子表格。**xlwings** 的优势之一是可以和 Windows 和 Appple/Mac 操作系统上的 **Excel** 一起使用。此外，尽管还处于早期发行阶段（本书编写时是 0.3 版本），但它有很好的文档。这个整体解决方案看起来很有前景，任何对 **Python** 与 **Excel** 集成感兴趣的人都应该试一试。

12.4　结语

Python 与 **Excel** 集成有多种选项。有些 **Python** 库——如 **xlwt** 或者 **xlsxwriter** 可以创建 **Excel** 电子表格。其他库如 **xlrd** 则可以读取任何电子表格文件，或者既可以读取、也可以写入电子表格文件。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

pandas 至少在某些任务上也有帮助。例如，在电子表格文件中写入较大的数据集，或者读取保存为这种文件格式的数据时。

但是，在本书编著期间，最强大的解决方案是 **DataNitro** 提出的，它可以紧密地集成这两个世界，其电子表格操纵能力与其他库类似甚至更好。此外，**DataNitro** 还有一些其他功能，例如向 **Excel** 电子表格输出 **Python** 图表。更重要的是，可以定义用户定义的 **Python** 函数（UDF），以与 **Excel** 内建函数相同的方式调用。新的开源库 **xlwings** 最近已经出现，在范围和能力上与 **DataNitro** 解决方案类似。

特别是，**DataNitro** 和 **xlwings** 方法使我们可以将 **Excel** 当作一个灵活和强大的通用 GUI——金融行业的几乎所有电脑都安装了它——并将其与 **Python** 的分析能力相结合，真可谓两全其美。

12.5 延伸阅读

对于本章中介绍的所有库和解决方案，下面是一些有益的 Web 资源：

- 对于 **xlrd** 和 **xlwt**，可以参见 http://www.python-excel.org 的在线文档；在 http://www.simplistix.co.uk/presentations/python-excel.pdf 还可以找到一个 PDF 格式的教程；
- **xlsxwriter** 在网站 http://xlswriter.readthedocs.org 上有很好的文档。
- **OpenPyxl** 的首页在：http://pythonhosted.org/oepnpyxl/。
- 关于 **PyXLL** 的详细信息，参见 https://www.pyxll.com。
- **DataNitro** 的免费试用和详细文档可以在 http://www.datanitro.com 上找到。
- 可以在 http://xlwings.org 上找到有关 **xlwings** 的文档和其他各种资源。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 13 章

面向对象和图形用户界面

首先解决问题，然后编写代码。

——Jon Johnson

面向对象技术既有爱好者，又有批评者。按照上面引用的格言，面向对象实现风格在由真正理解手上问题的程序员应用以及从抽象和概括中所得甚多的时候最有益处。反之，如果你不能准确地知道要做什么，更交互式和探索性的编程风格（如过程式编程）可能是更好的选择。

在本章中，我们不打算讨论使用面向对象方法的风险和优点。我们认为，这种方法在开发较为复杂的金融应用（参见本书第 3 部分中实现的项目）时有其用武之地，在这些案例中它可以带来许多重大的好处。在构建图形用户界面（GUI）时，面向对象方法通常是必不可少的。

因此，我们在本章中结合这两个主题，首先介绍 **Python** 类和对象的基本概念。有了这些知识之后，介绍 GUI 的开发就容易得多了。

13.1 面向对象

维基百科（http://bit.ly/oo_programming）为*面向对象编程*提供了如下定义：

> 面向对象编程（OOP）是一种编程范型，将概念表现为具有数据字段（描述对象属性）和相关过程（称为方法）的“对象”，对象通常是类的实例，通过类的相互作用设计应用程序和计算机软件。

上述定义已经提供了 **Python** 世界中用于类和对象的主要技术术语，在本节剩下的部分

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

将进一步澄清这些概念。

13.1.1 Python 类基础知识

我们首先定义一个新（对象）类。为此，使用 **class** 语句，该语句的应用方法类似于函数定义中的 **def** 语句。下面的代码定义名为 **ExampleOne** 的新 **Python** 类。该类只是“存在”而没有任何作用。**pass** 命令的作用正如其名——它只是“通过”，不做任何事情：

```
In [1]: class ExampleOne(object):
            pass
```

不过，**ExmpleOne** 类的存在使我们可以生成该类的实例，作为 **Python** 的一个新对象：

```
In [2]: c = ExampleOne()
```

此外，由于该类从通用的 **object** 类继承而来，它已经包含了一些内容。例如，下面的代码提供根据我们的类新生成的对象的字符串表现形式：

```
In [3]: c.__str__()
Out[3]: '<__main__.ExampleOne object at 0x7f8fcc28ef10>'
```

我们还可以使用 **type** 了解对象的类型——在本例中，就是类 **ExampleOne** 的一个实例：

```
In [4]: type(c)
Out[4]: __main__.ExampleOne
```

现在我们定义一个具有两个属性（如 a 和 b）的类。为此，我们定义一个特殊的方法 **init**，在每次实例化类时自动调用。注意，该对象本身也是这个函数的一个参数（通过 **Python** 约定关键字 **self**）：

```
In [5]: class ExampleTwo(object):
            def __init__(self, a, b):
                self.a = a
                self.b = b
```

现在，新类 **ExampleTwo** 的实例化取两个数值，第一个用于属性 **a**，第二个用于属性 **b**。注意，在前面的定义中，这些属性在内部（即类定义中）分别通过 **self.a** 和 **self.b** 引用：

```
In [6]: c = ExampleTwo(1, 'text')
```

类似地，我们可以访问对象 c 的属性值：

```
In [7]: c.a
Out[7]: 1
In [8]: c.b
Out[8]: 'text'
```

我们也可以简单地为属性赋新值，覆盖初始值：

```
In [9]: c.a = 100
In [10]: c.a
Out[10]: 100
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

Python 在类和对象的使用上颇为灵活。例如，对象的属性甚至可以在实例化之后定义，如下例：

```
In [11]: c = ExampleOne()
In [12]: c.first_name = 'Jason'
         c.last_name = 'Bourne'
         c.movies = 4
In [13]: print c.first_name, c.last_name, c.movies
Out[13]: Jason Bourne 4
```

下面的类定义引入类的方法。在本例中，除了自定义方法 **addition** 的定义之外，**ExampleThree** 和 **ExampleTwo** 相同：

```
In [14]: class ExampleThree(object):
             def __init__(self, a, b):
                 self.a = a
                 self.b = b
             def addition(self):
                 return self.a + self.b
```

实例化和以前一样，这次我们只使用属性 **a** 和 **b** 的整数值：

```
In [15]: c = ExampleThree(10, 15)
```

调用方法 **addition** 返回两个属性值的总和（只要属性的类型定义合适）：

```
In [16]: c.addition()
Out[16]: 25
In [17]: c.a += 10
         c.addition()
Out[17]: 35
```

面向对象编程的优势之一是可重用性。前面已经指出，**ExampleTwo** 和 **ExampleThree** 类定义的唯一不同之处是自定义方法定义。因此，定义 **ExampleThree** 类的另一种方式是使用 **ExampleTwo** 类并继承特殊函数 **init** 的定义：

```
In [18]: class ExampleFour(ExampleTwo):
             def addition(self):
                 return self.a + self.b
```

The behavior of instances of class ExampleFour is now exactly the same as that of instances of class ExampleThree:

```
In [19]: c = ExampleFour(10, 15)
In [20]: c.addition()
Out[20]: 25
```

Python 允许多重继承。然而，应该小心处理可读性和可维护性，特别是供其他人使用时：

```
In [21]: class ExampleFive(ExampleFour):
             def multiplication(self):
                 return self.a * self.b
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [22]: c = ExampleFive(10, 15)
In [23]: c.addition()
Out[23]: 25
In [24]: c.multiplication()
Out[24]: 150
```

例如，自定义方法定义不一定要包含在类定义中。它们可以放在其他位置，只要它们处于全局命名空间，就可以在类定义中使用。下面的代码说明了这种方法：

```
In [25]: def multiplication(self):
                return self.a * self.b
In [26]: class ExampleSix(ExampleFour):
             multiplication = multiplication
```

ExampleSix 类实例的表现也和前面的 **ExampleFive** 类的实例完全相同：

```
In [27]: c = ExampleSix(10, 15)
In [28]: c.addition()
Out[28]: 25
In [29]: c.multiplication()
Out[29]: 150
```

类/对象可能需要*私有*属性。这种属性通常由一个或者多个前导的下划线指明，如下面的类定义：

```
In [30]: class ExampleSeven(object):
             def __init__(self, a, b):
                 self.a = a
                 self.b = b
                 self.__sum = a + b
             multiplication = multiplication
             def addition(self):
                 return self.__sum
```

调用 **addition** 方法的结果和前面一样：

```
In [31]: c = ExampleSeven(10, 15)
In [32]: c.addition()
Out[32]: 25
```

在此，你不能直接访问私有属性 **sum**。但是，通过如下语法仍然可以做到：

```
In [33]: c._ExampleSeven__sum
Out[33]: 25
```

在定义 **ExampleSeven** 时，应该注意内部的工作机制。例如，属性值的变化不会改变 **addition** 方法调用的结果：

```
In [34]: c.a += 10
         c.a
Out[34]: 20
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [35]: c.addition()
Out[35]: 25
```

这当然是由于私有属性没有被更新：

```
In [36]: c._ExampleSeven__sum
Out[36]: 25
```

但是，调用 **multiplication** 方法的结果和预期的一样：

```
In [37]: c.multiplication()
Out[37]: 300
```

我们打算用另一个特殊的重要方法结束 **Python** 类和对象主要概念的介绍：**iter** 方法。这个方法每当要求在类的一个实例上循环时调用。首先，我们定义如下的人名列表：

```
In [38]: name_list = ['Sandra', 'Lilli', 'Guido', 'Zorro', 'Henry']
```

在 **Python** 中，直接在这种列表上循环（不使用整数计数器或者索引）是很平常的：

```
In [39]: for name in name_list:
             print name

Out[39]: Sandra
         Lilli
         Guido
         Zorro
         Henry
```

现在，我们将定义新的 **Python** 类，改列返回列表中的值，但是列表在迭代子开始从列表中返回值之前排序。**sorted_list** 类包含如下定义：

init

初始化 **elements** 属性，按照我们的预期，该属性是一个列表对象，将在实例化时排序。

iter

这个特殊方法在需要循环时调用；它需要 **next** 方法的定义。

next

这个方法定义每个循环步骤中发生的操作；它从索引值 **self.position = -1** 开始，每次调用时为该值加 1；然后返回处于当前索引位置 **self.position** 的 **elements** 值。

类定义如下：

```
In [40]: class sorted_list(object):
             def __init__(self, elements):
                 self.elements = sorted(elements)  # sorted list object
             def __iter__(self):
                 self.position = -1
                 return self
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        def next(self):
            if self.position == len(self.elements) - 1:
                raise StopIteration
            self.position += 1
            return self.elements[self.position]
```

现在用 **name_list** 对象实例化该类：

```
In [41]: sorted_name_list = sorted_list(name_list)
```

结果正是我们想要的——在新对象上循环，按照字母顺序返回元素：

```
In [42]: for name in sorted_name_list:
             print name
Out[42]: Guido
         Henry
         Lilli
         Sandra
         Zorro
```

原则上，我们已经复制了 **sorted** 函数的一次调用，该函数以**列表**对象为输入，返回的输出也是一个**列表**对象：

```
In [43]: type(sorted(name_list))
Out[43]: list
In [44]: for name in sorted(name_list):
             print name
Out[44]: Guido
         Henry
         Lilli
         Sandra
         Zorro
```

不过，我们的方法工作于一个全新类型的对象上——**sorted_list**：

```
In [45]: type(sorted_name_list)
Out[45]: __main__.sorted_list
```

对 **Python** 中精选的面向对象概念的简单介绍到此为止。在下面的讨论中，这些概念由介绍性的金融用例加以阐述。此外，第 3 部分大量使用面向对象编程方法，实现一个衍生品分析库。

13.1.2 简单的短期利率类

*折现*是金融学中最基本的概念之一。由于它是基础概念，所以定义一个*折现类*可能是值得的。在连续折现的固定短期利率世界中，日期 $t > 0$ 时的未来现金流与当前日期 $t = 0$ 之间的折现因子定义为 $D_0(t) = e^{-rt}$。

首先考虑如下函数定义，该函数返回给定未来日期和固定短期利率条件下的折现因子。注意，函数定义中使用一个 **NumPy** 通用函数进行指数函数计算，以实现向量化：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [46]: import numpy as np
         def discount_factor(r, t):
             ''' Function to calculate a discount factor.

             Parameters
             ==========
             r : float
                 positive, constant short rate
             t : float, array of floats
                 future date(s), in fraction of years;
                 e.g. 0.5 means half a year from now

             Returns
             =======
             df : float
                 discount factor
             '''
             df = np.exp(-r * t)
               # use of NumPy universal function for vectorization
             return df
```

图 13-1 展示了 5 年内不同固定短期利率下折现因子的表现。$t = 0$ 时的折现因子都等于 1；也就是说，当日现金流“没有折现”。然而，如果短期利率为 10%，5 年后到期的现金流将被折算为略高于 0.6 个货币单位（即 60%）。我们用如下代码生成图表：

```
In [47]: import matplotlib.pyplot as plt
         %matplotlib inline
In [48]: t = np.linspace(0, 5)
         for r in [0.01, 0.05, 0.1]:
             plt.plot(t, discount_factor(r, t), label='r=%4.2f' % r, lw=1.5)
         plt.xlabel('years')
         plt.ylabel('discount factor')
         plt.grid(True)
         plt.legend(loc=0)
```

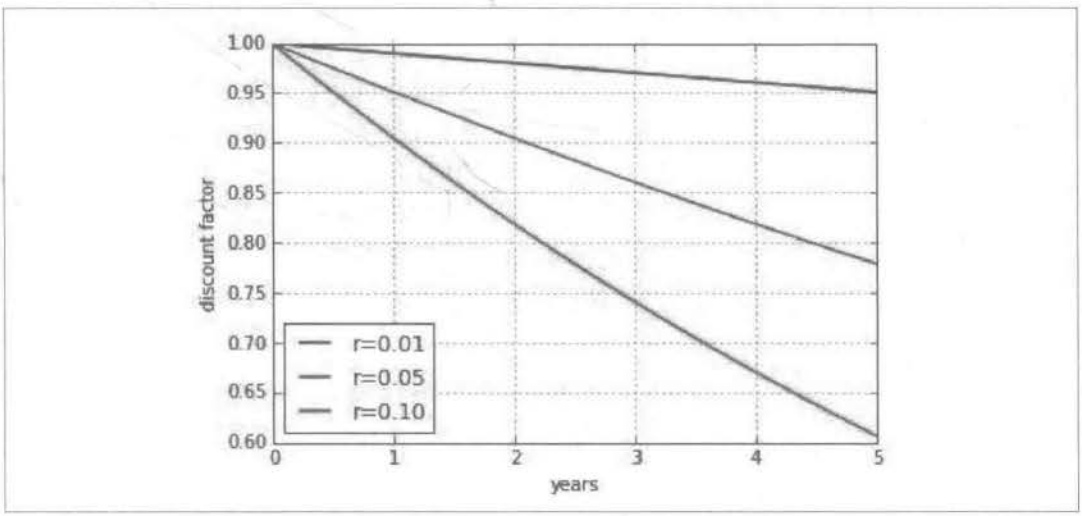


图 13-1　不同短期利率在 5 年内的折现因子

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

为了比较，现在我们来观察基于类的实现方法。我们将其称作 short_rate，因为这是核心的实体/对象，折现因子的求取通过一次方法调用实现：

```
In [49]: class short_rate(object):
             ''' Class to model a constant short rate object.

             Parameters
             ==========
             name : string
                 name of the object
             rate : float
                 positive, constant short rate

             Methods
             =======
             get_discount_factors :
                 returns discount factors for given list/array
                 of dates/times (as year fractions)
             '''
             def __init__(self, name, rate):
                 self.name = name
                 self.rate = rate
             def get_discount_factors(self, time_list):
                 ''' time_list : list/array-like '''
                 time_list = np.array(time_list)
                 return np.exp(-self.rate * time_list)
```

首先，定义 **sr** 为 **short_rate** 类的一个实例：

```
In [50]: sr = short_rate('r', 0.05)
In [51]: sr.name, sr.rate
Out[51]: ('r', 0.05)
```

为了从新对象中得到折现因子，需要一个包含年分数的时间列表：

```
In [52]: time_list = [0.0, 0.5, 1.0, 1.25, 1.75, 2.0]  # in year fractions
In [53]: sr.get_discount_factors(time_list)
Out[53]: array([ 1.        ,  0.97530991,  0.95122942,  0.93941306,  0.91621887,
                 0.90483742])
```

使用这个对象，像前面一样生成一个图表很简单（参见图 13-2）。主要的区别是我们首先更新属性 **rate**，然后为 **get_discount_factors** 提供时间列表 t：

```
In [54]: for r in [0.025, 0.05, 0.1, 0.15]:
             sr.rate = r
             plt.plot(t, sr.get_discount_factors(t),
                      label='r=%4.2f' % sr.rate, lw=1.5)
         plt.xlabel('years')
         plt.ylabel('discount factor')
         plt.grid(True)
         plt.legend(loc=0)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

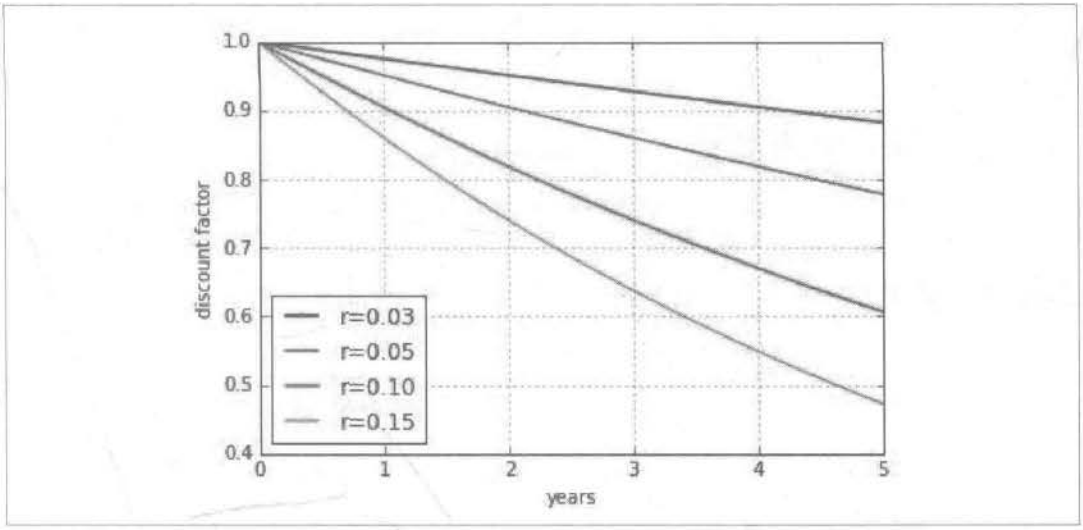


图 13-2　不同短期利率在 5 年内的折现因子

通常，折现因子“只是”达到某种目的的手段。例如，你可能想要用它们计算未来现金流的现值。利用我们的短期利率对象，在已知现金流和出现日期/时间的情况下，这是很容易做到的。考虑如下现金流示例，在例子中包含今天的负现金流和经过一年及两年之后的正现金流，这可能是某种投资机会的现金流配置：

```
In [55]: sr.rate = 0.05
         cash_flows = np.array([-100, 50, 75])
         time_list = [0.0, 1.0, 2.0]
```

有了 **time_list** 对象，折现因子只需要一个方法调用就可以得到：

```
In [56]: disc_facts = sr.get_discount_factors(time_list)
In [57]: disc_facts
Out[57]: array([ 1.        ,  0.95122942,  0.90483742])
```

现金流乘以折现因子就可以得到所有现金流的现值：

```
In [58]: # present values
         disc_facts * cash_flows
Out[58]: array([-100.        ,   47.56147123,   67.86280635])
```

投资理论中的典型决策原则之一是，决策者应该在给定（短期）利率（表示投资的机会成本）下净现值（NPV）为正数的项目上投资。在我们的例子中，NPV 就是简单地加总单个现值：

```
In [59]: # net present value
         np.sum(disc_facts * cash_flows)
Out[59]: 15.424277577732667
```

很显然，在短期利率为 5%时应该做出投资。对于 15%的利率又如何呢？此时 NPV 变成负数，不应该进行投资：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [60]: sr.rate = 0.15
         np.sum(sr.get_discount_factors(time_list) * cash_flows)
Out[60]: -1.4032346276182679
```

13.1.3 现金流序列类

通过前一个例子的体验，定义另一个类作为现金流序列的模型应该很简单。这个类应该提供方法，返回给定现金流系列（即现金流价值和日期/时间）现值和净现值的列表/数组：

```
In [61]: class cash_flow_series(object):
             ''' Class to model a cash flow series.

             Attributes
             ==========
             name : string
                 name of the object
             time_list : list/array-like
                 list of (positive) year fractions
             cash_flows : list/array-like
                 corresponding list of cash flow values
             short_rate : instance of short_rate class
                 short rate object used for discounting

             Methods
             =======
             present_value_list :
                 returns an array with present values
             net_present_value :
                 returns NPV for cash flow series
             '''
             def __init__(self, name, time_list, cash_flows, short_rate):
                 self.name = name
                 self.time_list = time_list
                 self.cash_flows = cash_flows
                 self.short_rate = short_rate
             def present_value_list(self):
                 df = self.short_rate.get_discount_factors(self.time_list)
                 return np.array(self.cash_flows) * df
             def net_present_value(self):
                 return np.sum(self.present_value_list())
```

我们用来自前一个例子的所有对象实例化该类：

```
In [62]: sr.rate = 0.05
         cfs = cash_flow_series('cfs', time_list, cash_flows, sr)
In [63]: cfs.cash_flows
Out[63]: array([-100,   50,   75])
In [64]: cfs.time_list
Out[64]: [0.0, 1.0, 2.0]
```

现在，可以将现值和 NPV 与前面的结果对比，幸运的是，得到的结果相同：

```
In [65]: cfs.present_value_list()
Out[65]: array([-100.        ,   47.56147123,   67.86280635])
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [66]: cfs.net_present_value()
Out[66]: 15.424277577732667
```

前一个例子的步骤有进一步推广的潜力。选项之一是定义新类，提供计算不同短期利率下 NPV 的方法——即*敏感度分析*。当然，我们从 **cash_flow_series** 类继承：

```
In [67]: class cfs_sensitivity(cash_flow_series):
             def npv_sensitivity(self, short_rates):
                 npvs = []
                 for rate in short_rates:
                     sr.rate = rate
                     npvs.append(self.net_present_value())
                 return np.array(npvs)
In [68]: cfs_sens = cfs_sensitivity('cfs', time_list, cash_flows, sr)
```

例如，定义一个包含不同短期利率的列表，就可以轻松地比较结果 NPV：

```
In [69]: short_rates = [0.01, 0.025, 0.05, 0.075, 0.1, 0.125, 0.15, 0.2]
In [70]: npvs = cfs_sens.npv_sensitivity(short_rates)
         npvs
Out[70]: array([ 23.01739219,  20.10770244,  15.42427758,  10.94027255,
                  6.64667738,   2.53490386,  -1.40323463,  -8.78945889])
```

图 13-3 以图形方式展示了结果。较粗的水平线（在坐标 0 上）展示了对应（短期）利率下有利可图的投资与应该避免的投资之间的分界点：

```
In [71]: plt.plot(short_rates, npvs, 'b')
         plt.plot(short_rates, npvs, 'ro')
         plt.plot((0, max(short_rates)), (0, 0), 'r', lw=2)
         plt.grid(True)
         plt.xlabel('short rate')
         plt.ylabel('net present value')
```

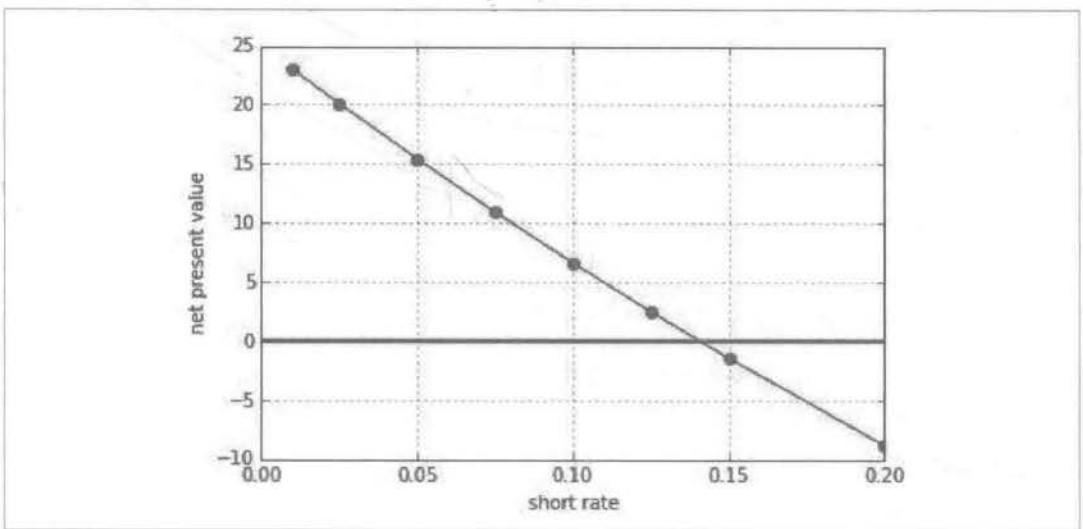


图 13-3　不同短期利率下的现金流列表净现值

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

13.2 图形用户界面

和开发者或者数据科学家相比，对于大部分计算机用户来说，图形用户界面（GUI）是他们所要使用的。这种 GUI 不仅带来视觉上的吸引力和简单性，还可以比交互性脚本、命令行接口或者 shell 更好地引导和控制用户交互。下面，我们以前一小节的例子为基础，为短期利率和现金流序列类构建简单的 GUI。

为了构建 GUI，我们使用 **traits** 库，这个库的文档可以在 http://code.enthought.com/projects/traits/docs/html/index.html 上找到。**traits** 库通常用于在现有类基础上快速构建 GUI，很少用于较复杂的应用程序。下面，我们将重新实现前面的两个示例类，考虑使用 GUI 与对应类的实例交互。

13.2.1 带 GUI 的短期利率类

首先，我们需要导入 **traits.api** 子库：

```
In [72]: import numpy as np
         import traits.api as trapi
```

为了定义新的 **short_rate** 类，我们从 **HasTraits** 类中继承。还要注意，在如下的类定义中，**traits** 有自己的数据类型，这些类型通常紧密地与 GUI 的视觉元素交织在一起——换言之，**traits** 知道使用哪些图形元素（例如用于文本字段的元素）（半）自动地构建 GUI：

```
In [73]: class short_rate(trapi.HasTraits):
             name = trapi.Str
             rate = trapi.Float
             time_list = trapi.Array(dtype=np.float, shape=(5,))
             def get_discount_factors(self):
                 return np.exp(-self.rate * self.time_list)
```

基于 **traits** 的类的实例化和往常一样：

```
In [74]: sr = short_rate()
```

然而，通过调用 **configure_traits** 方法（从 **HasTraits** 继承），可以自动生成一个 GUI，我们可以使用这个 GUI 输入新对象 **sr** 的属性值：

```
In [75]: sr.configure_traits()
```

图 13-4 展示了这样一个简单的 GUI，本例中该界面还是空白的（没有在不同字段中输入数值）。注意，下方的 5 个字段都属于“时间列表”——这是默认生成的布局。

图 13-5 展示了同样的简单 GUI，不过这次每个字段都有输入值。按下 OK 按钮将输入字段的值赋予对应的对象属性。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

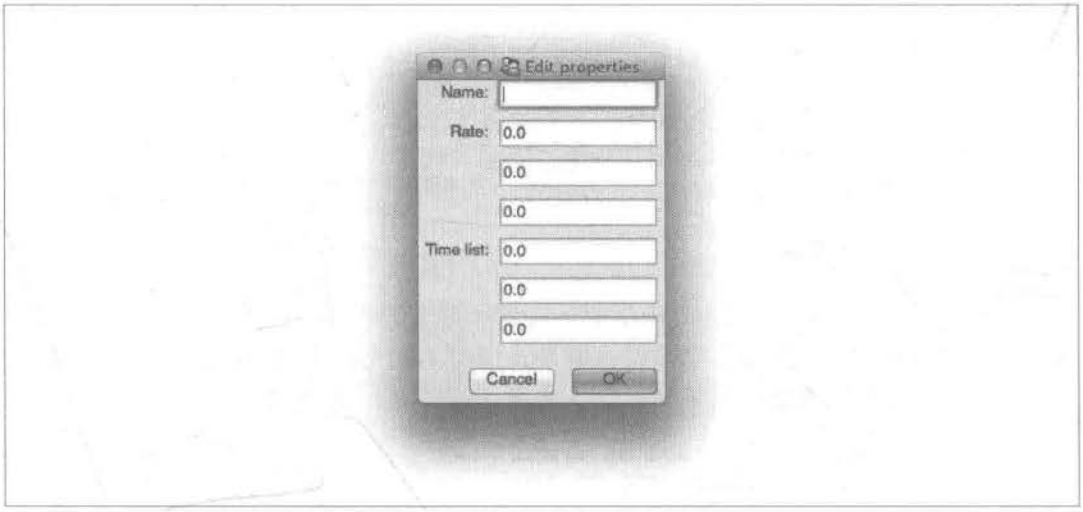


图 13-4　traits GUI 截图（空）

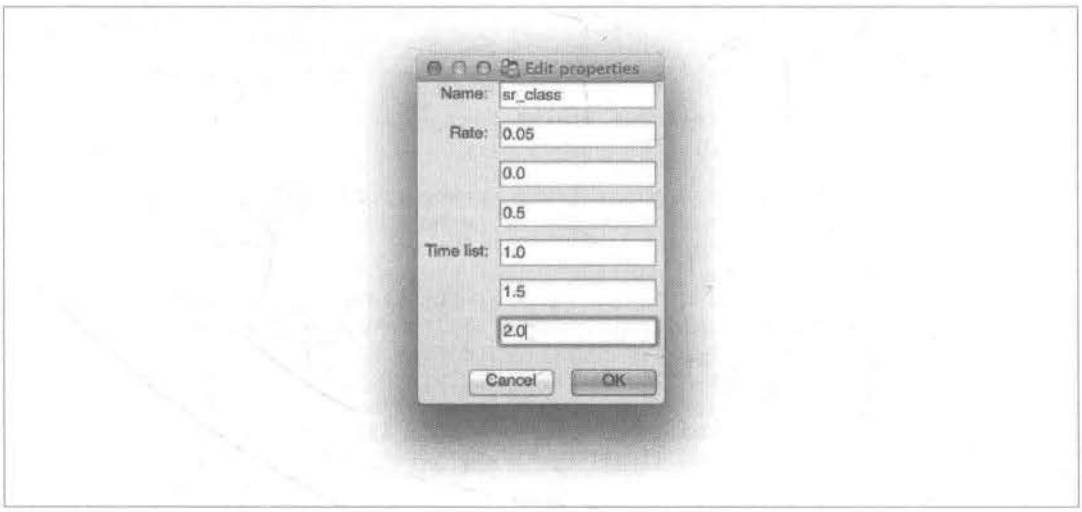


图 13-5　traits GUI 截图（有数据）

实际上，这和如下的代码行结果相同：

```
In [76]: sr.name = 'sr_class'
         sr.rate = 0.05
         sr.time_list = [0.0, 0.5, 1.0, 1.5, 2.0]
```

通过提供 **traits** 专用的数据类型，**traits** 可以生成正确的视觉元素，通过 GUI 实现这些操作——即用于 **sr.name** 的一个文本输入字段和 5 个用于列表对象 **sr.time_list** 的输入元素。

输入操作之后，新对象的行为和前一小节中的 **short_rate** 相同：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [77]: sr.rate
Out[77]: 0.05
In [78]: sr.time_list
Out[78]: array([ 0. ,  0.5,  1. ,  1.5,  2. ])
In [79]: sr.get_discount_factors()
Out[79]: array([ 1.        ,  0.97530991,  0.95122942,  0.92774349,  0.90483742])
```

13.2.2 值的更新

迄今为止，使用 **traits** 的新 **short_rate** 类可以为该类实例的初始化属性输入数据。但是，GUI 通常也被用于显示结果。通常应该避免在通过 GUI 提供输入数据，让用户通过交互式脚本访问结果。为此，需要另一个子库 **traitsui.api**：

```
In [80]: import traits.api as trapi
         import traitsui.api as trui
```

这个子库可以在同一个类/对象上生成不同的视图，还提供更多的选项，如标签和格式。下面的类定义中，关键是按下 Updated 按钮时发生的情况。在本例中，调用私有方法 **_update_fired**，该方法更新包含折现因子的列表对象。然后，这个更新后的列表在 GUI 窗口中显示，前提条件是用户已经提供了所有输入参数：

```
In [81]: class short_rate(trapi.HasTraits):
             name = trapi.Str
             rate = trapi.Float
             time_list = trapi.Array(dtype=np.float, shape=(1, 5))
             disc_list = trapi.Array(dtype=np.float, shape=(1, 5))
             update = trapi.Button
             def _update_fired(self):
                 self.disc_list = np.exp(-self.rate * self.time_list)
             v = trui.View(trui.Group(trui.Item(name = 'name'),
                      trui.Item(name='rate'),
                      trui.Item(name='time_list',
                                label='Insert Time List Here'),
                      trui.Item('update', show_label=False),
                      trui.Item(name='disc_list',
                                label='Press Update for Factors'),
                      show_border=True, label='Calculate Discount Factors'),
                      buttons = [trui.OKButton, trui.CancelButton],
                      resizable = True)
```

实例化和配置的实现和以前一样：

```
In [82]: sr = short_rate()
In [83]: sr.configure_traits()
```

图 13-6 展示了新的增强型 GUI，目前仍是空白的。可以看到新的元素，如 Update 按钮和折现因子的输出字段。

图 13-7 展示了新的 GUI“运行中”发生的情况。为对象属性提供值并按下 Update 按钮，会返回计算出的折现因子——这次显示在 GUI 窗口中。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

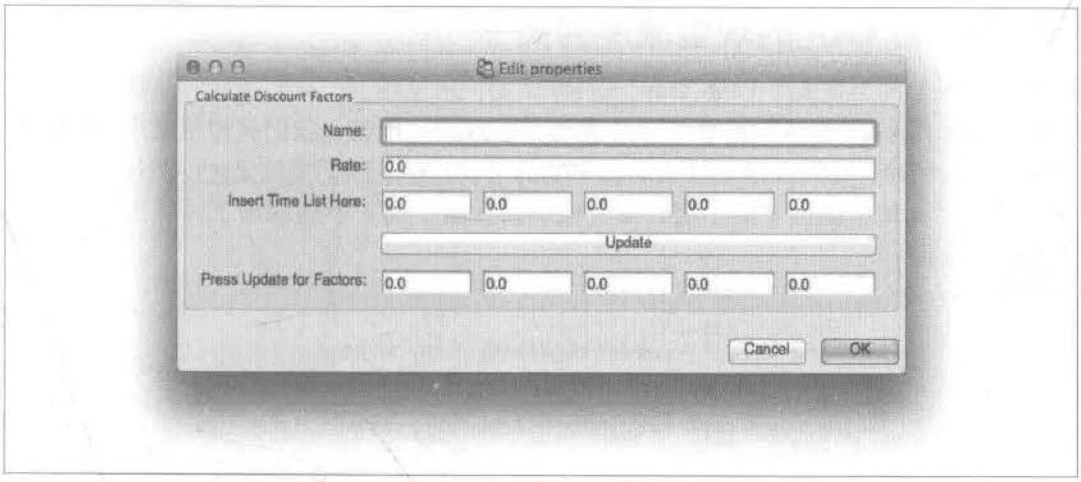


图 13-6　带更新功能的 traits GUI 截图（空）

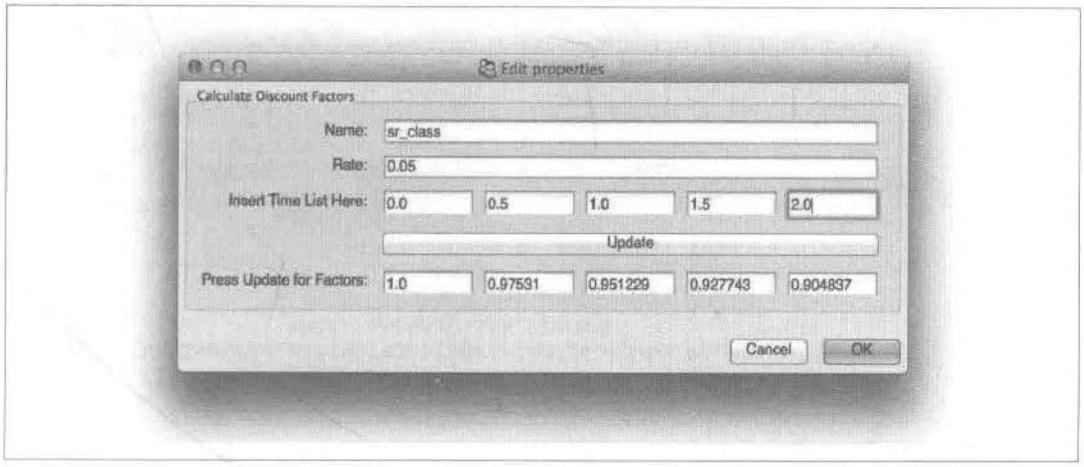


图 13-7　带更新功能的 traits GUI（更新之后）

下面的 **Python** 一步一步地说明没有 GUI 时的等价操作：

```
In [84]: sr.name = 'sr_class'
         sr.rate = 0.05
         sr.time_list = np.array(([0.0, 0.5, 1.0, 1.5, 2.0],), dtype=np.float32)
```

第二步，更新包含折现因子的列表对象：

```
In [85]: sr._update_fired()
```

最后，输出计算/更新后的包含折现因子的列表：

```
In [86]: sr.disc_list
Out[86]: array([[ 1.        ,  0.97530991,  0.95122942,  0.92774349,  0.90483742
         ]])
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

13.2.3 带 GUI 的现金流序列类

本节最后一个例子是关于 **cash_flow_series** 类的。从原理上，我们在前一个例子中已经了解 **traits** 在 GUI 窗口中显示结果的基本工作方式。现在，我们只想给整个故事增添一些曲折：例如，可以轻松更改短期利率值的滑块。在下面的类定义中，滑块用 **Range** 函数实现，在其中提供了最小值、最大值和默认值。例子中还增加了计算现值和净现值的输出字段：

```
In [87]: class cash_flow_series(trapi.HasTraits):
             name = trapi.Str
             short_rate = trapi.Range(0.0, 0.5, 0.05)
             time_list = trapi.Array(dtype=np.float, shape=(1, 6))
             cash_flows = trapi.Array(dtype=np.float, shape=(1, 6))
             disc_values = trapi.Array(dtype=np.float, shape=(1, 6))
             present_values = trapi.Array(dtype=np.float, shape=(1, 6))
             net_present_value = trapi.Float
             update = trapi.Button
             def _update_fired(self):
                 self.disc_values = np.exp(-self.short_rate * self.time_list)
                 self.present_values = self.disc_values * self.cash_flows
                 self.net_present_value = np.sum(self.present_values)
             v = trui.View(trui.Group(trui.Item(name = 'name'),
                       trui.Item(name='short_rate'),
                       trui.Item(name='time_list', label='Time List'),
                       trui.Item(name='cash_flows', label='Cash Flows'),
                       trui.Item('update', show_label=False),
                       trui.Item(name='disc_values',
                                 label='Discount Factors'),
                       trui.Item(name='present_values',
                                 label='Present Values'),
                       trui.Item(name='net_present_value',
                                 label='Net Present Value'),
                       show_border=True, label='Calculate Present Values'),
                       buttons = [trui.OKButton, trui.CancelButton],
                       resizable = True)
```

除了稍微复杂一点的类定义之外，用法仍然不变：

```
In [88]: cfs = cash_flow_series()
In [89]: cfs.configure_traits()
```

图 13-8 展示了尚未进行任何操作时（空白）的新 GUI。注意滑块和用于现金流值、现值和净现值的新字段。

图 13-9 展示了已经输入数据，但尚未进行其他操作时的 GUI。

最后，图 13-10 显示了包含输入和结果数据的 GUI——也就是在按下 **Update** 按钮之后的情况。这个例子仍然相当简单，但是结果几乎可以称作一个*应用程序*。我们实现了如下功能：

输入

GUI 可以输入数据初始化任何对象属性。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

逻辑

应用逻辑可以计算折现因子、限制和 NPV。

输出

GUI 显示向输入数据应用逻辑的结果。

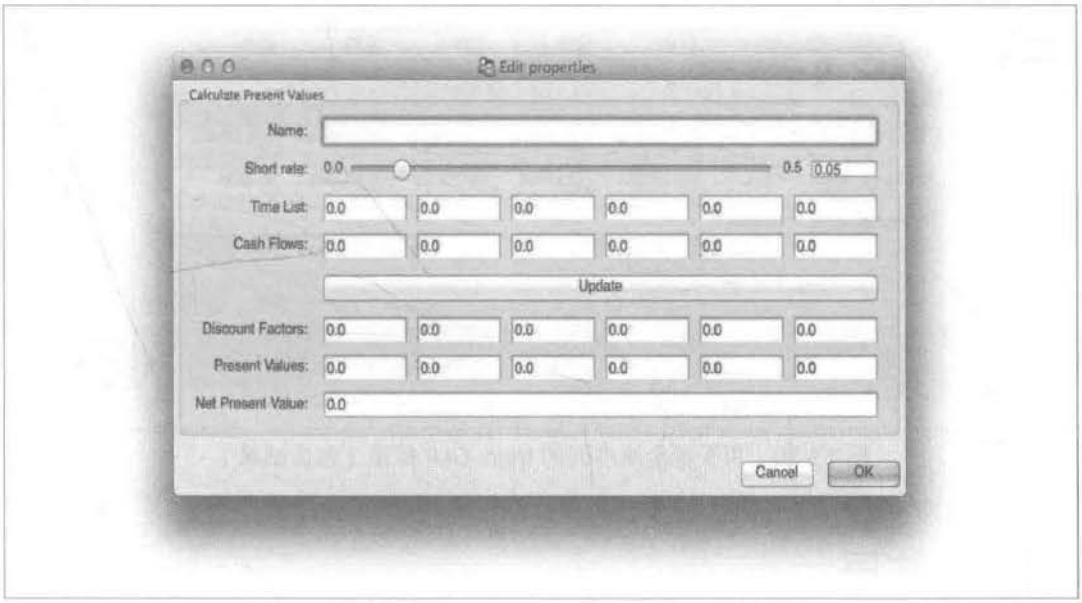


图 13-8　用于现金流序列的 traits GUI 截图（空）

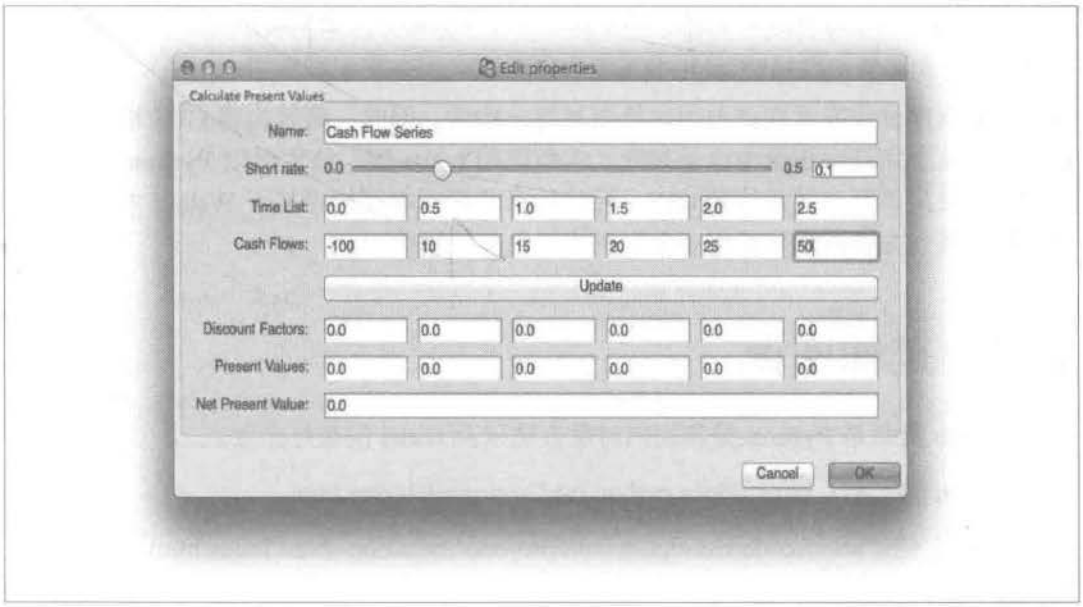


图 13-9　用于现金流序列的 traits GUI 截图（包含输入数据）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

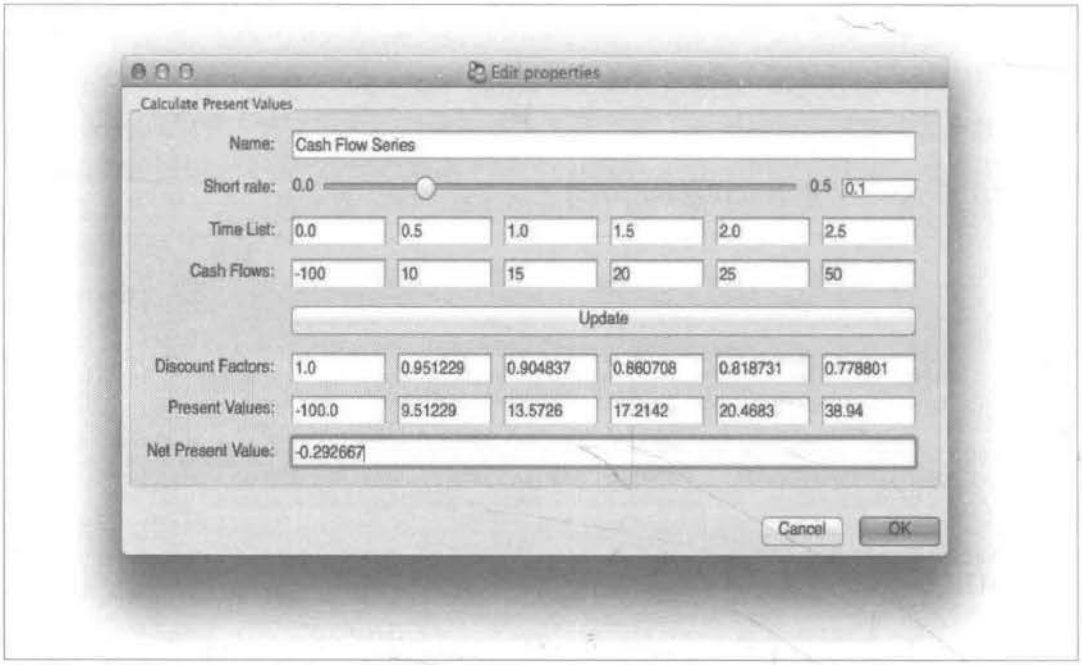


图 13-10　用于现金流序列的 traits GUI 截图（包含结果）

13.3　结语

面向对象范型是现代应用开发必不可少的工具。**Python** 提供了相当灵活的框架以定义客户定义类和使用这些类的实例。本章仅提供了 **Python** 类定义的基础知识——本书第 3 部分阐述（金融）**Python** 类在更复杂和现实的应用场景中的使用。

现代应用程序设计通常在图形用户界面基础上构建。因此，高效构建 GUI 相当重要，即使在快速应用开发场景中也是如此。本章使用 **traits** 库，该库可以 **Python** 风格的面向对象方法简单而高效地构建 GUI。后续的章节说明如何构建基于 **Web** 技术的 GUI，这种技术替代方案近来甚至用于金融机构的内部应用中。

13.4　延伸阅读

下面的 Web 资源是 **Python** 类和面向对象方法以及 **traits** 的很好出发点。

- **Python** 类文档：https://docs.python.org/2/tutorial/classes.html。
- **traits** 文档：http://code.enthought.com/projects/traits/docs/html/index.html。

书籍形式的帮助资源有：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- Downey, Allen (2012): *Think Python*. O'Reilly, Sebastopol, CA.
- Goodrich, Michael et al. (2013): *Data Structures and Algorithms in Python*. JohnWiley & Sons, Hoboken, NJ.
- Langtangen, Hans Petter (2009): *A Primer on Scientific Programming with Python*. Springer Verlag, Berlin, Heidelberg.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 14 章

Web 集成

我曾经引述过，未来的公司都将是互联网公司。我仍然相信这一点，比以前更坚定。

——Andrew Grove

互联网（或者 Web）已经从独立的世界发展为一个无所不在、包罗万象的世界。它已经成为实现多种不同用例的技术平台。从金融的角度，下面这些方面特别值得注意：

数据供应/收集

Web 技术可以简单的方式（通常成本也更低）供应和收集数据；它通常还加速所有相关的过程。大型金融数据提供商（如 Bloomberg 和 Thomson Reuters）很大程度上依赖 Web 和相关的技术为金融界提供实时数据。

交易/购买/销售

使用 Web 还能方便金融证券交易；今天的私人投资者也能够使用专业交易设施（例如 Interactive Brokers 等在线经纪商），实施交易证券。

应用程序提供

软件即服务（SaaS）等模式使小型公司（如创业公司）和大型公司都能够高效地提供应用；即使最小的机构现在也可以非常低的成本联系到全球的目标受众。大企业从 Web 技术中得益，例如，它们用 Web 提供可以通过标准浏览器访问和使用的内部应用，而不需要在成百上千台不同的机器上安装这类应用程序。

通信

当然，Web 方便了组织内部和外部的通信；今天的大部分公司和金融通信从纸质文件转向 Web。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

商品化/可伸缩性

现在的 Web 技术还考虑了更好的可视化，使 Web 服务器和服务器成为每个人都能够以较低的可变价格租赁的商品，在需求变化时很容易伸缩；计算能力和存储容量越来越像我们每个人都从家里的插座中得到的电力。

用于 Web 的 **Python** 同样是一个宽泛的主题，无法在本书的一个章节中涵盖。不过，本章可以介绍一些金融方面的重要主题，特别是：

Web 协议

第一小节说明如何通过 **FTP** 传递文件，通过 HTTP 访问网站。

Web 图表绘制

Web 技术通常可以实现比标准方法更好的交互性和更好的实时支持，例如，对数据图表的支持；第二小节介绍绘图库 **Bokeh**，它用于生成交互性 Web 图表，实现金融数据的实时绘图。

Web 应用

Python 的优势之一是用于开发 Web 应用的强大 Web 框架；其中一个具有真正 **Python** 风格、相当流行的框架是 **Flask**。本章介绍用这一框架开发基于 Web 应用的技术。

Web 服务

Web 服务已经成为面向 Web 应用的重要方面；最后一节说明如何开发简单的 Web 服务，估算基于 VSTOXX 波动率指数的欧式期权价值。

14.1 Web 基础知识

本节简单概述处理 Web 技术和协议的一些精选的 **Python** 库，许多主题（如 **Python** 对电子邮件功能的处理）无法一一介绍。

14.1.1 ftplib

顾名思义，文件传输协议（FTP）是通过 Web 传输文件的一个协议[1]。**Python** 提供了一个名为 **ftplib** 的库（http://docs.python.org/2/library/ftplib.html），专用于处理 FTP：

```
In [1]: import ftplib
        import numpy as np
```

下面，我们将连接到一个 FTP 服务器，登录，向服务器传输一个文件，将其回传到本地计算机，并从服务器上删除该文件。首先是连接：

```
In [2]: ftp = ftplib.FTP('quant-platform.com')
```

1 细节和背景知识参见 http://en.wikipedia.org/wiki/Ftp。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

并不是所有 FTP 服务器都有密码保护，但是这个服务器是有保护的：

```
In [3]: ftp.login(user='python', passwd='python')
Out[3]: '230 Login successful.'
```

我们生成一个包含随机数据的 **NumPy ndarray** 对象，保存到磁盘作为传输文件：

```
In [4]: np.save('./data/array', np.random.standard_normal((100, 100)))
```

为了接下来的 FTP 文件传输，必须打开该文件供读取：

```
In [5]: f = open('./data/array.npy', 'r')
```

现在，通过 STOR 命令和目标文件名的组合，选择二进制传输，写入服务器：

```
In [6]: ftp.storbinary('STOR array.npy', f)
Out[6]: '226 Transfer complete.'
```

我们来查看一下 FTP 服务器的目录，确实，这个文件已经传输：

```
In [7]: ftp.retrlines('LIST')
Out[7]: -rw---    1 1001     1001        80080 Sep 29 11:05 array.npy

        '226 Directory send OK.'
```

反向的操作很类似，为了读取一个远程文件并保存到磁盘，我们需要打开一个新文件，这次使用**写入（write）**模式：

```
In [8]: f = open('./data/array_ftp.npy', 'wb').write
```

同样，选择二进制传输，使用 RETR 命令从 FTP 服务器读取文件：

```
In [9]: ftp.retrbinary('RETR array.npy', f)
Out[9]: '226 Transfer complete.'
```

现在不再需要将文件保存在服务器了，可以删除它：

```
In [10]: ftp.delete('array.npy')
Out[10]: '250 Delete operation successful.'
In [11]: ftp.retrlines('LIST')
Out[11]: '226 Directory send OK.'
```

最后，应该关闭 FTP 服务器连接：

```
In [12]: ftp.close()
```

在本地目录现在有两个文件，其中一个是在本地生成的，另一个是通过读取服务器文件生成的：

```
In [13]: !ls -n ./data
<<<<<<< HEAD
Out[13]: insgesamt 156
         -rw--- 1 1000 1000 77824 Sep 15 08:14 array_ftp.npy
         -rw--- 1 1000 1000 80080 Sep 15 08:14 array.npy
=======
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[13]: insgesamt 156
         -rw--- 1 1000 1000 77824 Sep 29 17:05 array_ftp.npy
         -rw--- 1 1000 1000 80080 Sep 29 17:05 array.npy
>>>>>>> 798603793467fffcd06a9df88edf091e339dec37
In [14]: !rm -f ./data/arr*
           # cleanup directory
```

目前为止发生的一切都没有采用加密（也就是说，完全不安全）。登录信息和数据都以可读的形式传输。但是，对于大部分应用程序，这些操作应该加密，使其他人不能读取数据或者窃取登录信息，做更危险的事情。

ftplib 可以通过函数 FTP_TLS 安全连接到 FTP 服务器。一旦建立了这样的安全连接，所有其他操作都保持不变：

```
In [15]: ftps = ftplib.FTP_TLS('quant-platform.com')
In [16]: ftps.login(user='python', passwd='python')
Out[16]: '230 Login successful.'
In [17]: ftps.prot_p()
Out[17]: '200 PROT now Private.'
In [18]: ftps.retrlines('LIST')
Out[18]: '226 Directory send OK.'
In [19]: ftps.close()
```

14.1.2 httplib

Web 上的另一个重要协议（可能是最重要的）是超文本传输协议（HTTP）[1]。这个协议每当（基于 HTML 的）网页在浏览器中显示时使用。处理 HTTP 的 **Python** 库是 **httplib**（http://docs.python.org/2/library/httplib.html）：

```
In [20]: import httplib
```

和 FTP 一样，首先需要 HTTP 服务器的连接：

```
In [21]: http = httplib.HTTPConnection('hilpisch.com')
```

一旦建立连接，就可以发送请求，例如请求 **index.htm** 页面（文件）：

```
In [22]: http.request('GET', '/index.htm')
```

使用 **getresponse** 方法测试请求是否成功：

```
In [23]: resp = http.getresponse()
```

返回的对象提供状态信息。幸运的是，请求是成功的：

```
In [24]: resp.status, resp.reason
Out[24]: (200, 'OK')
```

有了这个响应对象，就可以读取内容了：

1 细节和背景知识参见 http://en.wikipedia.org/wiki/Http。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [25]: content = resp.read()
         content[:100]
           # first 100 characters of the file
Out[25]: '<!doctype html>\n<html lang="en">\n\n\t<head>\n\t\t<meta charset="utf-
         8">\n\n\t\t<title>Dr. Yves J. Hilpisch \xe2\x80'
```

一旦拥有了特定网页的内容，就有许多潜在的用例。例如，你可能想要搜索某些信息。你可能知道，可以通过搜索 E（在这个很特殊的例子中）找到页面上的电子邮件地址。由于 **content** 是一个**字符串对象**，可以应用 **find** 方法搜索“E”[1]：

```
In [26]: index = content.find(' E ')
         index
Out[26]: 2071
```

利用搜索信息的索引值，可以检查对象后续的字符：

```
In [27]: content[index:index + 29]
Out[27]: ' E contact [at] dyjh [dot] de'
```

完成之后，同样应该关闭服务器连接：

```
In [28]: http.close()
```

14.1.3 urllib

另一个 **Python** 库支持**不同** Web 协议的使用，这个库称作 **urllib**（http://docs.python.org/2/libraray/urllib.html）。还有一个相关的库 **urllib2**（http://docs.python.org/2/library/urllib2.html）。两个库都设计为可以根据 URL（统一资源定位器）[2]的“统一”精神处理任意 Web 资源。例如，标准用例之一是通过 Web 读取 CSV 数据文件。首先导入 urllib：

```
In [29]: import urllib
```

这个库的函数应用方法和 **ftplib** 及 **httplib** 很类似。当然，我们需要一个代表感兴趣的 Web 资源（一般是 HTTP 或者 FTP 服务器）的 URL。本例中使用雅虎财经的 URL，以读取 CSV 格式的股价信息：

```
In [30]: url = 'http://ichart.finance.yahoo.com/table.csv?g=d&ignore=.csv'
         url += '&s=YHOO&a=01&b=1&c=2014&d=02&e=6&f=2014'
```

下一步，必须建立与资源的连接：

```
In [31]: connect = urllib.urlopen(url)
```

建立连接之后，调用连接对象上的 read 方法读出内容：

```
In [32]: data = connect.read()
```

本例的结果是雅虎自身的历史股价信息：

---

1 这个例子仅用于展示。一般来说，应该使用专用的库，如 lxml（http://lxml.de）或者 Beautiful Soup（http://www/crummy.com/software/BeautifulSoup/）。——原注

2 这些库有一些替代方案，如 Requests（http://docs/python-request.org），该库带有较现代化的 API。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [33]: print data
Out[33]: Date,Open,High,Low,Close,Volume,Adj Close
         2014-03-06,39.60,39.98,39.50,39.66,10626700,39.66
         2014-03-05,39.83,40.15,39.19,39.50,12536800,39.50
         2014-03-04,38.76,39.79,38.68,39.63,16139400,39.63
         2014-03-03,37.65,38.66,37.43,38.25,14714700,38.25
         2014-02-28,38.55,39.38,38.22,38.67,16957100,38.67
         2014-02-27,37.80,38.48,37.74,38.47,15489400,38.47
         2014-02-26,37.35,38.10,37.34,37.62,15778900,37.62
         2014-02-25,37.48,37.58,37.02,37.26,9756900,37.26
         2014-02-24,37.23,37.71,36.82,37.42,15738900,37.42
         2014-02-21,37.90,37.96,37.22,37.29,12351900,37.29
         2014-02-20,37.83,38.04,37.30,37.79,11155900,37.79
         2014-02-19,38.06,38.33,37.68,37.81,15851900,37.81
         2014-02-18,38.31,38.59,38.09,38.31,12096400,38.31
         2014-02-14,38.43,38.45,38.11,38.23,9975800,38.23
         2014-02-13,37.92,38.69,37.79,38.52,12088100,38.52
         2014-02-12,38.60,38.91,38.03,38.11,14088500,38.11
         2014-02-11,38.15,38.86,38.09,38.50,18348000,38.50
         2014-02-10,38.00,38.13,37.25,37.76,17642900,37.76
         2014-02-07,36.65,37.27,36.24,37.23,16178500,37.23
         2014-02-06,35.65,36.75,35.61,36.24,14250000,36.24
         2014-02-05,35.60,35.94,34.99,35.49,14022900,35.49
         2014-02-04,35.11,35.86,34.86,35.66,21082500,35.66
         2014-02-03,35.94,36.01,34.66,34.90,22195200,34.90
```

urllib 库还提供方便的函数，以自定义 URL 字符串。例如，你可能想要参数化搜索的股票代码和起始日期。为此，定义一个包含字符串替代部分的新 URL 字符串，可以在替代部分插入参数：

```
In [34]: url = 'http://ichart.finance.yahoo.com/table.csv?g=d&ignore=.csv'
         url += '&%s'  # for replacement with parameters
         url += '&d=06&e=30&f=2014'
```

urlencode 函数的参数是一个 Python 字典对象，包含参数名称和关联的值：

```
In [35]: params = urllib.urlencode({'s': 'MSFT', 'a': '05', 'b': 1, 'c': 2014})
```

结果是，可以在前述的 URL 字符串中插入一个字符串对象，以完成 URL：

```
In [36]: params
Out[36]: 'a=05&s=MSFT&b=1&c=2014'
In [37]: url % params
Out[37]: 'http://ichart.finance.yahoo.com/table.csv?g=d&ignore=.csv&a=05&s=MSFT&
         b=1&c=2014&d=06&e=30&f=2014'
```

利用这个新的 URL 字符串，建立连接并从连接中读取数据：

```
In [38]: connect = urllib.urlopen(url % params)
In [39]: data = connect.read()
```

结果仍然是股价数据，这次是 Microsoft 在更多日期中的股价信息：

```
In [40]: print data
Out[40]: Date,Open,High,Low,Close,Volume,Adj Close
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
2014-07-30,44.07,44.10,43.29,43.58,31921400,43.31
2014-07-29,43.91,44.09,43.64,43.89,27763100,43.62
2014-07-28,44.36,44.51,43.93,43.97,29684200,43.70
2014-07-25,44.30,44.66,44.30,44.50,26737700,44.22
2014-07-24,44.93,45.00,44.32,44.40,30725300,44.12
2014-07-23,45.45,45.45,44.62,44.87,52362900,44.59
2014-07-22,45.00,45.15,44.59,44.83,43095800,44.55
2014-07-21,44.56,45.16,44.22,44.84,37604400,44.56
2014-07-18,44.65,44.84,44.25,44.69,43407500,44.41
2014-07-17,45.45,45.71,44.25,44.53,82180300,44.25
2014-07-16,42.51,44.31,42.48,44.08,63318000,43.81
2014-07-15,42.33,42.47,42.03,42.45,28748700,42.19
2014-07-14,42.22,42.45,42.04,42.14,21881100,41.88
2014-07-11,41.70,42.09,41.48,42.09,24083000,41.83
2014-07-10,41.37,42.00,41.05,41.69,21854700,41.43
2014-07-09,41.98,41.99,41.53,41.67,18445900,41.41
2014-07-08,41.87,42.00,41.61,41.78,31218200,41.52
2014-07-07,41.75,42.12,41.71,41.99,21952400,41.73
2014-07-03,41.91,41.99,41.56,41.80,15969300,41.54
2014-07-02,41.73,41.90,41.53,41.90,20208100,41.64
2014-07-01,41.86,42.15,41.69,41.87,26917000,41.61
2014-06-30,42.17,42.21,41.70,41.70,30805500,41.44
2014-06-27,41.61,42.29,41.51,42.25,74640000,41.99
2014-06-26,41.93,41.94,41.43,41.72,23604400,41.46
2014-06-25,41.70,42.05,41.46,42.03,20049100,41.77
2014-06-24,41.83,41.94,41.56,41.75,26509100,41.49
2014-06-23,41.73,42.00,41.69,41.99,18743900,41.73
2014-06-20,41.45,41.83,41.38,41.68,47764900,41.42
2014-06-19,41.57,41.77,41.33,41.51,19828200,41.25
2014-06-18,41.61,41.74,41.18,41.65,27097000,41.39
2014-06-17,41.29,41.91,40.34,41.68,22518600,41.42
2014-06-16,41.04,41.61,41.04,41.50,24205300,41.24
2014-06-13,41.10,41.57,40.86,41.23,26310000,40.97
2014-06-12,40.81,40.88,40.29,40.58,29818900,40.33
2014-06-11,40.93,41.07,40.77,40.86,18040000,40.61
2014-06-10,41.03,41.16,40.86,41.11,15117700,40.85
2014-06-09,41.39,41.48,41.02,41.27,15019200,41.01
2014-06-06,41.48,41.66,41.24,41.48,24060500,41.22
2014-06-05,40.59,41.25,40.40,41.21,31865200,40.95
2014-06-04,40.21,40.37,39.86,40.32,23209000,40.07
2014-06-03,40.60,40.68,40.25,40.29,18068900,40.04
2014-06-02,40.95,41.09,40.68,40.79,18504300,40.54
```

urlretrieve 可以在一步中读取内容保存到磁盘，这在许多情况下都相当方便：

```
In [41]: urllib.urlretrieve(url % params, './data/msft.csv')
Out[41]: ('./data/msft.csv', <httplib.HTTPMessage instance at 0x7f92ca59afc8>)
```

对保存文件进行简单检查表明，我们确实读取和保存了和前面一样的内容：

```
In [42]: csv = open('./data/msft.csv', 'r')
         csv.readlines()[:5]

Out[42]: ['Date,Open,High,Low,Close,Volume,Adj Close\n',
          '2014-07-30,44.07,44.10,43.29,43.58,31921400,43.31\n',
          '2014-07-29,43.91,44.09,43.64,43.89,27763100,43.62\n',
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        '2014-07-28,44.36,44.51,43.93,43.97,29684200,43.70\n',
        '2014-07-25,44.30,44.66,44.30,44.50,26737700,44.22\n']
In [43]: !rm -f ./data/*
```

14.2 Web 图表绘制

第 5 章介绍了最流行的 **Python** 绘图库 **matplotlib**。但是，正如它在 2D 和 3D 图表绘制中表现出来的强大功能，它的优势在于静态图表绘制。实际上，**matplotlib** 也能够生成交互式图表，例如带有变量滑块的图表。但是可以肯定的说，这不是它的强项[1]。

本节从生成*静态*图表开始，然后介绍*交互式*图表绘制，最后介绍*实时*图表绘制。

14.2.1 静态图表绘制

首先，正如前一小节，我们以从雅虎财经 API 获取的金融时间序列为基础，建立一个简短的基准示例：

```
In [44]: import numpy as np
         import pandas as pd
         %matplotlib inline
```

第 6 章中已经说明过，使用 **pandas** 通常可以相当方便地从 Web 读取数据。我们甚至不必使用附加库（如 **urllib**）——几乎一切都发生在后台。下面的代码读取 Microsoft 公司的历史股价数据，并将数据保存在一个 **DataFrame** 对象中：

```
In [45]: url = 'http://ichart.yahoo.com/table.csv?s=MSFT&a=0&b=1&c=2009'
         data = pd.read_csv(url, parse_dates=['Date'])
```

pandas 以列名作为 x 和 y 坐标的参数在。结果如图 14-1 所示：

```
In [46]: data.plot(x='Date', y='Close')
```

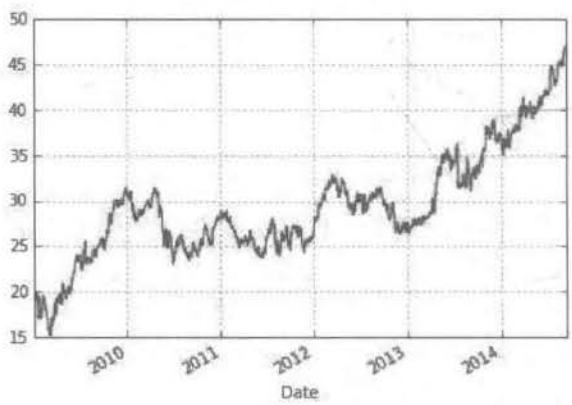


图 14-1 Microsoft 从 2009 年 1 月起的历史股价（matplotlib）

1 关于 matplotlib 交互式图表绘制的更多细节，参见该库的主页（http://www.matplotlib.org/）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 14-1 的图形和图表当然也可以用于 Web 环境。例如，将 **matplotlib** 生成的图表保存为 PNG（可移植网络图形）格式文件并将这些文件加入网站是很简单的。不过，最新的 Web 技术通常还提供交互性，例如*平移*或者*缩放*。

Bokeh（http://bokeh.pydata.org）库的明确目标是为 **Python** 提供现代化的交互式 Web 图表绘制。根据其网站上的说法：

> **Bokeh** 是用于大数据集的 **Python** 交互式可视化库，原生使用最新的 Web 技术。它的目标是以 Protovis/D3 的风格提供优雅、简练的新颖图形构造，同时向瘦客户端提供大数据上的高性能交互性。

上述描述中有 3 个要素值得注意：

大数据集

> 绘制大数据集的图表本身就是一个“绘图问题”。只要想象一下包含 100 万个点的散点图——通常，大部分信息会丢失；**Bokeh** 提供了这方面的内建帮助。

最新 Web 技术

> 一般来说，当今 Web 开发和可视化的首选语言是 **JavaScript**；它是 D3（数据驱动文档，http://d3js.org）等库的基础，**Bokeh** 也以 **JavaScript** 为基础。

高性能交互性

> 在 Web 上，人们习惯于实时交互（想想现代化的浏览器游戏），这在大数据集的可视化和交互时会成为问题；**Bokeh** 也提供了实现这一目标的内建功能。

在基本的层面上，**Bokeh** 的使用和 **matplotlib** 没有什么不同。但是，默认的输出通常不是一个标准的窗口或者 IPython Notebook（这也是一个选项），而是单独的 HTML 文件：

```
In [47]: import bokeh.plotting as bp
In [48]: bp.output_file("../images/msft_1.html", title="Bokeh Example (Static)")
         # use: bp.output_notebook("default")
         # for output within an IPython Notebook
```

在图表绘制上，**Bokeh** 提供了各种丰富的绘图风格，这些风格还在持续地改进。我们从最简单的例子开始，考虑下面的代码，它生成一个和 **pandas/matplotlib** 基准图表类似的线图。结果如图 14-2 所示，除了 x 和 y 坐标之外，其他所有参数都是可选的：

```
In [49]: bp.line(
             data['Date'],
                 # x coordinates
             data['Close'],
                 # y coordinates
             color='#0066cc',
                 # set a color for the line
             legend='MSFT',
                 # attach a legend label
             title='Historical Stock Quotes',
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        # plot title
    x_axis_type='datetime',
        # datetime information on x-axis
    tools = ''
)
bp.show()
```

按照 **matplotlib** 的传统，**Bokeh** 也有一个展厅（http://bokeh.pydata.org/docs/gallery.html/），展示不同的绘图风格。

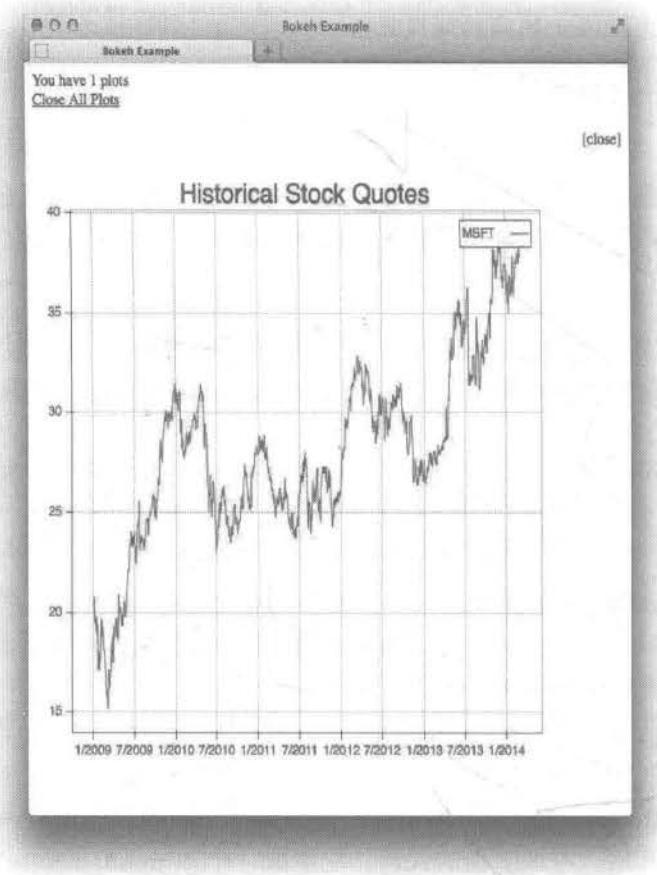


图 14-2　基于 HTML 的 Bokeh 图表截图

14.2.2　交互式图表绘制

下一步是为基于 Web 的图表增加交互性。可用的交互性元素（“工具”）包括：

pan

支持图表平移（就像用摄影机摇拍）；也就是说，相对于固定的画框移动图表（包括 x 和 y 坐标）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

wheel_zoom

使用鼠标滚轮实现图像缩放

box_zoom

用鼠标标记一个方框实现缩放

reset

重置图表的原始/默认视图

previewsave

生成可以保存为 PNG 格式的图表静态（位图）版本

下面的代码展示了这些工具的添加：

```
In [50]: bp.output_file("../images/msft_2.html",
                        title="Bokeh Example (Interactive)")
         bp.line(
             data['Date'],
             data['Close'],
             color='#0066cc',
             legend='MSFT',
             title='Historical Stock Quotes',
             x_axis_type = "datetime",
             tools = 'pan, wheel_zoom, box_zoom, reset, previewsave'
                 # adding a list of interactive tools
         )
         bp.show()
```

这段代码的输出如图 14-3 所示，其中使用了平移函数移动画框中的图表（将此图与图 14-2 比较）。

从原理上说，目前展示的所有功能也可以用 **matplotlib** 实现。实际上，在单独窗口上绘制图表时，**Bokeh** 的交互式工具在 **matplotlib** 中都存在。图 14-4 展示了在单独（**Python** 控制）窗口中显示的图 14-1 中 **pandas** 图表的缩放和平移版本。然而，和 **Bokeh** 不同的是，**matplotlib** 不能将这些功能“导出”到单独的图形文件。[1]

14.2.3 实时图表绘制

前一个小节说明，用 **Bokeh** 生成交互式、基于 Web 的图表是很简单的。但是，**Bokeh** 在实时可视化（例如高频金融数据）上更加出色。因此，本节包含了两个不同实时 API 的例子，一个使用 JSON（JavaScript 对象标记）格式的 FX（外汇）数据，另一个使用

1 matplotlib 能够导出的大部分图形格式本质上是静态的（即位图）。SVG（可缩放矢量图形）格式是一个反例，它可以用 Java Script/.ECMAScript 编程。该库的网站（http://www.matplotlib.org）提供一些这方面的示例。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

以 CSV 文本文件格式交付的日中分时股价数据。除了可视化方面，如何从这些 API 中读出数据也是很有趣的。

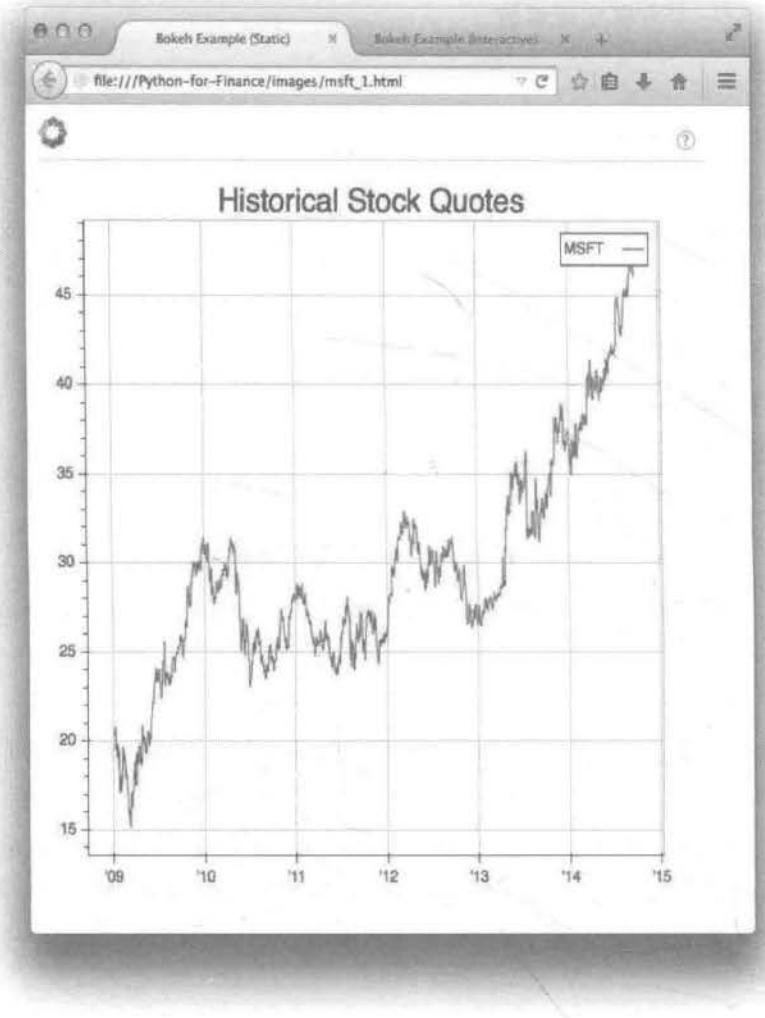


图 14-3　带有交互式元素、基于 HTML 的 Bokeh 图表截图

实时 FX 数据

我们的第一个例子基于 JSON 的外币汇率（FX）API。首先导入：

```
In [51]: import time
         import pandas as pd
         import datetime as dt
         import requests
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

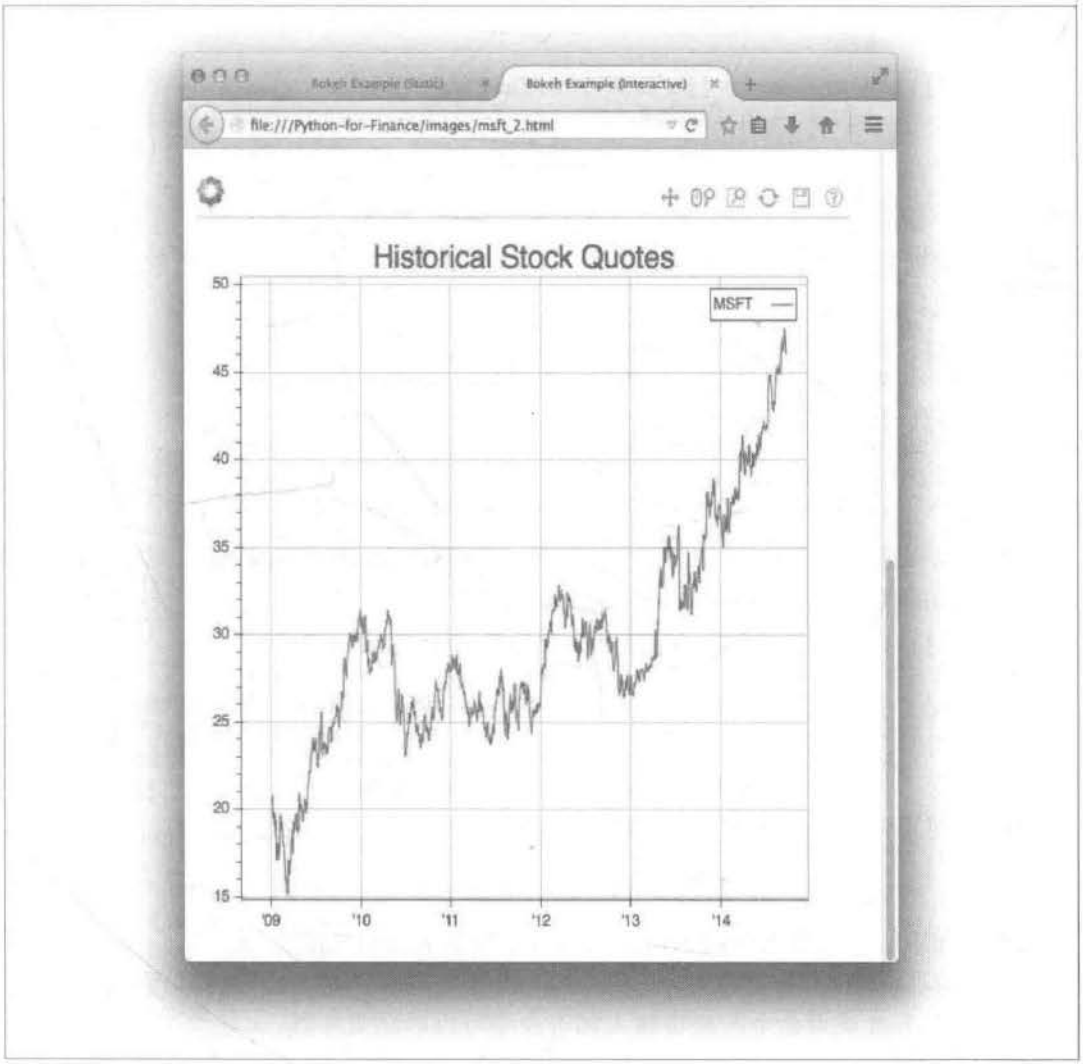


图 14-4　带有交互式元素、基于 pandas/matplotlib 的图表截图

我们使用的 API 来自在线 FX 经纪商 OANDA（http://www.oanda.com）。这家经纪商提供一个 API 沙箱，该沙箱提供和真实汇率相似的随机/虚拟数据。我们的例子基于欧元-美元汇率（参见 API 指南（http://developer.oanda.com/））：

```
In [52]: url = 'http://api-sandbox.oanda.com/v1/prices?instruments=%s'
         # real-time FX (dummy!) data from JSON API
```

我们使用 **requests** 库（http://docs.python-requests.org）连接到 API，这个库的目标是改进与 Web 交互时“人类”所用的接口：

```
In [53]: instrument = 'EUR_USD'
         api = requests.get(url % instrument)
```

打开连接之后，调用连接对象上的 **json** 方法可以读取 JSON 格式的数据：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [54]: data = api.json()
         data
Out[54]: {u'prices': [{u'ask': 1.25829,
            u'bid': 1.2582,
            u'instrument': u'EUR_USD',
            u'time': u'2014-09-29T06:14:34.749878Z'}]}
```

遗憾的是，数据还没有完全以我们希望的格式出现，因此，我们要做一些转换。下面的代码只取列表对象中存储在关键字“prices”下的第一个元素。结果对象是一个标准的 dict 对象：

```
In [55]: data = data['prices'][0]
         data
Out[55]: {u'ask': 1.25829,
          u'bid': 1.2582,
          u'instrument': u'EUR_USD',
          u'time': u'2014-09-29T06:14:34.749878Z'}
```

由于我们以很高的频率收集这样的小数据集，所以使用一个 **DataFrame** 对象存储所有对象。下面的代码初始化对应的 **DataFrame** 对象：

```
In [56]: ticks = pd.DataFrame({'bid': data['bid'],
                               'ask': data['ask'],
                               'instrument': data['instrument'],
                               'time': pd.Timestamp(data['time'])},
                               index=[pd.Timestamp(data['time']),])
           # initialization of ticks DataFrame
In [57]: ticks[['ask', 'bid', 'instrument']]
Out[57]:                                          ask     bid instrument
         2014-09-29 06:14:34.749878+00:00  1.25829  1.2582    EUR_USD
```

实现实时图表绘制需要做两件事情：实时数据收集和实时图表更新。有了 **Bokeh**，就可以使用 **Bokeh** 服务器完成这些工作，**Bokeh** 服务器在新数据出现时处理图表的实时更新。通过 **shell** 或者命令行接口输入如下命令可以启动服务器：

```
$ bokeh-server
```

服务器在后台运行，我们接着实现实时数据更新例程：

```
In [58]: import bokeh.plotting as bp
         from bokeh.objects import Glyph
```

在任何更新进行之前，必须更新一个对象。我们再次绘制一个线图——开始时的数据很少。输出被导入执行代码的 IPython Notebook 中。然而，实际上它再次被重定向到服务器，本例中可以通过 http://localhost:5006/访问：

```
In [59]: bp.output_notebook("default")
         bp.line(ticks['time'], ticks['bid'],
                 x_axis_type='datetime', legend=instrument)
Out[59]: Using saved session configuration for http://localhost:5006/
         To override, pass 'load_from_config=False' to Session
         <bokeh.objects.Plot at 0x7fdb7e1b2e10>
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

我们需要访问当前图表（即最近生成的图表）。调用 **curplot** 函数返回我们所要搜索的对象：

```
In [60]: bp.curplot()
Out[60]: <bokeh.objects.Plot at 0x7fdb7e1b2e10>
```

这个 **Plot** 对象由完成不同绘图任务（比如绘制网格（Grid）或者绘制表示金融数据的线（Glyph））的渲染对象组成。所有渲染对象都保存在列表属性 **renderers** 中：

```
In [61]: bp.curplot().renderers
Out[61]: [<bokeh.objects.DatetimeAxis at 0x7fdbaece6b50>,
          <bokeh.objects.Grid at 0x7fdb7e161190>,
          <bokeh.objects.LinearAxis at 0x7fdb7e161090>,
          <bokeh.objects.Grid at 0x7fdb7e1614d0>,
          <bokeh.objects.BoxSelectionOverlay at 0x7fdb7e161490>,
          <bokeh.objects.BoxSelectionOverlay at 0x7fdb7e161550>,
          <bokeh.objects.Legend at 0x7fdb7e161650>,
          <bokeh.objects.Glyph at 0x7fdb7e161610>]
```

下面的列表推导返回第一个 **Glyph** 类型的渲染对象：

```
In [62]: renderer = [r for r in bp.curplot().renderers
                     if isinstance(r, Glyph)][0]
```

对象的 **glyph** 属性包含 **Glyph** 类型对象——在本例中正如预期，是一个 **Line** 对象：

```
In [63]: renderer.glyph
Out[63]: <bokeh.glyphs.Line at 0x7fdb7e161590>
```

利用这个渲染对象，可以直接访问其数据源：

```
In [64]: renderer.data_source
Out[64]: <bokeh.objects.ColumnDataSource at 0x7fdb7e1b2ed0>
In [65]: renderer.data_source.data
Out[65]: {'x': 2014-09-29 06:14:34.749878+00:00    2014-09-29 06:14:34.749878+00
         :00
          Name: time, dtype: object, 'y': 2014-09-29 06:14:34.749878+00:00    1.
         2582
          Name: bid, dtype: float64}
In [66]: ds = renderer.data_source
```

这就是我们将要使用的对象，在新数据到来时将对其进行更新。下面的 while 循环仅在预先确定的时期运行。在循环中生成新的请求对象并读取 JSON 数据。新数据被附加到现有的 DataFrame 对象。渲染对象的 x 和 y 坐标更新，然后保存到当前会话：

```
In [67]: start = time.time()
         # run for 60 seconds
         while (time.time() - start) < 60:
             data = requests.get(url % instrument).json()
               # connect and read data
             data = dict(data['prices'][0])
               # transform data to dict object
             ticks = ticks.append(pd.DataFrame({'bid': data['bid'],
                                                'ask': data['ask'],
                                                'instrument': data['instrument'],
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                                    'time': pd.Timestamp(data['time'])},
                              index=[pd.Timestamp(data['time']),]))
  # append DataFrame object with new data to existing object
ds.data['x'] = ticks['time']
  # update x coordinates in rendering object
ds.data['y'] = ticks['bid']
  # update y coordinates in rendering object
bp.cursession().store_objects(ds)
  # store data objects
time.sleep(0.1)
  # wait for a bit
```

图 14-5 展示了图表绘制练习的输出——实时绘图的静态快照。这种方法及其底层技术当然有许多有趣的应用领域，既包括目前聚焦于实时、高频数据的金融领域，也包括其他许多领域。

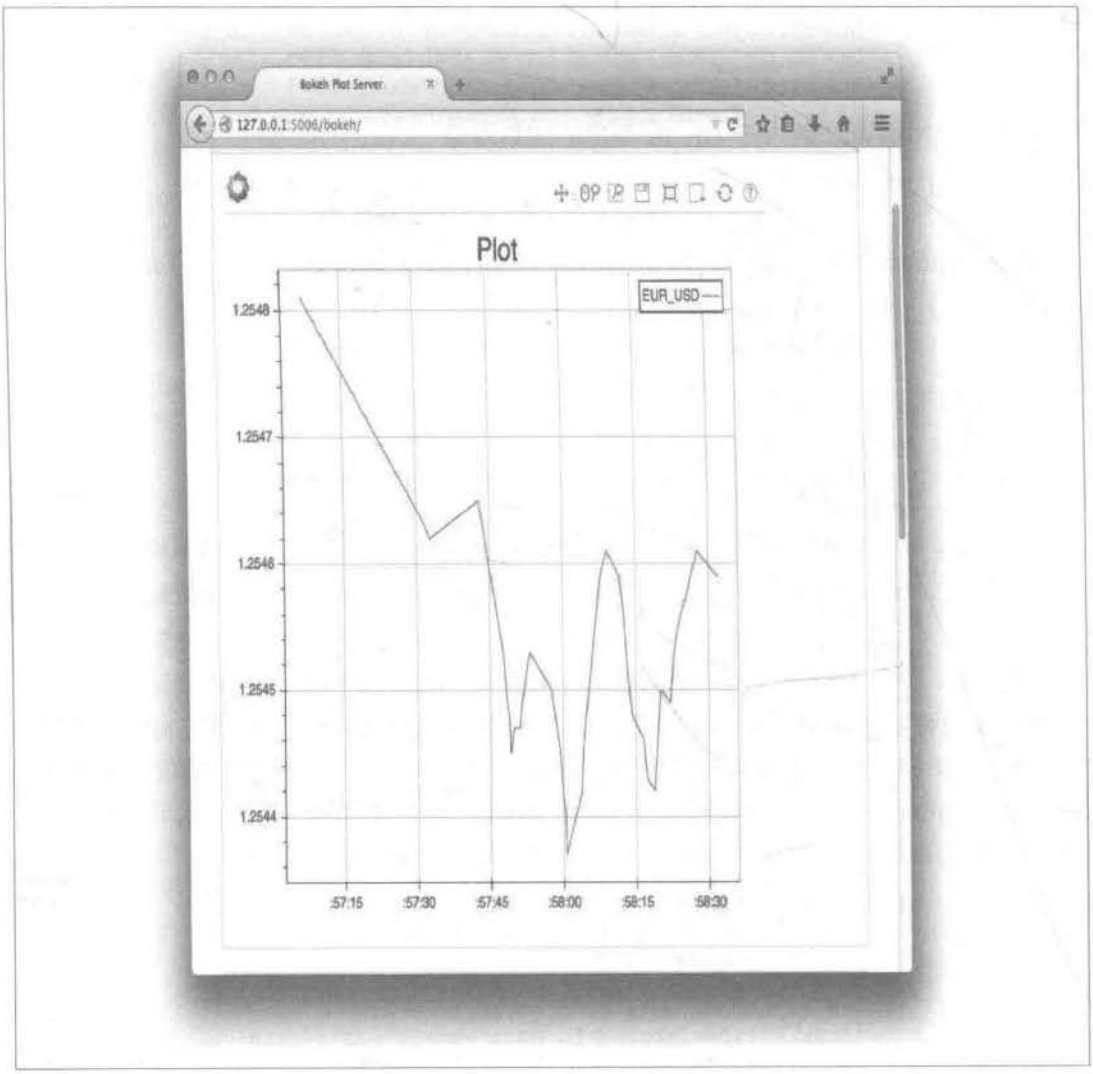


图 14-5　通过 Bokeh 服务器（汇率）制作的实时 Bokeh 图表截图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

实时股票报价

第二个例子使用实时的高频股价数据。首先，确保输出的导向正确（在本例中，导出到 **Bokeh** 服务器进行实时图表绘制）：

```
In [68]: bp.output_notebook("default")
Out[68]: Using saved session configuration for http://localhost:5006/
         To override, pass 'load_from_config=False' to Session
```

对我们下面要使用的数据源和 API，第 6 章已经提供了一个相关的例子。这是挪威在线经纪商 **Netfonds**（http://www.netfonds.no）提供的日间实时股价数据 API。API 和 Web 服务分别使用如下的基本 URL 格式：

```
In [69]: url1 = 'http://hopey.netfonds.no/posdump.php?'
         url2 = 'date=%s%s%s&paper=%s.O&csv_format=csv'
         url = url1 + url2
```

提供日期信息和感兴趣的股票代码，自定义 URL：

```
In [70]: today = dt.datetime.now()
         y = '%d' % today.year
           # current year
         m = '%02d' % today.month
           # current month, add leading zero if needed
         d = '%02d' % (today.day)
           # current day, add leading zero if needed
         sym = 'AAPL'
           # Apple Inc. stocks
In [71]: y, m, d, sym
Out[71]: ('2014', '09', '29', 'AAPL')
In [72]: urlreq = url % (y, m, d, sym)
         urlreq
Out[72]: 'http://hopey.netfonds.no/posdump.php?date=20140929&paper=AAPL.O&csv_fo
         rmat=csv'
```

有了正确的 URL 字符串，读取数据只需要一行代码：

```
In [73]: data = pd.read_csv(urlreq, parse_dates=['time'])
           # initialize DataFrame object
```

在上一个例子中，我们已经熟悉了以下的细节，首先初始化图表：

```
In [74]: bp.line(data['time'], data['bid'],
                 x_axis_type='datetime', legend=sym)
           # intial plot
Out[74]: <bokeh.objects.Plot at 0x7f92bedc8dd0>
```

其次，选择渲染对象：

```
In [75]: renderer = [r for r in bp.curplot().renderers
                     if isinstance(r, Glyph)][0]
         ds = renderer.data_source
```

第三步是一个 **while** 循环，每次循环都更新金融数据和图表：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [76]: start = time.time()
         while (time.time() - start) < 60:
             data = pd.read_csv(urlreq, parse_dates=['time'])
             data = data[data['time'] > dt.datetime(int(y), int(m), int(d),
                                                    10, 0, 0)]
               # only data from trading start at 10am
             ds.data['x'] = data['time']
             ds.data['y'] = data['bid']
             ds._dirty = True
             bp.cursession().store_objects(ds)
             time.sleep(0.5)
```

图 14-6 展示了输出结果——遗憾的是，还是只有实时图表的静态快照。

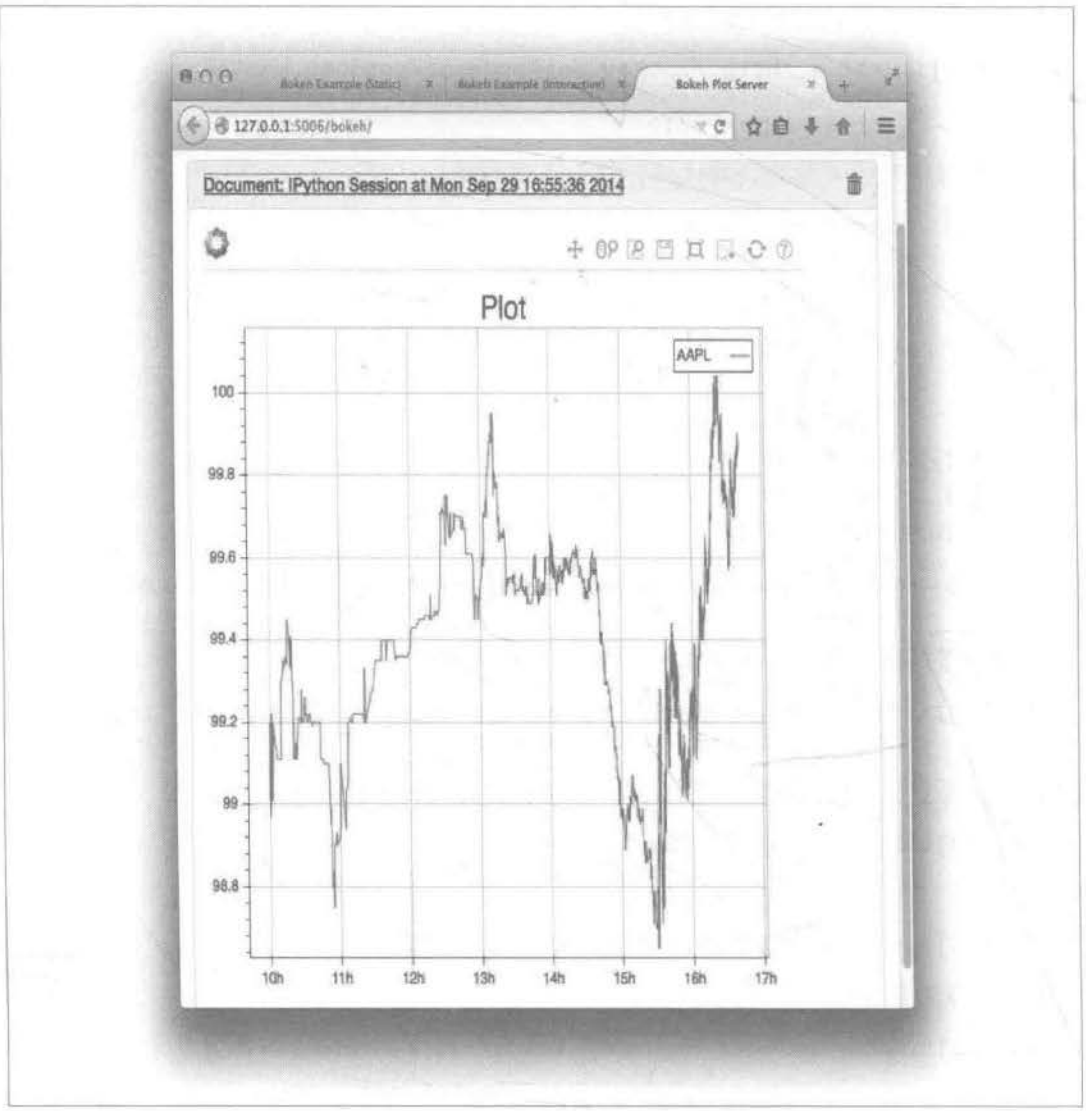


图 14-6　通过 Bokeh 服务器的实时 Bokeh 绘图截图（股票报价）

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

14.3 快速 Web 应用

如果将 Python 世界分成几个大陆，那么可能有*科学和金融学*大陆、*系统管理大陆*和 *Web 开发*大陆等。从这个概念上讲，Web 开发大陆可能是最大的一个，它有最多的人口（开发者）和房屋（应用程序）。

Python 作为 Web 开发的强大工具的原因之一是有很多不同的高级全栈式框架。正如 Python 网页（http://wiki.python.org/moin/WebFrameworks）所陈述的：

> Web 应用可以组合基本 HTTP 应用服务器、存储机制（如数据库）、模板引擎、请求调度器、验证模块和 AJAX 工具包。这些组件可能是单独提供或者和高级框架一起提供的。

最流行的框架有：

- **Django**（http://www.djangoproject.com）
- **Flask**（http://flask.pocoo.org）
- **Pyramid/Pylons**（http://www.pylonsproject.org）
- **TurboGears**（http://www/turbogears.org）
- **Zope**（http://zope2.zope.org）

可以肯定地说，没有哪个框架能够很好地适用于每个人及每个不同的应用类型[1]。所有框架都有自身的优势（有时候是劣势），选择何种框架很多时候是（对架构、风格、语法、API 等）偏好的问题。

Flask 是最近快速流行起来的框架之一。我们在此使用这个框架主要是因为：

Python 风格

> 以 **Flask** 为框架进行的应用开发有着真正的 **Python** 风格，由后台进程负责许多与 Web 相关的细节。

紧凑性

> 该框架不太复杂，因而可以很快地学会；它主要以其他地方广泛使用的标准组件和库作为基础。

文档

> **Flask** 的文档很好，在本书编著的时候，既有在线的 HTML 版本，也有大约 300

1 Python Web 框架的进一步信息参见 http://wiki.python.org/moin/WebFrameworks.Pyrhon 内容管理系统（CMS）的概述参见 http://wiki.python.org/moin/ContentManagementSystems。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

页的 PDF 版本[1]。

Flask 主要依赖以下两个库：

- Jinja2（http://jinja.pocoo.org/docs/），用于 **Python** 的 Web 模板语言/引擎
- Werkzeug（http://werkzeug.pocoo.org），用于 **Python** 的 **WSGI**（Web 服务器网管接口）

14.3.1 交易者的聊天室

我们现在将研究一个示例应用 Tradechat，该示例是一个交易者的聊天室，本质上依赖于 Flask 文档中教程所使用的示例，但是包含了几处变化，添加了一些更深入的功能[2]。

我们的基本思路是构建一个供交易者注册的基于 Web 应用程序，提供一个可以交换思路和谈论市场的中心聊天室。主屏幕允许登录用户输入文本，这些文本在按下某个按钮之后会添加到时间轴，表明添加评论的用户和时间。主屏幕还会以降序（从最新到最旧）显示所有历史条目。

14.3.2 数据建模

我们首先生成必要的目录。主目录将是 **tradechat**，此外，我们至少需要两个子目录 **static** 和 **templates**（按照 Flask 的惯例）：

```
$ mkdir tradechat
$ mkdir tradechat/static
$ mkdir tradechat/templates
```

为了存储数据——注册用户的数据和聊天室中的评论——我们使用 **SQLites**（参见 http://www.sqlite.org 和 http://docs.python.org/2/library/sqlite3.html）数据库。需要的两个不同表按照例 14-1 介绍的 SQL 模式生成，在此不讨论其细节。应该以文件名 **tables.sql** 将其保存在应用程序主目录 **tradechat** 之下。

例 14-1　在 SQLite3 中生成表的 SQL 模式

```
drop table if exists comments;
create table comments (
  id integer primary key autoincrement,
  comment text not null,
  user text not null,
```

1　尽管 Flask 框架相当新颖（2010 年开始），但是已经有关于它的书籍面世，参见 Grinberg（2014）。——原注

2　文档中的示例应用称为 Flaskr（https://github.com/mitsuhiko/flask/tree/master/examples/flaskr/），介绍了一个微博应用。我们的例子多多少少混合了 Flaskr 和 Mint twit（https://github.com/mitsuhiko/flask/tree/master/examples/minitwit）。——另一个 Flask 示例应用，类似于简单的 Twitter 克隆版本。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
  time text not null
);

drop table if exists users;
create table users (
  id integer primary key autoincrement,
  name text not null,
  password text not null
);
```

14.3.3 Python 代码

SQL 模式是下面的 **Python/Flask** 应用的主要输入。我们将一步一步地观察完整 **Python** 脚本的各个元素，该脚本存储在主目录 **tradechat** 下的 **tradechat.py** 文件中。

导入和数据库准备

首先，我们需要导入 **Flask** 中的几个库和一些主要函数，直接导入函数可以缩短代码，增加可读性：

```
# Tradechat
#
# A simple example for a web-based chat room
# based on Flask and SQLite3.
#

import os
import datetime as dt
from sqlite3 import dbapi2 as sqlite3
from flask import Flask, request, session, g, redirect, url_for, abort, \
     render_template, flash
```

整个应用程序围绕一个 **Flask** 对象运作，这个对象是框架主类的一个实例。在脚本执行时（如从 **Shell** 输入如下命令），用 **name** 实例化该类，使该对象继承应用程序的名称（**main**）：

```
# the application object from the main Flask class
app = Flask(__name__)
```

下一步是对新的应用程序对象进行一些配置。特别是，需要提供一个数据库文件名：

```
# override config from environment variable
app.config.update(dict(
    DATABASE=os.path.join(app.root_path, 'tradechat.db'),
      # the SQLite3 database file ("TC database")
    DEBUG=True,
    SECRET_KEY='secret_key',
      # use secure key here for real applications
))
app.config.from_envvar('TC_SETTINGS', silent=True)
  # do not complain if no config file exists
```

提供数据库路径和文件名之后，**connect_db** 函数连接到服务器并返回连接对象：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
def connect_db():
    ''' Connects to the TC database.''
    rv = sqlite3.connect(app.config['DATABASE'])
    rv.row_factory = sqlite3.Row
    return rv
```

Flask 使用 g 对象来存储全面数据和其他对象。例如，为大量用户提供服务的 Web 应用程序可以使用它来定期连接数据库。每当需要执行一个数据库操作时，再实例化一个连接对象的方法太过低效。我们可以在 g 对象的 sqlite_db 属性中存储这样一个连接对象。get_db 函数就使用了这种方法，因为只有当 g 对象中没有存储连接对象时，才打开一个 new 数据库连接。

```
def get_db():
    ''' Opens a new connection to the TC database. ''
    if not hasattr(g, 'sqlite_db'):
        # open only if none exists yet
        g.sqlite_db = connect_db()
    return g.sqlite_db
```

我们至少必须在数据库中创建表一次。第二次调用 init_db 函数将删除之前保存在数据库中保存的所有信息（根据使用的 SQL 模式）

```
def init_db():
    ''' Creates the TC database tables.''
    with app.app_context():
        db = get_db()
        with app.open_resource('tables.sql', mode='r') as f:
            db.cursor().executescript(f.read())
                # creates entries and users tables
        db.commit()
```

close_db 函数在 **g** 对象中存在数据库连接时将其关闭。在第一次（肯定也不是最后一次），我们会遇到一个 **Flask** 函数修饰符**@app.teardown_appcontext**，这个修饰符确保对应的函数在应用程序上下文卸载时调用——粗略地说，在应用程序的执行被用户或者错误/异常终止时：

```
@app.teardown_appcontext
def close_db(error):
    ''' Closes the TC database at the end of the request. ''
    if hasattr(g, 'sqlite_db'):
        g.sqlite_db.close()
```

核心功能

现在，在数据库基础架构上，可以继续实现应用程序的核心函数。首先，必须定义连接到应用程序主页/首页时发生什么。为此，我们使用 **Flask** 函数修饰符**@app.route("/")**。这样修饰的函数将在主页链接建立时调用。**show_entries** 函数本质上建立一个数据库连接，检索目前为止张贴的所有评论（可能一个都没有，也可能很多），并将其发送给基于模板的渲染引擎，根据模板和提供的数据返回一个 HTML 文档（很快就会详细介绍

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

模板的部分）：

```
@app.route('/')
def show_entries():
    ''' Renders all entries of the TC database. '''
    db = get_db()
    query = 'select comment, user, time from comments order by id desc'
    cursor = db.execute(query)
    comments = cursor.fetchall()
    return render_template('show_entries.html', comments=comments)
```

我们应该只允许注册用户在聊天室中发表评论。因此，必须为用户提供注册功能。为此，从技术上，我们必须允许应用程序渲染的供用户访问的 HTML 使用 POST 方法。注册时，用户必须提供*用户名*和*密码*。否则，将报告错误。register 函数只应该视为简单的演示，它缺乏现实世界应用程序中很重要的一些成分，例如检查用户名是否存在以及密码的加密（这里的密码用明文存储）。一旦用户成功注册，他们的状态自动更改为 logged_in，并通过 redirect(url_for("show_entries"))重定向到主页：

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    ''' Registers a new user in the TC database. '''
    error = None
    if request.method == 'POST':
        db = get_db()
        if request.form['username'] == '' or request.form['password'] == '':
            error = 'Provide both a username and a password.'
            # both fields have to be nonempty
        else:
            db.execute('insert into users (name, password) values (?, ?)',
                       [request.form['username'], request.form['password']])
            db.commit()
            session['logged_in'] = True
            # directly log in new user
            flash('You were sucessfully registered.')
            app.config.update(dict(USERNAME=request.form['username']))
            return redirect(url_for('show_entries'))
    return render_template('register.html', error=error)
```

对于这样的 Web 应用，可能有些“回头客”不需要或者不想要注册新帐户。因此，我们需要提供一个现有账户的登录表单，这就是 **login** 函数的功能，该函数功能和 **register** 很类似：

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    ''' Logs in a user. '''
    error = None
    if request.method == 'POST':
        db = get_db()
        try:
            query = 'select id from users where name = ? and password = ?'
            id = db.execute(query, (request.form['username'],
                                    request.form['password'])).fetchone()[0]
              # fails if record with provided username and password
              # is not found
            session['logged_in'] = True
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            flash('You are now logged in.')
            app.config.update(dict(USERNAME=request.form['username']))
            return redirect(url_for('show_entries'))
        except:
            error = 'User not found or wrong password.'
    return render_template('login.html', error=error)
```

一旦用户注册或者再次登录，他们就应该可以在聊天室里添加评论。**add_entry** 函数保存评论文本、评论用户的用户名以及准确的发布时间（精确到秒）。该函数还检查用户是否登录：

```
@app.route('/add', methods=['POST'])
def add_entry():
    """ Adds entry to the TC database. """
    if not session.get('logged_in'):
        abort(401)
    db = get_db()
    now = dt.datetime.now()
    db.execute('insert into comments (comment, user, time) values (?, ?, ?)',
            [request.form['text'], app.config['USERNAME'], str(now)[:-7]])
    db.commit()
    flash('Your comment was successfully added.')
    return redirect(url_for('show_entries'))
```

最后，用户必须注销才能结束会话。这就是函数 **logout** 支持的功能：

```
@app.route('/logout')
def logout():
    """ Logs out the current user. """
    session.pop('logged_in', None)
    flash('You were logged out')
    return redirect(url_for('show_entries'))
```

如果想将 **Python** 脚本作为独立应用运行，应该添加如下代码行，确保服务器启动，为应用提供服务：

```
# main routine
if __name__ == '__main__':
    init_db()  # comment out if data in current
               # TC database is to be kept
    app.run()
```

将上述部件组合起来，就得到如例 14-2 所示的 Python 脚本。

例 14-2　表现 Tradechat 应用程序核心的 Python 脚本

```
# Tradechat
#
# A simple example for a web-based chat room
# based on Flask and SQLite3.
#
import os
import datetime as dt
from sqlite3 import dbapi2 as sqlite3
from flask import Flask, request, session, g, redirect, url_for, abort, \
     render_template, flash
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
# the application object from the main Flask class
app = Flask(__name__)

# override config from environment variable
app.config.update(dict(
    DATABASE=os.path.join(app.root_path, 'tradechat.db'),
      # the SQLite3 database file ("TC database")
    DEBUG=True,
    SECRET_KEY='secret_key',
      # use secure key here for real applications
))
app.config.from_envvar('TC_SETTINGS', silent=True)
  # do not complain if no config file exists

def connect_db():
    "' Connects to the TC database."'
    rv = sqlite3.connect(app.config['DATABASE'])
    rv.row_factory = sqlite3.Row
    return rv


def get_db():
    "' Opens a new connection to the TC database. "'
    if not hasattr(g, 'sqlite_db'):
        # open only if none exists yet
        g.sqlite_db = connect_db()
    return g.sqlite_db

def init_db():
    "' Creates the TC database tables."'
    with app.app_context():
        db = get_db()
        with app.open_resource('tables.sql', mode='r') as f:
            db.cursor().executescript(f.read())
               # creates entries and users tables
        db.commit()


@app.teardown_appcontext
def close_db(error):
    "' Closes the TC database at the end of the request. "'
    if hasattr(g, 'sqlite_db'):
        g.sqlite_db.close()


@app.route('/')
def show_entries():
    "' Renders all entries of the TC database. "'
    db = get_db()
    query = 'select comment, user, time from comments order by id desc'
    cursor = db.execute(query)
    comments = cursor.fetchall()
    return render_template('show_entries.html', comments=comments)


@app.route('/register', methods=['GET', 'POST'])
def register():
    "' Registers a new user in the TC database. "'
    error = None
    if request.method == 'POST':
        db = get_db()
        if request.form['username'] == '' or request.form['password'] == '':
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            error = 'Provide both a username and a password.'
            # both fields have to be nonempty
        else:
            db.execute('insert into users (name, password) values (?, ?)',
                       [request.form['username'], request.form['password']])
            db.commit()
            session['logged_in'] = True
            # directly log in new user
            flash('You were sucessfully registered.')
            app.config.update(dict(USERNAME=request.form['username']))
            return redirect(url_for('show_entries'))
    return render_template('register.html', error=error)


@app.route('/login', methods=['GET', 'POST'])
def login():
    ''' Logs in a user. '''
    error = None
    if request.method == 'POST':
        db = get_db()
        try:
            query = 'select id from users where name = ? and password = ?'
            id = db.execute(query, (request.form['username'],
                                    request.form['password'])).fetchone()[0]
              # fails if record with provided username and password
              # is not found
            session['logged_in'] = True
            flash('You are now logged in.')
            app.config.update(dict(USERNAME=request.form['username']))
            return redirect(url_for('show_entries'))
        except:
            error = 'User not found or wrong password.'
    return render_template('login.html', error=error)

@app.route('/add', methods=['POST'])
def add_entry():
    ''' Adds entry to the TC database. '''
    if not session.get('logged_in'):
        abort(401)
    db = get_db()
    now = dt.datetime.now()
    db.execute('insert into comments (comment, user, time) values (?, ?, ?)',
            [request.form['text'], app.config['USERNAME'], str(now)[:-7]])
    db.commit()
    flash('Your comment was successfully added.')
    return redirect(url_for('show_entries'))


@app.route('/logout')
def logout():
    ''' Logs out the current user. '''
    session.pop('logged_in', None)
    flash('You were logged out')
    return redirect(url_for('show_entries'))

# main routine
if __name__ == '__main__':
    init_db()  # comment out if data in current
               # TC database is to be kept
    app.run()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

安全性

尽管本节的示例展示了用 Flask 开发的 Python Web 应用基本设计，但是它没有考虑安全性问题，这是 Web 应用中至关重要的一个方面。不过，Flask 和其他 Web 框架都提供了完整的工具集，可以处理典型的安全性问题。

14.3.4 模板

本质上，Flask 模板（**Jinja2**）和 **Python** 中的简单字符串替换类似：有一个基本的字符串指明替换的位置、替换的内容，还有一些将要插入字符串对象的数据。考虑如下的例子：

```
In [77]: '%d, %d, %d' % (1, 2, 3)
Out[77]: '1, 2, 3'
In [78]: '{}, {}, {}'.format(1, 2, 3)
Out[78]: '1, 2, 3'
In [79]: '{}, {}, {}'.format(*'123')
Out[79]: '1, 2, 3'
```

生成 HTML 页面的模板与此很类似。最大的差异是字符串对象“类似”HTML 文档（或者文档的一部分），并且有用于替代的命令，以及其他一些内容，例如渲染模板时的流程控制（如 for 循环）。遗漏的信息在渲染过程中添加，就像我们上一个例子中在字符串对象中添加整数一样。现在考虑下面的字符串对象，它包含标准 HTML 代码和一些模板专用代码：

```
In [80]: templ = """<!doctype html>
             Just print out <b>numbers</b> provided to the template.
             <br><br>
             {% for number in numbers %}
               {{ number }}
             {% endfor %}
         """
```

迄今为止，这还仅是一个字符串对象。必须根据它生成一个 **Jinja2** 模板（Template）对象，才能继续：

```
In [81]: from jinja2 import Template
In [82]: t = Template(templ)
```

这个模板对象有一个 **render** 方法，根据模板和一些输入值制作 HTML 代码——在本例中，是通过参数 **Numbers** 输入的一些数字：

```
In [83]: html = t.render(numbers=range(5))
```

HTML 代码仍然是一个字符串对象：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [84]: html
Out[84]: u'<!doctype html>\n  Just print out <b>numbers</b> provided to the temp
         late.\n  <br><br>\n  \n    0\n  \n    1\n  \n    2\n  \n    3\n  \n
         4\n  '
```

包含 HTML 代码的这种对象可以在 IPython Notebook 中渲染：

```
In [85]: from IPython.display import HTML
         HTML(html)
Out[85]: <IPython.core.display.HTML at 0x7fdb7e1eb890>
```

当然，模板所涉及的内容远非这个简单例子可以说明的（例如，继承）。更多细节可以在 http://jinja.pocoo.org 上找到。但是，Tradechat 应用的模板已经包含了一些重要特征。具体地说，我们需要如下模板：

layout.html

定义其他模板可以继承的基本布局

register.html

用于用户注册页面的模板

login.html

用户登录的对应模板

show_entries.html

显示聊天室中评论的主页，如果用户登录，还显示撰写和发表评论的文本字段

这些文件必须保存在 **templates** 目录中——这是使用 Flask 时的默认模板（子）目录。

例 14-3 展示了包含基本布局和一些元信息（如网站标题）的模板。这是供所有其他模板继承的模板。

例 14-3　Tradechat 应用基本布局所用的模板

```
<!doctype html>
<title>Tradechat</title>
<link rel=stylesheet type=text/css
     href="{{ url_for('static', filename='style.css') }}">
<div class=page>
  <h1>Tradechat</h1>
  <div class=metanav>
  {% if not session.logged_in %}
    <a href="{{ url_for('login') }}">log in</a><br>
    <a href="{{ url_for('register') }}">register</a>
  {% else %}
    <a href="{{ url_for('logout') }}">log out</a>
  {% endif %}
  </div>
  {% for message in get_flashed_messages() %}
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    <div class=flash>{{ message }}</div>
  {% endfor %}
  {% block body %}{% endblock %}
</div>
```

图 14-7 展示了第一次启动应用程序之后的主页截图。没有任何用户注册（当然也就没有用户登录），也没有发表任何评论。

图 14-7 Tradechat 的“空白”首页

例 14-4 提供用户注册页面的模板代码。模板中使用的表单允许用户通过 POST 方法向页面提供信息。

例 14-4 Tradechat 用户注册模板

```
{% extends "layout.html" %}
{% block body %}
  <h2>Register</h2>
  {% if error %}<p class=error><strong>Error:</strong> {{ error }}{% endif %}
  <form action="{{ url_for('register') }}" method=post>
    <dl>
      <dd><font size="-1">Username</font>
      <dd><input type=text name=username>
      <dd><font size="-1">Password</font>
      <dd><input type=password name=password>
      <dd><input type=submit value=Register>
    </dl>
  </form>
{% endblock %}
```

图 14-8 展示了注册页面的截图。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

图 14-8　Tradechat 注册页面截图

例 14-5 中展示的登录页面模板代码和注册页面的代码很相似。同样，用户可以通过一个表达提供登录信息。

例 14-5　Tradechat 用户登录模板

```
{% extends "layout.html" %}
{% block body %}
  <h2>Login</h2>
  {% if error %}<p class=error><strong>Error:</strong> {{ error }}{% endif %}
  <form action="{{ url_for('login') }}" method=post>
    <dl>
      <dd><font size="-1">Username</font>
      <dd><input type=text name=username>
      <dd><font size="-1">Password</font>
      <dd><input type=password name=password>
      <dd><input type=submit value=Login>
    </dl>
  </form>
{% endblock %}
```

图 14-9 中所示的登录页面不仅看上去和注册页面很类似，而且提供的主要功能也相同。

最后，例 14-6 提供了主页的模板代码。这个模板主要完成两项工作：

启用评论

如果用户登录，显示一个文本字段和一个 Post 按钮，用于发表评论。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

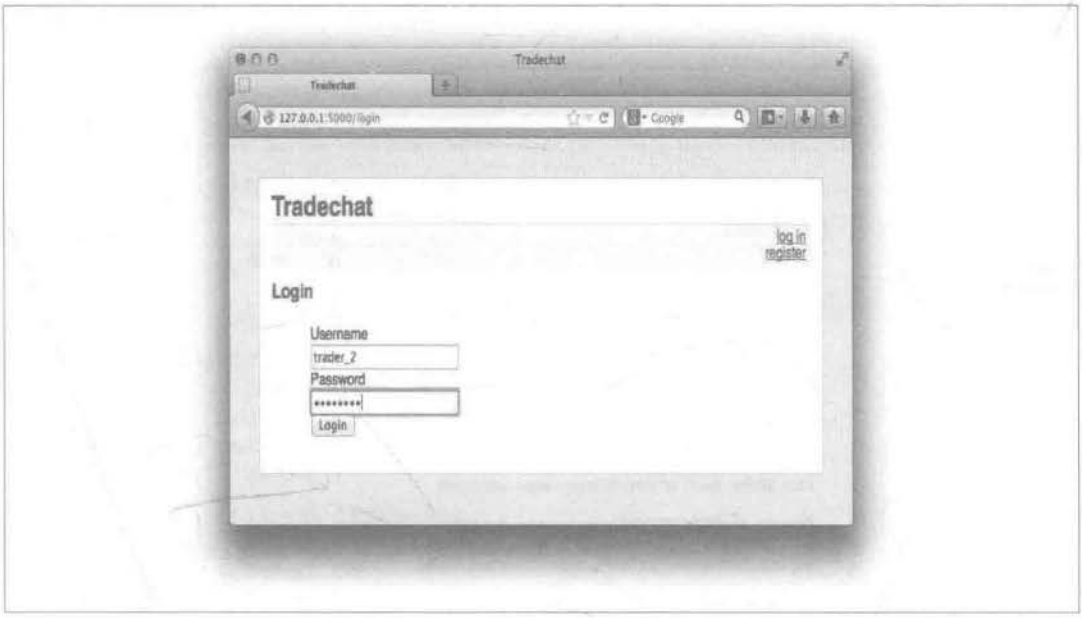


图 14-9　Tradechat 登录页面截图

显示评论

数据库中找到的所有评论以日期的倒序显示（最新的先显示，最旧的后显示）。

例 14-6　具有聊天室评论功能的 Tradechat 主页模板

```
{% extends "layout.html" %}
{% block body %}
  {% if session.logged_in %}
    <form action="{{ url_for('add_entry') }}" method=post class=add-comment>
      <dl>
        <dd>What's up?
        <dd><textarea name=text rows=3 cols=40></textarea>
        <dd><input type=submit value=Post>
      </dl>
    </form>
  {% endif %}
  <ul class=comments>
  {% for comment in comments %}
    <li>{{ comment.comment|safe }}
         <font size="-2">({{ comment.user }} @ {{ comment.time }})</font>
  {% else %}
    <li><em>No comments so far.</em>
  {% endfor %}
  </ul>
{% endblock %}
```

一旦用户登录并发表一些评论，主页显示文本字段和 Post 按钮，以及保存在数据库中的所有评论（参见图 14-10）。

从某种意义上，仅仅展示屏幕截图和模板是欺骗行为。在上述组合中遗漏了样式信息。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

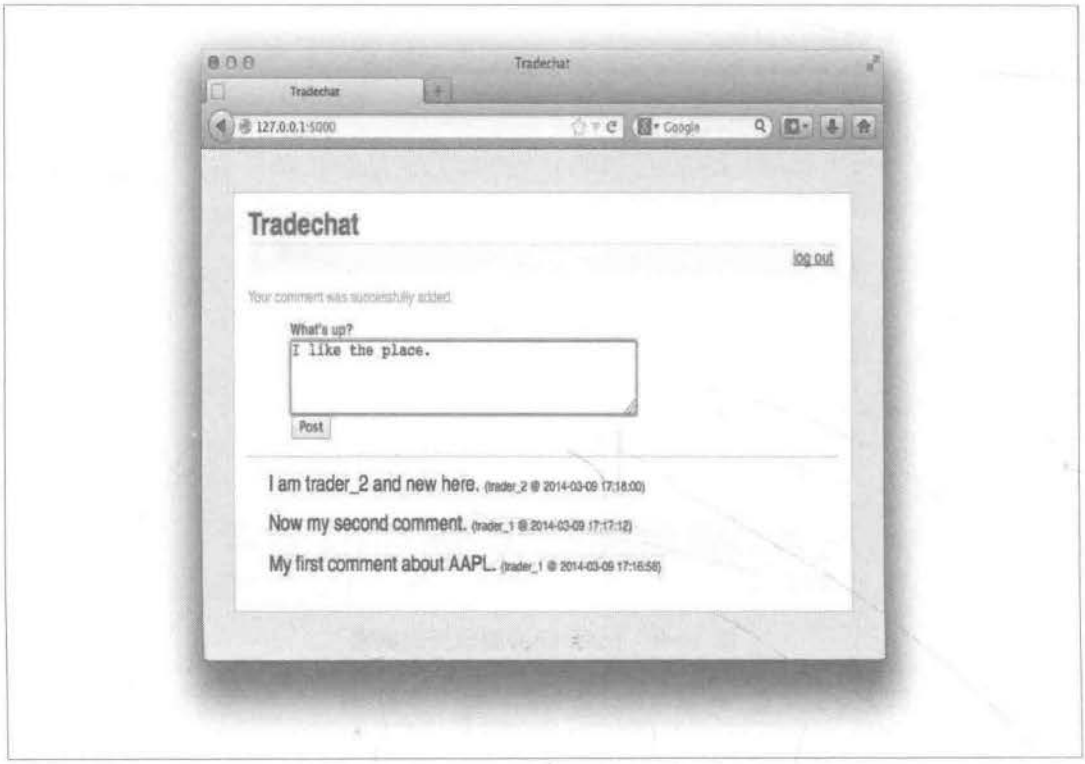


图 14-10　Tradechat 主页截图

14.3.5　样式化

当今网页和基于 Web 应用程序的样式化标准是 CSS（层叠样式表）。如果仔细观察单独的模板，就会发现许多地方有 **class=comments** 或者 **class=add-comment** 之类的参数。没有对应的 CSS 文件，这种参数化本质上是没有意义的。

因此，我们来看看保存在 **static**（子）目录中的 **style.css** 文件，该文件内容如例 14-7 所示。在文件中仍然可以看到前面提到的参数（**comments**、**add-comment**）。你还会找到对标准 HTML 标记的引用，如用于最高级标题的 h1。在自定义类名（如 **comments**）或者标准标记（如 h1）之后提供的所有信息定义或者改变相关对象的特定样式元素（例如，字体和大小）。

这些样式信息是定义 Tradechat 应用程序的最后成分，例如，它可以解释为什么“Tradechat”标题会显示为蓝色——原因是 **a, h1, h2 { color: #0066cc; }**)这一行代码。

例 14-7　Tradechat 应用的 CSS 样式表

```
body { font-family: sans-serif; background: #eee; }
a, h1, h2 { color: #0066cc; }
h1, h2 { font-family: 'Helvetica', sans-serif; margin: 0; }
h1 { font-size: 1.4em; border-bottom: 2px solid #eee; }
h2 { font-size: 1.0em; }
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
.page { margin: 2em auto; width: 35em; border: 1px solid #ccc;
                padding: 0.8em; background: white; }
.comments { list-style: none; margin: 0; padding: 0; }
.comments li { margin: 0.8em 1.2em; }
.comments li h2 { margin-left: -1em; }
.add-comment { color: #0066cc; font-size: 0.7em; border-bottom: 1px solid #ccc; }
.add-comment dl { font-weight: bold; }
.metanav { text-align: right; font-size: 0.8em; padding: 0.3em;
                margin-bottom: 1em; background: #fafafa; }
.flash { color: #b9b9b9; font-size: 0.7em; }
.error { color: #ff4629; font-size: 0.7em; padding: 0.5em; }
```

如果你跟着本书完成了每个步骤，**tradechat** 目录现在应该包含下面列出的文件：

```
In [86]: import os
         for path, dirs, files in os.walk('../python/tradechat'):
           print path
           for f in files:
             print f

Out[86]: ../python/tradechat
         tables.sql
         tradechat.db
         tradechat.py
         ../python/tradechat/static
         style.css
         ../python/tradechat/templates
         layout.html
         login.html
         register.html
         show_entries.html
```

现在，可以从 shell 运行主脚本，启动应用程序：

```
$ python tradechat.py
```

然后，你可以通过 Web 浏览器在 http://127.0.0.1:5000 访问该应用。单击 **register** 可以注册用户，提供用户名和密码之后，就可以发表评论。

14.4 Web 服务

本章的最后一个主题——也是非常有趣和重要的主题——是 Web 服务。Web 服务提供简单高效的手段，通过 Web 协议访问基于服务器的功能。例如，流量最高的 Web 服务之一是 Google 搜索功能。我们习惯于访问 http://www.google.com 并在网站上提供的搜索文本输入框中输入某些感兴趣的词语。但是，在按下回车或者“搜索”按钮之后发生的是页面将（来自搜索框和你的个人偏好设置的）所有信息转换为有些复杂的 URL。

例如，这种 URL 可能采取 http://www.google.de/search?num=5&q=yves+python 的形式。单击该链接并将其复制到浏览器中时，Google 搜索返回搜索引擎认为最匹配所提供词语（**q=Yves+Python**）的 5 个搜索结果（**num=5**）。然后，浏览器显示类似图 14-11 的页面。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

使用 Web 服务，任何面向数据和交易的金融服务可以通过 Web 技术提供。例如，雅虎财经和 Google 财经通过这种 Web 服务方式提供历史股价信息。更复杂的服务（如衍生品定价和风险分析）也可以通过这类服务提供（例如，基于 Web 的分析解决方案 DEXISION；参见 http://derivatives-analytics.com）。下面的例子阐述了期权定价服务的实现。

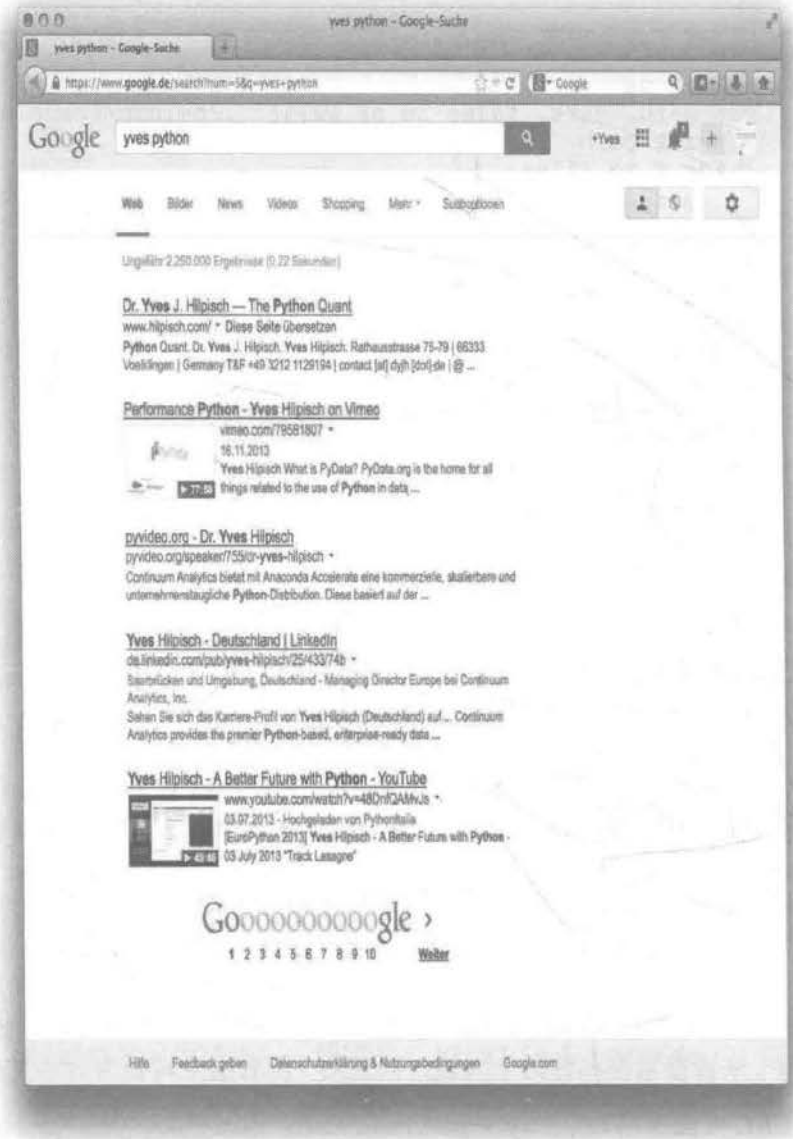


图 14-11　通过 Web 服务得到的 Google 搜索结果截图

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

14.4.1 金融模型

本节，我们将实现一个估算波动率期权（例如，某个波动率指数期权）价值的 Web 服务。我们使用 Gruenbichler 和 Longstaff（1996）提出的一个模型，他们通过公式 14-1 提供的平方根扩散，以直接方式建立了波动率过程模型（例如，波动率指数过程）。该过程因在波动率建模中展现的方便特性（如正性和均值回归）而闻名[1]。

公式 14-1　用于波动率建模的平方根扩散

$$dV_t = \kappa_V(\theta_V - V_t)dt + \sigma_V\sqrt{V_t}dZ \qquad (14\text{-}1)$$

公式 14-1　中的变量和参数含义如下：

V_t

波动率指数（例如 VSTOXX）在时间 t 的价值。

θ_V

波动率指数长期均值。

κ_V

V_t 恢复到 θ 的速度

Σ_V

波动率的波动率（Vol-vol）

θ_V，κ_V 和 Σ_V

假定为恒定的正数

Z_t

标准布朗运动

根据这一模型，Gruenbichler 和 Longstaff（1996）得出了公式 14-2 中的欧式看涨期权估值方程式。在这个方程式中，D(T)是对应的折现因子。参数 ζ 表示波动性风险的预期溢价，而 Q（•）是互补非中心 X^2 分布。

公式 14-2　Gruenbichler 和 Longstaff（1996）看涨期权方程式

$$\begin{aligned} C(V_0,K,T) &= D(T)\cdot e^{-\beta T}\cdot V_0\cdot Q(\gamma\cdot K \mid v+4,\lambda) \\ &+ D(T)\cdot\left(\frac{\alpha}{\beta}\right)\cdot(1-e^{-\beta T})\cdot Q(\gamma\cdot K \mid v+2,\lambda) \\ &- D(T)\cdot K\cdot Q(\gamma\cdot K \mid v,\lambda) \end{aligned} \qquad (14\text{-}2)$$

1　参见第 19 章中有关波动率期权的更大规模案例研究。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

$$\alpha = k\theta$$
$$\beta = k + \xi$$
$$r = \frac{4\beta}{\sigma^2(1-e^{-\beta T})}$$
$$\nu = \frac{4\alpha}{\sigma^2}$$
$$\lambda = \gamma \cdot e^{-\beta T} \cdot V$$

14.4.2 实现

和往常一样，公式 14-2 中的方程式很容易翻译成 **Python**。例 14-8 展示了包含这个估值函数的 **Python** 模块代码。我们将这个脚本称作 **vol_pricing_formula.py**，保存在 **volservice** 子目录下。

例 14-8　波动率期权估值的 Python 脚本

```
# Valuation of European volatility call options
# in Gruenbichler-Longstaff (1996) model
# square-root diffusion framework
# - semianalytical formula
#
from scipy.stats import ncx2
import numpy as np

# Semianalytical option pricing formula of GL96


def calculate_option_value(V0, kappa, theta, sigma, zeta, T, r, K):
    ''' Calculation of European call option price in GL96 model.

    Parameters
    ==========
    V0 : float
        current volatility level
    kappa : float
        mean reversion factor
    theta : float
        long-run mean of volatility
    sigma : float
        volatility of volatility
    zeta :
        volatility risk premium
    T : float
        time-to-maturity
    r : float
        risk-free short rate
    K : float
        strike price of the option
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    Returns
    =======
    value : float
        net present value of volatility call option
    '''
    D = np.exp(-r * T)  # discount factor

    # variables
    alpha = kappa * theta
    beta = kappa + zeta
    gamma = 4 * beta / (sigma ** 2 * (1 - np.exp(-beta * T)))
    nu = 4 * alpha / sigma ** 2
    lamb = gamma * np.exp(-beta * T) * V0
    cx1 = 1 - ncx2.cdf(gamma * K, nu + 4, lamb)
    cx2 = 1 - ncx2.cdf(gamma * K, nu + 2, lamb)
    cx3 = 1 - ncx2.cdf(gamma * K, nu, lamb)

    # formula for European call price
    value = (D * np.exp(-beta * T) * V0 * cx1
      + D * (alpha / beta) * (1 - np.exp(-beta * T))
      * cx2 - D * K * cx3)
    return value
```

为了简化 Web 服务的实现，我们编写一个方便的函数 **get_option_value**，该函数检查所有计算看涨期权价值所需的参数是否提供，保存在 **Python** 模块 **vol_priceing_service.py** 中，代码如例 14-9 所示。这个脚本还包含一个所有必备参数和参数简短说明的字典。该函数在一个或者多个参数遗漏的时候返回错误消息，详细说明遗漏的参数。如果在 Web 服务调用时提供了所有必要参数，该函数将调用 **vol_pricing_formula.py** 脚本中的 **calculate_option_value** 函数进行估价。

例 14-9　用于波动率期权估值和 Web 服务助手函数的 Python 脚本

```
# Valuation of European volatility options
# in Gruenbichler-Longstaff (1996) model
# square-root diffusion framework
# - parameter dictionary & web service function
#
from vol_pricing_formula import calculate_option_value

# model parameters

PARAMS={
    'V0' : 'current volatility level',
    'kappa' : 'mean reversion factor',
    'theta' : 'long-run mean of volatility',
    'sigma' : 'volatility of volatility',
    'zeta' : 'factor of the expected volatility risk premium',
    'T' : 'time horizon in years',
    'r' : 'risk-free interest rate',
    'K' : 'strike'
    }
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
# function for web service

def get_option_value(data):
    "' A helper function for web service. "'
    errorline = 'Missing parameter %s (%s)\n'
    errormsg = ''
    for para in PARAMS:
        if not data.has_key(para):
            # check if all parameters are provided
            errormsg += errorline % (para, PARAMS[para])
    if errormsg != '':
        return errormsg
    else:
        result = calculate_option_value(
                    float(data['V0']),
                    float(data['kappa']),
                    float(data['theta']),
                    float(data['sigma']),
                    float(data['zeta']),
                    float(data['T']),
                    float(data['r']),
                    float(data['K'])
                    )
        return str(result)
```

首先，添加前述的 Python 脚本路径：

```
In [87]: import sys
         sys.path.append("../python/volservice")
           # adjust if necessary to your path
```

我们使用 **werkzeug** 库处理基于 WSGI 应用的 Web 服务（回顾一下，**Werkzeug** 是 Flask 不可分割的一部分）。为此，需要从 **Werkzeug** 子库中导入一些函数：

```
In [88]: from werkzeug.wrappers import Request, Response
```

而且，为了接下来的 WSGI 核心应用，需要前面定义的 **get_option_value** 函数：

```
In [89]: from vol_pricing_service import get_option_value
```

剩下的唯一一件事是实现 WSGI 应用（函数）本身。在我们的例子中，这个函数如下：

```
In [90]: def application(environ, start_response):
             request = Request(environ)
               # wrap environ in new object
             text = get_option_value(request.args)
               # provide all parameters of call to function
               # get back either error message or option value
             response = Response(text, mimetype='text/html')
               # generate response object based on the returned text
             return response(environ, start_response)
```

这里，**environ** 是一个包含所有输入信息的字典。Request 函数包装了所有信息，使 **environ** 信息的访问更为便利。**start_response** 通常用于表示响应的开始。但是，Werkzeug

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

有一个负责处理响应的 **Response** 函数。

提供给 Web 服务的所有参数可以在 **request.args** 属性中找到，这也是我们提供给 **get_option_value** 函数的参数。该函数返回文本形式的错误消息，或者文本形式的期权价值估算。

为了更好地服务于这个函数（例如，通过本地 Web 服务器），我们将这个函数放入单独的 WSGI 脚本，并为其提供服务功能。例 14-10 展示了这个脚本（**vol_pricing.py**）的代码。

例 14-10　用于波动率期权估值和 Web 服务助手函数的 Python 脚本

```
#
# Valuation of European volatility options
# in Gruenbichler-Longstaff (1996) model
# square-root diffusion framework
# - WSGI application for web service
#
from vol_pricing_service import get_option_value
from werkzeug.wrappers import Request, Response
from werkzeug.serving import run_simple

def application(environ, start_response):
    request = Request(environ)
      # wrap environ in new object
    text = get_option_value(request.args)
      # provide all parameters of call to function
      # get back either error message or option value
    response = Response(text, mimetype='text/html')
      # generate response object based on the returned text
    return response(environ, start_response)

if __name__=='__main__':
    run_simple('localhost', 4000, application)
```

从合适的子目录（**volservice**）中，现在可以通过 **shell** 或者命令行界面执行如下命令，启动应用程序：

```
$ python vol_pricing.py
* Running on http://localhost:4000/
```

上述命令启动单独的 **Python** 过程，为 WSGI 应用提供服务。使用 **urllib**，可以访问这个 Web 服务的“全部能力”。在浏览器中复制这个 URL 并按下回车键，可以得到图 14-12 所示的结果。

然而，通常应该用不同的方式使用 Web 服务——例如，从 **IPython** 之类的脚本环境中。为此，可以使用 **urllib** 库提供的功能：

```
In [91]: import numpy as np
         import urllib
         url = 'http://localhost:4000/'
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

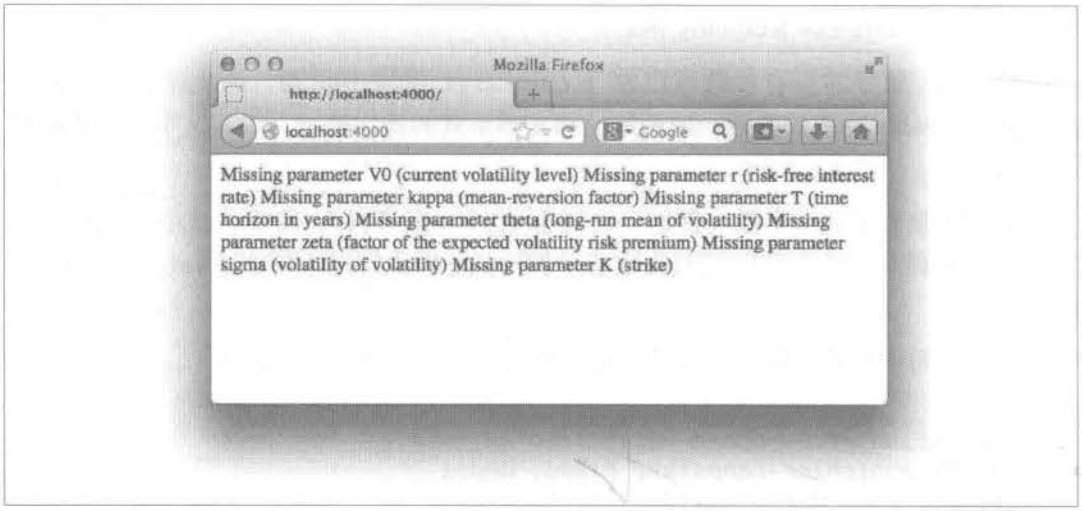


图 14-12　Web 服务错误消息截图

不带任何参数的简单 Web 服务调用返回如下错误消息，这和图 14-12 中的屏幕截图相同（除了格式化问题之外）：

```
In [92]: print urllib.urlopen(url).read()
Out[92]: Missing parameter V0 (current volatility level)
         Missing parameter r (risk-free interest rate)
         Missing parameter kappa (mean-reversion factor)
         Missing parameter T (time horizon in years)
         Missing parameter theta (long-run mean of volatility)
         Missing parameter zeta (factor of the expected volatility risk premium)
         Missing parameter sigma (volatility of volatility)
         Missing parameter K (strike)
```

当然，我们需要提供一些参数。因此，我们首先构建一个 URL 字符串对象，在后续的调用中可以替换具体参数值：

```
In [93]: urlpara = url + 'application?V0=%s&kappa=%s&theta=%s&sigma=%s&zeta=%s'
         urlpara += '&T=%s&r=%s&K=%s'
```

下面是可能出现的参数：

```
In [94]: urlval = urlpara % (25, 2.0, 20, 1.0, 0.0, 1.5, 0.02, 22.5)
         urlval
Out[94]: 'http://localhost:4000/application?V0=25&kappa=2.0&theta=20&sigma=1.0&z
         eta=0.0&T=1.5&r=0.02&K=22.5'
```

使用这个特别的 URL 字符串，可以像预想的那样返回期权价值：

```
In [95]: print urllib.urlopen(urlval).read()
Out[95]: 0.202937705934
```

有了这样的 Web 服务，当然可以多次调用，轻松地计算多种期权的价格：

```
In [96]: %%time
         urlpara = 'http://localhost:4000/application?V0=25&kappa=2.0'
         urlpara += '&theta=25&sigma=1.0&zeta=0.0&T=1&r=0.02&K=%s'
         strikes = np.linspace(20, 30, 50)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        results = []
        for K in strikes:
            results.append(float(urllib.urlopen(urlpara % K).read()))
        results = np.array(results)
Out[96]: CPU times: user 64 ms, sys: 20 ms, total: 84 ms
        Wall time: 196 ms
In [97]: results
Out[97]: array([ 4.91296701,  4.71661296,  4.52120153,  4.32692516,  4.1339945 ,
                 3.94264561,  3.75313813,  3.56575972,  3.38079846,  3.19858765,
                 3.01946028,  2.8437621 ,  2.67184576,  2.50406508,  2.34078693,
                 2.18230495,  2.02898213,  1.88111287,  1.738968  ,  1.60280064,
                 1.47281111,  1.34917004,  1.23204859,  1.12141092,  1.01739405,
                 0.9199686 ,  0.82907686,  0.74462353,  0.66647327,  0.59445387,
                 0.52843174,  0.46798166,  0.41300694,  0.36319553,  0.31824647,
                 0.27785656,  0.24171678,  0.20951651,  0.18094732,  0.1557064 ,
                 0.1334996 ,  0.11414975,  0.09710449,  0.08234678,  0.06958767,
                 0.05859317,  0.04915788,  0.04109348,  0.03422854,  0.02840802])
```

这种方法的优势之一是不用使用本地资源来获得结果，而使用 Web 服务器的资源——例如，服务器可能使用并行化技术。当然，在我们的例子中，所有资源都在本地，Web 服务器使用本地计算资源。图 14-13 以图形方式展示了估值结果，作为本节的结束：

```
In [98]: import matplotlib.pyplot as plt
        %matplotlib inline
        plt.plot(strikes, results, 'b')
        plt.plot(strikes, results, 'ro')
        plt.grid(True)
        plt.xlabel('strike')
        plt.ylabel('European call option value')
```

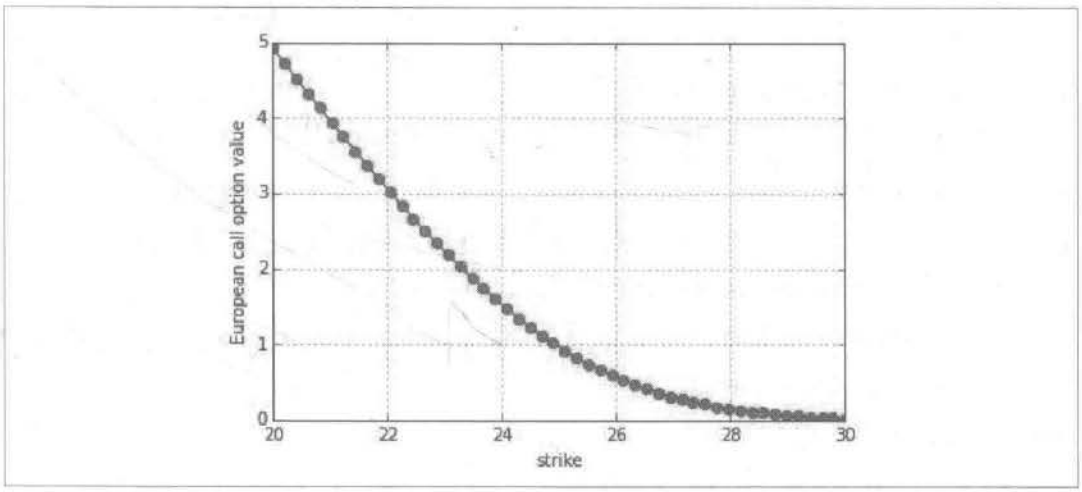


图 14-13　不同行权价下的欧式波动率看涨期权价值

Web 服务架构

Web 服务架构往往是提供基于 **Python** 分析功能（甚至整个应用程序）的一种强大而高效的替代方法。对于互联网或者私有网络的模式都同样如此。这种架构还简化了更新和维护，因为这些服务通常以集中化的方式提供。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

14.5 结语

近来，Web 技术几乎成为任何应用架构不可分割的一部分。它们不仅有利于与外部世界通信、向外部实体提供从简到繁的 Web 服务，也有助于组织内部（金融）服务。

本章首先阐述了与最常用**通信协议**（主要是 FTP 和 HTTP）相关的一些基本技术。本章还介绍如何实现交互式 *Web 图表绘制*、如何实时地与基于 Web 的金融数据 API（例如基于 **JSON**）接口，以及如何用 **Bokeh** 实时地可视化这种高频数据。

但是，Python 生态系统还提供了一些强大的*高级框架*，用于快速开发更为复杂的 Web 应用。我们使用近来流行的框架 Flask 实现了一个具备简单用户管理（注册和登录）功能的简单交易者聊天室应用，阐明了典型 Web 应用的所有要素——**Python** 实现的核心功能，**Jinja2** 模板和 **CSS** 样式。

最后，本章的最后一个小节介绍 Web 服务这一重要主题。使用 **Werkzeug** 库可以简化 WSGI 应用的处理，我们实现了一个以 **Gruenbichler** 和 Longstaff（1996）模型和公式为基础的基于 Web 波动率期权定价服务。

14.6 延伸阅读

下面的 Web 资源对本章介绍的主题很有帮助。

- **Python** 文档应该是本章说明的基本工具和技术的出发点之一：http://docs.python.org；还可以参考概述页面：http://docs.python.org/2/howto/ webservers.html。
- 关于 **Boke** 这个以 Web 为焦点的绘图库，应该参考其首页：http://bokeh.pydata.org
- 关于 Flask 的更多信息，可以从框架首页开始：http://flask.pocoo.org；也可以从 https://media.readthedocs.org/pdf/flask/latest/flask.pdf 下载 PDF 文档。
- 除了 **Python** 文档本身，更多关于 Web 服务的信息参考 Werkzeug 的首页：http://werkzeug.pocoo.org.

下面是 Flask 的参考书籍：

- Grinberg, Miguel (2014): Flask Web Development—Developing Web Applicationswith Python. O'Reilly, Sebastopol, CA.

最后是关于波动率期权的研究论文：

- Gruenbichler, Andreas and Francis Longstaff (1996): "Valuing Futures and Optionson Volatility." *Journal of Banking and Finance*, Vol. 20, pp. 985-1001.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第3部分

衍生品分析库

本部分关注规模较小、但是仍然很强大的蒙特卡洛模拟期权及衍生品定价实际应用的开发[1]。目标最终是一组 Python 类——我们称之为 DX 库（Derivatives AnalytiX，衍生品分析）——可以用来完成如下的工作。

建模

为了折现的目的建立短期利率模型；建立欧式和美式期权的模型，包括它们的标的风险因素以及相关的市场环境；建立更复杂的投资组合模型，包含多种期权和多种可能相关的标的风险因素。

模拟

根据几何布朗运动和跳跃扩散以及平方根扩散模拟风险因素；同时、连贯地模拟一些这类风险因素，不管它们是否相关。

估值

按照风险中立估值方法，估算任意收益的欧式和美式期权价值；以一致、全面的方式估算由这些期权组成的投资组合的价值。

风险管理

以独立于标的风险因素或者行权类型的方式，从数值上估算最重要的“期权希腊字母”——也就是期权/衍生品的 Delta 和 Vega 值。

---

1 期权交易以及相关主题（如市场基础知识以及所谓的“期权希腊字母”在期权风险管理中的角色）的简介和全面概述参见 James Bittman (2009)的《Trading Options as a Professional》(McGraw Hill，New York)。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

使用该库以市场为基础（使用经过检验的 VSTOXX 模型）估算 VSTOXX 波动率期权投资组合的价值并加以管理

本部分介绍的材料依赖于 DX 分析库（http://dx-analytics.com），该库是由本书作者和 Python Quants 股份有限公司（和 Python Quant 平台（http://quant-platform.com）相结合）开发和提供的。完整的版本可以进行复杂、多风险衍生物和交易账册的建模、定价和风险管理。

本部分分为如下几章。

- 第 15 章以理论和技术形式介绍估值框架。在理论上，资产定价基本定理和风险中立估值方法是核心。在技术上，这一章介绍用于风险中立折现和市场环境的 Python 类。
- 第 16 章关注基于几何布朗运动、跳跃扩散和平方根扩散过程的风险因素模拟；讨论一个普通类和三个特殊类。
- 第 17 章介绍根据单一潜在风险因素估算单一欧式或者美式衍生品价值；主要的组成部分仍然是一个普通类和两个特殊类。普通类可以独立于期权类型估算 Delta 和 Vega 值。
- 第 18 章介绍基于多种（可能相关）的潜在风险因素，估算可能包含多种衍生品的复杂投资组合的价值；介绍一个用于建立衍生品头寸模型的简单类和一个更复杂的、用于一致投资组合估值的类。
- 第 19 章用其他章节中开发的 DX 库估算 VSTOXX 波动率指数期权投资组合的价值并加以管理。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 15 章

估值框架

复利是有史以来最大的数学发现。

——阿尔伯特 • 爱因斯坦

本章介绍开发 DX 库所需的最基本概念，为其提供一个框架。我们将简短地回顾一下资产定价基本定理，这一理论为模拟和估值提供了理论背景。然后，本章介绍*日期处理*和*风险中立折现*。我们只采用最简单的情况——固定短期利率折现，但是更复杂和现实的模型很容易添加到库中。本章还介绍_market environment_（市场环境）的概念——这是后续章节中几乎所有其他类实例化所需的一组常量、列表和曲线。

15.1 资产定价基本定理

*资产定价基本定理*是现代金融理论和数学的基石和成功案例之一[1]。它的中心思想是*鞅*（Martingale）测度——也就是从折现后风险因素（随机过程）中消去漂移的概率测度。换句话说，在鞅测度下，所有风险因素随无风险短期利率漂移——而不随包含某种无风险短期利率之上风险溢价的任何其他市场利率漂移。

15.1.1 简单示例

考虑只有今天和明天两个交易日期的一个简单经济环境，其中包含一种高风险资产——某种股票和一种无风险资产——债券。今天的债券价格为 10 美元，明天清偿价值为 10 美元（0 利率）。股票今天价格为 10 美元，明日售出价格为 20 美元和 0 美元的

1 对所涉数学机制的全面研究和细节参见 Delbae 和 Schachermayer（2004）的著作。第 4 章中也有对 Hilpisch（2015）的简短介绍，特别是离散时间的版本。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

概率分别为 60%和 40%。债券的无风险收益为 0。股票的预期收益率为 $\frac{0.6\cdot 20+0.4\cdot 0}{10}-1=0.2$。这就是补偿股票风险特性的风险溢价。

现在考虑行权价为 15 美元的看涨期权。这种有 60%的概率取得 5 元收益，在其他情况下收益为 0 的未定权益，其公允价值是多大呢？例如，我们可以取得期望值，并将结果值折现（这里使用的 0 利率）。这种方法得到的价值是 0.6•5=3 美元，因为在股票价格上升至 20 美元时期权可以获得 5 美元的收益，其他情况下收益为 0。但是，还有另一种方法已经成功地应用到期权定价问题上：通过可交易证券的组合*复制*期权收益。很容易验证，购买 0.25 份股票就可以完全复制期权的收益（在 60%的情况下，我们的收益为 0.25•20=5 美元）。四分之一的股票成本仅为 2.5 美元而非 3 美元，在现实世界概率测度下的预期会*过高估计*期权的价值。

为什么会出现这种情况？现实世界的测度隐含着 20%的股票风险溢价，是因为股票中隐含的风险（盈利 100%或者损失 100%）"真的"无法依靠多样化或者对冲来消除。另一方面，有某种投资组合可以没有任何风险地复制期权的收益。这也意味着，卖出这种期权可以完全对冲任何风险[1]。这种由期权和对冲头寸组成的完全对冲投资组合必须得到等于无风险利率的收益，以避免套利机会（即"空手套白狼"、且概率为正数的机会）。

我们能否不通过预期估计看涨期权的价值？是的，可以做到。我们"只"需要改变概率，使风险资产（股票）随无风险利率（0）漂移。很显然，为两种情形同样设定 50%的（鞅）测度可以实现这一点；计算式为 $\frac{0.5\cdot 20+0.5\cdot 0}{10}-1=0$。现在，在新的鞅测度下计算期权收益，可以得到正确的（无套利）公允价值：0.5•5+0.5•0=2.5 美元。

15.1.2 一般结果

上述方法的迷人之处在于，它甚至可以用于最复杂的经济环境，例如连续时间模型（考虑连续的时间点）、大量风险资产、复杂的衍生品收益等。

因此，我们考虑离散时间中的一般市场模型[2]：

离散时间中的*一般市场模型* μ 是如下元素的集合：

- 有限状态空间 Ω
- 过滤 F

1 这种策略包括以 2.5 美元购入期权，以 2.5 美元购买 0.25 股股票。这种组合不管简单经济中出现什么情形，其收益均为 0。——原注

2 概率论概念参见 Williams（1991）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- 在（Ω）上定义的严格正概率测度 P
- 终止日期 $T \in N$，$T<\infty$
- K+1 个严格正向证券价格过程的集合 $S \equiv \{(S_t^k)_{t\in\{0,\cdots,T\}} : k \in \{0,\cdots,K\}\}$

我们将模型写作 μ={（Ω，（Ω），$\boldsymbol{F}$，P），T，$\boldsymbol{S}$}。

根据上述一般市场模型，我们可以这样描述资产定价基本定理[1]：

考虑一般市场模型 μ，根据*资产定价基本定理*，下面 3 个陈述等价：

- 市场模型 μ 中没有套利机会
- P 等价鞅测度集合 $\boldsymbol{Q}$ 不为空
- 一致线性定价系统集合 $\boldsymbol{P}$ 不为空

对于未定权益（期权、衍生品、期货、远期、掉期等），该定理的重要性可以用如下的推论阐述：

如果市场模型 μ 是无套利的，则对任何可获得（可复制）的未定权益（期权，衍生品等）V_T 存在一个相关的唯一价格 V_0，满足 $\forall Q \in \mathcal{Q} : V_0 = E_0^Q(e^{-rT}V_T)$，其中 e^{-rT} 是固定短期利率 r 对应的风险中立折现因子。

这一结果说明了该定理的重要性，并说明了我们上面介绍的简单推论适用于一般市场模型。

由于鞅测度的作用，这种估值方法也常常被称作鞅方法——或者，因为鞅测度下所有风险资产都随无风险利率漂移而被称作风险中立估值方法。对于我们的目的来说，第二个术语可能更好，因为在数值应用中，我们“简单”地让风险因素（随机过程）根据风险中立短期利率漂移。对于我们的应用，没有必要处理概率测度——但是，这些测度从理论上证明了我们所应用的核心理论结果和实施的技术方法。

最后，考虑一般市场模型中的市场完全性：

> 如果市场 μ 是无套利的，且每种未定权益（期权，衍生品等）都可获得（可复制），则称它是完全的。
>
> 假定市场模型 μ 是无套利的，当且仅当 $\mathcal{Q}$ 是单例（Sington，亦即存在唯一的 P 等价鞅测度）时该市场完全。

本节完成了对后续小节所需的技术背景的讨论。对这些概念、定义和结果的详细阐述参考 Hilpisch（2015）的第 5 章。

---

1 参见 Delbaen 和 Schachermayer（2004）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

15.2 风险中立折现

很明显，风险中立折现是风险中立估值方法的核心。因此，我们首先开发一个用于风险中立折现的 **Python** 类。但是，首先仔细地观察一下估值**相关日期**的建模和处理是值得的。

15.2.1 日期建模和处理

折现的必要前提之一是建立日期模型（参见附录 C）。为了估值的目的，通常将当日和一般市场模型的最后日期 T 之间的间隔分为离散的时间间隔。这些时间间隔可以是同构的（等长），也可以是异构的（不等长）。估值库应该能够处理异构时间间隔的一般情况，因为更简单的情况会自动包含。因此，我们使用日期列表，假定最小的相关时间间隔为*一天*。这意味着，我们不关心日间事件——如果需要处理这些事件，就必须建立*时间*模型（除了日期之外）[1]。

为了编辑一个相关日期列表，基本上可以采用两种方法：构建一个离散的*日期*列表（例如 **Python** 中的 **datetime.datetime** 对象）或者*年分数*（十进制数字，就像理论工作中常常做的）。

例如，下面的 **dates** 和 **fraction** 定义（大致）等价：

```
In [1]: import datetime as dt

In [2]: dates = [dt.datetime(2015, 1, 1), dt.datetime(2015, 7, 1),
                                  dt.datetime(2016, 1, 1)]

In [3]: (dates[1] - dates[0]).days / 365.
Out[3]: 0.4958904109589041

In [4]: (dates[2] - dates[1]).days / 365.
Out[4]: 0.5041095890410959

In [5]: fractions = [0.0, 0.5, 1.0]
```

它们只是大致等价，因为年分数很少处于某一天的开始（0 点），你只需要想想，将一年除以 50 的结果就明白了。

有时候，根据日期列表得出年分数是必要的。例 15-1 中介绍的 **get_year_delatas** 函数完成的就是这项工作。

例 15-1　从 datetime 对象列表或者数组得出年分数的函数

```
#
# DX Library Frame
# get_year_deltas.py
#
```

1　添加时间成分实际上很简单，但是为了易于讲解，在这里不这么做。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
import numpy as np

def get_year_deltas(date_list, day_count=365.):
    ''' Return vector of floats with day deltas in years.
    Initial value normalized to zero.

    Parameters
    ==========
    date_list : list or array
        collection of datetime objects
    day_count : float
        number of days for a year
    (to account for different conventions)

Results
=======
delta_list : array
    year fractions
'''

start = date_list[0]
delta_list = [(date - start).days / day_count
              for date in date_list]
return np.array(delta_list)
```

这个函数的应用如下：

```
In [1]: import datetime as dt

In [2]: dates = [dt.datetime(2015, 1, 1), dt.datetime(2015, 7, 1),
                                  dt.datetime(2016, 1, 1)]

In [3]: get_year_deltas(dates)
Out[4]: array([ 0.        ,  0.49589041,  1.        ])
```

建立短期利率模型时，这个函数的好处显而易见。

15.2.2 固定短期利率

我们将焦点放在短期利率折现的最简单情况——也就是短期利率*一直保持不变*的情况。许多期权定价模型，如 Black-Scholes-Merton（1973）、Merton（1976）和 Cox-Ross-Rubinstein）（1979）都做出这个假设[1]。我们假定连续折现，因为这是期权定价应用中常见的假设。在这种情况下，在给定未来日期 t 和固定短期利率 r 下，今天的一般折现因子可以由 $D_0(t)=e^{-rt}$ 得出。当然，在结束日有一个特例 $D_0(T)=e^{-rT}$。注意，t 和 T 都是年分数。

折现因子也可以解释为分别在 t 和 T 到期的*单位零息债券*（ZCB）的当日价值[2]。给定

1 例如，对于短期期权的定价，这个假设在许多情况下都满足。——原注

2 单位零息债券在到期时偿付一个货币单位，在当日和到期日之间没有利息。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

两个日期 $t \geq s \geq 0$，从 t 到 s 的折现因子可以由公式 $D_s(t) = D_0(t)/D_0(s) = e^{-rt}/e^{-rs} = e^{-rt} \cdot e^{rs} = e^{-r(t-s)}$ 给出。

例 15-2 提供了将上述考虑因素转换为 **Python** 代码的一个 **Python** 类[1]。

例 15-2　固定短期利率下的风险中立折现类

```
#
# DX Library Frame
# constant_short_rate.py
#
from get_year_deltas import *

class constant_short_rate(object):
    ''' Class for constant short rate discounting.

    Attributes
    ==========
    name : string
        name of the object
    short_rate : float (positive)
        constant rate for discounting

    Methods
    =======
get_discount_factors :
    get discount factors given a list/array of datetime objects
    or year fractions
    '''

    def __init__(self, name, short_rate):
        self.name = name
        self.short_rate = short_rate
        if short_rate < 0:
            raise ValueError('Short rate negative.')

    def get_discount_factors(self, date_list, dtobjects=True):
        if dtobjects is True:
            dlist = get_year_deltas(date_list)
        else:
            dlist = np.array(date_list)
        dflist = np.exp(self.short_rate * np.sort(-dlist))
        return np.array((date_list, dflist)).T
```

constant_short_rate 类的应用最好由一个简单、具体的例子说明。和前面一样，我们继续使用同一个 **datetime** 对象列表：

1 Python 面向对象开发的基本知识参见第 13 章。在本章和本部分的余下章节中，我们从标准的 PEP8 命名约定中得出 Python 类名。PEP 8 推荐为 Python 类名使用“词首字母大写”或者“驼峰式大小写”。在“接口已经记入文档并主要作为可调用对象的情况下”使用 PEP 8 提到的函数命名惯例作为有效的替代方案。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [1]: import datetime as dt

In [2]: dates = [dt.datetime(2015, 1, 1), dt.datetime(2015, 7, 1),
   ...:          dt.datetime(2016, 1, 1)]

In [3]: from constant_short_rate import *

In [4]: csr = constant_short_rate('csr', 0.05)

In [5]: csr.get_discount_factors(dates)
Out[5]:
array([[datetime.datetime(2015, 1, 1, 0, 0), 0.95122942450071402],
       [datetime.datetime(2015, 7, 1, 0, 0), 0.9755103387657228],
       [datetime.datetime(2016, 1, 1, 0, 0), 1.0]], dtype=object)
```

主要的结果是一个二维 **ndarray** 对象，包含成对的 **datetime** 对象和相关折现因子。该类（特别是对象 csr）通常也能处理年分数：

```
In [7]: deltas = get_year_deltas(dates)

In [8]: csr.get_discount_factors(deltas, dtobjects=False)
Out[8]:
array([[ 0.        ,  0.95122942],
       [ 0.49589041,  0.97551034],
       [ 1.        ,  1.        ]])
```

这个类负责其他类中需要的所有折现运算。

15.3 市场环境

市场环境“只是”其他数据和 **Python** 对象集合的一个名称。但是，使用这一抽象相当方便，因为它简化了许多操作，而且考虑到了循环特征的一致性建模[1]。市场环境主要由 3 个字典组成，它们存储如下类型的数据和 **Python** 对象：

常量

例如模型参数或者期权到期日。

列表

一般是对象序列，例如用于建立（风险）证券模型对象的列表对象。

曲线

用于折现的对象；例如，**constant_short_rate** 类的实例。

例 15-3 展示了 **market_environment** 类。字典对象的处理参见第 4 章。

例 15-3 建立由常量、列表和曲线组成的市场环境模型的类

1 这个概念的相关信息参见 Fletcher 和 Gardner（2009），他们大量使用市场环境。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
#
# DX Library Frame
# market_environment.py
#

class market_environment(object):
    ''' Class to model a market environment relevant for valuation.

    Attributes
    ==========
    name: string
        name of the market environment
    pricing_date : datetime object
        date of the market environment

    Methods
    =======
    add_constant :
        adds a constant (e.g. model parameter)
    get_constant :
        gets a constant
    add_list :
        adds a list (e.g. underlyings)
    get_list :
        gets a list
    add_curve :
        adds a market curve (e.g. yield curve)
    get_curve :
        gets a market curve
    add_environment :
        adds and overwrites whole market environments
        with constants, lists, and curves
    '''

    def __init__(self, name, pricing_date):
        self.name = name
        self.pricing_date = pricing_date
        self.constants = {}
        self.lists = {}
        self.curves = {}
def add_constant(self, key, constant):
    self.constants[key] = constant

def get_constant(self, key):
    return self.constants[key]

def add_list(self, key, list_object):
    self.lists[key] = list_object

def get_list(self, key):
    return self.lists[key]

def add_curve(self, key, curve):
    self.curves[key] = curve
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    def get_curve(self, key):
        return self.curves[key]

    def add_environment(self, env):
        # overwrites existing values, if they exist
        for key in env.constants:
            self.constants[key] = env.constants[key]
        for key in env.lists:
            self.lists[key] = env.lists[key]
        for key in env.curves:
            self.curves[key] = env.curves[key]
```

虽然在 **market_environment** 类中没有什么特殊的，但是一个简单的例子就可以说明使用该类的实例有多么方便：

```
In [1]: from market_environment import *

In [2]: import datetime as dt

In [3]: dates = [dt.datetime(2015, 1, 1), dt.datetime(2015, 7, 1),
                                 dt.datetime(2016, 1, 1)]

In [4]: csr = constant_short_rate('csr', 0.05)

In [5]: me_1 = market_environment('me_1', dt.datetime(2015, 1, 1))

In [6]: me_1.add_list('symbols', ['AAPL', 'MSFT', 'FB'])

In [7]: me_1.get_list('symbols')
Out[7]: ['AAPL', 'MSFT', 'FB']

In [8]: me_2 = market_environment('me_2', dt.datetime(2015, 1, 1))

In [9]: me_2.add_constant('volatility', 0.2)

In [10]: me_2.add_curve('short_rate', csr)  # add instance of discounting class

In [11]: me_2.get_curve('short_rate')
Out[11]: <constant_short_rate.constant_short_rate at 0x104ac3c90>

In [12]: me_1.add_environment(me_2)  # add complete environment

In [13]: me_1.get_curve('short_rate')
Out[13]: <constant_short_rate.constant_short_rate at 0x104ac3c90>

In [14]: me_1.constants
Out[14]: {'volatility': 0.2}

In [15]: me_1.lists
Out[15]: {'symbols': ['AAPL', 'MSFT', 'FB']}

In [16]: me_1.curves
Out[16]: {'short_rate': <constant_short_rate.constant_short_rate at 0x104ac3c90>}

In [17]: me_1.get_curve('short_rate').short_rate
Out[17]: 0.05
```

以上的例子说明了对这个相当通用的“存储”类的基本处理。对于实用的应用程序，首先收集市场数据和其他数据以及 **Python** 对象，然后实例化 **market_environment** 对

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

象并填入相关数据及对象。在一步中，就可以将其交付给需要保存在对应 **market_environment** 中数据对象的其他类。

例如，这种面向对象建模方法的重大优势之一是，**constant_short_rate** 类的实例可以存在于多个环境中。一旦实例被更新——例如，设置新的固定短期利率时——包含这个特殊折现类的所有 **market_environment** 实例都将被自动更新。

15.4 结语

本章为构建 **Python** 库、通过蒙特卡洛模拟估算期权和其他衍生品价值的较大项目提供一个框架。本章介绍资产定价基本定理，用相当简单的数值化示例加以阐述。在这一点上，为离散时间的一般市场模型提供了重要成果。

本章还为风险中立折现开发了一个 **Python** 类，以数值形式运用了资产定价基本定理的机制。根据 **Python datetime** 对象列表或者表示年分数的浮点对象，**constant_short_rate** 类的实例提供对应的折现因子（单位零息债券的现值）。

本章的最后介绍了相当通用的 **market_environment** 类，该类集合了用于建模、模拟、估值和其他目的的相关数据和 **Python** 对象。

为了简化未来的导入工作，我们将使用一个包装器模块 **dx_frame.py**，如例 15-4 所示。

例 15-4　框架组件的包装器模块

```
#
# DX Library Frame
# dx_frame.py
#
import datetime as dt

from get_year_deltas import get_year_deltas
from constant_short_rate import constant_short_rate
from market_environment import market_environment
```

下面的一条 **import** 语句可以在一步中使所有框架组件可用：

```
from dx_frame import *
```

对于 **Python** 库和模块套件，还有一个选择是将所有相关 **Python** 模块保存在一个（子）目录，并在该目录中放入一个特殊的 **init** 文件进行所有导入工作。例如，将所有模块保存在 **dx** 子目录下，例 15-5 中提供的文件可以完成导入。不过，要注意这个特殊文件的命名约定。

例 15-5　Python 打包文件

```
#
# DX Library
# packaging file
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
# __init__.py
#
import datetime as dt

from get_year_deltas import get_year_deltas
from constant_short_rate import constant_short_rate
from market_environment import market_environment
```

在这种情况下，只要使用目录名就可以一次完成所有导入：

```
from dx import *
```

也可以通过另一种方法：

```
import dx
```

15.5 延伸阅读

对于本章介绍的主题，下面是一些有用的书籍资源：

- Delbaen, Freddy and Walter Schachermayer (2004): *The Mathematics of Arbitrage*. Springer Verlag, Berlin, Heidelberg.
- Fletcher, Shayne and Christopher Gardner (2009): *Financial Modelling in Python*. John Wiley & Sons, Chichester, England.
- Hilpisch, Yves (2015): *Derivatives Analytics with Python*. Wiley Finance, Chichester, England. http://derivatives-analytics-with-python.com.
- Williams, David (1991): *Probability with Martingales*. Cambridge University Press, Cambridge, England.

定义本章中引用模型的原创研究论文，参考后续章节的“延伸阅读”小节。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 16 章

金融模型的模拟

科学的目的不是分析或者描述，而是制作这个世界的实用模型。

——Edward de Bono

第 10 章介绍了使用 **Python** 和 **NumPy** 实现随机过程的蒙特卡洛模拟的一些细节。本章应用前面介绍的基本技术实现模拟类，作为 DX 库的核心组件。我们将注意力限制在 3 个广泛使用的随机过程：

几何布朗运动

这是由 Black 和 Scholes（1973）的开创性工作引入期权定价文献的过程；它在本书中多次使用，尽管有一些著名的不足之处，并且在金融现实中得到越来越多的证实，但是仍是期权和衍生品估值的一个基准过程。

跳跃扩散

跳跃扩散由 Merton（1976）引入，在几何布朗运动（GBM）上增加了对数分布跳跃成分；这使我们可以考虑更多情况，例如短期虚值（OTM）期权的定价往往需要考虑大幅暴涨的可能性。换言之，以 GBM 作为金融模型往往无法令人满意地解释这种 OTM 期权的市场价值，而跳跃扩散可能做到。

平方根扩散

平方根扩散在金融学中的流行是因为 Cox、Ingersoll 和 Ross（1985）的努力，它被用于建立利率和波动率等均值回归数量的模型；除了均值回归之外，这种过程总是正向的，这通常是此类数量的理想特性。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

本章的第一小节继续使用方差缩减技术开发生成标准正态分布随机数的函数[1]。后续的小节开发 1 个泛型模拟类和 3 个特殊模拟类，每个特殊类用于前述的一个利率随机过程。

本章介绍的模型模拟的进一步细节参见 Hilpisch（2015）。本书还特别包含了一个基于 Merton 跳跃扩散（1976）的完整案例研究。

16.1 随机数生成

随机数生成是蒙特卡洛模拟的中心任务之一[2]。第 10 章说明如何使用 **Python** 和 **numpy.random** 等库生成不同分布的随机数。对于我们手上的项目，*标准正态分布*随机数最为重要。因此，创建一个方便的函数生成这种特殊类型的随机数是值得的。例 16-1 提供了一个这类函数。

例 16-1 生成标准正态分布随机数的函数

```
import numpy as np

def sn_random_numbers(shape, antithetic=True, moment_matching=True,
                      fixed_seed=False):
    ''' Returns an array of shape shape with (pseudo)random numbers
    that are standard normally distributed.

    Parameters
    ==========
    shape : tuple (o, n, m)
        generation of array with shape (o, n, m)
    antithetic : Boolean
        generation of antithetic variates
    moment_matching : Boolean
        matching of first and second moments
    fixed_seed : Boolean
        flag to fix the seed

    Results
    =======
    ran : (o, n, m) array of (pseudo)random numbers
    '''
    if fixed_seed:
        np.random.seed(1000)
    if antithetic:
        ran = np.random.standard_normal((shape[0], shape[1], shape[2] / 2))
        ran = np.concatenate((ran, -ran), axis=2)
    else:
        ran = np.random.standard_normal(shape)
    if moment_matching:
        ran = ran - np.mean(ran)
```

1 我们所指的“随机”数通常仅是“伪随机数”。——原注

2 随机数和随机变量生成的相关信息参见 Glasserman（2004）第 2 章。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            ran = ran / np.std(ran)
        if shape[0] == 1:
            return ran[0]
        else:
            return ran
```

本函数中使用的方差缩减技术，即反向路径和矩匹配在第 10 章中也有介绍[1]。

上述函数的应用很简单：

```
In [1]: from sn_random_numbers import *

In [2]: snrn = sn_random_numbers((2, 2, 2), antithetic=False,
   ...:                          moment_matching=False,
   ...:                          fixed_seed=True)

In [3]: snrn
Out[3]:
array([[[-0.8044583 ,  0.32093155],
        [-0.02548288,  0.64432383]],

       [[-0.30079667,  0.38947455],
        [-0.1074373 , -0.47998308]]])

In [4]: snrn_mm = sn_random_numbers((2, 3, 2), antithetic=False,
   ...:                          moment_matching=True,
   ...:                          fixed_seed=True)

In [5]: snrn_mm
Out[5]:
array([[[-1.47414161,  0.67072537],
        [ 0.01049828,  1.28707482],
        [-0.51421897,  0.80136066]],

       [[-0.14569767, -0.85572818],
        [ 1.19313679, -0.82653845],
        [ 1.3308292 , -1.47730025]]])

In [6]: snrn_mm.mean()
Out[6]: 1.8503717077085941e-17

In [7]: snrn_mm.std()
Out[7]: 1.0
```

这个函数将成为后面的模拟类中的主力干将。

16.2 泛型模拟类

面向对象建模——正如第 13 章中的介绍——允许属性和方法的继承。这是我们在构建模拟类时想要利用的：从一个*泛型*模拟类开始，它包含所有其他模拟类共享的属性和方法。

1 Glasserman(2004)的第 4 章提供了不同方差缩减技术的概述和理论细节。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

首先要注意的是，我们实例化任何模拟类的一个对象都“只”提供 3 个属性：

name

用作模型模拟对象名称的*字符串*对象

mar_env

market_environment 类的一个实例

corr

表示对象是否相关的一个标志（布尔型）

这再次说明了*市场环境*的作用：在一步中提供模拟和估值所需的所有数据和对象。泛型类的方法如下：

generate_time_grid

这个方法生成模拟所用的相关日期的“时间网格”。这个任务对于每个模拟类都相同。

get_instrument_values

每个模拟类都必须返回包含模拟金融工具价值的 **ndarray** 对象（例如模拟的股票价格，商品价格，波动率）。

例 16-2 提供了一个泛型模型模拟类。其中的方法利用了模型定制类提供的其他方法，例如 **self.generate_paths**。在我们拥有了专用、非泛型模拟类的全景时，这方面的细节就会变得很清晰。

例 16-2　泛型金融模型模拟类

```
#
# DX Library Simulation
# simulation_class.py
#
import numpy as np
import pandas as pd


class simulation_class(object):
    ''' Providing base methods for simulation classes.

    Attributes
    ==========
    name : string
        name of the object
    mar_env : instance of market_environment
        market environment data for simulation
    corr : Boolean
        True if correlated with other model object

    Methods
    =======
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    generate_time_grid :
        returns time grid for simulation
    get_instrument_values :
        returns the current instrument values (array)
    '''

    def __init__(self, name, mar_env, corr):
        try:
            self.name = name
            self.pricing_date = mar_env.pricing_date
            self.initial_value = mar_env.get_constant('initial_value')
            self.volatility = mar_env.get_constant('volatility')
            self.final_date = mar_env.get_constant('final_date')
            self.currency = mar_env.get_constant('currency')
            self.frequency = mar_env.get_constant('frequency')
            self.paths = mar_env.get_constant('paths')
            self.discount_curve = mar_env.get_curve('discount_curve')
            try:
                # if time_grid in mar_env take this
                # (for portfolio valuation)
                self.time_grid = mar_env.get_list('time_grid')
            except:
                self.time_grid = None
            try:
                # if there are special dates, then add these
                self.special_dates = mar_env.get_list('special_dates')
            except:
                self.special_dates = []
            self.instrument_values = None
            self.correlated = corr
            if corr is True:
                # only needed in a portfolio context when
                # risk factors are correlated
                self.cholesky_matrix = mar_env.get_list('cholesky_matrix')
                self.rn_set = mar_env.get_list('rn_set')[self.name]
                self.random_numbers = mar_env.get_list('random_numbers')
        except:
            print "Error parsing market environment."

    def generate_time_grid(self):
        start = self.pricing_date
        end = self.final_date
        # pandas date_range function
        # freq = e.g. 'B' for Business Day,
        # 'W' for Weekly, 'M' for Monthly
        time_grid = pd.date_range(start=start, end=end,
                                  freq=self.frequency).to_pydatetime()
        time_grid = list(time_grid)
        # enhance time_grid by start, end, and special_dates
        if start not in time_grid:
            time_grid.insert(0, start)
            # insert start date if not in list
        if end not in time_grid:
            time_grid.append(end)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            # insert end date if not in list
        if len(self.special_dates) > 0:
            # add all special dates
            time_grid.extend(self.special_dates)
            # delete duplicates
            time_grid = list(set(time_grid))
            # sort list
            time_grid.sort()
        self.time_grid = np.array(time_grid)

    def get_instrument_values(self, fixed_seed=True):
        if self.instrument_values is None:
            # only initiate simulation if there are no instrument values
            self.generate_paths(fixed_seed=fixed_seed, day_count=365.)
        elif fixed_seed is False:
            # also initiate resimulation when fixed_seed is False
            self.generate_paths(fixed_seed=fixed_seed, day_count=365.)
        return self.instrument_values
```

市场环境的解析嵌入一个 **try-except** 子句中，解析失败时会发起一个异常。为了保持代码的简洁，不实施任何完整性检查。例如，不管内容是否真的是折现类的一个实例，下面的代码行都被认为是“成功”。因此，在编译和将 **market_environment** 对象传递给任何模拟类时应该相当小心：

```
self.discount_curve = mar_env.get_curve('discount_curve')
```

表 16-1 展示了泛型和其他所有模拟类的 **market_environment** 对象必须包含的所有组成部分。

表 16-1　所有模拟类的市场环境元素

元素	类型	强制	描述
initial_value	常量	是	pricing_date（定价日）时的过程初始值
volatility	常量	是	过程的波动性系数
final_date	常量	是	模拟范围
currency	常量	是	金融实体的货币
frequency	常量	是	日期频率，和 pandas freq 参数相同
paths	常量	是	模拟路径数量
discount_curve	曲线	是	constant_short_rate 实例
time_grid	列表	否	相关日期的时间网格（在投资组合背景下）
random_numbers	列表	否	随机数数组（用于相关对象）
cholesky_matrix	列表	否	Cholesky 矩阵（用于相关对象）
rn_set	列表	否	包含指向相关随机数值指针的字典对象

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

与模型模拟对象关联有关的所有元素都将在后续的章节中解释。在本章中，我们将重点放在单一、无关联过程的模拟。类似地，传递 **time_grid** 的选项只在投资组合背景下适用，也将在后面解释。

16.3 几何布朗运动

几何布朗运动是公式 16-1 所描述的随机过程（参见第 10 章的公式 10-2，特别是参数和变量的含义）。过程的漂移已经设置为等于无风险固定短期利率 r，意味着我们在等价鞅测度下工作（参见第 15 章）。

公式 16-1　几何布朗运动的随机微分方程

$$dS_t = rS_t dt + \sigma S_t dZ_t \qquad (16\text{-}1)$$

公式 16-2 提供了上述微分方程用于模拟目的的欧拉离散化格式（更多细节参见第 10 章的公式 10-3）。我们工作于离散时间市场模型中（例如第 15 章中的一般市场模型 μ），使用有限相关日期集合 $0<t_1<t_2<\cdots<T$。

公式 16-2　模拟几何布朗运动的微分方程

$$S_{t_{m+1}} = S_{t_m} \exp\left(\left(r - \frac{1}{2}\sigma^2\right)(t_{m+1} - t_m) + \sigma\sqrt{t_{m+1} - t_m}\, z_t\right) \qquad (16\text{-}2)$$
$$0 \leqslant t_m < t_{m+1} \leqslant T$$

16.3.1 模拟类

例 16-3 提供了 GBM 模型的一个专用类。我们首先介绍完整的类，然后强调说明精选的一些特征。

例 16-3　几何布朗运动模拟类

```
#
# DX Library Simulation
# geometric_brownian_motion.py
#
import numpy as np

from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class

class geometric_brownian_motion(simulation_class):
    ''' Class to generate simulated paths based on
    the Black-Scholes-Merton geometric Brownian motion model.

    Attributes
    ==========
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    name : string
        name of the object
    mar_env : instance of market_environment
        market environment data for simulation
    corr : Boolean
        True if correlated with other model simulation object

    Methods
    =======
    update :
        updates parameters
    generate_paths :
    returns Monte Carlo paths given the market environment
    '''

def __init__(self, name, mar_env, corr=False):
    super(geometric_brownian_motion, self).__init__(name, mar_env, corr)

def update(self, initial_value=None, volatility=None, final_date=None):
    if initial_value is not None:
        self.initial_value = initial_value
    if volatility is not None:
        self.volatility = volatility
    if final_date is not None:
        self.final_date = final_date
    self.instrument_values = None

def generate_paths(self, fixed_seed=False, day_count=365.):
    if self.time_grid is None:
        self.generate_time_grid()
          # method from generic simulation class
    # number of dates for time grid
    M = len(self.time_grid)
    # number of paths
    I = self.paths
    # array initialization for path simulation
    paths = np.zeros((M, I))
    # initialize first date with initial_value
    paths[0] = self.initial_value
    if not self.correlated:
        # if not correlated, generate random numbers
        rand = sn_random_numbers((1, M, I),
                                 fixed_seed=fixed_seed)
    else:
        # if correlated, use random number object as provided
        # in market environment
        rand = self.random_numbers
    short_rate = self.discount_curve.short_rate
      # get short rate for drift of process
    for t in range(1, len(self.time_grid)):
        # select the right time slice from the relevant
        # random number set
        if not self.correlated:
            ran = rand[t]
        else:
            ran = np.dot(self.cholesky_matrix, rand[:, t, :])
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            ran = ran[self.rn_set]
        dt = (self.time_grid[t] - self.time_grid[t - 1]).days / day_count
          # difference between two dates as year fraction
        paths[t] = paths[t - 1] * np.exp((short_rate - 0.5
                                          * self.volatility ** 2) * dt
                                 + self.volatility * np.sqrt(dt) * ran)
          # generate simulated values for the respective date
    self.instrument_values = paths
```

在这个特例中，**market_environment** 对象只能包含表 16-1 中所示的数据和对象——即成分的最小集合。

update 方法的作用正如其名：它可以更新选择的模型重要参数。当然，**generate_paths** 方法更复杂一些，不过，它有一些内嵌注释，可以清晰地说明最重要的特征。理论上，考虑不同模型模拟对象之间的关联会给该方法带来一定的复杂度。这将会逐渐变得清晰，特别是在例 18-2 中。

16.3.2 用例

下面的交互式 **IPython** 会话说明了 **geometric_brownian_motion** 类的使用方法。首先，我们必须生成包含所有强制性元素的 **market_environment** 对象：

```
In [1]: from dx import *
In [2]: me_gbm = market_environment('me_gbm', dt.datetime(2015, 1, 1))
In [3]: me_gbm.add_constant('initial_value', 36.)
        me_gbm.add_constant('volatility', 0.2)
        me_gbm.add_constant('final_date', dt.datetime(2015, 12, 31))
        me_gbm.add_constant('currency', 'EUR')
        me_gbm.add_constant('frequency', 'M')
          # monthly frequency (respective month end)
        me_gbm.add_constant('paths', 10000)
In [4]: csr = constant_short_rate('csr', 0.05)
In [5]: me_gbm.add_curve('discount_curve', csr)
```

其次，我们初始化一个模型模拟对象：

```
In [6]: from dx_simulation import *
In [7]: gbm = geometric_brownian_motion('gbm', me_gbm)
```

第三步，我们可以使用这个对象。例如，生成和检查 **time_grid**。你将会注意到，我们在 **time_grid** 数组对象中有 13 个 **datetime** 对象（对应年的所有月末和 **pricing_date**）：

```
In [8]: gbm.generate_time_grid()
In [9]: gbm.time_grid
Out[9]: array([datetime.datetime(2015, 1, 1, 0, 0),
               datetime.datetime(2015, 1, 31, 0, 0),
               datetime.datetime(2015, 2, 28, 0, 0),
               datetime.datetime(2015, 3, 31, 0, 0),
               datetime.datetime(2015, 4, 30, 0, 0),
               datetime.datetime(2015, 5, 31, 0, 0),
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                datetime.datetime(2015, 6, 30, 0, 0),
                datetime.datetime(2015, 7, 31, 0, 0),
                datetime.datetime(2015, 8, 31, 0, 0),
                datetime.datetime(2015, 9, 30, 0, 0),
                datetime.datetime(2015, 10, 31, 0, 0),
                datetime.datetime(2015, 11, 30, 0, 0),
                datetime.datetime(2015, 12, 31, 0, 0)], dtype=object)
```

接下来，我们可以求得模拟金融工具价值：

```
In [10]: %time paths_1 = gbm.get_instrument_values()
Out[10]: CPU times: user 10.7 ms, sys: 2.91 ms, total: 13.6 ms
         Wall time: 12.8 ms
In [11]: paths_1
Out[11]: array([[ 36.        ,  36.        ,  36.        , …,  36.        ,
                  36.        ,  36.        ],
                [ 37.37221481,  38.08890977,  34.37156575, …,  36.22258915,
                  35.05503522,  39.63544014],
                [ 39.45866146,  42.18817025,  32.38579992, …,  34.80319951,
                  33.60600939,  37.62733874],
                …,
                [ 40.15717404,  33.16701733,  23.32556112, …,  37.5619937 ,
                  29.89282508,  30.2202427 ],
                [ 42.0974104 ,  36.59006321,  21.70771374, …,  35.70950512,
                  30.64670854,  30.45901309],
                [ 43.33170027,  37.42993532,  23.8840177 , …,  35.92624556,
                  27.87720187,  28.77424561]])
```

我们还要生成更高波动率下的金融工具价值：

```
In [12]: gbm.update(volatility=0.5)
In [13]: %time paths_2 = gbm.get_instrument_values()
Out[13]: CPU times: user 9.78 ms, sys: 1.36 ms, total: 11.1 ms
         Wall time: 10.2 ms
```

两组路径之间的差异在图 16-1 中显示：

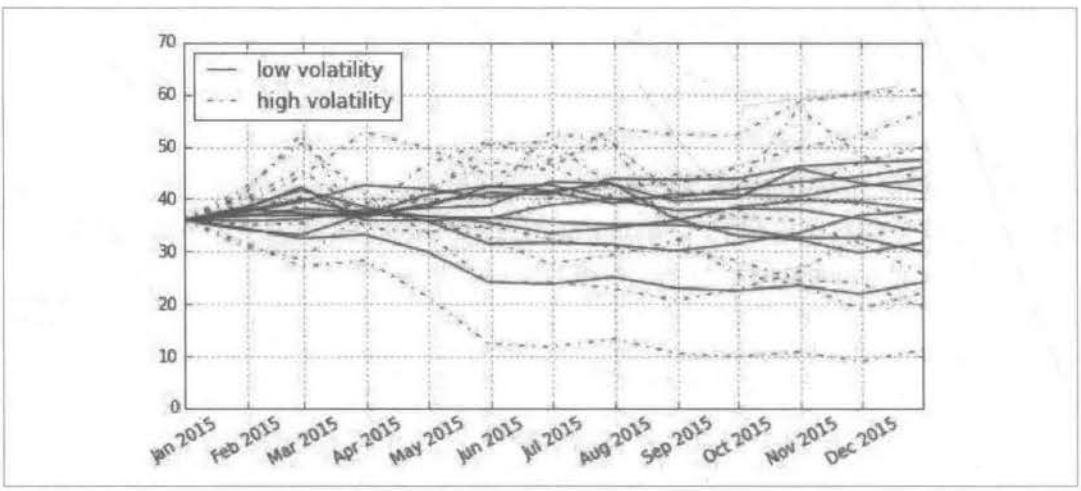


图 16-1　GBM 模拟类中的模拟路径

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [14]: import matplotlib.pyplot as plt
         %matplotlib inline
         plt.figure(figsize=(8, 4))
         p1 = plt.plot(gbm.time_grid, paths_1[:, :10], 'b')
p2 = plt.plot(gbm.time_grid, paths_2[:, :10], 'r-.')
plt.grid(True)
l1 = plt.legend([p1[0], p2[0]],
                ['low volatility', 'high volatility'], loc=2)
plt.gca().add_artist(l1)
plt.xticks(rotation=30)
```

16.4 跳跃扩散

有了 **geometric_brownian_motion** 类的背景知识，实现 Merton（1976）描述的跳跃扩散模型就很简单了。回顾一下跳跃扩散的随机微分方程（如公式 16-3 所示，参见第 10 章中的公式 10-8，尤其是参数和变量的含义）。

公式 16-3 Merton 跳跃扩散模型的随机微分方程

$$dS_t = (r - r_J)S_t dt + \sigma S_t dZ_t + J_t S_t dZN_t \qquad (16\text{-}3)$$

公式 16-4 提供了用于模拟目的的欧拉离散化格式（更详细的解释参见第 10 章的公式 10-9）。

公式 16-4 Merton 跳跃扩散模型的欧拉离散化

$$S_{t_{m+1}} = S_{t_m}\left(\exp\left(\left(r - r_J - \frac{1}{2}\sigma^2\right)(t_{m+1} - t_m) + \sigma\sqrt{t_{m+1} - t_m}\, z_t^1\right) + (e^{\mu_J + \delta z_t^2} - 1)y_t\right) \qquad (16\text{-}4)$$

$$0 \leqslant t_m < t_{m+1} \leqslant T$$

16.4.1 模拟类

例 16-4 提供了 **jump_diffusion** 模拟类的 **Python** 代码。这个类现在所包含的内容都在意料之中。当然，模型不同，但是设计和方法本质上相同。

例 16-4 跳跃扩散模拟类

```
#
# DX Library Simulation
# jump_diffusion.py
#
import numpy as np

from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class

class jump_diffusion(simulation_class):
    ''' Class to generate simulated paths based on
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    the Merton (1976) jump diffusion model.

    Attributes
    ==========
    name : string
        name of the object
    mar_env : instance of market_environment
        market environment data for simulation
    corr : Boolean
        True if correlated with other model object

    Methods
    =======
    update :
        updates parameters
    generate_paths :
    returns Monte Carlo paths given the market environment
"'

def __init__(self, name, mar_env, corr=False):
    super(jump_diffusion, self).__init__(name, mar_env, corr)
    try:
        # additional parameters needed
        self.lamb = mar_env.get_constant('lambda')
        self.mu = mar_env.get_constant('mu')
        self.delt = mar_env.get_constant('delta')
    except:
        print "Error parsing market environment."

def update(self, initial_value=None, volatility=None, lamb=None,
           mu=None, delta=None, final_date=None):
    if initial_value is not None:
        self.initial_value = initial_value
    if volatility is not None:
        self.volatility = volatility
    if lamb is not None:
        self.lamb = lamb
    if mu is not None:
        self.mu = mu
    if delta is not None:
        self.delt = delta
    if final_date is not None:
        self.final_date = final_date
    self.instrument_values = None

def generate_paths(self, fixed_seed=False, day_count=365.):
    if self.time_grid is None:
        self.generate_time_grid()
          # method from generic simulation class
    # number of dates for time grid
    M = len(self.time_grid)
    # number of paths
    I = self.paths
    # array initialization for path simulation
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        paths = np.zeros((M, I))
        # initialize first date with initial_value
        paths[0] = self.initial_value
        if self.correlated is False:
            # if not correlated, generate random numbers
            sn1 = sn_random_numbers((1, M, I),
                                    fixed_seed=fixed_seed)
    else:
        # if correlated, use random number object as provided
        # in market environment
        sn1 = self.random_numbers

    # standard normally distributed pseudorandom numbers
    # for the jump component
    sn2 = sn_random_numbers((1, M, I),
                            fixed_seed=fixed_seed)

    rj = self.lamb * (np.exp(self.mu + 0.5 * self.delt ** 2) - 1)

    short_rate = self.discount_curve.short_rate
    for t in range(1, len(self.time_grid)):
        # select the right time slice from the relevant
        # random number set
        if self.correlated is False:
            ran = sn1[t]
        else:
            # only with correlation in portfolio context
            ran = np.dot(self.cholesky_matrix, sn1[:, t, :])
            ran = ran[self.rn_set]
        dt = (self.time_grid[t] - self.time_grid[t - 1]).days / day_count
          # difference between two dates as year fraction
        poi = np.random.poisson(self.lamb * dt, I)
          # Poisson-distributed pseudorandom numbers for jump component
        paths[t] = paths[t - 1] * (np.exp((short_rate - rj
                                   - 0.5 * self.volatility ** 2) * dt
                                 + self.volatility * np.sqrt(dt) * ran)
                                 + (np.exp(self.mu + self.delt *
                                   sn2[t]) - 1) * poi)
    self.instrument_values = paths
```

当然，因为我们现在处理的是不同的模型，所以需要 **market_environment** 对象中的一组不同元素。除了用于 **geometric_brownian_motion** 类（见表 16-1）的元素之外，还增加了表 16-2 中的 3 种元素：即对数正态跳跃成分的参数：**lambda**、**mu** 和 **delta**。

表 16-2　jump_diffusion 类的特殊市场环境元素

元素	类型	强制	描述
lambda	常量	是	跳跃密度（概率，按年）
mu	常量	是	预期跳跃规模
delta	常量	是	跳跃规模的标准差

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

为了生成路径，由于跳跃成分的存在，这个类当然需要进一步的随机数。**generate_paths** 方法的内嵌注释强调了这些额外的随机数生成的两个位置。泊松分布随机数的生成参见第 10 章。

16.4.2 用例

下面，我们再次用例子交互式地说明模拟类 **jump_diffusion** 的使用方法。我们使用前一小节中为 GBM 对象定义的 **market_environment** 对象：

```
In [15]: me_jd = market_environment('me_jd', dt.datetime(2015, 1, 1))
In [16]: # add jump diffusion specific parameters
         me_jd.add_constant('lambda', 0.3)
         me_jd.add_constant('mu', -0.75)
         me_jd.add_constant('delta', 0.1)
```

对于这个环境，添加 GBM 模拟类的完整环境，完成需要的输入：

```
In [17]: me_jd.add_environment(me_gbm)
```

以这个 **market_environment** 对象为基础，可以实例化用于跳跃扩散的模拟类：

```
In [18]: from jump_diffusion import jump_diffusion
In [19]: jd = jump_diffusion('jd', me_jd)
```

由于我们所采用的建模方法，金融工具价值的生成现在在形式上是相同的。但是在本例中方法调用略慢一些，因为我们有跳跃成分，需要模拟更多的数值：

```
In [20]: %time paths_3 = jd.get_instrument_values()
Out[20]: CPU times: user 19.7 ms, sys: 2.92 ms, total: 22.6 ms
         Wall time: 21.9 ms
```

我们的目标仍然是比较两组不同的路径，例如，改变跳跃概率：

```
In [21]: jd.update(lamb=0.9)
In [22]: %time paths_4 = jd.get_instrument_values()
Out[22]: CPU times: user 26.3 ms, sys: 2.07 ms, total: 28.4 ms
         Wall time: 27.7 ms
```

图 16-2 比较分别来自高密度和低密度（跳跃概率）的两个集合的模拟路径。在图中可以看到，低密度的情况下跳跃较少，而高密度的情况下有多处跳跃：

```
In [23]: plt.figure(figsize=(8, 4))
         p1 = plt.plot(gbm.time_grid, paths_3[:, :10], 'b')
         p2 = plt.plot(gbm.time_grid, paths_4[:, :10], 'r-.')
         plt.grid(True)
         l1 = plt.legend([p1[0], p2[0]],
                         ['low intensity', 'high intensity'], loc=3)
         plt.gca().add_artist(l1)
         plt.xticks(rotation=30)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

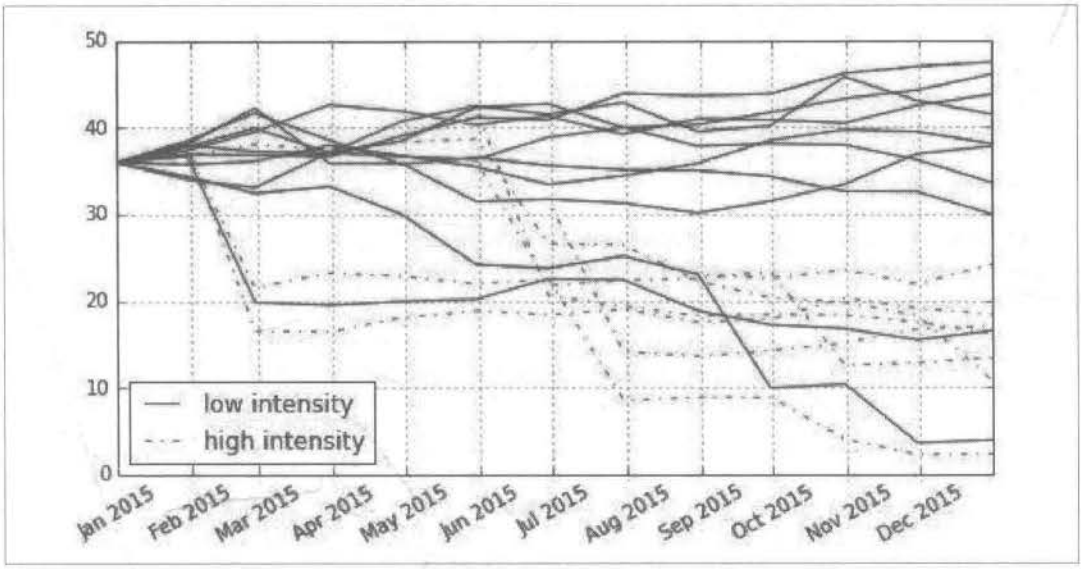


图 16-2　来自跳跃扩散模拟类的模拟路径

16.5　平方根扩散

我们要模拟的第三种随机过程是 Cox、Ingersoll 和 Ross（1985）用于建立随机短期利率模型的平方根扩散。公式 16-5 展示了这种过程的随机微分方程（进一步的细节参见第 10 章的公式 10-4）。

公式 16-5　平方根扩散的随机微分方程

$$\mathrm{d}x_t = \kappa(\theta - x_t)\mathrm{d}t + \sigma\sqrt{x_t}\,\mathrm{d}Z_t \qquad (16\text{-}5)$$

我们使用公式 16-6 中提供的离散化格式（参见第 10 章的公式 10-5，以及 10-6 中的精确替代格式）。

公式 16-6　平方根扩散的欧拉离散化

$$\begin{aligned} \tilde{x}_{t_{m+1}} &= \tilde{x}_{t_m} + \kappa(\theta - \tilde{x}_s^+)(t_{m+1} - t_m) + \sigma\sqrt{\tilde{x}_s^+}\sqrt{t_{m+1} - t_m}\,z_t \\ x_{t_{m+1}} &= \tilde{x}_{t_{m+1}}^+ \end{aligned} \qquad (16\text{-}6)$$

16.5.1　模拟类

例 16-5 提供了 **square_root_diffusion** 模拟类的 **Python** 代码。当然，除了模型和离散化格式不同之外，和其他两个特殊类相比，这个类没有什么新内容。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例 16-5　平方根扩散的模拟类

```
#
# DX Library Simulation
# square_root_diffusion.py
#
import numpy as np

from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class


class square_root_diffusion(simulation_class):
    ''' Class to generate simulated paths based on
    the Cox-Ingersoll-Ross (1985) square-root diffusion model.

    Attributes
    ==========
    name : string
        name of the object
    mar_env : instance of market_environment
        market environment data for simulation
    corr : Boolean
        True if correlated with other model object

    Methods
    =======
    update :
    updates parameters
generate_paths :
    returns Monte Carlo paths given the market environment
    '''

    def __init__(self, name, mar_env, corr=False):
        super(square_root_diffusion, self).__init__(name, mar_env, corr)
        try:
            self.kappa = mar_env.get_constant('kappa')
            self.theta = mar_env.get_constant('theta')
        except:
            print "Error parsing market environment."

    def update(self, initial_value=None, volatility=None, kappa=None,
               theta=None, final_date=None):
        if initial_value is not None:
            self.initial_value = initial_value
        if volatility is not None:
            self.volatility = volatility
        if kappa is not None:
            self.kappa = kappa
        if theta is not None:
            self.theta = theta
        if final_date is not None:
            self.final_date = final_date
        self.instrument_values = None

    def generate_paths(self, fixed_seed=True, day_count=365.):
        if self.time_grid is None:
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        self.generate_time_grid()
    M = len(self.time_grid)
    I = self.paths
    paths = np.zeros((M, I))
    paths_ = np.zeros_like(paths)
    paths[0] = self.initial_value
    paths_[0] = self.initial_value
    if self.correlated is False:
        rand = sn_random_numbers((1, M, I),
                                 fixed_seed=fixed_seed)
    else:
        rand = self.random_numbers

    for t in range(1, len(self.time_grid)):
        dt = (self.time_grid[t] - self.time_grid[t - 1]).days / day_count
        if self.correlated is False:
            ran = rand[t]
        else:
            ran = np.dot(self.cholesky_matrix, rand[:, t, :])
            ran = ran[self.rn_set]

        # full truncation Euler discretization
        paths_[t] = (paths_[t - 1] + self.kappa
                     * (self.theta - np.maximum(0, paths_[t - 1, :])) * dt
                     + np.sqrt(np.maximum(0, paths_[t - 1, :]))
                     * self.volatility * np.sqrt(dt) * ran)
        paths[t] = np.maximum(0, paths_[t])
    self.instrument_values = paths
```

表 16-3 列出了这个类特有的两个市场环境元素。

表 16-3　square_root_diffusion 类市场环境的特定元素

元素	类型	强制	描述
kappa	常量	是	均值回归因子
theta	常量	是	过程长期均值

16.5.2　用例

一个相当简短的用例可以说明模拟类的使用方法。和往常一样，我们需要一个市场环境，例如用于建立波动率（指数）过程模型的环境：

```
In [35]: me_srd = market_environment('me_srd', dt.datetime(2015, 1, 1))
In [36]: me_srd.add_constant('initial_value', .25)
         me_srd.add_constant('volatility', 0.05)
         me_srd.add_constant('final_date', dt.datetime(2015, 12, 31))
         me_srd.add_constant('currency', 'EUR')
         me_srd.add_constant('frequency', 'W')
         me_srd.add_constant('paths', 10000)
```

市场环境有两个元素是该类所特有的：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [37]: # specific to simulation class
         me_srd.add_constant('kappa', 4.0)
         me_srd.add_constant('theta', 0.2)
```

虽然我们在实施模拟时还不需要，但是泛型模拟类需要一个折现对象。从风险中立估值的角度可以证明这一需求，而风险中立估值是整个 DX 分析库的首要目标：

```
In [38]: # required but not needed for the class
         me_srd.add_curve('discount_curve', constant_short_rate('r', 0.0))
In [39]: from square_root_diffusion import square_root_diffusion
In [40]: srd = square_root_diffusion('srd', me_srd)
```

和以前一样，我们以 **market_environment** 对象作为输入，调用 **get_instrument_values** 方法获得模拟路径：

```
In [41]: srd_paths = srd.get_instrument_values()[:, :10]
```

图 16-3 显示单一模拟路径平均回归到长期均值 **theta**（虚线），说明了均值回归特性：

```
In [42]: plt.figure(figsize=(8, 4))
         plt.plot(srd.time_grid, srd.get_instrument_values()[:, :10])
         plt.axhline(me_srd.get_constant('theta'), color='r', ls='-', lw=2.0)
         plt.grid(True)
         plt.xticks(rotation=30)
```

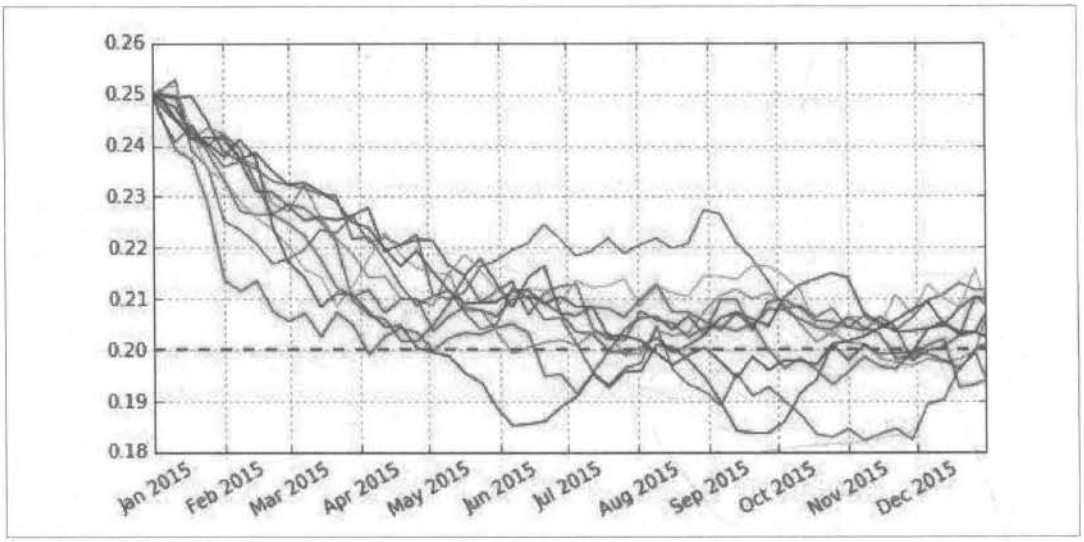


图 16-3　来自平方根扩散模拟类的模拟路径（虚线=长期均值 theta）

16.6　结语

本章开发我们感兴趣的 3 种随机过程（几何布朗运动、跳跃扩散和平方根扩散）模拟所需的所有工具和类，还介绍了一个可以方便地生成标准正态分布随机数的函数，然后介绍一个泛型模型模拟类。以此为基础，本章引入了 3 个专用模拟类并提供了这些

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

类的用例。

为了简化未来的导入，我们同样可以使用一个包装器模块 **dx_simulation.py**，如例 16-6 所示。

例 16-6　模拟组件的包装器模块

```
#
# DX Library Simulation
# dx_simulation.py
#
import numpy as np
import pandas as pd

from dx_frame import *
from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class
from geometric_brownian_motion import geometric_brownian_motion
from jump_diffusion import jump_diffusion
from square_root_diffusion import square_root_diffusion
```

和第一个包装器模块 **dx_frame.py** 一样，这样做的好处是用一条 **import** 语句就可以一次导入所有模拟组件：

```
from dx_simulation import *
```

由于 **dx_simulation.py** 还导入来自 **dx_frame.py** 的所有组件，这样的导入实际上输出目前为止开发的所有功能。对于 **dx** 目录中的增强版 **init** 文件也是如此，如例 16-7 所示。

例 16-7　增强版 Python 打包文件

```
#
# DX Library
# packaging file
# __init__.py
#
import numpy as np
import pandas as pd
import datetime as dt

# frame
from get_year_deltas import get_year_deltas
from constant_short_rate import constant_short_rate
from market_environment import market_environment

# simulation
from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class
from geometric_brownian_motion import geometric_brownian_motion
from jump_diffusion import jump_diffusion
from square_root_diffusion import square_root_diffusion
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

16.7 延伸阅读

下面是本章介绍的主题的一些实用参考书：

- Glasserman, Paul (2004): Monte Carlo Methods in Financial Engineering. Springer, New York.
- Hilpisch, Yves (2015): Derivatives Analytics with Python. Wiley Finance, Chichester, England. http://derivatives-analytics-with-python.com.

本章引用的原创论文如下：

- Black, Fischer and Myron Scholes (1973): "The Pricing of Options and Corporate Liabilities." Journal of Political Economy, Vol. 81, No. 3, pp. 638-659.
- Cox, John, Jonathan Ingersoll, and Stephen Ross (1985): "A Theory of the Term Structure of Interest Rates."Econometrica, Vol. 53, No. 2, pp. 385-407.
- Merton, Robert (1973): "Theory of Rational Option Pricing."Bell Journal of Economics and Management Science, Vol. 4, pp. 141-183.
- Merton, Robert (1976): "Option Pricing When the Underlying Stock Returns Are Discontinuous." Journal of Financial Economics, Vol. 3, No. 3, pp. 125-144.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 17 章

衍生品估值

衍生品是一个巨大、复杂的问题。

——Judd Gregg

期权和衍生品估值很久以来都属于华尔街“*火箭科学家*”（拥有物理学或者与数学相关的类似高要求学科博士学位的人们）的领域。然而，通过蒙特卡洛模拟等数值方法，这些模型的应用通常不像理论模型本身那么复杂。

对于采用*欧式行权方法*的期权和衍生品（即只可能在某个预先确定的日期行权）来说，上述说法尤为正确。对于采用*美式行权方法*的期权和衍生品来说则稍微复杂一些，这种行权可以在预先规定的一段时期内进行。本章介绍和使用*最小二乘蒙特卡洛*（LSM）算法，这种算法已经成为基于蒙特卡洛模拟的美式期权估值中的基准算法。

本章在结构上类似于第 16 章，首先介绍一个泛型估值类，然后提供两个专用估值类，一个用于欧式行权，另一个用于美式行权。

泛型估值类包含以数值形式估算最重要的期权“希腊字母”（*Delta* 和 *Vega*）的方法。因此，估值类不仅对估值很重要，对*风险管理*也很重要。

17.1 泛型估值类

和泛型模拟类一样，我们仅提供少数几个输入（本例是 4 个），实例化一个估值类对象：

name

表示模型模拟对象名称的字符串对象

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

underlying

代表标的的模拟类实例

mar_env

market_environment 类实例

payoff_func

包含期权/衍生品收益函数的 **Python** 字符串

泛型类有 3 个方法：

update

这个方法更新选择的估值参数（属性）。

delta

这个方法计算期权/衍生品的 **Delta** 值。

vega

这个方法计算期权/衍生品的 **Vega** 值。

有了前面几章中学习到的 DX 库背景知识，例 17-1 中介绍的泛型估值类几乎不言自明；在合适的地方还提供了内嵌注释。我们仍然首先提供完整的类，在后续的小节中重点介绍选择的主题。

例 17-1　泛型估值类

```
#
# DX Library Valuation
# valuation_class.py
#

class valuation_class(object):
    ''' Basic class for single-factor valuation.

    Attributes
    ==========
    name : string
        name of the object
    underlying :
        instance of simulation class
    mar_env : instance of market_environment
        market environment data for valuation
    payoff_func : string
        derivatives payoff in Python syntax
        Example: 'np.maximum(maturity_value - 100, 0)'
        where maturity_value is the NumPy vector with
        respective values of the underlying
        Example: 'np.maximum(instrument_values - 100, 0)'
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        where instrument_values is the NumPy matrix with
        values of the underlying over the whole time/path grid
Methods
=======
update:
    updates selected valuation parameters
delta :
    returns the Delta of the derivative
vega :
    returns the Vega of the derivative
'''

def __init__(self, name, underlying, mar_env, payoff_func=''):
    try:
        self.name = name
        self.pricing_date = mar_env.pricing_date
        try:
            self.strike = mar_env.get_constant('strike')
              # strike is optional
        except:
            pass
        self.maturity = mar_env.get_constant('maturity')
        self.currency = mar_env.get_constant('currency')
        # simulation parameters and discount curve from simulation object
        self.frequency = underlying.frequency
        self.paths = underlying.paths
        self.discount_curve = underlying.discount_curve
        self.payoff_func = payoff_func
        self.underlying = underlying
        # provide pricing_date and maturity to underlying
        self.underlying.special_dates.extend([self.pricing_date,
                                              self.maturity])
    except:
        print "Error parsing market environment."

def update(self, initial_value=None, volatility=None,
           strike=None, maturity=None):
    if initial_value is not None:
        self.underlying.update(initial_value=initial_value)
    if volatility is not None:
        self.underlying.update(volatility=volatility)
    if strike is not None:
        self.strike = strike
    if maturity is not None:
        self.maturity = maturity
        # add new maturity date if not in time_grid
        if not maturity in self.underlying.time_grid:
            self.underlying.special_dates.append(maturity)
            self.underlying.instrument_values = None
def delta(self, interval=None, accuracy=4):
    if interval is None:
        interval = self.underlying.initial_value / 50.
    # forward-difference approximation
    # calculate left value for numerical Delta
    value_left = self.present_value(fixed_seed=True)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        # numerical underlying value for right value
        initial_del = self.underlying.initial_value + interval
        self.underlying.update(initial_value=initial_del)
        # calculate right value for numerical delta
        value_right = self.present_value(fixed_seed=True)
        # reset the initial_value of the simulation object
        self.underlying.update(initial_value=initial_del - interval)
        delta = (value_right - value_left) / interval
        # correct for potential numerical errors
        if delta < -1.0:
            return -1.0
        elif delta > 1.0:
            return 1.0
        else:
            return round(delta, accuracy)

    def vega(self, interval=0.01, accuracy=4):
        if interval < self.underlying.volatility / 50.:
            interval = self.underlying.volatility / 50.
        # forward-difference approximation
        # calculate the left value for numerical Vega
        value_left = self.present_value(fixed_seed=True)
        # numerical volatility value for right value
        vola_del = self.underlying.volatility + interval
        # update the simulation object
        self.underlying.update(volatility=vola_del)
        # calculate the right value for numerical Vega
        value_right = self.present_value(fixed_seed=True)
        # reset volatility value of simulation object
        self.underlying.update(volatility=vola_del - interval)
        vega = (value_right - value_left) / interval
        return round(vega, accuracy)
```

泛型类 **valuation_class** 包含的主题之一是“希腊字母”的估算。这是我们应该仔细观察的。为此，考虑一个代表期权现值的连续可微分函数 $V(S_0, \sigma_0)$。期权的 Delta 值定义为标的当前价值 S_0 的一阶偏微分，即 $\Delta = \frac{\partial V(\bullet)}{\partial S_0}$。

假定现在我们从蒙特卡洛估值（参见第 10 章和本章后续的小节）中得到一个期权价值的蒙特卡洛估算函数 $\bar{V}(S_0, \sigma_0)$。期权 Delta 值的数值逼近由公式 17-1 给出[1]。这是泛型估值类的 **delta** 方法所实现的功能。该方法假定存在 **present_value** 方法，返回给定某组参数值下的蒙特卡洛估算函数。

公式 17-1　期权 Delta 值的数值解

$$\bar{\Delta} = \frac{\bar{V}(S_0 + \Delta S, \sigma_0) - \bar{V}(S_0, \sigma_0)}{\Delta S}, \Delta S > 0 \qquad (17\text{-}1)$$

1　通过蒙特卡洛模拟估算“希腊字母”的细节参见 Glassman(2004)的第 7 章。我们在此只使用前向差分格式。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

类似地，金融工具的 Vega 值定义为在当前（即期）波动率 σ_0 下现值的一阶偏微分，即 $\mathbf{V}=\frac{\partial V(\bullet)}{\partial \sigma_0}$。同样假设存在期权价值的蒙特卡洛估算函数，公式 17-2 提供了 Vega 的数值逼近公式。这是 **valuation_class** 类的 **vega** 方法实现的功能。

公式 17-2　期权 Vega 值的数值解

$$\mathbf{V}=\frac{\overline{V}(S_0,\sigma_0+\Delta\sigma)-\overline{V}(S_0,\sigma_0)}{\Delta\sigma},\Delta\sigma>0 \qquad (17\text{-}2)$$

注意，Delta 和 Vega 的讨论只基于有一个可微分函数或者期权现值的蒙特卡洛估算函数*存在*的情况。这是我们可以定义方法，以数值化方式估算这些数量，而无须了解蒙特卡洛估算函数的精确定义和数值实现的原因。

17.2　欧式行权

我们要专门化泛型估值类的第一种情况是*欧式行权*。为此，考虑生成期权价值蒙特卡洛估算函数的如下简化方法：

1. 进行 I 次风险中立测度下对应标的风险因子 S 的模拟，得出同样多的期权 T 到期日标的物模拟价值——即 $\overline{S}_T(i), i\in\{1,2,\cdots,I\}$

2. 计算到期日时每种模拟标的物价值的期权收益 h_T——即 $h_T(\overline{S}_T(i)), i\in\{1,2,\cdots,I\}$

3. 得出期权现值的蒙特卡洛估算函数 $\overline{V}_0\equiv e^{-rT}\frac{1}{I}\sum_{i=1}^{I}h_T(\overline{S}_T(i))$

17.3　估值类

例 17-2 展示了根据上述方法实现 **present_value** 方法的类。此外，该类还包含 **generate_payoff** 方法，生成模拟路径和给定路径的期权收益。这当然是蒙特卡洛估算函数的基础。

例 17-2　欧式行权估值类

```
#
# DX Library Valuation
# valuation_mcs_european.py
#
import numpy as np

from valuation_class import valuation_class

class valuation_mcs_european(valuation_class):
    ''' Class to value European options with arbitrary payoff
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    by single-factor Monte Carlo simulation.

    Methods
    =======
    generate_payoff :
        returns payoffs given the paths and the payoff function
    present_value :
        returns present value (Monte Carlo estimator)
    '''

    def generate_payoff(self, fixed_seed=False):
        '''
        Parameters
        ==========
        fixed_seed : Boolean
            use same/fixed seed for valuation
        '''
        try:
            # strike defined?
            strike = self.strike
        except:
            pass
        paths = self.underlying.get_instrument_values(fixed_seed=fixed_seed)
        time_grid = self.underlying.time_grid
        try:
            time_index = np.where(time_grid == self.maturity)[0]
            time_index = int(time_index)
        except:
            print "Maturity date not in time grid of underlying."
    maturity_value = paths[time_index]
    # average value over whole path
    mean_value = np.mean(paths[:time_index], axis=1)
    # maximum value over whole path
    max_value = np.amax(paths[:time_index], axis=1)[-1]
    # minimum value over whole path
    min_value = np.amin(paths[:time_index], axis=1)[-1]
    try:
        payoff = eval(self.payoff_func)
        return payoff
    except:
        print "Error evaluating payoff function."

def present_value(self, accuracy=6, fixed_seed=False, full=False):
    '''
    Parameters
    ==========
    accuracy : int
        number of decimals in returned result
    fixed_seed : Boolean
        use same/fixed seed for valuation
    full : Boolean
        return also full 1d array of present values
    '''
    cash_flow = self.generate_payoff(fixed_seed=fixed_seed)
    discount_factor = self.discount_curve.get_discount_factors(
                      (self.pricing_date, self.maturity))[0, 1]
    result = discount_factor * np.sum(cash_flow) / len(cash_flow)
    if full:
        return round(result, accuracy), discount_factor * cash_flow
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        else:
            return round(result, accuracy)
```

generate_payoff 方法提供了一些特殊的对象，用于期权收益的定义：

1. **strike** 是期权的行权价。
2. **maturity_value** 表示期权*到期日标的物*模拟价值。
3. **mean_value** 是从今天到到期日整条路径上标的物的*平均价*值。
4. **max_value** 是整条路径上标的物的*最大价值*。
5. **min_value** 是整条路径上标的物的*最小价值*。

后 3 个对象特别考虑了具有亚洲式（回顾式）特征的期权的高效处理。

17.3.1 用例

估值类 **valuation_mcs_european** 的应用最好通过一个特定的用例来说明。但是，在实例化估值之前，我们需要一个模拟对象——也就是用于估算期权价值的标的。从第 16 章起，我们就使用 **gemetric_brownian_motion** 类建立标的物模型。我们还使用对应用例的示例参数：

```
In [1]: from dx import *
In [2]: me_gbm = market_environment('me_gbm', dt.datetime(2015, 1, 1))
In [3]: me_gbm.add_constant('initial_value', 36.)
        me_gbm.add_constant('volatility', 0.2)
        me_gbm.add_constant('final_date', dt.datetime(2015, 12, 31))
        me_gbm.add_constant('currency', 'EUR')
        me_gbm.add_constant('frequency', 'M')
        me_gbm.add_constant('paths', 10000)
In [4]: csr = constant_short_rate('csr', 0.06)
In [5]: me_gbm.add_curve('discount_curve', csr)
In [6]: gbm = geometric_brownian_motion('gbm', me_gbm)
```

除了模拟对象之外，我们需要为期权提供一个市场环境，其中必须包含至少一个 **maturity** 和一个 **currency**。我们还可以选择提供一个 **strike**：

```
In [7]: me_call = market_environment('me_call', me_gbm.pricing_date)
In [8]: me_call.add_constant('strike', 40.)
        me_call.add_constant('maturity', dt.datetime(2015, 12, 31))
        me_call.add_constant('currency', 'EUR')
```

核心元素当然是收益函数，在这里以包含 Python 代码的一个字符串对象形式提供，这些代码由 **eval** 函数进行求值。我们打算定义欧式看涨期权，这种期权的收益为 $h_T=\max(S_T-K, 0)$，S_T 是到期日标的价值，K 是期权行权价。在 Python 和 NumPy 中——所有模拟值都采用向量化存储——该函数的形式如下：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [9]: payoff_func = 'np.maximum(maturity_value - strike, 0)'
```

现在，可以将所有成分组合起来，实例化 **valuation_mcs_european** 类：

```
In [10]: from valuation_mcs_european import valuation_mcs_european
In [11]: eur_call = valuation_mcs_european('eur_call', underlying=gbm,
                                           mar_env=me_call, payoff_func=payoff_func)
```

有了这个估值对象，所有感兴趣的数量都只需要一次方法调用即可得到。首先获得期权现值：

```
In [12]: %time eur_call.present_value()
Out[12]: CPU times: user 41.7 ms, sys: 11 ms, total: 52.7 ms
         Wall time: 44.6 ms

Out[12]: 2.180511
```

正如预期的那样，欧式看涨期权的 Delta 为正数——也就是说，期权的现值随标的物初始价值的增加而增加：

```
In [13]: %time eur_call.delta()
Out[13]: CPU times: user 10.9 ms, sys: 1.09 ms, total: 12 ms
         Wall time: 11.1 ms

         0.4596
```

Vega 的计算也类似，它说明在初始波动率增加 1%（例如从 24%增加到 25%）时，期权的现值增大。Vega 对于欧式看跌和看涨期权均为正数：

```
In [14]: %time eur_call.vega()
Out[14]: CPU times: user 15.2 ms, sys: 1.34 ms, total: 16.5 ms
         Wall time: 15.6 ms

         14.2782
```

一旦拥有了估值对象，就很容易实施现值和“希腊字母”的更全面分析。下面的代码计算标的初值从 34-46 欧元时的现值、Delta 和 Vega：

```
In [15]: %%time
         s_list = np.arange(34., 46.1, 2.)
         p_list = []; d_list = []; v_list = []
         for s in s_list:
             eur_call.update(initial_value=s)
             p_list.append(eur_call.present_value(fixed_seed=True))
             d_list.append(eur_call.delta())
             v_list.append(eur_call.vega())
Out[15]: CPU times: user 239 ms, sys: 8.18 ms, total: 248 ms
         Wall time: 248 ms
```

利用上述所有数值，就可以用图形方式检查结果。为此，我们使用例 17-3 所示的助手函数。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例 17-3　绘制期权统计图表的助手函数

```
#
# DX Library Valuation
# plot_option_stats.py
#
import matplotlib.pyplot as plt

def plot_option_stats(s_list, p_list, d_list, v_list):
    ''' Plots option prices, Deltas, and Vegas for a set of
    different initial values of the underlying.

    Parameters
    ==========
    s_list : array or list
        set of initial values of the underlying
    p_list : array or list
        present values
    d_list : array or list
        results for Deltas
    v_list : array or list
        results for Vegas
    '''
    plt.figure(figsize=(9, 7))
    sub1 = plt.subplot(311)
    plt.plot(s_list, p_list, 'ro', label='present value')
    plt.plot(s_list, p_list, 'b')
    plt.grid(True); plt.legend(loc=0)
    plt.setp(sub1.get_xticklabels(), visible=False)
    sub2 = plt.subplot(312)
    plt.plot(s_list, d_list, 'go', label='Delta')
    plt.plot(s_list, d_list, 'b')
    plt.grid(True); plt.legend(loc=0)
    plt.ylim(min(d_list) - 0.1, max(d_list) + 0.1)
    plt.setp(sub2.get_xticklabels(), visible=False)
    sub3 = plt.subplot(313)
    plt.plot(s_list, v_list, 'yo', label='Vega')
    plt.plot(s_list, v_list, 'b')
    plt.xlabel('initial value of underlying')
    plt.grid(True); plt.legend(loc=0)
```

导入该函数并向其提供估值结果，生成如图 17-1 所示的图形。

```
In [16]: from plot_option_stats import plot_option_stats
         %matplotlib inline

In [17]: plot_option_stats(s_list, p_list, d_list, v_list)
```

这说明，DX 库的使用尽管涉及很多数值计算，但是归结起来，采用的方法与有封闭式期权定价公式时可以媲美。不过，这种方法不仅适用于目前为止考虑的简单收益。使用相同的方法，可以处理复杂得多的收益。为此，我们考虑*常规和亚式收益*的组合：

```
In [18]: payoff_func = 'np.maximum(0.33 * (maturity_value + max_value) - 40, 0)'
           # payoff dependent on both the simulated maturity value
           # and the maximum value
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

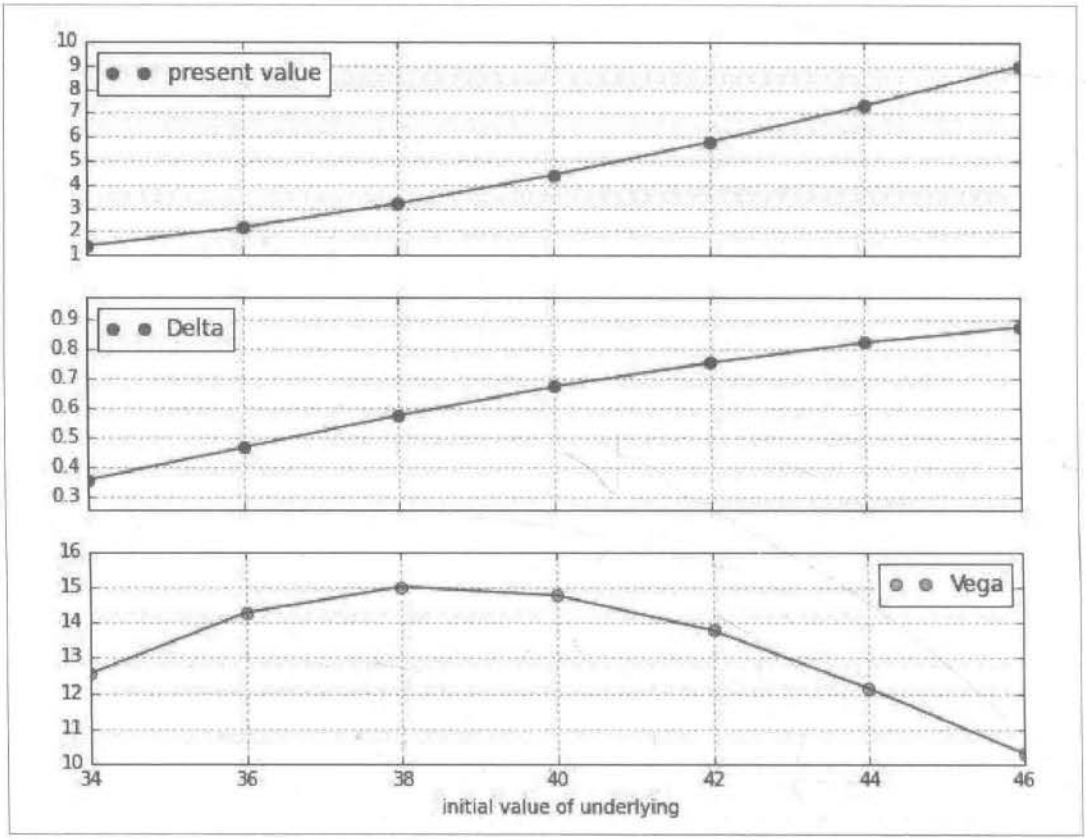


图 17-1　欧式看涨期权的现值、Delta 和 Vega 估算

其他一切保持不变：

```
In [19]: eur_as_call = valuation_mcs_european('eur_as_call', underlying=gbm,
                                              mar_env=me_call, payoff_func=payoff_func)
```

当然，所有统计数字都出现变化：

```
In [20]: %%time
         s_list = np.arange(34., 46.1, 2.)
         p_list = []; d_list = []; v_list = []
         for s in s_list:
             eur_as_call.update(s)
             p_list.append(eur_as_call.present_value(fixed_seed=True))
             d_list.append(eur_as_call.delta())
             v_list.append(eur_as_call.vega())
Out[20]: CPU times: user 286 ms, sys: 14.5 ms, total: 300 ms
         Wall time: 303 ms
```

图 17-2 显示，标的初始价值接近行权价（40）时 Delta 变为 1。从这个特定点起，标的初始价值的每次（鞅）增加导致期权价值有同等的（鞅）增加：

```
In [21]: plot_option_stats(s_list, p_list, d_list, v_list)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

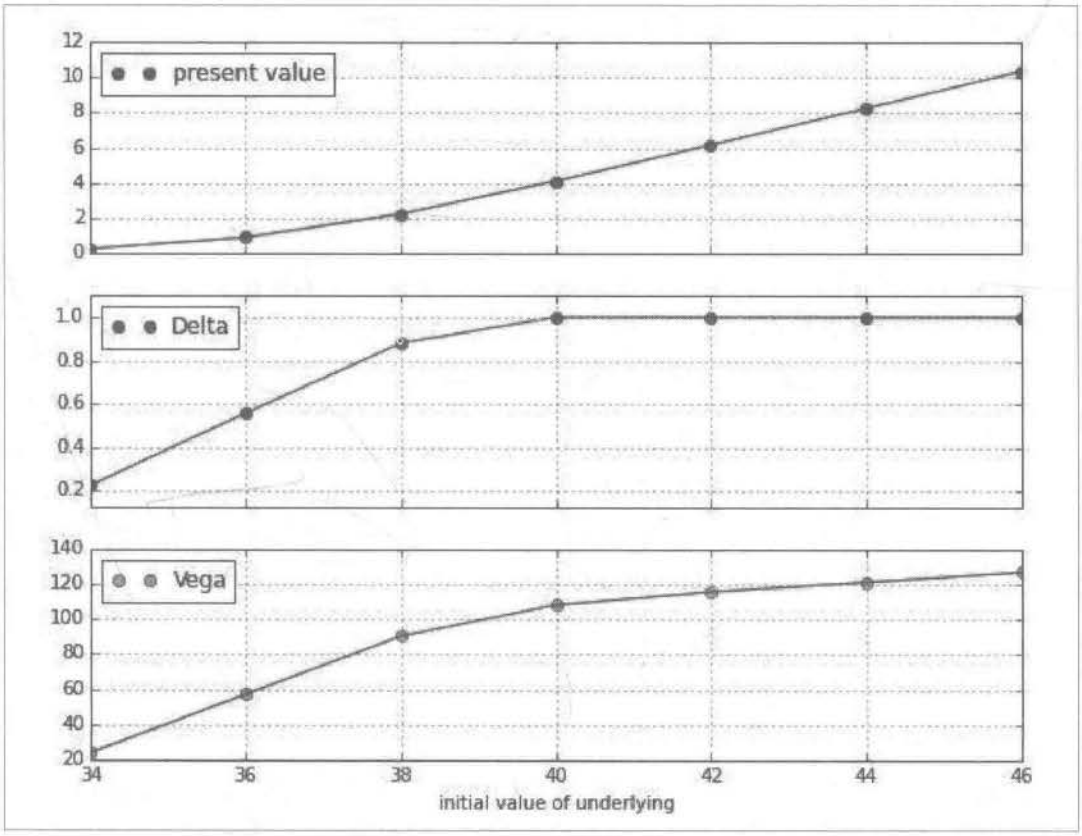


图 17-2　欧式-亚式看涨期权的现值、Delta 和 Vega 估算

17.4　美式行权

美式期权（或者百慕大式期权[1]）的估值比欧式期权要复杂得多。因此，在进入估值类之前，我们必须首先介绍更多估值理论。

17.4.1　最小二乘蒙特卡洛方法

Cox、Ross 和 Rubinstein（1979）提出了他们的二项式模型，在相同框架里用简单的数值化方法估算欧式和美式期权的价值，但是只有 Longstaff-Schwartz（2001）的模型令人满意地解决了用蒙特卡洛模拟（MCS）估计美式期权价值的问题。主要的问题是，MCS 本身是前移式算法，而美式期权的估值通常利用倒推法实现——从到期日开始估算美式期权的持续价值并倒推出现值。

1　美式行权指的是行权可以在一段固定期间中的每个时点（至少在交易时间内）进行的情况。百慕大行权通常指的是有多个离散行权日的情况，在数值应用中，美式行权由百慕大行权近似计算，在极限的情况下，行权日期的数量可能不限。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

Longstaff-Schwartz（2001）模型的深意在于使用了普通最小二乘回归[1]，根据所有可用模拟值的交点估算持续价值——对每条路径考虑如下因素：

- 标的模拟价值；
- 期权内在价值；
- 特定路径的实际持续价值。

在离散时间中，百慕大期权（以及极限情况下的美式期权）的价值由*最优截止问题*给出，对于有限时点集合 $0<t_1<t_2<\cdots<T$[4]，如公式 17-3 所示。

公式 17-3　百慕大期权离散时间中的最优截止问题

$$V_0 = \sup_{\tau\in\{0,t_1,t_2,\cdots,T\}} e^{-r\tau}\mathrm{E}_0^Q(h_\tau(S_\tau)) \qquad (17\text{-}3)$$

公式 17-4 提供了日期 $0\leqslant t_m< T$ 时美式期权的持续价值。这就是在美式期权 V_{tm+1} 后续日期价值的鞅测度下，日期 t_{m+1} 时的风险中立预期。

公式 17-4　美式期权持续价值

$$C_{t_m}(s) = e^{-r(t_{m+1}-t_m)}\mathrm{E}_{t_m}^Q(V_{t_{m+1}}(S_{t_{m+1}}) \mid St_m = s) \qquad (17\text{-}4)$$

美式期权 V_{tm} 在日期 t_m 时的价值也可以用公式 17-5 中的公式表示——立即行权的收益（内在价值）和不行权预期收益（持续价值）两者的最大值。

公式 17-5　美式权证在任何给定日期的价值

$$V_{t_m} = \max(h_{t_m}(s), C_{t_m}(s)) \qquad (17\text{-}5)$$

在公式 17-5 中，内在价值当然很容易计算。持续价值则稍微有些难度，Longstaff-Schwartz（2001）模型通过公式 17-6 中提供的回归公式近似计算该值。其中，i 表示当前模拟路径，D 是所使用回归基函数的数量，α^* 是最优回归参数，b_d 是编号为 d 的回归函数。

公式 17-6　基于回归的持续价值近似计算

$$\overline{C}_{t_m,i} = \sum_{d=1}^{D}\alpha^*_{d,t_m}\cdot b_d(S_{t_m,i}) \qquad (17\text{-}6)$$

最优回归参数是公式 17-7 中提出的最小二乘回归问题的解。式中，$Y_{t_m,i} \equiv e^{-r(t_{m+1}-t_m)}V_{t_{m+1},i}$

1　这就是这种算法通常缩写为 LSM（最小二乘蒙特卡洛）的原因。——原注

2　Kohler（2010）提供了美式期权估值理论的简洁概述，尤其是基于回归方法的使用。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

是路径 i 在日期 t_m 时的实际持续价值。

公式 17-7　普通最小二乘回归

$$\min_{\alpha_{1,t_m},\cdots,\alpha_{D,t_m}} \frac{1}{I}\sum_{i=1}^{I}\left(Y_{t_m,i}-\sum_{d=1}^{D}\alpha_{d,t_m}\cdot b_d(S_{t_m,i})\right)^2 \tag{17-7}$$

这就完成了通过 MCS 估算美式期权价值的基本（数学）工具集。

17.4.2　估值类

例 17-4 提供了采用美式行权方式的期权和衍生品的估值类。在 **present_value** 方法中 LSM 算法的实现有一个值得注意的步骤（在内嵌注释中也体现了）：*最优决策步骤*。这里重要的是，根据做出的决策，LSM 算法采用内在价值或者实际持续价值——而不是估算的持续价值[1]。

例 17-4　美式行权估值类

```
#
# DX Library Valuation
# valuation_mcs_american.py
#
import numpy as np

from valuation_class import valuation_class

class valuation_mcs_american(valuation_class):
    ''' Class to value American options with arbitrary payoff
    by single-factor Monte Carlo simulation.

    Methods
    =======
    generate_payoff :
        returns payoffs given the paths and the payoff function
    present_value :
        returns present value (LSM Monte Carlo estimator)
        according to Longstaff-Schwartz (2001)
    '''

    def generate_payoff(self, fixed_seed=False):
        '''
        Parameters
        ==========
        fixed_seed :
            use same/fixed seed for valuation
        '''
        try:
            strike = self.strike
        except:
            pass
        paths = self.underlying.get_instrument_values(fixed_seed=fixed_seed)
```

1　参加 Hilpisch(2015)的第 6 章。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        time_grid = self.underlying.time_grid
        try:
            time_index_start = int(np.where(time_grid == self.pricing_date)[0])
            time_index_end = int(np.where(time_grid == self.maturity)[0])
        except:
            print "Maturity date not in time grid of underlying."
        instrument_values = paths[time_index_start:time_index_end + 1]
        try:
            payoff = eval(self.payoff_func)
            return instrument_values, payoff, time_index_start, time_index_end
        except:
            print "Error evaluating payoff function."
    def present_value(self, accuracy=6, fixed_seed=False, bf=5, full=False):
        '''
        Parameters
        ==========
        accuracy : int
            number of decimals in returned result
        fixed_seed : boolean
            use same/fixed seed for valuation
        bf : int
            number of basis functions for regression
        full : Boolean
            return also full 1d array of present values
        '''
        instrument_values, inner_values, time_index_start, time_index_end = \
                    self.generate_payoff(fixed_seed=fixed_seed)
        time_list = self.underlying.time_grid[time_index_start:time_index_end + 1]
        discount_factors = self.discount_curve.get_discount_factors(
                            time_list, dtobjects=True)
        V = inner_values[-1]
        for t in range(len(time_list) - 2, 0, -1):
            # derive relevant discount factor for given time interval
            df = discount_factors[t, 1] / discount_factors[t + 1, 1]
            # regression step
            rg = np.polyfit(instrument_values[t], V * df, bf)
            # calculation of continuation values per path
        C = np.polyval(rg, instrument_values[t])
        # optimal decision step:
        # if condition is satisfied (inner value > regressed cont. value)
        # then take inner value; take actual cont. value otherwise
        V = np.where(inner_values[t] > C, inner_values[t], V * df)
    df = discount_factors[0, 1] / discount_factors[1, 1]
    result = df * np.sum(V) / len(V)
    if full:
        return round(result, accuracy), df * V
    else:
        return round(result, accuracy)
```

17.4.3 用例

正如前面选择的方法，用例可以说明 **valuation_mcs_american** 类的用法。这个用例重复 Longstaff 和 Schwartz（2001）原创论文表 1 所介绍的所有美式期权价值。标的和前面一样，是一个 **geometric_brownian_motion** 对象。标的物的起始参数如下：

```
In [22]: from dx_simulation import *
In [23]: me_gbm = market_environment('me_gbm', dt.datetime(2015, 1, 1))
In [24]: me_gbm.add_constant('initial_value', 36.)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
me_gbm.add_constant('volatility', 0.2)
me_gbm.add_constant('final_date', dt.datetime(2016, 12, 31))
me_gbm.add_constant('currency', 'EUR')
me_gbm.add_constant('frequency', 'W')
  # weekly frequency
me_gbm.add_constant('paths', 50000)
```

```
In [25]: csr = constant_short_rate('csr', 0.06)
In [26]: me_gbm.add_curve('discount_curve', csr)
In [27]: gbm = geometric_brownian_motion('gbm', me_gbm)
```

*期权类型*是一种有收益的美式看跌期权：

```
In [28]: payoff_func = 'np.maximum(strike - instrument_values, 0)'
```

论文表 1 中的第一种期权的行权期为 1 年，行权价自始至终均为 40 美元：

```
In [29]: me_am_put = market_environment('me_am_put', dt.datetime(2015, 1, 1))
In [30]: me_am_put.add_constant('maturity', dt.datetime(2015, 12, 31))
         me_am_put.add_constant('strike', 40.)
         me_am_put.add_constant('currency', 'EUR')
```

下一步是根据数值化假设实例化估值对象：

```
In [31]: from valuation_mcs_american import valuation_mcs_american
In [32]: am_put = valuation_mcs_american('am_put', underlying=gbm,
                                         mar_env=me_am_put, payoff_func=payoff_func)
```

美式看跌期权估值花费的时间比欧式期权长得多。我们不仅增加了路径的数量和估值的频率，由于向后推导和每个推导步骤的回归，该算法的计算要求也更高。用数值化方法得到的价值基金与原创论文中报告的准确值 4.478 相当接近：

```
In [33]: %time am_put.present_value(fixed_seed=True, bf=5)
Out[33]: CPU times: user 1.36 s, sys: 239 ms, total: 1.6 s
         Wall time: 1.6 s

         4.470627
```

由于 LSM 蒙特卡洛估算函数的最初构造，它表示数学上正确的美式期*权价值下界*[1]。因此，可以预期，这种数值化估算的结果低于任何数值化实例的真值。

作为替代的对偶估算函数还能提供上界[2]。将两者结合起来，两个不同的估算函数可以定义美式期权真值的区间。

这个用例的主要目标是复制原创论文中表 1 的所有美式期权价值。为此，我们只需要结合估算对象和一个嵌套循环。在最内层循环中，估值对象必须根据当时的参数更新：

```
In [34]: %%time
         ls_table = []
         for initial_value in (36., 38., 40., 42., 44.):
             for volatility in (0.2, 0.4):
```

1 主要原因是基于持续价值回归估算的“最优行权策略”未能达到“最优”的标准。——原注

2 得出上界的对偶算法和 Python 实现参见 Hilpisch(2015)的第 6 章。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                for maturity in (dt.datetime(2015, 12, 31),
                                 dt.datetime(2016, 12, 31)):
                    am_put.update(initial_value=initial_value,
                                  volatility=volatility,
                                  maturity=maturity)
                    ls_table.append([initial_value,
                                     volatility,
                                     maturity,
                                     am_put.present_value(bf=5)])
Out[34]: CPU times: user 31.1 s, sys: 3.22 s, total: 34.3 s
         Wall time: 33.9 s
```

下面是 Longstaff 和 Schwartz（2001）论文表的简化版本。总体来说，我们的数值化结果相当接近论文中报告的结果，在论文中使用了一些不同的参数（例如，他们使用两倍数量的路径）：

```
In [35]: print "S0  | Vola | T | Value"
         print 22 * "-"
         for r in ls_table:
             print "%d   | %3.1f  | %d | %5.3f" % \
                   (r[0], r[1], r[2].year - 2014, r[3])
Out[35]: S0  | Vola | T | Value
         --------
         36  | 0.2  | 1 | 4.444
         36  | 0.2  | 2 | 4.769
         36  | 0.4  | 1 | 7.000
         36  | 0.4  | 2 | 8.378
         38  | 0.2  | 1 | 3.210
         38  | 0.2  | 2 | 3.645
         38  | 0.4  | 1 | 6.066
         38  | 0.4  | 2 | 7.535
         40  | 0.2  | 1 | 2.267
         40  | 0.2  | 2 | 2.778
         40  | 0.4  | 1 | 5.203
         40  | 0.4  | 2 | 6.753
         42  | 0.2  | 1 | 1.554
         42  | 0.2  | 2 | 2.099
         42  | 0.4  | 1 | 4.459
         42  | 0.4  | 2 | 6.046
         44  | 0.2  | 1 | 1.056
         44  | 0.2  | 2 | 1.618
         44  | 0.4  | 1 | 3.846
         44  | 0.4  | 2 | 5.494
```

在这个用例结束的时候要注意，美式期权的“希腊字母” 估算在形式上和欧式期权相同—— 这是我们的方法和其他数值化方法（如二项式模型）相比的重大优势之一：

```
In [36]: am_put.update(initial_value=36.)
         am_put.delta()
Out[36]: -0.4655
In [37]: am_put.vega()
Out[37]: 17.3411
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

17.5 结语

本章介绍了基于蒙特卡洛模拟的欧式和美式期权数值化估值。本章介绍了一个泛型估值类 **valuation_class**，这个类提供了许多方法，例如估算两类期权最重要的“希腊字母”（Delta、Vega）的方法，这项方法独立于估值使用的模拟对象（风险因素/随机过程）。

在泛型估值类的基础上，本章介绍了两个专用类：**valuation_mcs_european** 和 **valuation_mcs_american**。欧式期权的估值类主要是第 15 章中介绍的风险中立估值方法简单实现与一个预期项的数值化估算（正如第 9 章中所讨论的，是蒙特卡洛模拟的积分）的结合。

美式期权的估值类需要某种基于回归的估值算法。这是因为美式期权的估值需要最优行权策略。这在理论上和数值化方法上都更复杂一些，但是，对应类的 **present_value** 方法仍然很简洁。

已经证明，DX 衍生品分析库所采用的方法是很有益的。不需要花费太多的精力，我们就可以估算具有如下特征的一大类期权的价值：

- 单一风险因素期权
- 欧式或者美式行权
- 任意收益

此外，我们可以估算这类期权最重要的“希腊字母”。为了简化未来的导入，我们再次使用一个包装器模块 **dx_valuation.py**，如例 17-5 所示。

例 17-5　DX 库所有组件（包含估值类）的包装器模块

```
#
# DX Library Valuation
# dx_valuation.py
#
import numpy as np
import pandas as pd

from dx_simulation import *
from valuation_class import valuation_class
from valuation_mcs_european import valuation_mcs_european
from valuation_mcs_american import valuation_mcs_american
```

我们还将再次改进 **dx** 目录中的 **init** 文件（参见例 17-6），使其保持一致。

例 17-6　改进的 Python 打包文件

```
#
# DX Library
# packaging file
# __init__.py
#
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
import numpy as np
import pandas as pd
import datetime as dt

# frame
from get_year_deltas import get_year_deltas
from constant_short_rate import constant_short_rate
from market_environment import market_environment

from plot_option_stats import plot_option_stats

# simulation
from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class
from geometric_brownian_motion import geometric_brownian_motion
from jump_diffusion import jump_diffusion
from square_root_diffusion import square_root_diffusion

# valuation
from valuation_class import valuation_class
from valuation_mcs_european import valuation_mcs_european
from valuation_mcs_american import valuation_mcs_american
```

17.6 延伸阅读

本章中的主题有如下参考书：

- Glasserman, Paul (2004): Monte Carlo Methods in Financial Engineering. Springer, New York.
- Hilpisch, Yves (2015): Derivatives Analytics with Python. Wiley Finance, Chichester, England. http://derivatives-analytics-with-python.com.

本章引用的原创论文：

- Cox, John, Stephen Ross, and Mark Rubinstein (1979): "Option Pricing: A Simplified Approach." *Journal of Financial Economics*, Vol. 7, No. 3, pp. 229-263.
- Kohler, Michael (2010): "A Review on Regression-Based Monte Carlo Methods forPricing American Options." In Luc Devroye et al. (eds.): *Recent Developments inApplied Probability and Statistics*. Physica-Verlag, Heidelberg, pp. 37-58.
- Longstaff, Francis and Eduardo Schwartz (2001): "Valuing American Options by Simulation: A Simple Least Squares Approach." *Review of Financial Studies*, Vol.14, No. 1, pp. 113-147.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 18 章

投资组合估值

价格是你所付出的，价值是你所得到的。

——沃伦•巴菲特

到目前为止，构建 DX 衍生品分析库的完整方法——以及相关的好处——应该已经相当清楚。通过严格地依赖蒙特卡洛模拟作为唯一的数**值**方法，我们几乎已经完成了分析库的模块化工作：

折现

constant_short_rate 类的实例负责处理对应的风险中立折现。

相关数据

相关数据、参数和其他输入保存在 **market_environment** 类的（多个）实例中。

模拟对象

相关风险因素（标的）的模型是如下 3 个模拟类之一的实例：

- **geometric_brownian_motion**
- **jump_diffusion**
- **square_root_diffusion**

估值对象

需要**估值**的期权和衍生品的模型是如下两个**估值**类之一的实例：

- **valuation_mcs_european**
- **valuation_mcs_american**

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

最后还需要一步：期权和衍生品**投资组合**（可能很复杂）的**估值**。为此，我们需要如下要素。

非冗余性

每种风险因素（标的）只能建模一次，可能供多个**估值**对象使用。

相关性

必须考虑风险因素之间的相关性。

头寸

例如，期权头寸可以是某个期权**合**约的整数倍。

不过，尽管在理论上允许（甚至要求）为模拟和**估值**对象分别提供币种，但是我们假定所要**估值**的**投资组合**仅以单一货币标价。这明显简化了**投资组合**中价**值**汇总的工作，因为我们无需考虑汇率和货币风险。

本章介绍两个新类：较为简单的一个类建立*衍生品头寸*模型，较复杂的一个类建立*衍生品投资组合*模型并进行**估值**。

18.1 衍生品头寸

理论上，衍生品头寸不过就是用于建立金融工具模型的一个**估值**对象和一个数量的**组合**。

18.1.1 类

例 18-1 提供了建立衍生品头寸模型的类。它主要是一个其他数据和对象的容器。此外，它提供 **get_info** 方法，打印保存在类实例中的数据和对象信息。

例 18-1 简单的衍生品头寸模型类

```
#
# DX Library Portfolio
# derivatives_position.py
#

class derivatives_position(object):
    ''' Class to model a derivatives position.

    Attributes
    ==========

    name : string
        name of the object
    quantity : float
        number of assets/derivatives making up the position
    underlying : string
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        name of asset/risk factor for the derivative
    mar_env : instance of market_environment
        constants, lists, and curves relevant for valuation_class
    otype : string
        valuation class to use
    payoff_func : string
        payoff string for the derivative

    Methods
    =======
    get_info :
        prints information about the derivative position
    '''
def __init__(self, name, quantity, underlying, mar_env, otype, payoff_func):
    self.name = name
    self.quantity = quantity
    self.underlying = underlying
    self.mar_env = mar_env
    self.otype = otype
    self.payoff_func = payoff_func


def get_info(self):
    print "NAME"
    print self.name, '\n'
    print "QUANTITY"
    print self.quantity, '\n'
    print "UNDERLYING"
    print self.underlying, '\n'
    print "MARKET ENVIRONMENT"
    print "\n**Constants**"
    for key, value in self.mar_env.constants.items():
        print key, value
    print "\n**Lists**"
    for key, value in self.mar_env.lists.items():
        print key, value
    print "\n**Curves**"
    for key in self.mar_env.curves.items():
        print key, value
    print "\nOPTION TYPE"
    print self.otype, '\n'
    print "PAYOFF FUNCTION"
    print self.payoff_func
```

为了定义衍生品头寸，我们需要提供如下信息，这几乎和**估值**类的实例化相同：

name

表示头寸名称的字符串对象

quantity

期权/衍生品数量

underlying

作为风险因素的模拟对象实例

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

mar_env

market_environment 实例

otype

字符串，“European”或者“American”

payoff_func

表示收益的 **Python** 字符串对象

18.1.2 用例

下面的交互式会话说明了该类的用法。不过，我们必须首先定义一个模拟对象——但是不是完整的；只需要最终的对象特性信息。这里，我们基本上坚持使用前两章中的数**值**化示例：

```
In [1]: from dx import *
```

为了定义衍生品头寸，我们不需要“完整的”**market_environment** 对象。遗漏的信息在后面（**投资组合估值**期间）模拟对象初始化时提供：

```
In [2]: me_gbm = market_environment('me_gbm', dt.datetime(2015, 1, 1))
In [3]: me_gbm.add_constant('initial_value', 36.)
        me_gbm.add_constant('volatility', 0.2)
        me_gbm.add_constant('currency', 'EUR')
```

不过，对于**投资组合估值**，需要另一个常量——所要使用的*模型*。这在后面的小节中将变得显而易见：

```
In [4]: me_gbm.add_constant('model', 'gbm')
```

模拟对象可用之后，就可以继续定义衍生品头寸：

```
In [5]: from derivatives_position import derivatives_position
In [6]: me_am_put = market_environment('me_am_put', dt.datetime(2015, 1, 1))
In [7]: me_am_put.add_constant('maturity', dt.datetime(2015, 12, 31))
        me_am_put.add_constant('strike', 40.)
        me_am_put.add_constant('currency', 'EUR')
In [8]: payoff_func = 'np.maximum(strike - instrument_values, 0)'
In [9]: am_put_pos = derivatives_position(
                     name='am_put_pos',
                     quantity=3,
                     underlying='gbm',
                     mar_env=me_am_put,
                     otype='American',
                     payoff_func=payoff_func)
```

关于这个对象的信息由 **get_info** 方法提供：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [10]: am_put_pos.get_info()
Out[10]: NAME
         am_put_pos

         QUANTITY
         3

         UNDERLYING
         gbm

         MARKET ENVIRONMENT

         **Constants**
         strike 40.0
         maturity 2015-12-31 00:00:00
         currency EUR

         **Lists**

         **Curves**

         OPTION TYPE
         American

         PAYOFF FUNCTION
         np.maximum(strike - instrument_values, 0)
```

18.2 衍生品投资组合

从**投资组合**的角度看，“相关市场”主要是由相关风险因素（标的）及其相关性，以及所要**估值**的衍生品和衍生品头寸**组**成的。从理论上，我们现在要处理的是一个一般市场模型 μ（如第 15 章中的定义），并对其应用**资**产定价基本定理（及其推论）[1]。

18.2.1 类

例 18-2 提供了一个稍复杂的 **Python** 类，根据**资**产定价基本定理实施**投资组合估值**——考虑了多种相关风险因素和多种衍生品头寸。该类的内嵌文档相当完备，特别是实现特定目的的段落。

例 18-2 估算衍生品投资组合的类

```
#
# DX Library Portfolio
# derivatives_portfolio.py
#
import numpy as np
```

1 在实践中，我们在此选择的方法有时候称作“总体估值”而不是“特定金融工具估值”。参考 Albanese、Gimonet 和 White（2010a）在《Risk Magazine》上发表的论文。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
import pandas as pd

from dx_valuation import *

# models available for risk factor modeling
models = {'gbm' : geometric_brownian_motion,
          'jd' : jump_diffusion,
          'srd' : square_root_diffusion}

# allowed exercise types
otypes = {'European' : valuation_mcs_european,
          'American' : valuation_mcs_american}


class derivatives_portfolio(object):
    ''' Class for building portfolios of derivatives positions.

    Attributes
    ==========
    name : str
        name of the object
    positions : dict
        dictionary of positions (instances of derivatives_position class)
    val_env : market_environment
        market environment for the valuation
    assets : dict
        dictionary of market environments for the assets
    correlations : list
        correlations between assets
    fixed_seed : Boolean
        flag for fixed rng seed

    Methods
    =======
    get_positions :
        prints information about the single portfolio positions
    get_statistics :
        returns a pandas DataFrame object with portfolio statistics
    '''

    def __init__(self, name, positions, val_env, assets,
                 correlations=None, fixed_seed=False):
        self.name = name
        self.positions = positions
        self.val_env = val_env
        self.assets = assets
        self.underlyings = set()
        self.correlations = correlations
        self.time_grid = None
        self.underlying_objects = {}
        self.valuation_objects = {}
        self.fixed_seed = fixed_seed
        self.special_dates = []
        for pos in self.positions:
            # determine earliest starting_date
            self.val_env.constants['starting_date'] = \
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                min(self.val_env.constants['starting_date'],
                    positions[pos].mar_env.pricing_date)
        # determine latest date of relevance
        self.val_env.constants['final_date'] = \
                max(self.val_env.constants['final_date'],
                    positions[pos].mar_env.constants['maturity'])
        # collect all underlyings
        # add to set; avoids redundancy
        self.underlyings.add(positions[pos].underlying)

    # generate general time grid
    start = self.val_env.constants['starting_date']
    end = self.val_env.constants['final_date']
    time_grid = pd.date_range(start=start,end=end,
                freq=self.val_env.constants['frequency']
                ).to_pydatetime()
    time_grid = list(time_grid)
    for pos in self.positions:
        maturity_date = positions[pos].mar_env.constants['maturity']
        if maturity_date not in time_grid:
            time_grid.insert(0, maturity_date)
            self.special_dates.append(maturity_date)
    if start not in time_grid:
        time_grid.insert(0, start)
    if end not in time_grid:
        time_grid.append(end)
    # delete duplicate entries
    time_grid = list(set(time_grid))
    # sort dates in time_grid
    time_grid.sort()
    self.time_grid = np.array(time_grid)
    self.val_env.add_list('time_grid', self.time_grid)

    if correlations is not None:
        # take care of correlations
        ul_list = sorted(self.underlyings)
        correlation_matrix = np.zeros((len(ul_list), len(ul_list)))
        np.fill_diagonal(correlation_matrix, 1.0)
        correlation_matrix = pd.DataFrame(correlation_matrix,
                             index=ul_list, columns=ul_list)
        for i, j, corr in correlations:
            corr = min(corr, 0.999999999999)
            # fill correlation matrix
            correlation_matrix.loc[i, j] = corr
            correlation_matrix.loc[j, i] = corr
        # determine Cholesky matrix
        cholesky_matrix = np.linalg.cholesky(np.array(correlation_matrix))

        # dictionary with index positions for the
        # slice of the random number array to be used by
        # respective underlying
        rn_set = {asset: ul_list.index(asset)
                  for asset in self.underlyings}

        # random numbers array, to be used by
        # all underlyings (if correlations exist)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        random_numbers = sn_random_numbers((len(rn_set),
                                    len(self.time_grid),
                                   self.val_env.constants['paths']),
                                   fixed_seed=self.fixed_seed)


        # add all to valuation environment that is
        # to be shared with every underlying
        self.val_env.add_list('cholesky_matrix', cholesky_matrix)
            self.val_env.add_list('random_numbers', random_numbers)
            self.val_env.add_list('rn_set', rn_set)

        for asset in self.underlyings:
            # select market environment of asset
            mar_env = self.assets[asset]
            # add valuation environment to market environment
            mar_env.add_environment(val_env)
            # select right simulation class
            model = models[mar_env.constants['model']]
            # instantiate simulation object
            if correlations is not None:
                self.underlying_objects[asset] = model(asset, mar_env,
                                                       corr=True)
            else:
                self.underlying_objects[asset] = model(asset, mar_env,
                                                       corr=False)

        for pos in positions:
            # select right valuation class (European, American)
            val_class = otypes[positions[pos].otype]
            # pick market environment and add valuation environment
            mar_env = positions[pos].mar_env
            mar_env.add_environment(self.val_env)
            # instantiate valuation class
            self.valuation_objects[pos] = \
                val_class(name=positions[pos].name,
                          mar_env=mar_env,
                          underlying=self.underlying_objects[
                                              positions[pos].underlying],
                          payoff_func=positions[pos].payoff_func)

    def get_positions(self):
        ''' Convenience method to get information about
        all derivatives positions in a portfolio. '''
        for pos in self.positions:
            bar = '\n' + 50 * '-'
            print bar
            self.positions[pos].get_info()
            print bar

    def get_statistics(self, fixed_seed=False):
        ''' Provides portfolio statistics. '''
        res_list = []
        # iterate over all positions in portfolio
        for pos, value in self.valuation_objects.items():
            p = self.positions[pos]
            pv = value.present_value(fixed_seed=fixed_seed)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            res_list.append([
                p.name,
                p.quantity,
                # calculate all present values for the single instruments
                pv,
                value.currency,
                # single instrument value times quantity
                pv * p.quantity,
                # calculate Delta of position
                value.delta() * p.quantity,
                # calculate Vega of position
                value.vega() * p.quantity,
            ])
        # generate a pandas DataFrame object with all results
        res_df = pd.DataFrame(res_list,
                    columns=['name', 'quant.', 'value', 'curr.',
                             'pos_value', 'pos_delta', 'pos_vega'])
        return res_df
```

18.2.2 用例

按照 DX 分析库的设计概略看来，建模能力限制在一个模拟类和一个**估值**类的**组合**，共有 6 种可能的**组合**：

```
models = {'gbm' : geometric_brownian_motion,
          'jd' : jump_diffusion
          'srd': square_root_diffusion}

otypes = {'European' : valuation_mcs_european,
          'American' : valuation_mcs_american}
```

在下面的交互式用例中，我们**组合**选择的元素，定义两种不同的衍生品头寸，将其**合并**为一个**投资组合**。

我们以 **derivatives_position** 类和来自前一小节的 **gbm** 及 **am_put_pos** 对象为基础构建这个用例，为了说明 **derivatives_position** 类的用法，我们定义一个附加的标的和附加的期权头寸。首先是一个 **jump_diffusion** 对象：

```
In [11]: me_jd = market_environment('me_jd', me_gbm.pricing_date)
In [12]: # add jump diffusion-specific parameters
         me_jd.add_constant('lambda', 0.3)
         me_jd.add_constant('mu', -0.75)
         me_jd.add_constant('delta', 0.1)
         # add other parameters from gbm
         me_jd.add_environment(me_gbm)
In [13]: # needed for portfolio valuation
         me_jd.add_constant('model', 'jd')
```

其次是基于这个新模拟对象的欧式看涨期权：

```
In [14]: me_eur_call = market_environment('me_eur_call', me_jd.pricing_date)
In [15]: me_eur_call.add_constant('maturity', dt.datetime(2015, 6, 30))
         me_eur_call.add_constant('strike', 38.)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
me_eur_call.add_constant('currency', 'EUR')
```

```
In [16]: payoff_func = 'np.maximum(maturity_value - strike, 0)'
```

```
In [17]: eur_call_pos = derivatives_position(
                        name='eur_call_pos',
                        quantity=5,
                        underlying='jd',
                        mar_env=me_eur_call,
                        otype='European',
                        payoff_func=payoff_func)
```

从**投资组合**的角度看，相关市场现在如下：

```
In [18]: underlyings = {'gbm': me_gbm, 'jd' : me_jd}
         positions = {'am_put_pos' : am_put_pos, 'eur_call_pos' : eur_call_pos}
```

现在我们抽取标的物之间的相关性。为**投资组合估值**编辑 **market_environment** 是实例化 **derivatives_portfolio** 类之前的最后一步：

```
In [19]: # discounting object for the valuation
         csr = constant_short_rate('csr', 0.06)
```

```
In [20]: val_env = market_environment('general', me_gbm.pricing_date)
         val_env.add_constant('frequency', 'W')
           # monthly frequency
         val_env.add_constant('paths', 25000)
         val_env.add_constant('starting_date', val_env.pricing_date)
         val_env.add_constant('final_date', val_env.pricing_date)
           # not yet known; take pricing_date temporarily
         val_env.add_curve('discount_curve', csr)
           # select single discount_curve for whole portfolio
```

```
In [21]: from derivatives_portfolio import derivatives_portfolio
```

```
In [22]: portfolio = derivatives_portfolio(
                            name='portfolio',
                            positions=positions,
                            val_env=val_env,
                            assets=underlyings,
                            fixed_seed=True)
```

现在，我们可以利用**估值**类的能力，获得刚刚定义的 **derivatives_portfolio** 对象的不同统计数字：

```
In [23]: portfolio.get_statistics()
```

```
Out[23]:
            name    quant.      value   curr.   pos_value   pos_delta   pos_vega
0   eur_call_pos         5   2.814638     EUR   14.073190      3.3605    42.7900
1    am_put_pos         3   4.472021     EUR   13.416063     -2.0895    30.5181
```

头寸价**值**总和、Delta 和 Vega 也很容易计算。这个**投资组合**的 Delta（几乎中立）和 Vega 稍微大一些：

```
In [24]: portfolio.get_statistics()[['pos_value', 'pos_delta', 'pos_vega']].sum()
           # aggregate over all positions
```

```
Out[24]: pos_value    27.489253
         pos_delta     1.271000
         pos_vega     73.308100
         dtype: float64
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

用 **get_positions** 方法可以很方便地得到所有头寸的完整概况——例如，这些输出可以用于报表目的（因为篇幅所限，这里略去）：

```
In [25]: portfolio.get_positions()
```

当然，也可以直接访问和使用 derivatives_portfolio 对象的所有（模拟、估值等）对象：

```
In [26]: portfolio.valuation_objects['am_put_pos'].present_value()
Out[26]: 4.450573
In [27]: portfolio.valuation_objects['eur_call_pos'].delta()
Out[27]: 0.6498
```

这种衍生品投资组合估值根据风险因素不相关的假设进行。这很容易通过检查两条模拟路径（每个模拟对象一条）进行验证：

```
In [28]: path_no = 777
         path_gbm = portfolio.underlying_objects['gbm'].get_instrument_values()[
                                                                    :, path_no]
         path_jd = portfolio.underlying_objects['jd'].get_instrument_values()[
                                                                    :, path_no]
```

图 18-1 直接比较选择的路径——没有发生跳跃扩散中的跳跃：

```
In [29]: import matplotlib.pyplot as plt
         %matplotlib inline
In [30]: plt.figure(figsize=(7, 4))
         plt.plot(portfolio.time_grid, path_gbm, 'r', label='gbm')
         plt.plot(portfolio.time_grid, path_jd, 'b', label='jd')
         plt.xticks(rotation=30)
         plt.legend(loc=0); plt.grid(True)
```

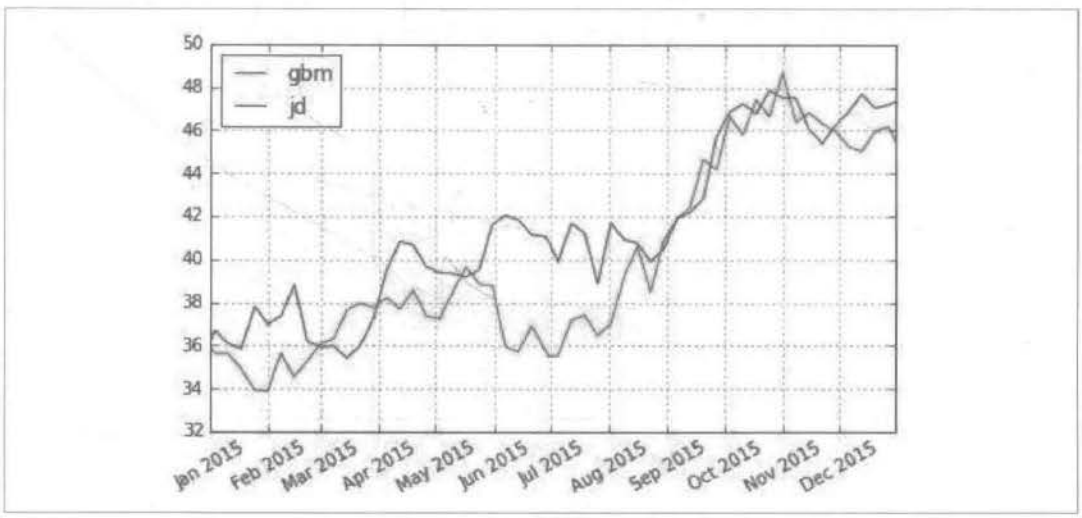


图 18-1　不相关的风险因素

现在考虑两种风险因素有高度正相关的情况：

```
In [31]: correlations = [['gbm', 'jd', 0.9]]
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

利用这些附加信息，实例化一个新的 **derivatives_portfolio** 对象：

```
In [32]: port_corr = derivatives_portfolio(
                            name='portfolio',
                            positions=positions,
                            val_env=val_env,
                            assets=underlyings,
                            correlations=correlations,
                            fixed_seed=True)
```

在这种情况下，对投资组合中头寸的价值没有直接影响：

```
In [33]: port_corr.get_statistics()
Out[33]:
           name   quant.      value   curr.    pos_value   pos_delta   pos_vega
0   eur_call_pos        5   2.804464    EUR    14.022320      3.3760    42.3500
1    am_put_pos        3   4.458565    EUR    13.375695     -2.0313    30.1416
```

但是，相关性在幕后起作用。为了得到图例，我们取和前面一样的两条路径：

```
In [34]: path_gbm = port_corr.underlying_objects['gbm'].\
                        get_instrument_values()[:, path_no]
         path_jd = port_corr.underlying_objects['jd'].\
                        get_instrument_values()[:, path_no]
```

图 18-2 现在显示，两种风险因素几乎完全并行地发展：

```
In [35]: plt.figure(figsize=(7, 4))
         plt.plot(portfolio.time_grid, path_gbm, 'r', label='gbm')
         plt.plot(portfolio.time_grid, path_jd, 'b', label='jd')
         plt.xticks(rotation=30)
         plt.legend(loc=0); plt.grid(True)
```

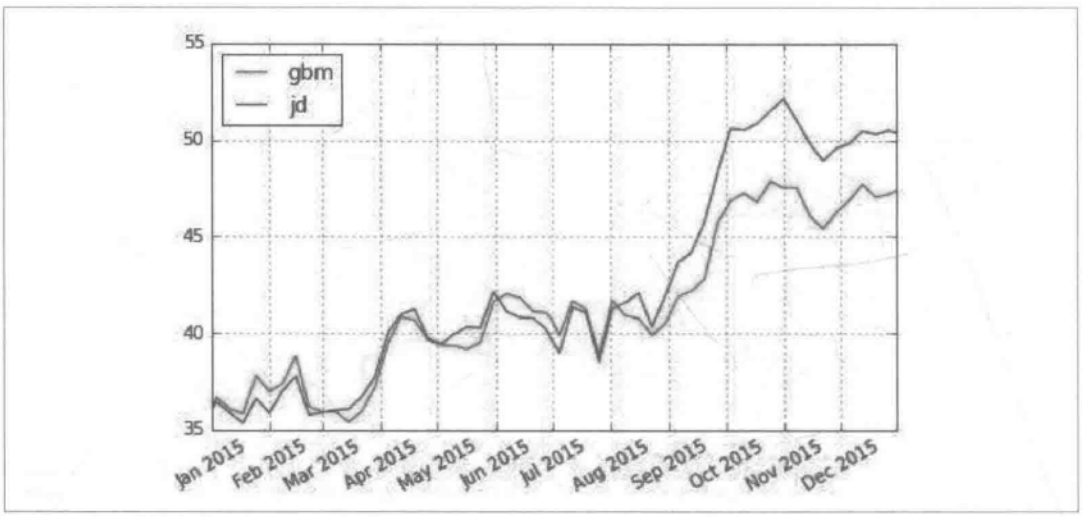


图 18-2　高度相关的风险因素

作为最后一个数值化和概念示例，*考虑投资组合现值的频率分布*。这是采用其他方法（如应用分析公式或者二项式期权定价模型）通常不可能生成的。我们通过计算现值并

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

传递参数标志 **full=True**，获得一组完整的各种期权头寸现值：

```
In [36]: pv1 = 5 * port_corr.valuation_objects['eur_call_pos'].\
                   present_value(full=True)[1]
         pv1
Out[36]: array([  0.        ,  22.55857473,   8.2552922 , ...,   0.        ,
                  0.        ,   0.        ])
In [37]: pv2 = 3 * port_corr.valuation_objects['am_put_pos'].\
                   present_value(full=True)[1]
         pv2
Out[37]: array([ 22.04450095,  10.90940926,  20.25092898, ...,  21.68232889,
                 17.7583897 ,   0.        ])
```

首先，我们比较两种头寸的频率分布。正如图 18-3 中所显示的，两种头寸的收益剖面有很大不同。注意，为了更好的可读性，我们限制了 x 和 y 轴：

```
In [38]: plt.hist([pv1, pv2], bins=25,
                  label=['European call', 'American put']);
         plt.axvline(pv1.mean(), color='r', ls='dashed',
                     lw=1.5, label='call mean = %4.2f' % pv1.mean())
         plt.axvline(pv2.mean(), color='r', ls='dotted',
                     lw=1.5, label='put mean = %4.2f' % pv2.mean())
         plt.xlim(0, 80); plt.ylim(0, 10000)
         plt.grid(); plt.legend()
```

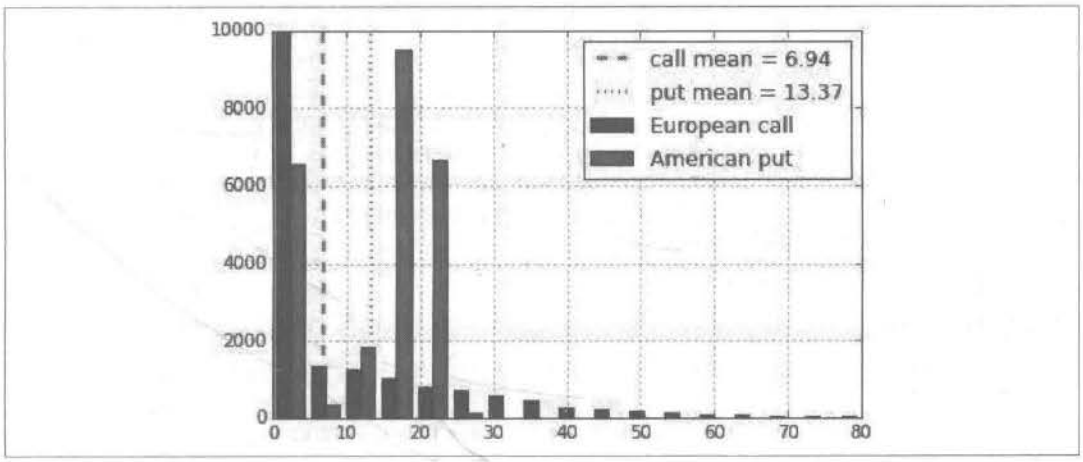


图 18-3　投资组合现值频率分布

下面的插图最终显示了投资组合现值的全部频率分布。在图 18-4 中可以清晰地看到组合看涨和看跌期权的多样化对冲效应：

```
In [39]: pvs = pv1 + pv2
         plt.hist(pvs, bins=50, label='portfolio');
         plt.axvline(pvs.mean(), color='r', ls='dashed',
                     lw=1.5, label='mean = %4.2f' % pvs.mean())
         plt.xlim(0, 80); plt.ylim(0, 7000)
         plt.grid(); plt.legend()
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

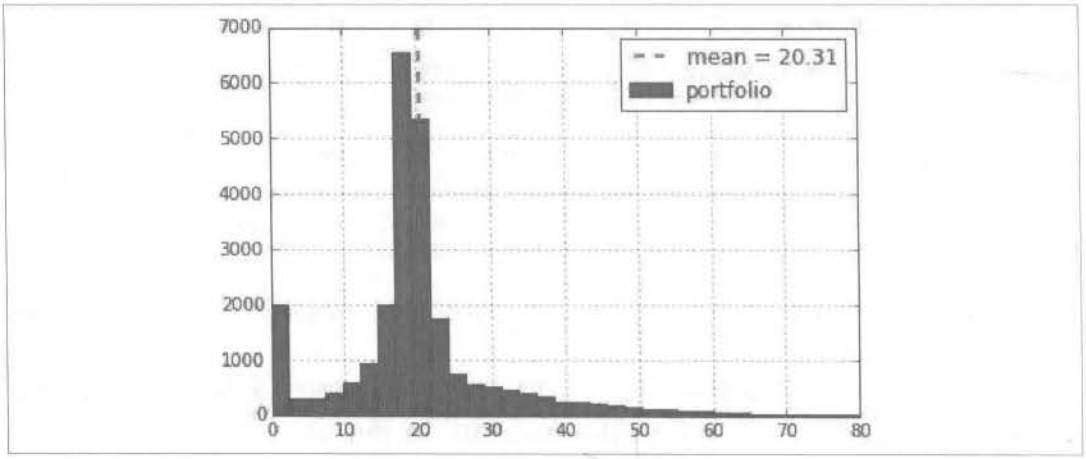


图 18-4　投资组合现值频率分布

以限制标准差为度量指标，两种风险因素之间的相关性对投资组合的风险有何影响？有相关性的投资组合的统计数字很容易计算：

```
In [40]: # portfolio with correlation
         pvs.std()
Out[40]: 16.736290069957963
```

类似地，对没有相关性的投资组合，计算如下：

```
In [41]: # portfolio without correlation
         pv1 = 5 * portfolio.valuation_objects['eur_call_pos'].\
                   present_value(full=True)[1]
         pv2 = 3 * portfolio.valuation_objects['am_put_pos'].\
                   present_value(full=True)[1]
         (pv1 + pv2).std()
Out[41]: 21.71542409437863
```

虽然均值保持恒定（忽略数值化误差），在这种计量方式下，相关性明显降低了投资组合风险。同样，这是使用其他数值化方法或者估值方法不可能得到的认识。

18.3　结语

本章介绍依赖多种可能相关的风险因素的衍生品头寸投资组合的估值和风险管理。为此，我们引入了一个新类 **derivatives_position**，以建立期权/衍生品头寸的模型。但是，主要的焦点放在 **derivatives_portfolio** 类，该类实现了一些相当复杂的任务。例如，该类负责以下工作：

- 风险因素之间的*相关性*（该类为所有风险因素的模拟生成一组一致性的随机数）；
- 在单一市场环境和一般估值环境以及衍生品头寸下的*模拟对象实例化*；

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

- 根据所有假设、所涉风险因素和衍生品头寸**生成投资组合统计数字**。

本章介绍的例子只能展示可由 DX 库和 **derivatives_portfolio** 类管理和估值的简单衍生品投资组合。对 DX 库的自然扩展应该添加更复杂的金融模型，例如随机波动率模型，并添加多重风险估值类建立依赖多种风险因素的衍生品（如欧洲一揽子期权或者美国最大看涨期权）模型并为之估值。在这一阶段，和资产定价基本定理（“全局估值”）一样通用的模块化建模以及估值框架应用展现出优势：风险因素的非冗余建模以及对风险因素之间相关性的考虑也对多风险衍生品的价值和“希腊字母”有影响。

例 18-3 是一个简短的最终包装器模块，在单条 **import** 语句中组合了 DX 分析库的所有组件。

例 18-3　组合所有 DX 组件的最终包装器模块

```
#
# DX Library Simulation
# dx_library.py
#
from dx_valuation import *
from derivatives_position import derivatives_position
from derivatives_portfolio import derivatives_portfolio
```

例 18-4 还提供了完整的 **dx** 目录 **init** 文件。

例 18-4　最终 Python 打包文件

```
#
# DX Library
# packaging file
# __init__.py
#
import numpy as np
import pandas as pd
import datetime as dt

# frame
from get_year_deltas import get_year_deltas
from constant_short_rate import constant_short_rate
from market_environment import market_environment
from plot_option_stats import plot_option_stats

# simulation
from sn_random_numbers import sn_random_numbers
from simulation_class import simulation_class
from geometric_brownian_motion import geometric_brownian_motion
from jump_diffusion import jump_diffusion
from square_root_diffusion import square_root_diffusion

# valuation
from valuation_class import valuation_class
from valuation_mcs_european import valuation_mcs_european
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
from valuation_mcs_american import valuation_mcs_american

# portfolio
from derivatives_position import derivatives_position
from derivatives_portfolio import derivatives_portfolio
```

18.4 延伸阅读

和前面关于 DX 衍生品分析库的几章一样，Glasserman（2004）是金融工程和应用中关于蒙特卡洛模拟的全面资源。Hilpisch（2015）也提供了最重要的蒙特卡洛算法的基于 Python 实现：

- Glasserman, Paul (2004): *Monte Carlo Methods in Financial Engineering*. Springer, New York.
- Hilpisch, Yves (2015): *Derivatives Analytics with Python*. Wiley Finance, Chichester, England. http://derivatives-analytics-with-python.com.

但是，利用蒙特卡洛模拟，以一致、无冗余的方式估算（复杂）衍生品投资组合价值方面没有什么研究。值得注意的一个例外（至少从概念上讲）是 Albanese、Gimonet 和 White（2010a）的简短论文。这几位作者组成的同一团队所撰写的白皮书更详细一些：

- Albanese, Claudio, Guillaume Gimonet, and Steve White (2010a): "Towards a Global Valuation Model." *Risk Magazine*, May issue. http://bit.ly/risk_may_2010.
- Albanese, Claudio, Guillaume Gimonet, and Steve White (2010b): "Global Valuationand Dynamic Risk Management." http://www.albanese.co.uk/Global_ Valuation_and_Dynamic_ Risk_Management.pdf.

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

第 19 章

波动率期权

我们正面对着极端的波动性。

——Carlos Ghosn

波动率衍生品已经成为重要的风险管理和交易工具。第一代期权定价金融模型只将波动率作为许多输入参数中的一个，而第二代模型和产品将*波动率视为单独的资产类型*。例如，1993 年推出的 VIX 波动率指数（参见 http://en.wikipedia.org/CBOE_Volatility_Index）从 2003 年开始被作为固定到期日为 30 日的 S&P 500 指数虚值看跌和看涨期权的加权隐含波动率度量。通常，30 天固定到期日*主指数*价值只能通过在指数较短和较长到期日的到期日价值之间插值计算——也就是，在不同到期日的两种*子指数*之间插值。

2005 年 Eurex（由德国德意志交易所运作的衍生品交易所，http://www.eurexchange.com/advanced-services/）推出的 VSTOXX 波动率指数的计算方法类似；但是，它基于 EURO STOXX 50 指数期权的隐含波动率[1]。

本章使用第 15 ~ 第 18 章开发的 DX 衍生品分析库估算 *VSTOXX 波动率指数美式看跌期权*投资组合的价值。时至今日，Eurex 只提供 VSTOXX 期货合约和欧式看涨及看跌期权。公开市场上没有任何 VSTOXX 美式期权。

这是银行买入和沽空对应交易所没有提供的指数期权的典型情况。为了简洁起见，我们假定美式看跌期权的到期日与某个交易期权序列的到期日一致。

我们采用来自 DX 库的 **square_root_diffusion** 类作为 VSTOXX 波动率指数的模型。这

1 VSTOXX 的计算方式细节和自行计算的方法——使用 Python 收集必要数据进行计算——参见基于 Python 的教程（http://www.eurexchange.com/advanced-services/vstoxx/）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

一模型满足数量（如波动率）建模的主要需求——即回归和正向性（参见第 10、第 14 和第 16 章）[1]。

本章特别实施了如下任务：

数据收集

我们需要 3 种类型数据，分别是 VSTOXX 指数本身、指数期货以及期权数据。

模型检验

为了以和市场一致的方式估算非交易期权，通常首先用期权报价检验所选择的模型，证明基于最优参数的模型尽可能复制市场价格。

投资组合估值

有了上述的所有数据和经过市场检验的 VSTOXX 波动率指数模型，最后一项任务是建立非交易权证的模型并为之估值。

19.1 VSTOXX 数据

本节逐步收集美式 VSTOXX 看跌期权估值所需的数据。首先，导入选择的数据收集及管理库：

```
In [1]: import numpy as np
        import pandas as pd
```

19.1.1 VSTOXX 指数数据

在第 6 章中，有一个基于 VSTOXX 和 EURO VSTOXX 50 指数的回归示例，我们还使用了如下的 VSTOXX 每日收盘价公开数据来源：

```
In [2]: url = 'http://www.stoxx.com/download/historical_values/h_vstoxx.txt'
        vstoxx_index = pd.read_csv(url, index_col=0, header=2,
                                   parse_dates=True, dayfirst=True)
In [3]: vstoxx_index.info()
Out[3]: <class 'pandas.core.frame.DataFrame'>
        DatetimeIndex: 4010 entries, 1999-01-04 00:00:00 to 2014-09-26 00:00:00
        Data columns (total 9 columns):
        V2TX    4010 non-null float64
        V6I1    3591 non-null float64
        V6I2    4010 non-null float64
        V6I3    3960 non-null float64
        V6I4    4010 non-null float64
        V6I5    4010 non-null float64
```

1 Gruenbichler 和 Longstaff（1996）提出的一个较早期波动率期权定价模型也基于平方根扩散。但是，他们只考虑了欧式期权，并为之提出了一个封闭形式的解决方案。该模型的研究和 Python 实现参见 http://www.eurexchange.com/advanced-services/vstoxx/。第 14 章中的 Web 服务示例也基于他们的模型和分析估值公式。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
V6I6    3995 non-null float64
V6I7    4010 non-null float64
V6I8    3999 non-null float64
dtypes: float64(9)
```

为了下面的期权分析，我们只需要 2014 年第一季度的 VSTOXX 指数数据。因此，可以删除 **DataFrame vstoxx_index** 中现在包含的更新或者更旧的数据：

```
In [4]: vstoxx_index = vstoxx_index[('2013/12/31' < vstoxx_index.index)
                                    & (vstoxx_index.index < '2014/4/1')]
```

观察这些数据可以发现，该数据集不仅包含了主指数 V2TX 的每日收盘价，还包括从 V6I1 到 V6I8 的所有子指数数据，最后一个数字代表到期日（1=最近到期日，8=最远到期日）。正如前面所指出的，主指数通常是两个子指数的插值，特别是 v6I1 和 V6I2，它们代表到期日小于 30 天的第一种情况，以及到期日在 30-60 天之间的第二种情况：

```
In [5]: np.round(vstoxx_index.tail(), 2)
Out[5]:
            V2TX   V6I1   V6I2   V6I3   V6I4   V6I5   V6I6   V6I7   V6I8
Date
2014-03-25  18.26  18.23  18.31  19.04  19.84  20.31  18.11  20.83  21.20
2014-03-26  17.59  17.48  17.70  18.45  19.42  20.00  20.26  20.45  20.86
2014-03-27  17.64  17.50  17.76  18.62  19.49  20.05  20.11  20.49  20.94
2014-03-28  17.03  16.68  17.29  18.33  19.30  19.83  20.14  20.38  20.82
2014-03-31  17.66  17.61  17.69  18.57  19.43  20.04  19.98  20.44  20.90
```

19.1.2 VSTOXX 期货数据

用于期货和期权的数据集并不是公开的，而是 Eurex 提供的以 VSTOXX 波动率指数为标的物的所有交易型金融工具每日收盘价的完整数据集。该数据集涵盖整个 2014 年第一季度的数据：

```
In [6]: vstoxx_futures = pd.read_excel('./source/vstoxx_march_2014.xlsx',
                                       'vstoxx_futures')
In [7]: vstoxx_futures.info()
Out[7]: <class 'pandas.core.frame.DataFrame'>
        Int64Index: 504 entries, 0 to 503
        Data columns (total 8 columns):
        A_DATE                      504 non-null datetime64[ns]
        A_EXP_YEAR                  504 non-null int64
        A_EXP_MONTH                 504 non-null int64
        A_CALL_PUT_FLAG             504 non-null object
        A_EXERCISE_PRICE            504 non-null int64
        A_SETTLEMENT_PRICE_SCALED   504 non-null int64
        A_PRODUCT_ID                504 non-null object
        SETTLE                      504 non-null float64
        dtypes: datetime64[ns](1), float64(1), int64(4), object(2)
```

有多列数据没有填入或者不需要，因此可以删除它们，不会丢失任何相关信息：

```
In [8]: del vstoxx_futures['A_SETTLEMENT_PRICE_SCALED']
        del vstoxx_futures['A_CALL_PUT_FLAG']
        del vstoxx_futures['A_EXERCISE_PRICE']
        del vstoxx_futures['A_PRODUCT_ID']
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

为简洁起见，我们更改其余列的名称：

```
In [9]: columns = ['DATE', 'EXP_YEAR', 'EXP_MONTH', 'PRICE']
        vstoxx_futures.columns = columns
```

按照市场的惯例，交易所交易期权的到期日为到期月的*第三个周五*。为此，创建一个助手函数 **third_friday**，给出指定年月第三个周五的日期，是很有帮助的：

```
In [10]: import datetime as dt
         import calendar

         def third_friday(date):
             day = 21 - (calendar.weekday(date.year, date.month, 1) + 2) % 7
             return dt.datetime(date.year, date.month, day)
```

对于 VSTOXX 期货和期权，在任何时候都有 8 个对应的到期日，每月的起始日不同，分别是当月的第三个周五（在该日之前）或者次月的第三个周五（该日前一天、当天和后一天）[1]。在我们的数据集中有 11 个对应到期日，从 2014 年 1 月到 2014 年 11 月：

```
In [11]: set(vstoxx_futures['EXP_MONTH'])
Out[11]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11}
```

我们计算所有月份的第三个周五的具体日期并在以后重用。注意，2014 年 4 月 18 日是德国公众假期，但是这与下面的分析无关：

```
In [12]: third_fridays = {}
         for month in set(vstoxx_futures['EXP_MONTH']):
             third_fridays[month] = third_friday(dt.datetime(2014, month, 1))
In [13]: third_fridays
Out[13]: {1: datetime.datetime(2014, 1, 17, 0, 0),
          2: datetime.datetime(2014, 2, 21, 0, 0),
          3: datetime.datetime(2014, 3, 21, 0, 0),
          4: datetime.datetime(2014, 4, 18, 0, 0),
          5: datetime.datetime(2014, 5, 16, 0, 0),
          6: datetime.datetime(2014, 6, 20, 0, 0),
          7: datetime.datetime(2014, 7, 18, 0, 0),
          8: datetime.datetime(2014, 8, 15, 0, 0),
          9: datetime.datetime(2014, 9, 19, 0, 0),
          10: datetime.datetime(2014, 10, 17, 0, 0),
          11: datetime.datetime(2014, 11, 21, 0, 0)}
```

在 **lamda** 函数中包装到期日字典对象，很容易应用 DataFrame 对象中对应的 EXP_MONTH 列。为了方便起见，我们将到期日和其他期货数据保存在一起：

```
In [14]: tf = lambda x: third_fridays[x]
         vstoxx_futures['MATURITY'] = vstoxx_futures['EXP_MONTH'].apply(tf)
In [15]: vstoxx_futures.tail()
Out[15]:           DATE  EXP_YEAR  EXP_MONTH  PRICE   MATURITY
         499 2014-03-31      2014          7  20.40 2014-07-18
         500 2014-03-31      2014          8  20.70 2014-08-15
         501 2014-03-31      2014          9  20.95 2014-09-19
         502 2014-03-31      2014         10  21.05 2014-10-17
         503 2014-03-31      2014         11  21.25 2014-11-21
```

1 VSTOXX 波动率衍生品的最后交易日是到期日的前两天。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

19.1.3 VSTOXX 期权数据

在任何时候，都有 8 种基于 VSTOXX 的期货进行交易。相比之下，期权当然要多得多，可以预期，波动率期权的数据集比期货数据集大得多。实际上，2014 年第一季度有将近 47000 个期权报价：

```
In [16]: vstoxx_options = pd.read_excel('./source/vstoxx_march_2014.xlsx',
                                        'vstoxx_options')
In [17]: vstoxx_options.info()
Out[17]: <class 'pandas.core.frame.DataFrame'>
         Int64Index: 46960 entries, 0 to 46959
         Data columns (total 8 columns):
         A_DATE                          46960 non-null datetime64[ns]
         A_EXP_YEAR                      46960 non-null int64
         A_EXP_MONTH                     46960 non-null int64
         A_CALL_PUT_FLAG                 46960 non-null object
         A_EXERCISE_PRICE                46960 non-null int64
         A_SETTLEMENT_PRICE_SCALED       46960 non-null int64
         A_PRODUCT_ID                    46960 non-null object
         SETTLE                          46960 non-null float64
         dtypes: datetime64[ns](1), float64(1), int64(4), object(2)
```

和前面一样，并非所有列都是必需的：

```
In [18]: del vstoxx_options['A_SETTLEMENT_PRICE_SCALED']
         del vstoxx_options['A_PRODUCT_ID']
```

重命名各列可以简化以后的查询：

```
In [19]: columns = ['DATE', 'EXP_YEAR', 'EXP_MONTH', 'TYPE', 'STRIKE', 'PRICE']
         vstoxx_options.columns = columns
```

我们再次使用 **tf** 函数将到期日和期权数据一同保存：

```
In [20]: vstoxx_options['MATURITY'] = vstoxx_options['EXP_MONTH'].apply(tf)
In [21]: vstoxx_options.head()
Out[21]:         DATE  EXP_YEAR  EXP_MONTH TYPE  STRIKE  PRICE   MATURITY
         0 2014-01-02      2014          1    C    1000   7.95 2014-01-17
         1 2014-01-02      2014          1    C    1500   3.05 2014-01-17
         2 2014-01-02      2014          1    C    1600   2.20 2014-01-17
         3 2014-01-02      2014          1    C    1700   1.60 2014-01-17
         4 2014-01-02      2014          1    C    1800   1.15 2014-01-17
```

期权合约以指数值的 100 倍为基准，因此，行权价也要相应进行比例调整。为了表示一个单位，我们将行权价除以 100，重新调整比例：

```
In [22]: vstoxx_options['STRIKE'] = vstoxx_options['STRIKE'] / 100.
```

现在，来自外部资源的所有数据都已经收集并准备好。如果有必要，可以保存这 3 个 DataFrame 对象供以后重用：

```
In [23]: save = False
         if save is True:
             import warnings
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
warnings.simplefilter('ignore')
h5 = pd.HDFStore('./source/vstoxx_march_2014.h5',
                 complevel=9, complib='blosc')
h5['vstoxx_index'] = vstoxx_index
h5['vstoxx_futures'] = vstoxx_futures
h5['vstoxx_options'] = vstoxx_options
h5.close()
```

19.2 模型检验

下一个重要步骤是用于按照可用市场数据估算 VSTOXX 期权价值的金融模型的检验。这个主题的深入讨论和 **Python** 示例代码参见 Hilpisch（2015），特别是第 11 章。

19.2.1 相关市场数据

检验模型的第一步是确定所使用的相关市场数据。对于这个例子，我们有如下假设：

- *定价日期*为 2014 年 3 月 31 日。
- *期权到期*月为 2014 年 8 月。

下面的 **Python** 代码定义 **pricing_date** 和 **maturity**，从对应的 DataFrame 对象读出 VSTOXX 的 **intial_value**，并读出对应到期日的 VSTOXX 期货 **forward** 值。

```
In [24]: pricing_date = dt.datetime(2014, 3, 31)
           # last trading day in March 2014
         maturity = third_fridays[10]
           # October maturity
         initial_value = vstoxx_index['V2TX'][pricing_date]
           # VSTOXX on pricing_date
         forward = vstoxx_futures[(vstoxx_futures.DATE == pricing_date)
                   & (vstoxx_futures.MATURITY == maturity)]['PRICE'].values[0]
```

在数据集中的许多期权报价里，我们只提取符合以下条件的数据：

- 从定价日期开始
- 对应的到期日
- 看涨期权溢价或者折价比例小于 20%

因此，我们使用如下代码：

```
In [25]: tol = 0.20
         option_selection = \
             vstoxx_options[(vstoxx_options.DATE == pricing_date)
                          & (vstoxx_options.MATURITY == maturity)
                          & (vstoxx_options.TYPE == 'C')
                          & (vstoxx_options.STRIKE > (1 - tol) * forward)
                          & (vstoxx_options.STRIKE < (1 + tol) * forward)]
```

上述代码保留如下期权报价，用于检验过程：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [26]: option_selection
Out[26]:              DATE  EXP_YEAR  EXP_MONTH TYPE  STRIKE  PRICE   MATURITY
         46482 2014-03-31      2014         10    C      17   4.85 2014-10-17
         46483 2014-03-31      2014         10    C      18   4.30 2014-10-17
         46484 2014-03-31      2014         10    C      19   3.80 2014-10-17
         46485 2014-03-31      2014         10    C      20   3.40 2014-10-17
         46486 2014-03-31      2014         10    C      21   3.05 2014-10-17
         46487 2014-03-31      2014         10    C      22   2.75 2014-10-17
         46488 2014-03-31      2014         10    C      23   2.50 2014-10-17
         46489 2014-03-31      2014         10    C      24   2.25 2014-10-17
         46490 2014-03-31      2014         10    C      25   2.10 2014-10-17
```

19.2.2 期权建模

为了检验 **square_root_diffusion** 模型，必须为前面选择的期权建立模型。这是 DX 分析库第一次起作用：到目前为止所做的一切“只是”下面的衍生品分析任务的准备。首先导入库：

```
In [27]: from dx import *
```

第一项任务是定义用于 VSTOXX 指数的 **market_environment** 对象，我们主要在该对象中保存前面收集和定义的数据：

```
In [28]: me_vstoxx = market_environment('me_vstoxx', pricing_date)
In [29]: me_vstoxx.add_constant('initial_value', initial_value)
         me_vstoxx.add_constant('final_date', maturity)
         me_vstoxx.add_constant('currency', 'EUR')
In [30]: me_vstoxx.add_constant('frequency', 'B')
         me_vstoxx.add_constant('paths', 10000)
In [31]: csr = constant_short_rate('csr', 0.01)
           # somewhat arbitrarily chosen here
In [32]: me_vstoxx.add_curve('discount_curve', csr)
```

检验过程的主要目标是得出 **square_root_diffusion** 模拟类的最优参数，也就是 **kappa**、**theta** 和 **volatility**。这些参数可以说是该类提供的*自由度*。其他所有参数通常是由市场或者手上的任务决定的。

尽管 3 个（最优）参数是通过数值化方法得出的，但是需要提供一些名义值，以实例化模拟类。对于 **volatility** 参数，我们采用数据集中的历史波动率：

```
In [33]: # parameters to be calibrated later
         me_vstoxx.add_constant('kappa', 1.0)
         me_vstoxx.add_constant('theta', 1.2 * initial_value)
         vol_est =  vstoxx_index['V2TX'].std() \
                     * np.sqrt(len(vstoxx_index['V2TX']) / 252.)
         me_vstoxx.add_constant('volatility', vol_est)
In [34]: vol_est
Out[34]: 1.0384283035169406
```

然后，我们为模拟类提供 **market_environment** 对象：

```
In [35]: vstoxx_model = square_root_diffusion('vstoxx_model', me_vstoxx)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

虽然 DX 库设计为完全模块化的，风险因素的建模独立（且非冗余）于估值的衍生品，但是对于 **market_environment** 对象则不一定是如此。这种对象可以用于标的风险因素和估值的期权。为了完成估值类所使用的市场环境，只需要添加 **strike**（行权价）的值和期权的 **maturity**（到期日）参数：

```
In [36]: me_vstoxx.add_constant('strike', forward)
         me_vstoxx.add_constant('maturity', maturity)
```

当然，实例化估值类还需要一个收益函数：

```
In [37]: payoff_func = 'np.maximum(maturity_value - strike, 0)'
In [38]: vstoxx_eur_call = valuation_mcs_european('vstoxx_eur_call',
                                vstoxx_model, me_vstoxx, payoff_func)
```

进行简单的健康检查，看看目前为止的模型在是否“大体上”有效：

```
In [39]: vstoxx_eur_call.present_value()
Out[39]: 0.379032
```

为了用前面选择的期权报价检验模型，我们需要建立所有相关欧式看涨期权的模型。它们本身的差别在于对应的行权价格；市场环境中的其他参数都相同。我们在一个字典对象中存储单独的估值对象，以 DataFrame 对象 **option_selection** 中期权报价的索引值作为字典对象的键值，用于唯一标识：

```
In [40]: option_models = {}
         for option in option_selection.index:
             strike = option_selection['STRIKE'].ix[option]
             me_vstoxx.add_constant('strike', strike)
             option_models[option] = \
                                 valuation_mcs_european(
                                         'eur_call_%d' % strike,
                                         vstoxx_model,
                                         me_vstoxx,
                                         payoff_func)
```

检验例程中的一个步骤需要更新所有估值对象并重新估算所有期权的价值。为了方便，我们将这些功能放入一个单独的函数：

```
In [41]: def calculate_model_values(p0):
             ''' Returns all relevant option values.

             Parameters
             ===========
             p0 : tuple/list
                 tuple of kappa, theta, volatility

             Returns
             =======
             model_values : dict
                 dictionary with model values
             '''
             kappa, theta, volatility = p0
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        vstoxx_model.update(kappa=kappa,
                            theta=theta,
                            volatility=volatility)
        model_values = {}
        for option in option_models:
            model_values[option] = \
              option_models[option].present_value(fixed_seed=True)
        return model_values
```

向函数 **calculate_model_values** 提供 **kappa**、**theta** 和 **volatility** 组成的元组，可以得到在其他条件不变的情况下，所有相关期权的模型期权价值：

```
In [42]: calculate_model_values((0.5, 27.5, vol_est))
Out[42]: {46482: 3.206401,
          46483: 2.412354,
          46484: 1.731028,
          46485: 1.178823,
          46486: 0.760421,
          46487: 0.46249,
          46488: 0.263662,
          46489: 0.142177,
          46490: 0.07219}
```

19.2.3 检验过程

期权定价模型的检验通常是一个凸优化问题。最广为使用的检验函数——即最小化函数——是给定期权报价下模型期权价值的*均方差（MSE）*。假定有 N 个相关期权、模型和市场报价，则以 MSE 为基础，用市场报价检验金融模型的问题可以用公式 19-1 表示。式中，C_n^* 和 C_n^{mod} 是第 n 个期权的市场价格和模型价格。p 是作为期权定价模型输入的参数集。

公式 19-1　根据均方差检验模型

$$\min_p \frac{1}{N}\sum_{n=1}^{N}(C_n^* - C_n^{mod}(p))^2 \qquad (19\text{-}1)$$

Python 函数 **mean_squared_error** 从技术上实现了这种模型检验方法，使用一个全局变量控制中间参数元组对象和结果 MSE 的输出：

```
In [43]: i = 0
         def mean_squared_error(p0):
             ''' Returns the mean-squared error given
             the model and market values.

             Parameters
             ==========
             p0 : tuple/list
                 tuple of kappa, theta, volatility

             Returns
             =======
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    MSE : float
        mean-squared error
    '''
    global i
    model_values = np.array(calculate_model_values(p0).values())
    market_values = option_selection['PRICE'].values
    option_diffs = model_values - market_values
    MSE = np.sum(option_diffs ** 2) / len(option_diffs)
      # vectorized MSE calculation
    if i % 20 == 0:
        if i == 0:
            print '%4s  %6s  %6s  %6s -> %6s' % \
                  ('i', 'kappa', 'theta', 'vola', 'MSE')
        print '%4d  %6.3f  %6.3f  %6.3f -> %6.3f' % \
                (i, p0[0], p0[1], p0[2], MSE)
    i += 1
    return MSE
```

同样，进行一次简短的检查，看看函数是否大体上适用：

```
In [44]: mean_squared_error((0.5, 27.5, vol_est))
Out[44]:    i   kappa   theta    vola ->    MSE
            0   0.500  27.500   1.038 ->  4.390

         4.3899900376937779
```

第 9 章介绍了用于凸优化问题的 **Python** 和 **Scipy** 函数。我们将应用它们，所以首先导入：

```
In [45]: import scipy.optimize as spo
```

下面的检验使用通过 **brute** 函数进行的*全局*优化和通过 **fmin** 函数进行的*局部*优化。首先是全局优化：

```
In [46]: %%time
         i = 0
         opt_global = spo.brute(mean_squared_error,
                                ((0.5, 3.01, 0.5),  # range for kappa
                                 (15., 30.1, 5.),  # range for theta
                                 (0.5, 5.51, 1)),  # range for volatility
                                finish=None)
Out[46]:    i   kappa   theta    vola ->    MSE
            0   0.500  15.000   0.500 -> 10.393
           20   0.500  30.000   1.500 ->  2.071
           40   1.000  25.000   3.500 ->  0.180
           60   1.500  20.000   5.500 ->  0.718
           80   2.000  20.000   1.500 ->  5.501
          100   2.500  15.000   3.500 ->  5.571
          120   2.500  30.000   5.500 -> 22.992
          140   3.000  30.000   1.500 -> 14.493
         CPU times: user 18.6 s, sys: 1.68 s, total: 20.3 s
         Wall time: 20.3 s
```

中间优化结果如下。MSE 已经相当低：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [47]: i = 0
         mean_squared_error(opt_global)
Out[47]:    i   kappa   theta    vola ->    MSE
            0   1.500  20.000   4.500 ->  0.008

         0.0076468730485555626
```

接下来，我们使用中间优化参数作为局部优化的输入：

```
In [48]: %%time
         i = 0
         opt_local = spo.fmin(mean_squared_error, opt_global,
                              xtol=0.00001, ftol=0.00001,
                              maxiter=100, maxfun=350)
Out[48]:    i   kappa   theta    vola ->    MSE
            0   1.500  20.000   4.500 ->  0.008
           20   1.510  19.235   4.776 ->  0.008
           40   1.563  18.926   4.844 ->  0.005
           60   1.555  18.957   4.828 ->  0.005
           80   1.556  18.947   4.832 ->  0.005
          100   1.556  18.948   4.831 ->  0.005
          120   1.556  18.948   4.831 ->  0.005
         Optimization terminated successfully.
                 Current function value: 0.004654
                 Iterations: 64
                 Function evaluations: 138
         CPU times: user 17.7 s, sys: 1.67 s, total: 19.3 s
         Wall time: 19.4 s
```

这次的结果如下：

```
In [49]: i = 0
         mean_squared_error(opt_local)
Out[49]:    i   kappa   theta    vola ->    MSE
            0   1.556  18.948   4.831 ->  0.005

         0.0046542736439999875
```

结果的模型价值为：

```
In [50]: calculate_model_values(opt_local)
Out[50]: {46482: 4.746597,
          46483: 4.286923,
          46484: 3.863346,
          46485: 3.474144,
          46486: 3.119211,
          46487: 2.793906,
          46488: 2.494882,
          46489: 2.224775,
          46490: 1.98111}
```

我们将这些结果保存在 DataFrame 对象 **option_selection** 中，计算与市场价格的差值：

```
In [51]: option_selection['MODEL'] = \
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
            np.array(calculate_model_values(opt_local).values())
option_selection['ERRORS'] = \
            option_selection['MODEL'] - option_selection['PRICE']
```

得到的结果如下：

```
In [52]: option_selection[['MODEL', 'PRICE', 'ERRORS']]
Out[52]:            MODEL  PRICE    ERRORS
         46482  4.746597   4.85 -0.103403
         46483  4.286923   4.30 -0.013077
         46484  3.863346   3.80  0.063346
         46485  3.474144   3.40  0.074144
         46486  3.119211   3.05  0.069211
         46487  2.793906   2.75  0.043906
         46488  2.494882   2.50 -0.005118
         46489  2.224775   2.25 -0.025225
         46490  1.981110   2.10 -0.118890
```

平均价格误差相对低，少于 1 分：

```
In [53]: round(option_selection['ERRORS'].mean(), 3)
Out[53]: -0.002
```

图 19-1 以图形方式展示了所有结果。观测到的最大差值出现在内在价值最低的看涨期权上：

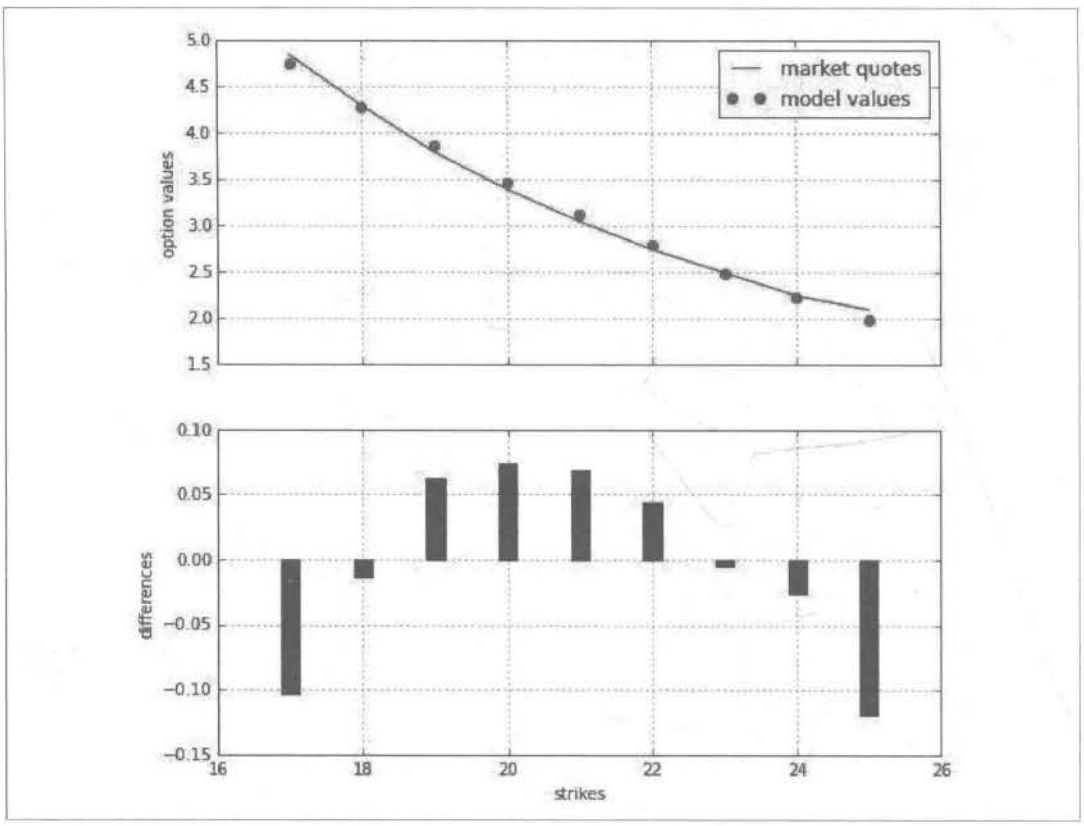


图 19-1　经过检验的 VSTOXX 看涨期权模型价值与市场报价的对比

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [54]: import matplotlib.pyplot as plt
         %matplotlib inline
         fix, (ax1, ax2) = plt.subplots(2, sharex=True, figsize=(8, 8))
         strikes = option_selection['STRIKE'].values
         ax1.plot(strikes, option_selection['PRICE'], label='market quotes')
         ax1.plot(strikes, option_selection['MODEL'], 'ro', label='model values')
         ax1.set_ylabel('option values')
         ax1.grid(True)
         ax1.legend(loc=0)
         wi = 0.25
         ax2.bar(strikes - wi / 2., option_selection['ERRORS'],
                 label='market quotes', width=wi)
         ax2.grid(True)
         ax2.set_ylabel('differences')
         ax2.set_xlabel('strikes')
```

19.3 基于 VSTOXX 的美式期权

未在交易所交易的期权估值和管理的重要先决条件之一是经过检验、尽可能与市场实际情况（即对应市场连续交易期权的报价）一致的模型。这是前一小节的主要成果。本节中使用这一主要成果估算 VSTOXX 美式看跌期权价值，这是未在市场中交易的一种衍生工具。我们假定一个投资组合，其中包含的美式看跌期权到期日及行权价与用于模型检验的欧式看涨期权相同。

19.3.1 期权头寸建模

用 DX 分析库估算衍生品投资组合价值的第一步是用 **market_environment** 对象定义相关风险因素。在这一阶段，该对象不一定必须完整；遗漏的数据和对象可以在投资组合估值时添加（例如 **paths** 或者 **frequency**）：

```
In [55]: me_vstoxx = market_environment('me_vstoxx', pricing_date)
         me_vstoxx.add_constant('initial_value', initial_value)
         me_vstoxx.add_constant('final_date', pricing_date)
         me_vstoxx.add_constant('currency', 'NONE')
```

当然，我们使用来自模型经验的最优参数：

```
In [56]: # adding optimal parameters to environment
         me_vstoxx.add_constant('kappa', opt_local[0])
         me_vstoxx.add_constant('theta', opt_local[1])
         me_vstoxx.add_constant('volatility', opt_local[2])
```

在投资组合的环境中，有必要指定模拟类/模型：

```
In [57]: me_vstoxx.add_constant('model', 'srd')
```

在美式看跌期权估值类的定义中，我们缺少的主要是一个合适的收益函数：

```
In [58]: payoff_func = 'np.maximum(strike - instrument_values, 0)'
```

和以前一样，所有美式期权的差别仅是它们的行权价。因此首先定义一个共享的 **market_environment** 对象很有意义：

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [59]: shared = market_environment('share', pricing_date)
         shared.add_constant('maturity', maturity)
         shared.add_constant('currency', 'EUR')
```

同样在所有相关期权上循环，选择相关的行权价，并用定义的 **market_environment** 对象逐个定义 **derivatives_position**：

```
In [60]: option_positions = {}
           # dictionary for option positions
         option_environments = {}
           # dictionary for option environments
         for option in option_selection.index:
             option_environments[option] = \
                 market_environment('am_put_%d' % option, pricing_date)
                 # define new option environment, one for each option
             strike = option_selection['STRIKE'].ix[option]
               # pick the relevant strike
             option_environments[option].add_constant('strike', strike)
               # add it to the environment
             option_environments[option].add_environment(shared)
               # add the shared data
             option_positions['am_put_%d' % strike] = \
                             derivatives_position(
                              'am_put_%d' % strike,
                             quantity=100.,
                             underlying='vstoxx_model',
                             mar_env=option_environments[option],
                             otype='American',
                             payoff_func=payoff_func)
```

注意，我们自始至终都使用 100 作为头寸的数量，这是典型的 VSTOXX 期权合约规模。

19.3.2 期权投资组合

要组成投资组合，我们必须指定几个参数，共同定义估值环境——也就是投资组合中所有对象共享的参数：

```
In [61]: val_env = market_environment('val_env', pricing_date)
         val_env.add_constant('starting_date', pricing_date)
         val_env.add_constant('final_date', pricing_date)
           # temporary value, is updated during valuation
         val_env.add_curve('discount_curve', csr)
         val_env.add_constant('frequency', 'B')
         val_env.add_constant('paths', 25000)
```

这个市场相当简单；它由单个风险因素组成：

```
In [62]: underlyings = {'vstoxx_model' : me_vstoxx}
```

将上述的所有对象组合起来，可以定义一个 **derivatives_portfolio** 对象：

```
In [63]: portfolio = derivatives_portfolio('portfolio', option_positions,
                                           val_env, underlyings)
```

估值需要花费不少时间，因为使用最小二乘蒙特卡洛方法估算多种美式期权的价值，

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

并且还要使用相同的高计算要求算法重新估算多个“希腊字母”：

```
In [64]: %time results = portfolio.get_statistics(fixed_seed=True)
Out[64]: CPU times: user 38.6 s, sys: 1.96 s, total: 40.6 s
         Wall time: 40.6 s
```

得到的结果 DataFrame 对象最好按照 **name** 列排序，以获得更好的统计数字对比视图：

```
In [65]: results.sort(columns='name')
```

Out[65]:

	name	quant.	value	curr.	pos_value	pos_delta	pos_vega
8	am_put_17	100	4.575197	EUR	457.5197	-24.85	102.77
1	am_put_18	100	5.203648	EUR	520.3648	-30.62	107.93
0	am_put_19	100	5.872686	EUR	587.2686	-33.31	107.79
2	am_put_20	100	6.578714	EUR	657.8714	-34.82	110.01
6	am_put_21	100	7.320523	EUR	732.0523	-39.46	105.20
7	am_put_22	100	8.081625	EUR	808.1625	-40.61	102.38
3	am_put_23	100	8.871962	EUR	887.1962	-43.26	104.37
4	am_put_24	100	9.664272	EUR	966.4272	-40.14	101.04
5	am_put_25	100	10.475168	EUR	1047.5168	-45.74	102.81

这个投资组合正如对美式看跌多头投资组合的预期，有较低（负数）的 Delta 和较高（正数）的 Vega：

```
In [66]: results[['pos_value','pos_delta','pos_vega']].sum()
Out[66]: pos_value    6664.3795
         pos_delta    -332.8100
         pos_vega      944.3000
         dtype: float64
```

19.4 结语

本章介绍了 DX 分析库更大规模、更实际的用例——估算基于 VSTOXX 波动率指数的非交易美式期权投资组合的价值。本章介绍了任何实际应用中都涉及的 3 个重要任务：

数据收集

最新、正确的市场数据是任何衍生品分析中建模与估值工作的基础；我们需要 VSTOXX 的指数数据、期货数据以及期权数据。

模型检验

为了以市场一致的方式估值、管理和对冲非交易期权，需要用期权市场报价（在到期日和行权价上相对应）检验模型参数。我们选择的模型是平方根扩散，这种方法适合于建立波动率指数的模型；检验结果相当不错，但是模型仅提供 3 种自由度（**kappa** 是均值回归因子，**theta** 是长期波动率，**volatility** 是波动率的波动率，或者称为“vol-vol”）。

投资组合估值

根据市场数据和经过检验的模型，建立了包含美式 VSTOXX 看跌期权投资组合的

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

模型，并生成重要统计数字（头寸价值，Delta 和 Vega）。

本章中的实际用例说明了 DX 库的灵活性和能力；它本质上使我们可以对付任何衍生品分析任务。基本方法和架构使这种应用大体上可以与欧式期权的 Black-Scholes-Merton 分析公式的基准用例相媲美。一旦定义了估值对象，就可以将与其类似的方式用于某个分析公式——不管表面下隐藏着怎样的事实，都需要大量应用数值化例程和算法。

19.5 延伸阅读

Eurex 的“VSTOXX 高级服务”教程页面提供了有关 VSTOXX 指数及相关波动率衍生品的丰富信息。这些页面还提供了许多易于使用的 Python 脚本，以复制教程中介绍的结果与分析：

- Eurex 的 VSTOXX 高级服务教程页面可以在 http://www.eurexchange.com/advanced-services/vstoxx/上找到，而事后检验应用在 http://www.eurexchange.com/ advanced-services/app2/上提供。

下面的书籍是本章介绍的主题很好的参考，特别是对期权定价模型的检验：

- Hilpisch, Yves (2015): *Derivatives Analytics with Python*. Wiley Finance, Chichester, England. http://derivatives-analytics-with-python.com.

对衍生品投资组合的一致性估值和管理，也可以参见第 18 章结尾处的提示。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

附录 A

精选的最佳实践

*最佳实践*通常指的是一些正式写下或者仅在日常生活中实践的规则，它们能够区分专业的 **Python** 开发者和偶尔使用 **Python** 的用户。这方面的最佳实践很多，本附录介绍较为重要的几种。

A.1 Python 语法

作为集成开发环境，Spyder（http://code.google.com/p/spyderlib/）真正有益的特性之一是自动语法和代码检查，这一功能检查 Python 代码与 Python 语法 PEP8 建议（http://www.python.org/dev/peps/pep-0008）的兼容性。但是“Python 增强建议 8”中规定了什么呢？原则上，有些代码格式规则能够确立公用标准，并使代码的可读性更好。在这个意义上，这种方法和书写及印刷自然语言时应用的语法规则没有太大的不同。

例如，考虑第 1 章中通过蒙特卡洛模拟方法估算欧洲看涨期权价值的例 1-1。首先，在例 A-1 中可以看到不遵循 PEP 8 的版本。它显得相当拥挤，因为没有空白行和空格（有时候又有太多的空格或者空行）。

例 A-1　不遵循 PEP 8 的 Python 脚本

```
#   Monte Carlo valuation of European call option
# in Black-Scholes-Merton model
#  bsm_mcs_euro_syntax_false.py
import numpy as np
#Parameter Values
S0=100.#initial index level
K=105.#strike price
T= 1.0#time-to-maturity
r=0.05#riskless short rate
sigma     =0.2#volatility
I=100000  # number of simulations
# Valuation Algorithm
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
z=np.random.standard_normal(I)#pseudorandom numbers
ST=S0*np.exp((r-   0.5*sigma**  2)+sigma*sqrt(T)*   z)#index values at maturity


hT=np.maximum(ST-K,0)#inner values at maturity
C0=np.exp(-r*T)*sum(hT)/I# Monte Carlo estimator
# Result Output
print"Value of the European Call Option %5.3f"%C0
```

现在，再看看例 A-2 中遵循 PEP 8 的版本（也就是例 1-1 中所看到的）。可读性的主要差异来源于两个事实：

- 使用空行表示代码块；
- 在 Python 运算符（如=或者*）周围和表示注释的“#”之前使用空格（这里使用两个空格）。

例 A-2　遵循 PEP 8 的 Python 脚本

```
#
# Monte Carlo valuation of European call option
# in Black-Scholes-Merton model
# bsm_mcs_euro_syntax_correct.py
#

import numpy as np

# Parameter Values
S0 = 100.  # initial index level
K = 105.  # strike price
T = 1.0  # time-to-maturity
r = 0.05  # riskless short rate
sigma = 0.2  # volatility

I = 100000  # number of simulations
# Valuation Algorithm
z = np.random.standard_normal(I)  # pseudorandom numbers
ST = S0 * np.exp((r - 0.5 * sigma ** 2) * T + sigma * np.sqrt(T) * z)
  # index values at maturity
hT = np.maximum(ST - K, 0)  # inner values at maturity
C0 = np.exp(-r * T) * np.sum(hT) / I  # Monte Carlo estimator

# Result Output
print "Value of the European Call Option %5.3f" % C0
```

虽然第一个版本完全可以由 Python 解释程序执行，但是第二个版本对程序员和任何其他试图理解它的人来说无疑都更易读。

函数和类使用某些特殊的规则。一般来说，在任何新函数（方法）定义和新的类定义之前应该有两个空行。对于函数，还应该使用缩进。通常，缩进通过空格而非制表符实现。作为一般原则，每级缩进使用 4 个空格[1]。

1　大部分（Python）编辑器允许我们配置按下 Tab 键时产生的空格数量。有些编辑器还允许半自动地用空格替换制表符。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

例 A-3　有多重缩进的 Python 函数

```
#
# Function to check prime characteristic of integer
# is_prime_no_doc.py
#


def is_prime(I):
    if type(I) != int:
        raise TypeError("Input has not the right type.")
    if I <= 3:
        raise ValueError("Number too small.")
    else:
        if I % 2 == 0:
            print "Number is even, therefore not prime."
        else:
            end = int(I / 2.) + 1
            for i in range(3, end, 2):
                if I % i == 0:
                    print "Number is not prime, it is divided by %d." % i
                    break
                if i >= end - 2:
                    print "Number is prime."
```

我们立刻注意到缩进在 Python 中所起的作用。多级缩进表示代码块，这里主要是由控制结构元素（例如 **if** 或者 **else**）或者循环（例如 **for** 循环）造成的。

控制结构元素在第 4 章中做了解释，但是即使你还不习惯 Python 语法，函数的基本工作原理也应该很清楚。表 A-1 列出了一些经常使用的 Python 运算符。表中的描述列中有问号时，该运算返回一个布尔对象（True 或者 False）。

表 A-1　Python 运算符（节选）

符号	描述
+	加法
-	减法
/	除法
*	乘法
%	取模
==	相等?
!=	不相等?
<	小于?
<=	小于或等于?
>	大于?
>=	大于或等于?

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

A.2 文档

Python 文档的两个主要元素是：

内嵌文档

内嵌文档原则上可以放在代码中的任何位置；由一个或者多个先导的#符号表示。通常，在#之前应该有至少两个空格。

文档字符串

这些字符串用于为 Python 函数（方法）和类提供文档，通常放在它们的定义中（在缩进代码的开始处）

例 A-2 中的代码包含了多个内嵌文档的示例。例 A-4 和例 A-3 一样展示了多个函数定义，但是这次添加了文档字符串。

例 A-4　包含文档字符串的 Python 函数 is_prime

```
#
# Function to check prime characteristic of integer
# is_prime_with_doc.py
#


def is_prime(I):
    "' Function to test for prime characteristic of an integer.

    Parameters
    ==========
    I : int
        number to be checked for prime characteristc

    Returns
    =======
    output: string
        states whether number is prime or not;
        if not, provide a prime factor

    Raises
    ======
    TypeError
        if argument is not an integer
    ValueError
        if the integer is too small (2 or smaller)

    Examples
    ========
    >>> is_prime(11)
    Number is prime.
    >>> is_prime(8)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    Number is even, therefore not prime.
    >>> is_prime(int(1e8 + 7))
    Number is prime.
    >>>
    '''
    if type(I) != int:
        raise TypeError("Input has not the right type.")
    if I <= 3:
        raise ValueError("Number too small.")
    else:
        if I % 2 == 0:
            print "Number is even, therefore not prime."
        else:
            end = int(I / 2.) + 1
            for i in range(3, end, 2):
                if I % i == 0:
                    print "Number is not prime, it is divided by %d." % i
                    break
                if i >= end - 2:
                    print "Number is prime."
```

通常，这些字符串提供有关如下元素的信息：

输入

提供的形参/实参及其格式（例如，**int**）

输出

函数/方法的返回值及其格式

错误

可能发生哪些（“特殊的”）错误

示例

函数/方法的使用示例

文档字符串的使用不仅对查看代码的人有帮助。大部分 Python 工具（如 IPython 和 Spyder）都可以直接访问这些文档和帮助来源。图 A-1 展示了 Spyder 的一个截图，其中在编辑器里显示 **is_prime** 函数，并在对象检查器（右上）中显示文档字符串。这说明了在函数和类中式中包含有意义文档字符串的益处。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

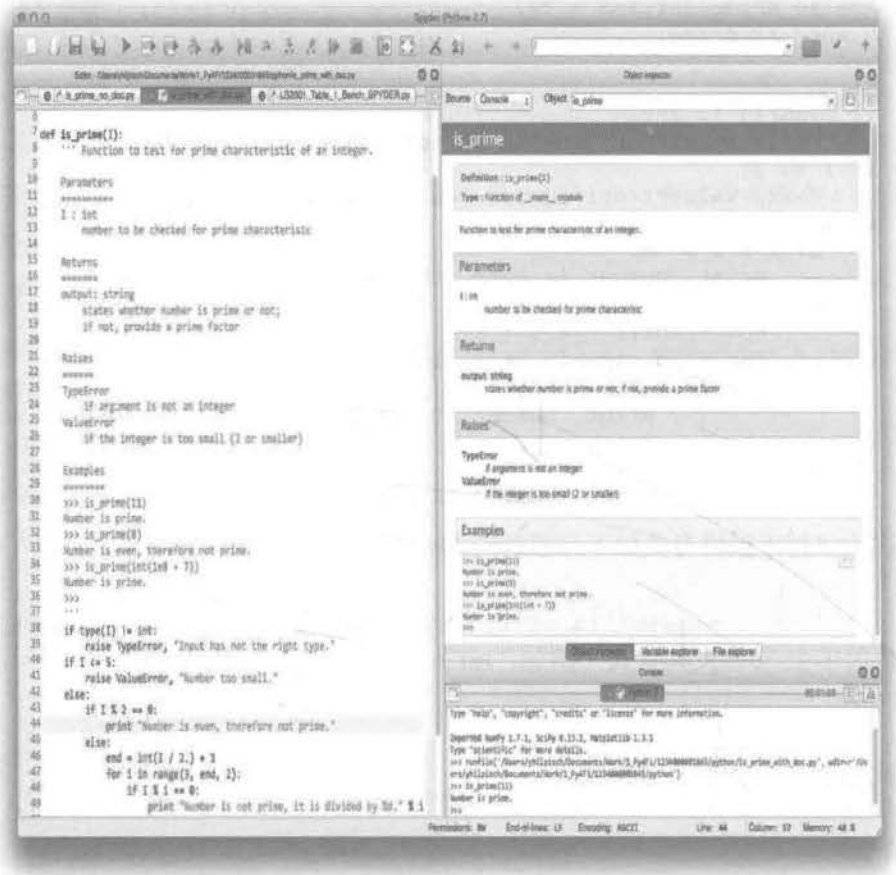


图 A-1　Spyder 的截图，显示了一个自定义函数和文档字符串

A.3　单元测试

我们所考虑的最后一个最佳实践是单元测试。在不同测试方法中，单元测试之所以被视为最佳实践，是因为它在相当基础的级别（一个单元）上测试 Python 代码。但是，它不测试各个单元的集成。一般来说，单元指的是函数、类或者类的方法。例 A-5 是一个简单的 Python 函数示例，它的测试很简单。

例 A-5　一个相当简单的 Python 函数

```
#
# Simple function to calculate
# the square of the square root
# of a positive number
# simple_function.py
#
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
from math import sqrt


def f(x):
    ''' Function to calculate the square of the square root.

    Parameters
    ==========
    x : float or int
        input number

    Returns
    =======
    fx : float
        square of the square root, i.e. sqrt(x) ** 2

    Raises
    ======
    TypeError
        if argument is neither float nor integer
    ValueError
        if argument is negative

    Examples
    ========
    >>> f(1)
    1
    >>> f(10.5)
    10.5
    '''
    if type(x) != float and type(x) != int:
        raise TypeError("Input has not the right type.")
    if x < 0:
        raise ValueError("Number negative.")
    fx = sqrt(x) ** 2
    return fx
```

有许多支持单元测试的工具，我们在下面将使用 **nose**（http://nose.readthedocs.org/en/latest/）。例 A-6 包含用于例 A-5 中函数 f 的测试套件。

例 A-6　函数 f 的测试套件

```
#
# Test suite for simple function f
# nose_test.py
#
import nose.tools as nt
from simple_function import f


def test_f_calculation():
    ''' Tests if function f calculates correctly. '''
    nt.assert_equal(f(4.), 4.)
    nt.assert_equal(f(1000), 1000)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
    nt.assert_equal(f(5000.5), 5000.5)


def test_f_type_error():
    "' Tests if type error is raised. "'
    nt.assert_raises(TypeError, f, 'test string')
    nt.assert_raises(TypeError, f, [3, 'string'])


def test_f_value_error():
    "' Tests if value error is raised. "'
    nt.assert_raises(ValueError, f, -1)
    nt.assert_raises(ValueError, f, -2.5)


def test_f_test_fails():
    "' Tests if function test fails. "'
    nt.assert_equal(f(5.), 10)
```

表 A-2 描述了所实现的测试函数。

表 A-2 简单函数 f 的测试函数

函数	描述
test_f_calculation	测试函数是否返回正确的结果
test_f_type_error	检查函数是否按照预期发起类型错误
test_f_value_error	检查函数是否按照预期发起值错误
test_f_test_fails	测试计算测试是否和预期一样失败（为了说明）

从命令行/shell，可以运行如下测试：

```
$ nosetests nose_test.py
..F
======================================================================
FAIL: Test if function test fails.
----------------------
Traceback (most recent call last):
   File "/Library/anaconda/lib/python2.7/site-packages/nose/case.py",
   line 197, in runTest self.test(*self.arg)
   File "//Users/yhilpisch/Documents/Work/Python4Finance/python/nose_test.py",
   line 30, in test_f_test_fails
     nt.assert_equal(f(5.), 10)
AssertionError: 5.000000000000001 != 10

----------------------
Ran 4 tests in 0.002s

FAILED (failures=1)
$
```

显然，前三个测试成功，而最后一个正如预期那样失败。使用这种工具——更重要的是实现严格的单元测试方法——可能需要预先做更多的工作，但是你和同事们将长期获益。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

附录 B

看涨期权类

例 B-1 包含一个 Black-Scholes-Merton（1973）模型（参见第 3 章，特别是例 3-1）下的欧洲看涨期权类定义。

例 B-1　Black-Scholes-Merton 看涨期权类实现

```
#
# Valuation of European call options in Black-Scholes-Merton Model
# incl. Vega function and implied volatility estimation
# - class-based implementation
# bsm_option_class.py
#

from math import log, sqrt, exp
from scipy import stats

class call_option(object):
    ''' Class for European call options in BSM model.

    Attributes
    ==========
    S0 : float
        initial stock/index level
    K : float
        strike price
    T : float
        maturity (in year fractions)
    r : float
        constant risk-free short rate
    sigma : float
        volatility factor in diffusion term

    Methods
    =======
    value : float
        return present value of call option
    vega : float
        return Vega of call option
    imp_vol: float
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
        return implied volatility given option quote
    '''

    def __init__(self, S0, K, T, r, sigma):
        self.S0 = float(S0)
        self.K = K
        self.T = T
        self.r = r
        self.sigma = sigma

    def value(self):
        ''' Returns option value. '''
        d1 = ((log(self.S0 / self.K)
            + (self.r + 0.5 * self.sigma ** 2) * self.T)
            / (self.sigma * sqrt(self.T)))
        d2 = ((log(self.S0 / self.K)
            + (self.r - 0.5 * self.sigma ** 2) * self.T)
            / (self.sigma * sqrt(self.T)))
        value = (self.S0 * stats.norm.cdf(d1, 0.0, 1.0)
            - self.K * exp(-self.r * self.T) * stats.norm.cdf(d2, 0.0, 1.0))
        return value

    def vega(self):
        ''' Returns Vega of option. '''
        d1 = ((log(self.S0 / self.K)
            + (self.r + 0.5 * self.sigma ** 2) * self.T)
            / (self.sigma * sqrt(self.T)))
        vega = self.S0 * stats.norm.cdf(d1, 0.0, 1.0) * sqrt(self.T)
        return vega

    def imp_vol(self, C0, sigma_est=0.2, it=100):
        ''' Returns implied volatility given option price. '''
        option = call_option(self.S0, self.K, self.T, self.r, sigma_est)
        for i in range(it):
            option.sigma -= (option.value() - C0) / option.vega()
        return option.sigma
```

这个类可以用于如下的交互式 IPython 会话：

```
In [1]: from bsm_option_class import call_option
In [2]: o = call_option(100., 105., 1.0, 0.05, 0.2)
        type(o)
Out[2]: bsm_option_class.call_option
In [3]: value = o.value()
        value
Out[3]: 8.0213522351431763
In [4]: o.vega()
Out[4]: 54.222833358480528
In [5]: o.imp_vol(C0=value)
Out[5]: 0.20000000000000001
```

这个期权类很容易用于可视化，例如，不同行权价和到期日的期权价值及 Vega 值，这最终是拥有该公式的主要好处之一。下面的 Python 代码生成不同到期日—行权价组合

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

的期权统计数字：

```
In [6]: import numpy as np
        maturities = np.linspace(0.05, 2.0, 20)
        strikes = np.linspace(80, 120, 20)
        T, K = np.meshgrid(strikes, maturities)
        C = np.zeros_like(K)
        V = np.zeros_like(C)
        for t in enumerate(maturities):
            for k in enumerate(strikes):
                o.T = t[1]
                o.K = k[1]
                C[t[0], k[0]] = o.value()
                V[t[0], k[0]] = o.vega()
```

首先，我们来观察期权的价值。为了绘制图表，需要导入一些库和函数：

```
In [7]: import matplotlib.pyplot as plt
        from mpl_toolkits.mplot3d import Axes3D
        from pylab import cm
        %matplotlib inline
```

下面代码的输出如图 B-1 所示：

```
In [8]: fig = plt.figure(figsize=(12, 7))
        ax = fig.gca(projection='3d')
        surf = ax.plot_surface(T, K, C, rstride=1, cstride=1,
                               cmap=cm.coolwarm, linewidth=0.5, antialiased=True)
        ax.set_xlabel('strike')
        ax.set_ylabel('maturity')
        ax.set_zlabel('European call option value')
        fig.colorbar(surf, shrink=0.5, aspect=5)
```

其次，得出看涨期权的 Vega 值，如图 B-2 所示：

```
In [9]: fig = plt.figure(figsize=(12, 7))
        ax = fig.gca(projection='3d')
        surf = ax.plot_surface(T, K, V, rstride=1, cstride=1,
                               cmap=cm.coolwarm, linewidth=0.5, antialiased=True)
        ax.set_xlabel('strike')
        ax.set_ylabel('maturity')
        ax.set_zlabel('Vega of European call option')
        fig.colorbar(surf, shrink=0.5, aspect=5)
        plt.show()
```

和第 3 章中的例 3-1 比较，例 B-1 的代码表现出一些优势：

- 更好的整体代码结构和可读性；
- 尽可能避免冗余定义；
- 更好的可重用性和更紧凑的方法调用。

这个期权类还能很好地帮助期权统计数字的可视化。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

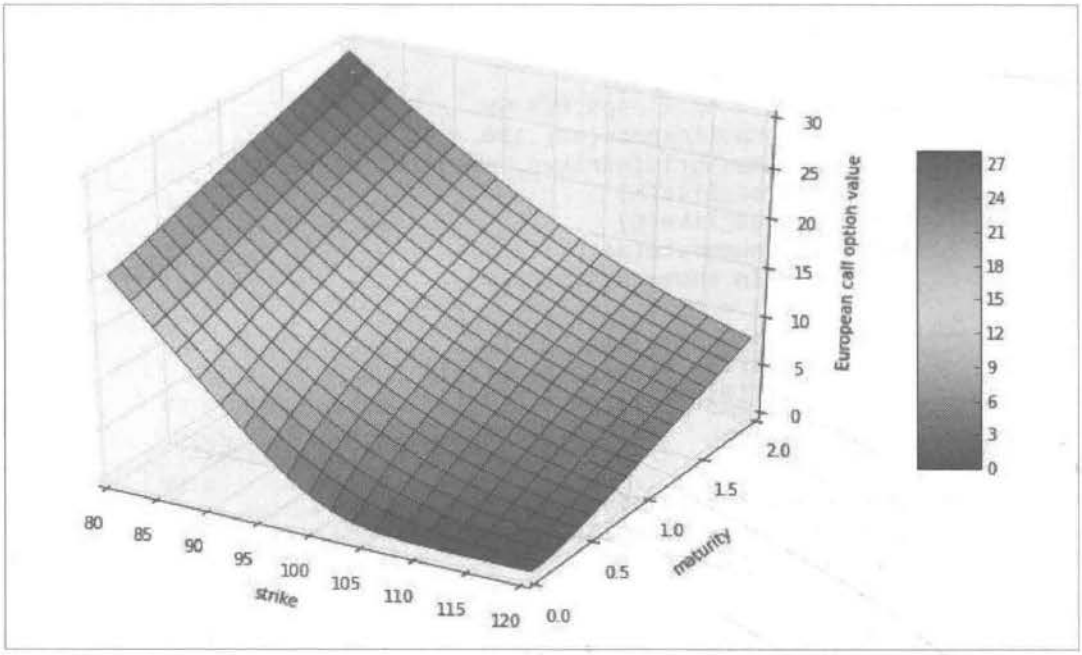


图 B-1　欧式看涨期权价值

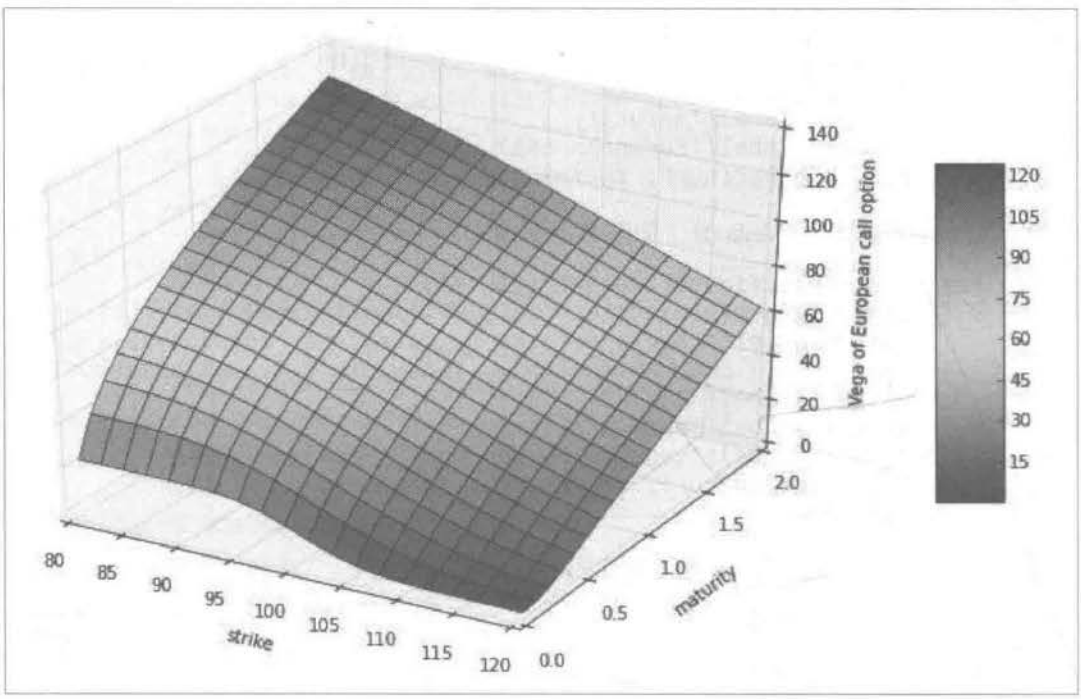


图 B-2　欧式看涨期权的 Vega 值

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

附录 C

日期和时间

正如大部分科学学科一样，日期和时间在金融学中起着重要的作用。本附录介绍 Python 编程中这一主题的不同方面。当然，本附录并不全面。不过，它可以作为支持日期和时间信息建模的 Python 生态系统主要领域的入门介绍。

C.1 Python

Python 标准库中的 **datetime** 模块实现了最重要的日期和时间相关任务[1]。我们首先导入这个模块：

```
In [1]: import datetime as dt
```

两个不同的函数提供当前日期和时间：

```
In [2]: dt.datetime.now()
Out[2]: datetime.datetime(2014, 9, 14, 19, 22, 24, 366619)
In [3]: to = dt.datetime.today()
        to
Out[3]: datetime.datetime(2014, 9, 14, 19, 22, 24, 491234)
```

结果是一个 **datetime** 对象：

```
In [4]: type(to)
Out[4]: datetime.datetime
```

weekday 方法提供指定 **datetime** 对象所代表的周日：

```
In [5]: dt.datetime.today().weekday()
          # zero-based numbering; 0 = Monday
Out[5]: 6
```

1 关于这个模块的更多信息参见在线文档（https://docs.python.org/2/library/datetime.html）。——原注

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

当然，**datetime** 对象也可以直接构造：

```
In [6]: d = dt.datetime(2016, 10, 31, 10, 5, 30, 500000)
        d
Out[6]: datetime.datetime(2016, 10, 31, 10, 5, 30, 500000)
In [7]: print d
Out[7]: 2016-10-31 10:05:30.500000
In [8]: str(d)
Out[8]: '2016-10-31 10:05:30.500000'
```

从这种对象，很容易提取其中的成分，例如年、月、日等信息：

```
In [9]: d.year
Out[9]: 2016
In [10]: d.month
Out[10]: 10
In [11]: d.day
Out[11]: 31
In [12]: d.hour
Out[12]: 10
```

可以用 **toordinal** 方法将日期信息转换为序数表示：

```
In [13]: o = d.toordinal()
         o
Out[13]: 736268
```

这也是一种工作方式，但是在此过程丢失了时间信息：

```
In [14]: dt.datetime.fromordinal(o)
Out[14]: datetime.datetime(2016, 10, 31, 0, 0)
```

另一方面，可以从 **datetime** 对象提取时间信息，形式是一个 **time** 对象：

```
In [15]: t = dt.datetime.time(d)
         t
Out[15]: datetime.time(10, 5, 30, 500000)
In [16]: type(t)
Out[16]: datetime.time
```

类似地，你可以仅提取日期信息，得到一个 **date** 对象：

```
In [17]: dd = dt.datetime.date(d)
         dd
Out[17]: datetime.date(2016, 10, 31)
```

某种精确度往往就足够了，为此，可以简单地用字面量代替 **datetime** 对象的某些属性：

```
In [18]: d.replace(second=0, microsecond=0)
Out[18]: datetime.datetime(2016, 10, 31, 10, 5)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

timedelta 是另一类对象，从其他日期-时间相关对象的数学运算得出：

```
In [19]: td = d - dt.datetime.now()
         td
Out[19]: datetime.timedelta(777, 52983, 863732)
In [20]: type(td)
Out[20]: datetime.timedelta
```

同样，可以直接访问属性提取详细信息：

```
In [21]: td.days
Out[21]: 777
In [22]: td.seconds
Out[22]: 52983
In [23]: td.microseconds
Out[23]: 863732
In [24]: td.total_seconds()
Out[24]: 67185783.863732
```

将 **datetime** 对象转换为不同表示以及从其他对象（如字符串对象）生成 **datetime** 对象都有多种方式。详情参见 **datetime** 模块的文档，下面是几个例子：

```
In [25]: d.isoformat()
Out[25]: '2016-10-31T10:05:30.500000'
In [26]: d.strftime("%A, %d. %B %Y %I:%M%p")
Out[26]: 'Monday, 31. October 2016 10:05AM'
In [27]: dt.datetime.strptime('2017-03-31', '%Y-%m-%d')
           # year first and four-digit year
Out[27]: datetime.datetime(2017, 3, 31, 0, 0)
In [28]: dt.datetime.strptime('30-4-16', '%d-%m-%y')
           # day first and two-digit year
Out[28]: datetime.datetime(2016, 4, 30, 0, 0)
In [29]: ds = str(d)
         ds
Out[29]: '2016-10-31 10:05:30.500000'
In [30]: dt.datetime.strptime(ds, '%Y-%m-%d %H:%M:%S.%f')
Out[30]: datetime.datetime(2016, 10, 31, 10, 5, 30, 500000)
```

除了 **now** 和 **today** 函数之外，还有 **utcnow** 函数，该函数给出 UTC（世界调整时间，过去称作格林威治标准时间 GMT）形式的日期和时间准确信息。这一时间和作者所在时区（CET，中欧标准时间）相差 2 个小时：

```
In [31]: dt.datetime.now()
Out[31]: datetime.datetime(2014, 9, 14, 19, 22, 28, 123943)
In [32]: dt.datetime.utcnow()
           #  Coordinated Universal Time
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[32]: datetime.datetime(2014, 9, 14, 17, 22, 28, 240319)
In [33]: dt.datetime.now() - dt.datetime.utcnow()
           # UTC + 2h = CET (summer)
Out[33]: datetime.timedelta(0, 7199, 999982)
```

datetime 模块的另一个类是 **tzinfo** 类，这是一个泛型时区类，包含 **utcoffset**、**dst** 和 **tzname** 方法。**dst** 是夏令时（Daylight Savint Time，DST）的缩写。UTC 时间的定义采用如下形式：

```
In [34]: class UTC(dt.tzinfo):
             def utcoffset(self, d):
                 return dt.timedelta(hours=0)
             def dst(self, d):
                 return dt.timedelta(hours=0)
             def tzname(self, d):
                 return "UTC"
```

UTC 可以作为 **datetime** 对象的一个属性，通过 replace 方法定义：

```
In [35]: u = dt.datetime.utcnow()
         u = u.replace(tzinfo=UTC())
           # attach time zone information
         u
Out[35]: datetime.datetime(2014, 9, 14, 17, 22, 28, 597383, tzinfo=<__main__.UTC
          object at 0x7f59e496ec10>)
```

类似地，在夏季 CET 有如下定义：

```
In [36]: class CET(dt.tzinfo):
             def utcoffset(self, d):
                 return dt.timedelta(hours=2)
             def dst(self, d):
                 return dt.timedelta(hours=1)
             def tzname(self, d):
                 return "CET + 1"
```

使用 **astimezone** 方法，可以简单地将基于 UTC 的 datetime 对象 u 转换为基于 CET 的对象：

```
In [37]: u.astimezone(CET())
Out[37]: datetime.datetime(2014, 9, 14, 19, 22, 28, 597383, tzinfo=<__main__.CET
          object at 0x7f59e79d8f10>)
```

Python 模块 pytz（http://pytz.sourceforge.net）实现全世界最重要的时区：

```
In [38]: import pytz
```

country_names 和 **country_timezones** 是包含国家和覆盖时区的字典对象：

```
In [39]: pytz.country_names['US']
Out[39]: u'United States'
In [40]: pytz.country_timezones['BE']
Out[40]: [u'Europe/Brussels']
In [41]: pytz.common_timezones[-10:]
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
Out[41]: ['Pacific/Wake',
          'Pacific/Wallis',
          'US/Alaska',
          'US/Arizona',
          'US/Central',
          'US/Eastern',
          'US/Hawaii',
          'US/Mountain',
          'US/Pacific',
          'UTC']
```

有了 **pytz**，通常没有必要定义自己的 **tzinfo** 对象：

```
In [42]: u = dt.datetime.utcnow()
         u = u.replace(tzinfo=pytz.utc)
         u
Out[42]: datetime.datetime(2014, 9, 14, 17, 22, 29, 503702, tzinfo=<UTC>)
In [43]: u.astimezone(pytz.timezone("CET"))
Out[43]: datetime.datetime(2014, 9, 14, 19, 22, 29, 503702, tzinfo=<DstTzInfo 'C
         ET' CEST+2:00:00 DST>)
In [44]: u.astimezone(pytz.timezone("GMT"))
Out[44]: datetime.datetime(2014, 9, 14, 17, 22, 29, 503702, tzinfo=<StaticTzInfo
          'GMT'>)
In [45]: u.astimezone(pytz.timezone("US/Central"))
Out[45]: datetime.datetime(2014, 9, 14, 12, 22, 29, 503702, tzinfo=<DstTzInfo 'U
         S/Central' CDT-1 day, 19:00:00 DST>)
```

C.2 NumPy

从 NumPy 1.7 起，已经有了原生的日期-时间信息支持（http://docs.scipy.org/doc/numpy/reference/arrays.datetime.html）。基本类称为 **datetime64**：

```
In [46]: import numpy as np
In [47]: nd = np.datetime64('2015-10-31')
         nd
Out[47]: numpy.datetime64('2015-10-31')
```

和 **datetime** 对象一样，**datetime64** 对象可以表示为字符串对象：

```
In [48]: np.datetime_as_string(nd)
Out[48]: '2015-10-31'
```

每个这类对象都保存在元数据，可以通过 **date_tmie_data** 方法访问。两个主要的组成部分是频率信息（例如，D 表示天）和单位（例如，我们的例子中 1 表示一天）：

```
In [49]: np.datetime_data(nd)
Out[49]: ('D', 1)
```

datetime64 对象很容易从 **datetime** 对象中构造：

```
In [50]: d
Out[50]: datetime.datetime(2016, 10, 31, 10, 5, 30, 500000)
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
In [51]: nd = np.datetime64(d)
         nd
Out[51]: numpy.datetime64('2016-10-31T11:05:30.500000+0100')
```

类似地，使用 **astype** 方法，可以将 **datetime64** 对象转换为 **datetime** 对象：

```
In [52]: nd.astype(dt.datetime)
Out[52]: datetime.datetime(2016, 10, 31, 10, 5, 30, 500000)
```

构造 **datetime64** 对象的另一种方法是提供一个字符串对象（例如包含年和月的字符串）和频率信息。注意，在下面的例子中，对象值默认为当月的第一天：

```
In [53]: nd = np.datetime64('2015-10', 'D')
         nd
Out[53]: numpy.datetime64('2015-10-01')
```

比较两个 **datetime64** 对象，在给出的信息相同时返回 True 值，即使详细程度不同：

```
In [54]: np.datetime64('2015-10') == np.datetime64('2015-10-01')
Out[54]: True
```

当然，也可以定义包含多个 **datetime64** 对象的 **ndarray** 对象：

```
In [55]: np.array(['2016-06-10', '2016-07-10', '2016-08-10'], dtype='datetime64')
Out[55]: array(['2016-06-10', '2016-07-10', '2016-08-10'], dtype='datetime64[D]')
In [56]: np.array(['2016-06-10T12:00:00', '2016-07-10T12:00:00',
                   '2016-08-10T12:00:00'], dtype='datetime64[s]')
Out[56]: array(['2016-06-10T12:00:00+0200', '2016-07-10T12:00:00+0200',
                '2016-08-10T12:00:00+0200'], dtype='datetime64[s]')
```

也可以使用 arange 函数生成一个日期范围。很容易使用不同的频率（例如，天、月或者周）：

```
In [57]: np.arange('2016-01-01', '2016-01-04', dtype='datetime64')
           # daily frequency as default in this case
Out[57]: array(['2016-01-01', '2016-01-02', '2016-01-03'], dtype='datetime64[D]')
In [58]: np.arange('2016-01-01', '2016-10-01', dtype='datetime64[M]')
           # monthly frequency
Out[58]: array(['2016-01', '2016-02', '2016-03', '2016-04', '2016-05', '2016-06',
                '2016-07', '2016-08', '2016-09'], dtype='datetime64[M]')
In [59]: np.arange('2016-01-01', '2016-10-01', dtype='datetime64[W]')[:10]
           # weekly frequency
Out[59]: array(['2015-12-31', '2016-01-07', '2016-01-14', '2016-01-21',
                '2016-01-28', '2016-02-04', '2016-02-11', '2016-02-18',
                '2016-02-25', '2016-03-03'], dtype='datetime64[W]')
```

还可以使用小于一天的频率，如小时或者秒（全部选项参见文档）：

```
In [60]: dtl = np.arange('2016-01-01T00:00:00', '2016-01-02T00:00:00',
                         dtype='datetime64[h]')
           # hourly frequency
         dtl[:10]
Out[60]: array(['2016-01-01T00+0100', '2016-01-01T01+0100', '2016-01-01T02+0100',
                '2016-01-01T03+0100', '2016-01-01T04+0100', '2016-01-01T05+0100',
                '2016-01-01T06+0100', '2016-01-01T07+0100', '2016-01-01T08+0100',
                '2016-01-01T09+0100'], dtype='datetime64[h]')
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

绘制日期-时间和时间序列数据的图表有时候很困难。**matplotlib** 对标准的 **datetime** 对象有很好的支持。将 **datetime64** 信息转换为 **datetime** 信息通常可以取得成功，正如下面的例子，其结果如图 C-1 所示：

```
In [61]: import matplotlib.pyplot as plt
         %matplotlib inline

In [62]: np.random.seed(3000)
         rnd = np.random.standard_normal(len(dtl)).cumsum() ** 2

In [63]: fig = plt.figure()
         plt.plot(dtl.astype(dt.datetime), rnd)
           # convert np.datetime to datetime.datetime
         plt.grid(True)
         fig.autofmt_xdate()
           # autoformatting of datetime x-ticks
```

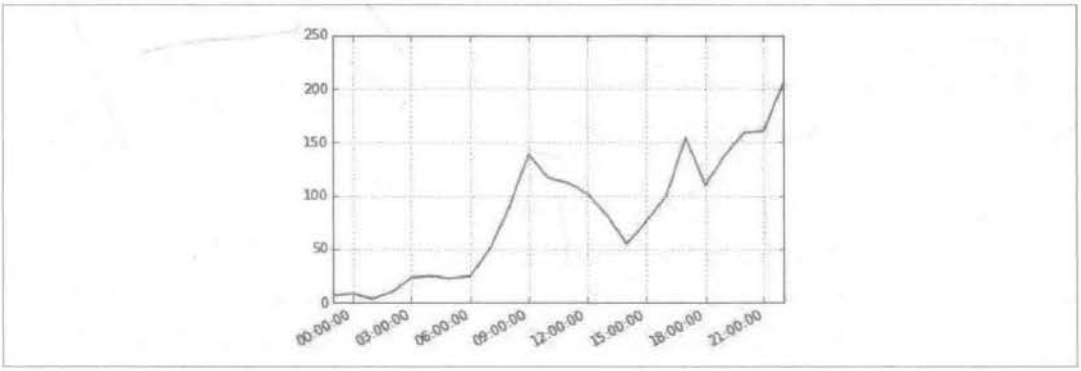


图 C-1　图表上 datetime.datetime 表示的 x 轴刻度自动格式化

最后，我们举例说明以秒和毫秒为频率的 **arange** 使用方法：

```
In [64]: np.arange('2016-01-01T00:00:00', '2016-01-02T00:00:00',
                   dtype='datetime64[s]')[:10]
           # seconds as frequency

Out[64]: array(['2016-01-01T00:00:00+0100', '2016-01-01T00:00:01+0100',
                '2016-01-01T00:00:02+0100', '2016-01-01T00:00:03+0100',
                '2016-01-01T00:00:04+0100', '2016-01-01T00:00:05+0100',
                '2016-01-01T00:00:06+0100', '2016-01-01T00:00:07+0100',
                '2016-01-01T00:00:08+0100', '2016-01-01T00:00:09+0100'], dtype='
         datetime64[s]')

In [65]: np.arange('2016-01-01T00:00:00', '2016-01-02T00:00:00',
                   dtype='datetime64[ms]')[:10]
           # milliseconds as frequency

Out[65]: array(['2016-01-01T00:00:00.000+0100', '2016-01-01T00:00:00.001+0100',
                '2016-01-01T00:00:00.002+0100', '2016-01-01T00:00:00.003+0100',
                '2016-01-01T00:00:00.004+0100', '2016-01-01T00:00:00.005+0100',
                '2016-01-01T00:00:00.006+0100', '2016-01-01T00:00:00.007+0100',
                '2016-01-01T00:00:00.008+0100', '2016-01-01T00:00:00.009+0100'],
         dtype='datetime64[ms]')
```

C.3　pandas

pandas 库在设计中专门考虑了时间序列数据。因此，该库提供了可以高效处理日期-

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

时间信息的类，如用于时间索引的 DatetimeIndex 类（参见文档（http://pandas.pydata.org/pandas-docs/stable/timeseries.html））：

```
In [66]: import pandas as pd
```

在 pandas 中，日期-时间信息通常存储在 Timestamp 对象中：

```
In [67]: ts = pd.Timestamp('2016-06-30')
         ts
Out[67]: Timestamp('2016-06-30 00:00:00')
```

这些对象很容易用 **to_datetime** 方法转换为常规的 **datetime** 对象：

```
In [68]: d = ts.to_datetime()
         d
Out[68]: datetime.datetime(2016, 6, 30, 0, 0)
```

类似地，**Timestamp** 对象很容易从 **datetime** 对象中构造：

```
In [69]: pd.Timestamp(d)
Out[69]: Timestamp('2016-06-30 00:00:00')
```

也可以从 **NumPy datetime64** 对象中构造：

```
In [70]: pd.Timestamp(nd)
Out[70]: Timestamp('2015-10-01 00:00:00')
```

另一个重要的类是 **DatetimeIndex** 类，这是 Timestamp 对象和一些强大的附加方法的集合（参见 http://bit.ly/date_range_doc 和 http://bit.ly/datetimeindex_doc）。这种对象可以用 date_range 函数实例化，该函数在构造时间索引方面相当灵活而强大（关于该函数的更多细节参见第 6 章）：

```
In [71]: dti = pd.date_range('2016/01/01', freq='M', periods=12)
         dti
Out[71]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2016-01-31, …, 2016-12-31]
         Length: 12, Freq: M, Timezone: None
```

对象的单个元素可以通过普通的索引操作访问：

```
In [72]: dti[6]
Out[72]: Timestamp('2016-07-31 00:00:00', offset='M')
```

DatetimeIndex 对象可以通过 **to_pydatetime** 方法转换为 **datetime** 对象的数组：

```
In [73]: pdi = dti.to_pydatetime()
         pdi
Out[73]: array([datetime.datetime(2016, 1, 31, 0, 0),
                datetime.datetime(2016, 2, 29, 0, 0),
                datetime.datetime(2016, 3, 31, 0, 0),
                datetime.datetime(2016, 4, 30, 0, 0),
                datetime.datetime(2016, 5, 31, 0, 0),
                datetime.datetime(2016, 6, 30, 0, 0),
                datetime.datetime(2016, 7, 31, 0, 0),
                datetime.datetime(2016, 8, 31, 0, 0),
                datetime.datetime(2016, 9, 30, 0, 0),
                datetime.datetime(2016, 10, 31, 0, 0),
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

```
                datetime.datetime(2016, 11, 30, 0, 0),
                datetime.datetime(2016, 12, 31, 0, 0)], dtype=object)
```

使用 DatetimeIndex 构造器可以进行反向的操作：

```
In [74]: pd.DatetimeIndex(pdi)
Out[74]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2016-01-31, …, 2016-12-31]
         Length: 12, Freq: None, Timezone: None
```

对于 **NumPy datetime64** 对象，必须使用 **astype** 方法：

```
In [75]: pd.DatetimeIndex(dtl.astype(pd.datetime))
Out[75]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2015-12-31 23:00:00, …, 2016-01-01 22:00:00]
         Length: 24, Freq: None, Timezone: None
```

pandas 可以绘制日期-时间信息的对应图表（参见图 C-2 和第 6 章）：

```
In [76]: rnd = np.random.standard_normal(len(dti)).cumsum() ** 2
In [77]: df = pd.DataFrame(rnd, columns=['data'], index=dti)
In [78]: df.plot()
```

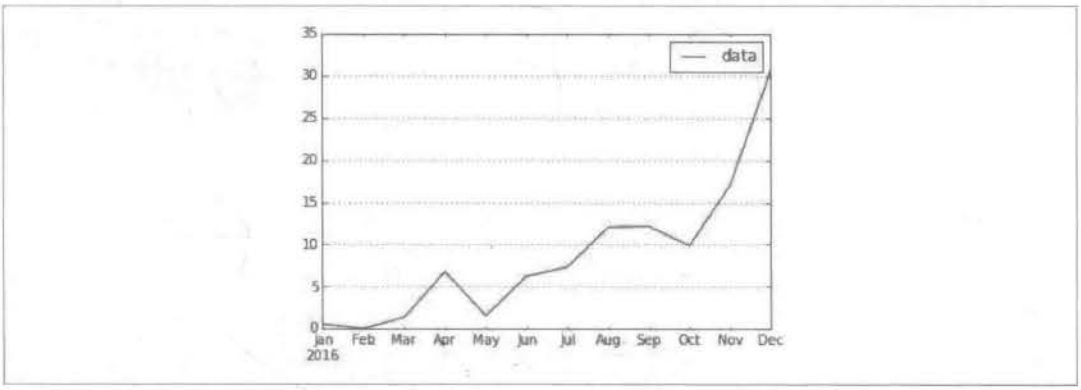


图 C-2　pandas 图表，Timestamp 表示的 x 轴刻度自动格式化

pandas 还很好地与 **pytz** 模块集成，管理时区：

```
In [79]: pd.date_range('2016/01/01', freq='M', periods=12,
                       tz=pytz.timezone('CET'))
Out[79]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2016-01-31 00:00:00+01:00, …, 2016-12-31 00:00:00+01:00]
         Length: 12, Freq: M, Timezone: CET
In [80]: dti = pd.date_range('2016/01/01', freq='M', periods=12, tz='US/Eastern')
         dti
Out[80]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2016-01-31 00:00:00-05:00, …, 2016-12-31 00:00:00-05:00]
         Length: 12, Freq: M, Timezone: US/Eastern
```

使用 **tz_convert** 方法，DatetimeIndex 对象可以从一个时区转换为另一个时区：

```
In [81]: dti.tz_convert('GMT')
Out[81]: <class 'pandas.tseries.index.DatetimeIndex'>
         [2016-01-31 05:00:00+00:00, …, 2016-12-31 05:00:00+00:00]
         Length: 12, Freq: M, Timezone: GMT
```


更多书籍资料请关注我的博客：http://readbook12.blog.163.com

作者介绍

Yves Hilpsch 是 Python Quants（德国）股份有限公司的创始人和任事股东，也是 Python Quants（纽约）有限责任公司的共同创办人。该集团提供基于 Python 的金融和衍生品分析软件（参见 http://pythonquants.com，http://quant-platfrom.com 和 http://dx-analytics.com），以及和 Python 及金融相关的咨询、开发和培训服务。

Yves 还是 *Derivatives Analytics with Python*（Wiley Finance，2015）的作者。作为获得数理金融学博士学位的商业管理专业研究生，他在萨尔州大学讲授计算金融学中的数值化方法课程。

封面说明

本书封面上的动物是伊斯帕尼奥拉长吻猬，这是一种生活于加勒比海伊斯帕尼奥拉岛（包含海地和多米尼加共和国）的濒危哺乳动物，在海地特别稀少，而在多米尼加共和国中稍微常见一些。

长吻猬以节肢动物、蠕虫、蜗牛和爬虫为食，偶尔也食用根、果实和叶子。长吻猬的重量为 1～2 磅，头和身体长 1 英尺，外加 10 英寸长的尾巴（略有误差）。这种古老的哺乳动物看上去像大的地鼠，多毛，上面为红褐色，下面较浅一些，而尾部、腿和突出的鼻子上则缺乏毛发。

长吻猬习性相当安静，往往游离于别人的视线之外。当它出门时，行动很笨拙，奔跑时常常绊倒。不过，作为一种夜间型物种，它有敏锐的听觉、嗅觉和触觉，据说，它的独特气味像“小山羊”。

它的下门牙槽分泌有毒的唾液，可以瘫痪和攻击无脊椎动物。长吻猬本身是少数有毒的哺乳动物之一。有时候，在相互争斗时会释放毒气，对长吻猬本身也可能致命。在最初的冲突之后，它们往往确立统治关系，并在同一块区域生活。家族可能长时间居住在一起，显然，它们只在洗澡时饮水。

O’Reilly 书籍封面上的许多动物都处于濒危状态；它们对世界很重要。要了解如何帮助它们，可以访问 animals.oreilly.com。

更多书籍资料请关注我的博客：http://readbook12.blog.163.com

O'REILLY®

Python金融大数据分析

金融行业已经以惊人的速度采用Python，一些最大的投资银行和对冲基金使用Python来构建核心的交易和风险管理系统。本书可以帮助开发人员和量化分析人员入门Python，并指导他们掌握Python在计量金融学上的重要应用。

本书通过大量的实用示例并以一个大型的真实案例研究为基础，讲解如何为基于蒙特卡洛模拟的衍生品和风险分析开发一个成熟的框架。本书大部分内容使用了交互式的IPython Notebooks，并包含了如下主题。

- **基础知识：Python数据结构，NumPy数组处理，用pandas进行时间序列分析，用matplotlib可视化，用PyTables进行高性能I/O操作，日期/时间信息处理和精选的最佳实践。**
- **金融主题：使用了NumPy、SciPy和SymPy的数学技术，例如回归和优化；用于蒙特卡洛模拟、风险价值、风险信用价值计算的推断统计学；用于正态性检验、均方差投资组合优化、主成分分析（PCA）和贝叶斯回归的统计学。**
- **特殊主题：用于金融算法的高性能Python，如向量化和并行化；Python与Excel的集成；以及构建基于Web技术的金融应用程序。**

"Python易于理解的语法、与C/C++的轻松集成以及各种数值计算工具，使其成为金融分析的自然选择。它正在快速替代主流金融机构中使用的语言和工具，并成为事实上的标准。"

——Kirat Singh
华盛顿Square Technologies公司
联合创始人、总裁兼CTO

Yves Hilpisch是Python Quants的创始人和任事股东，该公司是一家分析软件提供商和金融工程集团。Yves还在纽约和伦敦教授数理金融学，并组织有关Python计量金融学应用的聚会和会议。

PYTHON/FINANCE

封面设计：Ellie Volckhausen 张健

O'Reilly Media, Inc.授权人民邮电出版社出版

此简体中文版仅限于中国大陆（不包含中国香港、澳门特别行政区和中国台湾地区）销售发行
This Authorized Edition for sale only in the territory of People's Republic of China (excluding Hong Kong, Macao and Taiwan)

分类建议：计算机/程序设计/Python
人民邮电出版社网址：www.ptpress.com.cn

异步社区 www.epubit.com.cn
新浪微博 @人邮异步社区
投稿/反馈邮箱 contact@epubit.com.cn

ISBN 978-7-115-40445-9

ISBN 978-7-115-40445-9
定价：99.00 元

更多书籍资料请关注我的博客：http://readbook12.blog.163.com